

# 目录

|                                          |         |
|------------------------------------------|---------|
| Spring 实战 (第 6 版) MEAP                   | 1.1     |
| 欢迎                                       | 1.2     |
| 第 1 章 Spring 入门                          | 1.3     |
| 1.1 什么是 Spring?                          | 1.3.1   |
| 1.2 初始化 Spring 应用程序                      | 1.3.2   |
| 1.2.1 使用 Spring Tool Suite 初始化 Spring 项目 | 1.3.2.1 |
| 1.2.2 检查 Spring 项目结构                     | 1.3.2.2 |
| 1.3 编写 Spring 应用程序                       | 1.3.3   |
| 1.3.1 处理 web 请求                          | 1.3.3.1 |
| 1.3.2 定义视图                               | 1.3.3.2 |
| 1.3.3 测试控制器                              | 1.3.3.3 |
| 1.3.4 构建并运行应用程序                          | 1.3.3.4 |
| 1.3.5 了解 Spring Boot DevTools            | 1.3.3.5 |
| 1.3.6 回顾                                 | 1.3.3.6 |
| 1.4 俯瞰 Spring 风景线                        | 1.3.4   |
| 1.4.1 Spring 核心框架                        | 1.3.4.1 |
| 1.4.2 Spring Boot                        | 1.3.4.2 |
| 1.4.3 Spring Data                        | 1.3.4.3 |
| 1.4.4 Spring Security                    | 1.3.4.4 |
| 1.4.5 Spring Integration 和 Spring Batch  | 1.3.4.5 |
| 1.4.6 Spring Cloud                       | 1.3.4.6 |
| 1.4.7 Spring Native                      | 1.3.4.7 |
| 1.5 小结                                   | 1.3.5   |
| 第 2 章 开发 Web 应用程序                        | 1.4     |
| 2.1 展示信息                                 | 1.4.1   |
| 2.1.1 建立领域实体                             | 1.4.1.1 |
| 2.1.2 创建控制器类                             | 1.4.1.2 |
| 2.1.3 设计视图                               | 1.4.1.3 |
| 2.2 处理表单提交                               | 1.4.2   |
| 2.3 验证表单输入                               | 1.4.3   |
| 2.3.1 声明验证规则                             | 1.4.3.1 |
| 2.3.2 在表单绑定时执行验证                         | 1.4.3.2 |
| 2.3.3 显示验证错误                             | 1.4.3.3 |
| 2.4 使用视图控制器                              | 1.4.4   |
| 2.5 选择视图模板库                              | 1.4.5   |

|                                  |         |
|----------------------------------|---------|
| 2.5.1 缓存模板                       | 1.4.5.1 |
| 2.6 总结                           | 1.4.6   |
| 第 3 章 处理数据                       | 1.5     |
| 3.1 使用 JDBC 读写数据                 | 1.5.1   |
| 3.1.1 为持久化改造领域实体                 | 1.5.1.1 |
| 3.1.2 使用 JdbcTemplate            | 1.5.1.2 |
| 3.1.3 定义 schema 并预加载数据           | 1.5.1.3 |
| 3.1.4 插入数据                       | 1.5.1.4 |
| 3.2 使用 Spring Data JDBC          | 1.5.2   |
| 3.2.1 添加 Spring Data JDBC 到项目构建中 | 1.5.2.1 |
| 3.2.2 定义 Repository 接口           | 1.5.2.2 |
| 3.2.3 为持久化注解领域实体                 | 1.5.2.3 |
| 3.2.4 使用 CommandLineRunner 预加载数据 | 1.5.2.4 |
| 3.3 使用 Spring Data JPA 持久化数据     | 1.5.3   |
| 3.3.1 添加 Spring Data JPA 到工程中    | 1.5.3.1 |
| 3.3.2 注解领域实体                     | 1.5.3.2 |
| 3.3.3 声明 JPA Repository          | 1.5.3.3 |
| 3.3.4 自定义 Repository             | 1.5.3.4 |
| 3.4 总结                           | 1.5.4   |
| 第 4 章 处理非关系型数据                   | 1.6     |
| 4.1 使用 Cassandra Repository      | 1.6.1   |
| 4.1.1 启用 Spring Data Cassandra   | 1.6.1.1 |
| 4.1.2 理解 Cassandra 数据模型          | 1.6.1.2 |
| 4.1.3 Cassandra 持久化实体映射          | 1.6.1.3 |
| 4.1.4 编写 Cassandra Repository    | 1.6.1.4 |
| 4.2 编写 MongoDB Repository        | 1.6.2   |
| 4.2.1 启用 Spring Data MongoDB     | 1.6.2.1 |
| 4.2.2 MongoDB 持久化实体映射            | 1.6.2.2 |
| 4.2.3 编写 MongoDB Repository 接口   | 1.6.2.3 |
| 4.3 总结                           | 1.6.3   |
| 第 5 章 Spring 安全                  | 1.7     |
| 5.1 启用 Spring Security           | 1.7.1   |
| 5.2 配置身份验证                       | 1.7.2   |
| 5.2.1 内存用户详情服务                   | 1.7.2.1 |
| 5.2.2 自定义用户身份验证                  | 1.7.2.2 |
| 5.3 保护 web 请求                    | 1.7.3   |
| 5.3.1 保护请求                       | 1.7.3.1 |
| 5.3.2 创建自定义登录页面                  | 1.7.3.2 |

|                              |         |
|------------------------------|---------|
| 5.3.3 启用第三方验证                | 1.7.3.3 |
| 5.3.4 登出                     | 1.7.3.4 |
| 5.3.5 阻止跨站请求伪造攻击             | 1.7.3.5 |
| 5.4 启用方法级别保护                 | 1.7.4   |
| 5.5 了解您的用户                   | 1.7.5   |
| 5.6 小结                       | 1.7.6   |
| 第 6 章 使用配置属性                 | 1.8     |
| 6.1 微调自动配置                   | 1.8.1   |
| 6.1.1 理解 Spring 环境抽象         | 1.8.1.1 |
| 6.1.2 配置数据源                  | 1.8.1.2 |
| 6.1.3 配置嵌入式服务器               | 1.8.1.3 |
| 6.1.4 配置日志                   | 1.8.1.4 |
| 6.1.5 使用特殊的属性值               | 1.8.1.5 |
| 6.2 创建自己的配置属性                | 1.8.2   |
| 6.2.1 定义配置属性持有者              | 1.8.2.1 |
| 6.2.2 声明配置属性元数据              | 1.8.2.2 |
| 6.3 使用 profile 进行配置          | 1.8.3   |
| 6.3.1 定义特定 profile 的属性       | 1.8.3.1 |
| 6.3.2 激活 profile             | 1.8.3.2 |
| 6.3.3 有条件地使用 profile 创建 bean | 1.8.3.3 |
| 6.4 小结                       | 1.8.4   |
| 第 7 章 创建 REST 服务             | 1.9     |
| 7.1 编写 RESTful 控制器           | 1.9.1   |
| 7.1.1 从服务器获取数据               | 1.9.1.1 |
| 7.1.2 向服务器发送数据               | 1.9.1.2 |
| 7.1.3 更新服务器上的数据              | 1.9.1.3 |
| 7.1.4 删除服务器上的数据              | 1.9.1.4 |
| 7.2 启用后端数据服务                 | 1.9.2   |
| 7.2.1 调整资源路径和关系名称            | 1.9.2.1 |
| 7.2.2 分页和排序                  | 1.9.2.2 |
| 7.3 使用 REST 服务               | 1.9.3   |
| 7.3.1 GET 资源                 | 1.9.3.1 |
| 7.3.2 PUT 资源                 | 1.9.3.2 |
| 7.3.3 DELETE 资源              | 1.9.3.3 |
| 7.3.4 POST 资源数据              | 1.9.3.4 |
| 7.4 小结                       | 1.9.4   |
| 第 8 章 保护 REST 服务             | 1.10    |
| 8.1 OAuth 2 介绍               | 1.10.1  |

|                                       |          |
|---------------------------------------|----------|
| 8.2 创建验证服务器                           | 1.10.2   |
| 8.3 利用资源服务器保护 API                     | 1.10.3   |
| 8.4 开发客户端                             | 1.10.4   |
| 8.5 小结                                | 1.10.5   |
| 第 9 章 发送异步消息                          | 1.11     |
| 9.1 使用 JMS 发送消息                       | 1.11.1   |
| 9.1.1 设置 JMS                          | 1.11.1.1 |
| 9.1.2 使用 JmsTemplate 发送消息             | 1.11.1.2 |
| 9.1.3 接收 JMS 消息                       | 1.11.1.3 |
| 9.2 使用 RabbitMQ 和 AMQP                | 1.11.2   |
| 9.2.1 添加 RabbitMQ 到 Spring 中          | 1.11.2.1 |
| 9.2.2 使用 RabbitTemplate 发送消息          | 1.11.2.2 |
| 9.2.3 从 RabbitMQ 接收消息                 | 1.11.2.3 |
| 9.3 使用 Kafka 发送消息                     | 1.11.3   |
| 9.3.1 在 Spring 中设置 Kafka              | 1.11.3.1 |
| 9.3.2 使用 KafkaTemplate 发送消息           | 1.11.3.2 |
| 9.3.3 编写 Kafka 监听器                    | 1.11.3.3 |
| 9.4 总结                                | 1.11.4   |
| 第 10 章 集成 Spring                      | 1.12     |
| 10.1 声明简单的集成流                         | 1.12.1   |
| 10.1.1 使用 XML 定义集成流                   | 1.12.1.1 |
| 10.1.2 在 Java 中配置集成流                  | 1.12.1.2 |
| 10.1.3 使用 Spring Integration 的 DSL 配置 | 1.12.1.3 |
| 10.2 探索 Spring Integration            | 1.12.2   |
| 10.2.1 消息通道                           | 1.12.2.1 |
| 10.2.2 过滤器                            | 1.12.2.2 |
| 10.2.3 转换器                            | 1.12.2.3 |
| 10.2.4 路由                             | 1.12.2.4 |
| 10.2.5 分割器                            | 1.12.2.5 |
| 10.2.6 服务激活器                          | 1.12.2.6 |
| 10.2.7 网关                             | 1.12.2.7 |
| 10.2.8 通道适配器                          | 1.12.2.8 |
| 10.2.9 端点模块                           | 1.12.2.9 |
| 10.3 创建 Email 集成流                     | 1.12.3   |
| 10.4 总结                               | 1.12.4   |
| 第 11 章 Reactor 介绍                     | 1.13     |
| 11.1 理解响应式编程                          | 1.13.1   |
| 11.1.1 定义响应式流                         | 1.13.1.1 |

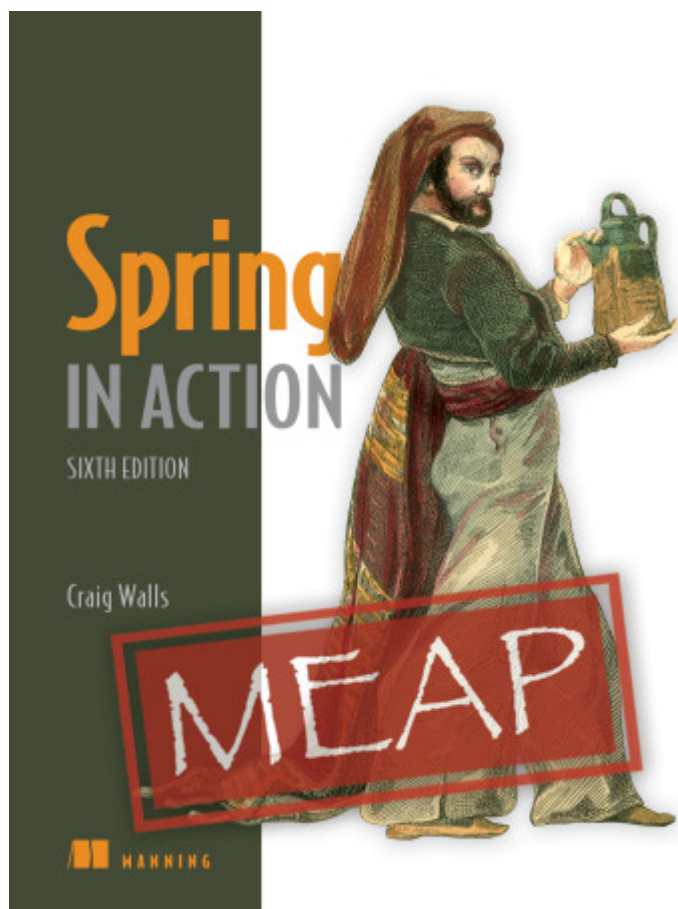


|                                 |          |
|---------------------------------|----------|
| 11.2 使用 Reactor                 | 1.13.2   |
| 11.2.1 图解响应式流                   | 1.13.2.1 |
| 11.2.2 添加 Reactor 依赖            | 1.13.2.2 |
| 11.3 通用响应式操作实战                  | 1.13.3   |
| 11.3.1 创建响应式类型                  | 1.13.3.1 |
| 11.3.2 响应式类型结合                  | 1.13.3.2 |
| 11.3.3 转换和过滤响应式流                | 1.13.3.3 |
| 11.3.4 对响应类型执行逻辑操作              | 1.13.3.4 |
| 11.4 总结                         | 1.13.4   |
| 第 12 章 开发响应式 API                | 1.14     |
| 12.1 使用 Spring WebFlux          | 1.14.1   |
| 12.1.1 Spring WebFlux 介绍        | 1.14.1.1 |
| 12.1.2 编写响应式 Controller         | 1.14.1.2 |
| 12.2 定义函数式请求处理程序                | 1.14.2   |
| 12.3 测试响应式 Controller           | 1.14.3   |
| 12.3.1 测试 GET 请求                | 1.14.3.1 |
| 12.3.2 测试 POST 请求               | 1.14.3.2 |
| 12.3.3 使用线上服务器进行测试              | 1.14.3.3 |
| 12.4 响应式消费 REST API             | 1.14.4   |
| 12.4.1 获取资源                     | 1.14.4.1 |
| 12.4.2 发送资源                     | 1.14.4.2 |
| 12.4.3 删除资源                     | 1.14.4.3 |
| 12.4.4 处理错误                     | 1.14.4.4 |
| 12.4.5 请求转换                     | 1.14.4.5 |
| 12.5 保护响应式 web API              | 1.14.5   |
| 12.5.1 配置响应式 Web 安全             | 1.14.5.1 |
| 12.5.2 配置响应式用户信息服务              | 1.14.5.2 |
| 12.6 总结                         | 1.14.6   |
| 第 13 章 响应式持久化数据                 | 1.15     |
| 13.1 使用 R2DBC                   | 1.15.1   |
| 13.1.1 为 R2DBC 定义实体             | 1.15.1.1 |
| 13.1.2 定义响应式 Repository         | 1.15.1.2 |
| 13.1.3 测试 R2DBC Repository      | 1.15.1.3 |
| 13.1.4 定义 OrderRepository 聚合根服务 | 1.15.1.4 |
| 13.2 使用 MongoDB 响应式保存文档         | 1.15.2   |
| 13.2.1 定义文档类型                   | 1.15.2.1 |
| 13.2.2 定义响应式 MongoDB Repository | 1.15.2.2 |
| 13.2.3 测试响应式 MongoDB Repository | 1.15.2.3 |

|                                               |          |
|-----------------------------------------------|----------|
| 13.3 使用 <b>Cassandra</b> 响应式保存数据              | 1.15.3   |
| 13.2.1 为 <b>Cassandra</b> 定义实体类               | 1.15.3.1 |
| 13.2.2 创建响应式 <b>Cassandra Repository</b>      | 1.15.3.2 |
| 13.2.3 测试响应式 <b>Cassandra Repository</b>      | 1.15.3.3 |
| 13.4 总结                                       | 1.15.4   |
| 第 14 章 使用 <b>RSocket</b>                      | 1.16     |
| 14.1 介绍 <b>RSocket</b>                        | 1.16.1   |
| 14.2 创建一个简单的 <b>RSocket</b> 服务端和客户端           | 1.16.2   |
| 14.2.1 使用 请求/响应 模型                            | 1.16.2.1 |
| 14.2.2 处理 请求/流 消息                             | 1.16.2.2 |
| 14.2.3 发送 即发即忘 消息                             | 1.16.2.3 |
| 14.2.4 双向发送消息                                 | 1.16.2.4 |
| 14.3 在 <b>WebSocket</b> 上进行 <b>RSocket</b> 传输 | 1.16.3   |
| 14.4 总结                                       | 1.16.4   |
| 第 15 章 使用 <b>Spring Boot Actuator</b>         | 1.17     |
| 15.1 介绍 <b>Actuator</b>                       | 1.17.1   |
| 15.1.1 配置 <b>Actuator</b> 基本路径                | 1.17.1.1 |
| 15.1.2 启用和禁用 <b>Actuator</b> 端点               | 1.17.1.2 |
| 15.2 使用 <b>Actuator</b> 端点                    | 1.17.2   |
| 15.2.1 获取重要的应用程序信息                            | 1.17.2.1 |
| 15.2.2 查看配置详细信息                               | 1.17.2.2 |
| 15.2.3 查看应用程序活动                               | 1.17.2.3 |
| 15.2.4 利用运行时指标                                | 1.17.2.4 |
| 15.3 自定义 <b>Actuator</b>                      | 1.17.3   |
| 15.3.1 向 <b>/info</b> 端点提供信息                  | 1.17.3.1 |
| 15.3.2 自定义健康指标                                | 1.17.3.2 |
| 15.3.3 注册自定义指标                                | 1.17.3.3 |
| 15.3.4 创建自定义端点                                | 1.17.3.4 |
| 15.4 保护 <b>Actuator</b>                       | 1.17.4   |
| 15.5 总结                                       | 1.17.5   |
| 第 16 章 管理 <b>Spring</b>                       | 1.18     |
| 16.1 使用 <b>SpringBoot Admin</b>               | 1.18.1   |
| 16.1.1 创建 <b>Admin</b> 服务端                    | 1.18.1.1 |
| 16.1.2 注册 <b>Admin</b> 客户端                    | 1.18.1.2 |
| 16.2 深入 <b>Admin</b> 服务端                      | 1.18.2   |
| 16.2.1 查看普通应用程序运行状况和信息                        | 1.18.2.1 |
| 16.2.2 观察关键指标                                 | 1.18.2.2 |
| 16.2.3 检查环境属性                                 | 1.18.2.3 |

|                         |          |
|-------------------------|----------|
| 16.2.4 查看并设置 log 级别     | 1.18.2.4 |
| 16.3 保护 Admin 服务端       | 1.18.3   |
| 16.3.1 在 Admin 服务端中启用登录 | 1.18.3.1 |
| 16.3.2 使用 Actuator 进行认证 | 1.18.3.2 |
| 16.4 总结                 | 1.18.4   |
| 第 17 章 使用 JMX 监控 Spring | 1.19     |
| 17.1 使用 Actuator MBean  | 1.19.1   |
| 17.2 创建自己的 MBean        | 1.19.2   |
| 17.3 发送通知               | 1.19.3   |
| 17.4 总结                 | 1.19.4   |
| 第 18 章 部署 Spring        | 1.20     |
| 18.1 权衡部署选项             | 1.20.1   |
| 18.2 构建可执行 JAR 文件       | 1.20.2   |
| 18.3 构建容器镜像             | 1.20.3   |
| 18.3.1 部署到 Kubernetes   | 1.20.3.1 |
| 18.3.2 启用优雅关闭           | 1.20.3.2 |
| 18.3.3 应用存活性和可用性        | 1.20.3.3 |
| 18.4 构建并部署 WAR 文件       | 1.20.4   |
| 18.5 终章                 | 1.20.5   |
| 18.6 总结                 | 1.20.6   |

## Spring 实战 (第 6 版) MEAP



Spring 实战（第 6 版）预计将于 2022 年 2 月出版。现在的翻译基于 Manning 出版社的 MEAP 版本。

地址: <https://www.manning.com/books/spring-in-action-sixth-edition>

LiveBook: <https://livebook.manning.com/book/spring-in-action-sixth-edition/welcome/v-7/6>

**强烈推荐您购买此书**

**尊重作者，保护版权**

第 6 版 GitHub 地址: <https://github.com/LeonLi0102/spring-in-action-v6-translate.git>

第 6 版 GitBook 地址: <https://leonli0102.github.io/spring-in-action-v6/>

部分章节的翻译来自于第 5 版:

第 5 版 Github 地址: <https://github.com/PotoYang/spring-in-action-v5-translate>

第 5 版 GitBook 地址: <https://potoyang.gitbook.io/spring-in-action-v5>

另一本书《云原生 **Spring** 实战》翻译进行中，欢迎参与。

地址: <https://github.com/LeonLi0102/cloud-native-spring-in-action-translate>

# 欢迎

感谢您购买《Spring 实战》第六版 MEAP。从过去的五个版本中一路走来，现在我很高兴在这个版本中为您介绍最新最好的 Spring。无论您是刚接触，还是想了解一些 Spring 新特性，本书都是您学习 Spring 的宝贵资源。

我试图让这个版本遵循一种亲身实践的风格，带领您完成应用程序构建的全过程。从初始化项目开始，一直到如何部署应用程序。

我们将发布开始的前五章内容。尽管还有一些尚未更新的内容，但这些章节与前一版没有太多不同。在第 1 章中，您将学习如何利用 Spring Initializr 和 Spring Boot 开始创建项目。在第 2 章中，我们将在此基础上，通过使用 Spring MVC 开发基于浏览器的功能。第 3 章，介绍使用 JDBC 和 Spring Data JPA 持久化数据。在第 4 章中，我们将看到使用 Spring Security 保护应用程序。最后，第 5 章将介绍如何使用配置属性来配置 Spring。

展望未来，我们将在第一部分中展开更多的有关数据持久化的内容，包括 Spring Data JDBC 和非关系数据库，如 MongoDB 和 Cassandra。在本书的第二部分中，我们将看到如何把应用程序与其他应用程序集成在一起。在第三部分，我们将深入研究 Spring 6 的响应式编程，并重新改造以前开发的组件，使其成为响应式式的。最后，第四部分我们将进一步讨论如何部署应用。

我们希望每隔几周对这本书进行一次更新，无论是新加章节，还是对现有章节进行修改。当您在阅读时，我邀请您访问 [liveBook 论坛](#)，您可以提问和留言。我非常重视您反馈，因为我在写这篇文章时，这些反馈对指导我的后续写作非常有价值。

—Craig Walls

# 第 1 章 Spring 入门

本章内容：

- Spring 和 Spring Boot 基础
- 初始化一个 Spring 项目
- 纵览 Spring

尽管希腊哲学家赫拉克利特（Heraclitus）并不是一个软件开发人员，但他似乎掌握了软件开发的精髓。因为有人曾引用他说过的话：“唯一不变的就是变化。”这句话深刻体现了软件开发的本质。

当 Rod Johnson 在书《*Expert One-on-One J2EE Design and Development*》（Wrox, 2002, <http://mng.bz/oVjy>）中介绍了 Spring 框架的最初形式后，我们今天开发应用程序的方式与一年前、五年前、十年前，甚至15年前都不一样了。

当时，开发的最常见的应用程序类型是基于浏览器的 web 应用程序，由关系数据库支持。虽然这种类型的开发仍然是相关的，而且 Spring 已经为这种类型的应用程序做好了很好的准备，但是我们现在还对开发由面向云的微服务组成的应用程序感兴趣，这些服务将数据持久化到各种数据库中。而对响应式编程的新兴趣在于通过非阻塞操作提供更大的可伸缩性和更好的性能。

随着软件开发的发展，Spring 框架也发生了变化，以解决现代开发问题，包括微服务和响应式编程。Spring 还通过引入 Spring Boot 来简化自己的开发模型。

无论您是开发简单的数据库支持的 web 应用程序，还是构建基于微服务的现代应用程序，Spring 都是帮助您实现目标的框架。本章是您使用 Spring 进行现代应用程序开发的第一步。

## 1.1 什么是 Spring?

我知道您可能很想开始编写 Spring 应用程序，我向您保证，在本章结束之前，您将开发一个简单的应用程序。但是首先，我得介绍一些 Spring 的基本概念，以帮助您了解 Spring 的变化。

任何不平凡的应用程序都由许多组件组成，每个组件负责自己的在整体应用程序中的那部分功能，并与其他应用程序元素协调以完成工作。在运行应用程序时，需要以某种方式创建这些组件并相互引用。

Spring 的核心是一个 **容器**，通常称为 **Spring 应用程序上下文**，用于创建和管理应用程序组件。这些组件（或 **bean**）在 Spring 应用程序上下文中连接在一起以构成一个完整的应用程序，就像将砖、灰浆、木材、钉子、管道和电线绑在一起以组成房屋。

将 **bean** 连接在一起的行为是基于一种称为 **依赖注入**（DI）的模式。依赖项注入的应用程序不是由组件自身创建和维护它们依赖的其他 **bean** 的生命周期，而是依赖于单独的实体（容器）来创建和维护所有组件，并将这些组件注入需要它们的 **bean**。通常通过构造函数参数或属性访问器方法完成此操作。

例如，假设在应用程序的许多组件中，要处理两个组件：**inventory service**（用于获取库存级别）和 **product service**（用于提供基本产品信息）。**product service** 取决于 **inventory service**，以便能够提供有关产品的完整信息。图 1.1 说明了这些 **bean** 与 Spring 应用程序上下文之间的关系。

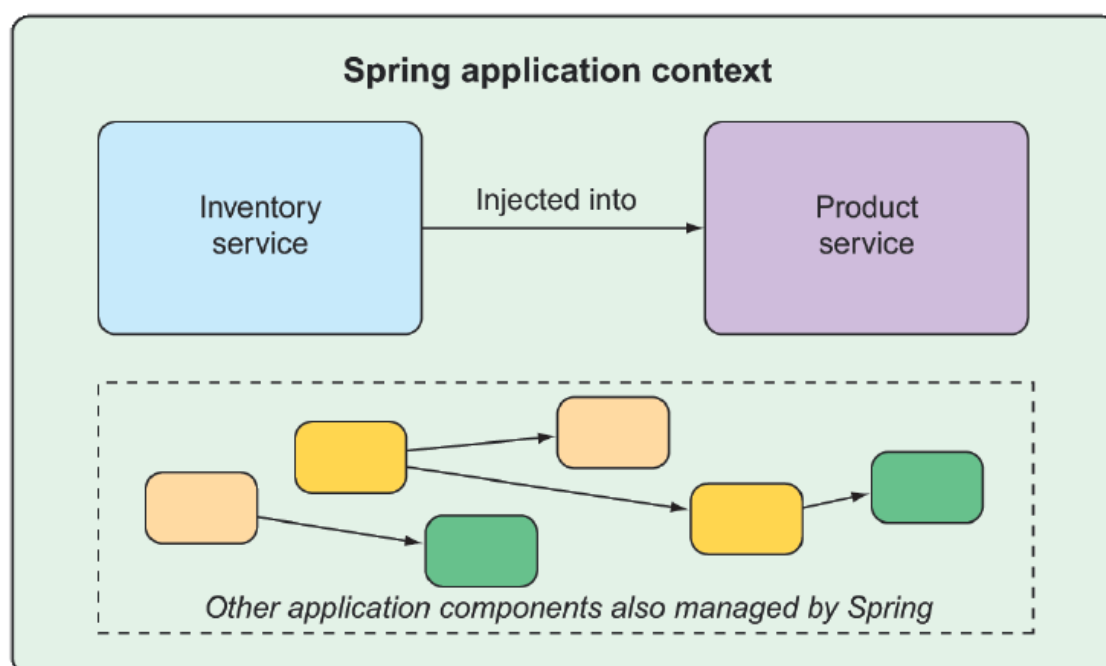


图 1.1 通过 Spring 上下文管理应用组件和注入

除了其核心容器之外，Spring 和完整的相关库产品组合还提供 Web 框架、各种数据持久性选项、安全框架与其他系统的集成、运行时监视、微服务支持、响应式编程模型以及许多其他功能，应用于现代应用程序开发。

从历史上看，引导 Spring 应用程序上下文将 **bean** 连接在一起的方式是使用一个或多个 XML 文件，这些文件描述了组件及其与其他组件的关系。例如，以下 XML 声明两个 **bean**，一个 **InventoryService bean** 和一个 **ProductService bean**，然后通过构造函数参数将 **InventoryService bean** 注入到 **ProductService** 中：



```

<bean id="inventoryService"
      class="com.example.InventoryService" />

<bean id="productService"
      class="com.example.ProductService" />
    <constructor-arg ref="inventoryService" />
</bean>

```

但是，在最新版本的 Spring 中，基于 Java 的配置更为常见。以下基于 Java 的配置类等效于 XML 配置：

```

@Configuration
public class ServiceConfiguration {
    @Bean
    public InventoryService inventoryService() {
        return new InventoryService();
    }
    @Bean
    public ProductService productService() {
        return new ProductService(inventoryService());
    }
}

```

`@Configuration` 注释向 Spring 表明这是一个配置类，它将为 Spring 应用程序上下文提供 beans。

配置的类方法带有 `@Bean` 注释，指示它们返回的对象应作为 beans 添加到应用程序上下文中（默认情况下，它们各自的 bean IDs 将与定义它们的方法的名称相同）。

与基于 XML 的配置相比，基于 Java 的配置具有多个优点，包括更高的类型安全性和改进的可重构性。即使这样，仅当 Spring 无法自动配置组件时，才需要使用 Java 或 XML 进行显式配置。

自动配置起源于 Spring 技术，即 *自动装配* 和 *组件扫描*。借助组件扫描，Spring 可以自动从应用程序的类路径中发现组件，并将其创建为 Spring 应用程序上下文中的 bean。通过自动装配，Spring 会自动将组件与它们依赖的其他 bean 一起注入。

最近，随着 Spring Boot 的推出，自动配置的优势已经远远超出了组件扫描和自动装配。Spring Boot 是 Spring 框架的扩展，它提供了多项生产力增强功能。这些增强功能中最著名的就是 *自动配置*，在这种配置中，Spring Boot 可以根据类路径中的条目、环境变量和其他因素，合理地猜测需要配置哪些组件，并将它们连接在一起。

这里想要展示一些演示自动配置的示例代码，但是并没有这样的代码，自动配置就如同风一样，可以看到它的效果，但是没有代码可以展示。我可以说“看！这是自动配置的示例！”事情发生、组件启用并且提供了功能，而无需编写代码。缺少代码是自动配置必不可少的要素，这使它如此出色。

Spring Boot 自动配置大大减少了构建应用程序所需的显式配置（无论是 XML 还是 Java）的数量。实际上，当完成本章中的示例时，将拥有一个正在运行的 Spring 应用程序，该应用程序仅包含一行 Spring 配置代码！

Spring Boot 极大地增强了 Spring 开发的能力，很难想象没有它如何开发 Spring 应用程序。因此，本书将 Spring 和 Spring Boot 视为一模一样。我们将尽可能使用 Spring Boot，并仅在必要时使用显式配置。而且，由于 Spring XML 配置是使用 Spring 的老派方式，因此我们将主要关注基于 Java 的 Spring 配置。

但是有这些功能就足够了，本书的标题包括 *实战* 这个词语，因此让我们动起来，立马开始使用 Spring 编写第一个应用程序。

## 1.2 初始化 Spring 应用程序

在本书的学习过程中，将创建 **Taco Cloud**，这是一个在线应用程序，用于订购由真人制作的最美味的食物 - 玉米饼。当然，将使用 **Spring**、**Spring Boot** 以及各种相关的库和框架来实现此目标。

初始化 **Spring** 应用程序的有多个选择。尽管我可以指导您逐步完成手动创建项目目录结构和定义构建规范的步骤，但这却浪费了时间，最好花费更多时间编写应用程序代码。因此，将依靠 **Spring Initializr** 来引导应用程序的创建。

**Spring Initializr** 既是一个基于浏览器的 **Web** 应用程序，又是一个 **REST API**，它们可以生成一个基本的 **Spring** 项目结构，可以使用所需的任何功能充实自己。使用 **Spring Initializr** 的几种方法如下：

- 从 **Web** 应用程序 <http://start.spring.io> 创建
- 使用 `curl` 命令从命令行创建
- 使用 **Spring Boot** 命令行接口从命令行创建
- 使用 **Spring Tool Suite** 创建一个新项目的时候
- 使用 **IntelliJ IDEA** 创建一个新项目的时候
- 使用 **NetBeans** 创建一个新项目的时候

我没有在本章中花费数页来讨论这些选项中的每一个，而是在附录中收集了这些详细信息。在本章以及整本书中，将展示如何使用 **Spring Tool Suite** 中对 **Spring Initializr** 的支持来创建一个新项目。

顾名思义，**Spring Tool Suite** 是一个绝佳的 **Spring** 开发环境，还提供了 **Eclipse**、**Visual Studio Code**、**Theia** **IDE** 的扩展插件。您可以下载 **Spring Tool Suite** 的可运行二进制文件，地址：<https://spring.io/tools>。**Spring Tool Suite** 提供了一个方便的 **Spring Boot Dashboard** 功能，可以在 **IDE** 中轻松启动、重启和或停止 **Spring** 应用程序。

如果您不是 **Spring Tool Suite** 用户，您仍然可以使用相关功能。请跳至附录，选择最适合您的 **Initializr** 选项代替后续章节中的安装说明。但是您要知道，在本书中，我偶尔还会引用特定于 **Spring Tool Suite** 的功能，例如 **Spring Boot Dashboard**。如果您不使用 **Spring Tool Suite**，则需要调整这些说明以适合您的 **IDE**。

### 1.2.1 使用 Spring Tool Suite 初始化 Spring 项目

要开始使用 Spring Tool Suite 中的新建 Spring 项目，请转到“文件”菜单并选择“新建”，然后选择“Spring Starter Project”。图 1.2 显示了要查找的菜单结构。

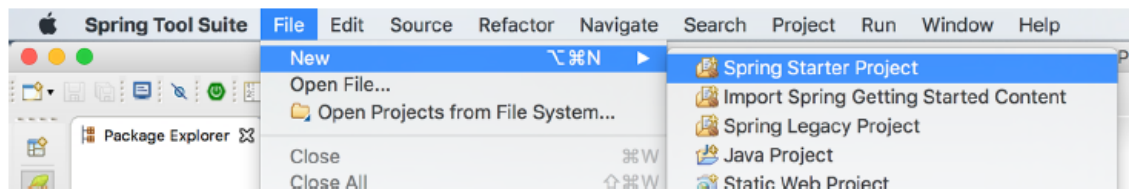


图 1.2 使用 Spring Tool Suite 的 Initializr 创建新项目

一旦选择了 Spring Starter Project，就会出现创建新的项目向导对话框（图 1.3）。向导的第一页要求提供一些常规项目信息，例如项目名称、描述和其他基本信息。如果您熟悉 Maven pom.xml 文件的内容，则可以将大多数数字段识别为以 Maven 构建规范结尾的项目。对于 Taco Cloud 应用程序，填写如图 1.3 所示的对话框，然后单击“下一步”。

**New Spring Starter Project**

Service URL:

Name:

☒ Use default location

Location:

Type:  Packaging:

Java Version:  Language:

Group:

Artifact:

Version:

Description:

Package:

Working sets

☐ Add project to working sets

Working sets:

图 1.3 填写 Taco Cloud 应用程序项目信息

向导的下一页使您可以选择要添加到项目中的依赖项（请参见图 1.4）。注意，在对话框顶部附近，您可以选择要作为项目基础的 **Spring Boot** 版本。默认为最新可用版本。通常，最好保持原样，除非您需要定位其他版本。

至于依赖项本身，您可以展开各个部分并手动查找所需的依赖项，或者在“可用”列表顶部的搜索框中搜索它们。对于 **Taco Cloud** 应用程序，选择图 1.4 中所示的依赖项。

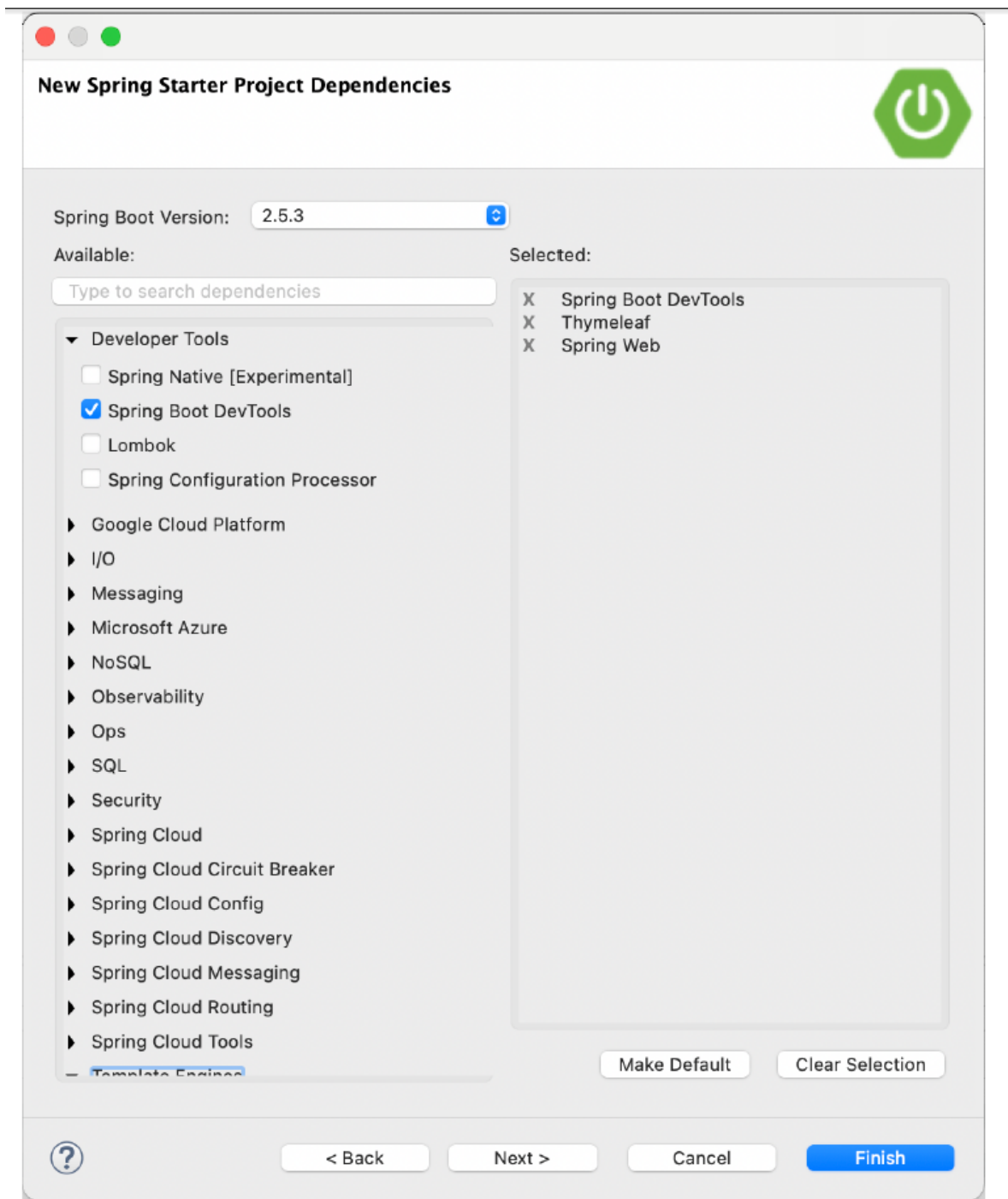
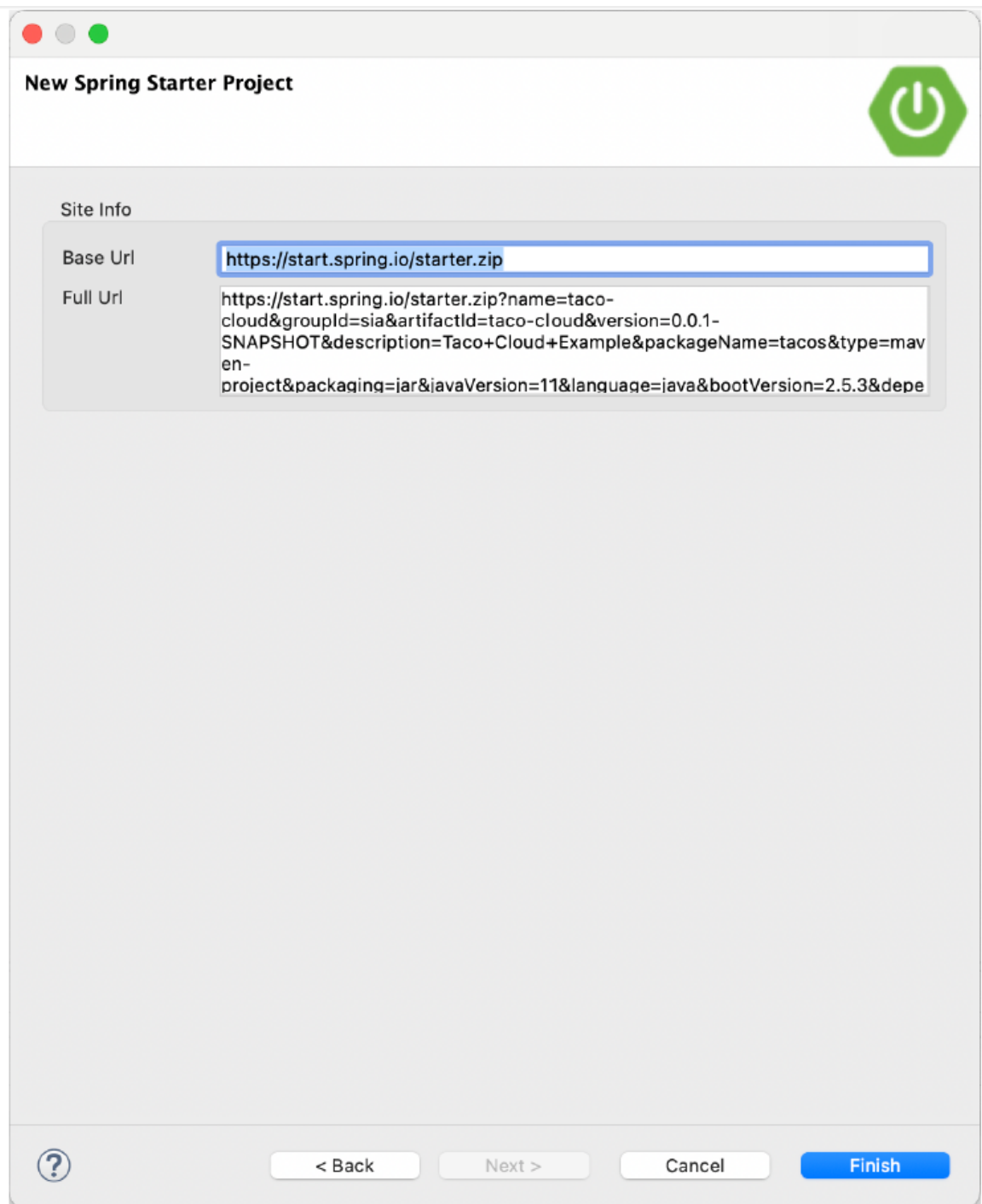


图 1.4 选择 **starter** 依赖

此时，可以单击完成以生成项目并将其添加到工作区。但是，如果感到有点危险，请再次单击“下一步”，以查看新的 **starter** 项目向导的最后一页，如图 1.5 所示。



New Spring Starter Project

Site Info

Base Url `https://start.spring.io/starter.zip`

Full Url `https://start.spring.io/starter.zip?name=taco-cloud&groupId=sia&artifactId=taco-cloud&version=0.0.1-SNAPSHOT&description=Taco+Cloud+Example&packageName=tacos&type=maven-project&packaging=jar&javaVersion=11&language=java&bootVersion=2.5.3&depe`

? < Back Next > Cancel Finish

图 1.5 指定备用 Initializr 地址（可选）

默认情况下，新项目向导在 <http://start.spring.io> 上调用 Spring Initializr 以生成项目。通常，不需要覆盖此默认值，这就是为什么可以在向导第二页上单击“完成”的原因。但是，如果由于某种原因要托管自己的 Initializr 克隆版本（也许是自己计算机上的本地副本，或者是在公司防火墙内运行的自定义克隆版本），那么将需要更改 Base Url 字段以指向 Initializr 实例，然后单击完成。

单击完成后，将从 Initializr 下载该项目并将其加载到工作区中。稍等片刻，使其加载和构建，然后就可以开始开发应用程序功能了。但是首先，让我们看一下 Initializr 所带来的好处。

## 1.2.2 检查 Spring 项目结构

在 IDE 中加载项目后，将其展开以查看其中包含的内容。图 1.6 显示了 Spring Tool Suite 中扩展的 Taco Cloud 项目。

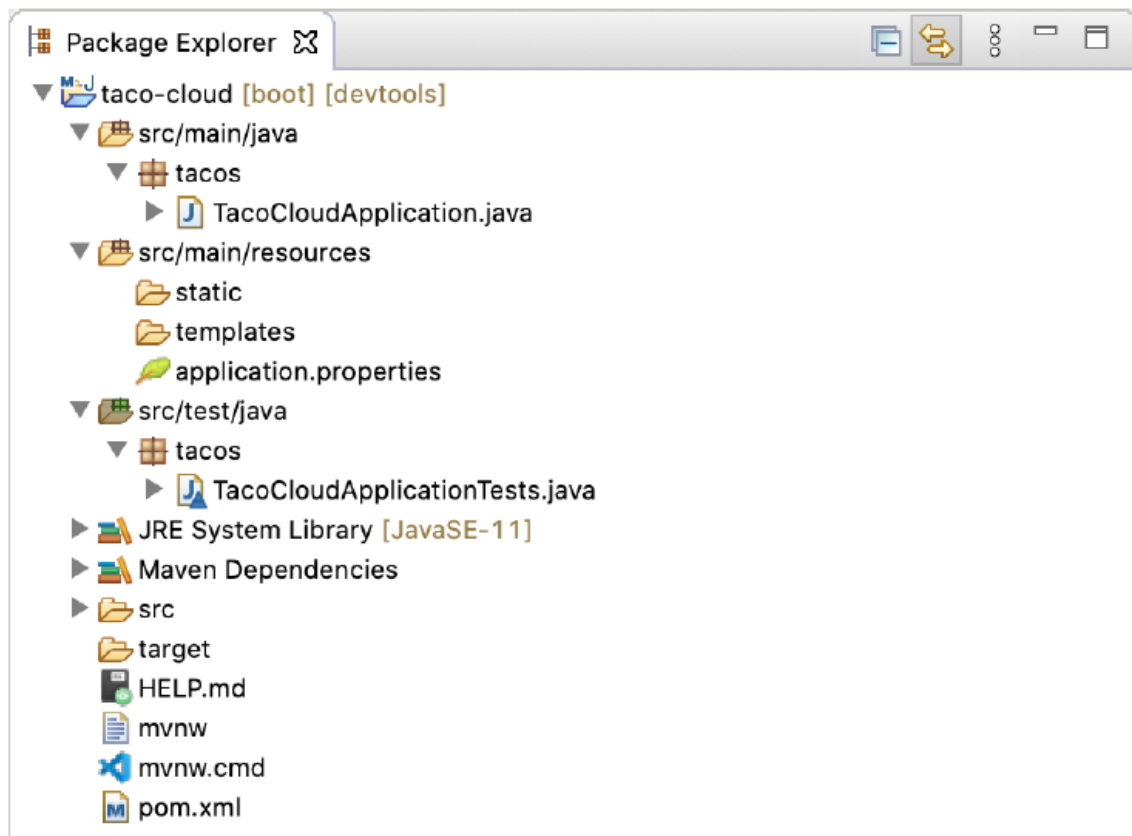


图 1.6 Spring Tool Suite 中的 Spring 项目结构

您可能会认为这是典型的 Maven 或 Gradle 项目结构，其中应用程序源代码位于 `src/main/java` 下，测试代码位于 `src/test/java` 下，非 Java 资源位于 `src/main/resources` 下。在该项目结构中，需要注意以下事项：

- `mvnw` 和 `mvnw.cmd` —— 这些是 Maven 包装器脚本。即使您的计算机上没有安装 Maven，也可以使用这些脚本构建项目。
- `pom.xml` —— 这是 Maven 构建规范，一会儿我们将对此进行更深入的研究。
- `TacoCloudApplication.java` —— 这是引导项目的 Spring Boot 主类。稍后，我们将在这节详细介绍。
- `application.properties` —— 该文件最初为空，但提供了一个可以指定配置属性的地方。我们将在本章中对此文件进行一些修改，但在第 5 章中将详细介绍配置属性。
- `static` —— 在此文件夹中，可以放置要提供给浏览器的任何静态内容（图像、样式表、JavaScript 等），最初为空。
- `templates` —— 在此文件夹中，放置用于向浏览器呈现内容的模板文件。最初为空，但很快会添加 Thymeleaf 模板。
- `TacoCloudApplicationTests.java` —— 这是一个简单的测试类，可确保成功加载 Spring 应用程序上下文。开发应用程序时，将添加更多的测试。

随着 Taco Cloud 应用程序的增长，将使用 Java 代码、图像、样式表、测试以及其他可帮助完成项目的附带材料来填充此准系统的项目结构。但是与此同时，让我们更深入地研究 Spring Initializr 提供的一些选项。

## 探索构建规范

填写 **Initializr** 表单时，指定应使用 **Maven** 构建项目。因此，**Spring Initializr** 给了您一个 **pom.xml** 文件，该文件已经填充了您所做的选择。下面的程序清单显示了 **Initializr** 提供的整个 **pom.xml** 文件。

程序清单 1.1 初始化 **Maven** 构建规范



```

<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
    https://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>2.5.3</version>
    <relativePath />
  </parent>
  <groupId>sia</groupId>
  <artifactId>taco-cloud</artifactId>
  <version>0.0.1-SNAPSHOT</version>
  <name>taco-cloud</name>
  <description>Taco Cloud Example</description>

  <properties>
    <java.version>11</java.version>
  </properties>

  <dependencies>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-thymeleaf</artifactId>
    </dependency>

    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-web</artifactId>
    </dependency>

    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-devtools</artifactId>
      <scope>runtime</scope>
      <optional>true</optional>
    </dependency>

    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-test</artifactId>
      <scope>test</scope>
      <exclusions>
        <exclusion>
          <groupId>org.junit.vintage</groupId>
          <artifactId>junit-vintage-engine</artifactId>
        </exclusion>
      </exclusions>
    </dependency>
  </dependencies>

  <build>
    <plugins>
      <plugin>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-maven-plugin</artifactId>
      </plugin>
    </plugins>
  </build>

  <repositories>
    <repository>

```

```

        <id>spring-milestones</id>
        <name>Spring Milestones</name>
        <url>https://repo.spring.io/milestone</url>
    </repository>
</repositories>
<pluginRepositories>
    <pluginRepository>
        <id>spring-milestones</id>
        <name>Spring Milestones</name>
        <url>https://repo.spring.io/milestone</url>
    </pluginRepository>
</pluginRepositories>
</project>

```

第一个值得注意的项是 `<parent>` 元素，更具体地说，注意它的 `<version>` 子元素。这指定您的项目将 `spring-boot-starter-parent` 作为它的父 POM。除此之外，这个父 POM 还为 Spring 项目中常用的几个库提供依赖项管理。对于父 POM 覆盖的那些库，不必指定版本，因为它是从父 POM 继承的。版本 `2.5.3`，表示您正在使用 Spring Boot 2.5.3，这样项目将使用继承自 Spring Boot 版本中定义的依赖项管理。除此之外，Spring Boot 的版本依赖关系管理 2.5.3 规定核心 Spring 框架的基础版本为 5.3.9。

在讨论依赖项时，请注意在 `<dependencies>` 元素下声明了四个依赖项。前三个看起来应该比较熟悉。它们直接对应于在单击 Spring Tool Suite 新建项目向导中的 Finish 按钮之前选择的 Web、Thymeleaf 以及 Spring Boot DevTools 依赖项。第四个依赖项提供了许多有用的测试功能，您不必选中包含它的方框，因为 Spring Initializr 假定（希望是正确的）您将编写测试。

您可能还会注意到，除 Spring Boot DevTools 以外的这些依赖项的 artifact ID 中都有 *starter* 这个词。Spring Boot starter 依赖项的特殊之处在于，它们本身通常没有任何库代码，而是间接地引入其他库。这些 starter 依赖提供了三个主要的好处：

- 构建的文件将会小得多，也更容易管理，因为不需要对每一个可能需要的库都声明一个依赖项。
- 可以根据它们提供的功能来考虑需要的依赖关系，而不是根据库名来考虑。如果正在开发一个 web 应用程序，那么将添加 web starter 依赖项，而不是一个编写 web 应用程序的各个库的清单。
- 不用担心 library 版本问题。可以相信的是，对于给定版本的 Spring Boot，可间接地引入的库的版本将是兼容的，只需要考虑使用的是哪个版本的 Spring Boot。

最后，构建规范以 Spring Boot 插件结束。这个插件执行一些重要的功能：

- 提供了一个 Maven 编译目标，让您能够使用 Maven 运行应用程序。这将在第 1.3.4 节中尝试实现这个目标。
- 确保所有的依赖库都包含在可执行的 JAR 文件中，并且在运行时类路径中可用。
- 在 JAR 文件中生成一个 manifest 文件，表示引导类（在本书例子中是 `TacoCloudApplication`）是可执行 JAR 的主类。

说到引导类，让我们打开它，仔细看看。

## 引导应用程序

因为将从一个可执行的 JAR 运行应用程序，所以在运行 JAR 文件时，有一个主类来执行是很重要的。还需要至少一个最小的 Spring 配置文件来引导应用程序。这就是将在 `TacoCloudApplication` 类中找到的内容：

程序清单 1.2 Taco Cloud 引导类

```
package tacos;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class TacoCloudApplication {

    public static void main(String[] args) {
        SpringApplication.run(TacoCloudApplication.class, args);
    }

}
```

虽然 `TacoCloudApplication` 中只有很少的代码，但是其中包含了相当丰富的内容。最强大的代码行之一也是最短的代码行之一。`@SpringBootApplication` 注释清楚地表明这是一个 **Spring** 引导应用程序。但是 `@SpringBootApplication` 中有更多的东西。

`@SpringBootApplication` 是一个组合了其他三个注释的复合应用程序：

- `@SpringBootConfiguration` —— 指定这个类为配置类。尽管这个类中还没有太多配置，但是如果需要，可以将 **Javabased Spring Framework** 配置添加到这个类中。实际上，这个注释是 `@Configuration` 注释的一种特殊形式。
- `@EnableAutoConfiguration` —— 启用 **Spring** 自动配置。稍后我们将详细讨论自动配置。现在，要知道这个注释告诉 **Spring Boot** 自动配置它认为需要的任何组件。
- `@ComponentScan` —— 启用组件扫描。这允许您声明其他带有 `@Component`、`@Controller`、`@Service` 等注释的类，以便让 **Spring** 自动发现它们并将它们注册为 **Spring** 应用程序上下文中的组件。

`TacoCloudApplication` 的另一个重要部分是 `main()` 方法。这个方法将在执行 **JAR** 文件时运行。在大多数情况下，这种方法是样板代码；编写的每个 **Spring** 引导应用程序都有一个类似或相同的方法（尽管类名不同）。

`main()` 方法调用 `SpringApplication` 类上的静态 `run()` 方法，该方法执行应用程序的实际引导，创建 **Spring** 应用程序上下文。传递给 `run()` 方法的两个参数是一个配置类和命令行参数。虽然传递给 `run()` 的配置类不必与引导类相同，但这是最方便、最典型的选择。

您可能不需要更改引导类中的任何内容。对于简单的应用程序，您可能会发现在引导类中配置一两个其他组件很方便，但是对于大多数应用程序，最好为任何没有自动配置的东西创建一个单独的配置类。您将在本书的整个过程中定义几个配置类，因此请注意这些细节。

## 测试应用程序

测试是软件开发的一个重要部分。您始终可以手动测试项目，这通过先构建它，然后从命令行运行它，如下所示：

```
$ ./mvnw package
...
$ java -jar target/taco-cloud-0.0.1-SNAPSHOT.jar
```

或者，由于我们使用的是 **Spring Boot**，**Spring Boot Maven** 插件使它变得更加简单：

```
$ ./mvnw spring-boot:run
```

但手动测试意味着涉及到人为因素，因此可能会出现人为错误和不一致的测试。自动化测试更加一致且可重复。

认识到这一点后，**Spring Initializr** 提供了一个测试类。下面的清单显示了基准测试类。

### 程序清单 1.3 基准应用测试

```
package tacos;

import org.junit.jupiter.api.Test;
import org.springframework.boot.test.context.SpringBootTest;

@SpringBootTest
public class TacoCloudApplicationTests {

    @Test
    public void contextLoads() {
    }

}
```

在 `TacoCloudApplicationTests` 中没有太多东西：类中的一个测试方法是空的。尽管如此，这个测试类确实执行了必要的检查，以确保 **Spring** 应用程序上下文能够成功加载。如果做了任何阻止创建 **Spring** 应用程序上下文的更改，则此测试将失败，这样您就可以通过解决问题来应对。

`@SpringBootTest` 告诉 **JUnit** 使用 **Spring Boot** 功能引导测试。与 `@SpringBootApplication` 一样，`@SpringBootTest` 是一个复合注解，它本身用 `@ExtendWith(SpringExtension.class)` 注解，以添加 **Spring** 测试功能到 **JUnit5** 中。不过，就目前而言，将其视为与 `main()` 方法调用 `SpringApplication.run()` 等效的测试类就足够了。在本书整个过程中，将多次看到 `@SpringBootTest`，我们将揭示它的一些功能。

最后，还有测试方法本身。尽管 `@SpringBootTest` 的任务是加载测试的 **Spring** 应用程序上下文，如果没有任何测试，它将没有任何作用。即使没有任何断言或任何类型的代码，这个空的测试方法也会调用两个注释完成它们的工作，并加载 **Spring** 应用程序上下文。如果运行过程中有任何问题，测试就会失败。

要从命令行运行该类 and 任何测试类，可以使用以下 **Maven** 命令：

```
$ ./mvnw test
```

至此，我们已经完成了对 **Spring Initializr** 提供的代码的回顾。看到了一些用于开发 **Spring** 应用程序的样板基础，但是仍然没有编写任何代码。现在，启动 **IDE**，掸掉键盘上的灰尘，并向 **Taco Cloud** 应用程序添加一些定制代码。

## 1.3 编写 Spring 应用程序

因为才刚刚开始，所以我们将对 Taco Cloud 应用程序的一个相对较小的更改开始，但是这个更改将展示 Spring 的很多优点。在刚刚开始的时候，添加到 Taco Cloud 应用程序的第一个功能是主页，这似乎是合适的。当您添加主页，您将创建两个代码构件：

- 一个处理主页请求的控制器类
- 一个视图模板，定义了主页的外观

因为测试很重要，所以还将编写一个简单的测试类来测试主页。但首先...我们来写这个控制器。

### 1.3.1 处理 web 请求

Spring 附带了一个强大的 web 框架，称为 Spring MVC。Spring MVC 的核心是控制器的概念，这是一个处理请求并使用某种信息进行响应的类。对于面向浏览器的应用程序，控制器的响应方式是可选地填充模型数据并将请求传递给视图，以生成返回给浏览器的 HTML。

您将在第 2 章学到很多关于 Spring MVC 的知识。但是现在，将编写一个简单的控制器类来处理根路径的请求（例如 `/`），并将这些请求转发到主页视图，而不填充任何模型数据。程序清单 1.4 显示了简单的控制器类。

程序清单 1.4 主页控制器

```
package tacos;

import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.GetMapping;

@Controller
public class HomeController {

    @GetMapping("/")
    public String home() {
        return "home";
    }
}
```

可以看到，这个类是用 `@Controller` 注释的。`@Controller` 本身并没有做多少事情。它的主要目的是将该类识别为组件扫描的组件。由于 `HomeController` 是用 `@Controller` 注释的，因此 Spring 的组件扫描会自动发现它，并在 Spring 应用程序上下文中创建一个 `HomeController` 实例作为 bean。

实际上，其他一些注释（包括 `@Component`、`@Service` 和 `@Repository`）的用途与 `@Controller` 类似。您可以用任何其他的注解来有效地注释 `HomeController`，它仍然可以工作。但是，选择 `@Controller` 更能描述该组件在应用程序中的角色。

`home()` 方法与控制器方法一样简单。它使用 `@GetMapping` 进行注释，以指示如果接收到根路径 `/` 的 HTTP GET 请求，则此方法应该处理该请求。除了返回 `home` 的 `String` 值外，它什么也不做。

此值被解释为视图的逻辑名称。如何实现该视图取决于几个因素，但是因为 Thymeleaf 在类路径中，所以可以使用 Thymeleaf 定义该模板。

#### 为什么是 Thymeleaf？

您可能想知道为什么选择 Thymeleaf 作为模板引擎。为什么不是 JSP？为什么不是 FreeMarker？为什么不是其他几个选项中的一个呢？

简单地说，我必须选择一些东西，我喜欢 Thymeleaf，相比其他选项更喜欢。尽管 JSP 看起来是一个不错的选择，但是在使用 JSP 进行 Spring 引导时仍然存在一些需要克服的挑战。我不想在第 1 章中掉进那个陷阱。不要紧，我们将在第 2 章中讨论其他模板选项，包括 JSP。

模板名称由逻辑视图名称派生而来，它的前缀是 `/templates/`，后缀是 `.html`。模板的结果路径是 `/templates/home.html`。因此，需要将模板放在项目的 `/src/main/resources/templates/home.html` 中。现在让我们创建该模板。

### 1.3.2 定义视图

为了保持您的主页简洁，它应该做的只是欢迎用户访问网站。下面的程序清单显示了定义 **Taco Cloud** 主页的基本 **Thymeleaf** 模板。

程序清单 1.5 **Taco Cloud** 主页模板

```
<!DOCTYPE html>
<html xmlns="http://www.w3.org/1999/xhtml"
      xmlns:th="http://www.thymeleaf.org">
  <head>
    <title>Taco Cloud</title>
  </head>

  <body>
    <h1>Welcome to...</h1>
    
  </body>
</html>
```

关于这个模板没有太多要讨论的。唯一值得注意的代码行是显示 **Taco Cloud** 标志的 `<img>` 标记。它使用一个 **Thymeleaf** 的 `th:src` 属性和一个 `@{...}` 表达式引用具有上下文相对路径的图片。除去这些，它只是一个 **Hello World** 页面。

但是让我们再多讨论一下这个图片。我将把它留给您来定义一个您喜欢的 **Taco Cloud** 标志。您需要将它放在项目中的恰当位置。

该图片是通过上下文相对路径 `/images/TacoCloud.png` 进行引用的。从我们对项目结构的回顾中可以想起，像图片这样的静态内容保存在 `/src/main/resources/static` 文件夹中。这意味着 **Taco Cloud** 标志图片也必须驻留在项目的 `/src/main/resources/static/images/TacoCloud.png` 中。

现在已经有了处理主页请求的控制器和呈现主页的视图模板，几乎已经准备好启动应用程序并看到它的实际运行效果了。但首先，让我们看看如何针对控制器编写测试。

### 1.3.3 测试控制器

在对 HTML 页面的内容进行断言时，测试 web 应用程序可能比较棘手。幸运的是，Spring 提供了一些强大的测试支持，使测试 web 应用程序变得很容易。

就主页而言，您将编写一个与主页本身复杂度相当的测试。您的测试将对根路径 `/` 执行一个 HTTP GET 请求并期望得到一个成功的结果，其中视图名称为 `home`，结果内容包含短语“Welcome to...”。下面的程序清单应该可以达到目的。

程序清单 1.6 主页控制器测试

```
package tacos;

import static org.hamcrest.Matchers.containsString;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.get;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.content;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.status;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.view;

import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.web.servlet.WebMvcTest;
import org.springframework.test.web.servlet.MockMvc;

@WebMvcTest(HomeController.class)
public class HomeControllerTest {

    @Autowired
    private MockMvc mockMvc;

    @Test
    public void testHomePage() throws Exception {
        mockMvc.perform(get("/"))
            .andExpect(status().isOk())
            .andExpect(view().name("home"))
            .andExpect(content().string(
                containsString("Welcome to...")));
    }
}
```

关于这个测试，您可能注意到的第一件事是，它与 `TacoCloudApplicationTests` 类在应用到它的注释方面略有不同。`HomeControllerTest` 使用 `@WebMvcTest` 注释，而不是 `@SpringBootTest` 标记。这是 Spring Boot 提供的一个特殊测试注释，它安排测试在 Spring MVC 应用程序的上下文中运行。更具体地说，在本例中，它安排 `HomeController` 在 Spring MVC 中注册，这样您就可以对它进行请求。

`@WebMvcTest` 还为测试 Spring MVC 提供了 Spring 支持。虽然可以让它启动服务器，但模拟 Spring MVC 的机制就足以满足您的目的了。测试类被注入了一个 `MockMvc` 对象中，以此用来测试来驱动模型。

`testHomePage()` 方法定义了对主页执行的测试。它从 `MockMvc` 对象开始，执行针对 `/`（根路径）的 HTTP GET 请求。该请求规定了下列期望值：

- 响应应该有一个 HTTP 200（OK）状态。
- 视图应该有一个合理的主页名称。
- 呈现的视图应该包含“Welcome to...”

您可以在您选择的 IDE 中或使用 Maven 运行测试，如下所示：



```
$ mvnw test
```

如果在 `MockMvc` 对象执行请求之后，这些期望中的任何一个都没有满足，那么测试就会失败。但是控制器和视图模板是为了满足这些期望而编写的，所以测试应该能够通过，或者至少能够通过一些表示测试通过的绿色提示。

控制器写好了，视图模板创建好了，测试通过了。看来您已经成功地实现了主页。但是，即使测试通过了，在浏览器中查看结果也会稍微让人更满意一些。毕竟，**Taco Cloud** 的客户也将这样看待它。让我们构建应用程序并运行它。

### 1.3.4 构建并运行应用程序

正如有多种方法可以初始化 Spring 应用程序一样，也有多种方法可以运行 Spring 应用程序。如果愿意，可以翻到附录部分，阅读一些更常见的运行 Spring 引导应用程序的方法。

因为选择使用 Spring Tool Suite 来初始化和处理项目，所以有一个称为 Spring Boot Dashboard 的便利功能可以帮助您在 IDE 中运行应用程序。Spring Boot Dashboard 显示为一个选项卡，通常位于 IDE 窗口的左下方。图 1.7 显示了 Spring Boot Dashboard 的注释截图。

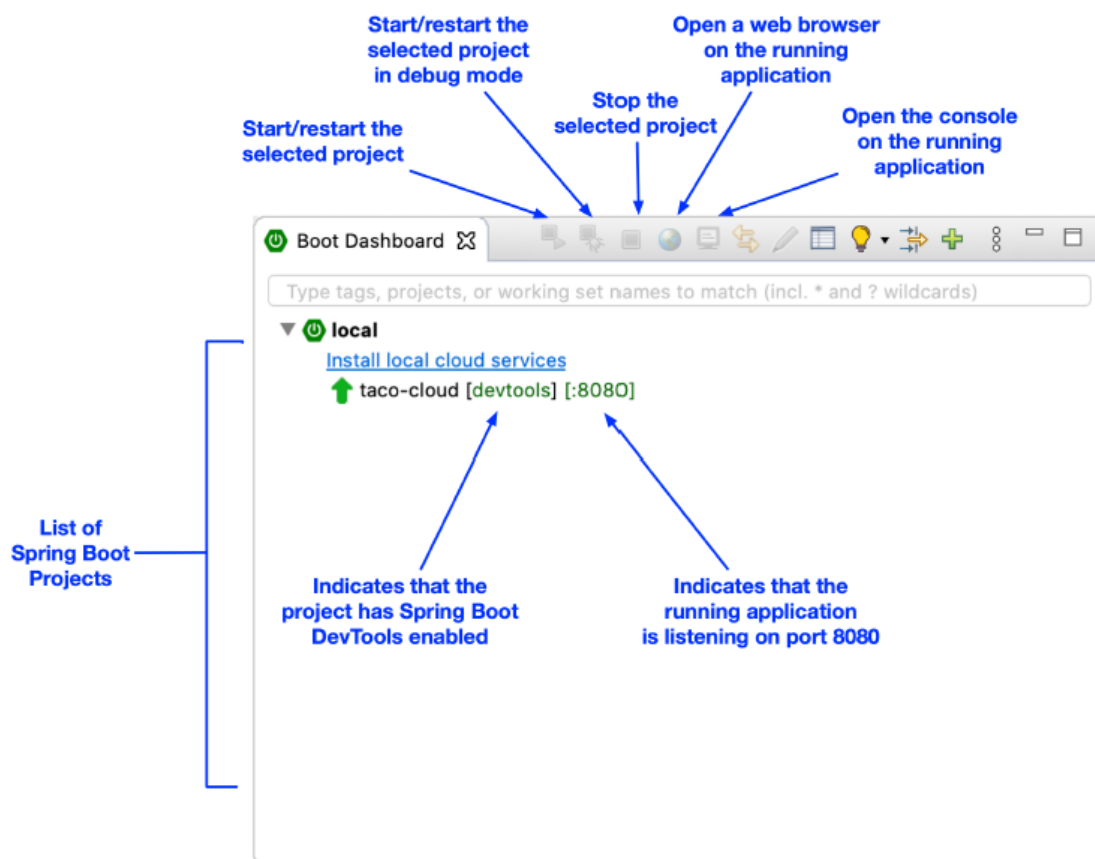


图 1.7 Spring Boot Dashboard 注释

虽然图 1.7 包含了一些最有用的细节，但我不想花太多时间来检查 Spring Boot Dashboard 所做的一切。现在需要知道的重要事情是如何使用它来运行 Taco Cloud 应用程序。确保 taco-cloud 应用程序在项目列表中突出显示（这是图 1.7 中显示的惟一应用程序），然后单击 start 按钮（最左边的按钮，其中有绿色三角形和红色正方形），应用程序应该会立即启动。

当应用程序启动时，将在控制台中看到一些 Spring ASCII 图飞过，然后看到一些日志条目描述应用程序启动时的步骤。在停止日志记录之前，将看到一个日志条目，其中说 Tomcat 在 port(s): 8080 (http) 上启动，这意味着已经准备好将 web 浏览器指向主页，以查看结果。

等一下，Tomcat 启动？何时将应用程序部署到 Tomcat？

Spring Boot 应用程序倾向于裹挟所有需要的东西，而不需要部署到某个应用服务器。您从未将应用程序部署到 Tomcat... 其实 Tomcat 是应用程序的一部分！（将在 1.3.6 小节中详细描述 Tomcat 如何成为应用程序的一部分的。）

现在应用程序已经启动，将 web 浏览器指向 <http://localhost:8080>（或单击 Spring Boot Dashboard 中地球仪样子的按钮），应该会看到类似图 1.8 所示的内容。如果您设计了自己的图标，那么结果可能不同，但是它与在图 1.8 中看到的应该相差不大。

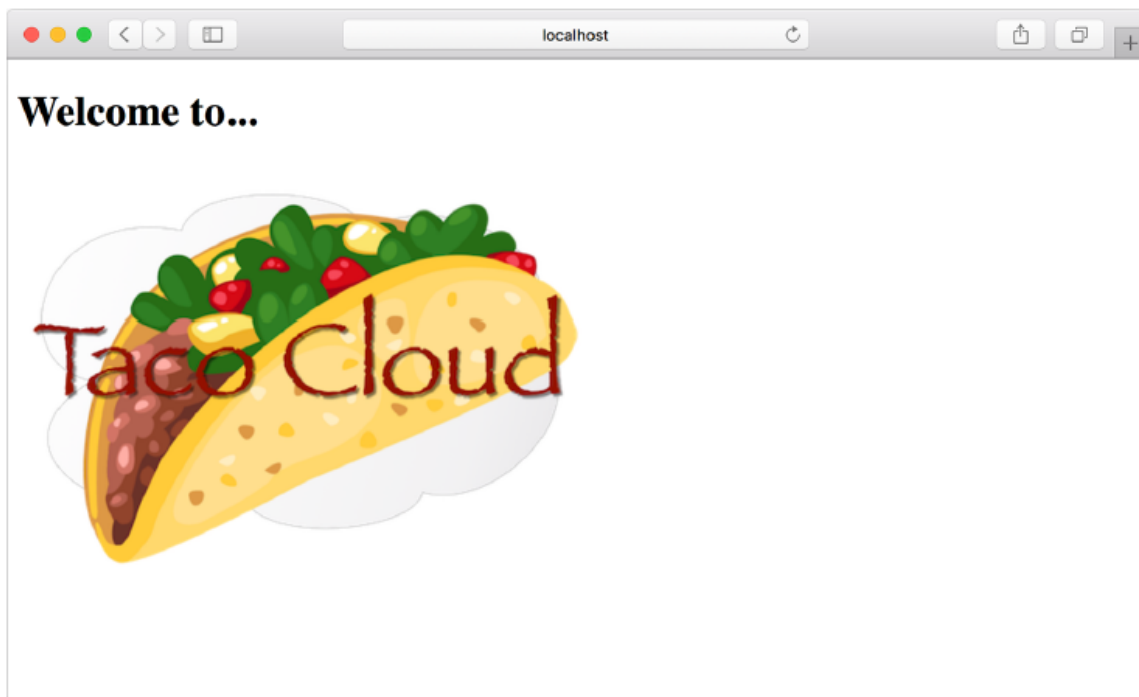


图 1.8 Taco Cloud 主页

它可能没什么好看的。但这并不是一本关于平面设计的书。主页的简陋外观现在已经足够了。它为您了解 Spring 提供了一个坚实的开端。

到目前为止，忽略了 DevTools。在初始化项目时将其作为依赖项进行选择。它作为一个依赖项出现在生成的 pom.xml 文件中。Spring Boot Dashboard 甚至显示项目已经启用了 DevTools。但是什么是 DevTools，它能为您做什么？让我们快速浏览一下 DevTools 的几个最有用的特性。

### 1.3.5 了解 Spring Boot DevTools

顾名思义，DevTools 为 Spring 开发人员提供了一些方便的开发同步工具。这些是：

- 当代码更改时自动重启应用程序
- 当以浏览器为目标的资源（如模板、JavaScript、样式表等）发生变化时，浏览器会自动刷新
- 自动禁用模板缓存
- 如果 H2 数据库正在使用，则在 H2 控制台中构建

理解 DevTools 不是 IDE 插件是很重要的，它也不要求您使用特定的 IDE。它在 Spring Tool Suite、IntelliJ IDEA 和 NetBeans 中工作得同样好。此外，由于它仅用于开发目的，所以在部署生产环境时禁用它本身是非常明智的。（我们将在第 19 章中讨论如何部署应用程序。）现在，让我们关注一下 Spring Boot DevTools 最有用的特性，首先是自动重启应用程序。

#### 自动重启应用程序

使用 DevTools 作为项目的一部分，将能够对项目中的 Java 代码和属性文件进行更改，并在短时间内查看这些更改的应用。DevTools 监视更改，当它看到某些内容发生更改时，它会自动重新启动应用程序。

更准确地说，当 DevTools 起作用时，应用程序被加载到 Java 虚拟机（JVM）中的两个单独的类加载器中。一个类装入器装入 Java 代码、属性文件以及项目的 `src/main/path` 中的几乎所有东西。这些项目可能会频繁更改。另一个类加载器加载了依赖库，它们不太可能经常更改。

当检测到更改时，DevTools 只重新加载包含项目代码的类加载器，并重新启动 Spring 应用程序上下文，但不影响其他类加载器和 JVM。尽管这一策略很微妙，但它可以略微减少启动应用程序所需的时间。

这种策略的缺点是对依赖项的更改在自动重新启动时不可用。这是因为类装入器包含依赖项库 不是自动重新加载。这意味着，每当在构建规范中添加、更改或删除依赖项时，都需要重新启动应用程序才能使这些更改生效。

#### 自动刷新浏览器和禁用模板缓存

默认情况下，模板选项（如 Thymeleaf 和 FreeMarker）被配置为缓存模板解析的结果，这样模板就不需要它们所服务的每个请求进行修复。这在生产中非常有用，因为它可以带来一些性能上的好处。

但是，缓存的模板在开发时不是很好。缓存的模板使它不可能在应用程序运行时更改模板，并在刷新浏览器后查看结果。即使做了更改，缓存的模板仍将继续使用，直到重新启动应用程序。

DevTools 通过自动禁用所有模板缓存来解决这个问题。对模板进行尽可能多的修改，并且要知道只有浏览器刷新才能看到结果。

但如果像我一样，甚至不想被点击浏览器的刷新按钮所累，如果能够立即在浏览器中进行更改并查看结果，那就更好了。幸运的是，DevTools 为我们这些懒得点击刷新按钮的人提供了一些特别的功能。

当 DevTools 起作用时，它会自动启用 LiveReload（<http://livereload.com/>）服务器和应用程序。就其本身而言，LiveReload 服务器并不是很有用。但是，当与相应的 LiveReload 浏览器插件相结合时，它会使得浏览器在对模板、图像、样式表、JavaScript 等进行更改时自动刷新——实际上，几乎所有最终提供给浏览器的更改都会自动刷新。

LiveReload 有针对 Google Chrome、Safari 和 Firefox 浏览器的插件。（对不起，ie 和 Edge 的粉丝们。）请访问 <http://livereload.com/extensions/>，了解如何为浏览器安装 LiveReload。

#### 在 H2 控制台中构建

虽然项目还没有使用数据库，但这将在第 3 章中进行更改。如果选择使用 H2 数据库进行开发，DevTools 还将自动启用一个 H2 控制台，您可以从 web 浏览器访问该控制台。只需将 web 浏览器指向 <http://localhost:8080/h2-console>，就可以深入了解应用程序正在处理的数据。

至此，已经编写了一个完整但简单的 Spring 应用程序。您将在本书的整个过程中扩展它。但是现在是回顾已经完成的工作以及 Spring 如何发挥作用的好时机。

### 1.3.6 回顾

回想一下是如何走到这一步的。简而言之，以下是构建基于 Spring 的 Taco Cloud 应用程序的步骤：

- 使用 **Spring Initializr** 创建了一个初始项目结构。
- 写了一个控制器类来处理主页请求。
- 定义了一个视图模板来呈现主页。
- 写了一个简单的测试类来检验上述工作。

看起来很简单，不是吗？除了启动项目的第一步之外，所采取的每一个行动都是为了实现创建主页的目标。

事实上，编写的几乎每一行代码都是针对这个目标的。不计算 `Java import` 语句，只计算控制器类中的两行代码，而视图模板中没有 **Spring** 的特定代码。尽管测试类的大部分都使用了 **Spring** 的测试支持，但是在测试上下文中，它的侵入性似乎要小一些。

这是使用 **Spring** 开发的一个重要好处。可以关注于满足应用程序需求的代码，而不是满足框架的需求。尽管确实需要不时地编写一些特定于框架的代码，但这通常只是代码库的一小部分。如前所述，**Spring**（通过 **Spring Boot**）可以被认为是 *无框架的框架*。

这到底是怎么回事？**Spring** 在幕后做了什么来确保您的应用程序需求得到满足？为了理解 **Spring** 在做什么，让我们从构建规范开始。

在 `pom.xml` 文件中，声明了对 **Web** 和 **Thymeleaf** 启动器的依赖。这两个依赖关系带来了一些其他的依赖关系，包括：

- **Spring MVC** 框架
- 嵌入式 **Tomcat**
- **Thymeleaf** 和 **Thymeleaf** 布局方言

它还带来了 **Spring Boot** 的自动配置库。当应用程序启动时，**Spring Boot** 自动配置自动检测这些库并自动执行：

- 在 **Spring** 应用程序上下文中配置 **bean** 以启用 **Spring MVC**
- 将嵌入式 **Tomcat** 服务器配置在 **Spring** 应用程序上下文中
- 为使用 **Thymeleaf** 模板呈现 **Spring MVC** 视图，配置了一个 **Thymeleaf** 视图解析器

简而言之，自动配置完成了所有繁重的工作，让您专注于编写实现应用程序功能的代码。如果您问我这样好不好，我会说这是一个很好的安排！

您的 **Spring** 之旅才刚刚开始。**Taco Cloud** 应用程序只涉及 **Spring** 提供的一小部分内容。在您开始下一步之前，让我们来俯瞰 **Spring** 的风景线，看看您在旅途中会遇到什么地标。

## 1.4 俯瞰 Spring 风景线

要了解 Spring 的风景线，只需查看完整版 **Spring Initializr web** 表单上的大量复选框列表即可。它列出了 100 多个依赖项选择，所以我不会在这里全部列出或者提供一个屏幕截图。但我鼓励您去看看。与此同时，我将提到一些亮点。

### 1.4.1 Spring 核心框架

正如您所期望的，Spring 核心框架是 Spring 领域中其他一切的基础。它提供了核心容器和依赖注入框架。但它也提供了一些其他的基本特性。

其中包括 Spring MVC 和 Spring web 框架。已经了解了如何使用 Spring MVC 编写控制器类来处理 web 请求。但是，您还没有看到的是，Spring MVC 也可以用于创建产生非 HTML 输出的 REST API。我们将在第 2 章深入研究 Spring MVC，然后在第 6 章中讨论如何使用它来创建 REST API。

Spring 核心框架还提供了一些基本数据持久性支持，特别是基于模板的 JDBC 支持。将在第 3 章中看到如何使用 JdbcTemplate。

Spring 具有对响应式编程的支持，包括一个新的响应式 web 框架——Spring WebFlux，它大量借鉴了 Spring MVC。将在第 3 部分中看到 Spring 的响应式编程模型，并在第 10 章中看到 Spring WebFlux。



## 1.4.2 Spring Boot

我们已经看到了 **Spring Boot** 的许多好处，包括启动依赖项和自动配置。在本书中我们确实会尽可能多地使用 **Spring Boot**，并避免任何形式的显式配置，除非绝对必要。但除了启动依赖和自动配置，**Spring Boot** 还提供了一些其他有用的特性：

- **Actuator** 提供了对应用程序内部工作方式的运行时监控，包括端点、线程 **dump** 信息、应用程序健康状况和应用程序可用的环境属性。
- 灵活的环境属性规范。
- 在核心框架的测试辅助之外，还有额外的测试支持。

此外，**Spring Boot** 提供了一种基于 **Groovy** 脚本的替代编程模型，称为 **Spring Boot CLI**（命令行界面）。使用 **Spring Boot CLI**，可以将整个应用程序编写为 **Groovy** 脚本的集合，并从命令行运行它们。我们不会在 **Spring Boot CLI** 上花太多时间，但是当它适合我们的需要时，我们会接触它。

**Spring Boot** 已经成为 **Spring** 开发中不可或缺的一部分；我无法想象开发一个没有它的 **Spring** 应用程序。因此，本书采用了以 **Spring Boot** 为中心的观点，当我提到 **Spring Boot** 正在做的事情时，您可能会发现我在使用 **Spring** 这个词。

### 1.4.3 Spring Data

尽管 **Spring** 核心框架提供了基本的数据持久性支持，但 **Spring Data** 提供了一些非常惊人的功能：将应用程序的数据存储库抽象为简单的 **Java** 接口，同时当定义方法用于如何驱动数据进行存储和检索的问题时，对方法使用了命名约定。

更重要的是，**Spring Data** 能够处理几种不同类型的数据库，包括关系型（**JPA**）、文档型（**Mongo**）、图型（**Neo4j**）等。在第 3 章中，将使用 **Spring Data** 来帮助创建 **Taco Cloud** 应用程序的存储库。

### 1.4.4 Spring Security

应用程序安全性一直是一个重要的主题，而且似乎一天比一天重要。幸运的是，Spring 在 Spring security 中有一个健壮的安全框架。

Spring Security 解决了广泛的应用程序安全性需求，包括身份验证、授权和 API 安全性。尽管 Spring Security 的范围太大，本书无法恰当地涵盖，我们将在第 4 章和第 12 章中讨论一些最常见的用例。

### 1.4.5 Spring Integration 和 Spring Batch

在某种程度上，大多数应用程序将需要与其他应用程序集成，甚至需要与同一应用程序的其他组件集成。为了满足这些需求，出现了几种应用程序集成模式。**Spring Integration** 和 **Spring Batch** 为基于 **Spring** 的应用程序提供了这些模式的实现。

**Spring Integration** 解决了实时集成，即数据在可用时进行处理。相反，**Spring Batch** 解决了批量集成的问题，允许在一段时间内收集数据，直到某个触发器（可能是一个时间触发器）发出信号，表示该处理一批数据了。将在第 9 章中研究 **Spring Batch** 和 **Spring Integration**。

## 1.4.6 Spring Cloud

在我写这篇文章的时候，应用程序开发领域正在进入一个新时代，在这个时代中，我们不再将应用程序作为单个部署单元来开发，而是将由几个称为 *微服务* 的单个部署单元组成应用程序。

微服务是一个热门话题，解决了几实际的开发和运行时问题。然而，在这样做的同时，他们也带来了自己的挑战。这些挑战都将由 Spring Cloud 直接面对，Spring Cloud 是一组用 Spring 开发云本地应用程序的项目。

Spring Cloud 覆盖了很多地方，这本书不可能涵盖所有的地方。关于 Spring Cloud 的更完整的讨论，我建议看看 Thomas Vitale 的《Cloud Native Spring in Action》（Manning, 2020, [www.manning.com/books/cloud-native-spring-in-action](http://www.manning.com/books/cloud-native-spring-in-action)）。

### 1.4.7 Spring Native

Spring 中一个相对较新的扩展是 **Spring Native** 项目。这个实验项目使用 GraalVM 本机镜像将 Spring 启动项目编译为本机可执行文件，使镜像的启动速度大大加快，并且更轻量级。

更多信息可访问 <https://github.com/spring-projects-experimental/spring-native>。

## 1.5 小结

- Spring 的目标是让开发人员轻松应对挑战，比如创建 web 应用程序、使用数据库、保护应用程序和使用服务。
- Spring Boot 构建在 Spring 之上，简化了依赖管理、自动配置和运行时监控，让 Spring 变得更加简单。
- Spring 应用程序可以使用 Spring Initializr 进行初始化，它是基于 web 的，并且在大多数 Java 开发环境中都支持它。
- 在 Spring 应用程序上下文中，组件（通常称为 bean）可以用 Java 或 XML 显式地声明，可以通过组件扫描进行发现，也可以用 Spring Boot 进行自动配置。

## 第 2 章 开发 Web 应用程序

本章内容：

- 在浏览器中展示模型数据
- 处理和验证表单输入
- 选择视图模板库

第一印象很重要：好的房屋门面能够让购房者进入房子之前被吸引；一辆车的喷漆工作将会比引擎盖下的东西吸引更多的人；文学作品中充满了一见钟情的故事。内在的东西很重要，但外在的 —— 先看到的 —— 才是重要的。

使用 **Spring** 构建的应用程序将执行各种操作，包括处理数据、从数据库中读取信息以及与其他应用程序进行交互。但是应用程序用户得到的第一印象来自于用户界面。在许多应用程序中，UI 界面是在浏览器中显示的 **web** 应用程序。

在第 1 章中，创建了第一个 **Spring MVC** 控制器来显示应用程序主页。但是 **Spring MVC** 能做的远不止简单地显示静态内容。在本章中，将开发 **Taco Cloud** 应用程序的第一个主要功能 —— 设计自定义 **Taco** 的能力。在此过程中，将深入研究 **Spring MVC**，并了解如何显示模型数据和处理表单输入。



## 2.1 展示信息

从根本上说，Taco Cloud 是一个可以在线订购玉米卷的地方。但除此之外，Taco Cloud 还希望让顾客能够表达自己的创意，从丰富的配料中设计定制的玉米卷。

因此，Taco Cloud web 应用程序需要一个页面来显示玉米卷制作艺术家可以从中选择的配料。选择的原料可能随时改变，所以不应该硬编码到 HTML 页面中。相反，应该从数据库中获取可用配料的列表，并将其提交给页面以显示给客户。

在 Spring web 应用程序中，获取和处理数据是控制器的工作。视图的工作是将数据渲染成 HTML 并显示在浏览器中。将创建以下组件来支持 Taco 创建页面：

- 一个定义玉米卷成分特性的领域实体类
- 一个 Spring MVC 控制器类，它获取成分信息并将其传递给视图
- 一个视图模板，在用户的浏览器中呈现一个成分列表

这些组件之间的关系如图 2.1 所示。

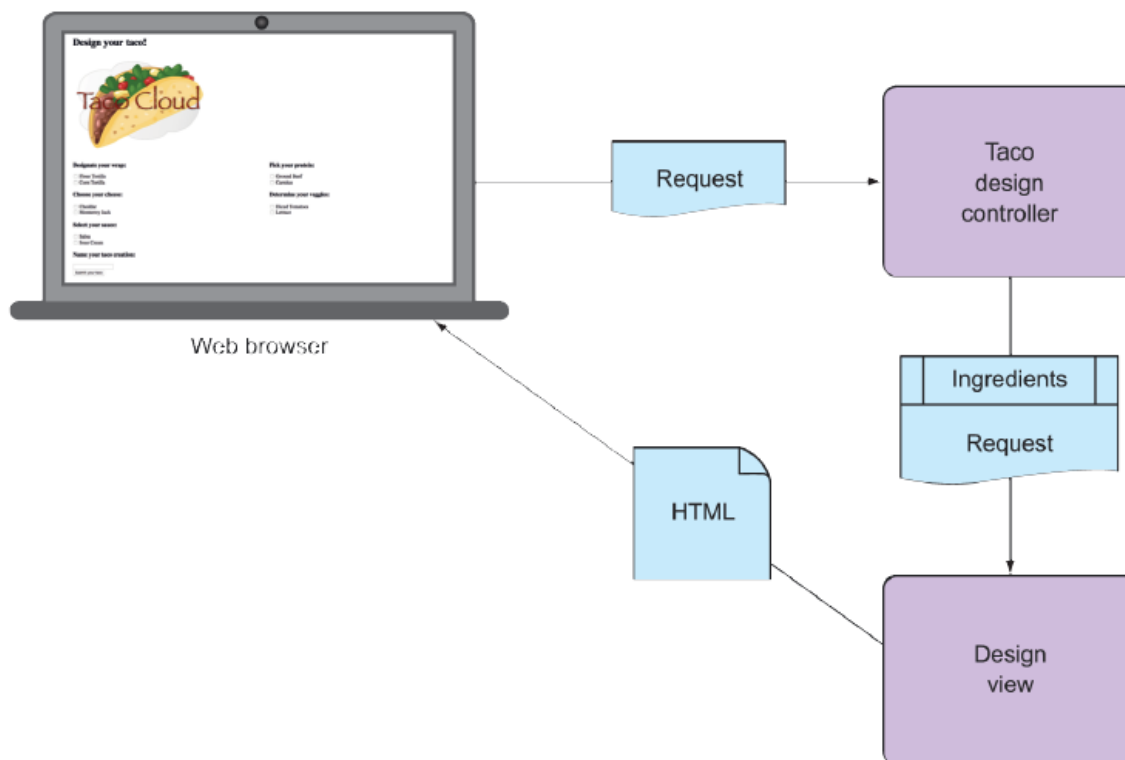


图 2.1 典型 Spring MVC 请求流程

由于本章主要讨论 Spring 的 web 框架，所以我们将把数据库的内容推迟到第 3 章。现在，控制器将单独负责向视图提供组件。在第 3 章中，将重写控制器，使其与从数据库中获取配料数据的存储库进行协作。

在编写控制器和视图之前，让我们先确定表示配料的域类型。这将为开发 web 组件奠定基础。

## 2.1.1 建立领域实体

应用程序的领域实体是它所处理的主题领域——影响应用程序理解的想法和概念。在 Taco Cloud 应用程序中，领域包括 Taco 设计、组成这些设计的成分、客户和客户下的 Taco 订单等对象。图 2.2 显示了这些实体以及它们是如何关联的。

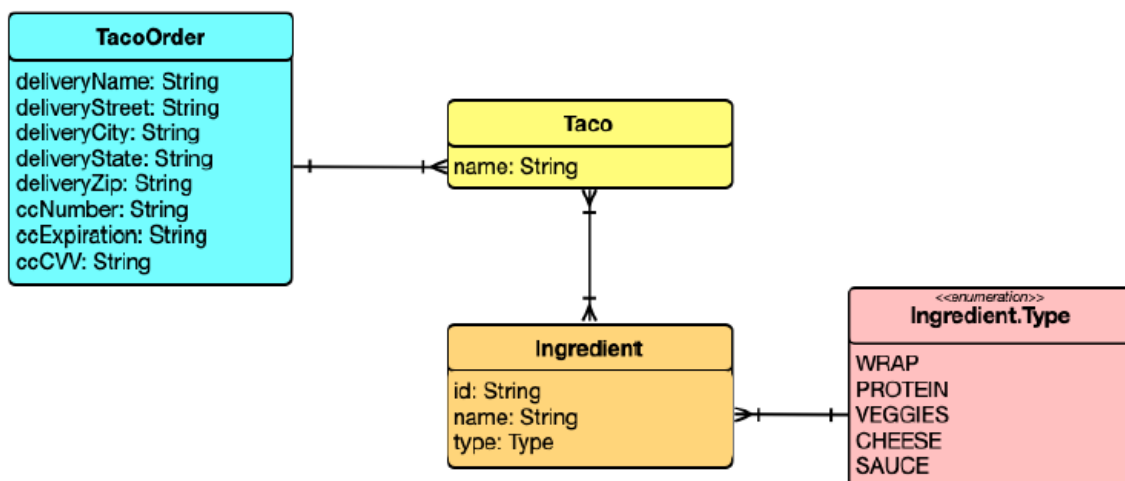


图 2.2 Taco Cloud 相关实体

首先，我们将关注玉米卷配料。在领域中，玉米卷配料是相当简单的对象。每一种都有一个名称和一个类型，这样就可以在视觉上对其进行分类（蛋白质、奶酪、酱汁等）。每一个都有一个 ID，通过这个 ID 可以轻松、明确地引用它。下面的成分类定义了需要的域对象。

### 程序清单 2.1 定义玉米卷的配料

```
package tacos;

import lombok.Data;

@Data
public class Ingredient {

    private final String id;
    private final String name;
    private final Type type;

    public enum Type {
        WRAP, PROTEIN, VEGGIES, CHEESE, SAUCE
    }
}
```

如您所见，这是一个普通的 Java 域类，定义了描述一个成分所需的三个属性。对于程序清单 2.1 中定义的 `Ingredient` 类，最不寻常的事情可能是它似乎缺少一组常用的 `getter` 和 `setter` 方法，更不用说像 `equals()`、`hashCode()`、`toString()` 等有用的方法。

在清单中看不到它们，部分原因是为了节省空间，但也因为使用了一个名为 `Lombok` 的出色库，它会在运行时自动生成这些方法。实际上，类级别的 `@Data` 注释是由 `Lombok` 提供的，它告诉 `Lombok` 生成所有缺少的方法，以及接受所有 `final` 属性作为参数的构造函数。通过使用 `Lombok`，可以让 `Ingredient` 的代码保持整洁。

Lombok 不是一个 Spring 库，但是它非常有用，没有它我很难开发。当我需要在一本书中保持代码示例简短明了时，它就成了我的救星。

要使用 Lombok，需要将其作为依赖项添加到项目中。如果正在使用 Spring Tool Suite，只需右键单击 pom.xml 文件并从 Spring 上下文菜单选项中选择 **Edit Starters** 即可。在第 1 章（图 1.4）中给出的依赖项的相同选择将出现，这样就有机会添加或更改所选的依赖项。找到 Lombok 选项，确保选中，然后单击 OK；Spring Tool Suite 将自动将其添加到构建规范中。

或者，可以使用 pom.xml 中的以下条目手动添加它：

```
<dependency>
  <groupId>org.projectlombok</groupId>
  <artifactId>lombok</artifactId>
</dependency>
```

如果您决定手动将 Lombok 添加到构建中，您还需要将其从 pom.xml 文件 `<build>` 部分中的 Spring Boot Maven 插件中排除：

```
<build>
  <plugins>
    <plugin>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-maven-plugin</artifactId>
      <configuration>
        <excludes>
          <exclude>
            <groupId>org.projectlombok</groupId>
            <artifactId>lombok</artifactId>
          </exclude>
        </excludes>
      </configuration>
    </plugin>
  </plugins>
</build>
```

Lombok 的魔力是在编译时应用的，所以它不需要在运行时可用。像这样排除会使它不出现在 JAR 或 WAR 文件中。

此依赖项将在开发时提供 Lombok 注释（如 `@Data`），并在运行时提供自动方法生成。但是还需要在 IDE 中添加 Lombok 作为扩展，否则 IDE 将会报错缺少方法和没有设置的最终属性。请访问 <https://projectlombok.org/>，以了解如何在 IDE 中安装 Lombok。

注意：为什么我的代码中有这么多错误提示呢？

再重复一遍，在使用 Lombok 时，必须安装 Lombok 插件到您的 IDE 中。没有它，您的 IDE 就不会意识到 Lombok 可提供 getter、setter 和其他方法，并会提示它们缺失。

Lombok 在许多流行的 IDE 中都受支持，包括 Eclipse、Spring Tool Suite、IntelliJ IDEA 和 Visual Studio Code。访问 <https://projectlombok.org/> 可获取有关如何将 Lombok 插件安装进您的 IDE 的详细信息。

您会发现 Lombok 非常有用，但它是可选的。如果不希望使用它，或是不需要它来开发 Spring 应用程序，那么请随意手动编写那些缺少的方法。继续……我将等待。完成后，将添加一些控制器来处理应用程序中的 web 请求。

配料只是玉米卷的基本组成部分。为了了解如何将所有配料组合在一起，我们定义 Taco 类：

程序清单 2.2 定义 **taco** 实体类

```

package tacos;
import java.util.List;
import lombok.Data;

@Data
public class Taco {

    private String name;

    private List<Ingredient> ingredients;

}

```

如您所见，**Taco** 是一个简单的 **Java** 对象，具有几个属性。此外，像 **Ingredient** 类一样，**Taco** 类也用 `@Data` 注释，以使 **Lombok** 在编译时自动生成基础 **JavaBean** 方法。

现在我们已经定义了 **Ingredient** 和 **Taco**，我们还需要定义客户如何指定他们想要订购的玉米卷，以及付款和送货方式信息，这就是 **TacoOrder** 类：

### 程序清单 2.3 **taco** 订单实体类

```

package tacos;
import java.util.List;
import java.util.ArrayList;
import lombok.Data;

@Data
public class TacoOrder {

    private String deliveryName;
    private String deliveryStreet;
    private String deliveryCity;
    private String deliveryState;
    private String deliveryZip;
    private String ccNumber;
    private String ccExpiration;
    private String ccCVV;

    private List<Taco> tacos = new ArrayList<>();

    public void addTaco(Taco taco) {
        this.tacos.add(taco);
    }

}

```

除了拥有比 **Ingredient** 和 **Taco** 更多的属性之外，**TacoOrder** 没有什么特别的。它是一个简单的实体类，有九个属性：5 个用于交付信息，3 个表示付款信息，1 个表示组成交易的 **Taco** 对象列表。还有一个 `addTaco()` 方法，它是为了方便将 **taco** 添加到订单中。

既然定义了实体类，我们就可以让它们工作了。让我们补充一些控制器来处理应用程序中的 **web** 请求。

## 2.1.2 创建控制器类

控制器是 Spring MVC 框架的主要参与者。它们的主要工作是处理 HTTP 请求，或者将请求传递给视图以呈现 HTML（浏览器显示），或者直接将数据写入响应体（RESTful）。在本章中，我们将重点讨论使用视图为 web 浏览器生成内容的控制器的类型。在第 6 章中，我们将讨论如何在 REST API 中编写处理请求的控制器。

对于 Taco Cloud 应用程序，需要一个简单的控制器来执行以下操作：

- 处理请求路径为 `/design` 的 HTTP GET 请求
- 构建配料列表
- 将请求和配料数据提交给视图模板，以 HTML 的形式呈现并发送给请求的 web 浏览器

下面的 `DesignTacoController` 类处理这些需求。

程序清单 2.2 Spring 控制器类的开始

```

package tacos.web;

import java.util.Arrays;
import java.util.List;
import java.util.stream.Collectors;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.ModelAttribute;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.SessionAttributes;

import lombok.extern.slf4j.Slf4j;
import tacos.Ingredient;
import tacos.Ingredient.Type;
import tacos.Taco;

@Slf4j
@Controller
@RequestMapping("/design")
@SessionAttributes("tacoOrder")
public class DesignTacoController {

    @ModelAttribute
    public void addIngredientsToModel(Model model) {
        List<Ingredient> ingredients = Arrays.asList(
            new Ingredient("FLTO", "Flour Tortilla", Type.WRAP),
            new Ingredient("COTO", "Corn Tortilla", Type.WRAP),
            new Ingredient("GRBF", "Ground Beef", Type.PROTEIN),
            new Ingredient("CARN", "Carnitas", Type.PROTEIN),
            new Ingredient("TMT0", "Diced Tomatoes", Type.VEGGIES),
            new Ingredient("LETC", "Lettuce", Type.VEGGIES),
            new Ingredient("CHED", "Cheddar", Type.CHEESE),
            new Ingredient("JACK", "Monterrey Jack", Type.CHEESE),
            new Ingredient("SLSA", "Salsa", Type.SAUCE),
            new Ingredient("SRCR", "Sour Cream", Type.SAUCE)
        );

        Type[] types = Ingredient.Type.values();
        for (Type type : types) {
            model.addAttribute(type.toString().toLowerCase(),
                filterByType(ingredients, type));
        }
    }

    @GetMapping
    public String showDesignForm(Model model) {
        model.addAttribute("taco", new Taco());
        return "design";
    }

    private Iterable<Ingredient> filterByType(
        List<Ingredient> ingredients, Type type) {
        return ingredients
            .stream()
            .filter(x -> x.getType().equals(type))
            .collect(Collectors.toList());
    }
}

```

关于 `DesignTacoController`，首先要注意的是在类级应用的一组注解。第一个是 `@Slf4j`，它是 Lombok 提供的注解，在运行时将自动生成类中的 `SLF4J`（Java 的简单日志门面，<https://www.slf4j.org/>）记录器。这个适当的注解具有与显式地在类中添加以下行相同的效果：

```
private static final org.slf4j.Logger log =
    org.slf4j.LoggerFactory.getLogger(DesignTacoController.class);
```

稍后您将使用这个 `Logger`。

下一个应用到 `DesignTacoController` 的注解是 `@Controller`。此注解用于将该类标识为控制器并将其标记为组件扫描的候选对象，以便 `Spring` 将发现该类并在 `Spring` 应用程序上下文中自动创建 `DesignTacoController` 实例作为 `bean`。

`DesignTacoController` 也用 `@RequestMapping` 注解。`@RequestMapping` 注解在类级应用时，指定该控制器处理的请求的类型。在本例中，它指定 `DesignTacoController` 将处理路径以 `/design` 开头的请求。

## 处理 GET 请求

类级别的 `@RequestMapping` 注解用于 `showDesignForm()` 方法时，可以用 `@GetMapping` 注解进行改进。`@GetMapping` 与类级别的 `@RequestMapping` 配对使用，指定当接收到 `/design` 的 HTTP GET 请求时，`showDesignForm()` 将用来处理请求。

`@GetMapping` 只是一系列请求映射注解中的一个。表 2.1 列出了 `Spring MVC` 中提供的所有请求映射注解的一部分。

表 2.1 `Spring MVC` 请求映射注解

| 注解                           | 描述                |
|------------------------------|-------------------|
| <code>@RequestMapping</code> | 通用请求处理            |
| <code>@GetMapping</code>     | 处理 HTTP GET 请求    |
| <code>@PostMapping</code>    | 处理 HTTP POST 请求   |
| <code>@PutMapping</code>     | 处理 HTTP PUT 请求    |
| <code>@DeleteMapping</code>  | 处理 HTTP DELETE 请求 |
| <code>@PatchMapping</code>   | 处理 HTTP PATCH 请求  |

当 `showDesignForm()` 处理 `/design` 的 GET 请求时，它实际上没有做多少工作。它所做的主要工作是返回一个字符串值“`design`”，这是视图的逻辑名称，将用于将模型渲染到浏览器的。但在此之前，它也会填充给定模型，在一个名为“`design`”的键下有一个空的 `Taco` 对象。这将使可以在表单上创作出一幅墨西哥玉米卷的杰作。

GET 请求似乎没有多大作用。但恰恰相反，`/design` 比 `showDesignForm()` 方法更复杂。您也会注意到，有一个名为 `addIngredientsToModel()` 的方法，该方法用 `@ModelAttribute` 注解。处理请求时也将调用此方法，并将构造一个要放入模型中的配料对象。该列表目前已硬编码。当我们到达第三章，您将从数据库中提取可用的玉米卷配料列表。

一旦准备好了原料列表，接下来的几行 `showDesignForm()` 将根据原料类型过滤该列表。然后将配料类型列表作为属性添加到传递到 `showDesignForm()` 的 `Model` 对象。`Model` 是一个对象，它在控制器和负责呈现数据的视图之间传输数据。最后，放置在 `Model` 类属性中的数据被复制到 `servlet` 响应属性中，视图可以在其中找到它们。`showDesignForm()` 方法最后返回“`design`”，这是将用于向浏览器呈现 `Model` 的视图的逻辑名称。

`DesignTacoController` 真的开始成形了。如果您现在运行应用程序并将您的浏览器指向 `/design` 路径，`DesignTacoController` 的 `showDesignForm()` 将被占用，它从存储库中获取数据并将其放在 `Model` 中，然后将请求传递给视图。但是因为还没有定义视图，所以请求会发生可怕的变化，导致 HTTP 500（`Internal Server Error`）错误。为了解决这个问题，让我们将注意力转移到视图上，其中的数据将用 HTML 进行修饰，并在用户的 web 浏览器中显示。





### 2.1.3 设计视图

控制器创建完成后，就该开始设计视图了。Spring 为定义视图提供了几个很好的选项，包括 `JavaServer Pages (JSP)`、`Thymeleaf`、`FreeMarker`、`Mustache` 和基于 `Groovy` 的模板。现在，我们将使用 `Thymeleaf`，这是我们在第 1 章开始项目时所做的选择。我们将在 2.5 节中考虑其他一些选项。

我们已经在第 1 章中添加了 `Thymeleaf` 依赖项。在运行时，`Spring Boot` 自动配置将看到 `Thymeleaf` 位于类路径中，并将自动创建支持 `Spring MVC` 的 `Thymeleaf` 视图 bean。

像 `Thymeleaf` 这样的视图库被设计成与任何特定的 web 框架解耦。因此，他们不知道 `Spring` 的模型抽象，并且无法处理控制器放置在模型中的数据。但是它们可以处理 `servlet` 请求属性。因此，在 `Spring` 将请求提交给视图之前，它将模型数据复制到请求属性中，而 `Thymeleaf` 和其他视图模板选项可以随时访问这些属性。

`Thymeleaf` 模板只是 `HTML` 与一些额外的元素属性，指导模板在渲染请求数据。例如，如果有一个请求属性，它的键是“`message`”，你希望它被 `Thymeleaf` 渲染成一个 `HTML` `<p>` 标签，你可以在你的 `Thymeleaf` 模板中写以下内容：

```
<p th:text="${message}">placeholder message</p>
```

当模板被呈现为 `HTML` 时，`<p>` 元素的主体将被 `servlet` 请求属性的值替换，其键值为“`message`”。`th:text` 是一个 `Thymeleaf` 的命名空间属性，用于需要执行替换的地方。`${}` 操作符告诉它使用请求属性的值（在本例中为“`message`”）。

`Thymeleaf` 还提供了另一个属性 `th:each`，它遍历元素集合，为集合中的每个项目呈现一次 `HTML`。当设计视图列出模型中的玉米卷配料时，这将非常方便。例如，要呈现“`wrap`”配料列表，可以使用以下 `HTML` 片段：

```
<h3>Designate your wrap:</h3>
<div th:each="ingredient : ${wrap}">
  <input th:field="*{ingredients}" type="checkbox"
    th:value="${ingredient.id}"/>
  <span th:text="${ingredient.name}">INGREDIENT</span><br/>
</div>
```

在这里，我们在 `<div>` 标签中填充 `th:each` 属性，用来对发现于 `wrap` 请求属性中的集合中的每一个项目进行重复呈现。在每次迭代中，成分项都绑定到一个名为 `ingredient` 的 `Thymeleaf` 变量中。

在 `<div>` 元素内部，有一个复选框 `<input>` 元素和一个 `<span>` 元素，用于为复选框提供标签。复选框使用 `Thymeleaf` 的 `th:value` 元素，它将把 `<input>` 元素的 `value` 属性呈现为在成分 `id` 属性中找到的值。`<span>` 元素使用 `th:text` 属性把“`INGREDIENT`”占位符替换为成分 `name` 属性的值。

当使用实际的模型数据呈现时，这个 `<div>` 循环迭代一次可能是这样的：

```
<div>
  <input name="ingredients" type="checkbox" value="FLT0" />
  <span>Flour Tortilla</span><br/>
</div>
```

最后，前面的 `Thymeleaf` 片段只是一个更大的 `HTML` 表单的一部分，通过它，玉米卷艺术家用户将提交他们美味的作品。完整的 `Thymeleaf` 模板（包括所有成分类型和表单）如下所示。

程序清单 2.5 完整的 `design-a-taco` 页面

```

<!DOCTYPE html>
<html xmlns="http://www.w3.org/1999/xhtml"
      xmlns:th="http://www.thymeleaf.org">
  <head>
    <title>Taco Cloud</title>
    <link rel="stylesheet" th:href="@{/styles.css}" />
  </head>

  <body>
    <h1>Design your taco!</h1>
    
    <form method="POST" th:object="${taco}">
      <div class="grid">
        <div class="ingredient-group" id="wraps">
          <h3>Designate your wrap:</h3>
          <div th:each="ingredient : ${wrap}">
            <input name="ingredients" type="checkbox" th:value="${ingredient.id}" />
            <span th:text="${ingredient.name}">INGREDIENT</span><br/>
          </div>
        </div>
        <div class="ingredient-group" id="proteins">
          <h3>Pick your protein:</h3>
          <div th:each="ingredient : ${protein}">
            <input name="ingredients" type="checkbox" th:value="${ingredient.id}" />
            <span th:text="${ingredient.name}">INGREDIENT</span><br/>
          </div>
        </div>
        <div class="ingredient-group" id="cheeses">
          <h3>Choose your cheese:</h3>
          <div th:each="ingredient : ${cheese}">
            <input name="ingredients" type="checkbox" th:value="${ingredient.id}" />
            <span th:text="${ingredient.name}">INGREDIENT</span><br/>
          </div>
        </div>
        <div class="ingredient-group" id="veggies">
          <h3>Determine your veggies:</h3>
          <div th:each="ingredient : ${veggies}">
            <input name="ingredients" type="checkbox" th:value="${ingredient.id}" />
            <span th:text="${ingredient.name}">INGREDIENT</span><br/>
          </div>
        </div>
        <div class="ingredient-group" id="sauces">
          <h3>Select your sauce:</h3>
          <div th:each="ingredient : ${sauce}">
            <input name="ingredients" type="checkbox" th:value="${ingredient.id}" />
            <span th:text="${ingredient.name}">INGREDIENT</span><br/>
          </div>
        </div>
        <div>
          <h3>Name your taco creation:</h3>
          <input type="text" th:field="*{name}" /><br/>
          <button>Submit your taco</button>
        </div>
      </form>
    </body>
  </html>

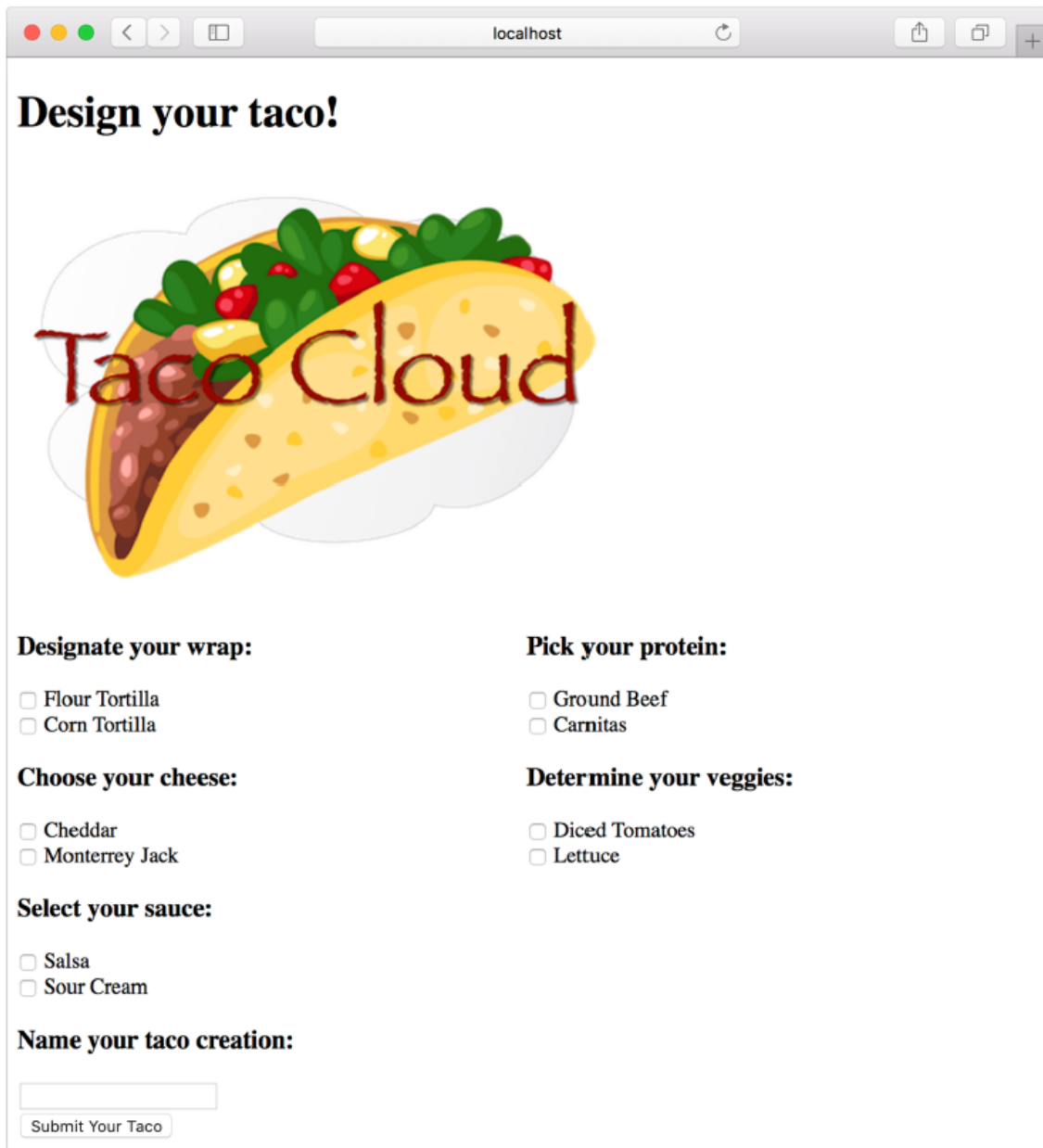
```

可以看到，对于每种类型的配料，都要重复 `<div>` 片段。还包括一个提交按钮和一个字段，用户可以在其中命名他们的创建。

值得注意的是，完整的模板包括 Taco Cloud 图标图片和一个指向样式表的 `<link>` 引用。在这两种情况下，Thymeleaf 的 `@{}` 操作符被用来产生一个上下文相关路径的静态工件，它们正在引用。正如在第 1 章中了解到的，Spring 启动应用程序中的静态内容是从类路径根目录的 `/static` 目录提供的。

现在控制器和视图已经完成，可以启动应用程序了。运行 Spring Boot 应用程序有许多方法。在第 1 章中，展示了如何运行这个应用程序，首先将它构建到一个可执行的 JAR 文件中，然后使用 `java -jar` 运行这个 JAR。展示了如何使用 `mvn spring-boot:run` 从构建中直接运行应用程序。

无论如何启动 Taco Cloud 应用程序，一旦启动，使用浏览器访问 <http://localhost:8080/design>。应该看到类似图 2.3 的页面。



**Design your taco!**



**Designate your wrap:**

- ☐ Flour Tortilla
- ☐ Corn Tortilla

**Pick your protein:**

- ☐ Ground Beef
- ☐ Carnitas

**Choose your cheese:**

- ☐ Cheddar
- ☐ Monterrey Jack

**Determine your veggies:**

- ☐ Diced Tomatoes
- ☐ Lettuce

**Select your sauce:**

- ☐ Salsa
- ☐ Sour Cream

**Name your taco creation:**

图 2.3 呈现的玉米卷设计页面

它看起来真不错！访问这个玉米卷艺术家呈现形式的网站，包含一个调色板的玉米卷成分，从中他们可以创建自己的杰作。但是当他们点击 `Submit Your Taco` 按钮时会发生什么呢？

`DesignTacoController` 还没有准备好接受玉米卷创作的请求。如果提交了设计表单，用户将看到一个错误。（具体来说，它将是一个 **HTTP 405** 错误：请求方法 “**POST**” 不受支持。）让我们通过编写更多处理表单提交的控制器代码来解决这个问题。

## 2.2 处理表单提交

如果在视图中查看 `<form>` 标签，可以看到它的 `method` 属性被设置为 `POST`。而且，`<form>` 没有声明 `action` 属性。这意味着在提交表单时，浏览器将收集表单中的所有数据，并通过 `HTTP POST` 请求将其发送到服务器，发送到显示表单的 `GET` 请求的同一路径——`/design` 路径。

因此，需要在该 `POST` 请求的接收端上有一个控制器处理程序方法。需要在 `DesignTacoController` 中编写一个新的处理程序方法来处理 `/design` 接口的 `POST` 请求。

在程序清单 2.4 中，使用 `@GetMapping` 注释指定 `showDesignForm()` 方法应该处理 `HTTP GET` 请求 `/design`。与 `@GetMapping` 处理 `GET` 请求一样，可以使用 `@PostMapping` 处理 `POST` 请求。为了处理玉米卷艺术家提交的设计，将以下程序清单中的 `processDesign()` 方法添加到 `DesignTacoController` 中。

程序清单 2.6 使用 `@PostMapping` 处理 `POST` 请求

```
@PostMapping
public String processTaco(Taco taco) {
    // Save the taco...
    // We'll do this in chapter 3
    log.info("Processing taco: " + taco);

    return "redirect:/orders/current";
}
```

当应用到 `processDesign()` 方法时，`@PostMapping` 与类级别 `@RequestMapping` 相协调，以表明 `processDesign()` 应该处理 `/design` 接口的 `POST` 请求。这正是需要处理的一个玉米卷艺术家提交的作品。

提交表单时，表单中的字段被绑定到 `Taco` 对象的属性（其类在下一个程序清单中显示），该对象作为参数传递给 `processDesign()`。从这里开始，`processDesign()` 方法可以对 `Taco` 对象做任何它想做的事情。

如果查看程序清单 2.5 中的表单，将看到几个 `checkbox` 元素，它们都带有 `ingredients` 名称和一个名为 `name` 的文本输入元素。表单中的这些字段直接对应于 `Taco` 类的 `ingredients` 和 `name` 属性。

表单上的 `Name` 字段只需要捕获一个简单的文本值。因此 `Taco` 的 `name` 属性的类型是 `String`。配料复选框也有文本值，但是因为可能选择了零个或多个配料，所以它们绑定到的 `ingredients` 属性是一个 `List<String>`，它将捕获每个选择的配料。

但是等等。如果“配料”复选框是文本值，但 `Taco` 对象中的配料列表为 `list<Ingredient>`，那么这根本就不匹配啊？您怎么能把 `["FLTO"、“GRBF”、“LETC”]` 这样的文本列表绑定到一个包含更丰富的对象，不仅包含 `ID`，还包含描述性名称和配料类型上呢？

这就是转换器派上用场的地方。转换器是实现 `Spring` 的 `Converter` 接口并实现其 `convert()` 方法，获取一个值并将其转换为另一个对象的类。要将字符串转换为配料，我们将使用 `IngredientByIdConverter` 如下所示：

程序清单 2.7 将字符串转换为配料对象

```

package tacos.web;

import java.util.HashMap;
import java.util.Map;

import org.springframework.core.convert.converter.Converter;
import org.springframework.stereotype.Component;

import tacos.Ingredient;
import tacos.Ingredient.Type;

@Component
public class IngredientByIdConverter implements Converter<String, Ingredient> {

    private Map<String, Ingredient> ingredientMap = new HashMap<>();

    public IngredientByIdConverter() {
        ingredientMap.put("FLTO", new Ingredient("FLTO", "Flour Tortilla", Type.WRAP));
        ingredientMap.put("COTO", new Ingredient("COTO", "Corn Tortilla", Type.WRAP));
        ingredientMap.put("GRBF", new Ingredient("GRBF", "Ground Beef", Type.PROTEIN));
        ingredientMap.put("CARN", new Ingredient("CARN", "Carnitas", Type.PROTEIN));
        ingredientMap.put("TMT0", new Ingredient("TMT0", "Diced Tomatoes", Type.VEGGIES));
        ingredientMap.put("LETC", new Ingredient("LETC", "Lettuce", Type.VEGGIES));
        ingredientMap.put("CHED", new Ingredient("CHED", "Cheddar", Type.CHEESE));
        ingredientMap.put("JACK", new Ingredient("JACK", "Monterrey Jack", Type.CHEESE));
        ingredientMap.put("SLSA", new Ingredient("SLSA", "Salsa", Type.SAUCE));
        ingredientMap.put("SRCR", new Ingredient("SRCR", "Sour Cream", Type.SAUCE));
    }

    @Override
    public Ingredient convert(String id) {
        return ingredientMap.get(id);
    }

}

```

因为我们还没有配料对象数据库，`IngredientByIdConverter` 的构造函数创建一个 `Map`，该 `Map` 的键是字符串类型，该字符串是配料的 ID，其值是配料对象。在第 3 章中，我们将对该转换器进行调整，从数据库中获取配料数据，而不是像这样硬编码。`convert()` 方法简单地获取一个配料 ID 字符串，并使用它来查找 `Map` 中的配料。

请注意，`IngredientByIdConverter` 用了 `@Component` 注解，使其成为 Spring 应用程序上下文中的 bean。Spring Boot 自动配置将发现这个，以及其他转换器 bean，并自动向 Spring MVC 注册它们，并在将请求参数转换为绑定属性时使用。

目前，`processDesign()` 方法对 `Taco` 对象没有任何作用。事实上，它什么都做不了。没关系。在第 3 章中，将添加一些持久性逻辑，将提交的 `Taco` 保存到数据库中。

与 `showDesignForm()` 方法一样，`processDesign()` 通过返回一个 `String` 结束。与 `showDesignForm()` 类似，返回的值指示将显示给用户的视图。但是不同的是，从 `processDesign()` 返回的值的前缀是“`redirect:`”，表示这是一个重定向视图。更具体地说，它表明在 `processDesign()` 完成之后，用户的浏览器应该被重定向到相对路径 `/order/current`。

这样做的想法源于，在创建了一个玉米卷之后，用户将被重定向到一个订单表单，他们可以从该表单下订单，以交付他们的玉米卷。但是还没有一个控制器来处理 `/orders/current` 请求。

根据现在对 `@Controller`、`@RequestMapping` 和 `@GetMapping` 的了解，可以轻松创建这样的控制器。它可能类似于下面的清单。

#### 程序清单 2.8 展现玉米卷订单表单的控制器

```
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestMapping;

import lombok.extern.slf4j.Slf4j;
import tacos.TacoOrder;

@Slf4j
@Controller
@RequestMapping("/orders")
public class OrderController {

    @GetMapping("/current")
    public String orderForm(Model model) {
        model.addAttribute("tacoOrder", new TacoOrder());
        return "orderForm";
    }
}
```

同样，可以使用 Lombok 的 `@Slf4j` 注释在运行时创建一个 SLF4J Logger 对象。稍后，将使用这个 Logger 来记录提交的订单的详细信息。

类级别的 `@RequestMapping` 指定该控制器中的任何请求处理方法都将处理路径以 `/orders` 开头的请求。当与方法级 `@GetMapping` 结合使用时，它指定 `orderForm()` 方法将处理 `/orders/current` 的 HTTP GET 请求。

至于 `orderForm()` 方法本身，它非常简单，只返回 `orderForm` 的逻辑视图名。在第 3 章中，一旦有了把创建的 `taco` 持久化到数据库的方法，将重新访问该方法并修改它，以使用 `taco` 对象的列表填充模型，这些对象将按顺序放置。

`orderForm` 视图由一个名为 `orderForm.html` 的 Thymeleaf 模板提供，如下面显示的。

#### 程序清单 2.9 `taco` 订单表单视图

```

<!DOCTYPE html>
<html xmlns="http://www.w3.org/1999/xhtml"
      xmlns:th="http://www.thymeleaf.org">
  <head>
    <title>Taco Cloud</title>
    <link rel="stylesheet" th:href="@{/styles.css}" />
  </head>

  <body>
    <form method="POST" th:action="@{/orders}" th:object="${order}">
      <h1>Order your taco creations!</h1>
      
      <a th:href="@{/design}" id="another">Design another taco</a><br/>
      <div th:if="${#fields.hasErrors()}">
        <span class="validationError">
          Please correct the problems below and resubmit.
        </span>
      </div>
      <h3>Deliver my taco masterpieces to...</h3>

      <label for="name">Name: </label>
      <input type="text" th:field="**{name}" />
      <br/>

      <label for="street">Street address: </label>
      <input type="text" th:field="**{street}" />
      <br/>

      <label for="city">City: </label>
      <input type="text" th:field="**{city}" />
      <br/>

      <label for="state">State: </label>
      <input type="text" th:field="**{state}" />
      <br/>

      <label for="zip">Zip code: </label>
      <input type="text" th:field="**{zip}" />
      <br/>

      <h3>Here's how I'll pay...</h3>

      <label for="ccNumber">Credit Card #: </label>
      <input type="text" th:field="**{ccNumber}" />
      <br/>

      <label for="ccExpiration">Expiration: </label>
      <input type="text" th:field="**{ccExpiration}" />
      <br/>

      <label for="ccCVV">CVV: </label>
      <input type="text" th:field="**{ccCVV}" />
      <br/>

      <input type="submit" value="Submit order" />
    </form>
  </body>
</html>

```

在大多数情况下，`orderForm.html` 视图是典型的 HTML/Thymeleaf 内容，没有什么值得注意的。但是注意，这里的 `<form>` 标记与程序清单 2.5 中使用的 `<form>` 标记不同，因为它还指定了一个表单操作。如果没有指定操作，表单将向呈现表单的相同 URL 提交 HTTP POST 请求。但是在这里，指定表单应该提交到 `/orders`（使用 Thymeleaf 的 `@{...}` 操作符作为上下文相关路径）。



因此，需要添加另外一个方法到 `OrderController` 类中，去处理 `/orders` 接口的 `POST` 请求。在进行到下一章之前，还没有办法将订单持久化，因此在这里简化它 —— 类似于在下一个程序清单中看到的内容。

#### 程序清单 2.10 处理 `taco` 订单提交

```
@PostMapping
public String processOrder(TacoOrder order) {
    log.info("Order submitted: " + order);
    return "redirect:/";
}
```

当调用 `processOrder()` 方法来处理提交的订单时，它将获得一个 `order` 对象，其属性绑定到提交的表单字段。`Order` 非常像 `Taco`，是一个相当简单的类，它携带订单信息。

现在已经开发了一个 `OrderController` 和 `order` 表单视图，可以开始尝试运行了。打开浏览器访问 <http://localhost:8080/design>，为您的玉米卷选择一些原料，然后点击 `Submit Your Taco` 按钮。应该会看到类似于图 2.4 所示的表单。

Order your taco creations!



[Design another taco](#)

**Deliver my taco masterpieces to...**

Name:

Street address:

City:

State:

Zip code:

**Here's how I'll pay...**

Credit Card #:

Expiration:

CVV:

图 2.4 taco 订单表单

在表单中填写一些字段，然后按 `Submit Order` 按钮。与此同时，请密切关注应用程序日志，以查看订单信息。当我尝试它，日志条目看起来像这样（重新格式化以适应这个页面的宽度）：

```
Order submitted: TacoOrder(deliveryName=Craig Walls, deliveryStreet=1234 7th
Street, deliveryCity=Somewhere, deliveryState=Who knows?, deliveryZip=zipzap,
ccNumber=Who can guess?, ccExpiration=Some day, ccCVV=See-vee-vee, tacos=[])
```

如果仔细查看来自测试订单的日志条目，可以看到，虽然 `processOrder()` 方法完成了它的工作并处理了表单提交，但是它让一些错误的信息进来了。表单中的大多数字段包含的数据可能是不正确的。需要添加一些验证，以确保提供的数据至少与所需的信息类型相似。

## 2.3 验证表单输入

当设计一个新的 **taco** 产品时，如果用户没有选择任何食材或者没有为他们的产品指定名称，该怎么办？当提交订单时，如果他们没有填写所需的地址字段，该怎么办？或者，如果他们在信用卡字段中输入的值甚至不是有效的信用卡号，该怎么办？

按照目前的情况，没有什么能阻止用户创建一个没有任何配料或空空如也的送货地址的玉米饼，甚至提交他们最喜欢的歌曲的歌词作为信用卡号码。这是因为还没有指定应该如何验证这些字段。

执行表单验证的一种方法是在 `processDesign()` 和 `processOrder()` 方法中加入一堆 `if/then` 块，检查每个字段以确保它满足适当的验证规则。但是这样做会很麻烦，并且难于阅读和调试。

幸运的是，Spring 支持 Java's Bean Validation API（也称为 JSR-303；<https://jcp.org/en/jsr/detail?id=303>）。这使得声明验证规则比在应用程序代码中显式地编写声明逻辑更容易。使用 Spring Boot，不需要做任何特殊的事情来将验证库添加到项目中，因为 Validation API 和 Validation API 的 Hibernate 实现作为 Spring Boot web 启动程序的临时依赖项自动添加到了项目中。

要在 Spring MVC 中应用验证，需要这样做：

- 工程中添加 Spring 校验 starter。
- 对要验证的类声明验证规则：特别是 **Taco** 类。
- 指定验证应该在需要验证的控制器方法中执行，具体来说就是：**DesignTacoController** 的 `processDesign()` 方法和 **OrderController** 的 `processOrder()` 方法。
- 修改表单视图以显示验证错误。

Validation API 提供了几个可以放在域对象属性上声明验证规则的注解。Hibernate 的 Validation API 实现甚至添加了更多的验证注解。通过将 Spring 校验 starter 添加到项目中，可以将两者都添加到项目构建中。Spring Boot Starters 向导中，勾选 I/O 下的“Validation”复选框将完成这个工作。但是如果您更喜欢手动编辑，那么在 Maven 的 `pom.xml` 文件可以手工添加以下内容：

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-validation</artifactId>
</dependency>
```

或者，如果您使用的是 Gradle，那么这就是您需要的依赖项：

```
implementation 'org.springframework.boot:spring-boot-starter-validation'
```

注意：是否需要明确添加 Spring Validation starter？

在 Spring Boot 的早期版本中，Spring Validation starter 是自动包含在 `web statert` 中的。从 Spring Boot 2.3.0 开始，如果要应用验证，则需要显式地将其添加到项目构建中。

添加好了校验 starter 以后，让我们看看如何应用这些注解来验证提交的 **Taco** 或 **TacoOrder**

### 2.3.1 声明验证规则

对于 **Taco** 类，希望确保 **name** 属性不是空的或 **null** 的，并且所选配料列表中至少有一项。下面的程序清单显示了一个更新后的 **Taco** 类，它使用 `@NotNull` 和 `@Size` 来声明这些验证规则。

程序清单 2.11 为 **Taco** 域类添加验证

```
package tacos;
import java.util.List;
import javax.validation.constraints.NotNull;
import javax.validation.constraints.Size;
import lombok.Data;

@Data
public class Taco {

    @NotNull
    @Size(min=5, message="Name must be at least 5 characters long")
    private String name;

    @NotNull
    @Size(min=1, message="You must choose at least 1 ingredient")
    private List<Ingredient> ingredients;
}
```

您会发现，除了要求 **name** 属性不为 **null**，同时您声明它应该有一个值是至少 5 个字符的长度。

当涉及到对提交玉米饼订单进行验证声明时，必须对 **Order** 类应用注解。对于地址的属性，只需要确保用户没有留下任何空白字段。对于这一点，将使用 **Hibernate Validator** 的 `@NotBlank` 注解。

支付领域的验证是一个比较奇特的存在。您不仅需要确保 **ccNumber** 属性不为空，还要确保它包含的是一个有效的信用卡号码的值。该 **ccExpiration** 属性必须符合 **MM/YY**（两位数的年/月）格式。而 **ccCVV** 属性必须是一个三位的数字。为了实现这种验证，需要使用一些其他的 **Java Bean Validation API** 注解，同时需要从 **Hibernate Validator** 集合中借用一些验证注解。下面程序清单列出了验证 **TacoOrder** 类所需要的改变。

程序清单 2.12 验证 **Order** 属性字段

```

package tacos;
import javax.validation.constraints.Digits;
import javax.validation.constraints.NotBlank;
import javax.validation.constraints.Pattern;
import org.hibernate.validator.constraints.CreditCardNumber;
import java.util.List;
import java.util.ArrayList;
import lombok.Data;

@Data
public class TacoOrder {

    @NotBlank(message="Delivery name is required")
    private String deliveryName;

    @NotBlank(message="Street is required")
    private String deliveryStreet;

    @NotBlank(message="City is required")
    private String deliveryCity;

    @NotBlank(message="State is required")
    private String deliveryState;

    @NotBlank(message="Zip code is required")
    private String deliveryZip;

    @CreditCardNumber(message="Not a valid credit card number")
    private String ccNumber;

    @Pattern(regexp="^(0[1-9]|1[0-2])([\\/])([1-9][0-9])$",
            message="Must be formatted MM/YY")
    private String ccExpiration;

    @Digits(integer=3, fraction=0, message="Invalid CVV")
    private String ccCVV;

    private List<Taco> tacos = new ArrayList<>();

    public void addTaco(Taco taco) {
        this.tacos.add(taco);
    }
}

```

可以看到，`ccNumber` 属性用 `@CreditCardNumber` 进行了注解。该注解声明属性的值必须是通过 Luhn 算法（[https://en.wikipedia.org/wiki/Luhn\\_algorithm](https://en.wikipedia.org/wiki/Luhn_algorithm)）检查过的有效信用卡号。这可以防止用户出错的数据和故意错误的数据，但不能保证信用卡号码实际上被分配到一个帐户，或该帐户可以用于交易。

不幸的是，没有现成的注解来验证 `ccExpiration` 属性的 `MM/YY` 格式。我已经应用了 `@Pattern` 注解，为它提供了一个正则表达式，以确保属性值符合所需的格式。如果想知道如何破译正则表达式，我建议查看许多在线正则表达式指南，包括 <http://www.regularexpressions.info/>。正则表达式语法是一门黑暗的艺术，当然也超出了本书的范围。

最后，用 `@Digits` 注解 `ccCVV` 属性，以确保值恰好包含三个数字。

所有的验证注解都包含一个消息属性，该属性定义了如果用户输入的信息不符合声明的验证规则的要求时将显示给用户的消息。

### 2.3.2 在表单绑定时执行验证

既然已经声明了应该如何验证 `Taco` 和 `TacoOrder`，那么我们需要重新访问每个控制器，并指定应该在将表单提交到各自的处理程序方法时执行验证。

要验证提交的 `Taco`，需要将 Java Bean Validation API 的 `@Valid` 注解添加到 `DesignTacoController` 的 `processDesign()` 方法的 `Taco` 参数中。

程序清单 2.13 验证 POST 来的 `Taco`

```
import javax.validation.Valid;
import org.springframework.validation.Errors;

...

@PostMapping
public String processTaco(@Valid @ModelAttribute("taco") Taco taco, Errors errors) {
    if (errors.hasErrors()) {
        return "design";
    }
    // Save the taco...
    // We'll do this in chapter 3
    log.info("Processing taco: " + taco);

    return "redirect:/orders/current";
}
```

`@Valid` 注解告诉 Spring MVC 在提交的 `Taco` 对象绑定到提交的表单数据之后，以及调用 `processDesign()` 方法之前，对提交的 `Taco` 对象执行验证。如果存在任何验证错误，这些错误的详细信息将在传递到 `processDesign()` 的错误对象中捕获。`processDesign()` 的前几行查询 `Errors` 对象，询问它的 `hasErrors()` 方法是否存在任何验证错误。如果有，该方法结束时不处理 `Taco`，并返回“design”视图名，以便重新显示表单。

要对提交的 `TacoOrder` 对象执行验证，还需要对 `OrderController` 的 `processOrder()` 方法进行类似的更改。

程序清单 2.14 验证 POST 来的 `TacoOrder`

```
@PostMapping
public String processOrder(@Valid TacoOrder order, Errors errors) {
    if (errors.hasErrors()) {
        return "orderForm";
    }

    log.info("Order submitted: " + order);
    return "redirect:/";
}
```

在这两种情况下，如果没有验证错误，则允许该方法处理提交的数据。如果存在验证错误，则请求将被转发到表单视图，以便用户有机会纠正其错误。

但是用户如何知道哪些错误需要改正呢？除非调出表单上的错误，否则用户将只能猜测如何成功提交表单。

### 2.3.3 显示验证错误

Thymeleaf 通过 `fields` 属性及其 `th:errors` 属性提供了对 `Errors` 对象的便捷访问。例如，要在信用卡号字段上显示验证错误，可以添加一个 `<span>` 元素，该元素将这些错误引用用于订单模板，如下所示。

程序清单 2.15 显示验证错误

```
<label for="ccNumber">Credit Card #: </label>
<input type="text" th:field="*{ccNumber}"/>
<span class="validationError"
      th:if="${#fields.hasErrors('ccNumber')}"
      th:errors="*{ccNumber}">CC Num Error</span>
```

除了可以用来设置错误样式以引起用户注意的 `class` 属性外，`<span>` 元素还使用 `th:if` 属性来决定是否显示 `<span>`。 `fields` 属性的 `hasErrors()` 方法检查 `ccNumber` 字段中是否有任何错误。如果有错误，`<span>` 将被渲染。

`th:errors` 属性引用 `ccNumber` 字段，并且假设该字段存在错误，它将用验证消息替换 `<span>` 元素的占位符内容。

如果在其他字段的订单表单周围使用类似的 `<span>` 标记，则在提交无效信息时可能会看到类似图 2.5 的表单。这些错误表明姓名、城市和邮政编码字段被留空，所有的支付字段都不符合验证标准。

localhost

# Order your taco creations!



[Design another taco](#)

Please correct the problems below and resubmit.

## Deliver my taco masterpieces to...

Name:  Name is required

Street address:

City:  City is required

State:

Zip code:  Zip code is required

## Here's how I'll pay...

Credit Card #:  Not a valid credit card number

Expiration:  Must be formatted MM/YY

CVV:  Invalid CVV

图 2.5 在订单表单上显示验证错误

现在 Taco Cloud 控制器不仅可以显示和捕获输入，还可以验证信息是否符合一些基本的验证规则。让我们后退一步，重新考虑第 1 章中的 HomeController，看看另一种实现。



## 2.4 使用视图控制器

到目前为止，已经为 Taco Cloud 应用程序编写了三个控制器。尽管每个控制器在应用程序的功能上都有不同的用途，但它们几乎都遵循相同的编程模型：

- 它们都用 `@Controller` 进行了注解，以表明它们是控制器类，应该由 Spring 组件扫描自动发现，并在 Spring 应用程序上下文中作为 bean 进行实例化。
- 除了 `HomeController` 之外，所有的控制器都在类级别上使用 `@RequestMapping` 进行注解，以定义控制器将处理的基本请求模式。
- 它们都有一个或多个方法，这些方法都用 `@GetMapping` 或 `@PostMapping` 进行了注解，以提供关于哪些方法应该处理哪些请求的细节。

即将编写的大多数控制器都将遵循这种模式。但是，如果一个控制器足够简单，不填充模型或流程输入（就像 `HomeController` 一样），那么还有另一种定义控制器的方法。请查看下一个程序清单，了解如何声明视图控制器——一个只将请求转发给视图的控制器。

程序清单 2.16 声明视图控制器

```
package tacos.web;

import org.springframework.context.annotation.Configuration;
import org.springframework.web.servlet.config.annotation.ViewControllerRegistry;
import org.springframework.web.servlet.config.annotation.WebMvcConfigurer;

@Configuration
public class WebConfig implements WebMvcConfigurer {

    @Override
    public void addViewControllers(ViewControllerRegistry registry) {
        registry.addViewController("/").setViewName("home");
    }

}
```

关于 `@WebConfig` 最值得注意的是它实现了 `WebMvcConfigurer` 接口。`WebMvcConfigurer` 定义了几个配置 Spring MVC 的方法。尽管它是一个接口，但它提供了所有方法的默认实现，因此只需覆盖所需的方法。在本例中，覆盖了 `addViewControllers()` 方法。

`addViewControllers()` 方法提供了一个 `ViewControllerRegistry`，可以使用它来注册一个或多个视图控制器。在这里，在注册表上调用 `addViewController()`，传入 `/`，这是视图控制器处理 GET 请求的路径。该方法返回一个 `ViewControllerRegistration` 对象，在该对象上立即调用 `setViewName()` 来指定 `home` 作为应该转发 `/` 请求的视图。

就像这样，已经能够用配置类中的几行代码替换 `HomeController`。现在可以删除 `HomeController`，应用程序的行为应该与以前一样。惟一需要做的其他更改是重新访问第 1 章中的 `HomeControllerTest`，从 `@WebMvcTest` 注解中删除对 `HomeController` 的引用，这样测试类就可以无错误地编译了。

这里，已经创建了一个新的 `WebConfig` 配置类来存放视图控制器声明。但是任何配置类都可以实现 `WebMvcConfigurer` 并覆盖 `addViewController()` 方法。例如，可以将相同的视图控制器声明添加到引导 `TacoCloudApplication` 类中，如下所示：

```
@SpringBootApplication
public class TacoCloudApplication implements WebMvcConfigurer {

    public static void main(String[] args) {
        SpringApplication.run(TacoCloudApplication.class, args);
    }

    @Override
    public void addViewControllers(ViewControllerRegistry registry) {
        registry.addViewController("/").setViewName("home");
    }
}
```

通过扩展现有的配置类，可以避免创建新的配置类，从而降低项目工件数量。但是我倾向于为每种配置（**web**、数据、安全性等等）创建一个新的配置类，保持应用程序引导配置的简洁。

说到视图控制器，更一般地说，是控制器将请求转发给的视图，到目前为止，已经为所有视图使用了 **Thymeleaf**。我非常喜欢 **Thymeleaf**，但也许你更喜欢应用程序视图的不同模板模型。让我们看看 **Spring** 支持的许多视图选项。

## 2.5 选择视图模板库

在大多数情况下，对视图模板库的选择取决于个人喜好。Spring 非常灵活，支持许多常见的模板选项。除了一些小的例外，所选择的模板库本身甚至不知道它是在 Spring 中工作的。表 2.2 列出了 Spring Boot 自动配置支持的模板选项。

表 2.2 支持的模板选项

| 模板                    | Spring Boot starter 依赖               |
|-----------------------|--------------------------------------|
| FreeMarker            | spring-boot-starter-freemarker       |
| Groovy Templates      | spring-boot-starter-groovy-templates |
| JavaServer Page (JSP) | None (provided by Tomcat or Jetty)   |
| Mustache              | spring-boot-starter-mustache         |
| Thymeleaf             | spring-boot-starter-thymeleaf        |

一般来说，可以选择想要的视图模板库，将其作为依赖项添加到构建中，然后开始在 `/templates` 目录中（在 Maven 或 Gradle 构建项目的 `src/main/resources` 目录下）编写模板。Spring Boot 将检测选择的模板库，并自动配置所需的组件来为 Spring MVC 控制器提供视图。

已经在 Taco Cloud 应用程序中用 Thymeleaf 实现了这一点。在第 1 章中，在初始化项目时选择了 Thymeleaf 复选框。这导致 Spring Boot 的 Thymeleaf starter 被包含在 `pom.xml` 文件中。当应用程序启动时，Spring Boot 自动配置会检测到 Thymeleaf 的存在，并自动配置 Thymeleaf bean。现在要做的就是开始在 `/templates` 中编写模板。

如果希望使用不同的模板库，只需在项目初始化时选择它，或者编辑现有的项目构建以包含新选择的模板库。

例如，假想使用 Mustache 而不是 Thymeleaf。没有问题。只需访问项目 `pom.xml` 文件，将：

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-thymeleaf</artifactId>
</dependency>
```

替换为：

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-mustache</artifactId>
</dependency>
```

当然，需要确保使用 Mustache 语法而不是 Thymeleaf 标签来编写所有模板。Mustache 的使用细节（或选择的任何模板语言）不在这本书的范围之内，但为了让你知道会发生什么，这里有一个从 Mustache 模板摘录过来的片段，这个片段渲染了玉米卷设计表单的成分列表中的一个：

```
<h3>Designate your wrap:</h3>
{{#wrap}}
<div>
  <input name="ingredients" type="checkbox" value="{{id}}" />
  <span>{{name}}</span><br/>
</div>
{{/wrap}}
```

这是 **Mustache** 与第 2.1.3 节中的 **Thymeleaf** 片段的等价替换。{{#wrap}} 块（以 {{/wrap}} 结尾）迭代 request 属性中的一个集合，该集合的键为 wrap，并为每个项目呈现嵌入的 HTML。{{id}} 和 {{name}} 标记引用项目的 id 和 name 属性（应该是一个 Ingredient）。

在表 2.2 中请注意，JSP 在构建中不需要任何特殊的依赖关系。这是因为 servlet 容器本身（默认情况下是 Tomcat）实现了 JSP 规范，因此不需要进一步的依赖关系。

但是如果选择使用 JSP，就会遇到一个问题。事实证明，Java servlet 容器——包括嵌入式 Tomcat 和 Jetty 容器——通常在 /WEB-INF 下寻找 jsp。但是如果将应用程序构建为一个可执行的 JAR 文件，就没有办法满足这个需求。因此，如果将应用程序构建为 WAR 文件并将其部署在传统的 servlet 容器中，那么 JSP 只是一个选项。如果正在构建一个可执行的 JAR 文件，必须选择 Thymeleaf、FreeMarker 或表 2.2 中的其他选项之一。

## 2.5.1 缓存模板

默认情况下，模板在第一次使用时只解析一次，解析的结果被缓存以供后续使用。对于生产环境来说，这是一个很好的特性，因为它可以防止对每个请求进行冗余的模板解析，从而提高性能。

但是，在开发时，这个特性并不那么好。假设启动了应用程序并点击了玉米卷设计页面，并决定对其进行一些更改。当刷新 **web** 浏览器时，仍然会显示原始版本。查看更改的惟一方法是重新启动应用程序，这非常不方便。

幸运的是，有一种方法可以禁用缓存。只需将 `templateappropriate` 高速缓存属性设置为 `false`。表 2.3 列出了每个支持的模板库的缓存属性。

表 2.3 启用/禁用模板缓存的属性

| 模板               | 缓存使能属性                                    |
|------------------|-------------------------------------------|
| Freemarker       | <code>spring.freemarker.cache</code>      |
| Groovy Templates | <code>spring.groovy.template.cache</code> |
| Mustache         | <code>spring.mustache.cache</code>        |
| Thymeleaf        | <code>spring.thymeleaf.cache</code>       |

默认情况下，所有这些属性都设置为 `true` 以启用缓存。可以通过将其缓存属性设置为 `false` 来禁用所选模板引擎的缓存。例如，要禁用 Thymeleaf 缓存，请在 `application.properties` 中添加以下行：

```
spring.thymeleaf.cache = false
```

惟一的问题是，在将应用程序部署到生产环境之前，一定要删除这一行(或将其设置为 `true`)。一种选择是在 `profile` 文件中设置属性。（我们将在第 5 章讨论 `profiles` 文件。）

一个更简单的选择是使用 Spring Boot 的 DevTools，就像我们在第 1 章中选择的那样。在 DevTools 提供的许多有用的开发时帮助中，它将禁用所有模板库的缓存，但在部署应用程序时将禁用自身（从而重新启用模板缓存）。

## 2.6 总结

- Spring 提供了一个强大的 web 框架，称为 Spring MVC，可以用于开发 Spring 应用程序的 web 前端。
- Spring MVC 是基于注解的，可以使用 `@RequestMapping`、`@GetMapping` 和 `@PostMapping` 等注解来声明请求处理方法。
- 大多数请求处理方法通过返回视图的逻辑名称来结束，例如一个 Thymeleaf 模板，请求（以及任何模型数据）被转发到该模板。
- Spring MVC 通过 Java Bean Validation API 和 Hibernate Validator 等验证 API 的实现来支持验证。
- 视图控制器可以用来处理不需要模型数据或处理的 HTTP GET 请求。
- 除了 Thymeleaf，Spring 还支持多种视图选项，包括 FreeMarker、Groovy Templates 和 Mustache。

## 第 3 章 处理数据

本章内容：

- 使用 Spring JdbcTemplate
- 创建 Spring Data JDBC Repository
- 使用 Spring Data 声明 JPA Repository

大多数应用程序提供的不仅仅是一张漂亮的脸。虽然用户界面可能提供与应用程序的交互，但它所呈现和存储的数据将应用程序与静态网站区分开来。

在 **Taco Cloud** 应用程序中，需要能够维护关于 **ingredients**、**tacos** 和 **orders** 的信息。如果没有一个数据库来存储这些信息，应用程序将无法比在第 2 章中开发的应用程序取得更大的进展。

在本章中，将向 **Taco Cloud** 应用程序添加数据持久化操作。首先使用 **Spring** 对 **JDBC**（**Java Database Connectivity**）的支持来消除样板代码。然后，将重新使用 **JPA**（**Java Persistence API**）处理数据存储库，从而消除更多代码。

## 3.1 使用 JDBC 读写数据

几十年来，关系数据库和 SQL 一直是数据持久化的首选。尽管近年来出现了许多替代数据库类型，但关系数据库仍然是通用数据存储的首选，而且不太可能很快被取代。

在处理关系数据时，Java 开发人员有多个选择。两个最常见的选择是 JDBC 和 JPA。Spring 通过抽象支持这两种方式，这使得使用 JDBC 或 JPA 比不使用 Spring 更容易。在本节中，我们将重点讨论 Spring 是如何支持 JDBC 的，然后在第 3.2 节中讨论 Spring 对 JPA 的支持。

Spring JDBC 支持起源于 `JdbcTemplate` 类。`JdbcTemplate` 提供了一种方法，通过这种方法，开发人员可以对关系数据库执行 SQL 操作，与通常使用 JDBC 不同的是，这里不需要满足所有的条件和样板代码。

为了更好地理解 `JdbcTemplate` 的作用，我们首先来看一个示例，看看如何在没有 `JdbcTemplate` 的情况下用 Java 执行一个简单的查询。

程序清单 3.1 不使用 `JdbcTemplate` 查询数据库

```
@Override
public Optional<Ingredient> findById(String id) {
    Connection connection = null;
    PreparedStatement statement = null;
    ResultSet resultSet = null;
    try {
        connection = dataSource.getConnection();
        statement = connection.prepareStatement(
            "select id, name, type from Ingredient");
        statement.setString(1, id);
        resultSet = statement.executeQuery();
        Ingredient ingredient = null;
        if(resultSet.next()) {
            ingredient = new Ingredient(
                resultSet.getString("id"),
                resultSet.getString("name"),
                Ingredient.Type.valueOf(resultSet.getString("type")));
        }
        return Optional.of(ingredient);
    } catch (SQLException e) {
        // ??? What should be done here ???
    } finally {
        if (resultSet != null) {
            try {
                resultSet.close();
            } catch (SQLException e) {}
        }
        if (statement != null) {
            try {
                statement.close();
            } catch (SQLException e) {}
        }
        if (connection != null) {
            try {
                connection.close();
            } catch (SQLException e) {}
        }
    }
    return null;
}
```



在程序清单 3.1 的某个地方，有几行代码用于查询数据库中的 `ingredients`。但是很难在 JDBC 的混乱代码中找到查询指针。它被创建连接、创建语句和通过关闭连接、语句和结果集来清理的代码所包围。

更糟糕的是，在创建连接或语句或执行查询时，可能会出现许多问题。这要求捕获一个 `SQLException`，这可能有助于（也可能无助于）找出问题出在哪里或如何解决问题。

`SQLException` 是一个被检查的异常，它需要在 `catch` 块中进行处理。但是最常见的问题，如未能创建到数据库的连接或输入错误的查询，不可能在 `catch` 块中得到解决，可能会重新向上抛出以求处理。相反，要是考虑使用 `JdbcTemplate` 的方法。

### 程序清单 3.2 使用 `JdbcTemplate` 查询数据库

```
private JdbcTemplate jdbcTemplate;

public Optional<Ingredient> findById(String id) {
    List<Ingredient> results = jdbcTemplate.query(
        "select id, name, type from Ingredient where id=?",
        this::mapRowToIngredient,
        id);
    return results.size() == 0 ?
        Optional.empty() :
        Optional.of(results.get(0));
}

private Ingredient mapRowToIngredient(ResultSet row, int rowNum)
    throws SQLException {
    return new Ingredient(
        row.getString("id"),
        row.getString("name"),
        Ingredient.Type.valueOf(row.getString("type")));
}
```

程序清单 3.2 中的代码显然比程序清单 3.1 中的原始 JDBC 示例简单得多；没有创建任何语句或连接。而且，在方法完成之后，不会对那些对象进行任何清理。最后，这样做不会存在任何在 `catch` 块中不能处理的异常。剩下的代码只专注于执行查询（调用 `JdbcTemplate` 的 `queryForObject()` 方法）并将结果映射到 `Ingredient` 对象（在 `mapRowToIngredient()` 方法中）。

程序清单 3.2 中的代码是使用 `JdbcTemplate` 在 `Taco Cloud` 应用程序中持久化和读取数据所需要做的工作的一个片段。让我们采取下一步必要的步骤来为应用程序配备 JDBC 持久化。我们将首先对域对象进行一些调整。

### 3.1.1 为持久化改造领域实体

在将对象持久化到数据库时，通常最好有一个惟一标识对象的字段。`Ingredient` 类已经有一个 `id` 字段，但是需要向 `Taco` 和 `TacoOrder` 添加 `id` 字段。

此外，了解何时创建 `Taco` 以及何时生成一个 `TacoOrder` 可能很有用。还需要向每个对象添加一个字段，以捕获保存对象的日期和时间。下面的程序清单显示了 `Taco` 类中需要的新 `id` 和 `createdAt` 字段。

程序清单 3.3 向 `Taco` 类添加 `id` 和 `timestamp` 字段

```
@Data
public class Taco {

    private Long id;

    private Date createdAt;

    ...
}
```

因为使用 `Lombok` 在运行时自动生成访问器方法，所以除了声明 `id` 和 `createdAt` 属性外，不需要做任何事情。它们将在运行时根据需要生成适当的 `getter` 和 `setter` 方法。`TacoOrder` 类也需要做类似的修改，如下所示：

```
@Data
public class TacoOrder implements Serializable {

    private static final long serialVersionUID = 1L;

    private Long id;

    private Date placedAt;
    // ...
}
```

同样，`Lombok` 会自动生成访问字段的方法，因此 `TacoOrder` 类只需要进行这些更改。（如果由于某种原因选择不使用 `Lombok`，那么需要自己编写这些方法。）

领域类现在已经为持久化做好了准备。让我们看看如何使用 `JdbcTemplate` 对它们进行数据库读写。

### 3.1.2 使用 JdbcTemplate

在开始使用 JdbcTemplate 之前，需要将它添加到项目类路径中。这很容易通过添加 Spring Boot 的 JDBC starter 依赖来实现：

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-jdbc</artifactId>
</dependency>
```

还需要一个存储数据的数据库。出于开发目的，嵌入式数据库也可以。我喜欢 H2 嵌入式数据库，所以我添加了以下依赖进行构建：

```
<dependency>
  <groupId>com.h2database</groupId>
  <artifactId>h2</artifactId>
  <scope>runtime</scope>
</dependency>
```

默认情况下，数据库名称是随机生成的。但这使得很难确定数据库连接 URL。因为需要使用 H2 控制台连接到数据库（使用 Spring Boot DevTools 访问 <http://localhost:8080/h2-console>，所以指定确定的数据库名称这是个好主意。这通过在 application.properties 中设置两个属性来实现：

```
spring.datasource.generate-unique-name=false
spring.datasource.name=tacocloud
```

或者，如果愿意，将 application.properties 重命名为 application.yml 并添加 YAML 格式的属性：

```
spring:
  datasource:
    generate-unique-name: false
    name: tacocloud
```

属性文件格式和 YAML 格式之间的选择取决于您。Spring Boot 都支持。鉴于 YAML 的结构增加了可读性，本书所有部分我们都将使用 YAML 格式来配置。

通过将 spring.datasource.generate-unique-name 属性设置为 false，我们可以看出 Spring 不再为数据库生成随机值。相反，它应该使用 spring.datasource.name 属性的值。在这种情况下，数据库名称将是“tacocloud”。因此，数据库 URL 将是 “jdbc:h2:mem:tacocloud”，您可以在 JDBC URL 中指定 H2 控制台连接。

稍后，将看到如何配置应用程序来使用外部数据库。但是现在，让我们继续编写一个获取和保存 Ingredient 数据的存储库。

## 定义 JDBC 存储库

Ingredient repository 需要执行以下操作：

- 查询所有的 Ingredient 使之变成一个 Ingredient 的集合对象
- 通过它的 id 查询单个 Ingredient
- 保存一个 Ingredient 对象

以下 IngredientRepository 接口将这三种操作定义为方法声明：

```

package tacos.data;

import java.util.Optional;

import tacos.Ingredient;

public interface IngredientRepository {

    Iterable<Ingredient> findAll();

    Optional<Ingredient> findById(String id);

    Ingredient save(Ingredient ingredient);

}

```

尽管该接口体现了需要 `Ingredient repository` 做的事情的本质，但是仍然需要编写一个使用 `JdbcTemplate` 来查询数据库的 `IngredientRepository` 的实现。下面的程序清单是编写实现的第一步。

#### 程序清单 3.4 使用 `JdbcTemplate` 开始编写 `Ingredient repository`

```

package tacos.data;

import java.sql.ResultSet;
import java.sql.SQLException;

import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.stereotype.Repository;

import tacos.Ingredient;

@Repository
public class JdbcIngredientRepository implements IngredientRepository {

    private JdbcTemplate jdbcTemplate;

    public JdbcIngredientRepository(JdbcTemplate jdbcTemplate) {
        this.jdbcTemplate = jdbcTemplate;
    }

    // ...

}

```

可以看到，`JdbcIngredientRepository` 使用了 `@Repository` 注解。这个注解是 `Spring` 定义的少数几个原型注解之一，包括 `@Controller` 和 `@Component`。通过使用 `@Repository` 对 `JdbcIngredientRepository` 进行注解，这样它就会由 `Spring` 组件在扫描时自动发现，并在 `Spring` 应用程序上下文中生成 `bean` 实例。

当 `Spring` 创建 `JdbcIngredientRepository` `bean` 时，通过 `@Autowired` 注解将 `JdbcTemplate` 注入到 `bean` 中。如果有多个构造函数，或者如果您只想显式地注入，那么可以使用 `@Autowired`：

```

@Autowired
public JdbcIngredientRepository(JdbcTemplate jdbcTemplate) {
    this.jdbcTemplate = jdbcTemplate;
}

```

构造函数将 `JdbcTemplate` 分配给一个实例变量，该变量将在其他方法中用于查询和插入数据库。谈到那些其他方法，让我们来看看 `findAll()` 和 `findById()` 的实现。

#### 程序清单 3.5 使用 `JdbcTemplate` 查询数据库

```

@Override
public Iterable<Ingredient> findAll() {
    return jdbcTemplate.query(
        "select id, name, type from Ingredient",
        this::mapRowToIngredient);
}

@Override
public Optional<Ingredient> findById(String id) {
    List<Ingredient> results = jdbcTemplate.query(
        "select id, name, type from Ingredient where id=?",
        this::mapRowToIngredient,
        id);
    return results.size() == 0 ?
        Optional.empty() :
        Optional.of(results.get(0));
}

private Ingredient mapRowToIngredient(ResultSet row, int rowNum)
    throws SQLException {
    return new Ingredient(
        row.getString("id"),
        row.getString("name"),
        Ingredient.Type.valueOf(row.getString("type")));
}

```

`findAll()` 和 `findById()` 都以类似的方式使用 `JdbcTemplate`。期望返回对象集合的 `findAll()` 方法使用了 `JdbcTemplate` 的 `query()` 方法。`query()` 方法接受查询的 SQL 以及 Spring 的 `RowMapper` 实现，以便将结果集中的每一行映射到一个对象。`findAll()` 还接受查询中所需的所有参数的列表作为它的最后一个参数。但是，在本例中，没有任何必需的参数。

相反，相反，`findById()` 方法需要在其查询中包含 `where` 子句，以比较 `id` 列的值与传递给方法的 `id` 参数的值。因此对 `query()` 的调用包括 `id` 参数。当查询执行时，“?”将被替换为此值。

如程序清单 3.5 所示，`findAll()` 和 `findById()` 的 `RowMapper` 参数作为 `mapRowToIngredient()` 方法的方法引用。当使用 `JdbcTemplate` 作为显式 `RowMapper` 实现的替代方案时，使用 Java 8 的方法引用和 `lambda` 非常方便。但是，如果出于某种原因，想要或是需要一个显式的 `RowMapper`，那么以下的 `findById()` 实现将展示如何做到这一点：

```

@Override
public Ingredient findById(String id) {
    return jdbcTemplate.queryForObject(
        "select id, name, type from Ingredient where id=?",
        new RowMapper<Ingredient>() {
            public Ingredient mapRow(ResultSet rs, int rowNum)
                throws SQLException {
                return new Ingredient(
                    rs.getString("id"),
                    rs.getString("name"),
                    Ingredient.Type.valueOf(rs.getString("type")));
            }
        }, id);
}

```

从数据库读取数据只是问题的一部分。在某些情况下，必须将数据写入数据库以便能够读取。因此，让我们来看看如何实现 `save()` 方法。

## 插入一行

`JdbcTemplate` 的 `update()` 方法可用于在数据库中写入或更新数据的任何查询。并且，如下面的程序清单所示，它可以用来将数据插入数据库。

#### 程序清单 3.6 使用 `JdbcTemplate` 插入数据

```
@Override
public Ingredient save(Ingredient ingredient) {
    jdbcTemplate.update(
        "insert into Ingredient (id, name, type) values (?, ?, ?)",
        ingredient.getId(),
        ingredient.getName(),
        ingredient.getType().toString());
    return ingredient;
}
```

因为没有必要将 `ResultSet` 数据映射到对象，所以 `update()` 方法要比 `query()` 简单得多。它只需要一个包含 SQL 的字符串来执行，以及为任何查询参数赋值。在本例中，查询有三个参数，它们对应于 `save()` 方法的最后三个参数，提供了 `Ingredient` 的 `id`、`name` 和 `type`。

完成了 `JdbcIngredientRepository` 后，现在可以将其注入到 `DesignTacoController` 中，并使用它来提供一个 `Ingredient` 对象列表，而不是使用硬编码的值（正如第 2 章中所做的那样）。`DesignTacoController` 的变化如下所示。

#### 程序清单 3.7 在控制器中注入并使用 `repository`

```
@Controller
@RequestMapping("/design")
@SessionAttributes("tacoOrder")
public class DesignTacoController {

    private final IngredientRepository ingredientRepo;

    @Autowired
    public DesignTacoController(
        IngredientRepository ingredientRepo) {
        this.ingredientRepo = ingredientRepo;
    }

    @ModelAttribute
    public void addIngredientsToModel(Model model) {
        Iterable<Ingredient> ingredients = ingredientRepo.findAll();
        Type[] types = Ingredient.Type.values();
        for (Type type : types) {
            model.addAttribute(type.toString().toLowerCase(),
                filterByType(ingredients, type));
        }
    }

    // ...
}
```

请注意，`addIngredientsToModel()` 方法的第 2 行现在调用了注入的 `IngredientRepository` 的 `findAll()` 方法。`findAll()` 方法从数据库中提取所有 `Ingredient`，然后将它们对应到模型的不同类型中。

现在我们有了一个 `IngredientRepository` 来从数据库中提取配料对象，我们还可以简化我们在第 2 章中创建的 `IngredientByIdConverter`，替换其中硬编码的配料 Map，通过简单调用 `IngredientRepository.findById()` 方法：

#### 程序清单 3.8 简化的 `IngredientByIdConverter`

```

package tacos.web;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.core.convert.converter.Converter;
import org.springframework.stereotype.Component;

import tacos.Ingredient;
import tacos.data.IngredientRepository;

@Component
public class IngredientByIdConverter implements Converter<String, Ingredient> {

    private IngredientRepository ingredientRepo;

    @Autowired
    public IngredientByIdConverter(IngredientRepository ingredientRepo) {
        this.ingredientRepo = ingredientRepo;
    }

    @Override
    public Ingredient convert(String id) {
        return ingredientRepo.findById(id).orElse(null);
    }

}

```

几乎已经准备好启动应用程序，并测试这些更改了。但是在开始从 **Ingredient** 表查询数据之前，应该先创建这个表，并写入一些 **Ingredient** 数据。

### 3.1.3 定义 schema 并预加载数据

除了 Ingredient 表之外，还需要一些保存订单和设计信息的表。图 3.1 说明了需要的表以及这些表之间的关系。

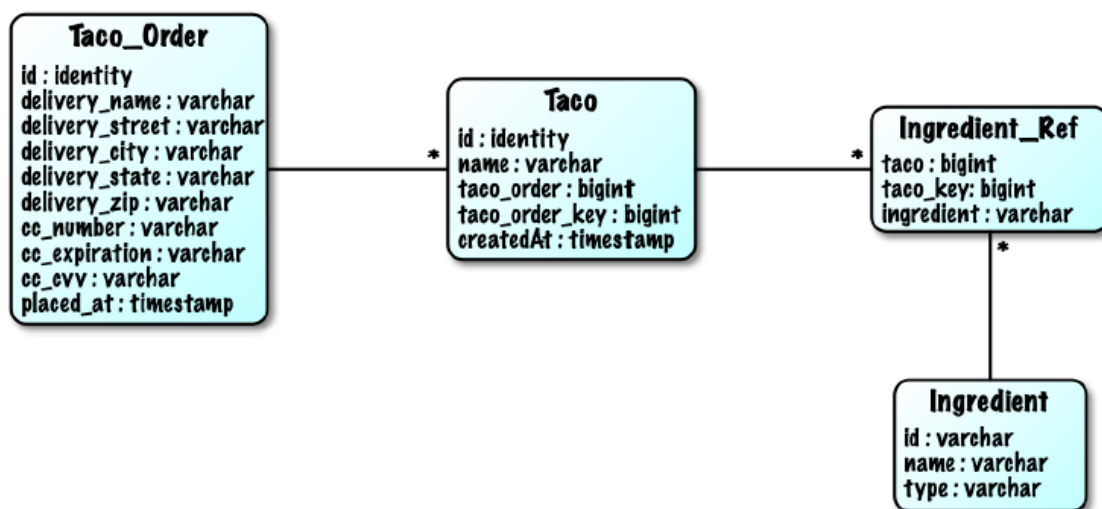


图 3.1 Taco Cloud Schema

图 3.1 中的表有以下用途：

- Taco\_Order - 保存着重要的订单细节
- Taco - 保存着关于 taco 设计的重要信息
- Ingredient\_Ref - 包含 Taco 表中每一行的一个或多行数据，将 Taco 映射到该 Taco 的 Ingredient
- Ingredient - 保存着原料信息

在我们的应用程序中，Taco 不能存在于 Taco\_Order 的上下文之外。因此，Taco\_Order 和 Taco 聚合关系，其中 Taco\_Order 是聚合根。另一方面，Ingredient 对象是其自身聚合的唯一成员，并且 Taco 通过 Ingredient\_Ref 引用。

注意：聚合和聚合根是领域驱动设计的核心概念，是一种设计方法，它提倡软件代码应与业务域匹配。虽然在 Taco Cloud 领域对象中，我们只简单应用领域驱动设计（DDD），但 DDD 不只是聚合和聚合根。更多信息请阅读 Eric Evans 的开创性作品《领域驱动设计：软件核心复杂性应对之道》。

下一个程序清单显示了创建表的 SQL 语句。

程序清单 3.9 定义 Taco Cloud 模式



```

create table if not exists Taco_Order (
    id identity,
    delivery_Name varchar(50) not null,
    delivery_Street varchar(50) not null,
    delivery_City varchar(50) not null,
    delivery_State varchar(2) not null,
    delivery_Zip varchar(10) not null,
    cc_number varchar(16) not null,
    cc_expiration varchar(5) not null,
    cc_cvv varchar(3) not null,
    placed_at timestamp not null
);

create table if not exists Taco (
    id identity,
    name varchar(50) not null,
    taco_order bigint not null,
    taco_order_key bigint not null,
    created_at timestamp not null
);

create table if not exists Ingredient_Ref (
    ingredient varchar(4) not null,
    taco bigint not null,
    taco_key bigint not null
);

create table if not exists Ingredient (
    id varchar(4) not null,
    name varchar(25) not null,
    type varchar(10) not null
);

alter table Taco
    add foreign key (taco_order) references Taco_Order(id);
alter table Ingredient_Ref
    add foreign key (ingredient) references Ingredient(id);

```

最大的问题是把这个 **Schema** 定义放在哪里。事实上，**Spring Boot** 已经有答案。

如果有一个名为 **schema.sql** 的文件。在应用程序的类路径根目录下执行 **sql**，然后在应用程序启动时对数据库执行该文件中的 **SQL**。因此，应该将程序清单 3.8 的内容写入一个名为 **schema.sql** 的文件中，然后放在项目的 **src/main/resources** 文件夹下。

还需要用一些 **Ingredient** 数据来预加载数据库。幸运的是，**Spring Boot** 还将执行一个名为 **data.sql** 的文件，这个文件位于根路径下。因此，可以使用 **src/main/resources/data.sql** 中的下面程序清单中的 **insert** 语句来加载包含 **Ingredient** 数据的数据库。

**程序清单 3.10** 预加载数据库

```

delete from Ingredient_Ref;
delete from Taco;
delete from Taco_Order;

delete from Ingredient;
insert into Ingredient (id, name, type)
    values ('FLTO', 'Flour Tortilla', 'WRAP');
insert into Ingredient (id, name, type)
    values ('COTO', 'Corn Tortilla', 'WRAP');
insert into Ingredient (id, name, type)
    values ('GRBF', 'Ground Beef', 'PROTEIN');
insert into Ingredient (id, name, type)
    values ('CARN', 'Carnitas', 'PROTEIN');
insert into Ingredient (id, name, type)
    values ('TMTO', 'Diced Tomatoes', 'VEGGIES');
insert into Ingredient (id, name, type)
    values ('LETC', 'Lettuce', 'VEGGIES');
insert into Ingredient (id, name, type)
    values ('CHED', 'Cheddar', 'CHEESE');
insert into Ingredient (id, name, type)
    values ('JACK', 'Monterrey Jack', 'CHEESE');
insert into Ingredient (id, name, type)
    values ('SLSA', 'Salsa', 'SAUCE');
insert into Ingredient (id, name, type)
    values ('SRCR', 'Sour Cream', 'SAUCE');

```

即使只开发了配料数据的 **repository**，也可以启动 **Taco Cloud** 应用程序并访问设计页面，查看 **JdbcIngredientRepository** 的运行情况。大胆的试试吧。后续可以继续编写用于持久化 **Taco**、**TacoOrder** 的 **repository**。

### 3.1.4 插入数据

到此，已经了解了如何使用 `JdbcTemplate` 向数据库写入数据。`JdbcIngredientRepository` 中的 `save()` 方法使用 `JdbcTemplate` 的 `update()` 方法将 `Ingredient` 对象保存到数据库中。

虽然这是第一个很好的例子，但是它可能有点太简单了。保存数据一般都会比 `JdbcIngredientRepository` 所做的更复杂。

在我们的设计中，`TacoOrder` 和 `Taco` 是聚合关系，其中 `TacoOrder` 是聚合根。换句话说，`Taco` 对象不存在于 `TacoOrder` 的上下文之外。所以，现在，我们只需要定义一个 `Repository` 来持久化 `TacoOrder` 对象，进而持久化 `Taco` 对象。这样的 `Repository` 在 `OrderRepository` 接口中定义，如下所示：

```
package tacos.data;

import java.util.Optional;

import tacos.TacoOrder;

public interface OrderRepository {

    TacoOrder save(TacoOrder order);

}
```

看起来很简单，对吧？没那么容易的。当保存 `TacoOrder` 时，还必须保存和它一起的那些 `Taco`。在保存 `Taco` 对象时，还需要保存一个对象，表示 `Taco` 和构成玉米卷的每个 `Ingredient` 之间的关联。`IngredientRef` 类定义了 `Taco` 和 `Ingredient` 之间的关系：

```
package tacos;

import lombok.Data;

@Data
public class IngredientRef {

    private final String ingredient;

}
```

这个 `save()` 方法，比之前创建的用于保存 `Ingredient` 对象的方法要更有趣。

`save()` 方法需要做的另一件事是确定保存后分配给的 ID。根据建表的 `Schema`，`Taco_Order` 表上的 `id` 属性是 `identity`，这意味着数据库将自动确定其值。但是如果数据库为您确定的值，您需要知道该值是什么，以便在 `save()` 方法返回的 `TacoOrder` 对象中返回。幸运的是，`Spring` 提供了一个 `GeneratedKeyHolder` 类可以帮助您实现这一点。但它涉及到使用 `Prepared Statement`，如下面的 `save()` 方法所示：

```

package tacos.data;

import java.sql.Types;
import java.util.Arrays;
import java.util.Date;
import java.util.List;
import java.util.Optional;

import org.springframework.asm.Type;
import org.springframework.jdbc.core.JdbcOperations;
import org.springframework.jdbc.core.PreparedStatementCreator;
import org.springframework.jdbc.core.PreparedStatementCreatorFactory;
import org.springframework.jdbc.support.GeneratedKeyHolder;
import org.springframework.stereotype.Repository;
import org.springframework.transaction.annotation.Transactional;

import tacos.IngredientRef;
import tacos.Taco;
import tacos.TacoOrder;

@Repository
public class JdbcOrderRepository implements OrderRepository {

    private JdbcOperations jdbcOperations;

    public JdbcOrderRepository(JdbcOperations jdbcOperations) {
        this.jdbcOperations = jdbcOperations;
    }

    @Override
    @Transactional
    public TacoOrder save(TacoOrder order) {
        PreparedStatementCreatorFactory pscf =
            new PreparedStatementCreatorFactory(
                "insert into Taco_Order "
                + "(delivery_name, delivery_street, delivery_city, "
                + "delivery_state, delivery_zip, cc_number, "
                + "cc_expiration, cc_cvv, placed_at) "
                + "values (?, ?, ?, ?, ?, ?, ?, ?, ?)",
                Types.VARCHAR, Types.VARCHAR, Types.VARCHAR,
                Types.VARCHAR, Types.VARCHAR, Types.VARCHAR,
                Types.VARCHAR, Types.VARCHAR, Types.TIMESTAMP
            );
        pscf.setReturnGeneratedKeys(true);

        order.setPlacedAt(new Date());
        PreparedStatementCreator psc =
            pscf.newPreparedStatementCreator(
                Arrays.asList(
                    order.getDeliveryName(),
                    order.getDeliveryStreet(),
                    order.getDeliveryCity(),
                    order.getDeliveryState(),
                    order.getDeliveryZip(),
                    order.getCcNumber(),
                    order.getCcExpiration(),
                    order.getCcCVV(),
                    order.getPlacedAt()
                ));

        GeneratedKeyHolder keyHolder = new GeneratedKeyHolder();
        jdbcOperations.update(psc, keyHolder);
        long orderId = keyHolder.getKey().longValue();
        order.setId(orderId);

        List<Taco> tacos = order.getTacos();
    }
}

```

```

    int i=0;
    for (Taco taco : tacos) {
        saveTaco(orderId, i++, taco);
    }
    return order;
}
}

```

在这个方法中似乎有很多东西。但是如果您把 `save()` 分解，就会有发现只有少数几个重要步骤。首先，创建 `PreparedStatementCreatorFactory`。它描述了插入查询以及输入字段的类型。因为您会稍后需要获取保存后的订单 ID，您还需要调用 `setReturnGeneratedKeys(true)`。

定义 `PreparedStatementCreatorFactory` 后，可以使用它创建 `PreparedStatementCreator`，从将要创建的 `TacoOrder` 对象传入值进行持久化。提供给 `PreparedStatementCreator` 的最后一个字段是订单发出的日期。您还需要对 `TacoOrder` 对象本身进行设置，以便 `TacoOrder` 将提供这些信息。

现在您有一个 `PreparedStatementCreator`，可以通过调用 `JdbcTemplate` 上的 `update()` 方法实际保存数据了。传入 `PreparedStatementCreator` 和 `GeneratedKeyHolder`。订单数据保存后，`GeneratedKeyHolder` 将包含由数据库生成的 id，并应复制到 `TacoOrder` 对象的 id 属性中。

此时，订单已保存，但还需要保存关联的 `Taco` 对象。您可以为订单中的每个玉米卷调用 `saveTaco()`。

`saveTaco()` 方法与 `save()` 方法非常相似，如下所示：

```

private long saveTaco(Long orderId, int orderKey, Taco taco) {
    taco.setCreatedAt(new Date());
    PreparedStatementCreatorFactory pscf =
        new PreparedStatementCreatorFactory(
            "insert into Taco "
            + "(name, created_at, taco_order, taco_order_key) "
            + "values (?, ?, ?, ?)",
            Types.VARCHAR, Types.TIMESTAMP, Type.LONG, Type.LONG
        );
    pscf.setReturnGeneratedKeys(true);

    PreparedStatementCreator psc =
        pscf.newPreparedStatementCreator(
            Arrays.asList(
                taco.getName(),
                taco.getCreatedAt(),
                orderId,
                orderKey));

    GeneratedKeyHolder keyHolder = new GeneratedKeyHolder();
    jdbcOperations.update(psc, keyHolder);
    long tacoId = keyHolder.getKey().longValue();
    taco.setId(tacoId);

    saveIngredientRefs(tacoId, taco.getIngredients());

    return tacoId;
}

```

一步一步地，`saveTaco()` 的结构与 `save()` 类似，只不过是保存 `Taco` 数据而不是 `TacoOrder` 数据。最后，它调用 `saveIngredientRefs()` 在 `Ingredient_Ref` 表插入一行，用于将 `Taco` 行链接到 `Ingredient` 行。这个 `saveIngredientRefs()` 方法如下所示：

```

private void saveIngredientRefs(
    long tacoId, List<IngredientRef> ingredientRefs) {
    int key = 0;
    for (IngredientRef ingredientRef : ingredientRefs) {
        jdbcOperations.update(
            "insert into Ingredient_Ref (ingredient, taco, taco_key) "
            + "values (?, ?, ?)",
            ingredientRef.getIngredient(), tacoId, key++);
    }
}

```

谢天谢地，`saveIngredientRefs()` 方法要简单得多。它循环列表中的 `Ingredient` 对象，将每个对象保存到 `Ingredient_Ref` 表中。它还有一个局部变量，被用作一个索引，以确保配料的顺序保持不变。

`OrderRepository` 剩下要做的就是，将其注入 `OrderController`，并在需要时使用它保存订单。下面的列表显示注入了 `repository` 所需的更改。

### 程序清单 3.11 注入并使用 `OrderRepository`

```

package tacos.web;
import javax.validation.Valid;

import org.springframework.stereotype.Controller;
import org.springframework.validation.Errors;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.SessionAttributes;
import org.springframework.web.bind.support.SessionStatus;

import tacos.TacoOrder;
import tacos.data.OrderRepository;

@Controller
@RequestMapping("/orders")
@SessionAttributes("tacoOrder")
public class OrderController {

    private OrderRepository orderRepo;

    public OrderController(OrderRepository orderRepo) {
        this.orderRepo = orderRepo;
    }

    // ...
    @PostMapping
    public String processOrder(@Valid TacoOrder order, Errors errors, SessionStatus sessionStatus) {
        if (errors.hasErrors()) {
            return "orderForm";
        }

        orderRepo.save(order);
        sessionStatus.setComplete();

        return "redirect:/";
    }
}

```

如您所见，构造函数将 `OrderRepository` 作为参数，将其分配给实例变量，并在 `processOrder()` 方法中使用。说到 `processOrder()` 方法，它已更改调用 `OrderRepository` 的 `save()` 方法，而不是记录 `TacoOrder` 对象。

Spring 的 `JdbcTemplate` 使操作关系数据库比使用纯 JDBC 要简单的多。但即使使用 `JdbcTemplate`，一些持久化任务仍然具有挑战性。尤其是在聚合中持久化嵌套对象时。要是有什么办法解决这个问题就好了。

让我们看一看 `Spring Data JDBC`，它使得使用 JDBC 变得非常简单——即使是持久化聚合对象时。

## 3.2 使用 Spring Data JDBC

Spring Data 项目是一个相当大的一揽子项目，由几个子项目组成，其中大多数是聚焦在对各种不同数据库类型进行持久化。一些最流行的 Spring Data 项目包括：

- Spring Data JDBC —— 针对关系数据库的 JDBC 持久化
- Spring Data JPA —— 针对关系数据库的 JPA 持久化
- Spring Data MongoDB —— Mongo 文档数据库的持久化
- Spring Data Neo4j —— 持久化到 Neo4j 图形数据库
- Spring Data Redis —— 持久化到 Redis 键值存储数据库
- Spring Data Cassandra —— 持久化到 Cassandra 列存储数据库

Spring Data 为所有这些项目提供的最有趣、最有用的特性之一是，基于 Repository 规范接口自动创建 Repository 的能力。因此，Spring Data 项目的持久化几乎没有持久化逻辑，只涉及编写一个或多个 Repository 接口。

让我们看看如何将 Spring Data JDBC 应用到我们的项目中，以简化 JDBC 的数据持久化。首先，需要将 Spring Data JDBC 添加到项目构建中。



### 3.2.1 添加 Spring Data JDBC 到项目构建中

Spring Data JDBC 有 Spring Boot 的 starter 依赖项。可添加到项目的 pom.xml 文件中，如下所示：

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-jdbc</artifactId>
</dependency>
```

您将不再需要提供 JdbcTemplate 的 JDBC starter，因此您可以删除如下依赖：

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-jdbc</artifactId>
</dependency>
```

但是，您仍然需要一个数据库，所以不要删除 H2 依赖项。

### 3.2.2 定义 Repository 接口

幸运的是，我们已经创建了 `IngredientRepository` 和 `OrderRepository`，定义存储库的工作已经完成。但我们需要对它们做一点微小的改变，以使它们与 `Spring Data JDBC` 一起工作。

`Spring Data` 将在运行时，自动为我们的存储库接口生成实现类。但它只会对扩展自 `Spring Data` 几个 `Repository` 接口的类执行此操作。所以，我们的存储库接口需要扩展自 `Repository`，以便 `Spring Data` 知道要自动创建实现类。例如，以下是您可能编写的 `IngredientRepository`，扩展自 `Repository`：

```
package tacos.data;
import java.util.Optional;
import org.springframework.data.repository.Repository;
import tacos.Ingredient;

public interface IngredientRepository
    extends Repository<Ingredient, String> {

    Iterable<Ingredient> findAll();

    Optional<Ingredient> findById(String id);

    Ingredient save(Ingredient ingredient);

}
```

如您所见，存储库接口是参数化的。第一个参数是要由该存储库持久化的对象——在本例中为 `Ingredient`。第二个参数是持久化对象的 ID 字段的类型。对于 `Ingredient`，是 `String`。

通过扩展 `Repository`，如上所示的 `IngredientRepository` 能够正常工作。`Spring Data` 还提供 `CrudRepository` 作为常见操作的基本接口，包括三个我们在 `IngredientRepository` 中定义的方法。因此，我们可以不扩展 `Repository`，而通常更方便的扩展 `CrudRepository`，如下所示：

```
package tacos.data;

import org.springframework.data.repository.CrudRepository;

import tacos.Ingredient;

public interface IngredientRepository
    extends CrudRepository<Ingredient, String> {

}
```

同样，我们的 `OrderRepository` 可以像这样扩展 `CrudRepository`：

```
package tacos.data;

import org.springframework.data.repository.CrudRepository;

import tacos.TacoOrder;

public interface OrderRepository
    extends CrudRepository<TacoOrder, Long> {

}
```

在这两种情况下，因为 `CrudRepository` 已经定义了所需的方法，所以没有必要在 `IngredientRepository` 和 `OrderRepository` 接口中显式定义它们。

现在您有了两个存储库。您可能认为您需要写这两个存储库的实现，包括 **Crudepository** 中定义的十几个方法。但 **Spring Data** 给您带来了好消息，根本不需要编写实现类！不管什么时候，只要应用程序启动了，**Spring Data** 将自动生成一个实现类。这意味着存储库从一开始就已经可以使用了。只需将它们注入控制器，您的工作就完成了。

此外，因为 **Spring Data** 在运行时将自动创建这些接口的实现，您不再需要显式实现 **JdbcIngredientRepository** 和 **JdbcOrderRepository**。您可以删除这两个类，根本不必有任何担心！

### 3.2.3 为持久化注解领域实体

我们唯一需要做的另一件事是为领域实体添加注解，这样 **Spring Data JDBC** 就可以知道如何进行持久化了。一般来说，这意味着使用 `@Id` 指定标识属性 —— 这样 **Spring Data** 将知道哪个字段表示对象的标识 —— 并且可以选择在类上使用 `@Table` 注解。例如，`TacoOrder` 类用了 `@Table` 和 `@Id` 注解，如下所示：

清单 3.15 为持久化准备 **Taco** 类。

```
package tacos;
import java.io.Serializable;
import java.util.ArrayList;
import java.util.Date;
import java.util.List;

import javax.validation.constraints.Digits;
import javax.validation.constraints.NotBlank;
import javax.validation.constraints.Pattern;

import org.hibernate.validator.constraints.CreditCardNumber;
import org.springframework.data.annotation.Id;
import org.springframework.data.relational.core.mapping.Table;

import lombok.Data;

@Data
@Table
public class TacoOrder implements Serializable {

    private static final long serialVersionUID = 1L;

    @Id
    private Long id;

    // ...

}
```

注解 `@Table` 是可选的。默认情况下，对象将映射到一个数据库表，表名是对象的类名称。在本例中，`TacoOrder` 映射到一个名为“`TacoOrder`”的表上。如果这对您来说是合适的，您就完全不用添加 `@Table` 注解，或者即使使用也不添加任何参数。但如果希望将对象映射到其他的表名上，则可以使用 `@Table` 参数来指定表名称，如下所示：

```
@Table("Taco_Cloud_Order")
public class TacoOrder {
    ...
}
```

如图所示，`TacoOrder` 将映射到一个名为“`Taco_Cloud_Order`”的表上。至于 `@Id` 注解，它将 `Id` 属性指定为 `TacoOrder` 的标识。`TacoOrder` 中的所有其他属性，将根据其属性名称自动映射到列上。例如，`deliveryName` 属性将自动映射到名为“`delivery_name`”的列上。如果要显式定义列名，您可以使用 `@Column` 注解的参数进行指定：

```
@Column("customer_name")
@NotBlank(message="Delivery name is required")
private String deliveryName;
```

您还需要将 `@Table` 和 `@Id` 注解应用于其他领域实体类。如 `Ingredient` 类:

清单 3.16 为持久化准备 `Ingredient` 类。

```
package tacos;

import org.springframework.data.annotation.Id;
import org.springframework.data.domain.Persistable;
import org.springframework.data.relational.core.mapping.Table;

import lombok.AccessLevel;
import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;

@Data
@Table
@AllArgsConstructor
@NoArgsConstructor(access=AccessLevel.PRIVATE, force=true)
public class Ingredient implements Persistable<String> {

    @Id
    private String id;

    // ...

}
```

还有 `Taco` 类:

清单 3.17 为持久化准备 `Taco` 类。

```
package tacos;

import java.util.ArrayList;
import java.util.Date;
import java.util.List;

import javax.validation.constraints.NotNull;
import javax.validation.constraints.Size;

import org.springframework.data.annotation.Id;
import org.springframework.data.relational.core.mapping.Table;

import lombok.Data;

@Data
@Table
public class Taco {

    @Id
    private Long id;

    // ...

}
```

至于 `IngredientRef`，它将自动映射到名为“`Ingredient_Ref`”的表上，这非常适合我们的应用。如果需要，可以使用 `@Table` 注解对其进行调整，但这完全不必要。“`Ingredient_Ref`”表没有标识列，因此无需在 `IngredientRef` 中使用 `@Id` 注解。

有了这些小小的改变，更不用说完全移除的 `JdbcIngredientRepository` 和 `JdbcOrderRepository` 类，您现在进行持久化的代码要少很多了。即使如此，它仍然完成了使用 `JdbcTemplate` 的存储库所做的一切。事实上，还可以做更多的事情，因为这两个存储库接口扩展自 `CrudRepository`，里边提供了十几种用于创建、读取、更新和删除对象的方法。

## 3.2.4 使用 CommandLineRunner 预加载数据

使用 `JdbcTemplate` 时，我们在应用程序启动时使用 `data.sql` 预加载了 `Ingredient` 数据，在创建数据源 `bean` 时对数据库执行插入操作。同样的方法也适用于 `Spring Data JDBC`。事实上，它对任何关系型数据库的持久化都有效。但是让我们看看另一种在启动时填充数据库的更灵活的方法。

`Spring Boot` 为应用程序启动时执行一些逻辑，提供了两个有用的接口：`CommandLineRunner` 和 `ApplicationRunner`。这两个函数式接口非常相似。都需要实现 `run()` 方法。当应用程序启动时，应用程序上下文中实现了 `CommandLineRunner` 或 `ApplicationRunner` 接口的任何 `bean`，将在应用程序上下文和所有 `bean` 生成之后，在其他任何事情发生之前，调用它们的 `run()` 方法。这为我们提供了一个方便的地方，以加载预置数据到数据库中。

因为 `CommandLineRunner` 和 `ApplicationRunner` 都是函数式接口，可以很方便的使用 `@Bean` 注解，在配置类中声明为 `Bean`，且返回一个 `lambda` 函数。例如，数据加载 `CommandLineRunner` `bean` 的例子：

```
@Bean
public CommandLineRunner dataLoader(IngredientRepository repo) {
    return args -> {
        repo.save(new Ingredient("FLT0", "Flour Tortilla", Type.WRAP));
        repo.save(new Ingredient("COTO", "Corn Tortilla", Type.WRAP));
        repo.save(new Ingredient("GRBF", "Ground Beef", Type.PROTEIN));
        repo.save(new Ingredient("CARN", "Carnitas", Type.PROTEIN));
        repo.save(new Ingredient("TMT0", "Diced Tomatoes", Type.VEGGIES));
        repo.save(new Ingredient("LETC", "Lettuce", Type.VEGGIES));
        repo.save(new Ingredient("CHED", "Cheddar", Type.CHEESE));
        repo.save(new Ingredient("JACK", "Monterrey Jack", Type.CHEESE));
        repo.save(new Ingredient("SLSA", "Salsa", Type.SAUCE));
        repo.save(new Ingredient("SRCR", "Sour Cream", Type.SAUCE));
    };
}
```

这里，`IngredientRepository` 被注入 `bean` 方法中，并在 `lambda` 中创建 `Ingredient` 对象。`CommandLineRunner` 的 `run()` 方法接受单个参数，该参数是字符串变量，其中包含所有运行应用的命令行参数。我们不需要这些配料对象加载到数据库中，因此 `args` 参数可以忽略。

或者，我们可以将数据加载器 `bean` 定义为 `ApplicationRunner` 的 `lambda` 实现，如下所示：

```
@Bean
public ApplicationRunner dataLoader(IngredientRepository repo) {
    return args -> {
        repo.save(new Ingredient("FLT0", "Flour Tortilla", Type.WRAP));
        repo.save(new Ingredient("COTO", "Corn Tortilla", Type.WRAP));
        repo.save(new Ingredient("GRBF", "Ground Beef", Type.PROTEIN));
        repo.save(new Ingredient("CARN", "Carnitas", Type.PROTEIN));
        repo.save(new Ingredient("TMT0", "Diced Tomatoes", Type.VEGGIES));
        repo.save(new Ingredient("LETC", "Lettuce", Type.VEGGIES));
        repo.save(new Ingredient("CHED", "Cheddar", Type.CHEESE));
        repo.save(new Ingredient("JACK", "Monterrey Jack", Type.CHEESE));
        repo.save(new Ingredient("SLSA", "Salsa", Type.SAUCE));
        repo.save(new Ingredient("SRCR", "Sour Cream", Type.SAUCE));
    };
}
```

`CommandLineRunner` 和 `ApplicationRunner` 之间的关键区别在于，传递给 `run()` 方法的参数。`CommandLineRunner` 接受字符串变量，该字符串命令上传递的参数的原始表示形式。但是 `ApplicationRunner` 接受 `ApplicationArguments` 参数，作为命令行解析组件，该参数提供了访问参数的一些方法。

例如，假设我们希望应用程序接受命令行中所带的参数，比如“`--version 1.2.3`”，需要在我们的加载器 `bean` 中使用这个参数。如果使用 `CommandLineRunner`，我们需要在数组中搜索“`-version`”，然后提取紧接其后的数组中的值。但是使用 `ApplicationRunner`，我们可以查询定的 `ApplicationArguments`，直接使用“`--version`”参数。如下所示：

```
public ApplicationRunner dataLoader(IngredientRepository repo) {  
    return args -> {  
        List<String> version = args.getOptionValues("version");  
        ...  
    };  
}
```

`getOptionValues()` 方法返回一个 `List<String>`，以允许选项参数可以多次指定。

但是，无论是 `CommandLineRunner` 还是 `ApplicationRunner`，我们都不需要命令行参数加载数据。因此，`args` 参数在我们的数据加载器 `bean` 中被忽略。

使用 `CommandLineRunner` 或 `ApplicationRunner` 进行初始数据加载的好处，是他们使用存储库来创建对象，而不是使用 `SQL` 脚本。这意味着它们对关系数据库或非关系数据库同样有效。在下一章中，当我们看到如何使用 `Spring Data` 持久化到非关系数据库，这就会显得非常方便。

但在此之前，让我们先看一看另一个 `Spring Data` 持久化关系数据库数据的项目：`Spring Data JPA`。



### 3.3 使用 Spring Data JPA 持久化数据

虽然 Spring Data JDBC 可以轻松地持久化数据，但 Java Persistence API（JPA）也是在关系数据库中处理数据的另一个常用选项。Spring Data JPA 提供了一个类似 JDBC 的 JPA 方式来持久化数据。

为了了解 Spring Data 是如何工作的，需要将本章前面介绍的基于 jdbc 的存储库替换为 Spring Data JPA 创建的存储库。但是首先，需要将 Spring Data JPA 添加到项目构建中。

### 3.3.1 添加 Spring Data JPA 到工程中

添加了 JPA starter 的 Spring Boot 应用程序就可以使用 Spring Data JPA。这个 starter 依赖不仅带来了 Spring Data JPA，还隐含添加了 Hibernate 作为 JPA 的实现：

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
```

如果想使用不同的 JPA 实现，那么至少需要排除 Hibernate 依赖，并包含所选择的 JPA 库。例如，要使用 EclipseLink 而不是 Hibernate，需要按如下方式更改构建：

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-jpa</artifactId>
  <exclusions>
    <exclusion>
      <groupId>org.hibernate</groupId>
      <artifactId>hibernate-core</artifactId>
    </exclusion>
  </exclusions>
</dependency>
<dependency>
  <groupId>org.eclipse.persistence</groupId>
  <artifactId>org.eclipse.persistence.jpa</artifactId>
  <version>2.7.6</version>
</dependency>
```

请注意，根据对 JPA 实现的选择，可能需要进行其他更改。详细信息请参阅选择的 JPA 实现的文档。现在，让我们重新查看领域对象并对它们进行注解，以实现 JPA 持久化。

### 3.3.2 注解领域实体

正如您已经在 Spring Data JDBC 中看到的，SpringData 在运行时做了一些惊人的事情，自动创建存储库实现。但不幸的是，这对领域对象添加 JPA 注解并没有多大帮助。您需要打开 Ingredient、Taco 和 TacoOrder 类，并添加一些注解。首先是 Ingredient 类：

程序清单 3.18 为 JPA 持久化注解 Ingredient 类

```
package tacos;

import javax.persistence.Entity;
import javax.persistence.Id;

import lombok.AccessLevel;
import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;

@Data
@Entity
@AllArgsConstructor
@NoArgsConstructor(access=AccessLevel.PRIVATE, force=true)
public class Ingredient {

    @Id
    private String id;
    private String name;
    private Type type;

    public static enum Type {
        WRAP, PROTEIN, VEGGIES, CHEESE, SAUCE
    }
}
```

为了将其声明为 JPA 实体，Ingredient 类必须使用 @Entity 注解。它的 id 属性必须使用 @Id 进行注解，以便将其指定为唯一标识数据库中实体的属性。注意，这是来自 javax.persistence 包的 @Id 注解，而不是 org.springframework.data.annotation 包中的 @Id 注解。

还要注意，我们不再需要 @Table 注解，也不需要实现 Persistable。虽然我们仍然可以在这里使用 @Table 注解，但在使用 JPA 时是不需要这么做的，表名默认设置为类的名称（本例中为“Ingredient”）。至于 Persistable，它只需要在使用 Spring Data JDBC 时，来确定是否要创建新实体还是更新现有实体。JPA 会自动测别出来。

除了特定于 JPA 的注解之外，还在类级别上添加了 @NoArgsConstructor 注解。JPA 要求实体有一个无参构造函数，所以 Lombok 的 @NoArgsConstructor 实现了这一点。但是要是不想使用它，可以通过将 access 属性设置为 AccessLevel.PRIVATE 来将其设置为私有。因为必须设置 final 属性，所以还要将 force 属性设置为 true，这将导致 Lombok 生成的构造函数，依据属性类型的不同会将它们设置为 null、0 或者 false。

您还将添加一个 @AllArgsConstructor，以便于使用所有属性初始化值创建 Ingredient 对象。

还添加了一个 @RequiredArgsConstructor。@Data 隐式地添加了一个必需的有参构造函数，但是当使用 @NoArgsConstructor 时，该构造函数将被删除。显式的 @RequiredArgsConstructor 确保除了私有无参数构造函数外，仍然有一个必需有参构造函数。

现在让我们转到 Taco 类，看看如何将其注解为 JPA 实体。

程序清单 3.19 把 Taco 注解为实体

```

package tacos;
import java.util.ArrayList;
import java.util.Date;
import java.util.List;

import javax.persistence.Entity;
import javax.persistence.GeneratedValue;
import javax.persistence.GenerationType;
import javax.persistence.Id;
import javax.persistence.ManyToMany;
import javax.validation.constraints.NotNull;
import javax.validation.constraints.Size;

import lombok.Data;

@Data
@Entity
public class Taco {

    @Id
    @GeneratedValue(strategy = GenerationType.AUTO)
    private Long id;

    @NotNull
    @Size(min=5, message="Name must be at least 5 characters long")
    private String name;

    private Date createdAt = new Date();

    @Size(min=1, message="You must choose at least 1 ingredient")
    @ManyToMany()
    private List<Ingredient> ingredients = new ArrayList<>();

    public void addIngredient(Ingredient ingredient) {
        this.ingredients.add(ingredient);
    }
}

```

与 `Ingredient` 一样，`Taco` 类现在使用 `@Entity` 注解，其 `id` 属性使用 `@Id` 注解。因为依赖于数据库自动生成 `id` 值，所以还使用 `@GeneratedValue` 注解 `id` 属性，指定自动生成策略。

要声明 `Taco` 及其相关 `Ingredient` 列表之间的关系，可以使用 `@ManyToMany` 注解 `ingredient` 属性。一个 `Taco` 可以有多个 `Ingredient`，一个 `Ingredient` 可以是很多 `Taco` 的一部分。

最后，让我们将 `Order` 对象注解为一个实体。下一个程序清单展示了新的 `Order` 类。

**程序清单 3.20 把 `Order` 注解为 JPA 实体**

```

package tacos;
import java.io.Serializable;
import java.util.ArrayList;
import java.util.Date;
import java.util.List;

import javax.persistence.CascadeType;
import javax.persistence.Entity;
import javax.persistence.GeneratedValue;
import javax.persistence.GenerationType;
import javax.persistence.Id;
import javax.persistence.OneToMany;
import javax.validation.constraints.Digits;
import javax.validation.constraints.NotBlank;
import javax.validation.constraints.Pattern;

import org.hibernate.validator.constraints.CreditCardNumber;

import lombok.Data;

@Data
@Entity
public class TacoOrder implements Serializable {

    private static final long serialVersionUID = 1L;

    @Id
    @GeneratedValue(strategy = GenerationType.AUTO)
    private Long id;

    private Date placedAt = new Date();

    ...

    @OneToMany(cascade = CascadeType.ALL)
    private List<Taco> tacos = new ArrayList<>();

    public void addTaco(Taco taco) {
        this.tacos.add(taco);
    }
}

```

如您所见，对 **Order** 的更改与对 **Taco** 的更改非常相似。值得注意的是，与 **Taco** 对象列表的关系用的是 `@OneToMany` 注解，表示玉米卷都是针对这一订单的。此外，级联参数设置为 `CascadeType.ALL`，这样，如果订单被删除，其相关的玉米卷也将被删除。

### 3.3.3 声明 JPA Repository

当您创建 `JdbcTemplate` 版本的存储库时，您明确声明了希望 `repository` 提供的方法。当使用 Spring Data JDBC 时，不用再明确实现类，而是扩展 `CrudRepository` 接口。事实上，`CrudRepository` 在 Spring Data JPA 下也可以很好的工作。例如，下面是一个新的 `IngredientRepository` 接口：

```
package tacos.data;

import org.springframework.data.repository.CrudRepository;

import tacos.Ingredient;

public interface IngredientRepository
    extends CrudRepository<Ingredient, String> {
}
```

事实上，我们将用于 Spring Data JPA 的 `IngredientRepository` 接口，与我们定义的用于 Spring Data JDBC 的那个完全一样。`CrudRepository` 接口通常可以在 Spring Data 的许多项目中使用，而不用考虑底层的持久化机制。类似地，您可以为 Spring Data JPA 定义 `OrderRepository`，这与为 Spring Data JDBC 定义的那个是一样的：

```
package tacos.data;

import org.springframework.data.repository.CrudRepository;

import tacos.TacoOrder;

public interface OrderRepository
    extends CrudRepository<TacoOrder, Long> {
}
```

`CrudRepository` 提供的方法非常适合实体的能用持久化需求。但是如果有一些特殊的需求呢？让我们看看如何自定义 `repository` 来执行特有的查询。

### 3.3.4 自定义 Repository

想象一下，除了 `CrudRepository` 提供的基本 CRUD 操作之外，还需要获取投递给指定邮政编码的所有订单。事实证明，通过在 `OrderRepository` 中添加以下方法声明可以很容易地解决这个问题：

```
List<TacoOrder> findByDeliveryZip(String deliveryZip);
```

在生成 repository 实现时，Spring Data 检查存储库接口中的任何方法，解析方法名称，并尝试在持久化对象的上下文中理解方法的用途（在本例中是 `TacoOrder`）。本质上，Spring Data 定义了一种小型的领域特定语言（DSL），其中持久化细节用 repository 中的方法签名表示。

Spring Data 知道这个方法是用来查找订单的，因为已经用 `TacoOrder` 参数化了 `CrudRepository`。方法名 `findByDeliveryZip()` 表明，该方法应该通过将其 `deliveryZip` 属性与作为参数，传递给匹配的方法来查找所有订单实体。

`findByDeliveryZip()` 方法非常简单，但是 Spring Data 也可以处理更有趣的方法名。repository 的方法由一个动词、一个可选的主语、单词 *by* 和一个谓词组成。在 `findByDeliveryZip()` 中，动词是 *find*，谓词是 *DeliveryZip*，主语没有指定，暗示是一个 `TacoOrder`。

让我们考虑另一个更复杂的例子。假设需要查询在给定日期范围内投递给指定邮政编码的所有订单。在这种情况下，当添加到 `OrderRepository` 时，下面的方法可能会被证明是有用的：

```
List<TacoOrder> readOrdersByDeliveryZipAndPlacedAtBetween(  
    String deliveryZip, Date startDate, Date endDate);
```

图 3.2 说明了在生成 repository 实现时，Spring Data 如何解析和理解

`readOrdersByDeliveryZipAndPlacedAtBetween()` 方法。可以看到，`readOrdersByDeliveryZipAndPlacedAtBetween()` 中的动词是 *read*。Spring Data 还将 *find*、*read* 和 *get* 理解为获取一个或多个实体的同义词。另外，如果只希望方法返回一个带有匹配实体计数的 `int`，也可以使用 *count* 作为动词。

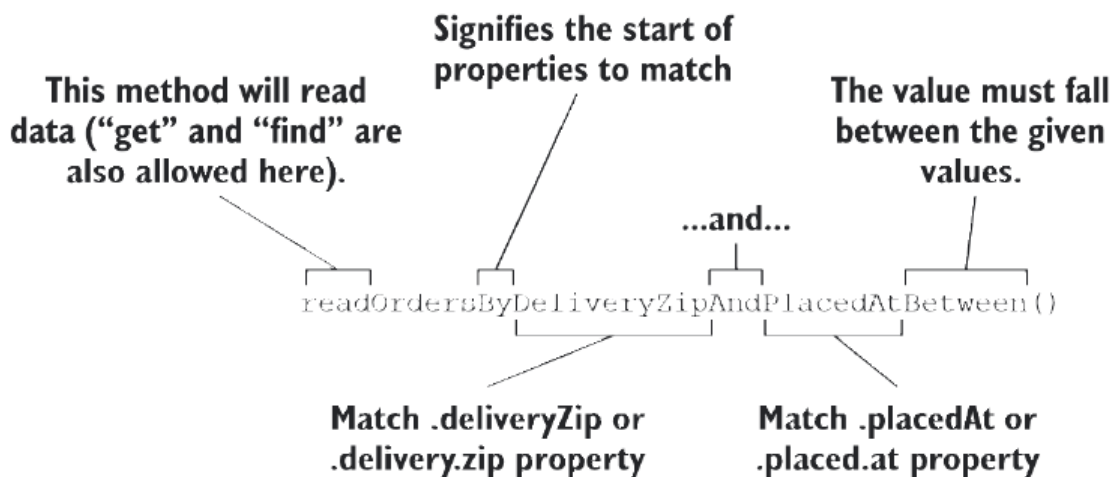


图 3.2 Spring Data 解析 repository 方法特征来确定如何运行查询语句

尽管该方法的主语是可选的，但在这里它表示 `Orders`。Spring Data 会忽略主题中的大多数单词，因此可以将方法命名为 `readPuppiesBy...` 它仍然可以找到 `TacoOrder` 实体，因为这是 `CrudRepository` 参数化的类型。

谓词跟在方法名中的 **By** 后面，是方法签名中最有趣的部分。在本例中，谓词引用两个 **TacoOrder** 属性：**deliveryZip** 和 **placedAt**。**deliveryZip** 属性必须与传递给方法的第一个参数的值一致。**Between** 关键字表示 **deliveryZip** 的值必须位于传入方法最后两个参数的值之间。

除了一个隐式的 **Equals** 操作和 **Between** 操作外，**Spring Data** 方法签名还可以包括以下任何操作：

- **IsAfter, After, IsGreaterThan, GreaterThan**
- **IsGreaterThanEqual, GreaterThanEqual**
- **IsBefore, Before, IsLessThan, LessThan**
- **IsLessThanEqual, LessThanEqual**
- **IsBetween, Between**
- **IsNull, Null**
- **IsNotNull, NotNull**
- **IsIn, In**
- **IsNotIn, NotIn**
- **IsStartingWith, StartingWith, StartsWith**
- **IsEndingWith, EndingWith, EndsWith**
- **IsContaining, Containing, Contains**
- **IsLike, Like**
- **IsNotLike, NotLike**
- **IsTrue, True**
- **IsFalse, False**
- **Is, Equals**
- **IsNot, Not**
- **IgnoringCase, IgnoresCase**

作为 **IgnoringCase** 和 **IgnoresCase** 的替代方法，可以在方法上放置 **AllIgnoringCase** 或 **AllIgnoresCase** 来忽略所有 **String** 比较的大小写。例如，考虑以下方法：

```
List<TacoOrder> findByDeliveryToAndDeliveryCityAllIgnoresCase(
    String deliveryTo, String deliveryCity);
```

最后，还可以将 **OrderBy** 放在方法名的末尾，以便根据指定的列对结果进行排序。例如，通过 **deliveryTo** 属性来订购：

```
List<TacoOrder> findByDeliveryCityOrderByDeliveryTo(String city);
```

虽然命名约定对于相对简单的查询很有用，但是对于更复杂的查询，不需要太多的想象就可以看出方法名称可能会失控。在这种情况下，可以随意将方法命名为任何想要的名称，并使用 **@Query** 对其进行注解，以显式地指定调用方法时要执行的查询，如下例所示：

```
@Query("Order o where o.deliveryCity='Seattle'")
List<TacoOrder> readOrdersDeliveredInSeattle();
```

在这个 **@Query** 的简单用法中，请求在西雅图交付的所有订单。但是也可以使用 **@Query** 来执行几乎任何想要的查询，即使通过遵循命名约定来实现查询很困难或不可能。

自定义查询方法也适用于 **Spring Data JDBC**，但有两个关键区别：

- 所有自定义查询方法都需要 **@query**。这是因为，与 **JPA** 不同，没有映射元数据以帮助 **Spring Data JDBC** 关联查询与方法称。



- `@Query` 中指定的所有查询必须是 SQL 查询，而不是 JPA 查询。

在下一章中，我们将扩展 **Spring Data** 在非关系数据库中使用。当我们这样做时，您将看到自定义查询方法的工作方式非常类似，尽管查询 `@Query` 中使用的语言将特定于底层数据库。

## 3.4 总结

- JdbcTemplate 大大简化了 JDBC 的工作。
- 当需要知道数据库生成的 id 时，可以同时使用 PreparedStatementCreator 和 KeyHolder。
- Spring Data JPA 使得 JPA 持久化就像编写存储库接口一样简单。

## 第 4 章 处理非关系型数据

本章内容：

- 将数据持久化到 **Cassandra**
- **Cassandra** 中的数据建模
- 在 **MongoDB** 中使用文档数据

丰富多彩的生活才乐趣横生。

您可能最喜欢冰淇淋的味道。您最喜欢的某种口味，通常是因为这比其他更能满足您的欲望。但是大多数人，尽管有自己喜欢的口味，还会不时尝试其他不同的口味。

数据库就像冰淇淋。几十年来，关系型数据库一直是用户最喜欢的类型。但是现在比以往任何时候都有更多的选择。所谓的“**NoSQL**”数据库提供了不同的概念和结构。尽管选择可能仍然在某种程度上取决于您的喜爱好，但有些数据库更适合于保存其他类型的数据。

幸运的是，**Spring Data** 支持了许多 **NoSQL** 数据库，包括 **MongoDB**、**Cassandra**、**Couchbase**、**Neo4j**、**Redis** 等等。而且幸运的是，无论选择哪个数据库，编程模型几乎是相同的。

本章没有足够的篇幅涵盖 **Spring Data** 支持的所有数据库类型。但为了让您了解 **Spring Data** 的其他“风味”，我们将研究两种流行的 **NoSQL** 数据库，**Cassandra** 和 **MongoDB**，并了解如何创建 **Repository** 进行数据持久化。让我们先看看如何使用 **Spring Data** 创建 **Cassandra Repository**。

## 4.1 使用 Cassandra Repository

Cassandra 是一个分布式、高性能、高可用、最终一致、分区行存储的 NoSQL 数据库。

这些词描述了 Cassandra 的主要特点，每一个都准确的说明了 Cassandra 的一种能力。简单地说，Cassandra 以数据行的方式处理数据，这些数据被写入到表中，这些表分区后，存储到多个节点上。没有一个节点存储所有数据，且任何给定的行，都是以跨多个节点的多个复本存储的，这种方式就消除了单点故障。

Spring Data Cassandra 为 Cassandra 提供了自动 Repository 支持。这和为关系数据库提供支持的 Spring Data JPA 有很大不同。此外，Spring Data Cassandra 提供了新的注解，用于将应用程序实体类型映射到后台数据库结构上。

在我们进一步探索 Cassandra 之前，必须始终牢记一点。就是尽管 Cassandra 与 Oracle、SQL Server 等关系型数据库有许多相似的概念，但 Cassandra 不是一个关系型数据库。Cassandra 在很多方面与关系型数据库不同，我将试图解释 Cassandra 的特质，以及通过 Spring Data 使用它。我鼓励您阅读 Cassandra

(<http://cassandra.apache.org/doc/latest/>) 的官方文档，以彻底全面的了解为什么 Cassandra 如此大受欢迎。

让我们从在 Taco Cloud 项目中启用 Spring Data Cassandra 开始。

### 4.1.1 启用 Spring Data Cassandra

要开始使用 Spring Data Cassandra，您需要为 Spring Boot 添加 Spring Data Cassandra 依赖。实际上有两个独立的 Spring Data Cassandra 依赖项可供选择：一个用于响应式数据持久化，另一个用于标准、非响应式持久化。

我们将在第 15 章后面讨论更多关于编写响应式 Repository 的内容。现在，我们使用非响应式 starter 依赖项：

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-cassandra</artifactId>
</dependency>
```

通过勾选 **Cassandra** 复选框，也可以在 Initializr 中获得此依赖项。

现在，重要的是理解这种依赖关系是代替了 Spring Data JPA 依赖项的。因为我们不是将 Taco Cloud 数据使用 JPA 持久化到关系数据库，而是使用 Spring Data 将数据持久化到 Cassandra 数据库。因此，您要移除 Spring Data JPA 以及任何关系数据库的依赖项（例如 JDBC 驱动程序或 H2 依赖项）。

Spring Data Cassandra stater 的依赖会带来一些下层依赖，其中主要包括 Spring Data Cassandra 的包。有了这些包位于运行时类路径中，自动配置会触发 Cassandra Repository 的创建。这意味着可以用很少的显式配置来编写 Cassandra Repository。

Cassandra 作为多节点的集群运行，这些节点共同充当一个完整的数据库系统。如果您没有 Cassandra 集群可以使用，则可以为其启动单节点集群，为开发目的使用 Docker 创建：

```
$ docker network create cassandra-net
$ docker run --name my-cassandra \
  --network cassandra-net \
  -p 9042:9042 \
  -d cassandra:latest
```

这将启动单节点集群，并使用端口（9042），供您的应用程序访问。

不过，您也需要提供一些必要配置。至少，您需要配置一个键空间的名称，Repository 将在该空间中进行操作。为此，首先需要创建这样一个键空间。

注意在 Cassandra 中，键空间是 Cassandra 节点中的一组表。它大致类似于关系数据库中的一个库。

尽管可以配置 Spring Data Cassandra 自动地创建键空间，但手动创建通常要容易得多（或者使用现有键空间）。使用 **Cassandra CQL (Cassandra Query Language)**，您可以为 Taco Cloud 应用程序创建键空间。您可以使用 Docker 启动 CQL shell，如下所示：

```
$ docker run -it --network cassandra-net --rm cassandra cqlsh my-cassandra
```

注意，如果此命令无法启动 CQL Shell，而出现一个错误，指示“Unable to connect to any servers”，请稍等一两分钟，然后重试。需要确保 Cassandra 集群已完全启动，然后再启动 CQL Shell。

当 shell 准备就绪后，请按如下方式使用 create keyspace 命令：

```
cqlsh> create keyspace tacocloud
... with replication={'class':'SimpleStrategy', 'replication_factor':1}
... and durable_writes=true;
```

简单地说，这将创建一个名为 **tacocloud** 的键空间，并在持久化时进行简单复制。通过将复制因子设置为 **2**，可以让 **Cassandra** 为每行数据保留一个副本。复制策略决定了如何处理复制。这个 **SimpleStrategy** 复制策略适合于单个数据中心（或演示环境）。但是如果您的 **Cassandra** 集群分布在多个数据中心，可以考虑 **NetworkTopologyStrategy** 复制策略。请参阅 **Cassandra** 有关文档，以了解相关复制策略信息，以及创建键空间的其他替代方法。

现在已经创建了键空间，您需要配置 `spring.cassandra.keyspace-name` 属性来告诉 **Spring Data Cassandra** 使用该键空间：

```
spring:
  data:
    cassandra:
      keyspace-name: taco_cloud
      schema-action: recreate
      local-datacenter: datacenter1
```

这里，还设置了 `spring.data.cassandra.schema-action` 为 **recreate-drop-unused**。此设置对于开发环境非常有用，因为它确保每次开始使用应用程序时，都会删除和重新创建任何表和用户定义类型。默认值 **none** 不执行任何操作，这在生产环境中设置是正常的，因为不会希望应用程序开始时，删除所有表。

最后，`spring.data.cassandra.local-datacenter` 属性标识本地数据中心，用于设置 **Cassandra** 的负载均衡策略。在单个节点中，设置了使用“**datacenter1**”这个值。有关 **Cassandra** 负载均衡策略的更多信息，以及如何设置本地数据中心，请参阅 **DataStax Cassandra** 驱动程序参考文件。

这些是使用本地运行的 **Cassandra** 数据库所需的唯一配置。但是，依据您自己的 **Cassandra** 集群配置，除了这两个属性之外，您还可以设置其他属性。

默认情况下，**Spring Data Cassandra** 假设 **Cassandra** 在本地运行并且监听端口 **9042**。如果你的环境中不是这样，比如在生产环境中，您可能需要设置 `spring.data.cassandra.contact-points` 和 `spring.data.cassandra.port` 属性：

```
spring:
  data:
    cassandra:
      keyspace-name: tacocloud
      local-datacenter: datacenter1
      contact-points:
        - casshost-1.tacocloud.com
        - casshost-2.tacocloud.com
        - casshost-3.tacocloud.com
      port: 9043
```

注意 `spring.data.cassandra.contact-points` 属性用于标识 **Cassandra** 的主机名。每一行指定一个正在运行 **Cassandra** 的主机节点。默认情况下，它设置为 **localhost**，但可以将其设置为主机名列表。它将尝试每个节点，直到它能够连接到其实的一个。这是为了确保 **Cassandra** 集群中没有单点故障，应用程序将能够通过给定的某一节点连接到群集。

您可能还需要为 **Cassandra** 集群指定用户名和密码。这可以设置 `spring.data.cassandra.username` 和 `spring.data.cassandra.password` 属性：

```
spring:
  data:
    cassandra:
      ...
      username: tacocloud
      password: s3cr3tP455w0rd
```

这样您的项目中启用并配置了 **Spring Data Cassandra**，您就可以将实体类型映射到 **Cassandra** 表和编写 **Repository** 了。但是首先，让我们先考虑一下 **Cassandra** 数据建模的几个基本点。

### 4.1.2 理解 Cassandra 数据模型

我已经提到过，Cassandra 与关系型数据库有很大的不同。在将实体类型映射到 Cassandra 表之前，先来了解一下 Cassandra 数据建模的一些方法。这些建模方式不同于以往关系型数据库建模。

这些是 Cassandra 数据建模中最重要的几点：

- Cassandra 表可以有任意数量的列，但不是所有的行都必须使用所有这些列。
- Cassandra 数据库可以被分割到多个分区中。给定表中的任何行可能由一个或多个分区管理，但在每个分区不太可能包含所有行。
- Cassandra 表有两种键：分区键和聚类键。对每一行的分区键执行哈希操作，以确定由哪个分区对该行进行管理。聚类键决定了在分区中维护的那些行的顺序（可能并不出现在查询结果中）。
- Cassandra 针对读取操作进行了高度优化。因此，对于高度非规范化的表，或跨多个表冗余数据保存的情况是比较适合的。（例如，客户信息保存在客户数据表，同时也在客户下的订单表中保存一份副本。）

总之，将 Taco Cloud 中的实体类型适配成以 Cassandra 存储，不只是简单地把几个 JPA 注解替换成 Cassandra 注解的问题。您需要考虑如何对实体数据进行重新建模。



### 4.1.3 Cassandra 持久化实体映射

在第 3 章中，您在实体类型（`Taco`、`Ingredient`、`Order` 等等）上使用 JPA 规范提供的注解。这些注解将实体类型映射到要持久化的关系型数据库表上。但这些注解在使用 **Cassandra** 进行持久化时不起作用，**Spring Data Cassandra** 提供了一组自己的注解，用于完成类似的映射功能。

让我们从最简单的 `Ingredient` 类开始，这个新的 `Ingredient` 类如下所示：

```
package tacos;

import org.springframework.data.cassandra.core.mapping.PrimaryKey;
import org.springframework.data.cassandra.core.mapping.Table;

import lombok.AccessLevel;
import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;
import lombok.RequiredArgsConstructor;

@Data
@AllArgsConstructor
@NoArgsConstructor(access=AccessLevel.PRIVATE, force=true)
@Table("ingredients")
public class Ingredient {

    @PrimaryKey
    private String id;
    private String name;
    private Type type;

    public static enum Type {
        WRAP, PROTEIN, VEGGIES, CHEESE, SAUCE
    }

}
```

`Ingredient` 类似乎否定了我所说的只需替换一些注解。在这里不用 JPA 持久化那样的 `@Entity` 注解，而是用 `@Table` 注解，以指示应该将 `Ingredient` 持久化到一张名为 `ingredients` 的表中。不是用 `@id` 注解在 `id` 属性上，而是用 `@PrimaryKey` 注解。到目前为止，您似乎只替换了很少的几个注解。

但别让 `Ingredient` 类欺骗了您。`Ingredient` 类是最简单的实体类型。当您处理 `Taco` 类时，事情会变得复杂。

程序清单 4.1 为 `Taco` 类添加 **Cassandra** 持久化注解

```

package tacos;
import java.util.ArrayList;
import java.util.Date;
import java.util.List;
import java.util.UUID;

import javax.validation.constraints.NotNull;
import javax.validation.constraints.Size;

import org.springframework.data.cassandra.core.cql.Ordering;
import org.springframework.data.cassandra.core.cql.PrimaryKeyType;
import org.springframework.data.cassandra.core.mapping.Column;
import org.springframework.data.cassandra.core.mapping.PrimaryKeyColumn;
import org.springframework.data.cassandra.core.mapping.Table;

import com.datastax.oss.driver.api.core.uuid.Uuids;

import lombok.Data;

@Data
@Table("tacos")
public class Taco {

    @PrimaryKeyColumn(type=PrimaryKeyType.PARTITIONED)
    private UUID id = Uuids.timeBased();

    @NotNull
    @Size(min = 5, message = "Name must be at least 5 characters long")
    private String name;

    @PrimaryKeyColumn(type=PrimaryKeyType.CLUSTERED,
        ordering=Ordering.DESCENDING)
    private Date createdAt = new Date();

    @Size(min=1, message="You must choose at least 1 ingredient")
    @Column("ingredients")
    private List<IngredientUDT> ingredients = new ArrayList<>();

    public void addIngredient(Ingredient ingredient) {
        this.ingredients.add(TacoUDRUtils.toIngredientUDT(ingredient));
    }
}

```

正如您所看到的，映射 **Taco** 类的内容更为复杂。与 **Ingredient** 一样，`@Table` 注解用于将 **TACO** 类标识为使用 **tacos** 表进行保存。但这是唯一与 **Ingredient** 类相似的地方。

**id** 属性仍然是主键，但它只是两个主键列中的一个。更具体地说，**id** 属性使用注解 `@PrimaryKeyColumn`，且设置类型为 `PrimaryKeyType.PARTITIONED`。这样设置指定了 **id** 属性作为分区键，用于确定每行 **taco** 应该将数据写入哪个 **Cassandra** 分区。

您还注意到 **id** 属性现在是 **UUID**，而不是 **Long** 类型。尽管不是强制的，但 **ID** 值属性通常为 **UUID** 类型。此外，新 **Taco** 对象的 **UUID** 是基于时间的 **UUID**。（但从数据库读取已有的 **Taco** 时，可能会覆盖该值）。

再往下一点，您会看到 **createdAt** 属性被映射为主键列的另一个属性。本例中，设置了 `@PrimaryKeyColumn` 的 **type** 属性为 `PrimaryKeyType.CLUSTERED`，它将 **createdAt** 属性指定为聚类键。如前所述，聚类键用于确定分区中的行数据的顺序。更具体地说，排序设置为降序。因此，在给定的分区中，较新行首先出现在 **tacos** 表中。

最后，**ingredients** 属性现在是一个 **IngredientUDT** 对象的列表。正如您所记得的，**Cassandra** 表是非规范化的，可能包含从其他表复制的数据。虽然 **ingredients** 表将作为所有可用 **Ingredient** 的记录表，但每个 **taco** 的 **Ingredient** 会在 **ingredients** 中重复出现。这不仅仅是简单地引用 **ingredients** 表中的一行或多行，而是在 **ingredients** 属性中包含完整数据。

但为什么要引入一个新的 `IngredientUDT` 类呢？为什么不重用 `Ingredient` 类呢？简单地说，包含数据集合的列，例如 `ingredients` 列，必须是基本类型（整数、字符串等）或用户自定义类型的集合。

在 `Cassandra` 中，用户自定义的类型和基本类型相比，允许您声明更丰富的表和列属性。通常，它们类似关系型数据库的外键。但与外键不同，外键只保存在另一个表的行数据中。但用户自定义类型的列，实际上可能携带从另一个表的行中复制的数据。对于 `tacos` 表中的 `ingredients` 列，它将包含所有 `ingredients` 的数据。

不能将 `Ingredient` 类用作自定义的类型，因为 `@Table` 注解已经将其映射为 `Cassandra` 中持久化的一个实体。因此，您必须创建一个新类，来定义如何在 `taco` 表上的 `ingredients` 列。`IngredientUDT` 类用于达到此目的（其中“UDT”是 `user defined type` 的缩写，表示用户自定义类型）：

```
package tacos;

import org.springframework.data.cassandra.core.mapping.UserDefinedType;

import lombok.AccessLevel;
import lombok.Data;
import lombok.NoArgsConstructor;
import lombok.RequiredArgsConstructor;

@Data
@RequiredArgsConstructor
@NoArgsConstructor(access = AccessLevel.PRIVATE, force = true)
@UserDefinedType("ingredient")
public class IngredientUDT {

    private final String name;

    private final Ingredient.Type type;

}
```

尽管 `IngredientUDT` 看起来很像 `Ingredient`，但它的映射简单的多。它用 `@UserDefinedType` 注解，以将其标识为用户自定义的类型。除此之外，它就是一个具有一些属性的简单类。

您还将注意到，`IngredientUDT` 类并不包含 `id` 属性。尽管它可能包含从 `Ingredient` 复制来的 `id` 属性的副本。事实上，用户自定义的类型可能包含您需要的任何属性，它不需要与任何表定义进行一对一的映射。

我意识到您现在可能没有一个清晰的完整视图，来理解用户自定义类型中的数据是如何关联，并持久化到库中的。图 4.1 显示了整个 `Taco Cloud` 的数据模型，包括用户自定义的类型。

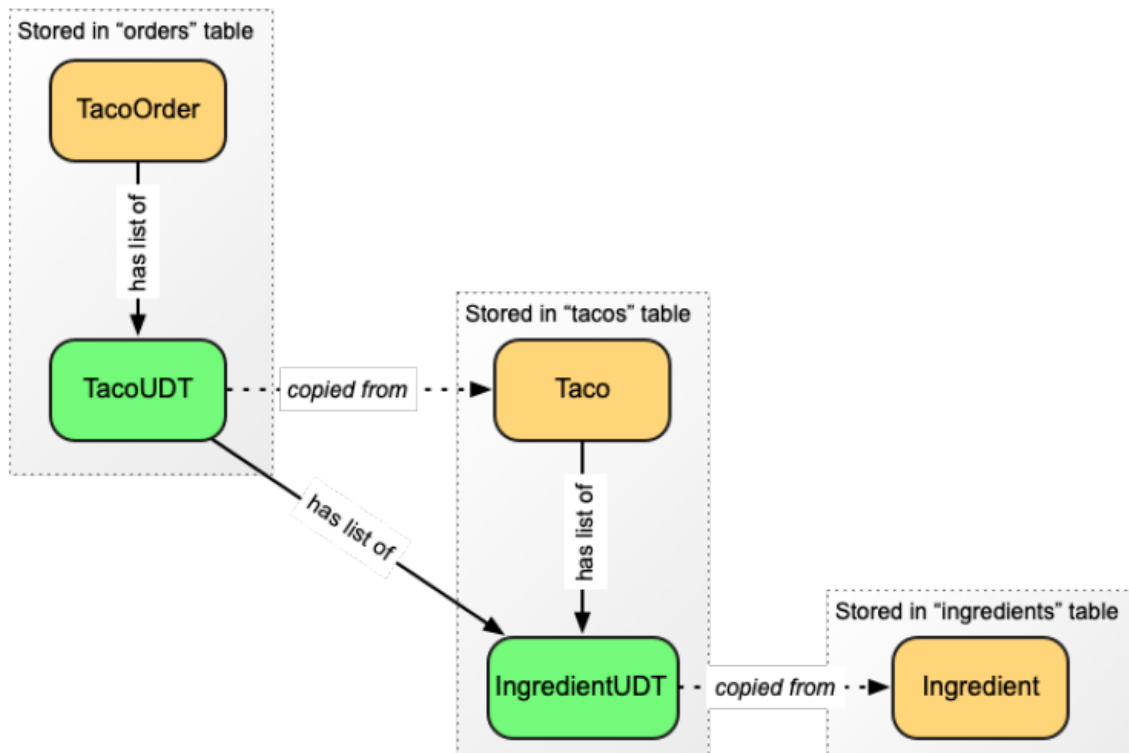


图 12.1 不用外链和关联, **Cassandra** 是反范式的, 用户自定义类型包含其他表中的数据复本。

具体到您刚刚创建的用户自定义类型, 请注意 **Taco** 有一个 **IngredientUDT**, 它保存从 **Ingredient** 对象复制的数据。当一个 **Taco** 被持久化的时候, 是 **Taco** 对象和其中的 **IngredientUDT** 列表被保存到 **tacos** 表中。**IngredientUDT** 的列表数据全部保存到 **ingredients** 列中。

另一种方法可以帮助您理解用户自定义类型的使用, 就是查询 **tacos** 表在数据库中的数据。使用 **CQL** 和 **Cassandra** 附带的 **cqlsh** 工具可以看到以下结果:

```

cqlsh:tacocloud> select id, name, createdAt, ingredients from tacos;

id      | name      | createdAt | ingredients
-----+-----+-----+-----
827390...| Carnivore | 2018-04...| [{name: 'Flour Tortilla', type: 'WRAP'},
                                {name: 'Carnitas', type: 'PROTEIN'},
                                {name: 'Sour Cream', type: 'SAUCE'},
                                {name: 'Salsa', type: 'SAUCE'},
                                {name: 'Cheddar', type: 'CHEESE'}]

(1 rows)

```

如您所见, **id**、**name** 和 **createdAt** 列包含简单值。它们与您熟悉的关系型数据的查询没有太大的不同。但是 **ingredients** 有点不同。因为这个列定义为包含用户自定义类型的集合 (由 **IngredientUDT** 定义), 它的值显示为一个 **JSON** 数组, 其中包含 **JSON** 对象。

您可能注意到图 4.1 中的其他用户自定义类型。您需要继续将其他实体映射到 **Cassandra** 表。还需要加一些注解, 包括 **TacoOrder** 类。下一个清单展示了为 **Cassandra** 持久化进行注解的 **TacoOrder** 类。

程序清单 4.2 映射 **TacoOrder** 类到 **Cassandra** 数据库的 **tacoorders** 表

```

package tacos;
import java.io.Serializable;
import java.util.ArrayList;
import java.util.Date;
import java.util.List;
import java.util.UUID;

import javax.validation.constraints.Digits;
import javax.validation.constraints.NotBlank;
import javax.validation.constraints.Pattern;

import org.hibernate.validator.constraints.CreditCardNumber;
import org.springframework.data.cassandra.core.mapping.Column;
import org.springframework.data.cassandra.core.mapping.PrimaryKey;
import org.springframework.data.cassandra.core.mapping.Table;

import com.datastax.oss.driver.api.core.uuid.Uuids;

import lombok.Data;

@Data
@Table("orders")
public class TacoOrder implements Serializable {

    private static final long serialVersionUID = 1L;

    @PrimaryKey
    private UUID id = Uuids.timeBased();

    private Date placedAt = new Date();

    // delivery and credit card properties omitted for brevity's sake

    @Column("tacos")
    private List<TacoUDT> tacos = new ArrayList<>();

    public void addTaco(TacoUDT taco) {
        this.tacos.add(taco);
    }
}

```

程序清单 4.2 故意省略了 **TacoOrder** 类的一些属性，这些属性本身并不适用对 **Cassandra** 数据建模的探讨。剩下的一些属性和映射，类似于 **Taco** 上的注解。**@Table** 用于将 **TacoOrder** 映射到 **tacoorders** 表。在里，由于您不关心排序，**id** 属性只需用 **@PrimaryKey** 注解，指定它既是一个分区键，又是一个具有默认顺序的聚类键。

**tacos** 属性很有趣，因为它是一个 **List<TacoUDT>**，而不是一个 **Taco** 列表。这里 **TacoOrder** 和 **Taco/TacoUDT** 之间的关系，类似于 **Taco** 和 **Ingredient/IngredientUDT** 的关系。也就是说，不是通过外键将表中的多行数据链接起来，而是在 **TacoOrder** 表中包含所有相关的 **taco** 数据，以优化表的读取速度。

至于 **TacoUDT** 类，它与 **IngredientUDT** 类非常相似，不过它包含引用其他用户定义类型的集合：

```
package tacos;

import java.util.List;
import org.springframework.data.cassandra.core.mapping.UserDefinedType;
import lombok.Data;

@Data
@UserDefinedType("taco")
public class TacoUDT {

    private final String name;
    private final List<IngredientUDT> ingredients;

}
```

尽管，重用在第 3 章中创建的实体类，或者把一些 JPA 注解换成 Cassandra 注解，应该更方便，但 Cassandra 持久化的本质特性决定了不能这样做。它要求您重新思考数据的建模方式。现在实体都已经映射了，可以编写 Repository 了。

### 4.1.4 编写 Cassandra Repository

正如您在第 3 章中看到的，编写一个基于 Spring Data 的 Repository，只需要声明扩展自 Spring Data 的基础接口。以及有选择性地，声明一些自定义查询的其他查询方法。事实证明，编写 Cassandra Repository 也没有什么不同。

事实上，在我们已经编写的 Repository 几乎不需要更改，就可以适配 Cassandra 持久化。例如，考虑我们在第 3 章中创建的 IngredientRepository:

```
package tacos.data;

import org.springframework.data.repository.CrudRepository;

import tacos.Ingredient;

public interface IngredientRepository
    extends CrudRepository<Ingredient, String> {
}
```

通过扩展 CrudRepository，IngredientRepository 现在准备好了持久化 ID 属性（对于 Cassandra 来说是主键属性）是 String 类型的 Ingredient 对象。真是太棒了！IngredientRepository 完全不用更改。

OrderRepository 所需的更改只要稍微多一些。当扩展 CrudRepository 时，指定的 ID 参数的类型不是 Long，而是 UUID。

```
package tacos.data;

import java.util.UUID;

import org.springframework.data.repository.CrudRepository;

import tacos.TacoOrder;

public interface OrderRepository
    extends CrudRepository<TacoOrder, UUID> {
}
```

Cassandra 很强大，当它与 Spring Data 结合，您就可以在 Spring 应用程序中体验到这些强大功能。现在让我们转移注意力，来看看另一个 Spring Data 支持的数据库：MongoDB。

## 4.2 编写 MongoDB Repository

MongoDB 是另一个著名的 NoSQL 数据库。Cassandra 是一个行存储数据库，MongoDB 一般被认为是一个文档数据库。更具体地说，MongoDB 以 BSON（Binary JSON）格式存储文档数据，并且提供了大致类似于其他数据库的数据查询方式。

与 Cassandra 一样，关键一点是理解 MongoDB 不是关系型数据库。管理 MongoDB 服务器集群，以及为 MongoDB 进行数据建模，都需要与处理其他类型数据库不同的思维方式。

与使用 Spring Data 结合 JPA 或 Cassandra 一样，使用 MongoDB 结合 Spring Data 也没有什么显著的不同。您需要使用注解，将实体类型映射到文档结构上。您将编写同样遵循相同编程模型的接口。当然，在做这些事情之前，您必须在项目中启用 Spring Data MongoDB。



## 4.2.1 启用 Spring Data MongoDB

要开始使用 Spring Data MongoDB，您需要添加 Spring Data MongoDB 的 starter 依赖。Spring Data MongoDB 有两个独立的 starter 可供选择：响应式的或者非响应式的。我们将在第 15 章中讨论响应式的持久化方案。现在，非响应式的 MongoDB，您可以添加以下依赖项：

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>
    spring-boot-starter-data-mongodb
  </artifactId>
</dependency>
```

在使用 Spring Initializr 创建工程时，可以通过勾选 MongoDB 复选框以添加这个依赖。

通过将 starter 添加到构建中，将触发自动配置以启用 Spring Data 支持编写 Repository 接口。就像第 3 章中为 JPA 或者本章前面为 Cassandra 编写的接口。

默认情况下，Spring Data MongoDB 假设 MongoDB 服务运行在本地，并且监听端口 27017。如果您的机器上安装了 Docker，那么获得 MongoDB 服务可以使用以下命令：

```
$ docker run -p 27017:27017 -d mongo:latest
```

为了方便测试和开发，您可以选择使用嵌入式 Mongo 数据库。为此，可以把 Flapdoodle Embedded MongoDB 依赖项添加到项目中：

```
<dependency>
  <groupId>de.flapdoodle.embed</groupId>
  <artifactId>de.flapdoodle.embed.mongo</artifactId>
  <!-- <scope>test</scope> -->
</dependency>
```

这个 Flapdoodle 嵌入式数据库，为测试和开发提供了便利。它是内存中的 Mongo 数据库，就像您在处理关系型数据库时使用 H2 那样。也就是说，您不需要运行单独的数据库服务，而是所有数据都放在内存中，并且重新启动应用程序时，所有数据将被清除。

嵌入式数据库对于开发和测试来说是很好的，但是一旦您的应用程序要发布到生产环境，您需要确保设置了一些必要属性，以便告知 Spring Data MongoDB 数据库在哪里，以及如何访问：

```
spring:
  data:
    mongodb:
      host: mongodb.tacocloud.com
      port: 27017
      username: tacocloud
      password: s3cr3tp455w0rd
      database: tacocloud
```

这些属性并不是都需要设置的。它们的作用是帮助 Spring Data MongoDB 了解到 Mongo 数据库未在本地运行。如果分解一下，每个属性所指定的配置如下：

- `spring.data.mongodb.host` —— Mongo 服务器的主机名（默认值：localhost）
- `spring.data.mongodb.port` —— Mongo 服务器正在侦听的端口（默认值：27017）
- `spring.data.mongodb.username` —— 用于访问 Mongo 数据库的用户名

- `spring.data.mongodb.password` —— 用于访问 Mongo 数据库的密码
- `spring.data.mongodb.database` —— 数据库名称（默认值：`test`）

现在，您的项目中启用了 **Spring Data MongoDB**。下一步，您需要对实体对象添加注解，以便通过 **MongoDB** 进行文档数据的持久化。

## 4.2.2 MongoDB 持久化实体映射

Spring Data MongoDB 提供了一组注解，用于将实体类映射到 MongoDB 的文档结构上。这组 Spring Data MongoDB 注解有好几个，其中有几个是最常用的：

- `@Id` —— 将属性指定为文档的 Id（在 Spring Data Commons 中）
- `@Document` —— 声明实体类型要持久化为 MongoDB 文档
- `@Field` —— 指定持久化文档中存储这个属性的字段名称（以及可选的顺序）
- `@Transient` —— 指定一下属性不需要持久化

在这几个注释中，只有 `@Id` 和 `@Document` 注解是必须有的。除非另行指定，否则没有使用 `@Field` 或 `@Transient` 注解的属性，将假定字段名等于属性名。

为 `Ingredient` 类添加这些注解后，可以得到以下结果：

```
package tacos;

import org.springframework.data.annotation.Id;
import org.springframework.data.mongodb.core.mapping.Document;

import lombok.AccessLevel;
import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;

@Data
@Document
@AllArgsConstructor
@NoArgsConstructor(access=AccessLevel.PRIVATE, force=true)
public class Ingredient {

    @Id
    private String id;
    private String name;
    private Type type;
    public static enum Type {
        WRAP, PROTEIN, VEGGIES, CHEESE, SAUCE
    }
}
```

如您所见，将 `@Document` 注解放置在类级别上，以指示 `Ingredient` 是一个文档实体，可以写入 Mongo 数据库并从中读取。默认情况下，`Collection` 名称（Mongo 中类似于关系数据库的 `Table`）基于类名，第一个字母小写。因为您没有进行指定，`Ingredient` 对象将持久化到名为 `ingredient` 的 `Collection` 中。但是您可以通过设置 `@Document` 的 `collection` 属性来改变这一点：

```
@Data
@AllArgsConstructor
@NoArgsConstructor(access=AccessLevel.PRIVATE, force=true)
@Document(collection="ingredients")
public class Ingredient {
    ...
}
```

您还注意到 `id` 属性已经添加了 `@Id` 注解，这指定了此属性作为持久化文档的 ID。您可以在任何属性上使用 `@Id` 注解，只要其类型是可序列化的，这包括 `String` 和 `Long`。在现在这种情况下，您的 `id` 属性是 `String` 类型，所以没有必要将其更改为其他类型。

到目前为止，进展还不错。您可能还记得在本章前面的内容中，为 **Cassandra** 添加映射时，**Ingredient** 类是容易处理的实体类型。其他类型，如 **Taco**，更复杂一些。让我们看看现在如何映射 **Taco** 类。

**MongoDB** 的文档持久化方法非常适合领域驱动的设计，在聚合根级别应用持久性的方法。**MongoDB** 中的文档通常定义为聚合根，聚合的成员作为子文档。

这对 **Taco Cloud** 意味着，因为 **Taco** 只作为 **TacoOrder** 根聚合时进行持久化，所以 **Taco** 类不需要注释为 **@Document**，也不需要添加 **@Id** 属性。**Taco** 类可以没有任何持久化注解：

```
package tacos;
import java.util.ArrayList;
import java.util.Date;
import java.util.List;

import javax.validation.constraints.NotNull;
import javax.validation.constraints.Size;

import lombok.Data;

@Data
public class Taco {

    @NotNull
    @Size(min=5, message="Name must be at least 5 characters long")
    private String name;

    private Date createdAt = new Date();

    @Size(min=1, message="You must choose at least 1 ingredient")
    private List<Ingredient> ingredients = new ArrayList<>();

    public void addIngredient(Ingredient ingredient) {
        this.ingredients.add(ingredient);
    }
}
```

但是，作为聚合根的 **TacoOrder** 类需要用 **@Document**，并有 **@Id** 的属性。

```

package tacos;
import java.io.Serializable;
import java.util.ArrayList;
import java.util.Date;
import java.util.List;

import javax.validation.constraints.Digits;
import javax.validation.constraints.NotBlank;
import javax.validation.constraints.Pattern;

import org.hibernate.validator.constraints.CreditCardNumber;
import org.springframework.data.annotation.Id;
import org.springframework.data.mongodb.core.mapping.Document;

import lombok.Data;

@Data
@Document
public class TacoOrder implements Serializable {

    private static final long serialVersionUID = 1L;

    @Id
    private String id;

    private Date placedAt = new Date();

    // other properties omitted for brevity's sake

    private List<Taco> tacos = new ArrayList<>();

    public void addTaco(Taco taco) {
        this.tacos.add(taco);
    }
}

```

为了简洁起见，我删掉了 **delivery** 和 **credit card** 字段。但是从剩下的属性来看，很明显，您只需要 `@Document` 和 `@Id` 注解，就像其他实体类型那样。

即便如此，您还是要为 **user** 属性加上 `@Field` 注解，以指定在持久化文档中，使用 **customer** 字段来进行存储。

从到目前为止的经验可以得知，映射 MongoDB 的 **User** 类应该同样容易：

```

@Data
@NoArgsConstructor(access=AccessLevel.PRIVATE, force=true)
@RequiredArgsConstructor
@Document
public class User implements UserDetails {

    private static final long serialVersionUID = 1L;

    @Id
    private String id;

    private final String username;

    private final String password;
    private final String fullname;
    private final String street;
    private final String city;
    private final String state;
    private final String zip;
    private final String phoneNumber;

    // UserDetails method omitted for brevity's sake
}

```

但是，请注意，`id` 属性已更改为 `String`（与 JPA 版本中的 `Long` 或 `Cassandra` 版本中的 `UUID` 类型不同）。正如我前面所说，`@Id` 可以应用于任何 `Serializable` 类型。但是，如果您选择使用 `String` 类型属性作为 ID，您将获得以下好处：`Mongo` 在保存时自动为其赋值（如果字段为 `null`）。通过选择 `String`，将分配一个数据库管理的 ID，并且无需手动设置该属性。

尽管有一些更高级和不寻常的情况，需要额外的映射配置。但在大多数情况下，`@Document` 和 `@Id` 以及偶尔使用 `@Field` 或 `@Transient`，对于 `MongoDB` 的映射就足够了。这几个注解就完全满足了 `Taco Cloud` 应用的类型映射工作。

剩下的就是编写 `Repository` 接口了。

### 4.2.3 编写 MongoDB Repository 接口

Spring Data MongoDB 提供了与 Spring Data JPA 和 Spring Data Cassandra 类似的，自动化的 Repository 支持。

您将首先定义一个 Repository，用于将 Ingredient 对象持久化为文档。您可以编写 IngredientRepository 来扩展 CrudRepository:

```
package tacos.data;

import org.springframework.data.repository.CrudRepository;

import tacos.Ingredient;

public interface IngredientRepository
    extends CrudRepository<Ingredient, String> {
}
```

等一下！它看起来与您在第 4.1 节，为 Cassandra 所写的 IngredientRepository 接口完全相同。实际上，确实是相同的接口，没有任何变化。这也突显了扩展 CrudRepository 的好处，在不同的数据库类型中更具可移植性。为 Cassandra 所写的接口，对于 MongoDB 也同样适用。

来看一下 OrderRepository 接口，您可以看到它非常简单:

```
package tacos.data;

import org.springframework.data.repository.CrudRepository;

import tacos.TacoOrder;

public interface OrderRepository
    extends CrudRepository<TacoOrder, String> {
}
```

与 IngredientRepository 一样，OrderRepository 扩展了 CrudRepository 以获得在其 insert() 方法中提供的优化。其他方面，与您迄今定义的其他一些 Repository 进行比较，就没有什么特别的了。但是要注意，扩展 CrudRepository 时的 ID 参数现在是 String，而不是 Long（对于 JPA）或 UUID（对于 Cassandra）。这是我们在 TacoOrder 中所做的修改，以支持自动生成 ID。

最后，使用 Spring Data MongoDB 与其他 Spring Data 项目并没有太大的不同。只是领域类型使用的注解不同。除了扩展 CrudRepository 时指定的 ID 参数，接口几乎完全相同。

## 4.3 总结

- Spring Data 支持 Cassandra、MongoDB、Couchbase 和 Redis 数据库。
- 应用程序中用于创建 **Repository** 的编程模型，对于不同的底层数据库差别不大。
- 使用非关系型数据库时需要了解，如何依据数据库最终存储数据的方式，来进行恰当的数据建模。



## 第 5 章 Spring 安全

本章内容：

- 自动配置 Spring Security
- 自定义用户存储
- 自定义登录页面
- 防御 CSRF 攻击
- 了解你的用户

你有没有注意到电视情景喜剧里的大多数人都不锁门？在《*Leave it to Beaver*》的时代，人们不锁门并不是什么稀罕事。但是，在我们关心隐私和安全的今天，我们却看到电视上的人物能够畅通无阻地进入他们的公寓和家中，这似乎很疯狂。

信息可能是我们现在拥有的最有价值的东西；骗子们正在寻找通过潜入不安全的应用程序来窃取我们的数据和身份的方法。作为软件开发人员，我们必须采取措施保护应用程序中的信息。无论是用用户名与密码保护的电子邮件帐户，还是用交易密码保护的经济帐户，安全性都是大多数应用程序的一个重要方面。

## 5.1 启用 Spring Security

保护 Spring 应用程序的第一步是将 Spring Boot security starter 依赖项添加到构建中。在项目的 pom.xml 文件中，添加以下 `<dependency>` 内容：

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-security</artifactId>
</dependency>
```

如果正在使用 Spring Tool Suite，这甚至更简单。右键单击 pom.xml 文件并从 Spring 上下文菜单中选择 `Edit Starters`。将出现“Starter Dependencies”对话框。检查 Security 类别下的 Spring Security，如图 5.1 所示。

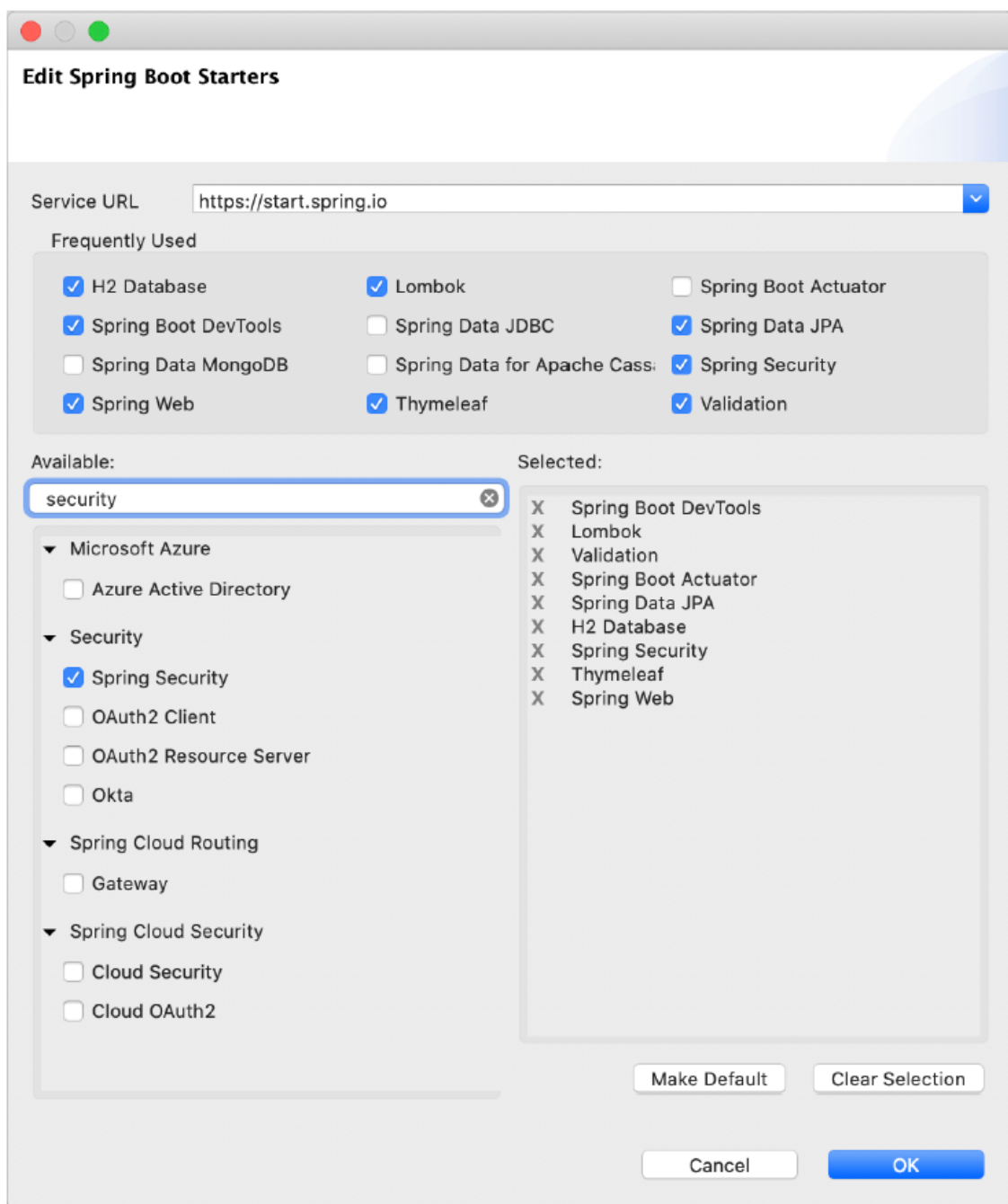


图 5.1 使用 **Spring Tool Suite** 添加 **security starter**

上面的依赖项是保护应用程序所需的唯一的東西。當应用程序启动時，自動配置將檢測類路徑中的 **Spring Security**，並設置一些基本的安全性配置。

如果想嘗試一下，启动应用程序并访问主页（或任何页面）。將提示使用 **HTTP** 基本身份验证对话框进行身份验证，如下所示：

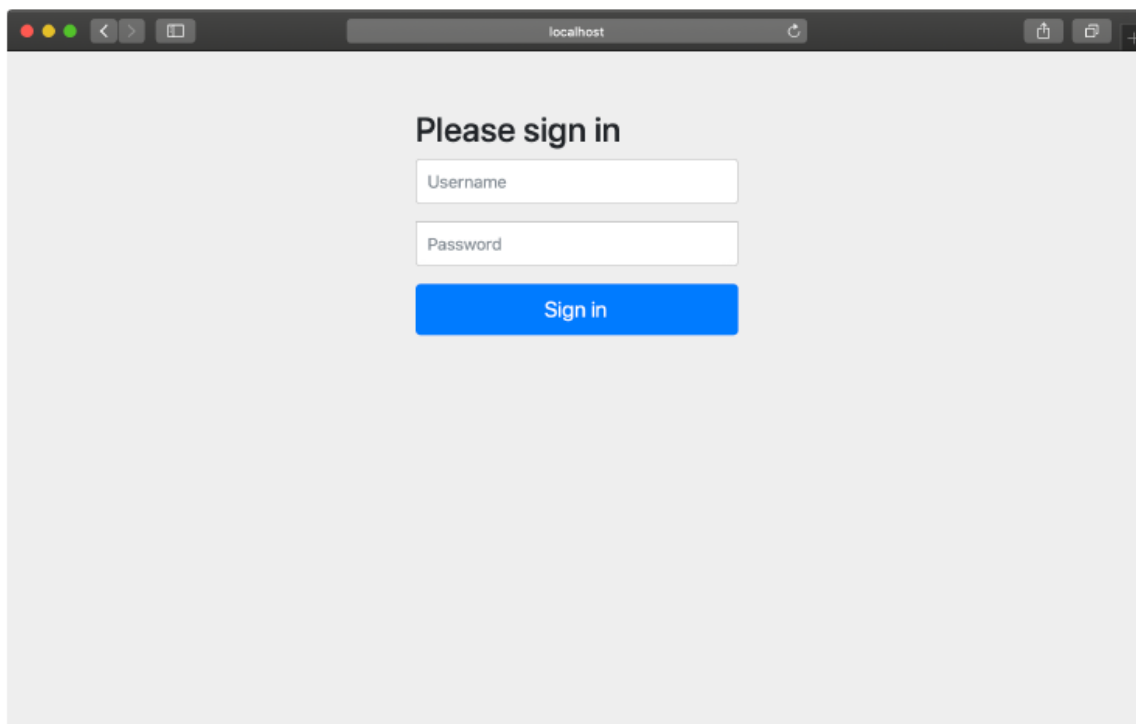


图 5.2 Spring Security 为您提供了一个免费的普通登录页面。

#### 小提示

无痕模式：当手动测试安全问题时，您可能会发现将浏览器设置无痕模式很有用。这将确保每次打开无痕窗口时都使用新的会话。您必须 每次重新登录到应用程序中，而可以确信任何安全方面更改已经生效，并且没有任何旧会话阻止您查看更改。

要想通过认证，需要提供用户名和密码。用户名是 **user**。至于密码，它是随机生成并写入了应用程序日志文件。日志条目应该是这样的：

```
Using generated security password: 087cfc6a-027d-44bc-95d7-cbb3a798a1ea
```

假设正确地输入了用户名和密码，将被授予对应用程序的访问权。

保护 Spring 应用程序似乎非常简单。Taco Cloud 应用程序的已经被保护了，我想我现在可以结束这一章，进入下一个主题了。但是在我们开始之前，让我们考虑一下自动配置提供了什么样的安全性。

只需要在项目构建中添加 **security starter**，就可以获得以下安全特性：

- 所有的 HTTP 请求路径都需要认证。
- 没有特定的角色或权限。
- 身份验证由 HTTP 基本身份验证提供。
- 只有一个用户；用户名是 **user**。

这是一个良好的开端，但我认为大多数应用程序（包括 Taco Cloud）的安全需求将与这些基本的安全特性有很大的不同。

如果要正确地保护 Taco Cloud 应用程序，还有更多的工作要做。至少需要配置 Spring Security 来完成以下工作：

- 提示使用登录页面进行身份验证，而不是使用 HTTP 基本对话框。
- 为多个用户提供注册页面，让新的 Taco Cloud 用户可以注册。

- 为不同的请求路径应用不同的安全规则。例如，主页和注册页面根本不需要身份验证。

为了满足对 **Taco Cloud** 的安全需求，必须编写一些显式的配置，覆盖自动配置提供的内容。首先需要配置一个合适的用户存储，这样就可以有多个用户。

## 5.2 配置身份验证

多年来，有几种配置 Spring Security 的方法，包括冗长的基于 xml 的配置。幸运的是，Spring Security 的几个最新版本都支持基于 Java 的配置，这种配置更容易读写。

在本章结束之前，已经在基于 Java 的 Spring Security 配置中配置了所有 Taco Cloud 安全需求。但是在开始之前，可以通过编写下面清单中所示的基本配置类来简化它。

清单 5.1 Spring Security 的基本配置类

```
package tacos.security;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;

@Configuration
public class SecurityConfig {

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }

}
```

这个基本的安全配置做了什么？嗯，不是很多，但是它确实离需要安全功能更近了一步。其实不多。主要的事情它只是声明了一个 PasswordEncoder bean，我们将在创建新用户和登录验证用户身份时使用。在本例中，我们使用的是 BCryptPasswordEncoder，它是 Spring Security 提供的密码编码器之一，包括：

- BCryptPasswordEncoder —— 应用 bcrypt 加强哈希加密
- NoOpPasswordEncoder —— 不应用任何编码
- Pbkdf2PasswordEncoder —— 应用 PBKDF2 加密
- SCryptPasswordEncoder —— 应用 scrypt 哈希加密
- StandardPasswordEncoder —— 应用 SHA-256 哈希加密

无论您使用哪种密码编码器，重要的是要密码在数据库中永远不会被解密。相反，用户在登录时输入的密码使用相同的算法加密，然后将其与数据库中的编码密码进行比较。比较在 PasswordEncoder 的 matches() 方法中执行。

除了密码编码器之外，我们还将在这个配置类中填充更多的 bean 以定义应用程序的安全性细节。我们将首先配置用户存储，可以处理多个用户身份验证。

为了配置用于身份验证的用户存储，您需要声明 UserDetailsService bean。UserDetailsService 接口相对简单，只有一个方法必须实现。以下是 UserDetailsService 接口：

```
public interface UserDetailsService {

    UserDetails loadUserByUsername(String username) throws UsernameNotFoundException;

}
```

loadUserByUsername() 方法接受用户名并使用它查找 UserDetails 对象，如果找不到给定用户名的用户，则会抛出 UsernameNotFoundException。

事实证明，**Spring Security** 提供了几个 **UserDetailsService** 实现，可以开箱即用。包括：

- 一个内存用户存储
- 基于 **JDBC** 的用户存储
- 由 **LDAP** 支持的用户存储

或者，您可以定制自己的实现，以满足特殊安全需求。

现在，让我们先尝试 **UserDetailsService** 的内存用户存储。

### 5.2.1 内存用户详情服务

用户信息可以保存在内存中。假设只有少数几个用户，这些用户都不可能改变。在这种情况下，将这些用户定义为安全配置的一部分可能非常简单。

例如，下一个清单显示了如何在内存用户存储中配置两个用户“buzz”和“woody”。程序清单 5.2 在内存用户存储中定义用户

清单 5.2 在内存用户服务 bean 中声明用户

```
@Bean
public UserDetailsService userDetailsService(PasswordEncoder encoder) {
    List<UserDetails> userList = new ArrayList<>();
    userList.add(new User(
        "buzz", encoder.encode("password"),
        Arrays.asList(new SimpleGrantedAuthority("ROLE_USER"))));
    userList.add(new User(
        "woody", encoder.encode("password"),
        Arrays.asList(new SimpleGrantedAuthority("ROLE_USER"))));
    return new InMemoryUserDetailsManager(userList);
}
```

这里创建了一个 Spring Security 用户对象列表，每个对象都有用户名、密码和密码一个或多个权限列表。然后使用这个用户列表创建了 InMemoryUserDetailsManager。

如果您现在试用该应用程序，您应该能够以“woody”或“buzz”身份登录，使用“password”作为密码。

内存中的用户存储应用于测试或非常简单的应用程序时非常方便，但是它不允许对用户进行简单的编辑。如果需要添加、删除或更改用户，则必须进行必要的更改，然后重新构建、部署应用程序。

对于 Taco Cloud 应用程序，希望客户能够注册并管理自己的用户帐户。使用这种方式就不适合了。因此让我们看看另一个使用数据库支持用户存储的选项。



### 5.2.2 自定义用户身份验证

在上一章中，决定了使用 **Spring Data JPA** 作为所有 **taco**、配料和订单数据的持久化选项。因此，以同样的方式持久化用户数据是有意义的，这样做的话，数据最终将驻留在关系型数据库中，因此可以使用基于 **JDBC** 的身份验证。但是更好的方法是利用 **Spring Data** 存储库来存储用户。

不过，还是要先做重要的事情，让我们创建表示和持久存储用户信息的域对象和存储库接口。

#### 定义用户实体

当 **Taco Cloud** 用户注册应用程序时，他们需要提供的不仅仅是用户名和密码。他们还会告诉您，他们的全名、地址和电话号码，这些信息可以用于各种目的，不限于重新填充订单（更不用说潜在的营销机会）。

为了捕获所有这些信息，将创建一个 **User** 类，如下所示。

程序清单 5.3 定义用户实体

```

package tacos;
import java.util.Arrays;
import java.util.Collection;
import javax.persistence.Entity;
import javax.persistence.GeneratedValue;
import javax.persistence.GenerationType;
import javax.persistence.Id;
import org.springframework.security.core.GrantedAuthority;
import org.springframework.security.core.authority.
    SimpleGrantedAuthority;
import org.springframework.security.core.userdetails.UserDetails;
import lombok.AccessLevel;
import lombok.Data;
import lombok.NoArgsConstructor;
import lombok.RequiredArgsConstructor;

@Entity
@Data
@NoArgsConstructor(access=AccessLevel.PRIVATE, force=true)
@RequiredArgsConstructor
public class User implements UserDetails {

    private static final long serialVersionUID = 1L;

    @Id
    @GeneratedValue(strategy=GenerationType.AUTO)
    private Long id;

    private final String username;
    private final String password;
    private final String fullname;
    private final String street;
    private final String city;
    private final String state;
    private final String zip;
    private final String phoneNumber;

    @Override
    public Collection<? extends GrantedAuthority> getAuthorities() {
        return Arrays.asList(new SimpleGrantedAuthority("ROLE_USER"));
    }

    @Override
    public boolean isAccountNonExpired() {
        return true;
    }

    @Override
    public boolean isAccountNonLocked() {
        return true;
    }

    @Override
    public boolean isCredentialsNonExpired() {
        return true;
    }

    @Override
    public boolean isEnabled() {
        return true;
    }
}

```

关于 **User** 类首先要注意的是，它与我们创建内存中用户详细信息服务时使用的 **User** 类不同。这一个有更多关于用户的详细信息，我们将用来完善 **taco** 订单，包括用户地址和联系信息。

您可能已经注意到 **User** 类比第 3 章中定义的任何其他实体都更加复杂。除了定义一些属性外，**User** 还实现了来自 **Spring Security** 的 **UserDetails** 接口。

**UserDetails** 的实现将向框架提供一些基本的用户信息，比如授予用户什么权限以及用户的帐户是否启用。

`getAuthorities()` 方法应该返回授予用户的权限集合。各种 `is___` 方法返回一个布尔值，指示用户的帐户是否已启用或过期。

对于 **User** 实体，`getAuthorities()` 方法仅返回一个集合，该集合指示所有用户将被授予 **ROLEUSER** 权限。而且，至少现在，**Taco Cloud** 还不需要禁用用户，所以所有的 `'is_'` 方法都返回 `true` 来表示用户处于活动状态。

定义了 **User** 实体后，现在可以定义存储库接口：

```
package tacos.data;
import org.springframework.data.repository.CrudRepository;
import tacos.User;

public interface UserRepository extends CrudRepository<User, Long> {

    User findByUsername(String username);

}
```

除了通过扩展 **CrudRepository** 提供的 **CRUD** 操作之外，**UserRepository** 还定义了一个 `findByUsername()` 方法，将在用户详细信息服务中使用该方法根据用户名查找 **User**。

如第 3 章所述，**Spring Data JPA** 将在运行时自动生成该接口的实现。因此，现在可以编写使用此存储库的自定义用户详细信息服务了。

## 创建用户详细信息服务

您可能还记得，**UserDetailsService** 接口只定义了一个 `loadUserByUsername()` 方法。这意味着它是一个函数式接口，可以使用 **lambda** 实现，而不是使用完整的实现类。既然我们真的需要 **UserDetailsService** 委托 **UserRepository**，它可以使用以下配置方法简单地声明为 **bean**：

程序清单 5.4 自定义用户详细信息服务

```
@Bean
public UserDetailsService userDetailsService(UserRepository userRepo) {
    return username -> {
        User user = userRepo.findByUsername(username);
        if (user != null) return user;

        throw new UsernameNotFoundException("User '" + username + "' not found");
    };
}
```

`userDetailsService()` 方法将 **UserRepository** 作为参数传入。创建 **bean** 后它返回一个 **lambda**，该 **lambda** 接受一个 `username` 参数，并使用它在给定的 **UserRepository** 上调用 `findByUsername()`。

`loadUserByUsername()` 方法只有一个简单的规则：不允许返回 `null`。因此如果调用 `findByUsername()` 返回 `null`，将会抛出一个 **UsernameNotFoundException**。除此之外，被找到的 **User** 将会被返回。

既然已经有了一个通过 JPA 存储库读取用户信息的自定义用户详细信息服务，那么首先需要的就是一种让用户进入数据库的方法。需要为 Taco Cloud 用户创建一个注册页面，以便注册该应用程序。

## 用户注册

尽管 Spring Security 处理安全性的很多方面，但它实际上并不直接涉及用户注册过程，因此将依赖于 Spring MVC 来处理该任务。下面程序清单中的 `RegistrationController` 类展示并处理注册表单。

程序清单 5.5 用户注册控制器

```
package tacos.security;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import tacos.data.UserRepository;
@Controller
@RequestMapping("/register")
public class RegistrationController {

    private UserRepository userRepo;
    private PasswordEncoder passwordEncoder;

    public RegistrationController(
        UserRepository userRepo, PasswordEncoder passwordEncoder) {
        this.userRepo = userRepo;
        this.passwordEncoder = passwordEncoder;
    }

    @GetMapping
    public String registerForm() {
        return "registration";
    }

    @PostMapping
    public String processRegistration(RegistrationForm form) {
        userRepo.save(form.toUser(passwordEncoder));
        return "redirect:/login";
    }
}
```

与任何典型的 Spring MVC 控制器一样，`RegistrationController` 使用 `@Controller` 进行注解，以将其指定为控制器，并将其标记为组件扫描。它还使用 `@RequestMapping` 进行注解，以便处理路径为 `/register` 的请求。

更具体地说，`registerForm()` 方法将处理 `/register` 的 GET 请求，它只返回注册的逻辑视图名。下面的程序清单显示了定义注册视图的 Thymeleaf 模板。

程序清单 5.6 Thymeleaf 注册表单视图

```

<!DOCTYPE html>
<html xmlns="http://www.w3.org/1999/xhtml"
      xmlns:th="http://www.thymeleaf.org">

  <head>
    <title>Taco Cloud</title>
  </head>

  <body>
    <h1>Register</h1>

    <form method="POST" th:action="@{/register}" id="registerForm">
      <label for="username">Username: </label>
      <input type="text" name="username" /><br/>
      <label for="password">Password: </label>
      <input type="password" name="password" /><br/>
      <label for="confirm">Confirm password: </label>
      <input type="password" name="confirm" /><br/>
      <label for="fullname">Full name: </label>
      <input type="text" name="fullname" /><br/>
      <label for="street">Street: </label>
      <input type="text" name="street" /><br/>
      <label for="city">City: </label>
      <input type="text" name="city" /><br/>
      <label for="state">State: </label>
      <input type="text" name="state" /><br/>
      <label for="zip">Zip: </label>
      <input type="text" name="zip" /><br/>
      <label for="phone">Phone: </label>
      <input type="text" name="phone" /><br/>
      <input type="submit" value="Register" />
    </form>
  </body>
</html>

```

提交表单时，HTTP POST 请求将由 `processRegistration()` 方法处理。`processRegistration()` 的 `RegistrationForm` 对象绑定到请求数据，并使用以下类定义：

```

package tacos.security;
import org.springframework.security.crypto.password.PasswordEncoder;
import lombok.Data;
import tacos.User;

@Data
public class RegistrationForm {

    private String username;
    private String password;
    private String fullname;
    private String street;
    private String city;
    private String state;
    private String zip;
    private String phone;

    public User toUser(PasswordEncoder passwordEncoder) {
        return new User(
            username, passwordEncoder.encode(password),
            fullname, street, city, state, zip, phone);
    }
}

```

在大多数情况下，**RegistrationForm** 只是一个支持 **Lombok** 的基本类，只有少量属性。但是 **toUser()** 方法使用这些属性创建一个新的 **User** 对象，**processRegistration()** 将使用注入的 **UserRepository** 保存这个对象。

毫无疑问，**RegistrationController** 被注入了一个密码编码器。这与之前声明的 **PasswordEncoder** bean 完全相同。在处理表单提交时，**RegistrationController** 将其传递给 **toUser()** 方法，该方法使用它对密码进行编码，然后将其保存到数据库。通过这种方式，提交的密码以编码的形式写入，用户详细信息服务将能够根据编码的密码进行身份验证。

现在 **Taco Cloud** 应用程序拥有完整的用户注册和身份验证支持。但是如果在此时启动它，您会注意到，如果不是提示您登录，您甚至无法进入注册页面。这是因为，默认情况下，所有请求都需要身份验证。让我们看看 **web** 请求是如何被拦截和保护，以便可以修复这种奇怪的先有鸡还是先有蛋的情况。

## 5.3 保护 web 请求

Taco Cloud 的安全需求应该要求用户在设计 **tacos** 或下订单之前进行身份验证。但是主页、登录页面和注册页面应该对未经身份验证的用户可用。

要配置这些安全规则，需要声明一个 **SecurityFilterChain** bean。下面的方法添加了 **@Bean** 注解，这是一个简化的（不太实用）**SecurityFilterChain** 类：

```
@Bean
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
    return http.build();
}
```

这个 **filterChain()** 方法接受 **HttpSecurity** 对象，充当的生成器，用于 **web** 级别安全配置。一旦安全配置通过 **HttpSecurity** 对象完成设置，调用 **build()** 方法将创建并返回一个 **SecurityFilterChain** 对象。

可以配置 **HttpSecurity** 的属性包括：

- 在允许服务请求之前，需要满足特定的安全条件
- 配置自定义登录页面
- 使用户能够退出应用程序
- 配置跨站请求伪造保护

拦截请求以确保用户拥有适当的权限是配置 **HttpSecurity** 要做的最常见的事情之一。让我们确保 **Taco Cloud** 的客户满足这些要求。

### 5.3.1 保护请求

需要确保 `/design` 和 `/orders` 的请求仅对经过身份验证的用户可用；应该允许所有用户发出所有其他请求。下面的配置就是这样做的：

```
@Bean
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
    return http
        .authorizeRequests()
            .antMatchers("/design", "/orders").hasRole("USER")
            .antMatchers("/", "**").permitAll()

        .and()
        .build();
}
```

对 `authorizeRequests()` 的调用返回一个对象

（`ExpressionUrlAuthorizationConfigurer.ExpressionInterceptUrlRegistry`），可以在该对象上指定 URL 路径和模式以及这些路径的安全需求。在这种情况下，指定两个安全规则：

- 对于 `/design` 和 `/orders` 的请求应该是授予 `ROLEUSER` 权限的用户的请求。传递给 `hasRole()` 的角色上不要包含“`ROLE`”前缀，否则会被 `hasRole()` 认为有相应权限。
- 所有的请求都应该被允许给所有的用户。

这些规则的顺序很重要。首先声明的安全规则优先于后声明的安全规则。如果交换这两个安全规则的顺序，所有请求都将应用 `permitAll()`，那么关于 `/design` 和 `/orders` 请求的规则将不起作用。

`hasRole()` 和 `permitAll()` 方法只是声明请求路径安全需求的两个方法。表 5.1 描述了所有可用的方法。

表 5.1 定义被保护路径的配置方法

| 方法                                      | 作用描述                                    |
|-----------------------------------------|-----------------------------------------|
| <code>access(String)</code>             | 如果 SpEL 表达式的值为 <code>true</code> ，则允许访问 |
| <code>anonymous()</code>                | 默认用户允许访问                                |
| <code>authenticated()</code>            | 认证用户允许访问                                |
| <code>denyAll()</code>                  | 无条件拒绝所有访问                               |
| <code>fullyAuthenticated()</code>       | 如果用户是完全授权的（不是记住用户），则允许访问                |
| <code>hasAnyAuthority(String...)</code> | 如果用户有任意给定的权限，则允许访问                      |
| <code>hasAnyRole(String...)</code>      | 如果用户有任意给定的角色，则允许访问                      |
| <code>hasAuthority(String)</code>       | 如果用户有给定的权限，则允许访问                        |
| <code>hasIpAddress(String)</code>       | 来自给定 IP 地址的请求允许访问                       |
| <code>hasRole(String)</code>            | 如果用户有给定的角色，则允许访问                        |
| <code>not()</code>                      | 拒绝任何其他访问方法                              |
| <code>permitAll()</code>                | 无条件允许访问                                 |
| <code>rememberMe()</code>               | 允许认证了的同时标记了记住我的用户访问                     |



表 5.1 中的大多数方法为请求处理提供了基本的安全规则，但是它们是自我限制的，只支持那些方法定义的安全规则。或者，可以使用 `access()` 方法提供 SpEL 表达式来声明更丰富的安全规则。Spring Security 扩展了 SpEL，包括几个特定于安全性的值和函数，如表 5.2 所示。

表 5.2 Spring Security 对 SpEL 的扩展

| Security 表达式                                                                    | 含义                                            |
|---------------------------------------------------------------------------------|-----------------------------------------------|
| <code>authentication</code>                                                     | 用户认证对象                                        |
| <code>denyAll</code>                                                            | 通常值为 <code>false</code>                       |
| <code>hasAnyAuthority(String... authorities)</code>                             | 如果用户有任何一项授权，则为 <code>true</code>              |
| <code>hasAnyRole(list of roles)</code>                                          | 如果用户有任何给定的角色，则为 <code>true</code>             |
| <code>hasAuthority(String authority)</code>                                     | 如果用户有给定的授权，则为 <code>true</code>               |
| <code>hasPermission(Object target, Object permission)</code>                    | 如果用户有对给定目标的给定的授权，则为 <code>true</code>         |
| <code>hasPermission(Object target, String targetType, Object permission)</code> | 如果用户有对给定目标的给定的授权，则为 <code>true</code>         |
| <code>hasRole(role)</code>                                                      | 如果用户有给定的角色，则为 <code>true</code>               |
| <code>hasIpAddress(IP Address)</code>                                           | 如果请求来自给定 IP 地址，则为 <code>true</code>           |
| <code>isAnonymous()</code>                                                      | 如果用户是默认用户，则为 <code>true</code>                |
| <code>isAuthenticated()</code>                                                  | 如果用户是认证了的，则为 <code>true</code>                |
| <code>isFullyAuthenticated()</code>                                             | 如果用户被完全认证了的（不是使用记住我进行认证），则为 <code>true</code> |
| <code>isRememberMe()</code>                                                     | 如果用户被标记为记住我后认证了，则为 <code>true</code>          |
| <code>permitAll()</code>                                                        | 通常值为 <code>true</code>                        |
| <code>principal</code>                                                          | 用户 principal 对象                               |

表 5.2 中的大多数安全表达式扩展对应于表 5.1 中的类似方法。实际上，使用 `access()` 方法以及 `hasRole()` 和 `permitAll` 表达式，可以按如下方式重写 `SecurityFilterChain` 配置。

程序清单 5.7 使用 Spring 表达式定义认证规则

```
@Bean
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
    return http
        .authorizeRequests()
            .antMatchers("/design", "/orders").access("hasRole('USER')")
            .antMatchers("/", "**").access("permitAll()")

        .and()
        .build();
}
```

乍一看，这似乎没什么大不了的。毕竟，这些表达式只反映了已经对方法调用所做的工作。但是表达式可以灵活得多。例如，假设（出于某种疯狂的原因）只想允许具有 `ROLE_USER` 权限的用户在周二（例如，在周二）创建新的 Taco；您可以重写 `SecurityFilterChain` bean 方法如下：

```

@Bean
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
    return http
        .authorizeRequests()
        .antMatchers("/design", "/orders")
        .access("hasRole('USER') && " +
            "T(java.util.Calendar).getInstance().get("+
            "T(java.util.Calendar).DAY_OF_WEEK) == " +
            "T(java.util.Calendar).TUESDAY")
        .antMatchers("/", "**").access("permitAll")

        .and()
        .build();
}

```

使用基于 SpEL 的安全约束，这种可能性实际上是无限的。我敢打赌，您已经在构思基于 SpEL 的有趣的安全约束了。

只需使用 `access()` 和程序清单 5.9 中的 SpEL 表达式，就可以满足 Taco Cloud 应用程序的授权需求。现在，让我们来看看如何定制登录页面来适应 Taco Cloud 应用程序的外观。

### 5.3.2 创建自定义登录页面

默认登录页面比您开始使用的笨重的 **HTTP basic** 对话框要好得多，但它仍然相当简单，与 **Taco Cloud** 中其他部分的外观风格不一致。要替换内置登录页面，首先需要告诉 **Spring Security** 您的登录页面路径。这可以通过调用 **HttpSecurity** 对象上的 `formLogin()` 来完成：

```
@Bean
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
    return http
        .authorizeRequests()
            .antMatchers("/design", "/orders").access("hasRole('USER')")
            .antMatchers("/", "**").access("permitAll()")

        .and()
            .formLogin()
                .loginPage("/login")

        .and()
            .build();
}
```

请注意，在调用 `formLogin()` 之前，需将此部分配置与前一部分配置连接起来，调用了 `and()`。`and()` 方法表示您已经完成了授权配置，并准备应用一些额外的 **HTTP** 配置。每当开始新的一部分配置时，就可以使用 `and()`。

连接完成后，调用 `formLogin()` 开始配置自定义登录表单。之后的 `loginPage()` 指定自定义登录页的路径。当 **Spring Security** 确定用户未经过身份验证且需要登录时，它将重定向到此路径上。

现在您需要提供一个控制器来处理该路径上的请求。因为您的登录页面只是一个简单的页面，可以简单的在 **WebConfig** 中声明它是一个视图。下面的 `addViewController()` 方法，设置登录页视图控制器，并将路径 `/` 映射到主控制器上：

```
@Override
public void addViewControllers(ViewControllerRegistry registry) {
    registry.addViewController("/").setViewName("home");
    registry.addViewController("/login");
}
```

最后，您需要定义登录页面本身。因为您用的是 **Thymeleaf** 模板引擎，以下 **Thymeleaf** 模板应该可以正常工作：

```
<!DOCTYPE html>
<html xmlns="http://www.w3.org/1999/xhtml"
      xmlns:th="http://www.thymeleaf.org">
  <head>
    <title>Taco Cloud</title>
  </head>

  <body>
    <h1>Login</h1>
    

    <div th:if="${error}">
      Unable to login. Check your username and password.
    </div>

    <p>New here? Click
      <a th:href="@{/register}">here</a> to register.</p>

    <form method="POST" th:action="@{/login}" id="loginForm">
      <label for="username">Username: </label>
      <input type="text" name="username" id="username" /><br/>

      <label for="password">Password: </label>
      <input type="password" name="password" id="password" /><br/>

      <input type="submit" value="Login"/>
    </form>
  </body>
</html>
```

关于这个登录页面，需要注意的是提交表单请求的路径，以及用户名和密码字段的名称。默认情况下，**Spring Security** 侦听路径 `/login` 的登录请求，用户名和密码字段应命名为 `username` 和 `password`。然而这是可配置的。例如，以下配置自定义了路径和字段名：

```
.and()
  .formLogin()
    .loginPage("/login")
    .loginProcessingUrl("/authenticate")
    .usernameParameter("user")
    .passwordParameter("pwd")
```

这里，您指定 **Spring Security** 应该侦听 `/authenticate` 路径的请求以验证登录。此外，用户名和密码字段现在应命名为 `user` 和 `pwd`。

默认情况下，当 **Spring Security** 确定需要登录时，一旦成功登录就会将用户直接导航到他们浏览的页面。如果用户直接访问登录页面，成功的登录会将它们带到根路径（例如：主页）。但您可以指定默认的成功页面：

```
.and()
  .formLogin()
    .loginPage("/login")
    .defaultSuccessUrl("/design")
```

按照这里的配置，如果用户直接访问登录页面后登录成功，他们将被引导到 `/design` 页面。

可选择的，您可以强制用户登录后进入 `/design` 页面，即使他们在登录之前，正在访问的其他页面。这通过将 `true` 作为第二个参数传递给 `defaultSuccessUrl`：

```
.and()  
.formLogin()  
.loginPage("/login")  
.defaultSuccessUrl("/design", true)
```

在 **web** 中进行身份验证的最常见方式就是使用用户名和密码进行登录。但是，让我们看一看其他方法，使用第三方登录页面来验证用户身份。

### 5.3.3 启用第三方验证

您可能在您一些网站上看到过“使用 Facebook 登录”、“使用 Twitter 登录”这样的链接或按钮。这种方式不要求用户在特定网站上登录，而是提供了一种通过另一个网站（如 Facebook）进行登录的方式，并且他们可能已经登录过那个网站了。

这种类型的身份验证基于 OAuth2 或 OpenID Connect（OIDC）。OAuth2 是一种授权规范，第 9 章，我们将进一步讨论如何使用它来保护 REST API，这里我们讨论用它实现第三方网站身份验证。OpenID Connect 是另一个基于 OAuth2 的安全规范，在第三方身份验证期间，用于将交互规范化。

为了在 Spring 应用程序中使用这种类型的身份验证，您需要添加 OAuth2 starter 客户端：

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-oauth2-client</artifactId>
</dependency>
```

然后，您至少需要配置一个或多个 OAuth2 或 OpenID 的身份验证服务器详细信息。Spring Security 支持使用 Facebook、Google、GitHub 和 Okta，您可以配置一些扩展属性，以支持其他客户端。

要使您的应用程序充当 OAuth2/OpenID 的客户端，一般需要设置的属性集如下：

```
spring:
  security:
    oauth2:
      client:
        registration:
          <oauth2 or openid provider name>:
            clientId: <client id>
            clientSecret: <client secret>
            scope: <comma-separated list of requested scopes>
```

例如，假设对于 Taco Cloud，我们希望用户能够使用 Facebook 登录。application.yml 中的以下配置将设置 OAuth2 客户端：

```
spring:
  security:
    oauth2:
      client:
        registration:
          facebook:
            clientId: <facebook client id>
            clientSecret: <facebook client secret>
            scope: email, public_profile
```

Client ID 和 secret 是识别 Facebook 应用程序的凭据。您可以通过在 <https://developers.facebook.com/> 中创建新的应用程序条目来获取。scope 属性指定应用程序将被授予的访问权限。在这种情况下，应用程序将可以访问用户的电子邮件地址，以及从他们的公共 Facebook 个人资料中获取基本信息。

在一个非常简单的应用程序中，这就是您所需要的。当用户试图访问某个页面需要身份验证时，他们的浏览器将重定向到 Facebook。如果他们还没有登录 Facebook，他们将看到 Facebook 登录页面。登录 Facebook 后，他们将被要求对您的应用程序进行授权，并授予所请求范围的权限。最后，他们身份验证完成，会重定向回您的应用程序。

但是，如果您已经通过声明 `SecurityFilterChainbean` 自定义了安全配置，那么您启用 OAuth2 登录验证的同时，也要添加以下安全配置：

```
@Bean
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
    return http
        .authorizeRequests()
            .mvcMatchers("/design", "/orders").hasRole("USER")
            .anyRequest().permitAll()
        .and()
            .formLogin()
                .loginPage("/login")
        .and()
            .oauth2Login()

        ...

        .and()
            .build();
}
```

您可能还希望同时提供传统用户名/密码登录和第三方登录。在这种情况下，您可以在配置中指定登录页面，如下所示：

```
.and()
    .oauth2Login()
        .loginPage("/login")
```

这将导致应用程序始终将用户带到应用程序提供的登录页面，他们可以像往常一样选择使用用户名和密码登录。但您也可以在同一个登录页面上提供一个链接，让他们有机会登录 **Facebook**。在登录页面的 HTML 模板中，这样的链接可能像如下这样：

```
<a th:href="/oauth2/authorization/facebook">Sign in with Facebook</a>
```

现在，您已经处理了登录问题，让我们转到身份验证问题的另一面，了解如何启用用户注销。

### 5.3.4 登出

与登录应用程序同样重要的是登出。要启用登出功能，只需调用 `HttpSecurity` 对象上的 `logout`:

```
.and()  
.logout()
```

这将设置一个安全筛选器来拦截发送到 `/logout` 的请求。因此，要提供登出功能，只需在应用程序的视图中添加登出表单和按钮：

```
<form method="POST" th:action="@{/logout}">  
  <input type="submit" value="Logout"/>  
</form>
```

当用户单击按钮时，他们的 `session` 将被清除，他们将退出应用程序。默认情况下，它们将被重定向到登录页面，在那里它们可以再次登录。但是，如果希望它们被发送到另一个页面，可以调用 `logoutSuccessFilter()` 来指定一个不同的登出后的登录页面：

```
.and()  
.logout()  
.logoutSuccessUrl("/")
```

在这个例子中，用户在登出后将被跳转到主页。



### 5.3.5 阻止跨站请求伪造攻击

跨站请求伪造（CSRF）是一种常见的安全攻击。它涉及到让用户在一个恶意设计的 **web** 页面上编写代码，这个页面会自动（通常是秘密地）代表经常遭受攻击的用户向另一个应用程序提交一个表单。例如，在攻击者的网站上，可能会向用户显示一个表单，该表单会自动向用户银行网站上的一个 **URL** 发送消息（该网站的设计可能很糟糕，很容易受到这种攻击），以转移资金。用户甚至可能不知道攻击发生了，直到他们注意到他们的帐户中少了钱。

为了防止此类攻击，应用程序可以在显示表单时生成 **CSRF token**，将该 **token** 放在隐藏字段中，然后将其存储在服务器上供以后使用。提交表单时，**token** 将与其他表单数据一起发送回服务器。然后服务器拦截请求，并与最初生成的 **token** 进行比较。如果 **token** 匹配，则允许继续执行请求。否则，表单一定是由一个不知道服务器生成的 **token** 的恶意网站呈现的。

幸运的是，**Spring Security** 有内置的 **CSRF** 保护。更幸运的是，它是默认启用的，不需要显式地配置它。只需确保应用程序提交的任何表单都包含一个名为 `_csrf` 的字段，该字段包含 **CSRF token**。

**Spring Security** 甚至可以通过将 **CSRF token** 放在名为 `_csrf` 的请求属性中来简化这一过程。因此，可以使用以下代码，在 **Thymeleaf** 模板的一个隐藏字段中呈现 **CSRF token**：

```
<input type="hidden" name="_csrf" th:value="${_csrf.token}"/>
```

如果使用 **Spring MVC** 的 **JSP** 标签库或带有 **Spring** 安全方言的 **Thymeleaf**，那么甚至不需要显式地包含一个隐藏字段，隐藏字段将自动呈现。

在 **Thymeleaf** 中，只需确保 `<form>` 元素的一个属性被前缀为 **Thymeleaf** 属性。因为让 **Thymeleaf** 将路径呈现为上下文相关是很常见的，所以这通常不是问题。例如，**Thymeleaf** 渲染隐藏字段所需要的仅仅是 `th:action` 属性：

```
<form method="POST" th:action="@{/login}" id="loginForm">
```

当然也可以禁用 **CSRF** 支持，但我不太愿意展示如何禁用。**CSRF** 保护很重要，而且在表单中很容易处理，所以没有理由禁用它，但如果你坚持禁用它，你可以这样调用 `disable()`：

```
.and()  
  .csrf()  
    .disable()
```

我再次提醒你不要禁用 **CSRF** 保护，特别是对于生产环境中的应用程序。

所有 **web** 层安全性现在都配置到 **Taco Cloud** 了。除此之外，现在有了一个自定义登录页面，并且能够根据 **JPA** 支持的用户存储库对用户进行身份验证。现在让我们看看如何获取有关登录用户的信息。

## 5.4 启用方法级别保护

虽然很容易考虑 **web** 请求级别的安全性，但这并不总是应用安全约束的最好方式。有时最好在用户执行操作时，再验证用户是否经过身份验证，并且具有足够的权限。

例如，假设出于管理目的，有一个服务类包含用于从数据库中清除所有订单的方法。使用注入的 **OrderRepository** 方法来实现，这可能看起来有点像下面这样：

```
public void deleteAllOrders() {
    orderRepository.deleteAll();
}
```

现在，假设有一个 **POST** 请求控制器调用 `deleteAllOrders()` 方法：

```
@Controller
@RequestMapping("/admin")
public class AdminController {

    private OrderAdminService adminService;

    public AdminController(OrderAdminService adminService) {
        this.adminService = adminService;
    }

    @PostMapping("/deleteOrders")
    public String deleteAllOrders() {
        adminService.deleteAllOrders();
        return "redirect:/admin";
    }

}
```

这很容易调整 **SecurityConfig**，以确保只允许授权的用户执行该 **POST** 请求，请执行以下操作：

```
.authorizeRequests()
...
.antMatchers(HttpMethod.POST, "/admin/**")
    .access("hasRole('ADMIN')")
....
```

这很好，可以防止任何未经授权的用户向“/admin/deleteOrders”发送 **POST** 请求，以防止所有订单从数据库中消失。

但是假设其他一些控制器方法也调用 `deleteAllOrders()`。您需要添加更多匹配器，以保护其他需要保护的控制器请求。

相反，我们可以直接在 `deleteAllOrders()` 方法上应用安全性配置，如下所示：

```
`@PreAuthorize` ("hasRole('ADMIN')")
public void deleteAllOrders() {
    orderRepository.deleteAll();
}
```

`@PreAuthorize` 注释采用 SpEL 表达式，如果表达式的计算结果为 **false**，将不会调用该方法。另一方面，如果表达式的计算结果为 **true**，那么这个方法就被允许了。在这种情况下，`@PreAuthorize` 检查用户是否有 `ROLE_ADMIN` 权限。如果是，则将调用该方法并删除所有订单。否则，它将停止执行。

如果 `@PreAuthorize` 阻止调用，则 Spring Security 将抛出 `AccessDeniedException`。这是一个未检查的异常，因此您不需要捕获它，除非您希望应用一些自定义行为。如果未捕获这个异常，它将向上抛出，最终被 Spring Security 的过滤器捕获并处理。或者使用 HTTP 403 页面，或者重定向到登录页面（如果用户正在登录）。

为了使 `@PreAuthorize` 工作，您需要启用全局方法安全。为此，您需要使用 `@EnableGlobalMethodSecurity` 注解安全配置类：

```
@Configuration
@EnableGlobalMethodSecurity
public class SecurityConfig extends WebSecurityConfigurerAdapter {
    ...
}
```

您会发现 `@PreAuthorize` 对于大多数方法级安全需求来说是一个有用的注解。但是要知道，还有一个不那么有用的 `@PostAuthorize` 注解。这个 `@PostAuthorize` 注解的工作原理与 `@PreAuthorize` 注解几乎相同，只是在调用并返回目标方法之前，不会计算其表达式的值。这允许表达式考虑该方法的返回值来决定是否允许方法执行。调例如，假设有一个方法通过其 ID 获取订单，但您希望限制它的使用，管理员或订单所属的用户除外。你可以用 `@PostAuthorize` 方式：

```
@PostAuthorize("hasRole('ADMIN') || " +
    "returnObject.user.username == authentication.name")
public TacoOrder getOrder(long id) {
    ...
}
```

在本例中，`TacoOrder` 中的 `returnObject` 从方法返回。如果用户属性的用户名等于身份验证的 `name` 属性，则允许。但是，为了知道这一点，需要先执行该方法，以便它能够返回 `TacoOrder` 对象。

但是等等！如果应用的条件不满足，由于安全性依赖于方法调用的返回值，那如何确保方法不被调用呢？这个先有鸡，还是先有蛋的难题，通过先允许方法调用，然后在表达式返回 **false** 时抛出 `AccessDeniedException` 来解决。

## 5.5 了解您的用户

通常，仅仅知道用户已经登录是不够的。通常重要的是要知道他们是谁，这样才能调整他们的体验。

例如，在 `OrderController` 中，当最初创建绑定到订单表单的 `TacoOrder` 时，如果能够用用户名和地址预先填充 `TacoOrder` 就更好了，这样他们就不必为每个订单重新输入它。也许更重要的是，在保存订单时，应该将 `TacoOrder` 与创建订单的用户关联起来。

为了在 `TacoOrder` 实体和 `User` 实体之间实现所需的连接，需要向 `TacoOrder` 类添加一个新属性：

```
@Data
@Entity
@Table(name="Taco_Order")
public class TacoOrder implements Serializable {

    ...

    @ManyToOne
    private User user;

    ...
}
```

此属性上的 `@ManyToOne` 注解表明一个订单属于单个用户，相反，一个用户可能有多个订单。（因为使用的是 `Lombok`，所以不需要显式地定义属性的访问方法。）

在 `OrderController` 中，`processOrder()` 方法负责保存订单。需要对其进行修改，以确定经过身份验证的用户是谁，并调用 `Order` 对象上的 `setUser()` 以将 `Order` 与该用户连接起来。

有几种方法可以确定用户是谁。以下是一些最常见的方法：

- 将 `java.security.Principal` 注入控制器方法
- 将 `org.springframework.security.core.Authentication` 对象注入控制器方法
- 使用 `org.springframework.security.core.context.SecurityContextHolder` 得到 `SecurityContext`
- 使用 `@AuthenticationPrincipal` 注解的方法参数（`@AuthenticationPrincipal` 在 `Spring Security` 的 `org.springframework.security.core.annotation` 名中）

例如，可以修改 `processOrder()` 来接受 `java.security.Principal` 类型的参数。然后可以使用从 `UserRepository` 根据规则名查找用户：

```
@PostMapping
public String processOrder(@Valid TacoOrder order, Errors errors,
    SessionStatus sessionStatus,
    Principal principal) {

    ...

    User user = userRepository.findByUsername(
        principal.getName());
    order.setUser(user);

    ...
}
```

这可以很好地工作，但是它会将与安全性无关的代码与安全代码一起丢弃。可以通过修改 `processOrder()` 来减少一些特定于安全的代码，以接受 **Authentication** 对象作为参数而不是 **Principal**：

```
@PostMapping
public String processOrder(@Valid TacoOrder order, Errors errors,
    SessionStatus sessionStatus,
    Authentication authentication) {

    ...

    User user = (User) authentication.getPrincipal();
    order.setUser(user);

    ...

}
```

有了身份验证，可以调用 `getPrincipal()` 来获取主体对象，在本例中，该对象是一个用户。注意，`getPrincipal()` 返回一个 `java.util.Object`，因此需要将其转换为 **User**。

然而，也许最干净的解决方案是简单地接受 `processOrder()` 中的用户对象，但是使用 `@AuthenticationPrincipal` 对其进行注解，以便它成为身份验证的主体：

```
@PostMapping
public String processOrder(@Valid TacoOrder order, Errors errors,
    SessionStatus sessionStatus,
    @AuthenticationPrincipal User user) {

    if (errors.hasErrors()) {
        return "orderForm";
    }

    order.setUser(user);

    orderRepo.save(order);
    sessionStatus.setComplete();

    return "redirect:/";
}
```

`@AuthenticationPrincipal` 的优点在于它不需要强制转换（与身份验证一样），并且将特定于安全性的代码限制为注释本身。当在 `processOrder()` 中获得 **User** 对象时，它已经准备好被使用并分配给订单了。

还有一种方法可以识别通过身份验证的用户是谁，尽管这种方法有点麻烦，因为它包含了大量与安全相关的代码。你可以从安全上下文获取一个认证对象，然后像这样请求它的主体：

```
Authentication authentication =
    SecurityContextHolder.getContext().getAuthentication();
User user = (User) authentication.getPrincipal();
```

尽管这个代码段充满了与安全相关的代码，但是它与所描述的其他方法相比有一个优点：它可以在应用程序的任何地方使用，而不仅仅是在控制器的处理程序方法中，这使得它适合在较低级别的代码中使用。

## 5.6 小结

- Spring Security 自动配置是一种很好的开始学习安全的方式，但大多数应用程序需要明确地配置安全，以满足其独特的安全需求。
- 用户细节可以在关系数据库、LDAP 或完全自定义实现支持的用户存储中进行管理。
- Spring Security 自动防御 CSRF 攻击。
- 通过 `SecurityContext` 对象（从 `SecurityContextHolder.getContext()` 中返回）或使用 `@AuthenticationPrincipal` 注入控制器中，可以获得认证用户的信息。

## 第 6 章 使用配置属性

本章内容：

- 微调自动配置 bean
- 将配置属性应用于应用程序组件
- 使用 Spring 配置文件

您还记得 iPhone 第一次出现的时候吗？一块由金属和玻璃制成的小平板几乎不符合人们对电话的认知。然而，它开创了现代智能手机时代，改变了我们交流的一切方式。尽管在很多方面，触控手机都比上一代的翻盖手机更简单、功能更强大，但在 iPhone 第一次发布时，很难想象一个只有一个按钮的设备怎么能用来打电话。

在某些方面，Spring Boot 自动配置是这样的。自动配置大大简化了 Spring 应用程序的开发。但是，在使用 Spring XML 配置中设置属性值和调用 bean 实例上 setter 方法十年之后，如何在没有显式配置的 bean 上设置属性并不是很明显。

幸运的是，Spring Boot 提供了一种配置属性的方法。配置属性不过是 Spring 应用程序上下文中 `@ConfigurationProperties` bean 上的属性。可以从几个属性源（包括 JVM 系统属性、命令行参数和环境变量）之一进行设置。我们将在第 6.2 节中看到，如何在我们自己的 bean 上使用 `@ConfigurationProperties`。但是 Spring Boot 本身提供了几个 `@ConfigurationProperties` 注解的 bean，我们先配置它们。

在本章中，将从实现 Taco Cloud 应用程序中的新功能中后退一步，以研究配置属性。在接下来的章节中，您学到的东西无疑会对您以后的学习很有帮助。我们将首先了解如何使用配置属性来微调 Spring Boot 自动配置的内容。

## 6.1 微调自动配置

在我们深入研究配置属性之前，有必要确定在 Spring 中有两种不同（但相关）的配置：

- **Bean wiring** —— 它声明应用程序组件将在 Spring 应用程序上下文中作为 bean 创建，以及它们应该如何相互注入。
- **Property injection** —— 在 Spring 应用程序上下文中设置 bean 的值。

在 Spring 的 XML 和基于 Java 的配置中，这两种类型的配置通常在同一个地方显式地声明。在 Java 配置中，`@Bean` 注解的方法可能实例化一个 bean，然后设置其属性的值。例如，考虑下面的 `@Bean` 方法，它为嵌入式 H2 数据库声明了一个数据源：

```
@Bean
public DataSource dataSource() {
    return new EmbeddedDatabaseBuilder()
        .setType(H2)
        .addScript("taco_schema.sql")
        .addScripts("user_data.sql", "ingredient_data.sql")
        .build();
}
```

这里的 `addScript()` 和 `addScripts()` 方法设置了一些带有 SQL 脚本名称的字符串属性，这些 SQL 脚本应该在数据源准备好后应用到数据库中。如果不使用 Spring Boot，那么这就是配置 `DataSource` bean 的方式，而自动配置使此方法完全没有必要。

如果 H2 依赖项在运行时类路径中可用，那么 Spring Boot 将在 Spring 应用程序上下文中自动创建适当的数据源 bean。bean 应用于 `schema.sql` 和 `data.sql` 脚本的读取。

但是，如果希望将 SQL 脚本命名为其他名称呢？或者，如果需要指定两个以上的 SQL 脚本怎么办？这就是配置属性的用武之地。但是在开始使用配置属性之前，需要了解这些属性的来源。



### 6.1.1 理解 Spring 环境抽象

Spring 环境抽象是任何可配置属性的一站式商店。它抽象了属性的起源，以便需要这些属性的 **bean** 可以从 Spring 本身使用它们。Spring 环境来自几个属性源，包括：

- JVM 系统属性
- 操作系统环境变量
- 命令行参数
- 应用程序属性配置文件

然后，它将这些属性聚合到单一的源中，从这个源中可以注入 Spring **bean**。图 6.1 演示了来自属性源的属性是如何通过 Spring 环境抽象流到 Spring **bean** 中的。

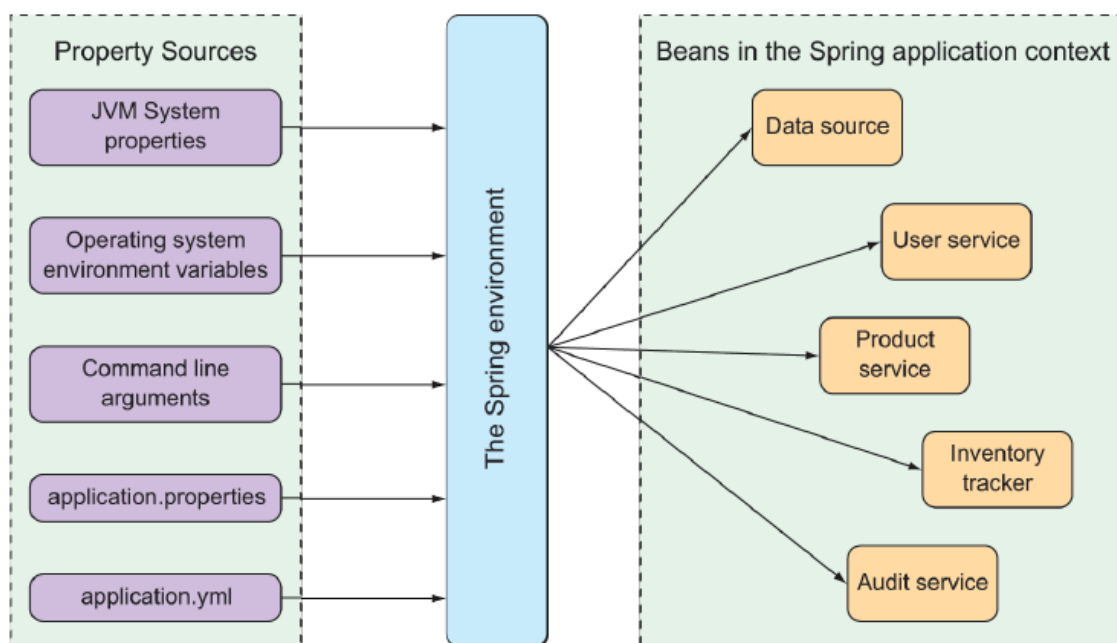


图 6.1 Spring 环境从属性源获取属性，并使它们能够在应用程序上下文中的 **bean** 获取。

通过 Spring Boot 自动配置的 **bean** 都可以通过从 Spring 环境中提取的属性进行配置。作为一个简单的例子，假设希望应用程序的底层 **Servlet** 容器侦听某些端口上的请求，而不是默认端口 8080。为此，通过在 `src/main/resources/application.properties` 文件中的 `server.port` 属性来指定一个不同的接口，如下所示：

```
server.port=9090
```

就我个人而言，我更喜欢在设置配置属性时使用 **YAML**。因此，我可能设置在 `/src/main/resources/application.yml` 文件中的 `server.port` 的值，而不是使用 `application.properties` 文件，如下所示：

```
server:
  port: 9090
```

如果希望在外部分配置该属性，还可以在启动应用程序时使用命令行参数指定端口：

```
$ java -jar tacocloud-0.0.5-SNAPSHOT.jar --server.port=9090
```

如果想让应用程序总是在一个特定的端口上启动，可以把它设置为一个操作系统环境变量：

```
$ export SERVER_PORT=9090
```

注意，在将属性设置为环境变量时，命名风格略有不同，以适应操作系统对环境变量名称的限制。**Spring** 能够将其分类并将 **SERVER\_PORT** 转译为 **server.port**。

正如我所说的，有几种设置配置属性的方法。当我们讲到第 14 章的时候，您会看到在一个集中的配置服务器中设置配置属性的另一种方法。实际上，可以使用几百个配置属性来调整 **Spring bean** 的行为。您已经看到了一些：本章中的 **server.port** 和前一章的 **security.user.name** 和 **security.user.password**。

在本章中不可能测试所有可用的配置属性。尽管如此，让我们来看看一些可能经常遇到的最有用的配置属性。我们将从几个属性开始，这些属性允许您调整自动配置的数据源。

## 6.1.2 配置数据源

此时，Taco Cloud 应用程序仍未完成，但是在准备部署应用程序之前，还有几个章节要处理一些问题。因此，作为数据源使用的嵌入式 H2 数据库非常适合目前为止需要的一切。但是，一旦将应用程序投入生产，可能需要考虑一个更持久的数据库解决方案。

虽然可以显式地配置 `DataSource` bean，但这通常是不必要的。相反，通过配置属性为数据库配置 URL 和凭据更简单。例如，如果打算开始使用 MySQL 数据库，可以将以下配置属性添加到 `application.yml`：

```
spring:
  datasource:
    url: jdbc:mysql://localhost/tacocloud
    username: tacodb
    password: tacopassword
```

虽然需要将适当的 JDBC 驱动程序添加到构建中，但通常不需要指定 JDBC 驱动程序类；Spring Boot 可以从数据库 URL 的结构中找到它。但如果有问题，可以试着设置 `spring.datasource.driver-class-name` 属性：

```
spring:
  datasource:
    url: jdbc:mysql://localhost/tacocloud
    username: tacouser
    password: tacopassword
    driver-class-name: com.mysql.jdbc.Driver
```

Spring Boot 在自动配置数据源 bean 时使用此连接配置。这个数据源 bean 将使用 HikariCP 连接池（如果在服务器上可用）。如果没有，Spring Boot 将查找并使用类路径上某个其他连接池实现：

- Tomcat JDBC 连接池
- Apache Commons DBCP2

虽然这些是通过自动配置提供的唯一连接池选项，但是您随时都可以显式配置数据源 bean，以使用其他连接池。

在本章前面，我们建议有一种方法可以指定，应用程序启动时要运行的数据库初始化脚本。在这种情况下，`spring.datasource.schema` 和 `spring.datasource.data` 属性被证明非常有用：

```
spring:
  datasource:
    schema:
      - order-schema.sql
      - ingredient-schema.sql
      - taco-schema.sql
      - user-schema.sql
    data:
      - ingredients.sql
```

可能显式数据源配置不是您的风格，您更喜欢在 JNDI 中配置数据源，并让 Spring 从那里查找它。在这种情况下，通过配置 `spring.datasource.jndi-name` 来设置数据源：

```
spring:
  datasource:
    jndi-name: java:/comp/env/jdbc/tacoCloudDS
```

如果设置了 `spring.datasource.jndi-name` 属性，那么其他数据源的连接属性（如果设置了）会被忽略。

### 6.1.3 配置嵌入式服务器

已经看到如何通过设置 `server.port` 来设置 `servlet` 容器。还没有让您看到的是，如果把 `server.port` 设置为 0 会发生什么：

```
server:
  port: 0
```

尽管正在显式地设置 `server.port` 为 0，但是服务器不会在端口 0 上启动。相反，它将从随机选择的可用端口启动。这在运行自动化集成测试以确保任何并发运行的测试不会在硬编码端口号上发生冲突时非常有用。在第 13 章中将看到，当不关心应用程序启动于哪个端口时，它也很有用，因为它是一个将从服务注册表中查找的微服务。

但是底层服务器不仅仅是一个端口。需要对底层容器做的最常见的事情之一是将其设置为处理 HTTPS 请求。要做到这一点，您必须做的第一件事是通过使用 JDK 的 `keytool` 命令行工具创建一个密钥存储：

```
$ keytool -keystore mykeys.jks -genkey -alias tomcat -keyalg RSA
```

您将会被问到几个关于您的名字和公司的问题，这些问题大部分都是无关紧要的。但当被要求输入密码时，记住您的密码。对于本例，我选择 `letmein` 作为密码。

接下来，需要设置一些属性，用于在嵌入式服务器中启用 HTTPS。可以在命令行中指定它们，但是那样会非常不方便。相反，可能会在 `application.properties` 或 `application.yml` 文件中设置它们。在 `application.yml` 中，属性可能是这样的：

```
server:
  port: 8443
  ssl:
    key-store: file:///path/to/mykeys.jks
    key-store-password: letmein
    key-password: letmein
```

在这里 `server.port` 属性设置为 8443，这是开发 HTTPS 服务器的常用选择。`server.ssl.key-store` 属性设置为创建密钥存储库文件的路径。这里显示了一个 `file://` URL 来从文件系统加载它，但是如果将它打包到应用程序 JAR 文件中，将使用一个 `classpath:` URL 来引用它。同时 `server.ssl.key-store-password` 和 `server.ssl.key-password` 属性都被设置为创建密钥存储时指定的密码值。

有了这些属性，应用程序应该侦听端口 8443 上的 HTTPS 请求。根据使用的浏览器，可能会遇到服务器无法验证其身份的警告。在开发期间从本地主机提供服务时，这没有什么可担心的。

## 6.1.4 配置日志

大多数应用程序都提供某种形式的日志记录。即使应用程序没有直接记录任何内容，应用程序使用的库也肯定会记录它们的活动。

默认情况下，Spring Boot 通过 Logback (<http://logback.qos.ch>) 配置日志，默认为 INFO 级别，然后写入控制台。在运行应用程序和其他示例时，可能已经在应用程序日志中看到了大量的 INFO 级别的日志条目。作为提醒，这里有一个默认日志格式的日志记录示例（为适合页面边距进行了换行）：

```
2021-07-29 17:24:24.187 INFO 52240 --- [nio-8080-exec-1] com.example.demo.Hello
    Here's a log entry.
2021-07-29 17:24:24.187 INFO 52240 --- [nio-8080-exec-1] com.example.demo.Hello
    Here's another log entry.
2021-07-29 17:24:24.187 INFO 52240 --- [nio-8080-exec-1] com.example.demo.Hello
    And here's one more.
```

要完全控制日志配置，可以在类路径的根目录（在 `src/main/resources` 中）创建 `log.xml` 文件。下面是一个简单的 `logback.xml` 文件的例子：

```
<configuration>
  <appender name="STDOUT" class="ch.qos.logback.core.ConsoleAppender">
    <encoder>
      <pattern>
        %d{HH:mm:ss.SSS} [%thread] %-5level %logger{36} - %msg%n
      </pattern>
    </encoder>
  </appender>
  <logger name="root" level="INFO"/>
  <root level="INFO">
    <appender-ref ref="STDOUT" />
  </root>
</configuration>
```

使用此新配置，以前的相同示例日志条目可能如下所示（为适合页面边距进行了换行）：

```
17:25:09.088 [http-nio-8080-exec-1] INFO com.example.demo.Hello
- Here's a log entry.
17:25:09.088 [http-nio-8080-exec-1] INFO com.example.demo.Hello
- Here's another log entry.
17:25:09.088 [http-nio-8080-exec-1] INFO com.example.demo.Hello
- And here's one more.
```

除了用于日志的模式外，Logback 配置或多或少与没有 `logback.xml` 文件时得到的默认配置相同。但是通过编辑 `logback.xml`，可以完全控制应用程序的日志文件。

**注意：** `logback.xml` 中包含的具体内容超出了本书的范围。有关更多信息，请参阅 Logback 的文档。

对日志配置最常见的更改是更改日志级别，可能还会指定应该写入日志的文件。使用 Spring Boot 配置属性，可以在不创建 `logback.xml` 文件的情况下进行这些更改。

要设置日志记录级别，需要创建以 `logging.level` 为前缀的属性，后面接上要为其设置日志级别的日志记录器的名称。例如，假设将 `root` 日志级别设置为 `WARN`，但是将 `Spring` 安全日志设置为 `DEBUG` 级别。可以像下面这样设置：

```
logging:
  level:
    root: WARN
    org:
      springframework:
        security: DEBUG
```

另外，可以将 **Spring Security** 包的名称折叠成一行，以便于阅读：

```
logging:
  level:
    root: WARN
    org.springframework.security: DEBUG
```

现在，假设希望将日志条目写入位于 `/var/logs/` 文件夹下的 `TacoCloud.log` 文件。`logging.file.path` 和 `logging.file.name` 属性可以帮助实现这一点：

```
logging:
  file:
    path: /var/logs/
    name: TacoCloud.log
  level:
    root: WARN
    org:
      springframework:
        security: DEBUG
```

假设应用程序对 `/var/logs/` 文件夹有写权限，那么日志将被写到 `/var/logs/TacoCloud.log` 文件中。默认情况下，日志文件在大小达到 **10 MB** 时就会进行循环写入。

### 6.1.5 使用特殊的属性值

在设置属性时，不限于将它们值声明为硬编码的字符串和数值。相反，可以从其他配置属性派生它们的值。

例如，假设（不管出于什么原因）想要设置一个名为 `greeting.welcome` 的属性，用于返回另一个名为 `spring.application.name` 的属性的值。为此，在设置 `greeting.welcome` 时可以使用 `${}` 占位符标记：

```
greeting:
  welcome: ${spring.application.name}
```

您甚至可以把这个占位符嵌入到其他文本中：

```
greeting:
  welcome: You are using ${spring.application.name}
```

正如您所看到的，使用配置属性配置 Spring 自己的组件可以很容易地将值注入这些组件的属性并调整自动配置。配置属性并不专属于 Spring 创建的 bean。只需稍加努力，就可以利用您自己的 bean 中的配置属性。接下来让我们来看看怎么做。



## 6.2 创建自己的配置属性

正如前面提到的，配置属性只不过是用来接受 Spring 环境抽象配置的 bean 的属性。没有提到的是如何指定这些 bean 来使用这些配置。

为了支持配置属性的属性注入，Spring Boot 提供了 `@ConfigurationProperties` 注释。当放置在任何 Spring bean 上时，它指定可以从 Spring 环境中的属性注入到该 bean 的属性。

为了演示 `@ConfigurationProperties` 是如何工作的，假设已经将以下方法添加到 `OrderController` 中，以列出经过身份验证的用户之前的订单：

```
@GetMapping
public String ordersForUser(
    @AuthenticationPrincipal User user, Model model) {

    model.addAttribute("orders",
        orderRepo.findByUserOrderByPlacedAtDesc(user));

    return "orderList";
}
```

除此之外，还需要向 `OrderRepository` 添加了必要的 `findByUserOrderByPlacedAtDesc()` 方法：

```
List<Order> findByUserOrderByPlacedAtDesc(User user);
```

请注意，此存储库方法是用 `OrderByPlacedAtDesc` 子句命名的。`OrderBy` 部分指定一个属性，通过该属性对结果排序——在本例中是 `placedAt` 属性。最后的 `Desc` 让排序按降序进行。因此，返回的订单列表将按时间倒序排序。

如前所述，在用户下了一些订单之后，这个控制器方法可能会很有用。但对于最狂热的 `taco` 鉴赏家来说，它可能会变得有点笨拙。在浏览器中显示的一些命令是有用的；一长串没完没了的订单只是噪音。假设希望将显示的订单数量限制为最近的 20 个订单，可以更改 `ordersForUser()`：

```
@GetMapping
public String ordersForUser(
    @AuthenticationPrincipal User user, Model model) {

    Pageable pageable = PageRequest.of(0, 20);
    model.addAttribute("orders",
        orderRepo.findByUserOrderByPlacedAtDesc(user, pageable));

    return "orderList";
}
```

随着这个改变，`OrderRepository` 跟着需要变为：

```
List<TacoOrder> findByUserOrderByPlacedAtDesc(
    User user, Pageable pageable);
```

这里，已经更改了 `findByUserOrderByPlacedAtDesc()` 方法的签名，以接受可分页的参数。可分页是 Spring Data 通过页码和页面大小选择结果子集的方式。在 `ordersForUser()` 控制器方法中，构建了一个 `PageRequest` 对象，该对象实现了 `Pageable` 来请求第一个页面（`page zero`），页面大小为 20，以便为用户获得最多 20 个最近的订单。

虽然这工作得非常好，但它让我感到有点不安，因为已经硬编码了页面大小。如果后来发现 20 个订单太多，而决定将其更改为 10 个订单，该怎么办？因为它是硬编码的，所以必须重新构建和重新部署应用程序。

可以使用自定义配置属性来设置页面大小，而不是硬编码页面大小。首先，需要向 **OrderController** 添加一个名为 **pageSize** 的新属性，然后在 **OrderController** 上使用 `@ConfigurationProperties` 注解，如下面的程序清单所示。

程序清单 6.1 在 **OrderController** 中使用配置属性

```
@Controller
@RequestMapping("/orders")
@SessionAttributes("order")
@ConfigurationProperties(prefix="taco.orders")
public class OrderController {

    private int pageSize = 20;

    public void setPageSize(int pageSize) {
        this.pageSize = pageSize;
    }

    ...

    @GetMapping
    public String ordersForUser(
        @AuthenticationPrincipal User user, Model model) {

        Pageable pageable = PageRequest.of(0, pageSize);
        model.addAttribute("orders",
            orderRepo.findByUserOrderByPlacedAtDesc(user, pageable));
        return "orderList";
    }
}
```

程序清单 6.1 中最重要的变化是增加了 `@ConfigurationProperties` 注解。其 **prefix** 属性设置为 **taco**。这意味着在设置 **pageSize** 属性时，需要使用一个名为 **taco.orders.pageSize** 的配置属性。

新的 **pageSize** 属性默认为 20。但是可以通过设置 **taco.orders.pageSize** 属性轻松地将其更改为想要的任何值。例如，可以在 **application.yml** 中设置此属性：

```
taco:
  orders:
    pageSize: 10
```

或者，如果需要在生产环境中进行快速更改，可以通过设置 **taco.orders.pageSize** 属性作为环境变量来重新构建和重新部署应用程序：

```
$ export TACO_ORDERS_PAGESIZE=10
```

可以设置配置属性的任何方法，都可以用来调整最近订单页面的大小。接下来，我们将研究如何在属性持有者中设置配置数据。

## 6.2.1 定义配置属性持有者

这里没有说 `@ConfigurationProperties` 必须设置在控制器或任何其他特定类型的 `bean` 上，`@ConfigurationProperties` 实际上经常放在 `bean` 上。在应用程序中，这些 `bean` 的惟一目的是作为配置数据的持有者，这使控制器和其他应用程序类不涉及特定于配置的细节，它还使得在几个可能使用该信息的 `bean` 之间共享公共配置属性变得很容易。

对于 `OrderController` 中的 `pageSize` 属性，可以将其提取到一个单独的类中。下面的程序清单以这种方式使用了 `OrderProps` 类。

程序清单 6.2 提取 `pageSize` 到持有者类

```
package tacos.web;
import org.springframework.boot.context.properties.
    ConfigurationProperties;
import org.springframework.stereotype.Component;
import lombok.Data;

@Component
@ConfigurationProperties(prefix="taco.orders")
@Data
public class OrderProps {

    private int pageSize = 20;

}
```

正如在 `OrderController` 中所做的，`pageSize` 属性默认为 20，同时 `OrderProps` 使用 `@ConfigurationProperties` 进行注解，以具有 `taco.orders` 前缀。它还带有 `@Component` 注解，因此 Spring 组件扫描时将自动发现它并在 Spring 应用程序上下文中将其创建为 `bean`。这很重要，因为下一步是将 `OrderProps bean` 注入到 `OrderController` 中。

关于配置属性持有者，没有什么特别的。它们是从 Spring 环境中注入属性的 `bean`。它们可以被注入到任何需要这些属性的其他 `bean` 中。对于 `OrderController`，这意味着从 `OrderController` 中删除 `pageSize` 属性，而不是注入并使用 `OrderProps bean`：

```
private OrderProps props;

public OrderController(OrderRepository orderRepo,
    OrderProps props) {
    this.orderRepo = orderRepo;
    this.props = props;
}

...

@GetMapping
public String ordersForUser(
    @AuthenticationPrincipal User user, Model model) {

    Pageable pageable = PageRequest.of(0, props.getPageSize());
    model.addAttribute("orders",
        orderRepo.findByUserOrderByPlacedAtDesc(user, pageable));

    return "orderList";
}
```

现在 `OrderController` 不再负责处理它自己的配置属性。这使得 `OrderController` 中的代码稍微整洁一些，并允许在任何其他需要它们的 `bean` 中重用 `OrderProps` 中的属性。此外，正在收集与一个地方的订单相关的配置属性：`OrderProps` 类。如果需要添加、删除、重命名或以其他方式更改其中的属性，只需要在 `OrderProps` 中应用这些更改。

例如，假设在其他几个 `bean` 中使用 `pageSize` 属性，这时最好对该属性应用一些验证，以将其值限制为不小于 5 和不大于 25。如果没有持有者 `bean`，将不得不对 `OrderController`、`pageSize` 属性以及使用该属性的所有其他类应用验证注解。但是因为已经将 `pageSize` 提取到 `OrderProps` 中，所以只需要更改 `OrderProps`：

```
package tacos.web;
import javax.validation.constraints.Max;
import javax.validation.constraints.Min;

import org.springframework.boot.context.properties.
    ConfigurationProperties;
import org.springframework.stereotype.Component;
import org.springframework.validation.annotation.Validated;

import lombok.Data;

@Component
@ConfigurationProperties(prefix="taco.orders")
@Data
@Validated
public class OrderProps {

    @Min(value=5, message="must be between 5 and 25")
    @Max(value=25, message="must be between 5 and 25")
    private int pageSize = 20;

}
```

尽管可以很容易地将 `@Validated`、`@Min` 和 `@Max` 注解应用到 `OrderController`（以及可以注入 `OrderProps` 的任何其他 `bean`），但这只会使 `OrderController` 更加混乱。通过使用配置属性持有者 `bean`，就在在一个地方收集了配置属性的细节，使得需要这些属性的类相对干净。

### 6.2.2 声明配置属性元数据

根据 IDE 的情况，你可能已经注意到 `application.yml`（或是 `application.properties`）中的 `taco.orders.pageSize` 属性有一个警告，说类似未知属性 `'taco'` 之类的东西。出现此警告是因为缺少关于刚刚创建的配置属性的元数据。图 6.2 显示了我将鼠标悬停在 Spring Tool Suite 中 `taco` 属性时的效果。

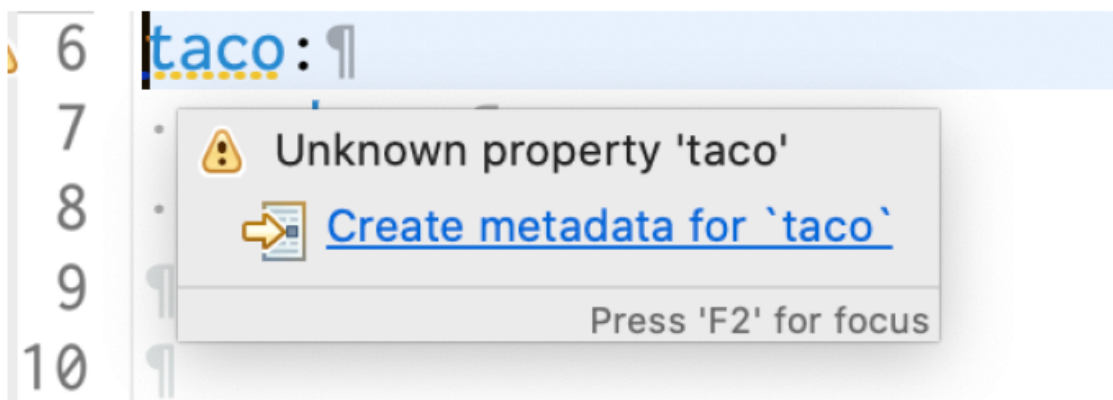


图 6.2 缺少配置属性元数据出现的警告

配置属性元数据是完全可选的，并不会阻止配置属性的工作。但是元数据对于提供有关配置属性的最小文档非常有用，特别是在 IDE 中。

例如，当我将鼠标悬停在 `security.user.password` 属性上时，如图 6.3 所示，虽然悬停帮助你获得的是最小的，但它足以帮助你了解属性的用途以及如何使用它。

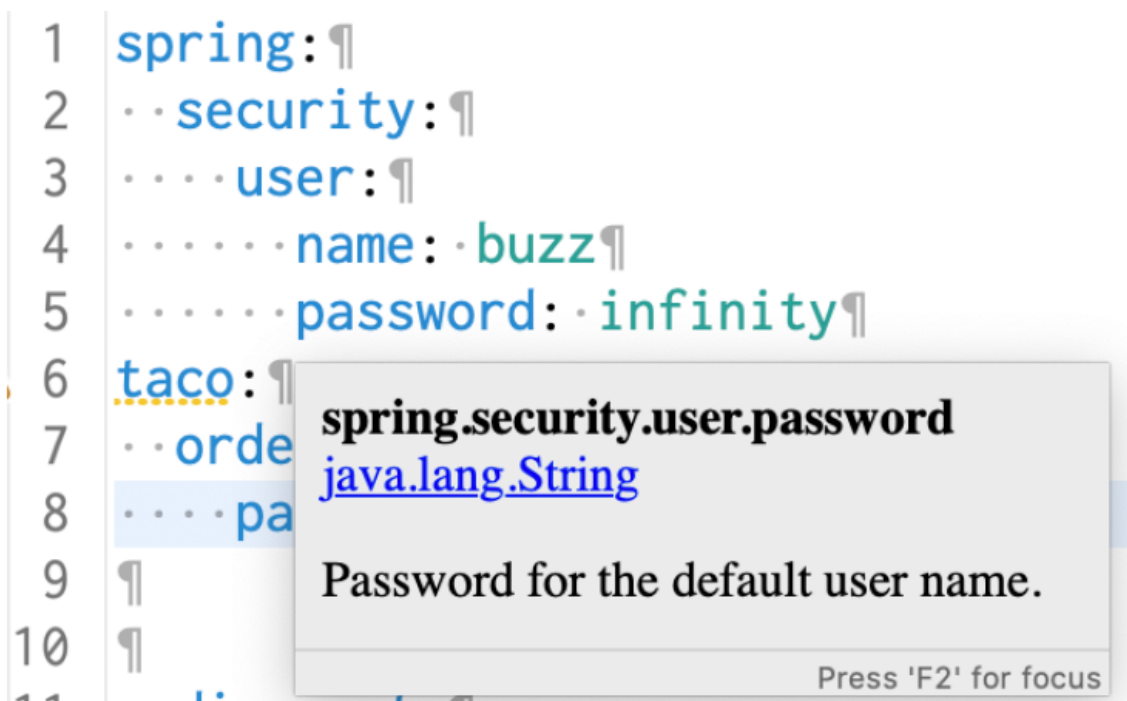


图 6.3 在 Spring Tool Suite 中悬停显示配置属性文档

为了帮助那些可能使用你定义的配置属性（甚至可能是你自己定义的）的人，通常最好是围绕这些属性创建一些元数据，至少它消除了 IDE 中那些恼人的黄色警告。

要为自定义配置属性创建元数据，需要在 META-INF（例如，在项目下的 `src/main/resources/META-INF` 中）中创建一个名为 `addition-spring-configuration-metadata.json` 的文件。

## 快速修复缺失的元数据

如果正在使用 **Spring Tool Suite**，则有一个用于创建丢失的属性元数据的快速修复选项。将光标放在缺少元数据警告的行上，然后按下 Mac 上的 **CMD-1** 或 Windows 和 Linux 上的 **Ctrl-1** 弹出的快速修复（参见图 6.4）。

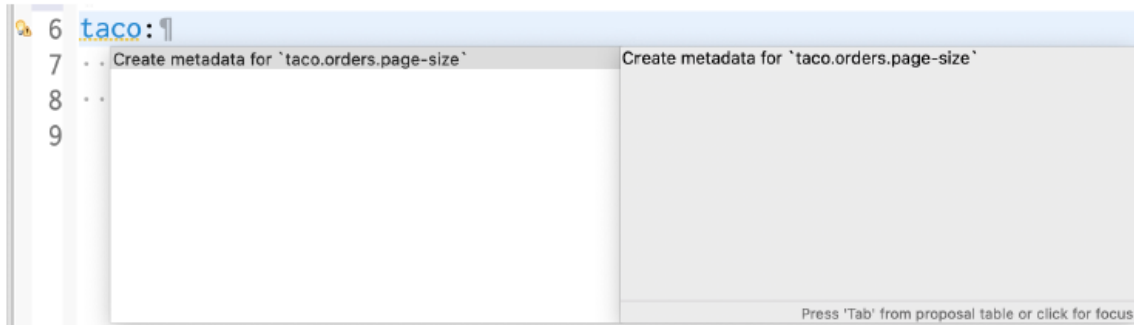


图 6.4 在 **Spring Tool Suite** 中使用快速弹出方式创建配置属性元数据

然后选择 `Create Metadata for...` 选项来为属性添加一些元数据（在 `additional-spring-configuration-metadata` 中）。如果该文件不存在，则创建该文件。

对于 `taco.orders.pageSize` 属性，可以用以下 JSON 设置元数据：

```
{
  "properties": [
    {
      "name": "taco.orders.page-size",
      "type": "java.lang.String",
      "description": "A description for 'taco.orders.page-size'"
    }
  ]
}
```

注意，元数据中引用的属性名是 `taco.orders.page-size`，实际在 `application.yml` 中的属性是 `pageSize`。Spring Boot 灵活的属性命名允许属性名的变化，比如 `taco.orders.page-size` 相当于 `taco.orders.pageSize`，所以使用哪种形式无关紧要。

写入 `additional-spring-configuration-metadata.json` 的初始元数据是一个不错的开始，但您可能需要对其进行一点编辑。首先，`pageSize` 属性不是一个 `String`，因此您需要将其更改为 `java.lang.Integer`。`description` 属性应更改为更能说明 `pageSize` 的用途。以下 JSON 显示了经过几次编辑后元数据的样子：

```
{
  "properties": [
    {
      "name": "taco.orders.page-size",
      "type": "java.lang.Integer",
      "description": "Sets the maximum number of orders to display in a list."
    }
  ]
}
```

有了这些元数据，警告就应该消失了。更重要的是，如果你悬停在 `taco.orders.pageSize` 属性，你将看到如图 6.5 所示的描述。

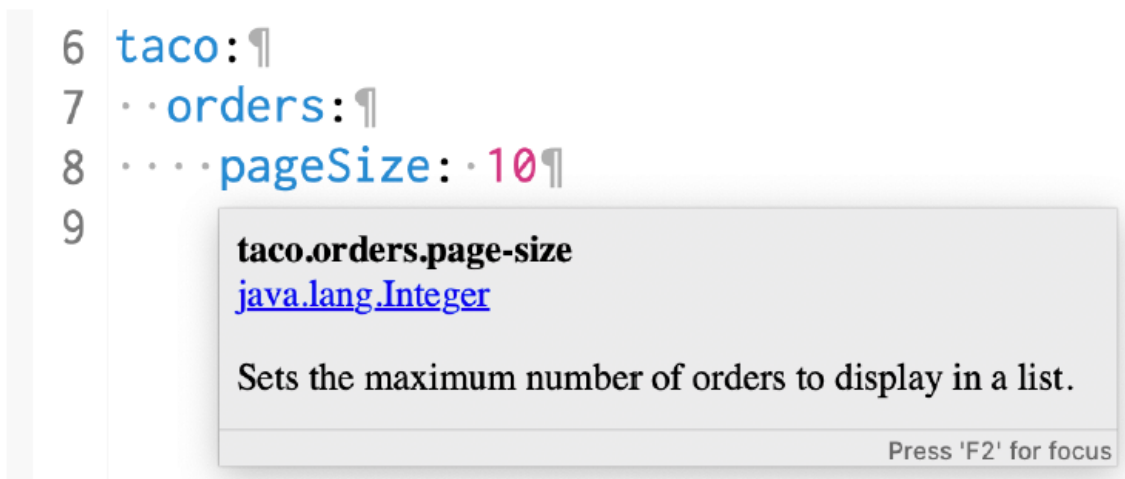


图 6.5 悬停显示自定义配置属性帮助

另外，如图 6.6 所示，可以从 IDE 获得自动完成帮助，就像 Springprovided 的配置属性一样。

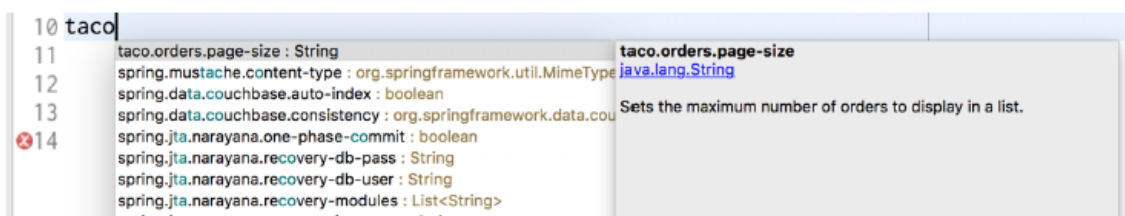


图 6.6 配置属性元数据让属性值自动填充

配置属性对于调整自动配置的组件和注入到应用程序 bean 中的细节非常有用。但是，如果需要为不同的部署环境配置不同的属性呢？让我们看看如何使用 Spring 配置文件来设置特定于环境的配置。

## 6.3 使用 profile 进行配置

当应用程序部署到不同的运行时环境时，通常会有一些配置细节不同。例如，数据库连接的细节在开发环境中可能与在 QA 环境中不一样，在生产环境中可能还不一样。在一个环境中唯一配置属性的一种方法是使用环境变量来指定配置属性，而不是在 `application.properties` 或 `application.yml` 中定义它们。

例如，在开发期间，可以依赖于自动配置的嵌入式 H2 数据库。但在生产中，可以将数据库配置属性设置为环境变量，如下所示：

```
% export SPRING_DATASOURCE_URL=jdbc:mysql://localhost/tacocloud
% export SPRING_DATASOURCE_USERNAME=tacouser
% export SPRING_DATASOURCE_PASSWORD=tacopassword
```

尽管这样做是可行的，但是将一两个以上的配置属性指定为环境变量就会变得有点麻烦。此外，没有跟踪环境变量更改的好方法，也没有在出现错误时轻松回滚更改的好方法。

相反，我更喜欢利用 Spring profile 文件。profile 文件是一种条件配置类型，其中根据运行时激活的 profile 文件应用或忽略不同的 bean、配置类和配置属性。

例如，假设出于开发和调试的目的，希望使用嵌入式 H2 数据库，并且希望将 Taco Cloud 代码的日志级别设置为 DEBUG。但是在生产中，需要使用一个外部 MySQL 数据库，并将日志记录级别设置为 WARN。在开发环境中，很容易不设置任何数据源属性并获得自动配置的 H2 数据库。至于 DEBUG 级别的日志记录，可以在 `application.yml` 中设置 `logging.level.tacos` 属性。

```
logging:
  level:
    tacos: DEBUG
```

这正是开发目的所需要的。但是，如果要将此应用程序部署到生产环境中，而不需要对 `application.yml` 进行进一步更改，仍然可以获得对于 `tacos` 包的调试日志和嵌入式 H2 数据库。需要的是定义一个具有适合生产的属性的 profile 文件。



### 6.3.1 定义特定 profile 的属性

定义特定 **profile** 文件的属性的一种方法是创建另一个仅包含用于生产的属性的 **YAML** 或属性文件。文件的名称应该遵循这个约定：**application-{profile 名称}.yaml** 或 **application-{profile 名称}.properties**。然后可以指定适合该配置文件的配置属性。例如，可以创建一个名为 **application-prod.yaml** 的新文件，包含以下属性：

```
spring:
  datasource:
    url: jdbc:mysql://localhost/tacocloud
    username: tacouser
    password: tacopassword
  logging:
    level:
      tacos: WARN
```

另一种指定特定 **profile** 文件的属性的方法只适用于 **YAML** 配置。它涉及在应用程序中将特定 **profile** 的属性与非 **profile** 的属性一起放在 **application.yaml** 中，由三个连字符分隔。将生产属性应用于 **application.yaml** 时，整个 **application.yaml** 应该是这样的：

```
logging:
  level:
    tacos: DEBUG

---
spring:
  profiles: prod

  datasource:
    url: jdbc:mysql://localhost/tacocloud
    username: tacouser
    password: tacopassword

logging:
  level:
    tacos: WARN
```

这个 **application.yaml** 文件由一组三重连字符（**---**）分成两个部分。第二部分为 **spring.profiles** 指定一个值，这个值指示了随后应用于 **prod** 配置文件的属性。另一方面，第一部分没有为 **spring.profiles** 指定值。因此，它的属性对于所有 **profile** 文件都是通用的，或者如果指定的 **profile** 文件没有设置其他属性，它就是默认的。

无论应用程序运行时哪个配置文件处于活动状态，**tacos** 包的日志级别都将通过默认配置文件中的属性设置为 **DEBUG**。但是，如果名为 **prod** 的配置文件是活动的，那么 **logging.level.tacos** 属性将会被重写为 **WARN**。同样，如果 **prod** 配置文件是活动的，那么数据源属性将设置为使用外部 **MySQL** 数据库。

通过创建使用 **application-{profile 名称}.yaml** 或 **application-{profile 名称}.properties** 这种模式命名的其他 **YAML** 或 **properties** 文件，可以为任意数量的 **profile** 文件定义属性。或者在 **application.yaml** 中再输入三个破折号通过 **spring.profiles** 来指定配置文件名称。然后添加需要的所有特定 **profile** 文件的属性。

### 6.3.2 激活 profile

设置特定 **profile** 属性没有什么意思，除非这些 **profile** 处于活动状态。但是要如何激活一个 **profile** 文件呢？让一个 **profile** 文件处于激活状态需要做的只是将 **spring.profiles.active** 属性的值指定为需要激活的 **profile** 的名称。例如，可以像下面这样设置 **application.yml** 中的这个属性：

```
spring:
  profiles:
    active:
      - prod
```

但是这可能是设定一个活动 **profile** 最糟糕的方式了。如果在 **application.yml** 中设置了激活的 **profile**，然后那个 **profile** 文件就变成了默认 **profile** 文件，那么就没有达到生产环境特定属性与开发环境特定属性分离的目的。相反，我推荐使用环境变量设置激活的 **profile**。在生产环境，像下面这样设置 **SPRING\_PROFILES\_ACTIVE**：

```
% export SPRING_PROFILES_ACTIVE=prod
```

这样设置完成后，部署于那台机器的任何应用程序将会使用 **prod profile**，同时相应的配置属性将优先于默认配置文件中的属性。

如果使用可执行的 **JAR** 文件来运行应用程序，你可能也可以通过命令行设置激活的 **profile** 文件，如下所示：

```
% java -jar taco-cloud.jar --spring.profiles.active=prod
```

请注意 **spring.profiles.active** 属性名包含的是复数单词 **profiles**。这意味着可以指定多个活动 **profiles** 文件。通常，这是一个逗号分隔的列表，当它设置一个环境变量：

```
% export SPRING_PROFILES_ACTIVE=prod,audit,ha
```

但是在 **YAML** 中，需要像下面这样指定它：

```
spring:
  profiles:
    active:
      - prod
      - audit
      - ha
```

同样值得注意的是，如果将 **Spring** 应用程序部署到 **Cloud Foundry** 中，一个名为 **cloud** 的配置文件会自动激活。如果 **Cloud Foundry** 是生产环境，那么需要确保在 **cloud profile** 文件中指定了特定于生产环境的属性。

事实证明，配置文件只对对在 **Spring** 应用程序中有条件地设置配置属性有用。让我们看看如何声明特定活动 **profile** 文件的 **bean**。

### 6.3.3 有条件地使用 profile 创建 bean

有时候，为不同的配置文件提供一组惟一的 bean 是很有用的。通常，不管哪个 profile 文件是活动的，Java 配置类中声明的任何 bean 都会被创建。但是，假设只有在某个配置文件处于活动状态时才需要创建一些 bean，在这种情况下，`@Profile` 注解可以将 bean 指定为只适用于给定的 profile 文件。

例如，在 `TacoCloudApplication` 中声明了一个 `CommandLineRunner` bean，用于在应用程序启动时加载嵌入式数据库中的配料数据。这对于开发来说很好，但是在生产应用程序中是不必要的（也是不受欢迎的）。为了防止在每次应用程序在生产部署中启动时加载配料数据，可以使用 `@Profile` 像下面这样注解

`CommandLineRunner` bean 方法：

```
@Bean
@Profile("dev")
public CommandLineRunner dataLoader(IngredientRepository repo,
    UserRepository userRepo, PasswordEncoder encoder) {

    ...

}
```

或是假设需要在 `dev profile` 或是 `qa profile` 被激活时创建 `CommandLineRunner`，在这种情况下，可以列出需要的 profile：

```
@Bean
@Profile({"dev", "qa"})
public CommandLineRunner dataLoader(IngredientRepository repo,
    UserRepository userRepo, PasswordEncoder encoder) {

    ...

}
```

这样配料数据只会在 `dev` 或是 `qa profile` 文件被激活时才会被加载。这就意味着需要在开发环境运行应用程序时，将 `dev profile` 激活。如果这个 `CommandLineRunner` bean 总是被创建，除非 `prod` 配置文件是活动的，那就更方便了。在这种情况下，您可以像这样应用 `@Profile`：

```
@Bean
@Profile("!prod")
public CommandLineRunner dataLoader(IngredientRepository repo,
    UserRepository userRepo, PasswordEncoder encoder) {

    ...

}
```

在这里，感叹号 `!` 否定了配置文件名称。实际上，它声明如果 `prod` 配置文件不是活动的，就会创建 `CommandLineRunner` bean。

也可以在整个 `@Configuration` 注解的类上使用 `@Profile`。例如，假设要将 `CommandLineRunner` bean 提取到一个名为 `DevelopmentConfig` 的单独配置类中。然后您可以用 `@Profile` 来注解 `DevelopmentConfig`：

```
@Profile({"!prod", "!qa"})
@Configuration
public class DevelopmentConfig {

    @Bean
    public CommandLineRunner dataLoader(IngredientRepository repo,
        UserRepository userRepo, PasswordEncoder encoder) {

        ...

    }

}
```

在这里，**CommandLineRunner** bean（以及在 **DevelopmentConfig** 中定义的任何其他 bean）仅在 **prod** 和 **qa** 配置文件都不活动的情况下才会被创建。

## 6.4 小结

- 可以使用 `@ConfigurationProperties` 注解 **Spring bean**，以支持从几个属性源之一注入值。
- 可以在命令行参数、环境变量、JVM 系统属性、属性文件或 YAML 文件等选项中设置配置属性。
- 配置属性可用于覆盖自动配置设置，包括指定数据源 URL 和日志级别的能力。
- **Spring profile** 文件可以与属性源一起使用，根据活动配置文件有条件地设置配置属性。

## 第 7 章 创建 REST 服务

本章内容：

- 在 Spring MVC 中定义 REST 端点
- 自动生成基于存储库的 REST 端点
- 消费 REST API

“浏览器已死！现在呢？”

大约十多年前，我听到有人说，web 浏览器正接近濒死状态，可能会被其他东西取代。但这是怎么回事呢？什么有可能取代几乎无处不在的 web 浏览器呢？如果不使用 web 浏览器，我们将如何消费越来越多的网站和在线服务？这的确是一个疯子的胡言乱语！

快进到今天，很明显，web 浏览器并没有消失，但它不再是上网的主要方式。移动设备、平板电脑、智能手表和语音设备现在都很常见，甚至许多基于浏览器的应用程序实际上都在运行 JavaScript 应用程序，而不是让浏览器成为服务器呈现内容的无声终端。

有了如此多的客户端选项，许多应用程序采用了一种常见的设计：将用户界面推进客户端，而服务器公开一个 API，通过该 API，所有类型的客户端都可以与后端进行交互。

在本章中，将使用 Spring 为 Taco Cloud 应用程序提供 REST API。将使用在第 2 章中所学到的关于 Spring MVC 的知识，使用 Spring MVC 控制器创建 RESTful 端点。还将自动公开第 3 章和第 4 章中定义的 Spring Data 存储库的 REST 端点。最后，我们将研究测试和保护这些端点的方法。

但首先，将从编写一些新的 Spring MVC 控制器开始，这些控制器公开带有 REST 端点的后端功能，以供丰富的 web 前端使用。

## 7.1 编写 RESTful 控制器

简而言之，REST API 与网站没有太大区别。两者都涉及对 HTTP 请求的响应。但关键的区别在于，网站是用 HTML 响应这些请求，而 REST API 通常以面向数据的格式（如 JSON 或 XML）响应。

在第 2 章中，使用 `@GetMapping` 和 `@PostMapping` 注解来获取和发送数据到服务器。在定义 REST API 时，这些相同的注释仍然很有用。此外，Spring MVC 还为各种类型的 HTTP 请求支持少量其他注解，如表 7.1 所示。

表 7.1 Spring MVC HTTP 请求处理注解

| 注解                           | HTTP 方法        | 典型用法   |
|------------------------------|----------------|--------|
| <code>@GetMapping</code>     | HTTP GET 请求    | 读取资源数据 |
| <code>@PostMapping</code>    | HTTP POST 请求   | 创建资源   |
| <code>@PutMapping</code>     | HTTP PUT 请求    | 更新资源   |
| <code>@PatchMapping</code>   | HTTP PATCH 请求  | 更新资源   |
| <code>@DeleteMapping</code>  | HTTP DELETE 请求 | 删除资源   |
| <code>@RequestMapping</code> | 通用请求处理         |        |

要查看这些注释的实际效果，将首先创建一个简单的 REST 端点，该端点获取一些最近创建的 taco。

### 7.1.1 从服务器获取数据

Taco Cloud 最酷的事情之一是它允许 Taco 狂热者设计他们自己的 Taco 作品，并与他们的 Taco 爱好者分享。为此，Taco Cloud 需要能够在单击最新设计链接时显示最近创建的 Taco 的列表。

为支持这个特性，我们增加一个 `/design/recent` 接口，使用 GET 请求，参数中包含 `recent`，返回一个最近设计的 taco 列表。我们创建一个新的控制器来处理这样的请求，下面的程序清单显示了完成这件事的控制器。

程序清单 7.2 处理 taco 设计 API 请求的 RESTful 控制器

```
package tacos.web.api;

import org.springframework.data.domain.PageRequest;
import org.springframework.data.domain.Sort;
import org.springframework.web.bind.annotation.CrossOrigin;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

import tacos.Taco;
import tacos.data.TacoRepository;

@RestController
@RequestMapping(path="/api/tacos",
    produces="application/json")
@CrossOrigin(origins="*")
public class TacoController {
    private TacoRepository tacoRepo;

    public TacoController(TacoRepository tacoRepo) {
        this.tacoRepo = tacoRepo;
    }

    @GetMapping(params="recent")
    public Iterable<Taco> recentTacos() {
        PageRequest page = PageRequest.of(
            0, 12, Sort.by("createdAt").descending());
        return tacoRepo.findAll(page).getContent();
    }
}
```

您可能感觉这个控制器的名字听起来很熟悉。在第 2 章中，您创建了一个处理类似类型请求的 `DesignTacoController`，但那个控制器是用于返回 HTML 结果的。这里的 `TacoController` 是一个 REST 控制器，正如 `@RestController` 注解所示。

`@RestController` 注解有两个用途。首先，它是一个像 `@Controller` 和 `@Service` 这样的原型注解，它通过组件扫描来标记一个类。但是与 REST 的讨论最相关的是，`@RestController` 注解告诉 Spring，控制器中的所有处理程序方法都应该将它们的返回值直接写入响应体，而不是在模型中被带到视图中进行呈现。

实际上，可以使用 `@Controller` 来注解 `TacoController`，就像使用任何 Spring MVC 控制器一样。但是，还需要使用 `@ResponseBody` 注解所有处理程序方法，以获得相同的结果。另一个选项是返回一个 `ResponseEntity` 对象，我们稍后将讨论它。

类级别的 `@RequestMapping` 注解与 `recentTacos()` 方法上的 `@GetMapping` 注解一起工作，以指定 `recentTacos()` 方法负责处理 `/design?recent` 接口的 GET 请求。



注意，`@RequestMapping` 注解还设置了一个 `produces` 属性。这指定了 `TacoController` 中的任何处理程序方法只在请求的 `Accept` 头包含 “`application/json`” 时才处理请求。这不仅限制了 API 只生成 JSON 结果，还允许另一个控制器（可能是第 2 章中的 `TacoController`）处理具有相同路径的请求，只要这些请求不需要 JSON 输出。

尽管这将 API 限制为基于 JSON 的，但是欢迎将 `produces` 设置为多个内容类型的字符串数组。例如，为了允许 XML 输出，可以向 `produces` 属性添加 “`text/html`”：

```
@RequestMapping(path="/api/tacos",
    produces={"application/json", "text/xml"})
```

在程序清单 7.2 中可能注意到的另一件事是，该类是用 `@CrossOrigin` 注解了的。由于应用程序的 Angular 部分将运行在独立于 API 的主机或端口上（至少目前是这样），web 浏览器将阻止 Angular 客户端使用 API。这个限制可以通过在服务器响应中包含 CORS（跨源资源共享）头来克服。Spring 使得使用 `@CrossOrigin` 注解应用 CORS 变得很容易。正如这里所应用的，`@CrossOrigin` 允许来自任何域的客户端使用 API。

`recentTacos()` 方法中的逻辑相当简单。它构造了一个 `PageRequest` 对象，该对象指定只想要包含 12 个结果的第一个（第 0 个）页面，结果按照 `taco` 的创建日期降序排序。简而言之就是您想要一打最新设计的 `tacos`。`PageRequest` 被传递到 `TacoRepository` 的 `findAll()` 方法的调用中，结果页面的内容被返回给客户机（如程序清单 7.1 所示，它将作为模型数据显示给用户）。

现在，您已经开始为您的客户端开发了 Taco Cloud API。出于测试目的，您可能还希望使用 `curl` 或 `HTTPie` 等命令行程序（<https://httpie.org/>）调用相关 API。例如，下面的命令行显示了如何获取最近创建的玉米卷：

```
$ curl localhost:8080/api/tacos?recent
```

如果您喜欢使用 `HTTPie`：

```
$ http :8080/api/tacos?recent
```

最初，数据库将是空的，因此来自这些请求的结果也将是空的。稍后我们将看到如何处理保存玉米卷的 `POST` 请求。但与此同时，您可以添加一个 `bean`，用 `CommandLineRunner` 预加载数据库中的一些测试数据。下面的 `CommandLineRunner bean` 方法显示了如何预加载一些配料，还有一些玉米卷：

```

@Bean
public CommandLineRunner dataLoader(
    IngredientRepository repo,
    UserRepository userRepo,
    PasswordEncoder encoder,
    TacoRepository tacoRepo) {
    return args -> {
        Ingredient flourTortilla = new Ingredient(
            "FLTO", "Flour Tortilla", Type.WRAP);
        Ingredient cornTortilla = new Ingredient(
            "COTO", "Corn Tortilla", Type.WRAP);
        Ingredient groundBeef = new Ingredient(
            "GRBF", "Ground Beef", Type.PROTEIN);
        Ingredient carnitas = new Ingredient(
            "CARN", "Carnitas", Type.PROTEIN);
        Ingredient tomatoes = new Ingredient(
            "TMTO", "Diced Tomatoes", Type.VEGGIES);
        Ingredient lettuce = new Ingredient(
            "LETC", "Lettuce", Type.VEGGIES);
        Ingredient cheddar = new Ingredient(
            "CHED", "Cheddar", Type.CHEESE);
        Ingredient jack = new Ingredient(
            "JACK", "Monterrey Jack", Type.CHEESE);
        Ingredient salsa = new Ingredient(
            "SLSA", "Salsa", Type.SAUCE);
        Ingredient sourCream = new Ingredient(
            "SRCR", "Sour Cream", Type.SAUCE);

        repo.save(flourTortilla);
        repo.save(cornTortilla);
        repo.save(groundBeef);
        repo.save(carnitas);
        repo.save(tomatoes);
        repo.save(lettuce);
        repo.save(cheddar);
        repo.save(jack);
        repo.save(salsa);
        repo.save(sourCream);

        Taco taco1 = new Taco();
        taco1.setName("Carnivore");
        taco1.setIngredients(Arrays.asList(
            flourTortilla, groundBeef, carnitas,
            sourCream, salsa, cheddar));
        tacoRepo.save(taco1);

        Taco taco2 = new Taco();
        taco2.setName("Bovine Bounty");
        taco2.setIngredients(Arrays.asList(
            cornTortilla, groundBeef, cheddar,
            jack, sourCream));
        tacoRepo.save(taco2);

        Taco taco3 = new Taco();
        taco3.setName("Veg-Out");
        taco3.setIngredients(Arrays.asList(
            flourTortilla, cornTortilla, tomatoes,
            lettuce, salsa));
        tacoRepo.save(taco3);
    };
}

```

现在，如果您尝试使用 `curl` 或 `HTTPIe` 向最近的 `tacos` 端点发出请求，您将得到类似下面这样的响应（为可读性而格式化了响应数据）：

```
$ curl localhost:8080/api/tacos?recent
[
  {
    "id": 4,
    "name": "Veg-Out",
    "createdAt": "2021-08-02T00:47:09.624+00:00",
    "ingredients": [
      { "id": "FLT0", "name": "Flour Tortilla", "type": "WRAP" },
      { "id": "COT0", "name": "Corn Tortilla", "type": "WRAP" },
      { "id": "TMT0", "name": "Diced Tomatoes", "type": "VEGGIES" },
      { "id": "LETC", "name": "Lettuce", "type": "VEGGIES" },
      { "id": "SLSA", "name": "Salsa", "type": "SAUCE" }
    ]
  },
  {
    "id": 3,
    "name": "Bovine Bounty",
    "createdAt": "2021-08-02T00:47:09.621+00:00",
    "ingredients": [
      { "id": "COT0", "name": "Corn Tortilla", "type": "WRAP" },
      { "id": "GRBF", "name": "Ground Beef", "type": "PROTEIN" },
      { "id": "CHED", "name": "Cheddar", "type": "CHEESE" },
      { "id": "JACK", "name": "Monterrey Jack", "type": "CHEESE" },
      { "id": "SRCR", "name": "Sour Cream", "type": "SAUCE" }
    ]
  },
  {
    "id": 2,
    "name": "Carnivore",
    "createdAt": "2021-08-02T00:47:09.520+00:00",
    "ingredients": [
      { "id": "FLT0", "name": "Flour Tortilla", "type": "WRAP" },
      { "id": "GRBF", "name": "Ground Beef", "type": "PROTEIN" },
      { "id": "CARN", "name": "Carnitas", "type": "PROTEIN" },
      { "id": "SRCR", "name": "Sour Cream", "type": "SAUCE" },
      { "id": "SLSA", "name": "Salsa", "type": "SAUCE" },
      { "id": "CHED", "name": "Cheddar", "type": "CHEESE" }
    ]
  }
]
```

现在，假设需要提供一个端点，该端点通过其 ID 获取单个 **taco**。通过在处理程序方法的路径中使用占位符变量并接受 **path** 变量的方法，可以捕获该 ID 并使用它通过存储库查找 **taco** 对象：

```
@GetMapping("/{id}")
public Optional<Taco> tacoById(@PathVariable("id") Long id) {
    return tacoRepo.findById(id);
}
```

因为控制器的基本路径是 `/api/tacos`，所以这个控制器方法处理 `/api/tacos/{id}` 的 GET 请求，其中路径的 `{id}` 部分是占位符。请求中的实际值指定给 `id` 参数，该参数通过 `@PathVariable` 映射到 `{id}` 占位符。

在 `tacoById()` 内部，将 `id` 参数传递给存储库的 `findById()` 方法来获取 **Taco**。`findById()` 返回一个 `Optional<Taco>`，因为可能没有具有给定 ID 的 **Taco**，控制器只是简单的返回 `Optional<Taco>`。

Spring 获取 `Optional<Taco>` 然后调用 `get()` 方法完成响应。如果 ID 不匹配任何已知的 **taco**，则返回 `null`，HTTP 响应状态码是 200（OK）。客户端会收到一个不能使用的响应，但是状态代码表明一切正常。更好的方法是返回一个带有 HTTP 404（NOT FOUND）状态的响应。

正如它目前所写的，没有简单的方法可以从 `tacoById()` 返回 404 状态代码。但如果您做一些小的调整，您可以设置适当的状态代码：

```
@GetMapping("/{id}")
public ResponseEntity<Taco> tacoById(@PathVariable("id") Long id) {
    Optional<Taco> optTaco = tacoRepo.findById(id);
    if (optTaco.isPresent()) {
        return new ResponseEntity<>(optTaco.get(), HttpStatus.OK);
    }
    return new ResponseEntity<>(null, HttpStatus.NOT_FOUND);
}
```

现在，`tacoById()` 不返回 `Taco` 对象，而是返回一个 `ResponseEntity<Taco>`。如果发现 `taco`，则将 `taco` 对象包装在 HTTP 状态为 OK 的 `ResponseEntity` 中（这是之前的行为）。但是，如果没有找到 `taco`，则在 `ResponseEntity` 中包装一个 `null`，并加上一个 HTTP status（NOT FOUND），以指示客户端试图获取一个不存在的 `taco`。

但是，定义返回信息的端点只是开始。如果 API 需要从客户端接收数据呢？让我们看看如何编写处理请求输入的控制器方法。

## 7.1.2 向服务器发送数据

到目前为止，API 能够返回一些最近创建的 **tacos**。但是这些 **tacos** 是如何产生的呢？

虽然您可以使用 **CommandLineRunner** bean 为测试而预加载一些数据，但最终 **Taco** 的数据来自用户，由他们进行 **Taco** 创作。因此 我们需要在 **TacoController** 中编写一个方法，来处理包含 **taco** 设计的请求，并将它们保存到数据库中。通过将以下 `postTaco()` 方法添加到 **TacoController**，您可以使控制器完成这项工作：

```
@PostMapping(consumes="application/json")
@ResponseStatus(HttpStatus.CREATED)
public Taco postTaco(@RequestBody Taco taco) {
    return tacoRepo.save(taco);
}
```

因为 `postTaco()` 将处理 HTTP POST 请求，所以它用 `@PostMapping` 注解，而不是 `@GetMapping`。这里没有指定 `path` 属性，因此 `postTaco()` 方法将处理对 `/api/tacos` 的请求。路径是在 **TacoController** 类中由注解 `@RequestMapping` 指定的。

您确实设置了 `consumes` 属性。`consumes` 属性是请求输入，`products` 是请求输出。这里使用 `consumes` 表示该方法将只处理内容为 `application/json` 类型的请求。

该方法的 **Taco** 参数用 `@RequestBody` 注解，以指示请求应转换为 **Taco** 对象并绑定到参数上。此注解很重要：如果没有它，Spring MVC 将假定您需要请求参数（查询参数或表单参数）。但是 `@RequestBody` 注解确保请求主体中的 JSON 绑定到 **Taco** 对象上。

一旦 `postTaco()` 接收到 **Taco** 对象后，就会将其传递给 **TacoRepository** 的 `save()` 方法。

这里在 `postTaco()` 方法上使用了 `@ResponseStatus(HttpStatus.CREATED)` 注解。在正常情况下（当没有抛出异常时），所有响应的 HTTP 状态码为 200（OK），表示请求成功。尽管 HTTP 200 响应总是好的，但它并不总是具有足够的描述性。对于 POST 请求，HTTP 状态 201（CREATED）更具描述性，它告诉客户机，请求不仅成功了，而且还创建了一个资源。在适当的地方使用 `@ResponseStatus` 将最具描述性和最准确的 HTTP 状态代码传递给客户端总是一个好想法。

虽然已经使用 `@PostMapping` 创建了一个新的 **Taco** 资源，但是 POST 请求也可以用于更新资源。即便如此，POST 请求通常用于创建资源，PUT 和 PATCH 请求用于更新资源。让我们看看如何使用 `@PutMapping` 和 `@PatchMapping` 更新数据。

### 7.1.3 更新服务器上的数据

在编写任何处理 HTTP PUT 或 PATCH 命令的控制器代码之前，应该花点时间考虑一下这个问题：为什么有两种不同的 HTTP 方法来更新资源呢？

虽然 PUT 经常用于更新资源数据，但它实际上是对 GET 语义的对立面。GET 请求用于将数据从服务器传输到客户机，而 PUT 请求用于将数据从客户机发送到服务器。

从这个意义上说，PUT 实际上是用于执行大规模替换操作，而不是更新操作。相反，HTTP PATCH 的目的是执行补丁或部分更新资源数据。

例如，假设希望能够更改订单上的地址，我们可以通过 REST API 实现这一点，可以用以下方式处理 PUT 请求：

```
@PutMapping(path="/{orderId}", consumes="application/json")
public TacoOrder putOrder(
    @PathVariable("orderId") Long orderId,
    @RequestBody TacoOrder order) {
    order.setId(orderId);
    return repo.save(order);
}
```

这可能行得通，但它要求客户端在 PUT 请求中提交完整的订单数据。从语义上讲，PUT 的意思是“把这个数据放到这个 URL 上”，本质上是替换任何已经存在的数据。如果订单的任何属性被省略，该属性的值将被 null 覆盖。甚至订单中的 taco 也需要与订单数据一起设置，否则它们将从订单中删除。

如果 PUT 完全替换了资源数据，那么应该如何处理只进行部分更新的请求？这就是 HTTP PATCH 请求和 Spring 的 @PatchMapping 的好处。可以这样写一个控制器方法来处理一个订单的 PATCH 请求：

```

@PatchMapping(path="/{orderId}", consumes="application/json")
public TacoOrder patchOrder(@PathVariable("orderId") Long orderId,
    @RequestBody TacoOrder patch) {

    TacoOrder order = repo.findById(orderId).get();
    if (patch.getDeliveryName() != null) {
        order.setDeliveryName(patch.getDeliveryName());
    }
    if (patch.getDeliveryStreet() != null) {
        order.setDeliveryStreet(patch.getDeliveryStreet());
    }
    if (patch.getDeliveryCity() != null) {
        order.setDeliveryCity(patch.getDeliveryCity());
    }
    if (patch.getDeliveryState() != null) {
        order.setDeliveryState(patch.getDeliveryState());
    }
    if (patch.getDeliveryZip() != null) {
        order.setDeliveryZip(patch.getDeliveryState());
    }
    if (patch.getCcNumber() != null) {
        order.setCcNumber(patch.getCcNumber());
    }
    if (patch.getCcExpiration() != null) {
        order.setCcExpiration(patch.getCcExpiration());
    }
    if (patch.getCcCVV() != null) {
        order.setCcCVV(patch.getCcCVV());
    }
    return repo.save(order);
}

```

这里要注意的第一件事是，`patchOrder()` 方法是用 `@PatchMapping` 而不是 `@PutMapping` 来注解的，这表明它应该处理 HTTP PATCH 请求而不是 PUT 请求。

但是 `patchOrder()` 方法比 `putOrder()` 方法更复杂一些。这是因为 Spring MVC 的映射注解（包括 `@PatchMapping` 和 `@PutMapping`）只指定了方法应该处理哪些类型的请求。这些注解没有规定如何处理请求。尽管 PATCH 在语义上暗示了部分更新，但是可以在处理程序方法中编写实际执行这种更新的代码。

对于 `putOrder()` 方法，接受订单的完整数据并保存它，这符合 HTTP PUT 的语义。但是为了使 `patchMapping()` 坚持 HTTP PATCH 的语义，该方法的主体需要更多语句。它不是用发送进来的新数据完全替换订单，而是检查传入订单对象的每个字段，并将任何非空值应用于现有订单。这种方法允许客户机只发送应该更改的属性，并允许服务器为客户机未指定的任何属性保留现有数据。

使用 PATCH 的方法不止一种

PATCH 方式应用于 `patchOrder()` 方法时，有两个限制：

- 如果传递的是 null 值，意味着没有变化，那么客户端如何指示字段应该设置为 null？
- 没有办法从一个集合中移除或添加一个子集。如果客户端想要从集合中添加或删除一条数据，它必须发送完整的修改后的集合。

对于应该如何处理 PATCH 请求或传入的数据应该是什么样子，确实没有硬性规定。客户端可以发送应用于特定 PATCH 请求的描述，这个描述包含着需要被应用于数据的更改，而不是发送实际的域数据。当然，必须编写请求处理程序来处理 PATCH 指令，而不是域数据。

在 `@PutMapping` 和 `@PatchMapping` 中，请注意请求路径引用了将要更改的资源。这与 `@GetMapping` 方法处理路径的方式相同。

现在已经了解了如何使用 `@GetMapping` 和 `@PostMapping` 来获取和发布资源。已经看到了使用 `@PutMapping` 和 `@PatchMapping` 更新资源的两种不同方法，剩下的工作就是处理删除资源的请求。



## 7.1.4 删除服务器上的数据

有时数据根本就不再需要了。在这些情况下，客户端需要发起 HTTP DELETE 请求删除资源。

Spring MVC 的 `@DeleteMapping` 可以方便地声明处理 DELETE 请求的方法。例如，假设需要 API 允许删除订单资源，下面的控制器方法应该可以做到这一点：

```
@DeleteMapping("/{orderId}")
@ResponseStatus(HttpStatus.NO_CONTENT)
public void deleteOrder(@PathVariable("orderId") Long orderId) {
    try {
        repo.deleteById(orderId);
    } catch (EmptyResultDataAccessException e) {}
}
```

至此，另一个映射注解的思想对您来说应该已经过时了。您已经看到了

`@GetMapping`、`@PostMapping`、`@PutMapping` 和 `@PatchMapping` —— 每一个都指定了一个方法应该处理对应的 HTTP 方法的请求。`@DeleteMapping` 用于 `deleteOrder()` 方法负责处理 `/orders/{orderId}` 的删除请求。

该方法中的代码实际用于执行删除订单操作。在本例中，它接受作为 URL 中的路径变量提供的订单 ID，并将其传递给存储库的 `deleteById()` 方法。如果调用该方法时订单存在，则将删除它。如果订单不存在，将抛出一个 `EmptyResultDataAccessException` 异常。

我选择捕获 `EmptyResultDataAccessException` 而不做任何事情。我的想法是，如果试图删除一个不存在的资源，其结果与在删除之前它确实存在的结果是一样的，也就是说，资源将不存在。它以前是否存在无关紧要。或者，我也可以编写 `deleteOrder()` 来返回一个 `ResponseEntity`，将 `body` 设置为 `null`，将 HTTP 状态代码设置为 NOT FOUND。

在 `deleteOrder()` 方法中需要注意的惟一点是，它使用 `@ResponseStatus` 进行了注解，以确保响应的 HTTP 状态是 204（NO CONTENT）。对于不再存在的资源，不需要将任何资源数据发送回客户机，因此对删除请求的响应通常没有正文，因此应该发送一个 HTTP 状态代码，让客户机知道不需要任何内容。

Taco Cloud API 已经开始成形了，客户端代码现在可以轻松地使用这个 API 来显示配料、接受订单和显示最近创建的 tacos。我们将在 7.3 节讨论如何编写 REST 客户端。接下来，让我们看看如何将超媒体添加到 Taco Cloud API 中。

## 7.2 启用后端数据服务

正如在第 3 章中看到的，Spring Data 拥有一种特殊的魔力，它根据在代码中定义的接口自动创建存储库的实现。但是 Spring Data 还有另一个技巧，可以为应用程序定义 API。

Spring Data REST 是 Spring Data 家族中的另一个成员，它为 Spring Data 创建的存储库自动创建 REST API。只需将 Spring Data REST 添加到构建中，就可以获得一个 API，其中包含所定义的所有存储库接口的操作。

要开始使用 Spring Data REST，需要在构建中添加以下依赖项：

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-rest</artifactId>
</dependency>
```

信不信由您，这就是在一个已经将 Spring Data 用于自动存储库的项目中公开 REST API 所需的全部内容。通过在构建中简单地使用 Spring Data REST starter，应用程序可以自动配置，从而为 Spring Data 创建的任何存储库（包括 Spring Data JPA、Spring Data Mongo 等）自动创建 REST API。

Spring Data REST 创建的 REST 端点至少与自己创建的端点一样好（甚至可能更好）。因此，在这一点上，可以做一些拆卸工作，并在继续之前删除到目前为止创建的任何 @RestController 注解的类。

要尝试 Spring Data REST 提供的端点，可以启动应用程序并开始查看一些 url。基于已经为 Taco Cloud 定义的存储库集，应该能够执行针对 Taco、Ingredient、Order 和 User 的 GET 请求。

例如，可以通过向 /ingredients 接口发出 GET 请求来获得所有 Ingredient 的列表。使用 curl，可能会得到这样的结果（经过删节，只显示第一个 Ingredient）：

```
$ curl localhost:8080/ingredients
{
  "_embedded" : {
    "ingredients" : [ {
      "name" : "Flour Tortilla",
      "type" : "WRAP",
      "_links" : {
        "self" : {
          "href" : "http://localhost:8080/ingredients/FLT0"
        },
        "ingredient" : {
          "href" : "http://localhost:8080/ingredients/FLT0"
        }
      }
    },
    ...
  ],
  "_links" : {
    "self" : {
      "href" : "http://localhost:8080/ingredients"
    },
    "profile" : {
      "href" : "http://localhost:8080/profile/ingredients"
    }
  }
}
```

哇！通过向构建中添加一个依赖项，不仅获得了 **Ingredient** 的端点，而且返回的资源也包含超链接！这些超链接是作为应用程序状态引擎的超媒体的实现，或简言之，**HATEOAS**。使用此 **API** 的客户端可以（可选）将这些超链接用作导航 **API** 和执行下一个请求的指南。

**Spring HATEOAS** 项目，为在您的应用程序中添加 **Spring MVC** 控制器响应超媒体链接提供了支持。但是 **Spring Data REST** 会自动将这些链接添加到其生成的 **API** 的响应中。

#### HATEOAS 或者非 HATEOAS？

**HATEOAS** 的总体思想是，它使客户机能够在 **API** 中导航，这与人类浏览网站的方式大致相同：通过跟踪链接。而不是在客户机中编码 **API** 细节，并让客户机构造对于每个请求的 **URL**。客户端可以从列表中按名称选择链接，并使用它发出下一个请求。这样，客户机就不需要编码以了解 **API** 的结构就可以使用。**API** 本身就是浏览 **API** 的路线图。另一方面，超链接确实会添加少量额外数据，有效负载增加了一些复杂性，要求客户端知道如何使用这些超链接导航。由于这个原因，如果在 **API** 中没有任何可用的参数，**API** 开发人员经常放弃 **HATEOAS**，客户端开发人员通常会忽略超链接。除了从 **Spring Data REST** 响应中获得的免费超链接，我们将忽略 **HATEOAS**，重点关注简单的非超媒体 **API**。

假装是这个 **API** 的客户端，您还可以使用 **curl** 来跟随 **self** 链接获得面粉玉米饼入口：

```
$ curl http://localhost:8080/ingredients/FLT0
{
  "name" : "Flour Tortilla",
  "type" : "WRAP",
  "_links" : {
    "self" : {
      "href" : "http://localhost:8080/ingredients/FLT0"
    },
    "ingredient" : {
      "href" : "http://localhost:8080/ingredients/FLT0"
    }
  }
}
```

为了避免过于分散注意力，在本书中我们不会浪费太多时间来深入研究 **Spring Data REST** 创建的每个端点和选项。但是应该知道，它还支持其创建的端点的 **POST**、**PUT** 和 **DELETE** 方法。没错：可以通过向 `/ingredients` 接口发送 **POST** 请求创建一个新的 **Ingredient**，然后通过向 `/ingredient/FLT0` 接口发送 **DELETE** 请求来从菜单上移除面粉玉米饼。

可能想要做的一件事是为 **API** 设置一个基本路径，这样它的端点是不同的，并且不会与编写的任何控制器发生冲突。（事实上，如果不删除先前创建的 **IngredientController**，它将干扰 **Spring Data REST** 提供的 `/ingredients` 端点。）要调整 **API** 的基本路径，请设置 `spring.data.rest` 基本路径属性：

```
spring:
  data:
    rest:
      base-path: /api
```

这将设置 **Spring Data REST** 端点的基本路径为 `/api`。因此，**Ingredient** 端点现在是 `/api/ingredients`。现在，通过请求一个 **tacos** 列表来使用这个新的基本路径：

```
$ curl http://localhost:8080/data-api/tacos
{
  "timestamp": "2018-02-11T16:22:12.381+0000",
  "status": 404,
  "error": "Not Found",
  "message": "No message available",
  "path": "/api/tacos"
}
```

噢？这并没有达到预期的效果。有一个 `Ingredient` 实体和一个 `IngredientRepository` 接口，其中 `Spring Data REST` 暴露 `/data-api/ingredients` 端点。因此，如果有一个 `Taco` 实体和一个 `TacoRepository` 接口，为什么 `Spring Data REST` 不能提供 `/data-api/tacos` 端点呢？

## 7.2.1 调整资源路径和关系名称

实际上，Spring Data REST 提供了处理 `tacos` 的端点。但是，尽管 Spring Data REST 非常智能，但它在暴露 `tacos` 端点方面的表现却稍微逊色一些。

在为 Spring Data 存储库创建端点时，Spring Data REST 尝试使关联多元化的实体类。对于 `Ingredient` 实体，端点是 `/data-api/ingredients`。对于 `TacoOrder`，它是 `/data-api/orders`。到目前为止，一切顺利。

但有时，比如“`taco`”，它会在一个字母上出错，这样复数形式就不太正确了。事实证明，Spring Data REST 将复数形式“`taco`”表示为“`tacoes`”，因此，要想对 `tacos` 发出请求，您必须请求 `/data-api/tacoes`：

```
$ curl localhost:8080/data-api/tacoes
{
  "_embedded" : {
    "tacoes" : [ {
      "name" : "Carnivore",
      "createdAt" : "2018-02-11T17:01:32.999+0000",
      "_links" : {
        "self" : {
          "href" : "http://localhost:8080/data-api/tacoes/2"
        },
        "taco" : {
          "href" : "http://localhost:8080/data-api/tacoes/2"
        },
        "ingredients" : {
          "href" : "http://localhost:8080/data-api/tacoes/2/ingredients"
        }
      }
    }
  ]
},
  "page" : {
    "size" : 20,
    "totalElements" : 3,
    "totalPages" : 1,
    "number" : 0
  }
}
```

您可能想知道我怎么知道“`taco`”会被误拼成“`tacoes`”。事实证明，Spring Data REST 还公开了一个 `home` 资源，其中包含所有公开端点的链接。只需向 API 基础路径发出 GET 请求即可获得：

```
$ curl localhost:8080/api
{
  "_links" : {
    "orders" : {
      "href" : "http://localhost:8080/data-api/orders"
    },
    "ingredients" : {
      "href" : "http://localhost:8080/data-api/ingredients"
    },
    "tacos" : {
      "href" : "http://localhost:8080/data-api/tacos{?page,size,sort}",
      "templated" : true
    },
    "users" : {
      "href" : "http://localhost:8080/data-api/users"
    },
    "profile" : {
      "href" : "http://localhost:8080/data-api/profile"
    }
  }
}
```

可以看到，**home** 资源显示了所有实体的链接。除了 **tacos** 链接之外，一切看起来都很好，其中关系名称和 URL 都有 “taco” 的单数复数形式。

好消息是，不必接受 Spring Data REST 的这个怪癖。通过向 **Taco** 类添加一个简单的注解，可以调整关系名称和路径：

```
@Data
@Entity
@RestResource(rel="tacos", path="tacos")
public class Taco {
    ...
}
```

**@RestResource** 注解让您给任何您想要的名称和路径的关系，在这个例子中，把它们都设定为了 “tacos”。现在当请求 **home** 资源的时候，将会看到 **tacos** 链接正确的复数形式：

```
"tacos" : {
  "href" : "http://localhost:8080/data-api/tacos{?page,size,sort}",
  "templated" : true
},
```

这还可以对端点的路径进行排序，这样就可以针对 `/data-api/tacos` 接口发起请求来使用 **taco** 资源了。

说到排序，让我们看看如何对 Spring Data REST 端点的结果进行排序。

## 7.2.2 分页和排序

您可能注意到了在 `home` 资源的链接中，全部都有 `page`、`size` 和 `sort` 参数。默认情况下，像是对 `/data-api/tacos` 这种集合资源请求的接口来说，将会从第一页返回每页 20 个数据项。但是可以根据请求的要求，通过指定特定的 `page` 和 `size` 参数来调整页面大小和哪一页。

举个例子，要请求 `tacos` 的页面大小为 5 的第一页，可以发起以下 GET 请求（使用 `curl`）：

```
$ curl "localhost:8080/data-api/tacos?size=5"
```

假设有大于 5 条 `tacos` 数据，可以通过添加 `page` 参数请求 `tacos` 数据的第二页：

```
$ curl "localhost:8080/data-api/tacos?size=5&page=1"
```

注意 `page` 参数是从 0 开始的，意思是请求第 1 页实际上是请求的第 2 页。（还会注意到许多 `shell` 命令行在请求中的 `&` 符号上出错，这就是为什么我在前面的 `curl` 命令中引用整个 URL 的原因。）

`sort` 参数允许根据实体的任何属性对结果列表进行排序。例如，需要一种方法来获取最近创建的 12 个 `tacos`，以便 UI 显示，可以通过指定以下分页和排序参数组合来做到这一点：

```
$ curl "localhost:8080/data-api/tacos?sort=createdAt,desc&page=0&size=12"
```

这里，`sort` 参数指定了应该根据 `createdAt` 属性进行排序，并按降序排序（以便最新的 `tacos` 排在前面）。页面和大小参数的指定确定了应该在第一个页面上看到 12 个 `tacos`。

这正是 UI 为了显示最近创建的 `tacos` 所需要的。它与在本章前面的 `TacoController` 中定义的 `/api/tacos?recent` 端点大致相同。

现在，让我们切换思路，看看如何编写客户端代码，来使用我们创建的 API 端点。

## 7.3 使用 REST 服务

您是否曾经去看电影，当电影开始的时候，您发现只有您一个人在电影院？从本质上说，这是一次私人观影的美妙经历。您可以选择任何您想要的座位，和屏幕上的人物交谈，甚至可以打开您的手机发推特谈论它，而不会有人因为破坏了他们的观影体验而生气。最棒的是，也没有其他人会为您毁了这部电影！

这种情况在我身上并不常见。但当它出现的时候，我在想如果我没有出现会发生什么。他们还会放映这部电影吗？英雄还会拯救世界吗？电影结束后，工作人员还会打扫影院吗？

没有观众的电影就像没有客户端的 API。它已经准备好接受和提供数据了，但是如果 API 从未被调用过，那么它真的是一个 API 吗？就像薛定谔的猫一样，在我们向它发出请求之前，我们无法知道 API 是活动的还是返回 HTTP 404 响应。

Spring 应用程序既提供 API，又向另一个应用程序的 API 发出请求，这种情况并不少见。事实上，在微服务的世界里，这正变得越来越普遍。因此，花点时间看看如何使用 Spring 与 REST API 交互是值得的。

Spring 应用程序可以通过以下方式使用 REST API：

- **RestTemplate** —— 一个由 Spring 核心框架提供的简单、同步 REST 客户端。
- **Traverson** —— 可感知超链接的同步 REST 客户端，由 Spring HATEOAS 提供，灵感来自同名的 JavaScript 库。
- **WebClient** —— 一个响应式、异步 REST 客户端。

现在，我们将主要关注 **RestTemplate**。在第 12 章讨论 Spring 的响应式 web 框架之前，我将推迟讨论 **WebClient**。如果您有兴趣 编写支持超链接的客户端，请查看以下位置的 **Traverson** 文档：

<https://docs.spring.io/spring-hateoas/docs/current/reference/html/#client>。

从客户的角度来看，与 REST 资源进行交互需要做很多工作 —— 主要是单调乏味的样板文件。使用低级 HTTP 库，客户端需要创建一个客户端实例和一个请求对象，执行请求，解释响应，将响应映射到域对象，并处理过程中可能抛出的任何异常。不管发送什么 HTTP 请求，所有这些样板文件都会重复。

为了避免这样的样板代码，Spring 提供了 **RestTemplate**。正如 **JdbcTemplate** 处理使用 JDBC 糟糕的那部分一样，**RestTemplate** 使你不必为调用 REST 资源而做单调的工作。

**RestTemplate** 提供了 41 个与 REST 资源交互的方法。与其检查它提供的所有方法，不如只考虑 12 个惟一的操作，每个操作都有重载，以形成 41 个方法的完整集合。表 7.2 描述了 12 种操作。

表 7.2 **RestTemplate** 定义的 12 个唯一操作，每一个都重载了，共提供 41 种方法（未完待续）



| 方法                                | 描述                                                                  |
|-----------------------------------|---------------------------------------------------------------------|
| <code>delete(...)</code>          | 对指定 URL 上的资源执行 HTTP DELETE 请求                                       |
| <code>exchange(...)</code>        | 对 URL 执行指定的 HTTP 方法，返回一个 <code>ResponseEntity</code> ，其中包含从响应体映射的对象 |
| <code>execute(...)</code>         | 对 URL 执行指定的 HTTP 方法，返回一个映射到响应体的对象                                   |
| <code>getForEntity(...)</code>    | 发送 HTTP GET 请求，返回一个 <code>ResponseEntity</code> ，其中包含从响应体映射的对象      |
| <code>getForObject(...)</code>    | 发送 HTTP GET 请求，返回一个映射到响应体的对象                                        |
| <code>headForHeaders(...)</code>  | 发送 HTTP HEAD 请求，返回指定资源 URL 的 HTTP 请求头                               |
| <code>optionsForAllow(...)</code> | 发送 HTTP OPTIONS 请求，返回指定 URL 的 Allow 头信息                             |
| <code>patchForObject(...)</code>  | 发送 HTTP PATCH 请求，返回从响应主体映射的结果对象                                     |
| <code>postForEntity(...)</code>   | 将数据 POST 到一个 URL，返回一个 <code>ResponseEntity</code> ，其中包含从响应体映射而来的对象  |
| <code>postForLocation(...)</code> | 将数据 POST 到一个 URL，返回新创建资源的 URL                                       |
| <code>postForObject(...)</code>   | 将数据 POST 到一个 URL，返回从响应主体映射的对象                                       |
| <code>put(...)</code>             | 将资源数据 PUT 到指定的 URL                                                  |

除了 TRACE 之外，`RestTemplate` 对于每个标准 HTTP 方式至少有一个方法。此外，`execute()` 和 `exchange()` 为使用任何 HTTP 方式发送请求提供了低层的通用方法。

表 7.2 中的大多数方法都被重载为三种方法形式：

- 一种是接受一个 `String` 作为 URL 规范，在一个变量参数列表中指定 URL 参数。
- 一种是接受一个 `String` 作为 URL 规范，其中的 URL 参数在 `Map<String, String>` 中指定。
- 一种是接受 `java.net.URI` 作为 URL 规范，不支持参数化 URL。

一旦了解了 `RestTemplate` 提供的 12 个操作以及每种变体的工作方式，就可以很好地编写调用资源的 REST 客户端了。

要使用 `RestTemplate`，需要创建一个实例：

```
RestTemplate rest = new RestTemplate();
```

或是将它声明为一个 `bean`，在需要它的时候将其注入：

```
@Bean
public RestTemplate restTemplate() {
    return new RestTemplate();
}
```

让我们通过查看支持四种主要 HTTP 方法（GET、PUT、DELETE 和 POST）的操作来探寻 `RestTemplate` 的操作。我们将从 `getForObject()` 和 `getForEntity()` —— GET 方法开始。

### 7.3.1 GET 资源

假设想从 Taco Cloud API 获取一个 `Ingredient` 数据。需要使用 `RestTemplate` 的 `getForObject()` 来获取 `Ingredient`。例如，下面的代码使用 `RestTemplate` 获取一个 `Ingredient` 对象的 ID：

```
public Ingredient getIngredientById(String ingredientId) {
    return rest.getForObject("http://localhost:8080/ingredients/{id}",
        Ingredient.class, ingredientId);
}
```

这里使用的是 `getForObject()` 变量，它接受一个字符串 URL 并为 URL 变量使用一个变量列表。传递给 `getForObject()` 的 `ingredientId` 参数用于填充给定 URL 中的 `{id}` 占位符。虽然在本例中只有一个 URL 变量，但重要的是要知道变量参数是按给定的顺序分配给占位符的。

`getForObject()` 的第二个参数是响应应该绑定的类型。在这种情况下，应该将响应数据（可能是 JSON 格式）反序列化为将要返回的 `Ingredient` 对象。

或者，可以使用映射来指定 URL 变量：

```
public Ingredient getIngredientById(String ingredientId) {
    Map<String, String> urlVariables = new HashMap<>();
    urlVariables.put("id", ingredientId);
    return rest.getForObject("http://localhost:8080/ingredients/{id}",
        Ingredient.class, urlVariables);
}
```

在这个例子中，`ingredientId` 的值被映射到 `id` 键上，当发出请求时，`{id}` 占位符被键为 `id` 的映射条目替换。

使用 URI 参数稍微复杂一些，需要在调用 `getForObject()` 之前构造一个 URI 对象，它类似于其他两中形式：

```
public Ingredient getIngredientById(String ingredientId) {
    Map<String, String> urlVariables = new HashMap<>();
    urlVariables.put("id", ingredientId);
    URI url = UriComponentsBuilder
        .fromHttpUrl("http://localhost:8080/ingredients/{id}")
        .build(urlVariables);
    return rest.getForObject(url, Ingredient.class);
}
```

这里的 URI 对象是根据字符串规范定义的，其占位符是根据映射中的条目填充的，这与前面的 `getForObject()` 形式非常相似。`getForObject()` 方法是获取资源的一种有效方法。但是，如果客户端需要的不仅仅是有效负载，可能需要考虑使用 `getForEntity()`。

`getForEntity()` 的工作方式与 `getForObject()` 非常相似，但它返回的不是表示响应有效负载的域对象，而是包装该域对象的 `ResponseEntity` 对象。`ResponseEntity` 允许访问附加的响应细节，比如响应头。

例如，假设除了 `Ingredient` 数据之外，还希望检查响应中的 `Date` 头信息，有了 `getForEntity()`，事情就简单多了：

```
public Ingredient getIngredientById(String ingredientId) {  
    ResponseEntity<Ingredient> responseEntity =  
        rest.getForEntity("http://localhost:8080/ingredients/{id}",  
            Ingredient.class, ingredientId);  
    log.info("Fetched time: " +  
        responseEntity.getHeaders().getDate());  
    return responseEntity.getBody();  
}
```

`getForEntity()` 方法使用与 `getForObject()` 相同的重载参数，因此可以将 `URL` 变量作为变量列表参数，或者使用 `URI` 对象调用 `getForEntity()`。

### 7.3.2 PUT 资源

对于发送 HTTP PUT 请求，`RestTemplate` 提供 `put()` 方法。`put()` 的所有三个重载方法都接受一个将被序列化并发送到给定 URL 的对象。至于 URL 本身，可以将其指定为 `URI` 对象或 `String`。与 `getForObject()` 和 `getForEntity()` 类似，URL 变量可以作为变量参数列表或 `Map` 提供。

假设想要用来自一个新的 `Ingredient` 对象的数据来替换配料资源。下面的代码应该可以做到这一点：

```
public void updateIngredient(Ingredient ingredient) {
    rest.put("http://localhost:8080/ingredients/{id}",
            ingredient, ingredient.getId());
}
```

这里 URL 以 `String` 的形式给出，并有一个占位符，该占位符由给定的 `Ingredient` 对象的 `id` 属性替换。要发送的数据是 `Ingredient` 对象本身。`put()` 方法返回 `void`，因此不需要处理返回值。

### 7.3.3 DELETE 资源

假设 Taco Cloud 不再提供一种配料，并希望将其作为一种选项完全删除。要做到这一点，可以从 `RestTemplate` 中调用 `delete()` 方法：

```
public void deleteIngredient(Ingredient ingredient) {  
    rest.delete("http://localhost:8080/ingredients/{id}",  
               ingredient.getId());  
}
```

在本例中，仅将 URL（指定为 `String`）和 URL 变量值赋给 `delete()`。但是，与其他 `RestTemplate` 方法一样，可以将 URL 指定为 `URI` 对象，或者将 URL 参数指定为 `Map`。

### 7.3.4 POST 资源数据

现在，假设向 Taco Cloud 菜单添加了一种新 `Ingredient`。向 `.../ingredients` 端点发起 HTTP POST 请求就能实现添加，这个请求的请求体需要包含 `Ingredient` 数据。`RestTemplate` 有三种发送 POST 请求的方法，每种方法都有相同的重载变量来指定 URL。如果想在 POST 请求后收到新创建的 `Ingredient` 资源，可以像这样使用 `postForObject()`：

```
public Ingredient createIngredient(Ingredient ingredient) {
    return rest.postForObject("http://localhost:8080/ingredients",
                             ingredient, Ingredient.class);
}
```

`postForObject()` 方法的这种形式采用 `String` 作为 URL 规范，要发送到服务器的对象以及响应主体应该绑定到的域类型。虽然在本例中没有利用它，但第四个参数可以是 URL 变量值的 `Map` 或要替换到 URL 中的参数的变量列表。

如果客户对新创建的资源的位置有更多的需求，那么可以调用 `postForLocation()`：

```
public java.net.URI createIngredient(Ingredient ingredient) {
    return rest.postForLocation("http://localhost:8080/ingredients",
                               ingredient);
}
```

注意，`postForLocation()` 的工作方式与 `postForObject()` 非常相似，只是它返回的是新创建资源的 `URI`，而不是资源对象本身。返回的 `URI` 派生自响应的 `Location` 头信息。如果同时需要位置和响应负载，可以调用

`postForEntity()`：

```
public Ingredient createIngredient(Ingredient ingredient) {
    ResponseEntity<Ingredient> responseEntity =
        rest.postForEntity("http://localhost:8080/ingredients",
                           ingredient,
                           Ingredient.class);
    log.info("New resource created at " +
             responseEntity.getHeaders().getLocation());
    return responseEntity.getBody();
}
```

虽然 `RestTemplate` 方法的用途不同，但是它们的使用方式非常相似。这使得你很容易精通 `RestTemplate` 并在客户端代码中使用它。

## 7.4 小结

- Spring MVC 可以创建端点，控制器遵循与以浏览器为目标的控制器相同的编程模型。
- 控制器处理程序方法可以使用 `@ResponseBody` 进行注解，也可以返回 `ResponseEntity` 对象，从而绕过模型，直接将数据写入响应体。
- `@RestController` 注解简化了 REST 控制器，无需在处理程序方法上使用 `@ResponseBody`。
- Spring Data 存储仓库可以使用 Spring Data REST 自动公开为 REST API。

## 第 8 章 保护 REST 服务

本章内容：

- 使用 OAuth 2 保护 API
- 创建授权服务器
- 将资源服务器添加到 API
- 使用 OAuth 2 安全 API

您有没有使用过代客泊车服务？这是一个简单的概念。当您去商店、酒店、剧院或餐厅，把车钥匙递给门口的服务人员，他们会给您找个停车位。然后，当您回来时，他们会把您的车还给您。大概是因为我看了太多次“春天不是读书天”电影，我总是不愿意把我的车钥匙交给一个陌生人，怕他们不能替我好好保管我的车。

尽管如此，代客泊车涉及到信任他人照看您的汽车。许多的较新的汽车提供“代客泊车钥匙”，这是一种只能用来打开车门和启动发动机的特殊钥匙。这样，您授予的信任的数量在范围上是有限的。泊车人员不能用专用钥匙打开手套箱或行李箱。

在分布式应用程序中，软件系统之间的信任至关重要。即使是在一个简单的情况下，当客户端应用程序使用后端 API 时，重要的是客户端是可信的，并且任何其他未授权人试图使用 API 都将被阻止。和泊车人员一样，您授予客户的信任仅限于履行其职责所需的功能。

保护 REST API 与保护基于浏览器的 web 应用程序不同。在本章中，我们将研究 OAuth2，一个专门为 API 创建的授权规范。在此过程中，我们将了解 Spring Security 对 OAuth2 的支持。首先，让我们先看看 OAuth 2 是如何工作的。



## 8.1 OAuth 2 介绍

假设，我们想要创建一个新的后台应用程序来管理 Taco Cloud 应用。更具体地说，我们希望这个新应用程序，能够管理 Taco Cloud 网站上提供的配料。

在开始为管理应用程序编写代码之前，我们需要添加一些 Taco Cloud API 的新接口以支持配料管理。清单 8.1 所示的 REST 控制器 提供了列表查询、添加和删除配料的三个接口。

程序清单 8.1 管理可用材料的控制器

```
package tacos.web.api;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.HttpStatus;
import org.springframework.web.bind.annotation.CrossOrigin;
import org.springframework.web.bind.annotation.DeleteMapping;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.ResponseStatus;
import org.springframework.web.bind.annotation.RestController;

import tacos.Ingredient;
import tacos.data.IngredientRepository;

@RestController
@RequestMapping(path="/api/ingredients", produces="application/json")
@CrossOrigin(origins="*")
public class IngredientController {

    private IngredientRepository repo;

    @Autowired
    public IngredientController(IngredientRepository repo) {
        this.repo = repo;
    }

    @GetMapping
    public Iterable<Ingredient> allIngredients() {
        return repo.findAll();
    }

    @PostMapping
    @ResponseStatus(HttpStatus.CREATED)
    public Ingredient saveIngredient(@RequestBody Ingredient ingredient) {
        return repo.save(ingredient);
    }

    @DeleteMapping("/{id}")
    @ResponseStatus(HttpStatus.NO_CONTENT)
    public void deleteIngredient(@PathVariable("id") String ingredientId) {
        repo.deleteById(ingredientId);
    }
}
```

很好！现在我们需要做的就是开始管理应用程序，根据需要调用 Taco Cloud 应用程序上的接口添加或删除材料。

但是等等。这个 API 还没有任何安全性。如果我们的后端应用程序可以使 HTTP 请求添加和删除配料，其他人也可以。甚至使用 `curl` 命令行，就可以添加如下新配料：

```
$ curl localhost:8080/ingredients \
-H"Content-type: application/json" \
-d'{"id":"FISH","name":"Stinky Fish", "type":"PROTEIN"}'
```

甚至可以使用 `curl` 删除存在的配料：

```
$ curl localhost:8080/ingredients/GRBF -X DELETE
```

此 API 是主应用程序的一部分，可供任何人使用；事实上，GET 接口是由 `home.html` 中的用户界面使用的。因此，很明显，我们至少需要保护 POST 和 DELETE 接口。

一种选择是使用 HTTP 基本身份验证来保护接口。这可以通过向处理程序方法添加 `@PreAuthorize` 来完成，如下所示：

```
@PostMapping
@PreAuthorize("#{hasRole('ADMIN')}")
public Ingredient saveIngredient(@RequestBody Ingredient ingredient) {
    return repo.save(ingredient);
}

@DeleteMapping("/{id}")
@PreAuthorize("#{hasRole('ADMIN')}")
public void deleteIngredient(@PathVariable("id") String ingredientId) {
    repo.deleteById(ingredientId);
}
```

或者，也可以通过安全配置来保护接口：

```
@Override
protected void configure(HttpSecurity http) throws Exception {
    http
        .authorizeRequests()
            .antMatchers(HttpMethod.POST, "/ingredients").hasRole("ADMIN")
            .antMatchers(HttpMethod.DELETE, "/ingredients/**").hasRole("ADMIN")
            ...
}
```

注意：是否使用“ROLE\_”前缀？

Spring Security 中的权限可以采取多种形式，包括角色，权限，以及（稍后我们将看到）OAuth2 作用域。具体而言，角色是一种以“ROLE”为前缀的特殊形式的权限。当使用方法或 SpEL 表达式直接处理角色时，例如，`hasRole()` 方法会推断包含有“ROLE”前缀。因此，调用 `hasRole("ADMIN")` 会检查授权名称“ROLEADMIN”。所以这种情况下调用方法和函数时不需显式使用“ROLE”前缀（事实上，这样做会导致出现双份“ROLE”前缀）。更一般地说，Spring Security 其他处理权限的方法和功能也可以用于检查角色。但在这种情况下，您必须显式添加“ROLE”前缀。例如，如果您选择使用 `hasAuthority()` 而不是 `hasRole()`，您需要传入“ROLE\_ADMIN”而不是“ADMIN”。

无论哪种方式，向“/ingredients”提交 POST 或 DELETE 请求的能力，都需要提交者还提供具有“ROLE\_ADMIN”权限的凭据。例如，使用 `curl` 中，可以使用 `-u` 参数指定凭据，如下所示：

```
$ curl localhost:8080/ingredients \
-H"Content-type: application/json" \
-d'{"id":"FISH","name":"Stinky Fish", "type":"PROTEIN"}' \
-u admin:l3tm31n
```

虽然 HTTP Basic 会锁定 API，但它相当简陋。它要求客户端 API 共享用户凭证，这可能会使用凭据到处留存副本。此外，尽管 HTTP 基本凭据在请求的标头中是 Base64 编码的，但如果黑客以某种方式截获了请求，可以很容易地获得未编码的原始凭证，然后使用凭证进行非法操作。如果发生这种情况，就需要立即更改密码，而所有客户端中都要进行更新，并重新进行身份验证。

如果 API 不要求用户在每个请求上都标识自己，只是请求一些令牌来证明他们有权访问资源？这大致就像一张体育赛事的门票。要进入游戏，栅门处的人不需要知道您是谁；他们只需要知道您有一张有效的票。如果是这样的话，那么您就允许访问。

这大概就是 OAuth 2 授权的工作原理。客户端请求访问令牌 —— 类似于代客泊车钥匙 —— 来自授权服务器，并获得用户的明确许可。那个令牌允许他们代表授权用户与 API 交互。在任何时候，令牌可能会过期或被吊销，而无需更改用户的密码。在这种情况下，客户端只需要请求一个新的访问令牌就可以继续操作。该流程如图 8.1 所示。

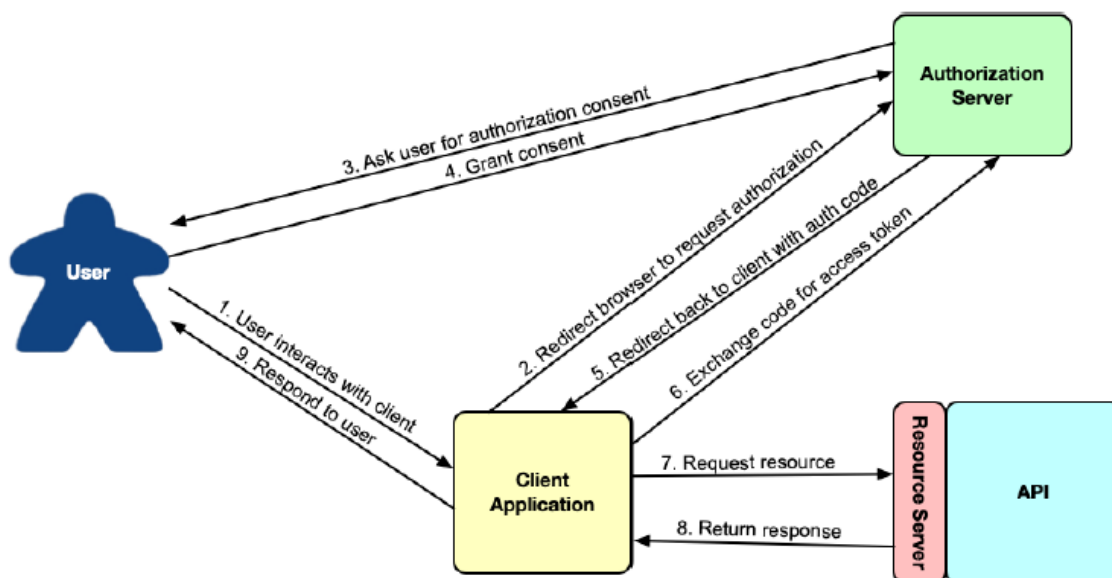


图 8.1 OAuth 2 的授权 Code 流。

OAuth 2 是一个非常丰富的安全规范，可以通过多种方式使用它。图 8.1 中 描述的被称为授权码授权。官方还支持的其他 OAuth 2 规范包括：

- 隐式授权：与授权码授权一样，隐式授权重定向到用户的浏览器到授权服务器以获得用户同意。但当重新定向回来时，除了在请求中提供授权码外，访问令牌也在请求中。虽然最初是为在浏览器中运行的 JavaScript 客户端设计的，但是通常不再建议使用，请优先考虑授权码授权。
- 用户凭据（或密码）授权：此种方式不会发生重定向，并且可能会发生错误，甚至连网络浏览器都没有。相反，客户端应用程序获取用户的凭据，并将其直接交换为访问令牌。这个流程似乎适合于不是基于浏览器的客户端，但现代应用程序通常倾向于要求用户在浏览器中访问网站并执行授权 Code 授权，以避免处理用户的凭据。
- 客户端凭据授权：类似于用户凭据授权，只不过不是将用户的凭据转换为访问令牌，而是客户端自己用凭据来申请访问令牌。但是，授予的令牌的范围仅限于执行非以用户为中心的操作，不能用于代表用户。

为实现我们的功能，我们重点关注获取 JSON 的授权 Code 流，授予 Web Token（JWT）访问令牌。这将涉及到创建一些可以在一起工作的应用程序，包括：

- 授权服务器：授权服务器的任务是，代表客户端应用程序从用户处获取权限。如果用户授予权限，则授权服务器向客户端应用程序提供一个访问令牌，然后就可以使用该令牌授权访问 PI。
- 资源服务器：资源服务器只是 API 的另一个名称，由 OAuth 2 进行保护。虽然资源服务器是 API 本身的一部分，但为了便于讨论，这两个概念通常被视为两个截然不同的概念。资源服务器限制对资源的访问，除非请求提供有效的访问令牌和必要的权限范围。我们的项目会在第 6 章中开始编写 Taco Cloud API，一旦我们添加一点安全配置，就可以作为我们的资源服务器。
- 客户端应用程序：客户端应用程序是希望使用 API 的程序，需要有权限才能执行此操作。我们将为 Taco Cloud 构建一个简单的管理应用程序，此程序能够添加新的配料。
- 用户：这是使用客户端应用程序并授予应用程序权限的人员，客户端程序是代表他们来访问资源服务器 API 的。

在授权 Code 流中，当客户端获得访问令牌时，应用程序和授权服务器之间存在一系列浏览器重定向。它从客户端开始，将用户浏览器重定向到授权服务器，请求特定权限（或“范围”）。然后，授权服务器要求用户登录并同意请求的授权权限。用户授予同意后，授权服务器将重定向浏览器返回到客户端，客户端可以使用 Code 交换访问令牌。一旦客户端有一个访问令牌，然后通过传递该令牌与资源服务器 API 进行交互，它位于每个请求 Header 的“Authorization”字段中。

虽然我们将把重点限制在 OAuth 2 的特定使用上，但我们鼓励您阅读 OAuth 2 规范，深入研究这个主题（<https://oauth.net/2/>）或阅读关于该主题的下列任何一本书：

- 《OAuth 2 实战》：[\[https://www.manning.com/books/oauth-2-in-action\]](https://www.manning.com/books/oauth-2-in-action)<https://www.manning.com/books/oauth-2-in-action>
- microservices security in action: [\[https://www.manning.com/books/microservices-security-in-action\]](https://www.manning.com/books/microservices-security-in-action)<https://www.manning.com/books/microservices-security-in-action>
- API Security in Action: [\[https://www.manning.com/books/api-security-in-action\]](https://www.manning.com/books/api-security-in-action)<https://www.manning.com/books/api-security-in-action>

您可能还想看看名为“Protecting User Data with Spring Security and OAuth 2”的 LiveProject。  
（<https://www.manning.com/liveproject/protecting-user-data-with-spring-security-and-oauth2>）。

几年来，有一个名为 Spring Security for OAuth 的项目，提供了对 OAuth 1.0a 和 OAuth 2 的支持。它独立于 Spring Security，但由相同的团队开发。近年来，Spring Security 团队吸收了客户和资源服务器组件到 Spring Security 中。

至于授权服务器，决定不将其包括在 Spring Security 中。相反，鼓励开发人员使用来自不同供应商的授权服务器，如 Okta，谷歌等。但是，由于开发者社区的需求，Spring Security 团队启动了一个 Spring Authorization Server 项目。该项目被标记为“experimental”项目，并且旨在最终由社区驱动。但这是一个开始试验 OAuth 2，无需注册其他授权服务器实现的好方式。

在本章的其余部分中，我们将看到 OAuth2 如何使用 Spring Security。在此过程中，我们将创建两个新项目，一个授权服务器项目和一个客户端项目，我们将修改现有的 Taco Cloud 项目，使其 API 充当资源服务器。我们将首先使用 Spring Authorization Server 创建一个授权服务器。

## 8.2 创建验证服务器

授权服务器的工作主要是代表用户发出访问令牌。像前面提到过，有几种授权服务器实现可供选择。但是我们将使用 **Spring Authorization Server**。**Spring Authorization Server** 是实验性的，并没有实现所有的 OAuth 2 授权类型，但它确实实现了授权 Code 授予和客户端凭据授予。

授权服务器是一个独立应用程序，不同于提供 API 的服务，也有别于客户端。因此，要开始使用 **Authorization Server**，您需要创建一个新的 **Spring Boot** 项目，选择（至少）**web** 和 **security starter**。对于我们的授权服务器，用户将使用 **JPA** 存储在关系数据库中，因此请确保添加 **JPA starter** 和 **H2** 依赖。如果您用 **Lombok** 来处理 **getter**、**setter**、**constructor** 等等，确保也包括它。

**Spring Authorization Server**（尚未）作为 **Initializr** 设定项提供。所以如果您的项目已经创建，您需要手动添加 **Spring Authorization Server** 依赖。例如，这里是 **pom.xml** 文件中您需要包含的 **Maven** 依赖项：

```
<dependency>
  <groupId>org.springframework.security.experimental</groupId>
  <artifactId>spring-security-oauth2-authorization-server</artifactId>
  <version>0.1.2</version>
</dependency>
```

下一步，因为我们将在开发机器上运行所有这些程序（至少现在是这样），所以您需要确保主 **Taco Cloud** 应用程序和授权服务器没有端口冲突。将以下条目添加到项目的 **application.yml** 文件，将使授权服务器运行在端口 9000 上：

```
server:
  port: 9000
```

现在，让我们深入了解授权服务器将使用的基本安全配置。清单 8.2 显示了一个非常简单的 **Spring** 安全配置类，它支持基于表单的登录，并要求对所有请求进行身份验证。

程序清单 8.2 基于表单登录的基本安全配置。

```

package tacos.authorization;
import org.springframework.context.annotation.Bean;
import org.springframework.security.config.annotation.web.builders.
    HttpSecurity;
import org.springframework.security.config.annotation.web.configuration.
    EnableWebSecurity;
import org.springframework.security.core.userdetails.UserDetailsService;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.web.SecurityFilterChain;

import tacos.authorization.users.UserRepository;

@EnableWebSecurity
public class SecurityConfig {

    @Bean
    SecurityFilterChain defaultSecurityFilterChain(HttpSecurity http)
        throws Exception {
        return http
            .authorizeRequests(authorizeRequests ->
                authorizeRequests.anyRequest().authenticated()
            )

            .formLogin()

            .and().build();
    }

    @Bean
    UserDetailsService userDetailsService(UserRepository userRepo) {
        return username -> userRepo.findByUsername(username);
    }

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }
}

```

注意，`UserDetailsService` 与 `TacoUserRepository` 一起工作，通过用户的用户名查找用户。为了只关注配置授权服务器本身，我们将跳过有关 `TacoUserRepository` 的细节，但只需说它看起来很像我们从第 3 章开始创建的基于 `Spring Data` 的存储库。

关于 `TacoUserRepository`，唯一值得注意的是（为了方便测试）您可以在 `CommandLineRunner` bean 中使用它，通过几个测试预填充用户到数据库：

```

@Bean
public ApplicationRunner dataLoader(
    UserRepository repo, PasswordEncoder encoder) {
    return args -> {
        repo.save(
            new User("habuma", encoder.encode("password"), "ROLE_ADMIN"));
        repo.save(
            new User("tacocheff", encoder.encode("password"), "ROLE_ADMIN"));
    };
}

```

现在，我们可以开始应用配置来启用授权服务器了。第一步，配置授权服务器就是创建一个新的配置类来导入一些授权服务器的通用配置。下面列出的 `AuthorizationServerConfig` 是一个良好的开端：

```

@Configuration(proxyBeanMethods = false)
public class AuthorizationServerConfig {

    @Bean
    @Order(Ordered.HIGHEST_PRECEDENCE)
    public SecurityFilterChain authorizationServerSecurityFilterChain(HttpSecurity http)
        throws Exception {
        OAuth2AuthorizationServerConfiguration
            .applyDefaultSecurity(http);
        return http
            .formLogin(Customizer.withDefaults())
            .build();
    }

    ...
}

```

`authorizationServerSecurityFilterChain()` 方法定义 `SecurityFilterChain`，为 OAuth 2 授权服务器设置一些默认行为和一个默认的表单登录页面。`@Order` 注解的优先级为 `Ordered.HIGHEST_PRECEDENCE`。为了确保由于某种原因，如果声明了此类型的其他 bean，则此 bean 将优先于 bean。

在大多数情况下，就是样板配置。但如果您愿意，可以对其进行更深入的分析并进行自定义配置。现在，我们就使用默认值。

有一个组件不是样板配置，因此不是由 `OAuth2AuthorizationServerConfiguration` 提供的，就是客户端存储库。客户端存储库是类似于用户详细信息服务或用户存储库，不同之处在于不是维护详细信息用户，而是为请求授权需要询问的客户。它由 `RegisteredClientRepository` 接口定义，该接口如下所示：

```

public interface RegisteredClientRepository {

    @Nullable
    RegisteredClient findById(String id);

    @Nullable
    RegisteredClient findByClientId(String clientId);

}

```

在生产环境中，您可以编写 `RegisteredClientRepository` 的实现类，从数据库或其他数据来源检索客户端详细信息。但是，Spring Authorization Server 提供了一个现成的内存实现，非常适合用于演示和测试目的。我们鼓励您实现适合自己的 `RegisteredClientRepository`。我们这里将使用内存中的实现，向授权服务器注册单个客户端。添加以下 bean 方法到 `AuthorizationServerConfig` 中：

```

@Bean
public RegisteredClientRepository registeredClientRepository(
    PasswordEncoder passwordEncoder) {
    RegisteredClient registeredClient =
        RegisteredClient.withId(UUID.randomUUID().toString())
            .clientId("taco-admin-client")
            .clientSecret(passwordEncoder.encode("secret"))
            .clientAuthenticationMethod(
                ClientAuthenticationMethod.BASIC)
            .authorizationGrantType(AuthorizationGrantType.AUTHORIZATION_CODE)
            .authorizationGrantType(AuthorizationGrantType.REFRESH_TOKEN)
            .redirectUri(
                "http://127.0.0.1:9090/login/oauth2/code/taco-admin-client")
            .scope("writeIngredients")
            .scope("deleteIngredients")
            .scope(OidcScopes.OPENID)
            .clientSettings(
                clientSettings -> clientSettings.requireUserConsent(true))
            .build();
    return new InMemoryRegisteredClientRepository(registeredClient);
}

```

正如您所看到的，**RegisteredClient** 中有很多信息。自上而下，我们的客户端是这样定义的：

- **ID**：一个随机的、唯一的标识符。
- **客户端ID**：类似于用户名，但不是用户，而是客户端。在这种情况下是“taco-admin-client”。
- **客户端密码**：类似于客户端的密码。这里我们用的“secret”这个词作为客户端机密。
- **授权授予类型**：此客户端将支持的 OAuth 2 授权类型。在这种情况下，我们启用授权 Code 和刷新令牌授权。
- **重定向URL**：在获得授权后，授权服务器可以重定向到的一个或多个已注册 URL。这增加了另一个安全级别，防止了任意应用程序无法接收授权码，而该授权码可用于交换令牌
- **作用域**：允许此客户端请求的一个或多个 OAuth 2 作用域。这里我们设置三个作用域：“writeIngredients”、“deleteIngredients”和常量 `OidcScopes.OPENID`，解析为“openid”。当稍后使用授权服务器作为 Taco Cloud 管理程序的单点登录解决方案时，“openid”范围是必需的。
- **客户端设置**：这是一个 `lambda`，允许我们自定义客户端设置。在这种情况下，在授予请求的范围之前，我们需要得到明确的用户同意。没有这个配置，授权范围将在用户登录后隐式授予。

最后，由于我们的授权服务器将生成 JWT 令牌，因此这些令牌需要使用 **JSON Web Key (JWK)** 作为签名密钥。因此，我们将需要一些 **bean** 来生成 JWK。添加以下 **bean** 方法（和私有 **helper** 方法）到 **AuthorizationServerConfig** 中就能为我们完成这件工作：



```

@Bean
public JWKSSource<SecurityContext> jwkSource() {
    RSAKey rsaKey = generateRsa();
    JWKSSet jwkSet = new JWKSSet(rsaKey);
    return (jwkSelector, securityContext) -> jwkSelector.select(jwkSet);
}

private static RSAKey generateRsa() {
    KeyPair keyPair = generateRsaKey();
    RSAPublicKey publicKey = (RSAPublicKey) keyPair.getPublic();
    RSAPrivateKey privateKey = (RSAPrivateKey) keyPair.getPrivate();
    return new RSAKey.Builder(publicKey)
        .privateKey(privateKey)
        .keyID(UUID.randomUUID().toString())
        .build();
}

private static KeyPair generateRsaKey() {
    try {
        KeyPairGenerator keyPairGenerator = KeyPairGenerator.getInstance("RSA");
        keyPairGenerator.initialize(2048);
        return keyPairGenerator.generateKeyPair();
    } catch (Exception e) {
        return null;
    }
}

@Bean
public JwtDecoder jwtDecoder(JWKSSource<SecurityContext> jwkSource) {
    return OAuth2AuthorizationServerConfiguration.jwtDecoder(jwkSource);
}

```

这里似乎比较复杂。但总而言之，**JWKSSource** 创建了 2048 位的 RSA 密钥对，用于对令牌进行签名。令牌将使用私钥进行签名。然后，资源服务器可以通过获取来自授权服务器的公钥来进行验证。我们将在创建资源服务器时作进一步讨论。

我们的授权服务器的所有部分现在都已就绪。剩下要做的就是试试启动它。构建并运行应用程序，您应该有一个正在侦听 9000 端口的授权服务器。

由于我们还没有客户端，您可以使用 **web** 浏览器和 **curl** 命令行工具。首先，将 **web** 浏览器指向 [http://localhost:9000/.../\\_/taco-admin-client&scope=writeIngredients+deleteIngredients](http://localhost:9000/.../_/taco-admin-client&scope=writeIngredients+deleteIngredients)。您将看到一个类似图 8.2 的登录页。

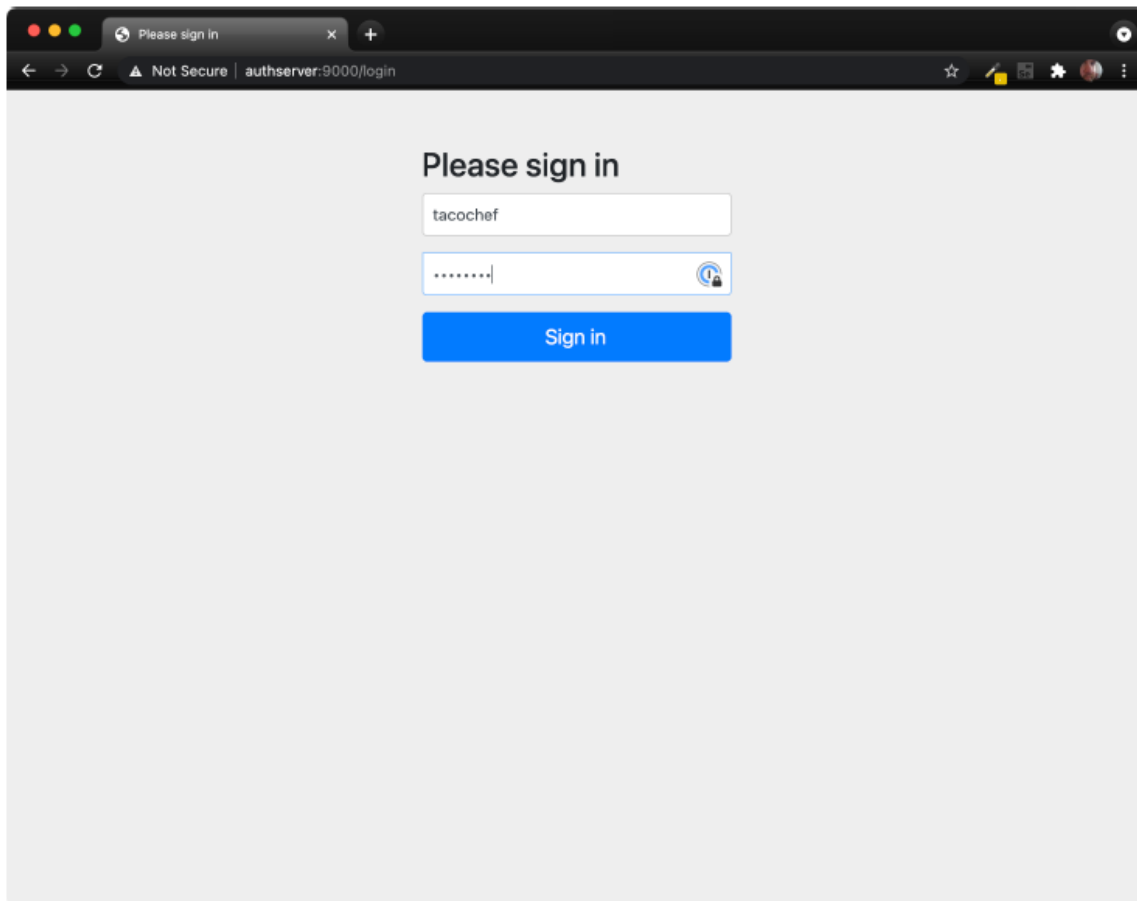


图 8.2 授权服务器登录页。

登录后（使用“tacocheff”和“password”或某些用户名/密码组合登录，只要 TacoUserRepository 下的数据库有对应数据），将要求您同意上请求的作用域，如图 8.3 所示。

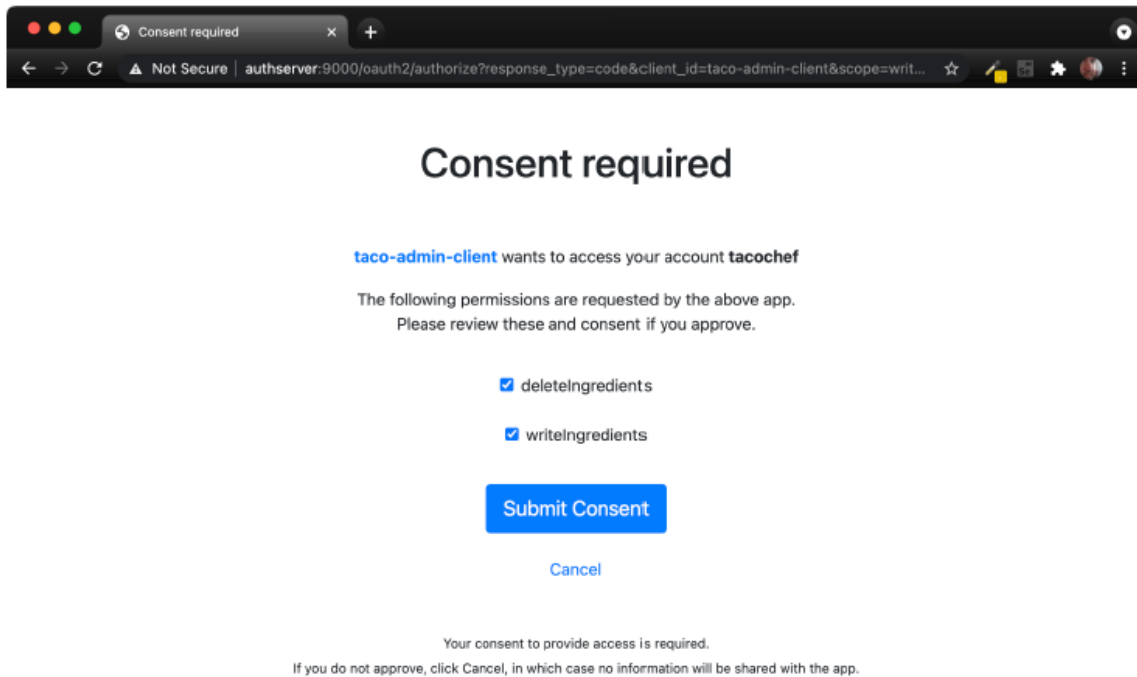


图 8.3 授权服务器同意页面。

授予同意后，浏览器将重定向回客户端 URL。既然我们没有客户端，那里可能什么都没有，您会收到一个错误。但没关系，我们假装是客户机，所以我们将自己从 URL 获取授权码。

查看浏览器的地址栏，您将看到 URL 有一个 `code` 参数。复制该参数的整个值，并在以下 `curl` 命令行中使用它来代替 `$code`：

```
$ curl localhost:9000/oauth2/token \
-H"Content-type: application/x-www-form-urlencoded" \
-d"grant_type=authorization_code" \
-d"redirect_uri=http://127.0.0.1:9090/login/oauth2/code/taco-admin-client" \
-d"code=$code" \
-u taco-admin-client:secret
```

在这里，我们将用收到的授权码交换访问令牌。有效载荷采用“`application/x-www-form-urlencoded`”格式，并发送授权类型（“`authorization_code`”）、重定向URI（用于附加安全性）和授权码。如果一切顺利，那么您将收到一个 JSON 响应（格式化后）如下所示：

```
{
  "access_token":"eyJraWQ...",
  "refresh_token":"HOzHA5s...",
  "scope":"deleteIngredients writeIngredients",
  "token_type":"Bearer",
  "expires_in":"299"
}
```

那么，我们如何使用访问令牌呢？我们大概会向 **Taco Cloud API** 发送请求，作为“Authorization”请求头的一部分。像这样：

```
$ curl localhost:8080/ingredients \
-H"Content-type: application/json" \
-H"Authorization: Bearer eyJraWQ..." \
-d{'id':"FISH","name":"Stinky Fish", "type":"PROTEIN"}
```



# JWT

[Debugger](#) [Libraries](#) [Introduction](#) [Ask](#) [Get a T-shirt](#)

[Crafted by](#)  [Auth0](#)

## Encoded

PASTE A TOKEN HERE

```
eyJraWQioiJhZGZlNGM5NC05ZGZiLlRiZmYtYjUyOC0yYmVhNmMzMTIzNzAiLCJ0eXAiOiJKV1QiLCJhbGciOiJSUzI1NiJ9.eyJzdWIiOiJ0YWNvY2hlZiIsImF1ZCI6InRhY28tYWRtaW4tY2xpZW50IiwibmJmIjoxNjI0OTk5NjE4LjZyZjZwZSI6WyJkZWxlZGVJbmdyZWRpZW50cyIsIndyaXRlSW5ncmVkaWVudHMlLCJvcGVuaWQiXSwiaXNzIjoiaHR0cDpcL1wvYXV0aHNlcnZlcj05MDAwIiwiaXhwIjoxNjI0OTk5OTE4LjYyYXQiojE2MjE5OTk2MTgsImp0aSI6ImFiZDkwZTk5LWVmODEtNGY0Zi04NjU4LWRRMmZiYyY2YzN2QyYiJ9.N5yTrB0AJ2MAT28hc60nnSLI_1tPpLWFVq0tPdPECMRjwSLJrw7353Wt4WrDc9lCqKKiu7bYczNWELeKSbV5Vz7KMa5YgPu6PQ22t6ap4VfnotZ0UazHCr2fclPc10RQ30XZCuhXmRkNu0gHJN5r9fhW2v_ffD7sRupbpnl6aD2TpxsHqm65eaQ-nMVnm2-J5phTlebxbnRbXzGNGJwHhsZzRTLZTvc6Nlc7kmtva4zCIXJhsWZpi2lS4nz5u02tEfDyPvKcA7H2MByz3PTwoxxUF9iUyPzIqRkEVP2HIJmvti4kDFmA6w-WQxaxa3_yJcIYGfkr-62S-CZ47g
```

## Decoded

EDIT THE PAYLOAD AND SECRET

### HEADER: ALGORITHM & TOKEN TYPE

```
{
  "kid": "adfe4c94-9dfb-4bff-b528-2bea6c312378",
  "typ": "JWT",
  "alg": "RS256"
}
```

### PAYLOAD: DATA

```
{
  "sub": "tacocheff",
  "aud": "taco-admin-client",
  "nbf": 1621999618,
  "scope": [
    "deleteIngredients",
    "writeIngredients",
    "openid"
  ],
  "iss": "http://authserver:9000",
  "exp": 1621999918,
  "iat": 1621999618,
  "jti": "abd90e99-af81-4f4f-8658-df322cf37d1b"
}
```

### VERIFY SIGNATURE

```
RSASHA256(
  base64UrlEncode(header) + "." +
  base64UrlEncode(payload),
  Public Key or Certificate. Enter it in plain text only if you
```

如您所见，令牌被解码为三个部分：报头、有效负载和签名。仔细查看有效负载可以发现，该令牌是代表名为“tacoche”的用户，令牌具有“WriteElements”和“DeleteComponents”作用域。正是我们想要的！

224

```
$ curl localhost:9000/oauth2/token \  
-H"Content-type: application/x-www-form-urlencoded" \  
-d"grant_type=refresh_token&refresh_token=H0zHA5s..." \  
-u taco-admin-client:secret
```

对此请求的响应，与使用授权码交换初始访问令牌的请求返回，是基本相同的。仅多了新的访问令牌。

虽然将访问令牌粘贴到 <https://jwt.io> 很有趣，但访问令牌真正的威力和目的是用于访问 **API**。让我们看看如何在 **Taco Cloud API** 上启用资源服务器。

## 8.3 利用资源服务器保护 API

资源服务器实际上只是一个位于 API 前面的过滤器，确保需要授权的资源包括具有所需范围的有效访问令牌。Spring Security 提供了一个 OAuth2 资源服务器实现，您可以将其添加到现有的 API，将以下依赖项添加到 `tacocloud-security` 项目的构建中：

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-oauth2-resource-server</artifactId>
</dependency>
```

您还可以通过选择“OAuth2 Resource Server”来添加资源服务器依赖项，如果创建项目时使用的是 Initializr。

有了依赖关系后，下一步是声明发送到 `/ingredients` 的 POST 请求需要“`writeIngredients`”作用域，并且对 `/ingredients` 的 DELETE 请求需要“`deleteIngredients`”作用域。下面摘录的是来自 `tacocloud-security` 项目的 `SecurityConfig` 类，显示了如何完成此配置：

```
@Override
protected void configure(HttpSecurity http) throws Exception {
    http
        .authorizeRequests()
        ...
        .antMatchers(HttpMethod.POST, "/api/ingredients")
            .hasAuthority("SCOPE_writeIngredients")
        .antMatchers(HttpMethod.DELETE, "/api//ingredients")
            .hasAuthority("SCOPE_deleteIngredients")
        ...
}
```

对于每个接口，都会调用 `.hasAuthority()` 方法来指定所需的作用域。请注意，作用域的前缀为“SCOPE\_”，表示它们应该匹配针对请求这些资源时给出的访问令牌中的 OAuth 2 作用域。

在这个配置类中，我们还需要启用资源服务器：

```
@Override
protected void configure(HttpSecurity http) throws Exception {
    http
        ...
        .and()
            .oauth2ResourceServer(oauth2 -> oauth2.jwt())
        ...
}
```

这里为 `oauth2ResourceServer()` 方法提供了一个 lambda，用于配置资源服务器。在这里，它只是启用 JWT 令牌（与不透明令牌相反），以便资源服务器可以检查令牌的内容，以查找它包含哪些安全声明。具体来说，它将查看令牌是否包括“`writeIngredients`”和/或“`deleteIngredients`”作用域，这是我们所保护的两个接口所适用的作用域。

不过，它不会按设置值直接信任令牌。要确保令牌是由受信的授权服务器生成的，它将使用与创建令牌签名私钥匹配的公钥来进行验证。我们需要配置资源服务器知道从何处获取公钥，请执行以下操作。以下属性将指定资源服务器访问的授权服务器上的 JWK 的 URL，并从那里获取公钥：

```
spring:
  security:
    oauth2:
      resourceserver:
        jwt:
          jwk-set-uri: http://localhost:9000/oauth2/jwks
```

现在我们的资源服务器已经准备好了！构建 **Taco Cloud** 应用程序并启动它。您可以像这样使用 **curl** 进行尝试：

```
$ curl localhost:8080/ingredients \
-H"Content-type: application/json" \
-d'{"id":"CRKT", "name":"Legless Crickets", "type":"PROTEIN"}'
```

请求应该会失败，并返回 **HTTP 401** 响应码。这是因为我们已经配置了接口需要“**writeIngredients**”作用域，但我们在请求上尚未提供有效作用域的访问令牌。

要想请求成功并添加新的配料，您需要使用我们在上一节中使用的流，获得相应的访问令牌，确保我们的请求在将浏览器转到授权服务器时具有“**writeIngredients**”和“**deleteIngredients**”作用域。然后，像这样使用 **curl** 在“**Authorization**”请求头中提供访问令牌（用“**\$token**”代替实际访问令牌）：

```
$ curl localhost:8080/ingredients \
-H"Content-type: application/json" \
-d'{"id":"SHMP", "name":"Coconut Shrimp", "type":"PROTEIN"}' \
-H"Authorization: Bearer $token"
```

这一次应该会创建出新的配料。您可以使用 **curl** 或 **HTTP** 客户端以执行对接口的 **GET** 请求：

```
$ curl localhost:8080/ingredients
[
  {
    "id": "FLT0",
    "name": "Flour Tortilla",
    "type": "WRAP"
  },
  ...
  {
    "id": "SHMP",
    "name": "Coconut Shrimp",
    "type": "PROTEIN"
  }
]
```

“椰子虾”配料，现在被列在从该工厂返回的所有配料清单的末尾。新建确实成功了！

我们知道访问令牌 5 分钟后就会过期。如果令牌过期，请求会再次返回 **HTTP 401** 响应码。您可以使用与访问令牌一起使用的刷新令牌，通过向授权服务器发出请求来获取新的访问令牌（将实际刷新令牌替换为“**\$refreshToken**”）：

```
$ curl localhost:9000/oauth2/token \
-H"Content-type: application/x-www-form-urlencoded" \
-d"grant_type=refresh_token&refresh_token=$refreshToken" \
-u taco-admin-client:secret
```

有了一个新创建的访问令牌，您可以继续为随心所欲的创建新的配料了。

既然我们已经保护了 `/ingredients` 接口，那么就应该对其他 API 也使用相同的技术来进行保护了。例如，`/orders` 接口不应该让任何类型的请求打开，即使是 HTTP GET 请求，因为这将允许黑客轻松获取客户信息。我让您自己来处理对其他 API 接口的安全保护工作。

使用 `curl` 管理 `tacocloud` 应用程序只适合简单修补，以了解 OAuth 2 令牌是如何在资源保护方面发挥作用的。但最终我们想要的是一个可用于管理配料的真实客户端应用程序。现在我们来关注一下如何创建 OAuth 客户端，该客户端将获取访问令牌并向 API 发出请求。



## 8.4 开发客户端

在 OAuth 2 授权过程中，客户端应用程序的角色是获取访问令牌和代表用户向资源服务器发出请求。因为我们使用的是 OAuth 2 授权代码流，这意味着当客户端应用程序确定用户尚未通过身份验证时，它应该将用户的浏览器重定向到授权服务器以获取用户的同意。然后，当授权服务器将控制重定向回客户端时，客户端必须将收到的授权码交换访问令牌。

第一件事：客户端在其类路径中需要 Spring Security 的 OAuth 2 客户端支持。以下 starter 依赖项完成这件工作：

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-oauth2-client</artifactId>
</dependency>
```

这不仅为应用程序 OAuth 2 客户端提供了我们稍后将利用的功能，它也带来了 Spring Security 本身。这使我们能够为应用程序编写一些安全配置。下面的 SecurityFilterChain bean 设置 Spring Security，以便所有请求都需要身份验证：

```
@Bean
SecurityFilterChain defaultSecurityFilterChain(HttpSecurity http) throws Exception {
    http
        .authorizeRequests(
            authorizeRequests -> authorizeRequests.anyRequest().authenticated()
        )
        .oauth2Login(
            oauth2Login ->
                oauth2Login.loginPage("/oauth2/authorization/taco-admin-client")
        )
        .oauth2Client(withDefaults());
    return http.build();
}
```

此外，这个 SecurityFilterChain bean 还增加了一些 OAuth 2 客户端功能。具体来说，它在路径“/oauth2/authorization/taco admin client”处设置登录页面。但是这个不是一个普通的需要用户名和密码的登录页面。相反，它接受授权码，将其交换为访问令牌，并使用访问令牌确定使用者。换句话说，这是授权服务器在用户访问授权后，将重定向到的路径。

我们还需要配置有关授权服务器和应用程序的 OAuth 2 的客户详细信息。这是在配置属性中完成的。例如下面的 application.yml 中配置名为“taco-admin-client”的客户端文件：

```
spring:
  security:
    oauth2:
      client:
        registration:
          taco-admin-client:
            provider: tacocloud
            client-id: taco-admin-client
            client-secret: secret
            authorization-grant-type: authorization_code
            redirect-uri: "http://127.0.0.1:9090/login/oauth2/code/{registrationId}"
            scope: writeIngredients,deleteIngredients,openid
```

这将注册一个名为“aco-admin-client”的 Spring Security OAuth 2 客户端。这个注册详细信息包括客户的凭据（client-id 和 client-secret）、授权类型（authorization-grant-type）、请求的作用域（scope）和重定向 URI（redirect-uri）。请注意，给 redirect-uri 的值有一个占位符，引用客户端的 ID，即“taco-admin-client”。因此，重定向 URI 设置为 `""http://127.0.0.1:9090/login/oauth2/code/taco-admin-client"`，这与我们先前已配置的 OAuth 2 登录地址相同。

但是授权服务器本身呢？我们应该在哪里告诉客户应该将用户的浏览器重定向到哪里？provider 属性就做这个的，尽管是间接的。这个 provider 属性设置为“tacocloud”，这是对一组单独配置的引用。它描述了“tacocloud”的授权服务器。该配置在同一 application.yml 文件中配置如下：

```
spring:
  security:
    oauth2:
      client:
        ...
      provider:
        tacocloud:
          issuer-uri: http://authserver:9000
```

提供程序配置所需的唯一属性是 issuer-uri 此属性标识授权服务器的基本 URI。在本例中，它指的是名称为“authserver”的服务器主机。假设您在本地运行这些示例，这只是“localhost”的另一个别名。在大多数基于 Unix 的操作系统上，这可以添加以下行到 /etc/hosts 文件中：

```
127.0.0.1 authserver
```

如果修改 /etc/hosts 在您的计算机上不起作用，请参阅操作系统的文档，查找如何创建自定义主机的详细信息。

基于基本 URL，Spring Security 的 OAuth 2 客户端，将假定授权 URL、令牌 URL 和其他授权服务器配置信息都使用默认值。但是，如果您正在使用的授权服务器不想使用这些默认值，您可以明确地配置授权详细信息，如下所示：

```
spring:
  security:
    oauth2:
      client:
        provider:
          tacocloud:
            issuer-uri: http://authserver:9000
            authorization-uri: http://authserver:9000/oauth2/authorize
            token-uri: http://authserver:9000/oauth2/token
            jwk-set-uri: http://authserver:9000/oauth2/jwks
            user-info-uri: http://authserver:9000/userinfo
            user-name-attribute: sub
```

我们已经看到过这些 URI 中的大多数，例如授权、令牌和 JWK 集 URI。然而，user-info-uri 是新的。这个 URI 用来获取基本用户信息，尤其是用户的用户名。对该 URI 的请求应返回 JSON，包含 user-name-attribute 属性以标识用户。但是，请注意，在使用 Spring Authorization Server 时，不需要创建该 URI 的端点；Spring Authorization Server 将自动地公开用户信息端点。

现在，应用程序进行权限验证，以及从授权服务器获取访问令牌的所有部分都已就绪。不需要做更多的事情，您就可以启动应用程序。向应用程序上的任何 URL 发出请求，将重定向到授权服务器以进行权限申请，当授权服务器重定向回来时，Spring Security's OAuth 2 的内部将在重定向中接收到的授权码交换访问令牌。现在，我们如何使用该令牌呢？

假设我们有一个服务，它使用 `RestTemplate` 调用 Taco Cloud API。下面的 `RestIngredientService` 实现展示了这样一个类。类中提供了两种方法：一个用于获取配料列表，另一种用于保存新的配料：

```
package tacos;

import java.util.Arrays;
import org.springframework.web.client.RestTemplate;

public class RestIngredientService implements IngredientService {

    private RestTemplate restTemplate;

    public RestIngredientService() {
        this.restTemplate = new RestTemplate();
    }

    @Override
    public Iterable<Ingredient> findAll() {
        return Arrays.asList(restTemplate.getForObject(
            "http://localhost:8080/api/ingredients",
            Ingredient[].class));
    }

    @Override
    public Ingredient addIngredient(Ingredient ingredient) {
        return restTemplate.postForObject(
            "http://localhost:8080/api/ingredients",
            ingredient,
            Ingredient.class);
    }
}
```

对 `/ingredients` 端点的 HTTP GET 请求是不安全的，因为只要 Taco Cloud API 在本地主机端口 8080 上侦听，`findAll()` 方法就可以正常工作。但是 `addIngredient()` 方法可能会因 HTTP 401 响应而失败，因为我们已保护 `/ingredients`，要求“`writesIngredients`”作用域。唯一的办法是在请求头的 `Authorization` 字段中提交具有“`writesIngredients`”作用域的访问令牌。

幸运的是，Spring Security 的 OAuth 2 客户端应该在完成授权码流之后就有了访问令牌。我们需要做的就是确保访问令牌在请求中出现。为此，我们将构造函数修改一下，把拦截器附加到它创建的 `RestTemplate` 上：

```

public RestIngredientService(String accessToken) {
    this.restTemplate = new RestTemplate();
    if (accessToken != null) {
        this.restTemplate
            .getInterceptors()
            .add(getBearerTokenInterceptor(accessToken));
    }
}

private ClientHttpRequestInterceptor
    getBearerTokenInterceptor(String accessToken) {
    ClientHttpRequestInterceptor interceptor =
        new ClientHttpRequestInterceptor() {
        @Override
        public ClientHttpResponse intercept(
            HttpRequest request, byte[] bytes,
            ClientHttpRequestExecution execution) throws IOException {
            request.getHeaders().add("Authorization", "Bearer " + accessToken);
            return execution.execute(request, bytes);
        }
    };
    return interceptor;
}

```

构造函数现在接受一个 **String** 参数，以传入访问令牌。使用这个令牌，它附加一个客户端请求拦截器，该拦截器将 **Authorization** 请求头添加到每个请求中，通过 **RestTemplate**，请求头的值为“Bearer”，后边跟着令牌。为了保持构造函数整洁，客户机拦截器是在单独的私有方法中创建的。

只剩下一个问题：访问令牌来自哪里？下面的 **bean** 方法是这件事发生的地方：

```

@Bean
@RequestScope
public IngredientService ingredientService(
    OAuth2AuthorizedClientService clientService) {
    Authentication authentication =
        SecurityContextHolder.getContext().getAuthentication();

    String accessToken = null;

    if (authentication.getClass()
        .isAssignableFrom(OAuth2AuthenticationToken.class)) {
        OAuth2AuthenticationToken oauthToken =
            (OAuth2AuthenticationToken) authentication;
        String clientRegistrationId =
            oauthToken.getAuthorizedClientRegistrationId();
        if (clientRegistrationId.equals("taco-admin-client")) {
            OAuth2AuthorizedClient client =
                clientService.loadAuthorizedClient(
                    clientRegistrationId, oauthToken.getName());
            accessToken = client.getAccessToken().getTokenValue();
        }
    }
    return new RestIngredientService(accessToken);
}

```

首先，请注意 **bean** 使用了 `@RequestScope` 注解声明具有请求作用域功能。这意味着每个请求都将创建一个新的 **bean** 实例。**bean** 必须是请求作用域，因为它需要从 **SecurityContext** 拉取授权。这是由 **Spring Security** 的一个过滤器在每次请求时填充的。应用程序启动时没有 **SecurityContext**，创建的是默认作用域的 **bean**。

在返回 `RestIngredientService` 实例之前，`bean` 方法检查身份验证情况，这是作为 `OAuth2AuthenticationToken` 实现的。如果通过，那就意味着它将拥有令牌。然后验证令牌是否用于名为“`taco-admin-client`”的客户端。如果是这样，则从授权客户端提取令牌并传递给 `RestIngredientService` 的构造函数。有了令牌，`RestIngredientService` 在向 Taco Cloud API 发出请求时不会遇到任何问题，这也代表用户正确授权了应用程序。

## 8.5 小结

- 使用 OAuth 2 是保护 API 的常用方法，它比简单的 HTTP 基本身份验证更健壮。
- 授权服务器为客户端颁发访问令牌，以便在以下情况下代表用户行事：向 API 发出请求（或在客户端令牌流的情况下代表 API）。
- 资源服务器位于 API 前面，以验证是否存在有效的、未过期的令牌，并提供访问 API 资源所需的作用域。
- Spring Authorization Server 是一个实现 OAuth 2 的实验性质的授权服务器。
- Spring Security 支持创建资源服务器和创建客户端。从授权服务器获取访问令牌，并通过资源服务器发出请求。

## 第 9 章 发送异步消息

本章内容：

- 异步消息
- 使用 JMS、RabbitMQ 和 Kafka 发送消息
- 从 Broker 拉取消息
- 监听消息

现在是星期五下午 4 点 55 分。您还有几分钟就要开始期待已久的假期了。虽然您有足够的时间开车去机场赶飞机，但是在您打包好行李准备离开的时候，您需要确保您的老板和同事们知道您当前工作的状态，这样下周一他们可以从您中断的地方接着做。不幸的是，您的一些同事已经提前离开去过周末了，并且您的老板一直在开会。您该怎么做呢？

最实用的办法就是，向老板和同事发一封简短的电子邮件，详细说明您的工作进展情况，并承诺寄张明信片。这样做既能传达您的工作的进度，又能赶上飞机。您不知道他们在哪里，也不知道他们什么时候会读这封邮件，但您知道他们最终会回到自己的办公桌上读到这封邮件。与此同时，您正在去机场的路上。

*同步* 通信有它的适用场景，这是我们在 **REST** 中所看到的。但这并不是开发人员可以使用的惟一的应用程序间通信方式。*异步* 消息传递是一种间接地将消息从一个应用程序发送到另一个应用程序而无需等待响应的方式。这种间接方式解耦了相互通信的应用程序，并带来了更好的伸缩性。

在本章中，将使用异步消息传递，把订单从 **Taco Cloud** 网站发送到另一个独立应用程序 **Taco Cloud** 厨房，在那里将准备 **tacos**。我们分别考虑 **Spring** 为异步消息传递提供的三个选项：**Java** 消息服务（**JMS**）、**RabbitMQ** 和高级消息队列协议（**AMQP**）以及 **Apache Kafka**。除了基本的消息发送和接收之外，我们还将了解 **Spring** 对消息驱动 **POJO** 的支持：一种类似于 **EJB** 的消息驱动 **bean**（**MDB**）的消息接收方式。

## 9.1 使用 JMS 发送消息

JMS 是一个 Java 标准，它定义了一个用于使用消息代理的公共 API。自 2001 年首次引入以来，JMS 一直是 Java 中异步消息传递的首选方法。在 JMS 之前，每个消息代理都有一个专用 API，这使得应用程序的消息代码在代理之间的可移植性更差。但是有了 JMS，所有兼容的实现都可以通过公共接口进行处理，这与 JDBC 为关系数据库操作提供公共接口的方式非常相似。

Spring 通过称为 `JmsTemplate` 的基于模板的抽象来支持 JMS。使用 `JmsTemplate`，很容易从生产者端跨队列和主题发送消息，并在消费者端接收这些消息。Spring 还支持消息驱动 POJO 的概念：简单的 Java 对象以异步方式对队列或主题上到达的消息做出响应。

我们将探讨 Spring 的 JMS 支持，包括 `JmsTemplate` 和消息驱动 POJO。我们的重点是 Spring 对 JMS 消息发送的支持，但是如果您想知道更多关于 JMS 的信息，可以阅读由 Bruce Snyder、Dejan Bosanac 和 Rob Davies 合著的《ActiveMQ 实战》一书（Manning 出版社，2011）。

在可以发送和接收消息之前，需要一个消息代理，它可以在生产者和消费者之间传递这些消息。让我们通过在 Spring 中设置消息代理来开始对 Spring JMS 的探索。



## 9.1.1 设置 JMS

在使用 JMS 之前，必须将 JMS 客户端添加到项目的构建中。使用 Spring Boot，这个过程简单的不能再简单了，需要做的仅仅是将 **starter** 依赖添加到构建中。但是，首先必须决定是使用 Apache ActiveMQ，还是使用较新的 Apache ActiveMQ Artemis Broker。

如果使用 ActiveMQ，需要添加以下依赖到项目的 pom.xml 文件中：

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-activemq</artifactId>
</dependency>
```

如果选择 ActiveMQ Artemis，starter 如下所示：

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-artemis</artifactId>
</dependency>
```

使用 Spring Initializr（或 IDE 附带的 Initializr 前端页面）时，还可以选择以下选项中的任何一个：**Spring for Apache ActiveMQ 5**、**Spring for Apache ActiveMQ Artemis**，如图 9.1 所示，这是 <https://start.spring.io> 网页的屏幕截图。

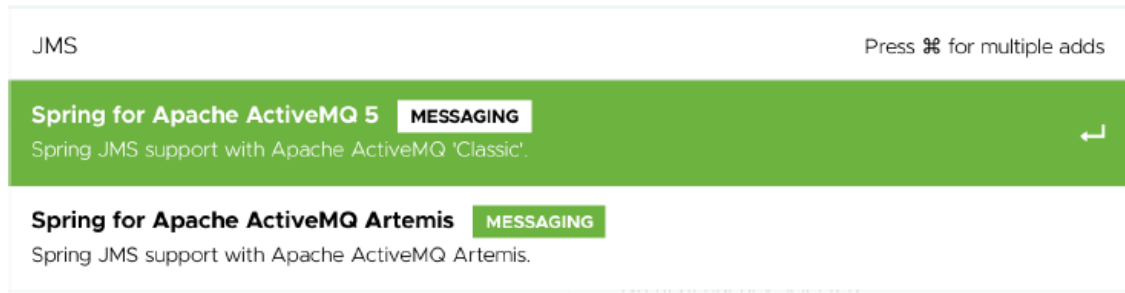


图 9.1 Spring Initializr 中提供的 ActiveMQ 和 Artemis 选项。

Artemis 是 ActiveMQ 的下一代重新实现，实际上这让 ActiveMQ 成为一个遗留选项。因此，对于 Taco Cloud，将选择 Artemis。但是，这种选择最终对如何编写发送和接收消息的代码几乎没有影响。唯一显著的区别在于如何配置 Spring 来创建与 Broker 的连接。

提示：启动一个 Artemis 代理。

您需要启动一个 Artemis 代理才能运行这一章中提供的代码。如果您还没有运行 Artemis 实例，您可以按照 Artemis 文档中的说明进行操作 <https://activemq.apache.org/latest/using-server.html>。

默认情况下，Spring 假设 Artemis Broker 正在监听 localhost 的 61616 端口。对于开发目的，这是可以的，但是一旦准备好将应用程序发送到生产环境中，就需要设置一些属性来告诉 Spring 如何访问代理。表 9.1 列出了最有用的属性。

表 9.1 用于配置 Artemis 代理的位置和凭据属性

| 属性                                   | 描述                  |
|--------------------------------------|---------------------|
| <code>spring.artemis.host</code>     | broker 主机           |
| <code>spring.artemis.port</code>     | broker 端口           |
| <code>spring.artemis.user</code>     | 用于访问 broker 的用户（可选） |
| <code>spring.artemis.password</code> | 用于访问 broker 的密码（可选） |

例如，考虑应用程序中的以下条目。可能用于非开发设置的 yml 文件：

```
spring:
  artemis:
    host: artemis.tacocloud.com
    port: 61617
    user: tacoweb
    password: 13tm31n
```

这将设置 Spring，以创建到监听 `artemis.tacocloud.com`（端口 61617）的 Artemis Broker 的 broker 连接。它还设置将与该 broker 交互的应用程序的凭据，凭据是可选的，但建议用于生产部署。

如果要使用 ActiveMQ 而不是 Artemis，则需要使用表 9.2 中列出的 ActiveMQ 特定的属性。

表 9.2 用于配置 ActiveMQ 代理的位置和凭据属性

| 属性                                      | 描述                     |
|-----------------------------------------|------------------------|
| <code>spring.activemq.broker-url</code> | Broker 的 URL           |
| <code>spring.activemq.user</code>       | 用于访问 Broker 的用户（可选）    |
| <code>spring.activemq.password</code>   | 用于访问 Broker 的密码（可选）    |
| <code>spring.activemq.in-memory</code>  | 是否启动内存 Broker（默认：true） |

请注意，不是为 Broker 的主机名和端口提供单独的属性，而是使用单个属性 `spring.activemq.broker-url` 指定 ActiveMQ Broker 的地址。URL 应该是 `tcp://` URL，如下面的 YAML 片段所示：

```
spring:
  activemq:
    broker-url: tcp://activemq.tacocloud.com
    user: tacoweb
    password: 13tm31n
```

无论选择 Artemis 还是 ActiveMQ，当 Broker 在本地运行时，都不需要为开发环境配置这些属性。

但是，如果使用 ActiveMQ，则需要设置 `spring.activemq.in-memory` 属性为 `false`，以防止 Spring 启动内存中的 Broker。内存中的 Broker 可能看起来很有用，但它只在发布和消费同一个应用的消息时有用（这一点用处有限）。

在继续之前，将希望安装并启动一个 Artemis（或 ActiveMQ）Broker，而不是使用嵌入式 Broker。与其在这里重复安装说明，我建议您参考 Broker 文档了解详细信息：

- *Artemis* —— <https://activemq.apache.org/artemis/docs/latest/using-server.html>
- *ActiveMQ* —— <http://activemq.apache.org/getting-started.html#GettingStarted-PreInstallationRequirements>

有了构建中的 **JMS starter** 和等待将消息从一个应用程序传递到另一个应用程序的 **Broker**，就可以开始发送消息了。

## 9.1.2 使用 JmsTemplate 发送消息

在构建中有 JMS starter 依赖（无论 Artemis 还是 ActiveMQ），Spring Boot 将会自动配置 JmsTemplate，这样就可以将其注入并使用它发送和接收消息了。

JmsTemplate 是 Spring JMS 集成支持的核心。与 Spring 的其他面向模板的组件非常相似，JmsTemplate 消除了大量与 JMS 协同工作所需的样板代码。如果没有 JmsTemplate，将需要编写代码来创建与消息代理的连接和会话，并编写更多代码来处理在发送消息过程中可能抛出的任何异常。JmsTemplate 专注于真正想做的事情：发送消息。

JmsTemplate 有几个发送消息的有用方法，包括：

```
// 发送原始消息
void send(MessageCreator messageCreator) throws JmsException;
void send(Destination destination, MessageCreator messageCreator)
    throws JmsException;
void send(String destinationName, MessageCreator messageCreator)
    throws JmsException;

// 发送转换自对象的消息
void convertAndSend(Object message) throws JmsException;
void convertAndSend(Destination destination, Object message)
    throws JmsException;
void convertAndSend(String destinationName, Object message)
    throws JmsException;

// 发送经过处理后从对象转换而来的消息
void convertAndSend(Object message,
    MessagePostProcessor postProcessor) throws JmsException;
void convertAndSend(Destination destination, Object message,
    MessagePostProcessor postProcessor) throws JmsException;
void convertAndSend(String destinationName, Object message,
    MessagePostProcessor postProcessor) throws JmsException;
```

实际上只有两个方法，`send()` 和 `convertAndSend()`，每个方法都被重载以支持不同的参数。如果仔细观察，会发现 `convertAndSend()` 的各种形式可以分为两个子类。在试图理解所有这些方法的作用时，请考虑以下细分：

- `send()` 方法需要一个 `MessageCreator` 来制造一个 `Message` 对象。
- `convertAndSend()` 方法接受一个 `Object`，并在后台自动将该 `Object` 转换为一条 `Message`。
- 三种 `convertAndSend()` 方法会自动将一个 `Object` 转换成一条 `Message`，但也会接受一个 `MessagePostProcessor`，以便在 `Message` 发送前对其进行定制。

此外，这三个方法类别中的每一个都由三个重载的方法组成，它们是通过指定 JMS 目的地（队列或主题）的方式来区分的：

- 一个方法不接受目的地参数，并将消息发送到默认目的地。
- 一个方法接受指定消息目的地的 `Destination` 对象。
- 一个方法接受一个 `String`，该 `String` 通过名称指定消息的目的地。

要使这些方法工作起来，请考虑下面程序清单中的 `JmsOrderMessagingService`，它使用 `send()` 方法的最基本形式。

程序清单 9.1 使用 `.send()` 发送订到到默认目的地

```

package tacos.messaging;
import javax.jms.JMSException;
import javax.jms.Message;
import javax.jms.Session;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.jms.core.JmsTemplate;
import org.springframework.jms.core.MessageCreator;
import org.springframework.stereotype.Service;

@Service
public class JmsOrderMessagingService implements OrderMessagingService {
    private JmsTemplate jms;

    @Autowired
    public JmsOrderMessagingService(JmsTemplate jms) {
        this.jms = jms;
    }

    @Override
    public void sendOrder(TacoOrder order) {
        jms.send(new MessageCreator() {
            @Override
            public Message createMessage(Session session)
                throws JMSException {
                return session.createObjectMessage(order);
            }
        });
    }
}

```

`sendOrder()` 方法调用 `jms.send()`，传递 `MessageCreator` 的匿名内部类实现。该实现重写 `createMessage()` 以从给定的 `Order` 对象创建新的对象消息。

因为特定于 JMS 的 `JmsOrderMessagingService` 实现了更通用的 `OrderMessagingService` 接口，我们可以通过将此服务注入 `OrderApiController` 并在创建订单时调用 `sendOrder()`：

```

@RestController
@RequestMapping(path="/api/orders",
    produces="application/json")
@CrossOrigin(origins="*")
public class OrderApiController {

    private OrderRepository repo;
    private OrderMessagingService messageService;

    public OrderApiController(
        OrderRepository repo,
        OrderMessagingService messageService) {
        this.repo = repo;
        this.messageService = messageService;
    }

    @PostMapping(consumes="application/json")
    @ResponseStatus(HttpStatus.CREATED)
    public TacoOrder postOrder(@RequestBody TacoOrder order) {
        messageService.sendOrder(order);
        return repo.save(order);
    }

    ...
}

```

现在，当您通过 **Taco Cloud** 网站创建订单时，应向代理消息，以便其他应用程序接收订单信息。不过我们的程序还没法接收消息，如果想收到这条信息。您也可以使用 **Artemis** 控制台查看消息队列数据。请参阅 **Artemis** 文档 <https://activemq.apache.org/components/artemis/documentation/latest/management-console.html> 以了解如何执行此操作的详细信息。

我不太清楚您怎么想，但我认为清单 9.1 中的代码虽然简单，但有点笨拙。代码中包括声明匿名内部类，但只是提供给另一个类进行简单的方法调用。如果发现 **MessageCreator** 是一个函数式接口，您可以使用 **lambda** 表达式使 `sendOrder()` 方法看起来更简洁：

```

@Override
public void sendOrder(TacoOrder order) {
    jms.send(session -> session.createObjectMessage(order));
}

```

但是请注意，对 `jms.send()` 的调用没有指定目的地。为了实现这一点，还必须使用 `spring.jms.template.default-destination` 属性指定一个默认的目的地名称。例如，可以在 `application.yml` 中设置属性：

```

spring:
  jms:
    template:
      default-destination: tacocloud.order.queue

```

在许多情况下，使用缺省目的地是最简单的选择。它让您指定一次目的地名称，允许代码只关心发送消息，而不关心消息被发送到哪里。但是，如果需要将消息发送到缺省目的地之外的目的地，则需要将该目的地指定为 `send()` 方法的参数。

一种方法是传递目标对象作为 `send()` 的第一个参数。最简单的方法是声明一个 **Destination bean**，然后将其注入执行消息传递的 **bean**。例如，下面的 **bean** 声明了 **Taco Cloud** 订单队列 **Destination**：

```
public Destination orderQueue() {
    return new ActiveMQQueue("tacocloud.order.queue");
}
```

这个 **bean** 方法可以添加到，需要通过 **JMS** 接收发送消息的应用程序的任何配置类中。为了便于分类管理，最好将其添加到专门的消息配置类中，例如 **MessagingConfig**。

需要注意的是，这里使用的 **ActiveMQQueue** 实际上来自于 **Artemis**（来自 `org.apache.activemq.artemis.jms.client` 包）。如果正在使用 **ActiveMQ**（而不是 **Artemis**），那么还有一个名为 **ActiveMQQueue** 的类（来自 `org.apache.activemq.command` 包）。

如果这个 **Destination** bean 被注入到 **JmsOrderMessagingService** 中，那么可以在调用 `send()` 时使用它来指定目的地：

```
private Destination orderQueue;

@Autowired
public JmsOrderMessagingService(JmsTemplate jms,
    Destination orderQueue) {
    this.jms = jms;
    this.orderQueue = orderQueue;
}

...

@Override
public void sendOrder(TacoOrder order) {
    jms.send(
        orderQueue,
        session -> session.createObjectMessage(order));
}
```

使用类似这样的 **Destination** 对象指定目的地，使您有机会配置 **Destination**，而不仅仅是目的地的名称。但是在实践中，几乎只指定了目的地名称，将名称作为 `send()` 的第一个参数通常更简单：

```
@Override
public void sendOrder(TacoOrder order) {
    jms.send(
        "tacocloud.order.queue",
        session -> session.createObjectMessage(order));
}
```

虽然 `send()` 方法并不是特别难以使用（特别是当 **MessageCreator** 以 **lambda** 形式给出时），但是提供 **MessageCreator** 还是会增加一些复杂性。如果只需要指定要发送的对象（以及可选的目的地），不是会更简单吗？这简要地描述了 **convertAndSend()** 的工作方式，让我们来看看。

## 在发送前转换消息

**JmsTemplates** 的 `convertAndSend()` 方法不需要提供 **MessageCreator**，从而简化了消息发布。相反，将要直接发送的对象传递给 `convertAndSend()`，在发送之前会将该对象转换为消息。

例如，`sendOrder()` 的以下新实现使用 `convertAndSend()` 将 **Order** 发送到指定的目的地：

```
@Override
public void sendOrder(TacoOrder order) {
    jms.convertAndSend("tacocloud.order.queue", order);
}
```

与 `send()` 方法一样，`convertAndSend()` 将接受 **Destination** 或 **String** 值来指定目的地，或者可以完全忽略目的地来将消息发送到默认目的地。

无论选择哪种形式的 `convertAndSend()`，传递给 `convertAndSend()` 的 **Order** 都会在发送之前转换为消息。实际上，这是通过 **MessageConverter** 实现的，它完成了将对象转换为消息的复杂工作。

## 配置消息转换器

**MessageConverter** 是 Spring 定义的接口，它只有两个用于实现的方法：

```
public interface MessageConverter {
    Message toMessage(Object object, Session session)
        throws JMSEException, MessageConversionException;
    Object fromMessage(Message message)
}
```

这个接口的实现很简单，都不需要创建自定义实现。Spring 已经提供了一些有用的实现，就像表 9.3 中描述的那样。

表 9.3 用于常见转换任务的 **Spring** 消息转换器（全部在 `org.springframework.jms.support.converter` 包中）

| 消息转换器                                  | 功能                                                                                                                                                                   |
|----------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>MappingJackson2MessageConverter</b> | 使用 Jackson 2 JSON 库对消息进行与 JSON 的转换                                                                                                                                   |
| <b>MarshallingMessageConverter</b>     | 使用 JAXB 对消息进行与 XML 的转换                                                                                                                                               |
| <b>MessagingMessageConverter</b>       | 使用底层 <b>MessageConverter</b> （用于有效负载）和 <b>JmsHeaderMapper</b> （用于将 Jms 信息头映射到标准消息标头）将 <b>Message</b> 从消息传递抽象转换为 <b>Message</b> ，并从 <b>Message</b> 转换为 <b>Message</b> |
| <b>SimpleMessageConverter</b>          | 将 <b>String</b> 转换为 <b>TextMessage</b> ，将字节数组转换为 <b>BytesMessage</b> ，将 <b>Map</b> 转换为 <b>MapMessage</b> ，将 <b>Serializable</b> 转换为 <b>ObjectMessage</b>             |

**SimpleMessageConverter** 是默认的消息转换器，但是它要求发送的对象实现 **Serializable** 接口。这样要求可能还不错，但是可能更喜欢使用其他的消息转换器，如 **MappingJackson2MessageConverter**，来避免上述限制。

为了应用不同的消息转换器，需要做的是将选择的转换器声明为一个 **bean**。例如，下面这个 **bean** 声明将会使用 **MappingJackson2MessageConverter** 而不是 **SimpleMessageConverter**：

```
@Bean
public MappingJackson2MessageConverter messageConverter() {
    MappingJackson2MessageConverter messageConverter =
        new MappingJackson2MessageConverter();
    messageConverter.setTypeIdPropertyName("_typeId");
    return messageConverter;
}
```



注意一下，您在返回 `MappingJackson2MessageConverter` 之前调用了 `setTypeIdPropertyName()`。这是非常重要的，因为它使接收者知道要将传入消息转换成什么类型。默认情况下，它将包含被转换类型的完全限定类名。但这有点不灵活，要求接收方也具有相同的类型，具有相同的完全限定类名。

为了实现更大的灵活性，可以通过调用消息转换器上的 `setTypeIdMappings()` 将合成类型名称映射到实际类型。例如，对消息转换器 `bean` 方法的以下更改将合成订单类型 ID 映射到 `Order` 类：

```
@Bean
public MappingJackson2MessageConverter messageConverter() {
    MappingJackson2MessageConverter messageConverter =
        new MappingJackson2MessageConverter();
    messageConverter.setTypeIdPropertyName("_typeId");

    Map<String, Class<?>> typeIdMappings = new HashMap<String, Class<?>>();
    typeIdMappings.put("order", TacoOrder.class);
    messageConverter.setTypeIdMappings(typeIdMappings);

    return messageConverter;
}
```

与在消息的 `_typeId` 属性中发送完全限定的类名不同，将发送值 `order`。在接收应用程序中，将配置类似的消息转换器，将 `order` 映射到它自己对 `order` 的理解。订单的实现可能在不同的包中，有不同的名称，甚至有发送者 `Order` 属性的一个子集。

## 后期处理消息

让我们假设，除了利润丰厚的网络业务，**Taco Cloud** 还决定开几家实体 **Taco** 连锁店。考虑到他们的任何一家餐馆也可以成为 **web** 业务的执行中心，他们需要一种方法来将订单的来源传达给餐馆的厨房。这将使厨房工作人员能够对商店订单采用与网络订单不同的流程。

在 `Order` 对象中添加一个新的 `source` 属性来携带此信息是合理的，可以用 **WEB** 来填充在线订单，用 **STORE** 来填充商店中的订单。但这将需要更改网站的 `Order` 类和厨房应用程序的 `Order` 类，而实际上，这些信息只需要为 **taco** 准备人员提供。

更简单的解决方案是在消息中添加一个自定义头信息，以承载订单的源。如果正在使用 `send()` 方法发送 **taco** 订单，这可以通过调用消息对象上的 `setStringProperty()` 轻松实现：

```
jms.send("tacocloud.order.queue",
    session -> {
        Message message = session.createObjectMessage(order);
        message.setStringProperty("X_ORDER_SOURCE", "WEB");
    });
```

这里的问题是没有使用 `send()`。通过选择使用 `convertAndSend()`，`Message` 对象是在幕后创建的，并且不能访问它。

幸运的是，有一种方法可以在发送消息之前调整在幕后创建的 `Message`。通过将 `MessagePostProcessor` 作为最后一个参数传递给 `convertAndSend()`，可以在消息创建之后对其进行任何操作。下面的代码仍然使用 `convertAndSend()`，但是它也使用 `MessagePostProcessor` 在消息发送之前添加 `X_ORDER_SOURCE` 头信息：

```
jms.convertAndSend("tacocloud.order.queue", order, new MessagePostProcessor() {
    @Override
    public Message postProcessMessage(Message message) throws JMSException {
        message.setStringProperty("X_ORDER_SOURCE", "WEB");
        return message;
    }
});
```

可能注意到了 **MessagePostProcessor** 是一个函数接口，这意味着可以使用 **lambda** 将其简化为匿名内部类：

```
jms.convertAndSend("tacocloud.order.queue", order,
    message -> {
        message.setStringProperty("X_ORDER_SOURCE", "WEB");
        return message;
    });
```

尽管只需要这个特定的 **MessagePostProcessor** 来处理对 `convertAndSend()` 的调用，但是可能会发现自己使用同一个 **MessagePostProcessor** 来处理对 `convertAndSend()` 的几个不同调用。在这些情况下，也许方法引用是比 **lambda** 更好的选择，避免了不必要的代码重复：

```
@GetMapping("/convertAndSend/order")
public String convertAndSendOrder() {
    TacoOrder order = buildOrder();
   .jms.convertAndSend("tacocloud.order.queue", order,
        this::addOrderSource);
    return "Convert and sent order";
}

private Message addOrderSource(Message message) throws JMSException {
    message.setStringProperty("X_ORDER_SOURCE", "WEB");
    return message;
}
```

已经看到了几种发送消息的方法。但是，如果没有人收到信息，就没有什么用处。让我们看看如何使用 **Spring** 和 **JMS** 接收消息。

### 9.1.3 接收 JMS 消息

在消费消息时，可以选择 *拉模型*（代码请求消息并等待消息到达）或 *推模型*（消息可用时将消息传递给代码）。

**JmsTemplate** 提供了几种接收消息的方法，但它们都使用拉模型。调用其中一个方法来请求消息，线程会发生阻塞，直到消息可用为止（可能是立即可用，也可能需要一段时间）。

另一方面，还可以选择使用推模型，在该模型中，定义了一个消息监听器，它在消息可用时被调用。

这两个选项都适用于各种用例。人们普遍认为推模型是最佳选择，因为它不会阻塞线程。但是在某些用例中，如果消息到达得太快，侦听器可能会负担过重。拉模型允许使用者声明他们已经准备好处理新消息。

让我们看看接收消息的两种方式。我们将从 **JmsTemplate** 提供的拉模型开始。

#### 使用 **JmsTemplate** 接收消息

**JmsTemplate** 提供多个用于拉模式的方法，包括以下这些：

```
Message receive() throws JmsException;
Message receive(Destination destination) throws JmsException;
Message receive(String destinationName) throws JmsException;

Object receiveAndConvert() throws JmsException;
Object receiveAndConvert(Destination destination) throws JmsException;
Object receiveAndConvert(String destinationName) throws JmsException;
```

可以看到，这 6 个方法是 **JmsTemplate** 中的 `send()` 和 `convertAndSend()` 方法的镜像。`receive()` 方法接收原始消息，而 `receiveAndConvert()` 方法使用配置的消息转换器将消息转换为域类型。对于其中的每一个，可以指定 **Destination** 或包含目的地名称的 **String**，也可以从缺省目的地获取一条消息。

要查看这些操作，需要编写一些代码来从 `tacocloud.order.queue` 的目的地拉取 **Order**。下面的程序清单显示了 **OrderReceiver**，这是一个使用 `JmsTemplate.receive()` 接收 **Order** 数据的服务组件。

#### 程序清单 9.2 从队列中拉取订单

```
package tacos.kitchen.messaging.jms;
import javax.jms.Message;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.jms.core.JmsTemplate;
import org.springframework.jms.support.converter.MessageConverter;
import org.springframework.stereotype.Component;

@Component
public class JmsOrderReceiver implements OrderReceiver {
    private JmsTemplate jms;
    private MessageConverter converter;

    @Autowired
    public JmsOrderReceiver(JmsTemplate jms, MessageConverter converter) {
        this.jms = jms;
        this.converter = converter;
    }

    public TacoOrder receiveOrder() {
        Message message = jms.receive("tacocloud.order.queue");
        return (TacoOrder) converter.fromMessage(message);
    }
}
```

这里，使用了一个 `String` 来指定从何处拉取订单。`receive()` 方法返回一个未转换的 `Message`。但是真正需要的是 `Message` 中的 `Order`，所以接下来要做的就是使用注入的消息转换器来转换消息。消息中的类型 ID 属性将指导转换器将其转换为 `Order`，但是它是作为一个 `Object` 返回的，在返回它之前需要进行转换。

接收原始 `Message` 对象在某些需要检查消息属性和标题的情况下可能很有用，但是通常只需要有效载荷。将有效负载转换为域类型需要两个步骤，需要将消息转换器注入组件。当只关心消息的有效负载

时，`receiveAndConvert()` 要简单得多。下面的程序清单显示了如何修改 `JmsOrderReceiver` 来使用

`receiveAndConvert()` 而不是 `receive()`。

### 程序清单 9.3 接收已经转换的 `Order` 对象

```
package tacos.kitchen.messaging.jms;

import org.springframework.jms.core.JmsTemplate;
import org.springframework.stereotype.Component;
import tacos.TacoOrder;
import tacos.kitchen.OrderReceiver;

@Component
public class JmsOrderReceiver implements OrderReceiver {

    private JmsTemplate jms;

    public JmsOrderReceiver(JmsTemplate jms) {
        this.jms = jms;
    }

    @Override
    public TacoOrder receiveOrder() {
        return (TacoOrder) jms.receiveAndConvert("tacocloud.order.queue");
    }
}
```

这个新版本的 `JmsOrderReceiver` 有一个 `receiveOrder()` 方法，该方法已经减少到只有一行。不再需要注入 `MessageConverter`，因为所有的消息转换都将在 `receiveAndConvert()` 中完成。

在继续之前，让我们考虑一下如何在 `Taco Cloud` 厨房应用程序中使用 `receiveOrder()`。在 `Taco Cloud` 的一个厨房里，一名食品加工人员可能会按下一个按钮或采取一些行动，表示他们已经准备好开始制作 `tacos` 了。

此时，`receiveOrder()` 将被调用，而对 `receive()` 或 `receiveAndConvert()` 的调用将被阻塞。在订单消息准备好之前，不会发生任何其他事情。一旦订单到达，它将从 `receiveOrder()` 中返回，其信息用于显示订单的详细信息，以便食品加工人员开始工作。这似乎是拉模型的自然选择。

现在，让我们通过声明 `JMS` 监听器来了解推模型是如何工作的。

## 声明消息监听器

在拉模型中，接收消息需要显式调用 `receive()` 或 `receiveAndConvert()` 方法，与拉模型不同，消息监听器是一个被动组件，在消息到达之前是空闲的。

要创建对 `JMS` 消息作出响应的消息监听器，只需使用 `@JmsListener` 对组件中的方法进行注解。下面程序清单显示了一个新的 `OrderListener` 组件，它被动地监听消息，而不是主动地请求消息。

### 程序清单 9.4 监听订单的 `OrderListener` 组件

```

package tacos.kitchen.messaging.jms.listener;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.context.annotation.Profile;
import org.springframework.jms.annotation.JmsListener;
import org.springframework.stereotype.Component;

import tacos.TacoOrder;
import tacos.kitchen.KitchenUI;

@Profile("jms-listener")
@Component
public class OrderListener {

    private KitchenUI ui;

    @Autowired
    public OrderListener(KitchenUI ui) {
        this.ui = ui;
    }

    @JmsListener(destination = "tacocloud.order.queue")
    public void receiveOrder(TacoOrder order) {
        ui.displayOrder(order);
    }
}

```

`receiveOrder()` 方法由 `JmsListener` 注解，以监听 `tacocloud.order.queue` 目的地的消息。它不处理 `JmsTemplate`，也不被应用程序代码显式地调用。相反，Spring 中的框架代码将等待消息到达指定的目的地，当消息到达时，`receiveOrder()` 方法将自动调用，并将消息的 `Order` 有效负载作为参数。

在许多方面，`@JmsListener` 注解类似于 Spring MVC 的请求映射注释之一，比如 `@GetMapping` 或 `@PostMapping`。在 Spring MVC 中，用一个请求映射方法注解的方法对指定路径的请求做出响应。类似地，使用 `@JmsListener` 注解的方法对到达目的地的消息做出响应。

消息监听器通常被吹捧为最佳的选择，因为它们不会阻塞，并且能够快速处理多个消息。然而，在 **Taco Cloud** 应用程序的上下文中，它们可能不是最佳选择。食品加工是系统中的一个重要瓶颈，可能无法在接到订单时快速准备 **taco**。当一个新的订单显示在屏幕上时，食品加工者可能已经完成了一半的订单。厨房用户界面需要在订单到达时对其进行缓冲，以避免给厨房员工带来过重的负担。

这并不是说消息监听器不好。相反，当消息可以快速处理时，它们是完美的选择。但是，当消息处理程序需要能够根据自己的时间请求更多消息时，`JmsTemplate` 提供的拉模型似乎更合适。

因为 **JMS** 是由标准 **Java** 规范定义的，并且受到许多消息 **Broker** 的支持，所以它是 **Java** 中消息传递的常用选择。但是 **JMS** 有一些缺点，尤其是作为 **Java** 规范，它的使用仅限于 **Java** 应用程序。**RabbitMQ** 和 **Kafka** 等较新的消息传递选项解决了这些缺点，并且适用于 **JVM** 之外的其他语言和平台。让我们把 **JMS** 放在一边，看看如何使用 **RabbitMQ** 进行 **taco** 订单消息传递。

## 9.2 使用 RabbitMQ 和 AMQP

RabbitMQ 可以说是 AMQP 最优秀的实现，它提供了比 JMS 更高级的消息路由策略。JMS 消息使用接收方将从中检索它们的目的地的名称来寻址，而 AMQP 消息使用交换器的名称和路由键来寻址，它们与接收方正在监听的队列解耦。交换器和队列之间的这种关系如图 9.2 所示。

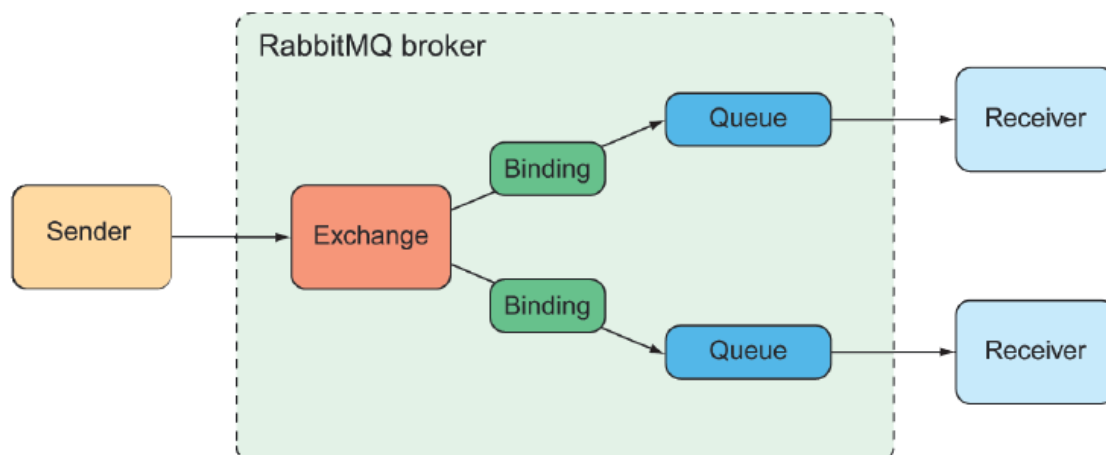


图 9.2 发送到 RabbitMQ 交换器的消息被路由到一个或多个队列，依据键和绑定关系。

当消息到达 RabbitMQ broker 时，它将转到它所寻址的交换器。交换器负责将其路由到一个或多个队列，具体取决于交换器的类型、交换器与队列之间的绑定以及消息的路由键的值。

有几种不同的交换方式，包括以下几种：

- **Default** —— 一种特殊的交换器，通过 broker 自动创建。它将消息路由到与消息的路由键的值同名的队列中。所有的队列将会自动地与交换器绑定。
- **Direct** —— 路由消息到消息路由键的值与绑定值相同的队列。
- **Topic** —— 将消息路由到一个或多个队列，其中绑定键（可能包含通配符）与消息的路由键匹配。
- **Fanout** —— 将消息路由到所有绑定队列，而不考虑绑定键或路由键。
- **Headers** —— 与 topic 交换器类似，只是路由基于消息头值而不是路由键。
- **Dead letter** —— 对无法交付的消息（意味着它们不匹配任何已定义的交换器与队列的绑定）的全部捕获。

最简单的交换形式是 **Default** 和 **Fanout**，因为它们大致对应于 JMS 队列和主题。但是其他交换允许定义更灵活的路由方案。

需要理解的最重要的一点是，消息是用路由键发送到交换器的，它们是从队列中使用的。它们如何从一个交换到一个队列取决于绑定定义以及什么最适合相应的情况。

使用哪种交换类型以及如何定义从交换到队列的绑定与 Spring 应用程序中消息的发送和接收方式关系不大。因此，我们将重点讨论如何编写使用 RabbitMQ 发送和接收消息的代码。

注意 有关如何最好地将队列绑定到交换器的更详细讨论，请参见 Gavin Roy 的《RabbitMQ in Depth》（Manning, 2017），或由 Alvaro Videla 和 Jason J.W. Williams 合著的《RabbitMQ 实战》（Manning, 2012）。

## 9.2.1 添加 RabbitMQ 到 Spring 中

在开始使用 Spring 发送和接收 RabbitMQ 消息之前，需要将 Spring Boot 的 AMQP starter 依赖项添加到构建中，以取代在前一节中添加的 Artemis 或 ActiveMQ starter：

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-amqp</artifactId>
</dependency>
```

将 AMQP starter 添加到构建中将触发自动配置，该配置将创建 AMQP 连接工厂和 RabbitTemplate bean，以及其他支持组件。只需添加此依赖项，就可以开始使用 Spring 从 RabbitMQ broker 发送和接收消息，表 9.4 中列出了一些有用的属性。

表 9.4 配置 RabbitMQ broker 的位置和凭据的属性

| 属性                        | 描述                           |
|---------------------------|------------------------------|
| spring.rabbitmq.addresses | 一个逗号分隔的 RabbitMQ Broker 地址列表 |
| spring.rabbitmq.host      | Broker 主机（默认为 localhost）     |
| spring.rabbitmq.port      | Broker 端口（默认为 5672）          |
| spring.rabbitmq.username  | 访问 Broker 的用户名（可选）           |
| spring.rabbitmq.password  | 访问 Broker 的密码（可选）            |

出于开发目的，可能有一个 RabbitMQ Broker，它不需要在本地机器上运行身份验证，监听端口 5672。当还在开发阶段时，这些属性可能不会有太大的用处，但是当应用程序进入生产环境时，它们无疑会很有用。

注意：运行 RabbitMQ 代理。

如果您还没有 RabbitMQ 代理可以使用，那么有几种在本地计算机上运行 RabbitMQ 的选项。查阅官方 RabbitMQ 文档 <https://www.rabbitmq.com/download.html>，以获得运行 RabbitMQ 的最新说明。

例如，假设在进入生产环境时，RabbitMQ Broker 位于一个名为 rabbit.tacocloud.com 的服务器上，监听端口 5673，并需要凭据。在这种情况下，应用程序中的以下配置。当 prod 配置文件处于活动状态时，yaml 文件将设置这些属性：

```
spring:
  profiles: prod
  rabbitmq:
    host: rabbit.tacocloud.com
    port: 5673
    username: tacoweb
    password: l3tm31n
```

现在 RabbitMQ 被配置到了应用程序中了，是时候使用 RabbitTemplate 发送消息了。

## 9.2.2 使用 RabbitTemplate 发送消息

Spring 对于 RabbitMQ 消息支持的核心就是 `RabbitTemplate`。`RabbitTemplate` 提供一套与 `JmsTemplate` 类似的方法。但是对于 RabbitMQ，在工作方式上还是有一些细微的差别。

关于使用 `RabbitTemplate` 发送消息，`send()` 和 `convertAndSend()` 方法与来自 `JmsTemplate` 的同名方法并行。但是不同于 `JmsTemplate` 方法，它只将消息路由到给定的队列或主题，`RabbitTemplate` 方法根据交换和路由键发送消息。下面是一些用 `RabbitTemplate` 发送消息的最有用的方法：

```
// 发送原始消息
void send(Message message) throws AmqpException;
void send(String routingKey, Message message) throws AmqpException;
void send(String exchange, String routingKey, Message message)
    throws AmqpException;

// 发送从对象转换过来的消息
void convertAndSend(Object message) throws AmqpException;
void convertAndSend(String routingKey, Object message)
    throws AmqpException;
void convertAndSend(String exchange, String routingKey, Object message) throws AmqpException;

// 发送经过处理后从对象转换过来的消息
void convertAndSend(Object message, MessagePostProcessor mPP)
    throws AmqpException;
void convertAndSend(String routingKey, Object message,
    MessagePostProcessor messagePostProcessor)
    throws AmqpException;
void convertAndSend(String exchange, String routingKey, Object message,
    MessagePostProcessor messagePostProcessor)
    throws AmqpException;
```

这些方法与 `JmsTemplate` 中的孪生方法遵循类似的模式。前三个 `send()` 方法都发送一个原始消息对象。接下来的三个 `convertAndSend()` 方法接受一个对象，该对象将在发送之前在后台转换为消息。最后三个 `convertAndSend()` 方法与前三个方法类似，但是它们接受一个 `MessagePostProcessor`，可以在消息对象发送到代理之前使用它来操作消息对象。

这些方法与对应的 `JmsTemplate` 方法不同，它们接受 `String` 值来指定交换和路由键，而不是目的地名称（或 `Destination` 对象）。不接受交换的方法将把它们的消息发送到默认交换。同样，不接受路由键的方法将使用默认路由键路由其消息。

让我们用 `RabbitTemplate` 发送 `taco` 订单。一种方法是使用 `send()` 方法，如程序清单 9.5 所示。但是在调用 `send()` 之前，需要将 `Order` 对象转换为消息。如果不是因为 `RabbitTemplate` 使用 `getMessageConverter()` 方法使其消息转换器可用，这可能是一项乏味的工作。

程序清单 9.5 使用 `RabbitTemplate.send()` 发送消息



```

package tacos.messaging;

import org.springframework.amqp.core.Message;
import org.springframework.amqp.core.MessageProperties;
import org.springframework.amqp.rabbit.core.RabbitTemplate;
import org.springframework.amqp.support.converter.MessageConverter;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import tacos.Order;

@Service
public class RabbitOrderMessagingService
    implements OrderMessagingService {
    private RabbitTemplate rabbit;

    @Autowired
    public RabbitOrderMessagingService(RabbitTemplate rabbit) {
        this.rabbit = rabbit;
    }

    public void sendOrder(TacoOrder order) {
        MessageConverter converter = rabbit.getMessageConverter();
        MessageProperties props = new MessageProperties();
        Message message = converter.toMessage(order, props);
        rabbit.send("tacocloud.order", message);
    }
}

```

有了 `MessageConverter` 之后，将 `Order` 转换为消息就很简单了。必须使用 `MessageProperties` 提供任何消息属性，但是如果不需设置任何此类属性，则可以使用 `MessageProperties` 的缺省实例。然后，剩下的就是调用 `send()`，将交换键和路由键（两者都是可选的）与消息一起传递。在本例中，只指定了与消息一起的路由键：`tacocloud.order`，因此将使用缺省交换。

说到默认交换，默认交换名称是 `""`（一个空 `String`），它对应于 `RabbitMQ Broker` 自动创建的默认交换。同样，默认的路由键是 `""`（其路由取决于所涉及的交换和绑定）。可以通过设置 `spring.rabbitmq.template.exchange` 和 `spring.rabbitmq.template.routing-key` 属性来覆盖这些缺省值：

```

spring:
  rabbitmq:
    template:
      exchange: tacocloud.orders
      routing-key: kitchens.central

```

在这种情况下，所有发送的消息都将自动发送到名为 `tacocloud.orders` 的交换器。如果在 `send()` 或 `convertAndSend()` 调用中也未指定路由键，则消息将有一个 `kitchens.central` 的路由键。

从消息转换器创建消息对象非常简单，但是使用 `convertAndSend()` 让 `RabbitTemplate` 处理所有的转换工作就更容易了：

```

public void sendOrder(Order order) {
    rabbit.convertAndSend("tacocloud.order", order);
}

```

## 配置消息转换器

默认情况下，使用 `SimpleMessageConverter` 执行消息转换，`SimpleMessageConverter` 能够将简单类型（如 `String`）和可序列化对象转换为消息对象。但是 `Spring` 为 `RabbitTemplate` 提供了几个消息转换器，包括以下内容：

- `Jackson2JsonMessageConverter` —— 使用 `Jackson 2 JSON` 处理器将对象与 `JSON` 进行转换
- `MarshallingMessageConverter` —— 使用 `Spring Marshaller` 和 `Unmarshaller` 进行转换
- `SerializerMessageConverter` —— 使用 `Spring` 的序列化和反序列化抽象转换 `String` 和任何类型的本地对象
- `SimpleMessageConverter` —— 转换 `String`、字节数组和序列化类型
- `ContentTypeDelegatingMessageConverter` —— 基于 `contentType` 头信息将对象委托给另一个 `MessageConverter`
- `MessagingMessageConverter` —— 将消息转换委托给底层 `MessageConverter`，将消息头委托给 `AmqpHeaderConverter`

如果需要修改消息转换器，需要做的是配置 `MessageConverter` bean，例如，对于基于 `JSON` 的消息对话，可以像下面这样配置 `Jackson2JsonMessageConverter`：

```
@Bean
public MessageConverter messageConverter() {
    return new Jackson2JsonMessageConverter();
}
```

`Spring Boot` 的自动配置将会发现这个 bean 并 `RabbitTemplate` 的缺省的消息转换器那里。

## 设置消息属性

与 `JMS` 一样，可能需要在发送的消息中设置一些标题。例如，假设需要为通过 `Taco Cloud` 网站提交的所有订单发送一个 `X_ORDER_SOURCE`。在创建 `Message` 对象时，可以通过提供给消息转换器的 `MessageProperties` 实例设置消息头。

重新访问程序清单 9.5 中的 `sendOrder()` 方法，只需要添加一行代码来设置标题：

```
public void sendOrder(TacoOrder order) {
    MessageConverter converter = rabbit.getMessageConverter();
    MessageProperties props = new MessageProperties();
    props.setHeader("X_ORDER_SOURCE", "WEB");
    Message message = converter.toMessage(order, props);
    rabbit.send("tacocloud.order", message);
}
```

但是，在使用 `convertAndSend()` 时，不能快速访问 `MessageProperties` 对象。不过，`MessagePostProcessor` 可以做到这一点：

```
public void sendOrder(TacoOrder order) {
    rabbit.convertAndSend("tacocloud.order.queue", order,
        new MessagePostProcessor() {
            @Override
            public Message postProcessMessage(Message message)
                throws AmqpException {
                MessageProperties props = message.getMessageProperties();
                props.setHeader("X_ORDER_SOURCE", "WEB");
                return message;
            }
        });
}
```

这里，在 `convertAndSend()` 中使用 `MessagePostProcessor` 的匿名内部类进行实现。在 `postProcessMessage()` 方法中，首先从消息中获取 `MessageProperties`，然后调用 `setHeader()` 来设置 `X_ORDER_SOURCE` 头信息。

现在已经了解了如何使用 `RabbitTemplate` 发送消息，接下来让我们将注意力转移到从 `RabbitMQ` 队列接收消息的代码上。

### 9.2.3 从 RabbitMQ 接收消息

使用 `RabbitTemplate` 发送消息与使用 `JmsTemplate` 发送消息差别不大。事实证明，从 `RabbitMQ` 队列接收消息与从 `JMS` 接收消息并没有太大的不同。

与 `JMS` 一样，有两个选择：

- 使用 `RabbitTemplate` 从队列中拉取消息
- 获取被推送到 `@RabbitListener` 注解的方法中的消息

让我们从基于拉模型的 `RabbitTemplate.receive()` 方法开始。

### 使用 RabbitTemplate 接收消息

`RabbitTemplate` 有多个从队列中拉取消息的方法，一部分最有用的方法如下所示：

```
// 接收消息
Message receive() throws AmqpException;
Message receive(String queueName) throws AmqpException;
Message receive(long timeoutMillis) throws AmqpException;
Message receive(String queueName, long timeoutMillis) throws AmqpException;

// 接收从消息转换过来的对象
Object receiveAndConvert() throws AmqpException;
Object receiveAndConvert(String queueName) throws AmqpException;
Object receiveAndConvert(long timeoutMillis) throws AmqpException;
Object receiveAndConvert(String queueName, long timeoutMillis)
    throws AmqpException;

// 接收从消息转换过来的类型安全的对象
<T> T receiveAndConvert(ParameterizedTypeReference<T> type)
    throws AmqpException;
<T> T receiveAndConvert(String queueName, ParameterizedTypeReference<T> type)
    throws AmqpException;
<T> T receiveAndConvert(long timeoutMillis, ParameterizedTypeReference<T> type)
    throws AmqpException;
<T> T receiveAndConvert(String queueName, long timeoutMillis,
    ParameterizedTypeReference<T> type) throws AmqpException;
```

这些方法是前面描述的 `send()` 和 `convertAndSend()` 方法的镜像。`send()` 用于发送原始 `Message` 对象，而 `receive()` 从队列接收原始 `Message` 对象。同样地，`receiveAndConvert()` 接收消息，并在返回消息之前使用消息转换器将其转换为域对象。

但是在方法签名方面有一些明显的区别。首先，这些方法都不以交换键或路由键作为参数。这是因为交换和路由键用于将消息路由到队列，但是一旦消息在队列中，它们的下一个目的地就是将消息从队列中取出的使用者。使用应用程序不需要关心交换或路由键，队列是在消费应用程序是仅仅需要知道一个东西。

许多方法接受一个 `long` 参数来表示接收消息的超时。默认情况下，接收超时为 0 毫秒。也就是说，对 `receive()` 的调用将立即返回，如果没有可用的消息，则可能返回空值。这与 `receive()` 方法在 `JmsTemplate` 中的行为有明显的不同。通过传入超时值，可以让 `receive()` 和 `receiveAndConvert()` 方法阻塞，直到消息到达或超时过期。但是，即使使用非零超时，代码也要准备好处理返回的 `null` 值。

让我们看看如何将其付诸实践。下面程序清单显示了 `OrderReceiver` 的一个新的基于 `Rabbit` 的实现，它使用 `RabbitTemplate` 来接收订单。

程序清单 9.6 使用 `RabbitTemplate` 从 `RabbitMQ` 拉取订单

```

package tacos.kitchen.messaging.rabbit;
import org.springframework.amqp.core.Message;
import org.springframework.amqp.rabbit.core.RabbitTemplate;
import org.springframework.amqp.support.converter.MessageConverter;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Component;

@Component
public class RabbitOrderReceiver {
    private RabbitTemplate rabbit;
    private MessageConverter converter;

    @Autowired
    public RabbitOrderReceiver(RabbitTemplate rabbit) {
        this.rabbit = rabbit;
        this.converter = rabbit.getMessageConverter();
    }

    public TacoOrder receiveOrder() {
        Message message = rabbit.receive("tacocloud.orders");
        return message != null
            ? (TacoOrder) converter.fromMessage(message)
            : null;
    }
}

```

`receiveOrder()` 方法是所有操作发生的地方。它调用所注入的 `RabbitTemplate` 上的 `receive()` 方法来从 `tacocloud.queue` 中获取订单。它不提供超时值，因此只能假设调用立即返回 `Message` 或 `null`。如果返回一条 `Message`，则使用 `RabbitTemplate` 中的 `MessageConverter` 将 `Message` 转换为 `Order`。另一方面，如果 `receive()` 返回 `null`，则返回 `null`。

根据实际情况的不同，可能容忍一个小的延迟。例如，在 **Taco Cloud** 厨房项目的头顶显示器中，如果没有订单信息出现，可以等待一下，可以决定等 30 秒后再放弃。然后，可以将 `receiveOrder()` 方法更改为传递一个 30,000 毫秒的延迟后再调用 `receive()`：

```

public TacoOrder receiveOrder() {
    Message message = rabbit.receive("tacocloud.order.queue", 30000);
    return message != null
        ? (TacoOrder) converter.fromMessage(message)
        : null;
}

```

如果您和我一样，看到这样一个硬编码的数字会让您有点不舒服。那么创建一个带 `@ConfigurationProperties` 注解的类是个好想法，这样就可以使用 **Spring Boot** 的配置属性来配置超时。如果不是 **Spring Boot** 已经提供了这样的配置属性，我也会觉得硬编码的数字很不舒服。如果希望通过配置设置超时，只需删除 `receive()` 调用中的超时值，并在配置中使用 `spring.rabbitmq.template.receive-timeout` 属性设置它：

```

spring:
  rabbitmq:
    template:
      receive-timeout: 30000

```

回到 `receiveOrder()` 方法，请注意，必须使用 `RabbitTemplate` 中的消息转换器来将传入 `Message` 对象转换为 `Order` 对象。但是如果 `RabbitTemplate` 携带了一个消息转换器，为什么它不能进行转换呢？这正是 `receiveAndConvert()` 方法的用途。使用 `receiveAndConvert()`，可以像这样重写 `receiveOrder()`：

```
public TacoOrder receiveOrder() {
    return (TacoOrder) rabbit.receiveAndConvert("tacocloud.order.queue");
}
```

那就简单多了，不是吗？所看到的唯一麻烦的事情就是从 **Object** 到 **Order** 的转换。不过，除了演员阵容，还有另一种选择。相反，您可以对 `receiveAndConvert()` 传递一个 **ParameterizedTypeReference** 来直接接收一个 **Order** 对象：

```
public TacoOrder receiveOrder() {
    return rabbit.receiveAndConvert("tacocloud.order.queue",
        new ParameterizedTypeReference<Order>() {});
}
```

这是否比类型转换更好还值得商榷，但它是一种比类型转换更安全的方法。使用 `receiveAndConvert()` 的 **ParameterizedTypeReference** 的惟一要求是消息转换器必须是 **SmartMessageConverter** 的实现；**Jackson2JsonMessageConverter** 是唯一可以选择的開箱即用的实现。

**JmsTemplate** 提供的拉模型适用于许多用例，但通常最好有监听消息并在消息到达时调用的代码。让我们看看如何编写响应 **RabbitMQ** 消息的消息驱动 **bean**。

## 使用监听器处理 **RabbitMQ** 消息

对于消息驱动的 **RabbitMQ** **bean**，**Spring** 提供了 **RabbitListener**，相当于 **RabbitMQ** 中的 **JmsListener**。要指定当消息到达 **RabbitMQ** 队列时应该调用某个方法，请在相应的 **bean** 方法上使用 `@RabbitTemplate` 进行注解。

例如，下面的程序清单显示了 **OrderReceiver** 的 **RabbitMQ** 实现，它被注解为监听订单消息，而不是使用 **RabbitTemplate** 来轮询订单消息。

程序清单 9.7 声明一个方法作为 **RabbitMQ** 消息监听器

```
package tacos.kitchen.messaging.rabbit.listener;

import org.springframework.amqp.rabbit.annotation.RabbitListener;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Component;

import tacos.TacoOrder;
import tacos.kitchen.KitchenUI;

@Component
public class OrderListener {

    private KitchenUI ui;

    @Autowired
    public OrderListener(KitchenUI ui) {
        this.ui = ui;
    }

    @RabbitListener(queues = "tacocloud.order.queue")
    public void receiveOrder(TacoOrder order) {
        ui.displayOrder(order);
    }
}
```

这与程序清单 9.4 中的代码非常相似。实际上，唯一改变的是监听器注解——从 `@JmsListener` 变为了 `@RabbitListener`。尽管 `@RabbitListener` 非常棒，但这种近乎复制的代码让我对 `@RabbitListener` 没什么可说的，而我之前还没有对 `@JmsListener` 说过。它们都非常适合编写从各自的 **broker** 推送给它们的消息的代码——**JMS broker** 用于 `@JmsListener`，**RabbitMQ broker** 用于 `@RabbitListener`。

虽然在前面的段落中可能感觉到了 `@RabbitListener` 不是那么让人兴奋。事实上，`@RabbitListener` 与 `@JmsListener` 的工作方式非常相似，这一点非常令人兴奋！这意味着在使用 **RabbitMQ** 与 **Artemis** 或 **ActiveMQ** 时，不需要学习完全不同的编程模型。同样令人兴奋的是 **RabbitTemplate** 和 **JmsTemplate** 之间的相似性。

在结束本章时，让我们继续关注 **Spring** 支持的另一个消息传递中间件：**Apache Kafka**。

## 9.3 使用 Kafka 发送消息

Apache Kafka 是我们在本章中研究的最新消息传递选项。乍一看，Kafka 是一个消息代理，就像 ActiveMQ、Artemis 或 Rabbit 一样。但是 Kafka 有一些独特的技巧。

Kafka 被设计为在集群中运行，提供了巨大的可伸缩性。通过将其 topic 划分到集群中的所有实例中，它具有很强的弹性。RabbitMQ 主要处理 exchange 中的队列，而 Kafka 仅利用 topic 来提供消息的发布/订阅。

Kafka topic 被复制到集群中的所有 broker 中。集群中的每个节点充当一个或多个 topic 的 leader，负责该 topic 的数据并将其复制到集群中的其他节点。

更进一步说，每个 topic 可以分成多个分区。在这种情况下，集群中的每个节点都是一个 topic 的一个或多个分区的 leader，但不是整个 topic 的 leader。该 topic 的职责由所有节点分担。图 9.2 说明了这是如何工作的。

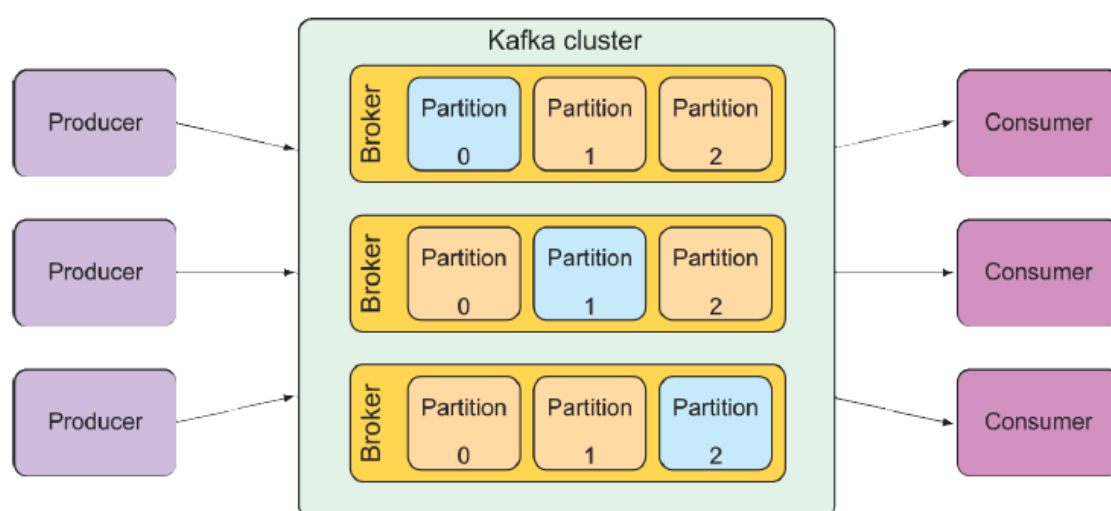


图 9.3 Kafka 集群由多个 broker 组成，每一个都作为 topic 分区的 leader。

由于 Kafka 独特的构建风格，我鼓励您阅读由 Dylan Scott、Viktor Gamov 和 Dave Klein 合著的《Kafka 实战》（Manning, 2021）。在本书中我们将重点讨论如何使用 Spring 向 Kafka 发送和接收消息。



### 9.3.1 在 Spring 中设置 Kafka

要开始使用 Kafka 进行消息传递，需要将适当的依赖项添加到构建中。但是，与 JMS 和 RabbitMQ 不同，Kafka 没有 Spring Boot starter。不过还是只需要一个依赖：

```
<dependency>
  <groupId>org.springframework.kafka</groupId>
  <artifactId>spring-kafka</artifactId>
</dependency>
```

这个依赖项将 Kafka 所需的一切都带到项目中。更重要的是，它的存在将触发 Kafka 的 Spring Boot 自动配置，它将在 Spring 应用程序上下文中生成一个 `KafkaTemplate`。您所需要做的就是注入 `KafkaTemplate` 并开始发送和接收消息。

然而，在开始发送和接收消息之前，应该了解一些在使用 Kafka 时会派上用场的属性。具体来说就是，`KafkaTemplate` 默认在 `localhost` 上运行 Kafka broker，并监听 9092 端口。在开发应用程序时，在本地启动 Kafka broker 是可以的，但是在进入生产环境时，需要配置不同的主机和端口。

注意：安装 Kafka 群集。

如果要运行本章中的示例，您需要一个可用的 Kafka 集群。请参阅 Kafka 官方文档

<https://kafka.apache.org/quickstart>，这是一个开始学习如何在您的计算机上本地运行 Kafka 的好地方。

`spring.kafka.bootstrap-servers` 属性设置一个或多个 Kafka 服务器的位置，用于建立到 Kafka 集群的初始连接。例如，如果集群中的 Kafka 服务器之一运行在 `Kafka.tacocloud.com` 上，并监听 9092 端口，那么可以在 YAML 中像这样配置它的位置：

```
spring:
  kafka:
    bootstrap-servers:
      - kafka.tacocloud.com:9092
```

但是注意 `spring.kafka.bootstrap-servers` 属性是复数形式，它接受一个列表。因此，可以在集群中为它提供多个 Kafka 服务器：

```
spring:
  kafka:
    bootstrap-servers:
      - kafka.tacocloud.com:9092
      - kafka.tacocloud.com:9093
      - kafka.tacocloud.com:9094
```

这些配置适用于名为 `Kafka.tacocloud.com` 的主机上的 Kafka 引导服务器。如果如果您在本地运行 Kafka 集群（可能在开发过程中），那么您需要要改用 `localhost`，请执行以下操作：

```
spring:
  kafka:
    bootstrap-servers:
      - localhost:9092
```

在项目中设置了 Kafka 之后，就可以发送和接收消息了。首先来看看 `KafkaTemplate` 将 `Order` 对象发送给 Kafka。

### 9.3.2 使用 `KafkaTemplate` 发送消息

在许多方面，`KafkaTemplate` 与 `JMS` 和 `RabbitMQ` 类似。与此同时，它也是不同的，尤其是在我们考虑它发送消息的方法时：

```
ListenableFuture<SendResult<K, V>> send(String topic, V data);
ListenableFuture<SendResult<K, V>> send(String topic, K key, V data);
ListenableFuture<SendResult<K, V>> send(String topic,
    Integer partition, K key, V data);
ListenableFuture<SendResult<K, V>> send(String topic,
    Integer partition, Long timestamp, K key, V data);
ListenableFuture<SendResult<K, V>> send(ProducerRecord<K, V> record);
ListenableFuture<SendResult<K, V>> send(Message<?> message);
ListenableFuture<SendResult<K, V>> sendDefault(V data);
ListenableFuture<SendResult<K, V>> sendDefault(K key, V data);
ListenableFuture<SendResult<K, V>> sendDefault(Integer partition,
    K key, V data);
ListenableFuture<SendResult<K, V>> sendDefault(Integer partition,
    Long timestamp, K key, V data);
```

注意到的第一件事是没有 `convertAndSend()` 方法。这是因为 `KafkaTemplate` 是用的泛型，同时能够在发送消息时直接处理域类型。在某种程度上，所有的 `send()` 方法都在做 `convertAndSend()` 的工作。

再者 `send()` 和 `sendDefault()` 的参数，它们与 `JMS` 和 `Rabbit` 中使用的参数完全不同。当使用 `Kafka` 发送消息时，可以指定以下参数来指导如何发送消息：

- 发送消息的 **topic**（`send()` 方法必要的参数）
- 写入 **topic** 的分区（可选）
- 发送记录的键（可选）
- 时间戳（可选；默认为 `System.currentTimeMillis()`）
- **payload**（必须）

**topic** 和 **payload** 是两个最重要的参数。分区和键对如何使用 `KafkaTemplate` 几乎没有影响，除了作为 `send()` 和 `sendDefault()` 的参数用于提供额外信息。出于我们的目的，我们将把重点放在将消息有效负载发送到给定主题上，而不考虑分区和键。

对于 `send()` 方法，还可以选择发送一个 `ProducerRecord`，它与在单个对象中捕获所有上述参数的类型差不多。也可以发送 `Message` 对象，但是这样做需要将域对象转换为 `Message`。通常，使用其他方法比创建和发送 `ProducerRecord` 或 `Message` 对象更容易。

使用 `KafkaTemplate` 及其 `send()` 方法，可以编写一个基于 `kafka` 的 `OrderMessagingService` 实现。下面的程序清单显示了这样一个实现。

程序清单 8.8 使用 `KafkaTemplate` 发送订单

```

package tacos.messaging;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.kafka.core.KafkaTemplate;
import org.springframework.stereotype.Service;
import tacos.TacoOrder;

@Service
public class KafkaOrderMessagingService
    implements OrderMessagingService {

    private KafkaTemplate<String, TacoOrder> kafkaTemplate;

    @Autowired
    public KafkaOrderMessagingService(
        KafkaTemplate<String, TacoOrder> kafkaTemplate) {
        this.kafkaTemplate = kafkaTemplate;
    }

    @Override
    public void sendOrder(TacoOrder order) {
        kafkaTemplate.send("tacocloud.orders.topic", order);
    }
}

```

在 `OrderMessagingService` 的这个实现中，`sendOrder()` 方法使用注入的 `KafkaTemplate` 的 `send()` 方法向名为 `tacocloud.orders.topic` 的主题发送 `Order`。代码中除了使用 "Kafka" 这个名称外，这与为 `JMS` 和 `Rabbit` 编写的代码没有太大的不同。而且，就像其他实现一样，对于 `OrderMessagingService`，可以将其注入 `OrderApiController` 当通过 `/api/orders` 下单时，可以通过 `Kafka` 发送消息。

在我们创建消息接收器的 `Kafka` 实现之前，您需要一个控制台来查看发送的内容。`Kafka` 有几种可用的管理控制台，包括 `Offset Explorer` (<https://www.kafkatool.com/>) 以及 `Confluent's Apache Kafka UI` (<https://www.confluent.io/product/confluent-platform/gui-driven-management-and-monitoring/>)。

如果设置了默认主题，可以稍微简化 `sendOrder()` 方法。首先，通过设置 `spring.kafka.template.default-topic` 属性，将默认主题设置为 `tacocloud.orders.topic`：

```

spring:
  kafka:
    bootstrap-servers:
      - localhost:9092
    template:
      default-topic: tacocloud.orders.topic

```

然后，在 `sendOrder()` 方法中，可以调用 `sendDefault()` 而不是 `send()`，并且不指定主题名称：

```

@Override
public void sendOrder(TacoOrder order) {
    kafkaTemplate.sendDefault(order);
}

```

现在已经编写了消息发送代码了，让我们将注意力转向编写从 `Kafka` 接收这些消息的代码。

### 9.3.3 编写 Kafka 监听器

除了 `send()` 和 `sendDefault()` 的惟一方法签名之外，`KafkaTemplate` 与 `JmsTemplate` 和 `RabbitTemplate` 的不同之处在于它不提供任何接收消息的方法。这意味着使用 `Spring` 消费来自 `Kafka` 主题的消息的唯一方法是编写消息监听器。

对于 `Kafka`，消息监听器被定义为被 `@KafkaListener` 注解的方法。`@KafkaListener` 注解大致类似于 `@JmsListener` 和 `@RabbitListener`，其使用方式大致相同。下面程序清单显示了为 `Kafka` 编写的基于 `listener` 的订单接收程序。

程序清单 9.9 使用 `@KafkaListener` 接收订单

```
package tacos.kitchen.messaging.kafka.listener;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.kafka.annotation.KafkaListener;
import org.springframework.stereotype.Component;
import tacos.Order;
import tacos.kitchen.KitchenUI;

@Component
public class OrderListener {

    private KitchenUI ui;

    @Autowired
    public OrderListener(KitchenUI ui) {
        this.ui = ui;
    }

    @KafkaListener(topics="tacocloud.orders.topic")
    public void handle(TacoOrder order) {
        ui.displayOrder(order);
    }
}
```

`handle()` 方法由 `@KafkaListener` 注解，表示当消息到达名为 `tacocloud.orders.topic` 的主题时应该调用它。正如程序清单 8.9 中所写的，只为 `handle()` 方法提供了一个 `Order`（payload）参数。但是，如果需要来自消息的其他元数据，它也可以接受一个 `ConsumerRecord` 或 `Message` 对象。

例如，`handle()` 的以下实现接受一个 `ConsumerRecord`，这样就可以记录消息的分区和时间戳：

```
@KafkaListener(topics="tacocloud.orders.topic")
public void handle(
    TacoOrder order, ConsumerRecord<String, TacoOrder> record) {
    log.info("Received from partition {} with timestamp {}",
        record.partition(), record.timestamp());

    ui.displayOrder(order);
}
```

类似地，可以使用 `Message` 而不是 `ConsumerRecord`，并达到同样的效果：

```
@KafkaListener(topics="tacocloud.orders.topic")
public void handle(Order order, Message<Order> message) {
    MessageHeaders headers = message.getHeaders();
    log.info("Received from partition {} with timestamp {}",
        headers.get(KafkaHeaders.RECEIVED_PARTITION_ID),
        headers.get(KafkaHeaders.RECEIVED_TIMESTAMP));
    ui.displayOrder(order);
}
```

值得注意的是，消息有效负载也可以通过 `ConsumerRecord.value()` 或 `Message.getPayload()` 获得。这意味着可以通过这些对象请求 **Order**，而不是直接将其作为 `handle()` 的参数。

## 9.4 总结

- 异步消息传递在通信的应用程序之间提供了一个间接层，允许更松散的耦合和更大的可伸缩性。
- Spring 支持 JMS、RabbitMQ 或 Apache Kafka 的异步消息传递。
- 应用程序可以使用基于模板的客户端（`JmsTemplate`、`RabbitTemplate` 或 `KafkaTemplate`）通过消息 broker 发送消息。
- 接收应用程序可以使用相同的基于模板的客户端，在基于 pull 的模型中使用消息。
- 也可以通过向 bean 方法应用消息监听器注解（`@JmsListener`、`@RabbitListener` 或 `@KafkaListener`）将消息推送给消费者。

## 第 10 章 集成 Spring

本章内容：

- 实时数据处理
- 定义集成流程
- 使用 **Spring Integration** 的 **Java DSL** 定义
- 集成电子邮件、文件系统和其他外部系统

我在旅行中遇到的最令人沮丧的事情之一就是在长途飞行中，飞机上的互联网连接很差或者根本不存在。我喜欢利用我的飞行时间完成一些工作，包括写这本书。在需要获取库或查找 **Java** 文档时，如果没有网络连接，那就尴尬了，也完成不了多少工作。在这种场合下，我学会了带本纸质书来读。

正如我们需要连接到互联网来提高生产力一样，许多应用程序也必须连接到外部系统来执行它们的工作。应用程序可能需要读取或发送电子邮件、与外部 **API** 交互或对写入数据库的数据作出响应。而且，由于数据是从这些外部系统获取或写入的，应用程序可能需要以某种方式处理数据，以便将其转换到或从应用程序自己的领域。

在本章中，将看到如何使用 **Spring Integration** 的通用集成模式。**Spring Integration** 是由 **Gregor Hohpe** 和 **Bobby Woolf** 在《企业级集成模式》一书中编目的许多集成模式的实现。每个模式都被实现为一个组件，消息通过该组件传输管道中的数据。使用 **Spring** 配置，可以将这些组件组装到数据流经的管道中。让我们从定义一个简单的集成流开始，它引入了 **Spring Integration** 的许多特性。

## 10.1 声明简单的集成流

一般来说，Spring Integration 支持创建集成流，应用程序可以通过这些集成流接收或发送数据到应用程序本身的外部资源。应用程序可以与之集成的一种资源是文件系统。因此，在 Spring Integration 的众多组件中，有用于读写文件的通道适配器。

为了熟悉 Spring Integration，将创建一个向文件系统写入数据的集成流。首先，需要将 Spring Integration 添加到项目构建中。对于 Maven，必要的依赖关系如下：

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-integration</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-integration-file</artifactId>
</dependency>
```

第一个依赖项是 Spring Integration 的 Spring Boot starter。无论 Spring Integration 流可能与什么集成，这种依赖关系都是开发 Spring Integration 流所必需的。与所有 Spring Boot starter 依赖项一样，它也存在于 Initializr 的复选框表中。

第二个依赖项是 Spring Integration 的文件端点模块。此模块是用于与外部系统集成的二十多个端点模块之一。我们将在第 10.2.9 节中更多地讨论端点模块。但是，就目前而言，要知道文件端点模块提供了将文件从文件系统提取到集成流或将数据从流写入文件系统的能力。

接下来，需要为应用程序创建一种将数据发送到集成流的方法，以便将数据写入文件。为此，将创建一个网关接口，如下面所示。

程序清单 10.1 消息网关接口，用于将方法转换为消息

```
package sia6;

import org.springframework.integration.annotation.MessagingGateway;
import org.springframework.integration.file.FileHeaders;
import org.springframework.messaging.handler.annotation.Header;

@MessagingGateway(defaultRequestChannel="textInChannel")
public interface FileWriterGateway {

    void writeToFile(
        @Header(FileHeaders.FILENAME) String filename,
        String data);

}
```

尽管它是一个简单的 Java 接口，但是关于 FileWriterGateway 还有很多要说的。首先会注意到它是由 @MessagingGateway 注解的。这个注解告诉 Spring Integration 在运行时生成这个接口的实现——类似于 Spring Data 如何自动生成存储库接口的实现。当需要编写文件时，代码的其他部分将使用这个接口。

@MessagingGateway 的 defaultRequestChannel 属性表示，对接口方法的调用产生的任何消息都应该发送到给定的消息通道。在本例中，声明从 writeToFile() 调用产生的任何消息都应该发送到名为 textInChannel 的通道。



对于 `writeToFile()` 方法，它接受一个文件名作为字符串，另一个字符串包含应该写入文件的文本。关于此方法签名，值得注意的是 `filename` 参数使用 `@Header` 进行了注解。在本例中，`@Header` 注解指示传递给 `filename` 的值应该放在消息头中（指定为 `FileHeaders`），解析为 `file_name` 的文件名，而不是在消息有效负载中。另一方面，数据参数值则包含在消息有效负载中。

现在已经有了一个消息网关，还需要配置集成流。尽管添加到构建中的 `Spring Integration starter` 依赖项支持 `Spring Integration` 的基本自动配置，但仍然需要编写额外的配置来定义满足应用程序需求的流。声明集成流的三个配置选项包括：

- XML 配置
- Java 配置
- 使用 DSL 进行 Java 配置

我们将对 `Spring Integration` 的这三种风格的配置进行讲解，从最原始的 XML 配置开始。

### 10.1.1 使用 XML 定义集成流

尽管在本书中我避免使用 XML 配置，但 Spring Integration 在 XML 中定义的集成流方面有着悠久的历史。因此，我认为值得至少展示一个 XML 定义的集成流示例。下面的程序清单显示了如何在 XML 中配置流。

程序清单 10.2 使用 Spring XML 配置定义集成流

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xmlns:int="http://www.springframework.org/schema/integration"
       xmlns:int-file="http://www.springframework.org/schema/integration/file"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
                           http://www.springframework.org/schema/beans/spring-beans.xsd
                           http://www.springframework.org/schema/integration
                           http://www.springframework.org/schema/integration/spring-integration.xsd
                           http://www.springframework.org/schema/integration/file
                           http://www.springframework.org/schema/integration/file/spring-integration-file.xsd">

  <int:channel id="textInChannel" />

  <int:transformer id="upperCase"
    input-channel="textInChannel"
    output-channel="fileWriterChannel"
    expression="payload.toUpperCase()" />

  <int:channel id="fileWriterChannel" />

  <int-file:outbound-channel-adapter id="writer"
    channel="fileWriterChannel"
    directory="/tmp/sia6/files"
    mode="APPEND"
    append-new-line="true" />
</beans>
```

分析一下程序清单 10.2 中的 XML：

- 配置了一个名为 `textInChannel` 的通道，这与为 `FileWriterGateway` 设置的请求通道是相同的。当在 `FileWriterGateway` 上调用 `writeToFile()` 方法时，结果消息被发布到这个通道。
- 配置了一个转换器来接收来自 `textInChannel` 的消息。它使用 Spring Expression Language (SpEL) 表达式在消息有效负载上调用 `toUpperCase()`。然后将大写操作的结果发布到 `fileWriterChannel` 中。
- 配置了一个名为 `fileWriterChannel` 的通道，此通道用作连接转换器和外部通道适配器的管道。
- 最后，使用 `int-file` 命名空间配置了一个外部通道适配器。这个 XML 命名空间由 Spring Integration 的文件模块提供，用于编写文件。按照配置，它将接收来自 `fileWriterChannel` 的消息，并将消息有效负载写到一个文件中，该文件的名称在消息的 `file_name` 头中指定，该文件位于 `directory` 属性中指定的目录中。如果文件已经存在，则将用换行来追加文件，而不是覆盖它。

图 10.1 显示了该流程，使用了与《企业级集成模式》中图形元素样式相同的图形。

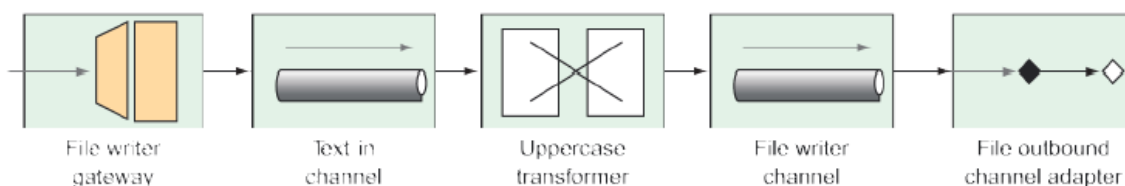


图 10.1 文件写入集成流程

该流由五个组件组成：网关、两个通道、转换器和通道适配器。这些只是可以组装成集成流的组件的一小部分。我们会在第 10.2 节中探索这些组件以及 Spring Integration 支持的其他组件。

如果希望在 Spring Boot 应用程序中使用 XML 配置，则需要将 XML 作为资源导入 Spring 应用程序。最简单的方法是在应用程序的 Java 配置类上使用 Spring 的 `@ImportResource` 注解：

```
@Configuration
@ImportResource("classpath:/filewriter-config.xml")
public class FileWriterIntegrationConfig { ... }
```

尽管基于 XML 的配置很好地服务于 Spring Integration，但大多数开发人员对使用 XML 越来越谨慎。（正如我所说的，我在本书中避免使用 XML 配置）让我们把这些尖括号放在一边，将注意力转向 Spring Integration 的 Java 配置风格。

## 10.1.2 在 Java 中配置集成流

大多数现代 Spring 应用程序都避开了 XML 配置，而采用了 Java 配置。实际上，在 Spring Boot 应用程序中，Java 配置是自动配置的自然补充。因此，如果要集成流添加到 Spring Boot 应用程序中，那么在 Java 中定义该流是很有意义的。

作为如何使用 Java 配置编写集成流的示例，请查看下面的程序清单。这显示了与以前相同的文件编写集成流，但这次是用 Java 编写的。

程序清单 10.3 使用 Java 配置定义集成流

```
package sia6;

import java.io.File;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.integration.annotation.ServiceActivator;
import org.springframework.integration.annotation.Transformer;
import org.springframework.integration.file.FileWritingMessageHandler;
import org.springframework.integration.file.support.FileExistsMode;
import org.springframework.integration.transformer.GenericTransformer;

@Configuration
public class FileWriterIntegrationConfig {

    @Bean
    @Transformer(inputChannel="textInChannel",
        outputChannel="fileWriterChannel")

    public GenericTransformer<String, String> upperCaseTransformer() {
        return text -> text.toUpperCase();
    }

    @Bean
    @ServiceActivator(inputChannel="fileWriterChannel")
    public FileWritingMessageHandler fileWriter() {
        FileWritingMessageHandler handler =
            new FileWritingMessageHandler(new File("/tmp/sia6/files"));
        handler.setExpectReply(false);
        handler.setFileExistsMode(FileExistsMode.APPEND);
        handler.setAppendNewLine(true);
        return handler;
    }

}
```

使用 Java 配置，可以声明两个 bean：一个转换器和一个文件写入消息处理程序。这里转换器是 `GenericTransformer`。因为 `GenericTransformer` 是一个函数接口，所以能够以 `lambda` 的形式提供在消息文本上调用 `toUpperCase()` 的实现。转换器的 bean 使用 `@Transformer` 进行注解，并将其指定为集成流中的转换器，该转换器接收名为 `textInChannel` 的通道上的消息，并将消息写入名为 `fileWriterChannel` 的通道。

至于文件写入 bean，它使用 `@ServiceActivator` 进行了注解，以指示它将接受来自 `fileWriterChannel` 的消息，并将这些消息传递给由 `FileWritingMessageHandler` 实例定义的服务。`FileWritingMessageHandler` 是一个消息处理程序，它使用消息的 `file_name` 头中指定的文件名将消息有效负载写入指定目录中的文件。与 XML 示例一样，将 `FileWritingMessageHandler` 配置为用换行符附加到文件中。

`FileWritingMessageHandler` bean 配置的一个独特之处是调用 `setExpectReply(false)` 来指示服务激活器不应该期望应答通道（通过该通道可以将值返回到流中的上游组件）。如果不调用 `setExpectReply()`，则文件写入 bean 默认为 `true`，尽管管道仍按预期工作，但将看到记录了一些错误，说明没有配置应答通道。

还会看到不需要显式地声明通道。如果不存在具有这些名称的 bean，就会自动创建 `textInChannel` 和 `fileWriterChannel` 通道。但是，如果希望对通道的配置方式有更多的控制，可以像这样显式地将它们构造为 bean：

```
@Bean
public MessageChannel textInChannel() {
    return new DirectChannel();
}
...
@Bean
public MessageChannel fileWriterChannel() {
    return new DirectChannel();
}
```

可以说，Java 配置选项更易于阅读，也更简洁，而且与我在本书中所追求的纯 Java 配置完全一致。但是，通过 Spring Integration 的 Java DSL（领域特定语言）配置风格，它可以变得更加精简。

### 10.1.3 使用 Spring Integration 的 DSL 配置

让我们进一步尝试定义文件编写集成流。这一次，仍然使用 Java 定义它，但是将使用 Spring Integration 的 Java DSL。不是为流中的每个组件声明一个单独的 bean，而是声明一个定义整个流的 bean。

程序清单 10.4 为设计集成流提高流式 API

```
package sia6;

import java.io.File;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.integration.dsl.IntegrationFlow;
import org.springframework.integration.dsl.IntegrationFlows;
import org.springframework.integration.dsl.MessageChannels;
import org.springframework.integration.file.dsl.Files;
import org.springframework.integration.file.support.FileExistsMode;

@Configuration
public class FileWriterIntegrationConfig {

    @Bean
    public IntegrationFlow fileWriterFlow() {
        return IntegrationFlows
            .from(MessageChannels.direct("textInChannel"))
            .<String, String>transform(t -> t.toUpperCase())
            .handle(Files
                .outboundAdapter(new File("/tmp/sia6/files"))
                .fileExistsMode(FileExistsMode.APPEND)
                .appendNewLine(true))
            .get();
    }
}
```

这个新配置尽可能简洁，用一个 bean 方法捕获整个流。IntegrationFlows 类初始化了这个构建者 API，可以从该 API 声明流。

在程序清单 10.4 中，首先从名为 textInChannel 的通道接收消息，然后该通道转到一个转换器，使消息有效负载大写。在转换器之后，消息由出站通道适配器处理，该适配器是根据 Spring Integration 的文件模块中提供的文件类型创建的。最后，调用 get() 构建要返回的 IntegrationFlow。简而言之，这个 bean 方法定义了与 XML 和 Java 配置示例相同的集成流。

注意，与 Java 配置示例一样，不需要显式地声明通道 bean。虽然引用了 textInChannel，但它是由 Spring Integration 自动创建的，因为没有使用该名称的现有通道 bean。但是如果需要，可以显式地声明通道 bean。

至于连接转换器和外部通道适配器的通道，甚至不需要通过名称引用它。如果需要显式配置通道，可以在流定义中通过调用 channel() 的名称引用：

```

@Bean
public IntegrationFlow fileWriterFlow() {
    return IntegrationFlows
        .from(MessageChannels.direct("textInChannel"))
        .<String, String>transform(t -> t.toUpperCase())
        .channel(MessageChannels.direct("FileWriterChannel"))
        .handle(Files
            .outboundAdapter(new File("/tmp/sia6/files"))
            .fileExistsMode(FileExistsMode.APPEND)
            .appendNewLine(true))
        .get();
}

```

在使用 **Spring Integration** 的 **Java DSL**（与任何流式 **API** 一样）时要记住的一件事是，必须巧妙地使用空白来保持可读性。在这里给出的示例中，我小心地缩进了行以表示相关代码块。对于更长、更复杂的流，甚至可以考虑将流的一部分提取到单独的方法或子流中，以获得更好的可读性。

现在已经看到了使用三种不同配置风格定义的简单流，让我们回过头来看看 **Spring Integration** 的全貌。

## 10.2 探索 Spring Integration

Spring Integration 涵盖了许多集成场景。试图将所有这些内容都包含在一个章节中，就像试图将大象装进一个信封一样。我将展示一张 **Spring Integration** 大象的照片，而不是对 **Spring Integration** 进行全面的讨论，以便让您了解它是如何工作的。然后，将创建一个向 **Taco Cloud** 应用程序添加功能的集成流。

集成流由以下一个或多个组件组成。在编写更多代码之前，我们将简要地了解一下这些组件在集成流中所扮演的角色：

- **Channels** —— 将信息从一个元素传递到另一个元素。
- **Filters** —— 有条件地允许基于某些标准的消息通过流。
- **Transformers** —— 更改消息值或将消息有效负载从一种类型转换为另一种类型。
- **Routers** —— 直接将信息发送到几个渠道之一，通常是基于消息头。
- **Splitters** —— 将收到的信息分成两条或多条，每条都发送到不同的渠道。
- **Aggregators** —— 与分离器相反，它将来自不同渠道的多条信息组合成一条信息。
- **Service activators** —— 将消息传递给某个 **Java** 方法进行处理，然后在输出通道上发布返回值。
- **Channel adapters** —— 将通道连接到某些外部系统或传输。可以接受输入，也可以向外部系统写入。
- **Gateways** —— 通过接口将数据传递到集成流。

在定义文件写入集成流时，您已经看到了其中的一些组件。**FileWriterGateway** 接口是将应用程序提交的文本写入文件的网关。还定义了一个转换器来将给定的文本转换为大写；然后声明一个服务网关，它执行将文本写入文件的任务。这个流有两个通道：**textInChannel** 和 **fileWriterChannel**，它们将其他组件相互连接起来。现在，按照承诺快速浏览一下集成流组件。



## 10.2.1 消息通道

消息通道意指消息移动的集成管道（见图 10.2）。它们是连接 Spring Integration 所有其他部分的管道。

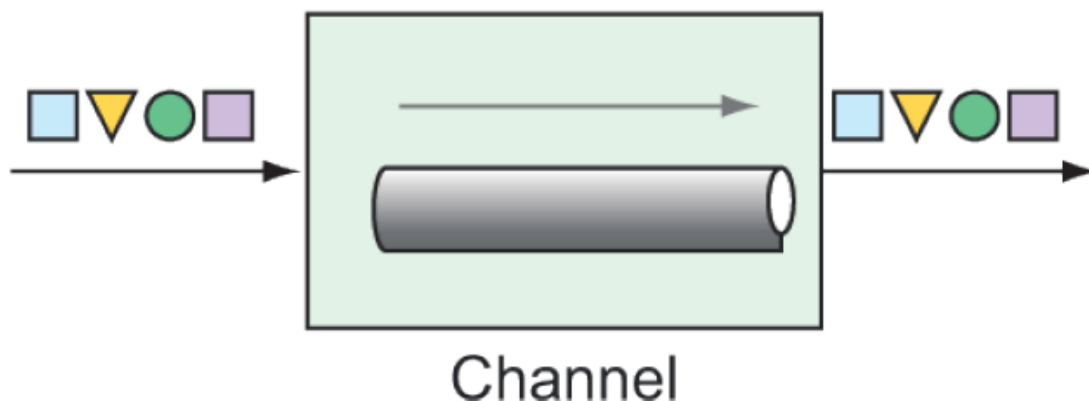


图 10.2 消息通道是数据在网络中的其他组件之间流动的管道整合流程。

Spring Integration 提供了多个管道的实现，包括以下这些：

- **PublishSubscribeChannel** —— 消息被发布到 **PublishSubscribeChannel** 后又被传递给一个或多个消费者。如果有多个消费者，他们都将收到消息。
- **QueueChannel** —— 消息被发布到 **QueueChannel** 后被存储到一个队列中，直到消息被消费者以先进先出（FIFO）的方式拉取。如果有多个消费者，他们中只有一个能收到消息。
- **PriorityChannel** —— 与 **QueueChannel** 类似，但是与 FIFO 方式不同，消息被冠以 **priority** 的消费者拉取。
- **RendezvousChannel** —— 与 **QueueChannel** 期望发送者阻塞通道，直到消费者接收这个消息类似，这种方式有效的同步了发送者与消费者。
- **DirectChannel** —— 与 **PublishSubscribeChannel** 类似，但是是通过在与发送方相同的线程中调用消费者来将消息发送给单个消费者，此通道类型允许事务跨越通道。
- **ExecutorChannel** —— 与 **DirectChannel** 类似，但是消息分派是通过 **TaskExecutor** 进行的，在与发送方不同的线程中进行，此通道类型不支持事务跨通道。
- **FluxMessageChannel** —— **Reactive Streams Publisher** 基于 **Project Reactor Flux** 的消息通道。（我们将在第 12 章讨论 **Reactive Streams**、**Reactor** 和 **Flux**）

在 Java 配置和 Java DSL 样式中，输入通道都是自动创建的，默认是 **DirectChannel**。但是，如果希望使用不同的通道实现，则需要显式地将通道声明为 **bean** 并在集成流中引用它。例如，要声明

**PublishSubscribeChannel**，需要声明以下 **@Bean** 方法：

```
@Bean
public MessageChannel orderChannel() {
    return new PublishSubscribeChannel();
}
```

然后在集成流定义中通过名称引用这个通道。例如，如果一个服务 **activator bean** 正在使用这个通道，那么可以在 **@ServiceActivator** 的 **inputChannel** 属性中引用它：

```
@ServiceActivator(inputChannel="orderChannel")
```

或者，如果使用 **Java DSL** 配置方式，需要通过调用 **channel()** 方法引用它：

```
@Bean
public IntegrationFlow orderFlow() {
    return IntegrationFlows
        ...
        .channel("orderChannel")
        ...
        .get();
}
```

需要注意的是，如果使用 **QueueChannel**，则必须为使用者配置一个轮询器。例如，假设已经声明了一个这样的 **QueueChannel** bean：

```
@Bean
public MessageChannel orderChannel() {
    return new QueueChannel();
}
```

需要确保将使用者配置为轮询消息通道。在服务激活器的情况下，**@ServiceActivator** 注解可能是这样的：

```
@ServiceActivator(inputChannel="orderChannel",
    poller=@Poller(fixedRate="1000"))
```

在本例中，服务激活器每秒（或 1,000 ms）从名为 **orderChannel** 的通道轮询一次。

## 10.2.2 过滤器

过滤器可以放置在集成管道的中间，以允许或不允许消息进入流中的下一个步骤（如图 10.3）。

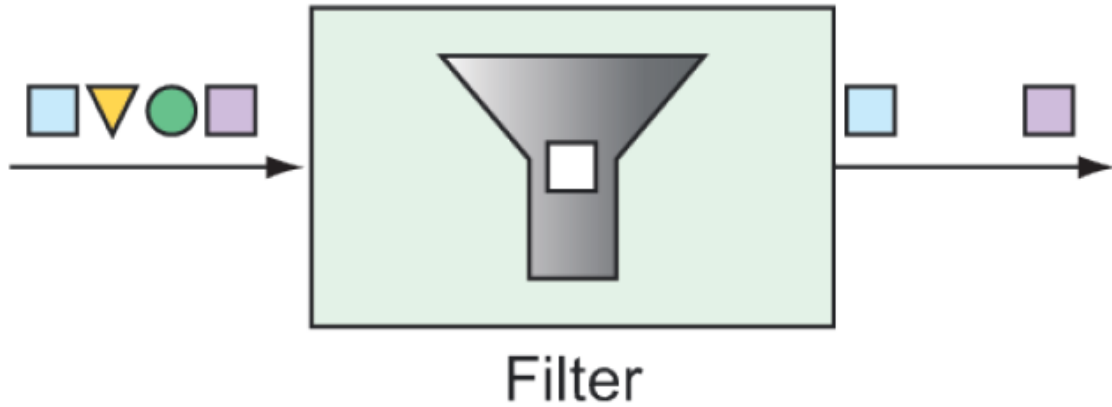


图 10.3 基于某些条件的过滤器允许或不允许消息在管道中继续。

例如，假设包含整数值的消息通过名为 `numberChannel` 的通道发布，但是只希望偶数传递到名为 `evenNumberChannel` 的通道。在这种情况下，可以使用 `@Filter` 注解声明一个过滤器，如下所示：

```
@Filter(inputChannel="numberChannel",
        outputChannel="evenNumberChannel")
public boolean evenNumberFilter(Integer number) {
    return number % 2 == 0;
}
```

或者，如果您使用 `Java DSL` 配置风格来定义集成流，您可以这样调用 `filter()`：

```
@Bean
public IntegrationFlow evenNumberFlow(AtomicInteger integerSource) {
    return IntegrationFlows
        ...
        .<Integer>filter((p) -> p % 2 == 0)
        ...
        .get();
}
```

在本例中，使用 `lambda` 表达式实现过滤器。但是，事实上，`filter()` 方法是接收一个 `GenericSelector` 作为参数。这意味着可以实现 `GenericSelector` 接口，而不是引入一个简略的 `lambda` 表达式实现过滤。

### 10.2.3 转换器

转换器对消息执行一些操作，通常会产生不同的消息，并且可能会产生不同的负载类型（如图 10.4）。转换可以是一些简单的事情，例如对数字执行数学运算或操作 `String` 字符串值；转换也会很复杂，例如使用表示 ISBN 的 `String` 字符串值来查找并返回相应书籍的详细信息。

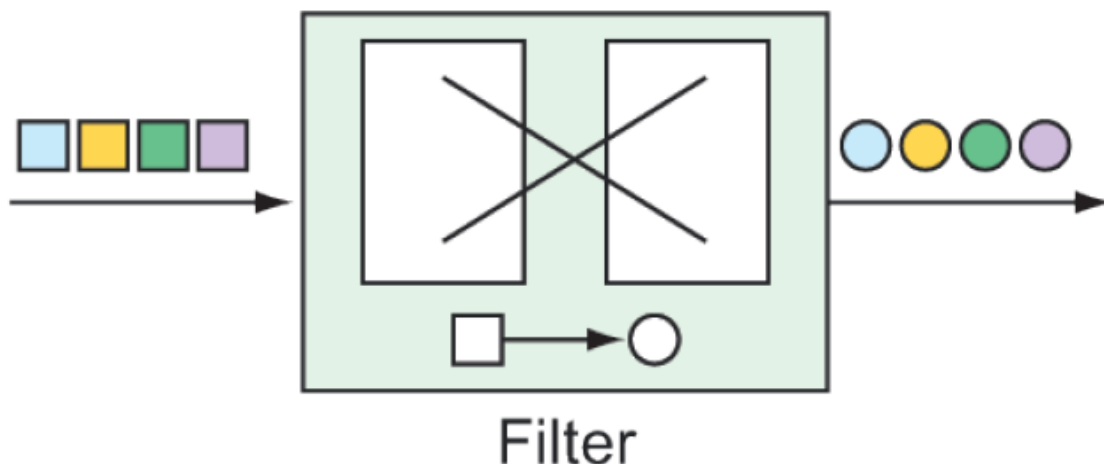


图 10.4 转换器在消息流经集成流时对其进行变形。

例如，假设正在一个名为 `numberChannel` 的通道上发布整数值，并且希望将这些数字转换为包含等效罗马数字的 `String` 字符串。在这种情况下，可以声明一个 `GenericTransformer` 类型的 `bean`，并添加 `@Transformer` 注解，如下所示：

```
@Bean
@Transformer(inputChannel="numberChannel",
             outputChannel="romanNumberChannel")
public GenericTransformer<Integer, String> romanNumTransformer() {
    return RomanNumbers::toRoman;
}
```

通过 `@Transformer` 注解将 `bean` 指定为 `transformer bean`，它从名为 `numberChannel` 的通道接收整数值，并使用 `toRoman()`（`toRoman()` 方法是在一个名为 `RomanNumbers` 的类中静态定义的，并在这里通过方法引用进行引）的静态方法进行转换，得到的结果被发布到名为 `romanNumberChannel` 的通道中。

在 `Java DSL` 配置风格中，调用 `transform()` 甚至更简单，将方法引用传递给 `toRoman()` 方法即可：

```
@Bean
public IntegrationFlow transformerFlow() {
    return IntegrationFlows
        ...
        .transform(RomanNumbers::toRoman)
        ...
        .get();
}
```

虽然在两个 `transformer` 代码示例中都使用了方法引用，但是要知道 `transformer` 也可以使用 `lambda` 表达式。或者，如果 `transformer` 比较复杂，需要单独的成为一个 `Java` 类，可以将它作为 `bean` 注入流配置，并将引用传递给 `transform()` 方法：

```

@Bean
public RomanNumberTransformer romanNumberTransformer() {
    return new RomanNumberTransformer();
}
@Bean
public IntegrationFlow transformerFlow(
    RomanNumberTransformer romanNumberTransformer) {
    return IntegrationFlows
        ...
        .transform(romanNumberTransformer)
        ...
        .get();
}

```

在这里，声明了一个 `RomanNumberTransformer` 类型的 bean，它本身是 `Spring Integration` 的 `Transformer` 或 `GenericTransformer` 接口的实现。bean 被注入到 `transformerFlow()` 方法，并在定义集成流时传递给 `transform()` 方法。

## 10.2.4 路由

基于某些路由标准的路由器允许在集成流中进行分支，将消息定向到不同的通道（如图 10.5）。

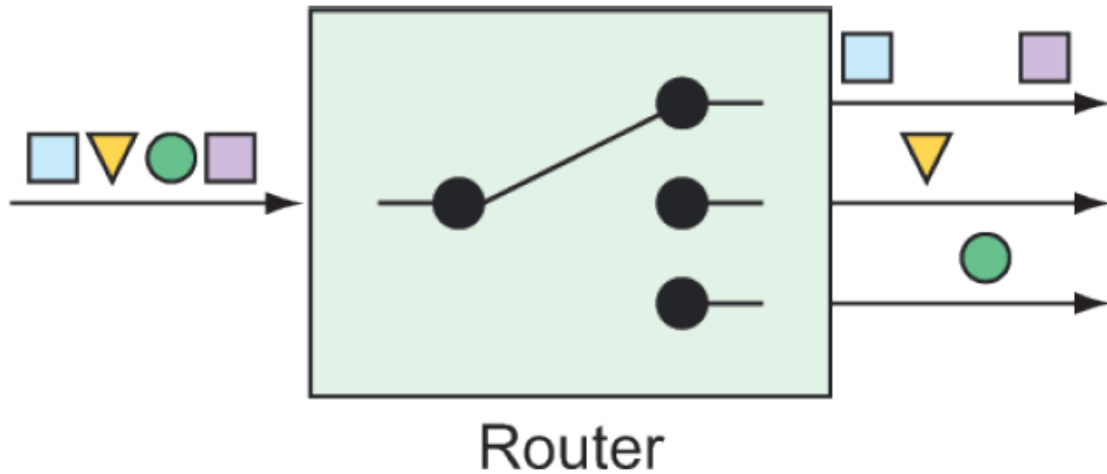


图 10.5 基于应用于路由器的一些标准，路由器将消息直接发送到不同的通道信息。

例如，假设有一个名为 `numberChannel` 的通道，整数值通过它流动。假设希望将所有偶数消息定向到一个名为 `evenChannel` 的通道，而将奇数消息定向到一个名为 `oddChannel` 的通道。要在集成流中创建这样的路由，可以声明一个 `AbstractMessageRouter` 类型的 `bean`，并使用 `@Router` 注解该 `bean`：

```
@Bean
@Router(inputChannel="numberChannel")
public AbstractMessageRouter evenOddRouter() {
    return new AbstractMessageRouter() {
        @Override
        protected Collection<MessageChannel>
            determineTargetChannels(Message<?> message) {
            Integer number = (Integer) message.getPayload();
            if (number % 2 == 0) {
                return Collections.singleton(evenChannel());
            }
            return Collections.singleton(oddChannel());
        }
    };
}

@Bean
public MessageChannel evenChannel() {
    return new DirectChannel();
}

@Bean
public MessageChannel oddChannel() {
    return new DirectChannel();
}
```

这里声明的 `AbstractMessageRouter` `bean` 接受来自名为 `numberChannel` 的输入通道的消息。定义为匿名内部类的实现检查消息有效负载，如果它是偶数，则返回名为 `evenChannel` 的通道（在路由器 `bean` 之后声明为 `bean`）。否则，通道有效载荷中的数字必须为奇数；在这种情况下，将返回名为 `oddChannel` 的通道（也在 `bean` 声明方法中声明）。

在 `Java DSL` 形式中，路由器是通过在流定义过程中调用 `route()` 来声明的，如下所示：

```

@Bean
public IntegrationFlow numberRoutingFlow(AtomicInteger source) {
    return IntegrationFlows
        ...
        .<Integer, String>route(n -> n%2==0 ? "EVEN":"ODD", mapping -> mapping
            .subFlowMapping("EVEN", sf -> sf
                .<Integer, Integer>transform(n -> n * 10)
                .handle((i,h) -> { ... })
            )
            .subFlowMapping("ODD", sf -> sf
                .transform(RomanNumbers::toRoman)
                .handle((i,h) -> { ... })
            )
        )
        .get();
}

```

虽然仍然可以声明 **AbstractMessageRouter** 并将其传递给 `route()`，但是本例使用 **lambda** 表达式来确定消息有效负载是奇数还是偶数。如果是偶数，则返回一个偶数的字符串值。如果是奇数，则返回奇数。然后使用这些值来确定哪个子映射将处理消息。

## 10.2.5 拆分器

有时，在集成流中，将消息拆分为多个独立处理的消息可能很有用。**Splitter**（如图 10.6）将为分割并处理这些消息。

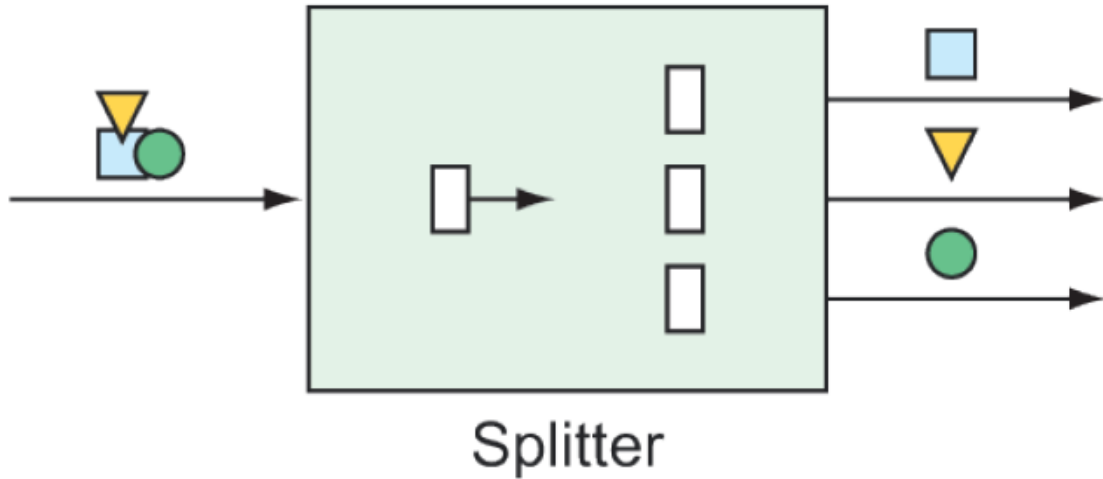


图 10.6 拆分器将消息分解为两个或多个可由单独的子流。

**Splitter** 在很多情况下都很有用，但是有两个基本用例可以使用 **Splitter**：

- 消息有效载荷，包含单个消息有效载荷相同类型的项的集合。例如，携带产品列表的消息可能被分成多个消息，每个消息的有效负载是一个产品。
- 信息有效载荷，携带的信息虽然相关，但可以分为两种或两种以上不同类型的信息。例如，购买订单可能包含交付、帐单和行项目信息。交付细节可能由一个子流程处理，账单由另一个子流程处理，每一项则由另一个子流程处理。在这个用例中，**Splitter** 后面通常跟着一个路由器，它根据有效负载类型路由消息，以确保正确的子流处理数据。

当将消息有效负载拆分为两个或多个不同类型的消息时，通常只需定义一个 **POJO** 即可，该 **POJO** 提取传入的有效负载的各个部分，并将它们作为集合的元素返回。

例如，假设希望将携带购买订单的消息拆分为两条消息：一条携带账单信息，另一条携带项目列表。下面的 **OrderSplitter** 将完成这项工作：

```
public class OrderSplitter {
    public Collection<Object> splitOrderIntoParts(PurchaseOrder po) {
        ArrayList<Object> parts = new ArrayList<>();
        parts.add(po.getBillingInfo());
        parts.add(po.getLineItems());
        return parts;
    }
}
```

然后，可以使用 `@Splitter` 注解将 **OrderSplitter** bean 声明为集成流的一部分，如下所示：

```
@Bean
@Splitter(inputChannel="poChannel",
        outputChannel="splitOrderChannel")
public OrderSplitter orderSplitter() {
    return new OrderSplitter();
}
```



在这里，购买订单到达名为 `poChannel` 的通道，并被 `OrderSplitter` 分割。然后，将返回集合中的每个项作为集成流中的单独消息发布到名为 `splitOrderChannel` 的通道。在流的这一点上，可以声明一个 `PayloadTypeRouter` 来将账单信息和项目，并路由到它们自己的子流：

```
@Bean
@Router(inputChannel="splitOrderChannel")
public MessageRouter splitOrderRouter() {
    PayloadTypeRouter router = new PayloadTypeRouter();
    router.setChannelMapping(
        BillingInfo.class.getName(), "billingInfoChannel");
    router.setChannelMapping(
        List.class.getName(), "lineItemsChannel");
    return router;
}
```

顾名思义，`PayloadTypeRouter` 根据消息的有效负载类型将消息路由到不同的通道。按照这里的配置，将有效负载为类型为 `BillingInfo` 的消息路由到一个名为 `billingInfoChannel` 的通道进行进一步处理。至于项目信息，它们在 `java.util.List` 集合包中；因此，可以将 `List` 类型的有效负载映射到名为 `lineItemsChannel` 的通道中。

按照目前的情况，流分为两个子流：一个是 `BillingInfo` 对象流，另一个是 `List<LineItem>` 流。但是，如果想进一步分割它，而不是处理 `LineItem` 列表，而是分别处理每个 `LineItem`，该怎么办呢？要将列表拆分为多个消息（每个行项对应一条消息），只需编写一个方法（而不是 `bean`），该方法使用 `@Splitter` 进行注解，并返回 `LineItems` 集合，可能类似如下：

```
@Splitter(inputChannel="lineItemsChannel", outputChannel="lineItemChannel")
public List<LineItem> lineItemSplitter(List<LineItem> lineItems) {
    return lineItems;
}
```

当携带 `List<LineItem>` 的有效负载的消息到达名为 `lineItemsChannel` 的通道时，它将传递到 `lineItemSplitter()` 方法。根据 `Splitter` 的规则，该方法必须返回要 `Splitter` 的项的集合。在本例中，已经有了 `LineItems` 的集合，因此只需直接返回该集合。因此，集合中的每个 `LineItem` 都以其自己的消息形式发布到名为 `lineItemChannel` 的通道。

如果您想使用 `Java DSL` 来声明相同的 `Splitter/Router` 配置，您可以调用 `split()` 和 `route()`：

```
return IntegrationFlows
...
    .split(orderSplitter())
    .<Object, String> route(
        p -> {
            if (p.getClass().isAssignableFrom(BillingInfo.class)) {
                return "BILLING_INFO";
            } else {
                return "LINE_ITEMS";
            }
        }, mapping -> mapping
        .subFlowMapping("BILLING_INFO", sf -> sf
            .<BillingInfo> handle((billingInfo, h) -> {
                ...
            })
        .subFlowMapping("LINE_ITEMS", sf -> sf
            .split()
            .<LineItem> handle((lineItem, h) -> {
                ...
            })
        )
    ).get();
```

流定义的 DSL 形式当然更简洁，如果不是说更难理解的话。跟随通过将 `lambda` 提取到方法，可以稍微清理一下这一点。例如 以下三种方法可用于替换流定义中使用的 `lambda`：

```
private String route(Object p) {
    return p.getClass().isAssignableFrom(BillingInfo.class)
        ? "BILLING_INFO"
        : "LINE_ITEMS";
}
private BillingInfo handleBillingInfo(
    BillingInfo billingInfo, MessageHeaders h) {
    // ...
}
private LineItem handleLineItems(
    LineItem lineItem, MessageHeaders h) {
    // ...
}
```

然后，可以使用如下方法引用重写集成流：

```
return IntegrationFlows
    ...
    .split()
    .route(
        this::route,
        mapping -> mapping
            .subFlowMapping("BILLING_INFO", sf -> sf
                .<BillingInfo> handle(this::handleBillingInfo))
            .subFlowMapping("LINE_ITEMS", sf -> sf
                .split()
                .<LineItem> handle(this::handleLineItems)));
```

无论哪种方式，它都使用与 Java 配置示例相同的 `OrderSplitter` 来分割订单。在订单被分割之后，它按其类型路由到两个单独的子流。

## 10.2.6 服务激活器

服务激活器从输入信道接收消息并发送这些消息给的 **MessageHandler**，如图 10.7。

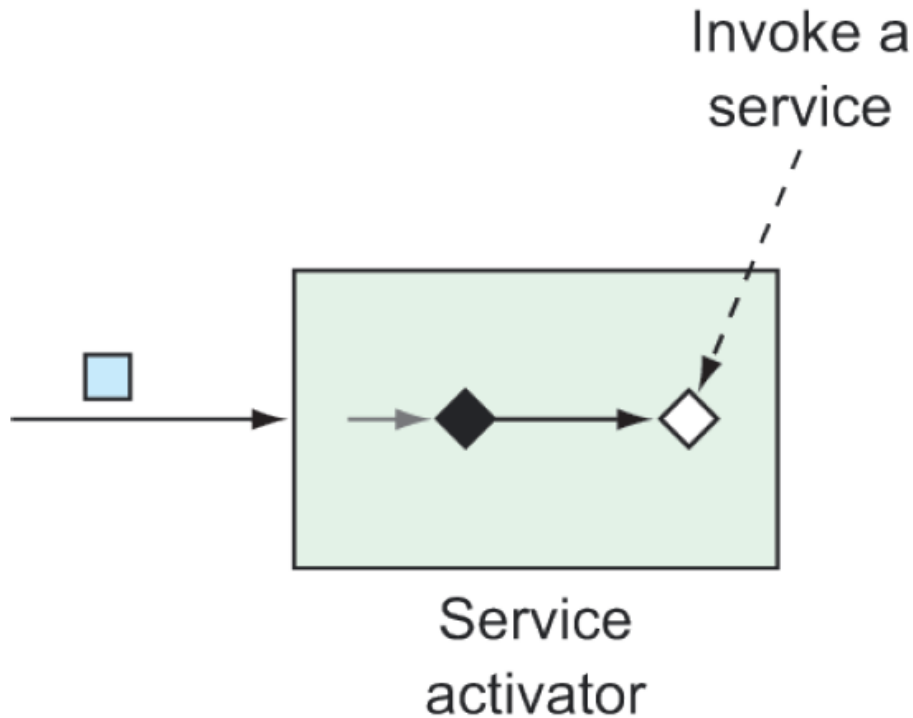


图 10.7 服务激活器在收到消息时通过 **MessageHandler** 调用某些服务。

Spring 集成提供了多种的 **MessageHandler** 实现开箱即用（**PayloadTypeRouter** 就是 **MessageHandler** 的实现），但您会经常需要提供一些定制实现充当服务激活。作为一个例子，下面的代码说明了如何声明的 **MessageHandler** bean，构成为一个服务激活器：

```
@Bean
@ServiceActivator(inputChannel="someChannel")
public MessageHandler sysoutHandler() {
    return message -> {
        System.out.println("Message payload: " + message.getPayload());
    };
}
```

通过 `@ServiceActivator` 注解 bean，将其指定为一个服务激活器，从所述信道处理消息命名 `someChannel`。至于 **MessageHandler** 的本身，它是通过一个 **lambda** 实现。虽然这是一个简单的 **MessageHandler**，给定的消息时，它发出其有效载荷的标准输出流。

另外，可以声明一个服务激活器，用于在返回一个新的有效载荷之前处理传入的消息。在这种情况下，这个 bean 应该是一个 **GenericHandler** 而非的 **MessageHandler**：

```

@Bean
@ServiceActivator(inputChannel="orderChannel",
    outputChannel="completeChannel")
public GenericHandler<EmailOrder> orderHandler(
    OrderRepository orderRepo) {
    return (payload, headers) -> {
        return orderRepo.save(payload);
    };
}

```

在这种情况下，服务激活器是一个 **GenericHandler**，其中的有效载荷为 **Order** 类型。当订单到达，它是通过 **repository** 进行保存；保存 **Order** 后产生的结果被发送到名称为 **completeChannel** 的输出通道。

注意，**GenericHandler** 不仅给出了有效载荷，还有消息头（即使该示例不使用任何形式的头信息）。同时也可以通过传递了 **MessageHandler** 或 **GenericHandler** 去调用在流定义中的 **handler()** 方法，来使用在 **Java DSL** 配置式中的服务激活器：

```

public IntegrationFlow someFlow() {
    return IntegrationFlows
        ...
        .handle(msg -> {
            System.out.println("Message payload: " + msg.getPayload());
        })
        .get();
}

```

在这种情况下，**MessageHandler** 是作为一个 **lambda**，但也可以将它作为一个参考方法甚至是一个类，它实现了 **MessageHandler** 接口。如果给它一个 **lambda** 或方法引用，要知道，它是接受一个消息作为参数。

类似地，如果服务激活器不是流的结束，**handler()** 可以写成接受 **GenericHandler** 参数。从之前应用订单存储服务激活器来看，可以使用 **Java DSL** 对流程进行配置：

```

public IntegrationFlow orderFlow(OrderRepository orderRepo) {
    return IntegrationFlows
        ...
        .<EmailOrder>handle((payload, headers) -> {
            return orderRepo.save(payload);
        })
        ...
        .get();
}

```

当利用 **GenericHandler** 时，**lambda** 表达式或方法参考接受该消息的有效载荷和报头作为参数。另外，如果选择在一个流程的结束使用 **GenericHandler**，需要返回 **null**，否则会得到这表明有没有指定输出通道的错误。

## 10.2.7 网关

网关是通过一个应用程序可以将数据提交到一个集成信息流和接收这是该流的结果的响应的装置。通过 **Spring Integration** 实现的，网关是实现为应用程序可以调用将消息发送到集成信息流的接口（如图 10.8）。

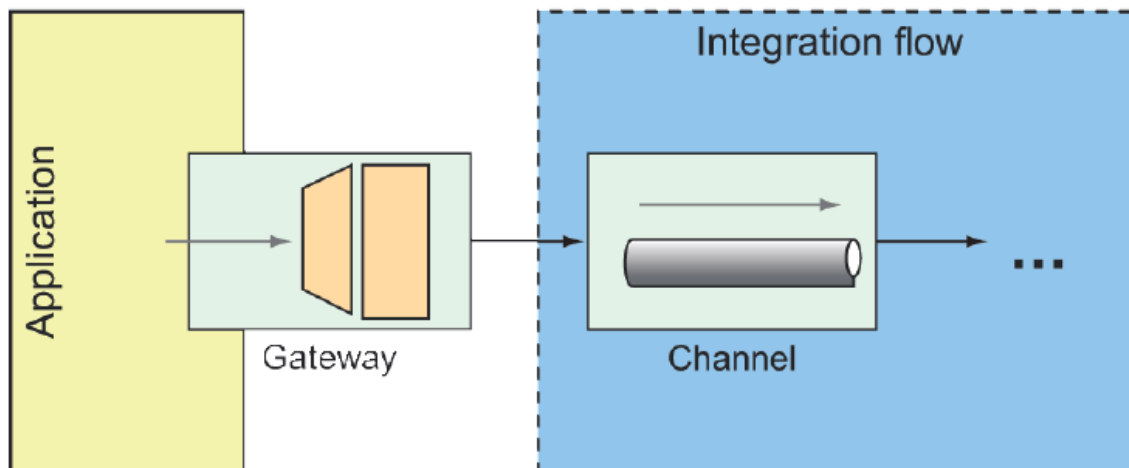


图 10.8 服务网关是应用程序可以通过其向用户提交消息的接口整合流程。

您已经见过 **FileWriterGateway** 消息网关的例子。**FileWriterGateway** 是单向网关，它的方法接受 **String** 作为参数，将其写入到文件中，返回 **void**。同样，编写一个双向网关也很容易。当写网关接口时，确保该方法返回某个值发布到集成流程即可。

作为一个例子，假设一个网关处理接受一个 **String** 的简单集成信息流，并把特定的 **String** 转成大写。网关接口可能是这个样子：

```
package sia6;
import org.springframework.integration.annotation.MessagingGateway;
import org.springframework.stereotype.Component;

@Component
@MessagingGateway(defaultRequestChannel="inChannel",
    defaultReplyChannel="outChannel")
public interface UpperCaseGateway {
    String uppercase(String in);
}
```

令人惊叹的是，没有必要实现这个接口。**Spring Integration** 自动提供运行时实现，这个实现会使用特定的通道进行数据的发送与接收。

当 `uppercase()` 被调用时，给定的 **String** 被发布到名为 `inChannel` 的集成流通道中。而且，不管流是如何定义的或是它是做什么的，在当数据到达名为 `outChannel` 通道时，它从 `uppercase()` 方法中返回。

至于 `uppercase` 集成流，它只有一个单一的步骤，把 **String** 转换为大写一个简单的集成流。以下是使用 **Java DSL** 配置：

```
@Bean
public IntegrationFlow uppercaseFlow() {
    return IntegrationFlows
        .from("inChannel")
        .<String, String> transform(s -> s.toUpperCase())
        .channel("outChannel")
        .get();
}
```

正如这里所定义的，流程启动于名为 **inChannel** 的通道获得数据输入的时候。然后消息的有效负载通过转换器去执行变成大写字母的操作，这里的操作都使用 **lambda** 表达式进行定义。消息的处理结果被发布到名为 **outChannel** 的通道中，这个通道就是已经被声明为 **UpperCaseGateway** 接口的答复通道。

## 10.2.8 通道适配器

通道适配器代表集成信息流的入口点和出口点。数据通过入站信道适配器的方式进入到集成流中，通过出站信道适配器的方式离开集成流。如图 10.9。

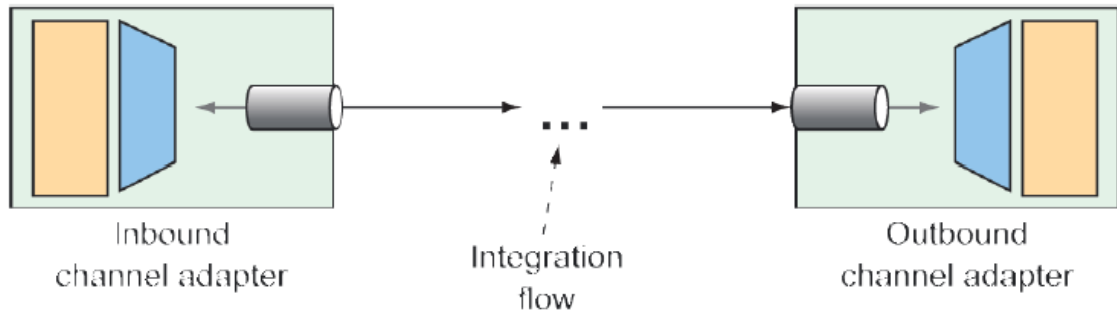


图 10.9 通道适配器是集成流的入口和出口点。

入站信道的适配器可以采取多种形式，这取决于它们引入到流的数据源。例如，声明一个入站通道适配器，它采用从 `AtomicInteger` 到流递增的数字。使用 `Java` 配置，它可能是这样的：

```
@Bean
@InboundChannelAdapter(
    poller=@Poller(fixedRate="1000"), channel="numberChannel")
public MessageSource<Integer> numberSource(AtomicInteger source) {
    return () -> {
        return new GenericMessage<>(source.getAndIncrement());
    };
}
```

此 `@Bean` 方法声明了一个入站信道适配器 `bean`，后面跟随着 `@InboundChannelAdapter` 注解，它们每 1 秒（1000 ms）从注入的 `AtomicInteger` 提交一个数字到名 `numberChannel` 的通道中。

当使用 `Java` 配置时，`@InboundChannelAdapter` 意味着是一个入站通道适配器，`from()` 方法就是使用 `Java DSL` 来定义流的时候，表明它是怎么处理的。下面对于流定义的一个片段展示了在 `Java DSL` 配置中类似的输入通道适配器：

```
@Bean
public IntegrationFlow someFlow(AtomicInteger integerSource) {
    return IntegrationFlows
        .from(integerSource, "getAndIncrement",
            c -> c.poller(Pollers.fixedRate(1000)))
        ...
        .get();
}
```

通常情况下，通道适配器通过的 `Spring Integration` 的多端点模块之一进行提供。举个例子，假设需要一个入站通道适配器，用它来监视指定的目录，同时将任何写入到那个目录中的文件作为消息，提交到名为 `file-channel` 的通道中。下面的 `Java` 配置使用 `FileReadingMessageSource` 从 `Spring Integration` 的文件端点模块来实现这一目标：

```

@Bean
@InboundChannelAdapter(channel="file-channel",
    poller=@Poller(fixedDelay="1000"))
public MessageSource<File> fileReadingMessageSource() {
    FileReadingMessageSource sourceReader = new FileReadingMessageSource();
    sourceReader.setDirectory(new File(INPUT_DIR));
    sourceReader.setFilter(new SimplePatternFileListFilter(FILE_PATTERN));
    return sourceReader;
}

```

当在 **Java DSL** 中写入同样的 **file-reading** 进站通道适配器时，来自 **Files** 类的 `inboundAdapter()` 方法达到的同样的目的。出站通道适配器位于集成信息流的最后位置，将最终消息扇出到应用程序或是其他系统中：

```

@Bean
public IntegrationFlow fileReaderFlow() {
    return IntegrationFlows
        .from(Files.inboundAdapter(new File(INPUT_DIR))
            .patternFilter(FILE_PATTERN))
        .get();
}

```

服务激活器（作为消息处理的实现）往往是为出站通道适配器而存在的，特别是当数据需要被扇出到应用程序本身的时候。

值得一提的，**Spring Integration** 的端点模块为几种常见的用例提供了有用的消息处理程序。如在程序清单 10.3 中所看到的 **FileWritingMessageHandler** 出站通道适配器，这就是一个很好的例子。说到 **Spring Integration** 端点模块，让我们快速浏览一下准备使用的集成端点模块。



## 10.2.9 端点模块

Spring Integration 可以让您创建自己的通道适配器，这是很棒的。但是，更棒的是 Spring Integration 提供了包含通道超过两打的端点模块适配器，包括入站和出站，用于与各种常用外部系统进行集成，如表 10.1 所示。

表 10.1 Spring Integration 提供了二十多个用于集成外部系统的端点模块。

| 模块                        | 依赖的 <b>Artifact ID</b>       |
|---------------------------|------------------------------|
| AMQP                      | spring-integration-amqp      |
| Spring application events | spring-integration-event     |
| RSS and Atom              | spring-integration-feed      |
| Filesystem                | spring-integration-file      |
| FTP/FTPS                  | spring-integration-ftp       |
| GemFire                   | spring-integration-gemfire   |
| HTTP                      | spring-integration-http      |
| JDBC                      | spring-integration-jdbc      |
| JPA                       | spring-integration-jpa       |
| JMS                       | spring-integration-jms       |
| Email                     | spring-integration-mail      |
| MongoDB                   | spring-integration-mongodb   |
| MQTT                      | spring-integration-mqtt      |
| R2DBC                     | spring-integration-r2dbc     |
| Redis                     | spring-integration-redis     |
| RMI                       | spring-integration-rmi       |
| SFTP                      | spring-integration-sftp      |
| STOMP                     | spring-integration-stomp     |
| Stream                    | spring-integration-stream    |
| Syslog                    | spring-integration-syslog    |
| TCP/UDP                   | spring-integration-ip        |
| WebFlux                   | spring-integration-webflux   |
| Web Services              | spring-integration-ws        |
| WebSocket                 | spring-integration-websocket |
| XMPP                      | spring-integration-xmpp      |
| ZeroMQ                    | spring-integration-zeromq    |
| ZooKeeper                 | spring-integration-zookeeper |

从表 10.1 可以清楚的看出 **Spring Integration** 提供了一套广泛的组件，以满足众多集成的需求。大多数应用程序一点都不会用到 **Spring Integration** 提供的功能。但是，如果您需要它们，很好，**Spring Integration** 几乎都能覆盖到。

更重要的是，本章在表 10.1 中列出模块，不可能涵盖提供的所有通道适配器。您已经看到，使用文件系统模块写入到文件系统的例子。而您很快就要使用电子邮件模块读取电子邮件。

每个端点模块提供通道适配器，当使用 **Java** 配置时，可以被声明为 **bean**，当时应 **Java DSL** 配置时，可以通过静态方法进行引用。鼓励您去探索您最感兴趣的任何端点模块。您会发现它们的使用方法相当一致。但现在，让我们把关注点转向电子邮件端点模块，看看在 **Taco Cloud** 应用程序中如何使用它。

## 10.3 创建 Email 集成流

Taco Cloud 应该能让它的用户通过 email 提交他们的 taco 设计和放置订单。您在报纸上通过发送传单和放置外卖广告，邀请大家通过电子邮件发送 taco 订单。这样做很成功！不幸的是，它有点太过于"成功"了。有这么多的电子邮件中，您必须雇用临时工做一些，无非就是阅读完所有的信件，并提交订单的详细信息到订购系统来的工作。

在本节中，将实现一个集成信息流，用于轮询 Taco Cloud 收件箱中的 taco 订单电子邮件，并解析邮件订单的详细信息，然后提交订单到 Taco Cloud 进行处理。总之，您将从邮箱端点模块中使用入站通道适配器，用于把 Taco Cloud 收件箱中的邮件提取到集成中。

下一步，在集成信息流中，电子邮件将被解析为订单对象，接着被扇出到另外一个向 Taco Cloud 的 REST API 提交订单的处理器中，在那里，它们将如同其他订单一样被处理。首先，让我们定义一个简单的配置属性的类，来捕获如何处理 Taco Cloud 电子邮件的细节：

```
package tacos.email;

import org.springframework.boot.context.properties.ConfigurationProperties;
import org.springframework.stereotype.Component;
import lombok.Data;

@Data
@ConfigurationProperties(prefix="tacocloud.email")
@Component
public class EmailProperties {

    private String username;
    private String password;
    private String host;
    private String mailbox;
    private long pollRate = 30000;

    public String getImapUrl() {
        return String.format("imaps://%s:%s@%s/%s",
            this.username, this.password, this.host, this.mailbox);
    }
}
```

正如您所看到的，EmailProperties 使用 `get()` 方法来产生一个 IMAP URL。流就使用这个 URL 连接到 Taco Cloud 的电子邮件服务器，然后轮询电子邮件。所捕获的属性中包括，用户名、密码、IMAP服务器的主机名、轮询的邮箱和该邮箱被轮询频率（默认为 30 秒轮询一次）。

该 EmailProperties 类是在类的级别使用了 `@ConfigurationProperties` 注解，注解中 `prefix` 被设置为 `tacocloud.email`。这意味着，可以在 `application.yml` 配置文件中详细配置 email 的信息：

```
tacocloud:
  email:
    host: imap.tacocloud.com
    mailbox: INBOX
    username: taco-in-flow
    password: 1L0v3T4c0s
    poll-rate: 10000
```

当然，此处显示的电子邮件服务器配置是由以下部分组成的。您需要把它调整到匹配您将使用的电子邮件服务器详细信息。

此外，您可能在 IDE 中收到“unknown property”警告。那是因为 IDE 是寻找元数据以理解这些属性的含义。警告不会破坏实际代码，可以忽略它们。或者您可以消除它们，将以下依赖项添加到构建中（也可以使用 Spring Initializr 勾选“Spring Configuration Processor”选项）：

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-configuration-processor</artifactId>
  <optional>true</optional>
</dependency>
```

此依赖项包括对自动生成自定义元数据的支持，例如我们用于配置电子邮件服务器详细信息的属性。

现在，让我们使用 `EmailProperties` 来配置集成流。您的目标是什么 `create` 看起来有点像图 10.10。

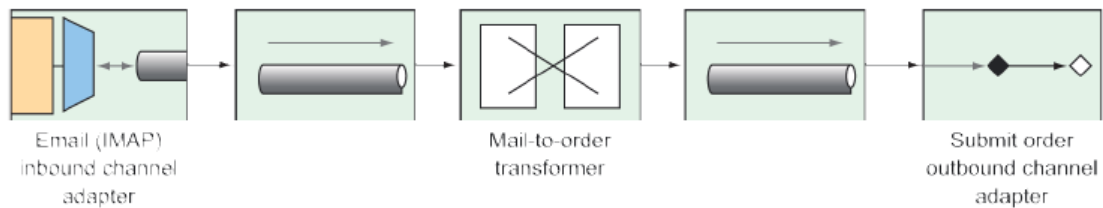


图 10.10 通过电子邮件接受塔可订单的集成流程。

定义这个流程时有两种选择：

- 定义在 *Taco Cloud* 应用程序本身里面 -- 在流结束的位置，服务激活器将调用定义了的存储库来创建 `taco` 订单。
- 定义在一个单独的应用程序中 -- 在流结束的位置，服务激活器将发送 `POST` 请求到 `Taco Cloud API` 来提交 `taco` 订单。

无论选择那种服务激活器的实现，对流本身没有什么影响。但是，因为您需要一些类型来代表的 `taco`、`order` 和 `ingredient`，这些需要与您在 `Taco Cloud` 应用程序中定义的那些有一些不一样。因此在一个单独的应用程序中集成信息流，可以避免与现有的域类型混淆进行。

还可以选择使用 `XML` 配置、`Java` 配置或 `Java DSL` 来定义流。我比较喜欢 `Java DSL` 的优雅，哈哈，这就是您会用到的。随意啦，您如果对一点点额外的挑战有兴趣，可以使用其他配置风格来书写这个流。现在，让我们来看看在 `Java DSL` 配置下的 `taco` 订单电子邮件流。

程序清单 9.5 定义一个集成流接收电子邮件并将它们作为订单提交。

```

package tacos.email;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.integration.dsl.IntegrationFlow;
import org.springframework.integration.dsl.IntegrationFlows;
import org.springframework.integration.dsl.Pollers;
import org.springframework.integration.mail.dsl.Mail;

@Configuration
public class TacoOrderEmailIntegrationConfig {

    @Bean
    public IntegrationFlow tacoOrderEmailFlow(
        EmailProperties emailProps,
        EmailToOrderTransformer emailToOrderTransformer,
        OrderSubmitMessageHandler orderSubmitHandler) {

        return IntegrationFlows
            .from(Mail.imapInboundAdapter(emailProps.getImapUrl()),
                e -> e.poller(
                    Pollers.fixedDelay(emailProps.getPollRate())))
            .transform(emailToOrderTransformer)
            .handle(orderSubmitHandler)
            .get();
    }
}

```

**taco** 订单电子邮件流（在 `tacoOrderEmailFlow()` 方法中的定义）是由三个不同的部分组成：

- **IMAP 电子邮件进站信道适配器**—— 根据 `EmailProperties` 的 `getImapUrl()` 方法返回的 IMP URL 来创建信道适配器，根据 `pollRate` 属性来设定轮询延时。进来的电子邮件被移交到它连接到转换器的通道。
- **一种将电子邮件转换为订单对象的转换器**—— 在 `EmailToOrderTransformer` 中实现的转换器，其被注入到 `tacoOrderEmailFlow()` 方法中。从转换中所产生的订单通过另外一个通道扇出到最终组件中。
- **\*处理程序（作为出站信道适配器）**—— 处理程序接收一个订单对象，并将其提交到 **Taco Cloud** 的 REST API。

可以通过将 **Email** 端点模块作为项目构建的依赖项，就可以对 `Mail.imapInboundAdapter()` 进行调用。**Maven** 的依赖关系如下所示：

```

<dependency>
  <groupId>org.springframework.integration</groupId>
  <artifactId>spring-integration-mail</artifactId>
</dependency>

```

`EmailToOrderTransformer` 这个类，通过继承 `AbstractMailMessageTransformer` 的方式，实现了 **Spring Integration** 中的 `Transformer` 接口，如程序清单 10.6 所示。

程序清单 10.6 使用集成转换器将到来的邮件转换为 **taco** 订单。

```

package tacos.email;

import java.io.IOException;
import java.util.ArrayList;
import java.util.List;
import javax.mail.Message;
import javax.mail.MessagingException;
import javax.mail.internet.InternetAddress;
import org.apache.commons.text.similarity.LevenshteinDistance;
import org.springframework.integration.mail.transformer
    .AbstractMailMessageTransformer;
import org.springframework.integration.support
    .AbstractIntegrationMessageBuilder;
import org.springframework.integration.support.MessageBuilder;
import org.springframework.stereotype.Component;

@Component
public class EmailToOrderTransformer
    extends AbstractMailMessageTransformer<EmailOrder> {

    private static final String SUBJECT_KEYWORDS = "TACO ORDER";

    @Override
    protected AbstractIntegrationMessageBuilder<EmailOrder>
        doTransform(Message mailMessage) throws Exception {
        EmailOrder tacoOrder = processPayload(mailMessage);
        return MessageBuilder.withPayload(tacoOrder);
    }

    private EmailOrder processPayload(Message mailMessage) {
    try {
        String subject = mailMessage.getSubject();
        if (subject.toUpperCase().contains(SUBJECT_KEYWORDS)) {
            String email =
                ((InternetAddress) mailMessage.getFrom()[0]).getAddress();
            String content = mailMessage.getContent().toString();
            return parseEmailToOrder(email, content);
        }
    } catch (MessagingException e) {}
    } catch (IOException e) {}
    return null;
    }

    private EmailOrder parseEmailToOrder(String email, String content) {
        EmailOrder order = new EmailOrder(email);
        String[] lines = content.split("\\r?\\n");
        for (String line : lines) {
            if (line.trim().length() > 0 && line.contains(":")) {
                String[] lineSplit = line.split(":");
                String tacoName = lineSplit[0].trim();
                String ingredients = lineSplit[1].trim();
                String[] ingredientsSplit = ingredients.split(",");
                List<String> ingredientCodes = new ArrayList<>();
                for (String ingredientName : ingredientsSplit) {
                    String code = lookupIngredientCode(ingredientName.trim());
                    if (code != null) {
                        ingredientCodes.add(code);
                    }
                }
                Taco taco = new Taco(tacoName);
                taco.setIngredients(ingredientCodes);
                order.addTaco(taco);
            }
        }
        return order;
    }
}

```

```

    }

    private String lookupIngredientCode(String ingredientName) {
        for (Ingredient ingredient : ALL_INGREDIENTS) {
            String ucIngredientName = ingredientName.toUpperCase();
            if (LevenshteinDistance.getDefaultInstance()
                .apply(ucIngredientName, ingredient.getName()) < 3 ||
                ucIngredientName.contains(ingredient.getName()) ||
                ingredient.getName().contains(ucIngredientName)) {
                return ingredient.getCode();
            }
        }
        return null;
    }
}

private static Ingredient[] ALL_INGREDIENTS = new Ingredient[] {
    new Ingredient("FLTO", "FLOUR TORTILLA"),
    new Ingredient("COTO", "CORN TORTILLA"),
    new Ingredient("GRBF", "GROUND BEEF"),
    new Ingredient("CARN", "CARNITAS"),
    new Ingredient("TMT0", "TOMATOES"),
    new Ingredient("LETC", "LETTUCE"),
    new Ingredient("CHED", "CHEDDAR"),
    new Ingredient("JACK", "MONTERREY JACK"),
    new Ingredient("SLSA", "SALSA"),
    new Ingredient("SRCR", "SOUR CREAM")
};
}

```

**AbstractMailMessageTransformer** 是处理其有效载荷为电子邮件消息的基类，其着重于，从到来的消息中将邮件信息提取到，通过 `doTransform()` 方法传入的 **Message** 对象中。

在 `doTransform()` 方法中，把 **Message** 传递到名 `processPayload()` 的 **private** 方法中，在其中将电子邮件解析为 **Order** 对象。这里的 **Order** 对象与 **Taco Cloud** 主应用程序中的 **Order** 对象虽然有些相似，但是还是不同的，它稍微简单一些：

```

package tacos.email;
import java.util.ArrayList;
import java.util.List;
import lombok.Data;

@Data
public class EmailOrder {
    private final String email;
    private List<Taco> tacos = new ArrayList<>();

    public void addTaco(Taco taco) {
        this.tacos.add(taco);
    }
}

```

与用于在客户的整个交付和账单信息不同，这个 **Order** 类只携带了客户的电子邮件。

将电子邮件解析为 **taco** 订单是一个有意义的事情。事实上，即使是一个简单的实现，都会涉及到几十行代码。而这几十行的代码对 **Spring Integration** 和如何实现转换器的讨论是没有更多的帮助的。因此，为了节省空间，准备放下对 `processPayload()` 方法的详细实现。

该 **EmailToOrderTransformer** 做的最后一件事就是返回包含 **Order** 对象有效载荷的 **MessageBuilder**。由 **MessageBuilder** 产生的消息，被发送到集成信息流的最后一个部分：消息处理器，推送订单到 **Taco Cloud API**。**OrderSubmitMessageHandler**，如下所示，实现了 **Spring Integration** 的 **GenericHandler** 接口，用于处理携带 **Order** 有效载荷的消息。

程序清单 10.7 通过消息处理器传输订单到 Taco Cloud API。

```
package tacos.email;

import org.springframework.integration.handler.GenericHandler;
import org.springframework.messaging.MessageHeaders;
import org.springframework.stereotype.Component;
import org.springframework.web.client.RestTemplate;

@Component
public class OrderSubmitMessageHandler
    implements GenericHandler<EmailOrder> {

    private RestTemplate rest;
    private ApiProperties apiProps;

    public OrderSubmitMessageHandler(ApiProperties apiProps, RestTemplate rest) {
        this.apiProps = apiProps;
        this.rest = rest;
    }
    @Override
    public Object handle(EmailOrder order, MessageHeaders headers) {
        rest.postForObject(apiProps.getUrl(), order, String.class);
        return null;
    }
}
```

为了满足 `GenericHandler` 接口的要求，`OrderSubmitMessageHandler` 重写 `handle()` 方法。这个方法接收传入 `Order` 对象，并使用注入的 `RestTemplate` 通过 POST 请求中注入 `ApiProperties` 对象捕获的 URL 提交订单。最后，`handler()` 方法返回 `null` 以指示流处理结束。

`ApiProperties` 是为了避免在调用 `postForObject()` 方法时对 URL 进行硬编码。这是一个配置属性文件，看起来像这样：

```
@package tacos.email;

import org.springframework.boot.context.properties.ConfigurationProperties;
import org.springframework.stereotype.Component;
import lombok.Data;

@Data
@ConfigurationProperties(prefix = "tacocloud.api")
@Component
public class ApiProperties {
    private String url;
}
```

在 `application.yml`，Taco Cloud API 的 URL 可能会像这样被配置：

```
tacocloud:
  api:
    url: http://localhost:8080/orders/fromEmail
```

为了使 `RestTemplate` 在项目中可用，它被注入到 `OrderSubmitMessageHandler` 中，需要将 Spring Boot web starter 中添加到项目构建中：



```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>
```

这使得 `RestTemplate` 在 `classpath` 中可用，这也触发了 `Spring MVC` 的自动配置。作为一个独立的 `Spring Integration` 流，应用程序并不需要 `Spring MVC`，或是嵌入的 `Tomcat`。因此，您应该在 `application.yml` 中禁用 `Spring MVC` 的自动配置：

```
spring:
  main:
    web-application-type: none
```

`spring.main.web-application-type` 属性可以被设置为 `servlet`、`reactive` 或是 `none`，当 `Spring MVC` 在 `classpath` 中时，自动配置将这个值设置为 `servlet`。但是这里需要将其重写为 `none`，这样 `Spring MVC` 和 `Tomcat` 就不会自动配置了。（我们将在 12 章讨论将其作为一个响应式 `web` 应用程序的意义）。

## 10.4 总结

- **Spring Integration** 允许定义数据在进入或离开应用程序时可以通过的流。
- **Integration** 流可以以 **XML**、**Java** 或 **Java DSL** 配置的风格进行定义。
- 消息网关和通道适配器充当集成信息流的入口和出口。
- 消息可以被转化，分割，聚集，路由和由服务活化器在流动的过程中进行处理。
- 消息通道连接集成流的组件。

## 第 11 章 Reactor 介绍

### 本章内容

- 理解响应式编程
- Reactor 项目
- 响应式的操作数据

您曾经是否订阅过报纸或杂志呢？互联网已经大幅度削减了传统出版物的用户群，但有当订阅的报纸是与当天发生的事件跟上的最佳途径之一的时候，传统媒体还是有一定机会的。您可以每个清晨细数投递来的报纸上的实事，在早餐或在上班路上进行阅读。

现在假设在订阅然后付费后，好几天去了，都没有收到任何报纸。又过了几天，您致电报纸销售处问道，为什么还没有收到您每日的报纸。试想一下，如果他们解释说，“您订阅的是整整一年的报纸，一年的报纸现在尚未全部完成，但是放心，当全年的报纸都准备完成后，您一定会得到所有的报纸。”这样您该会有多吃惊。

值得庆幸的是，不是所有的订阅都是这样工作的。报纸还有一定的时效性。在出版后，它们被尽快的投递出去，这样保证了当报纸被阅读时，内容仍然是最新的。此外，当您正在阅读最新的内容时，本报记者正在写后续的新故事，同时印刷机也在并行的出版下一个版本。

当我们开发应用程序时，有两种风格的代码我们可以写：命令式和响应式：

- *同步式*的代码很像是假想的荒谬的报纸订阅的情况。这是一套串行任务，每次运行一个，完成前一个任务后再完成后一个。数据是批量进行处理的，在前面的任务没有完成批量数据处理前，不能将工作移交到下一个任务。
- *响应式*的代码很像是真正的报纸订阅的情况。定义一组任务去处理数据，但这些任务可并行运行。每个任务处理这些数据的一个子集，当它处理另外一个子集的时候，把处理完成的数据交给下一个任务。

在本章中，我们将暂时离开 Taco Cloud 应用程序，来探索 Reactor 项目。Reactor 是响应式编程的一个库，这个库是 Spring 家族的一部分。而且，因为它是 Spring 对响应式编程支持的基础，在使用 Spring 建立响应式 controller 和 repository 之前，很重要的一步是让您理解 Reactor。在我们开始理解 Reactor 之前，让我们来快速浏览响应式编程的要领。

## 11.1 理解响应式编程

响应式编程是对命令式编程进行替代的一个范例。这种替代的存在是因为响应式编程解决了命令式编程的限制。通过了解这些限制，可以更好地把握响应式模式的好处。

注意：响应式编程不是银弹。不应该从这章或是其他任何对于响应式编程的讨论中，推断出命令式编程是魔鬼而响应式编程是天使。与作为一个开发人员学习的任何东西一样，响应式编程在某些地方很合适，在某些地方完全没有，应该对症下药。

如果和许多开发者一样，都是从命令式编程起步的。很自然地，现在您所写的大多数代码都是命令式的。命令式编程是非常直观的，现在的学生在他们学校的 **STEM** 课程中很轻松地学习它，它很强大，以至于它构成了大部分的代码，驱动着最大的企业。

这个想法很简单：您写的代码就是一行接一行的指令，按照它们的顺序一次一条地出现。一个任务被执行，程序就需要等到它执行完了，才能执行下一个任务。每一步，数据都需要完全获取到了才能被处理，因此它需要作为一个整体来处理。

这样做还行吧...直到它不得行了。当正在执行的任务被阻塞了，特别是它是一个 **IO** 任务，例如将数据写入到数据库或从远程服务器获取数据，那么调用该任务的线程将无法做任何事情，直到任务完成。说穿了，阻塞的线程就是一种浪费。

大多数编程语言，包括 **Java**，支持并发编程。在 **Java** 中启动另一个线程并将其发送到执行某项工作的分支上是相当容易的，而调用线程则继续执行其他工作。尽管很容易创建线程，但这些线程可能最终会阻塞自己。在管理在多线程里面的并发是很具有挑战性的。更多的线程意味着更高的复杂度。

相反，响应式编程是函数式和声明式的。响应式编程涉及描述通过该数据流的 **pipeline** 或 **stream**，而不是描述的一组按顺序执行的步骤。响应式流处理数据时只要数据是可用的就进行处理，而不是需要将数据作为一个整体进行提供。事实上，输入数据可以是无穷的（例如，一个地点的实时温度数据的恒定流）。

注意：如果您是 **Java** 函数式编程新手，您可能想看看由 Pierre-Yves Saumont 《Java 函数式编程》（Manning, 2017），或者 Micha Pachta 的 《Grokking Functional Programming》（Manning, 2021）。

应用于一个真实世界的类比就是，将命令式编程看做一个装水的气球，响应式编程看做花园里面的水管。两者都是在炎热的夏天让您的朋友惊喜并沉浸其中的方式。但是它们的执行风格是不同的：

- 一个水气球一次能携带的它的有效载荷，在撞击的那一刻浸湿了它预定的目标。然而，水球的容量是有限的，如果您想用水泡更多的人（或把同一个人淋得更湿），您唯一的选择就是增加水球的数量。
- 一根花园水龙头带将其有效载荷作为一股水流从水龙头流向喷嘴。花园水龙头接的水带的容量可能是有限的，但在打水仗的过程中水是源源不断的。只要水从水龙头进入软管，就会一直通过软管然后从喷嘴喷出。同一个花园软管是很容易扩展的，您和朋友们可以玩得更尽兴。

命令式编程就类似打水仗中的水球，本质上没有什么问题，但是拿着类似响应式编程的水管的人，在可扩展性和性能方面是有优势的。

### 11.1.1 定义响应式流

Reactive Streams 是 2013 年底由 Netflix、Lightbend 和 Pivotal（Spring 背后的公司）的工程师发起的一项计划。响应式流旨在为无阻塞异步 Backpressure 流处理提供一个标准。

我们已经谈到了响应式编程的异步特性：它使我们能够并行执行任务以获得更大的可伸缩性。

Backpressure（译者注：<https://www.zhihu.com/question/49618581/answer/237078934>）是一种手段，通过对用户愿意处理的数据量设定限制，数据消费者可以避免被生产速度过快的数据淹没。

#### Java Streams 与 Reactive Streams 对比

在 Java 流和响应式流之间有很大的相似性。首先，它们的名字中都含有 Streams。它们也都为处理数据提供函数式接口。事实上，稍后当学到容器的时候，您会看到，其实它们有很多共同操作。

然而，Java 流通常是同步的，同时只能处理有限数据集。它们本质上是使用函数式进行集合迭代的一种手段。

响应式流支持任何大小的数据集，包括无限数据集的异步处理。它们使实时处理数据成为了可能。

响应式流的规范可以通过四个接口定义来概括：Publisher，Subscriber，Subscription 和 Processor。Publisher 为每一个 Subscription 的 Subscriber 生产数据。Publisher 接口声明了一个 subscribe() 方法，通过这个方法 Subscriber 可以订阅 Publisher：

```
public interface Publisher<T> {
    void subscribe(Subscriber<? super T> subscriber);
}
```

Subscriber 一旦进行了订阅，就可以从 Publisher 中接收消息，这些消息都是通过 Subscriber 接口中的方法进行发送：

```
public interface Subscriber<T> {
    void onSubscribe(Subscription sub);
    void onNext(T item);
    void onError(Throwable ex);
    void onComplete();
}
```

Subscriber 通过调用 onSubscribe() 函数将会收到第一个消息。当 Publisher 调用 onSubscribe()，它通过一个 Subscription 对象将消息传输给 Subscriber。消息是通过 Subscription 进行传递的，Subscriber 可以管理他自己的订阅内容：

```
public interface Subscription {
    void request(long n);
    void cancel();
}
```

Subscriber 可以调用 request() 去请求被发送了的数据，或者调用 cancel() 来表明他对接收的数据不感兴趣，并取消订阅。当调用 request() 时，Subscriber 通过传递一个 long 值的参数来表示它将会接收多少数据。这时就会引进 backpressure，用以阻止 Publisher 发送的数据超过 Subscriber 能够处理的数据。在 Publisher 发送了足够的被请求的数据后，Subscriber 可以再次调用 request() 来请求更多的数据。

一旦 **Subscriber** 已经接收到数据，数据就通过流开始流动了。每一个 **Publisher** 发布的项目都会通过调用 `onNext()` 方法将数据传输到 **Subscriber**。如果出现错误，`onError()` 方法将被调用。如果 **Publisher** 没有更多的数据需要发送了，同时也不会再生产任何数据了，将会调用 `onComplete()` 方法来告诉 **Subscriber**，它已经结束了。

对于 **Processor** 接口而言，它连接了 **Subscriber** 和 **Publisher**：

```
public interface Processor<T, R>
    extends Subscriber<T>, Publisher<R> {}
```

作为 **Subscriber**，**Processor** 将会接收数据然后以一定的方式处理这些数据。然后它会摇身一变，变为一个 **Publisher**，将处理的结果发布到 **Subscriber**。

正如您所看到的，响应式流规范相当地简单。关于如何从 **Publisher** 开始建立起一个数据处理的通道，这也是一件很容易的事情了，通过将数据不输入或是输入到多个 **Processor** 中，然后将最终结果传递到 **Subscriber** 中就行了。

然而，响应流接口不适合的是，组成这样一个功能性方式的流。**Reactor** 工程实现了响应式流的规范，它提供由响应式流组成的函数式 **API**。正如您将在后面的章节中看到的，**Reactor** 是 **Spring** 响应式编程模型的基础。在本章的剩余部分，我们将探索 **Reactor** 工程。

## 11.2 使用 Reactor

响应式编程需要我们从与命令式编程完全不同的角度去思考。响应式编程是通过直接建立一个用于数据流通的管道，而不是描述一系列需要进行的步骤。作为数据流通的管道，它可以被改变或以某种方式被使用。

例如，假设您想到利用一个人的名字，把它的所有字母变为大写，然后用它来创建一个问候语，最后将它打印出来。在命令式编程模型中，代码会是这个样子：

```
String name = "Craig";
String capitalName = name.toUpperCase();
String greeting = "Hello, " + capitalName + "!";
System.out.println(greeting);
```

在命令式编程中，每一行为一步，一步接着一部，同时绝对是在同一个线程中。每一步都会阻塞直到完成才能进行下一步动作。

相反，函数式的响应式代码可以以下面这种方式达到目的：

```
Mono.just("Craig")
    .map(n -> n.toUpperCase())
    .map(cn -> "Hello, " + cn + "!")
    .subscribe(System.out::println);
```

不要太担心这个例子中的细节；我们很快将讨论所有关于 `just()`、`map()` 和 `subscribe()` 的操作。就目前而言，要了解的是，虽然响应式的例子似乎仍然遵循一步一步的模式，但是这确实是一个用于数据流的管道。在管道的每个阶段，数据被以某种方式修改了，但是不能知道哪一步操作被哪一个线程执行了的。它们可能在同一个线程也可能不是。

例子中的 `Mono` 是 `Reactor` 的两个核心类型之一，另一个是 `Flux`。两者都是响应式流的 `Publisher` 的实现。`Flux` 表示零个、一个或多个（可能是无限个）数据项的管道。`Mono` 特定用于已知的数据返回项不多于一个的响应式类型。

### Reactor 与 RxJava

如果您已经熟悉 `RxJava` 或 `ReactiveX`，您可能会认为 `Mono` 和 `Flux` 听起来很像 `Observable` 和 `Single`。事实上，它们在语义上近似相等，甚至提供许多相同的操作。

尽管我们在本书中重点讨论 `Reactor`，但可以在 `Reactor` 和 `RxJava` 类型之间进行转换。此外，您将在下面的章节中看到 `Spring` 还可以使用 `RxJava` 的类型。

在前面的例子中实际上有三个 `Mono`。`just()` 操作创建了第一个。当 `Mono` 发出一个值时，该值将被赋予大写操作的 `map()` 并用于创建另一个 `Mono`。当第二个 `Mono` 发布其数据时，被赋予第二个 `map()` 操作来执行一些字符串连接，其结果用于创建第三个 `Mono`。最后，对 `Mono` 的 `subscribe()` 进行调用，接收数据并打印它。

### 11.2.1 图解响应式流

响应式流通常使用弹珠图（Marble Diagram）进行绘制。弹珠图最简单的形式就是，在最上面画出数据流经 Flux 或是 Mono 的时间线，在中间画出操作，在最下面画出 Flux 或是 Mono 结果的时间线。图 11.1 展示了 Flux 的弹珠图模板。正如您所看到的，当数据流通过原始的 Flux 后，它通过一些操作进行处理，通过数据流处理后产生一个新的 Flux。

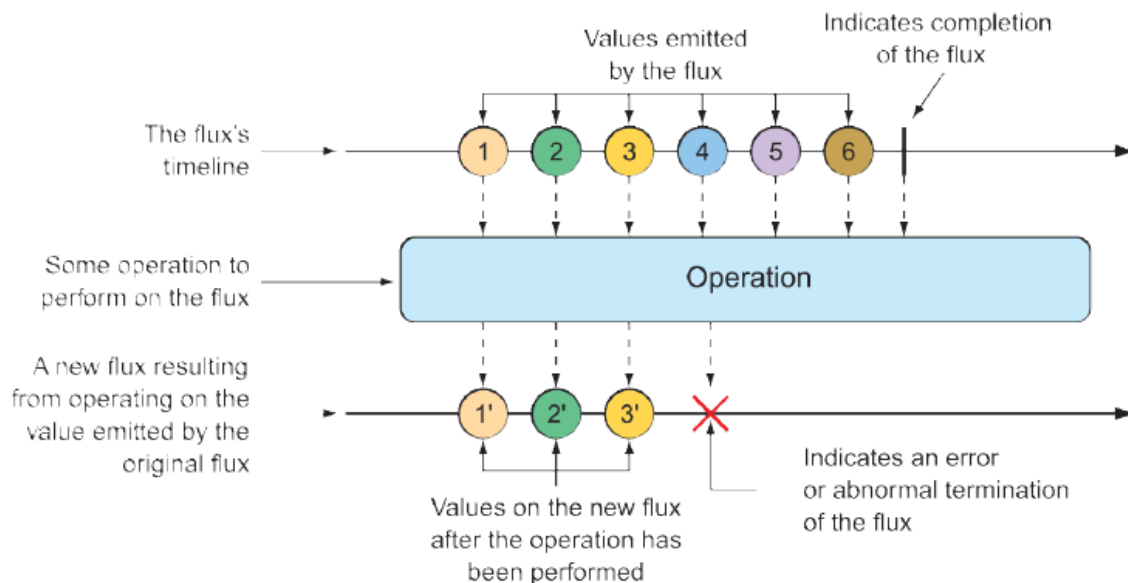


图 11.1 Flux 基本流的可视化弹珠图

图 11.2 展示了一个类似的弹珠图，但是对于 Mono 而言的。正如您所看到的，关键的区别在于 Mono 会有零个或一个数据项，或一个错误。

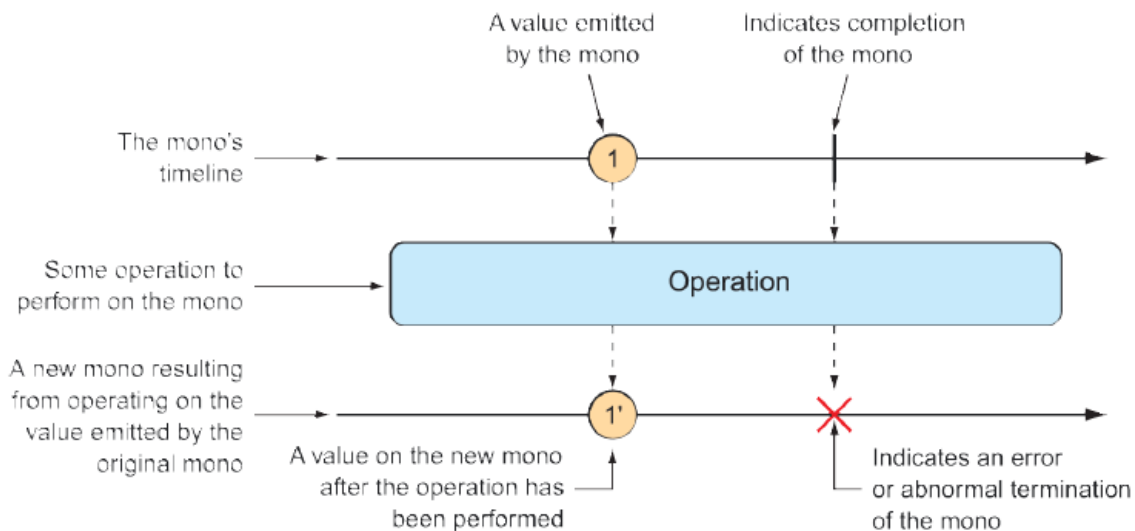


图 11.2 Flux 基本流的可视化弹珠图

在 11.3 节中，我们将探讨 Flux 和 Mono 支持的许多操作，将使用弹珠图来想象它们是如何工作的。



## 11.2.2 添加 Reactor 依赖

让我们开始使用 **Reactor** 吧，把以下依赖添加到项目构建中：

```
<dependency>
  <groupId>io.projectreactor</groupId>
  <artifactId>reactor-core</artifactId>
</dependency>
```

**Reactor** 还提供了测试支持。您会写在您的 **Reactor** 代码周围写很多的测试，所以您肯定会想把这个依赖添加到项目构建中：

```
<dependency>
  <groupId>io.projectreactor</groupId>
  <artifactId>reactor-test</artifactId>
  <scope>test</scope>
</dependency>
```

我假定您要向 **Spring Boot** 项目中添加这些依赖，它可以为您处理的依赖管理，所以没有必要指定依赖的 `<version>` 元素。但是如果您想在非 **Spring Boot** 项目中使用 **Reactor**，那么您需要在构建中设置 **Reactor** 的 BOM（物料清单）。以下依赖项管理条目将 **Reactor** 的“2020.0.4”版本添加到构建中：

```
<dependencyManagement>
  <dependencies>
    <dependency>
      <groupId>io.projectreactor</groupId>
      <artifactId>reactor-bom</artifactId>
      <version>2020.0.4</version>
      <type>pom</type>
      <scope>import</scope>
    </dependency>
  </dependencies>
</dependencyManagement>
```

本章中我们将使用的示例都是独立的，与 **Taco Cloud** 无关。因此，最好创建一个全新的 **Spring** 项目，在构建中使用 **Reactor** 依赖项，并从那里开始工作。

现在，**Reactor** 在您的项目构建中了，可以使用 **Mono** 和 **Flux** 开始创建响应式管道了。对于本章的其余部分，我们将使用 **Mono** 和 **Flux** 提供的一些操作。

## 11.3 通用响应式操作实战

**Flux** 和 **Mono** 是 **Reactor** 提供的最重要的组成部分，而这两个响应式类型所提供的操作就是粘合剂，这些操作将它们结合在一起，来创建数据流动的通道。在 **Flux** 和 **Mono** 之间，存在超过 500 种操作，其中的每一个可以被归类为：

- 创建操作
- 联合操作
- 传输操作
- 逻辑处理操作

把 500 个操作都印在这里来看看它们是如何工作的，这是件有趣的事情，但是在这一章节中没有这么大的空间给它。因此我在这个章节中选择了几个最有用的操作，我们先从创建操作开始。

注意：哪里有 **Mono** 的例子吗？**Mono** 和 **Flux** 有很多相同的操作，所以它没有必要展示两次同样的操作。此外，**Mono** 虽然是有用的，但是对比 **Flux** 的操作来说，**Mono** 看上去还是没有那么有趣。在我们所写的例子中，大多数使用的都是 **Flux**，**Mono** 通常情况下有与 **Flux** 对等的操作。

### 11.3.1 创建响应式类型

当长时间使用 Spring 中的响应式类型时，会从 `repository` 或是 `service` 中得到 `Flux` 或是 `Mono`，因此需要您自己创建一个。但是有时候您需要创建一个新的响应式发布者。

`Reactor` 为创建 `Flux` 和 `Mono` 提供了多个操作。在本节中，我们将介绍一些最有用的创建操作。

#### 从对象进行创建

如果想从 `Flux` 或是 `Mono` 创建一个或多个对象，可以 `Flux` 或 `Mono` 中的静态方法 `just()` 去创建一个响应式类型，其中的数据由这些对象驱动。例如，下面这个测试方法就是使用 5 个 `String` 对象来创建一个 `Flux`：

```
@Test
public void createAFlux_just() {
    Flux<String> fruitFlux = Flux
        .just("Apple", "Orange", "Grape", "Banana", "Strawberry");
}
```

这样就创建了一个 `Flux`，但它没有订阅者。要是没有订阅者，数据不会流动。以花园软管的思路进行类比，您已经把软管接到出水口了，另一端就是从自来水公司流出的水。但是水不会流动，除非您打开水龙头。对响应式类型的订阅就是打开数据流的方式。

要添加一个订阅者，可以调用 `Flux` 中的 `subscribe()` 方法：

```
fruitFlux.subscribe(
    f -> System.out.println("Here's some fruit: " + f);
);
```

`subscribe()` 中的 `lambda` 表达式实际上是 `java.util.Consumer`，用于创建响应式流的 `Subscriber`。由于调用了 `subscribe()` 方法，数据开始流动了。在这个例子中，不存在中间操作，因此数据直接从 `Flux` 流到了 `Subscriber`。

为了在运行过程中观察响应式类型，一个好方法就是将 `Flux` 或 `Mono` 打印到控制台里面。但是，测试 `Flux` 或 `Mono` 更好的方式是使用 `Reactor` 中的 `StepVerifier`。给定一个 `Flux` 或 `Mono`，`StepVerifier` 订阅这个响应式类型，然后对流中流动的数据应用断言，最后验证流以预期方式完成。

例如，为了验证规定的数据流经 `fruitFlux`，可以写一个测试，如下所示：

```
StepVerifier.create(fruitFlux)
    .expectNext("Apple")
    .expectNext("Orange")
    .expectNext("Grape")
    .expectNext("Banana")
    .expectNext("Strawberry")
    .verifyComplete();
```

这个例子中，`StepVerifier` 订阅了 `Flux`，然后对每一个匹配到的期望的水果名字做断言。最后，它验证了 `Strawberry` 是由 `Flux` 生成的，对 `Flux` 的验证完毕。

在本章余下的示例中，将使用 `StepVerifier` 编写测试用例以验证某些行为，并帮助您了解某些操作是如何工作的，从而了解一些 `Reactor` 最有用的操作。

#### 从集合创建

Flux 也可从任何的集合创建，如 `Iterable` 或是 `Java Stream`。图 10.3 使用弹珠图绘制了这是如何运行的：

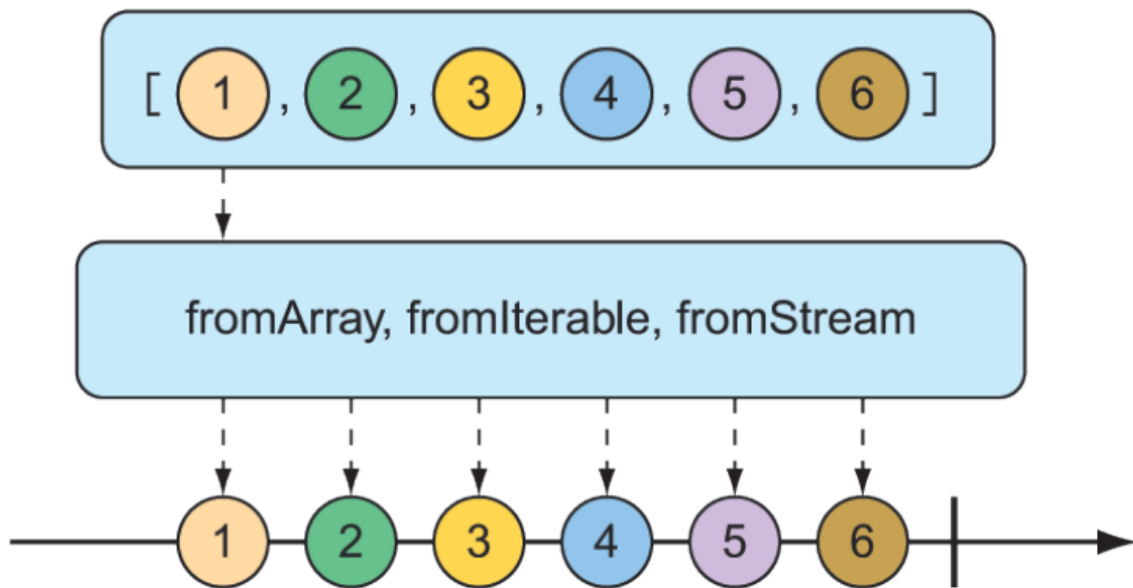


图 11.3 Flux 可以从数组、`Iterable` 或 `Stream` 中创建。

为了从数组创建一个 Flux，调用静态方法 `fromArray()`，然后将数组作为数据源传入：

```
@Test
public void createAFlux_fromArray() {
    String[] fruits = new String[] {
        "Apple", "Orange", "Grape", "Banana", "Strawberry" };

    Flux<String> fruitFlux = Flux.fromArray(fruits);
    StepVerifier.create(fruitFlux)
        .expectNext("Apple")
        .expectNext("Orange")
        .expectNext("Grape")
        .expectNext("Banana")
        .expectNext("Strawberry")
        .verifyComplete();
}
```

因为当您从对象列表中创建 Flux 的时候，源数组包含了您使用到的相同的水果名称，所以被 Flux 所命中的数据有相同的值。这样一来，您就在验证这个 Flux 之前使用相同的 `StepVerifier`。

如果您需要从 `java.util.List`、`java.util.Set` 或任何实现了 `java.lang.Iterable` 接口的类创建 Flux，您可以将它传入静态方法 `fromIterable()` 中：

```

@Test
public void createAFlux_fromIterable() {
    List<String> fruitList = new ArrayList<>();
    fruitList.add("Apple");
    fruitList.add("Orange");
    fruitList.add("Grape");
    fruitList.add("Banana");
    fruitList.add("Strawberry");

    Flux<String> fruitFlux = Flux.fromIterable(fruitList);

    StepVerifier.create(fruitFlux)
        .expectNext("Apple")
        .expectNext("Orange")
        .expectNext("Grape")
        .expectNext("Banana")
        .expectNext("Strawberry")
        .verifyComplete();
}

```

或是，如果您突然想要把您用得顺手的 **Java Stream** 作为 **Flux** 的源，您将会用到 `fromStream()` 方法：

```

@Test
public void createAFlux_fromStream() {
    Stream<String> fruitStream =
        Stream.of("Apple", "Orange", "Grape", "Banana", "Strawberry");

    Flux<String> fruitFlux = Flux.fromStream(fruitStream);

    StepVerifier.create(fruitFlux)
        .expectNext("Apple")
        .expectNext("Orange")
        .expectNext("Grape")
        .expectNext("Banana")
        .expectNext("Strawberry")
        .verifyComplete();
}

```

这里还是一样地使用 **StepVerifier** 去验证需要发布到 **Flux** 的数据。

## 生成 **Flux** 数据

有时您没有任何数据可供使用，只需要使用 **Flux** 作为计数器，发出一个随每个新值递增的数字。要创建计数器 **Flux**，可以使用静态 `range()` 方法。图 11.4 展示了 `range()` 是如何工作的。

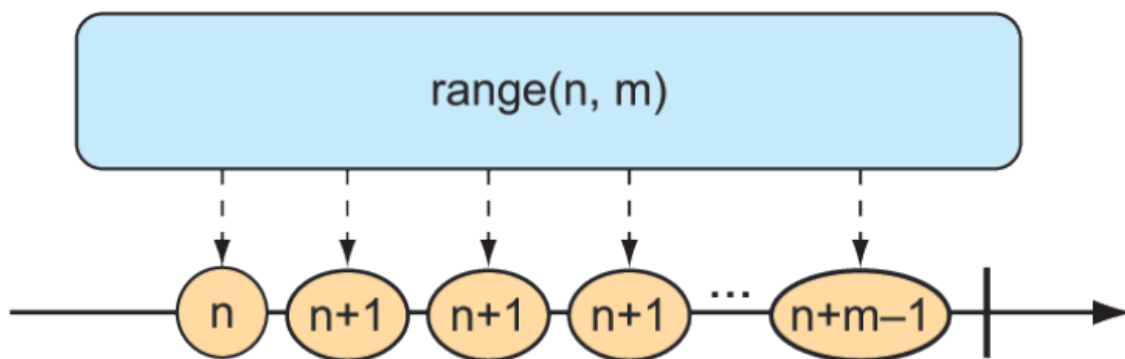


图 11.4 从 **Range** 结果创建 **Flux** 会导致消息的计数器发布。

下面的测试方法展示了如何创建一个范围的 Flux:

```
@Test
public void createAFlux_range() {
    Flux<Integer> intervalFlux =
        Flux.range(1, 5);
    StepVerifier.create(intervalFlux)
        .expectNext(1)
        .expectNext(2)
        .expectNext(3)
        .expectNext(4)
        .expectNext(5)
        .verifyComplete();
}
```

在本例中, 创建的范围 Flux 的起始值为 1, 结束值为 5。StepVerifier 证明它将发布五个项目, 即从 1 到 5 的整数。

另一个类似 range() 的 Flux 创建方法是 interval()。与 range() 方法一样, interval() 创建一个发出递增值的 Flux。但是 interval() 的特殊之处在于, 您不必给它一个起始值和结束值, 而是指定一个持续时间或一个值的发出频率。图 11.5 展示了 interval() 创建方法的弹珠图。

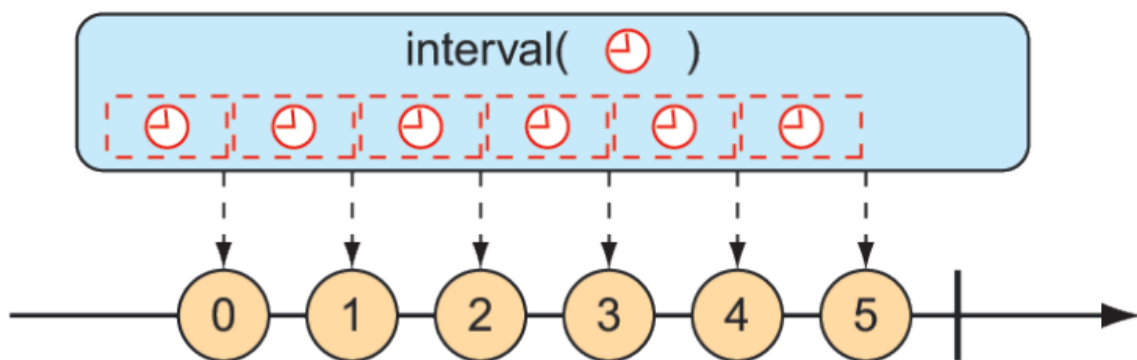


图 11.5 从一个区间创建的 Flux 周期性地发布条目。

例如, 可以使用静态的 interval() 方法来创建每秒发送一个值的 Flux, 如下所示:

```
@Test
public void createAFlux_interval() {
    Flux<Long> intervalFlux =
        Flux.interval(Duration.ofSeconds(1))
            .take(5);
    StepVerifier.create(intervalFlux)
        .expectNext(0L)
        .expectNext(1L)
        .expectNext(2L)
        .expectNext(3L)
        .expectNext(4L)
        .verifyComplete();
}
```

请注意, 间隔 Flux 发出的值以 0 开始, 并在每个连续项上递增。另外, 由于 interval() 没有给定最大值, 因此它可能永远运行。因此, 还可以使用 take() 操作将结果限制为前 5 个条目。我们将在下一节中详细讨论 take() 操作。

### 11.3.2 响应式类型结合

您可能发现您需要将两种响应式类型以某种方式合并到一起。或者，在其他情况下，您可能需要将 Flux 分解成多个响应式类型。在本节中，我们将研究 Reactor 中 Flux 和 Mono 的结合和分解操作。

#### 合并响应式类型

假设您有两个 Flux 流，并需要建立一个汇聚结果的 Flux，它会因为能够得到上流的 Flux 流，所以能够产生数据。为了将一个 Flux 与另一个合并，可以使用 `mergeWith()` 操作，如在图 11.6 展示的弹珠图一样：

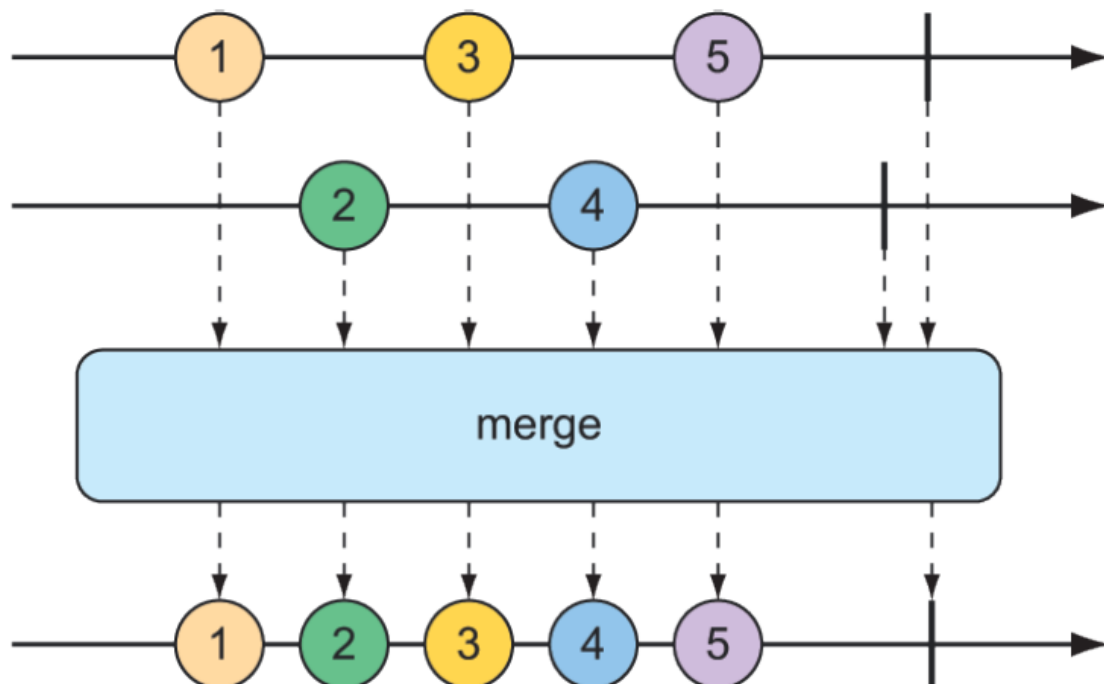


图 11.6 合并两个 Flux 会将它们的消息交织成一个新的 Flux。

例如，假设第一个 Flux 其值是电视和电影人物的名字，第二个 Flux 其值是食品的名称。下面的测试方法将展示如何使用 `mergeWith()` 方法合并两个 Flux 对象：

```

@Test
public void mergeFluxes() {

    Flux<String> characterFlux = Flux
        .just("Garfield", "Kojak", "Barbossa")
        .delayElements(Duration.ofMillis(500));

    Flux<String> foodFlux = Flux
        .just("Lasagna", "Lollipops", "Apples")
        .delaySubscription(Duration.ofMillis(250))
        .delayElements(Duration.ofMillis(500));

    Flux<String> mergedFlux = characterFlux.mergeWith(foodFlux);

    StepVerifier.create(mergedFlux)
        .expectNext("Garfield")
        .expectNext("Lasagna")
        .expectNext("Kojak")
        .expectNext("Lollipops")
        .expectNext("Barbossa")
        .expectNext("Apples")
        .verifyComplete();
}

```

通常情况下，**Flux** 会尽可能快的快地发送数据。因此，需要在创建 **Flux** 的时候使用 `delayElements()` 操作，用来将数据发送速度减慢 —— 每 0.5s 发送一个数据。此外，您将 `delaySubscription()` 操作应用于 **foodFlux**，使得它在延迟 250ms 后才会发送数据，因此 **foodFlux** 将会在 **characterFlux** 之后执行。

合并这两个 **Flux** 对象后，新的合并后的 **Flux** 被创建。当 **StepVerifier** 订阅合并后的 **Flux** 时，它会依次订阅两个 **Flux** 源。

合并后的 **Flux** 发出的数据的顺序，与源发出的数据的时间顺序一致。由于两个 **Flux** 都被设置为固定频率发送数据，因此值会通过合并后的 **Flux** 交替出现 —— **character...food...character...food** 一直这样下去。如何其中任何一个 **Flux** 的发送时间被修改了的话，您可能会看到 2 个 **charater** 跟在 1 个 **food** 后面或是 2 个 **food** 跟在 1 个 **character** 后面的情况。

因为 `mergeWith()` 不能保证源之间的完美交替，所以可能需要考虑使用 `zip()` 操作。当两个 **Flux** 对象压缩在一起时，会产生一个新的 **Flux**，该 **Flux** 生成一个元组，其中元组包含来自每个源 **Flux** 的一个项。图 11.7 说明了如何将两个 **Flux** 对象压缩在一起。



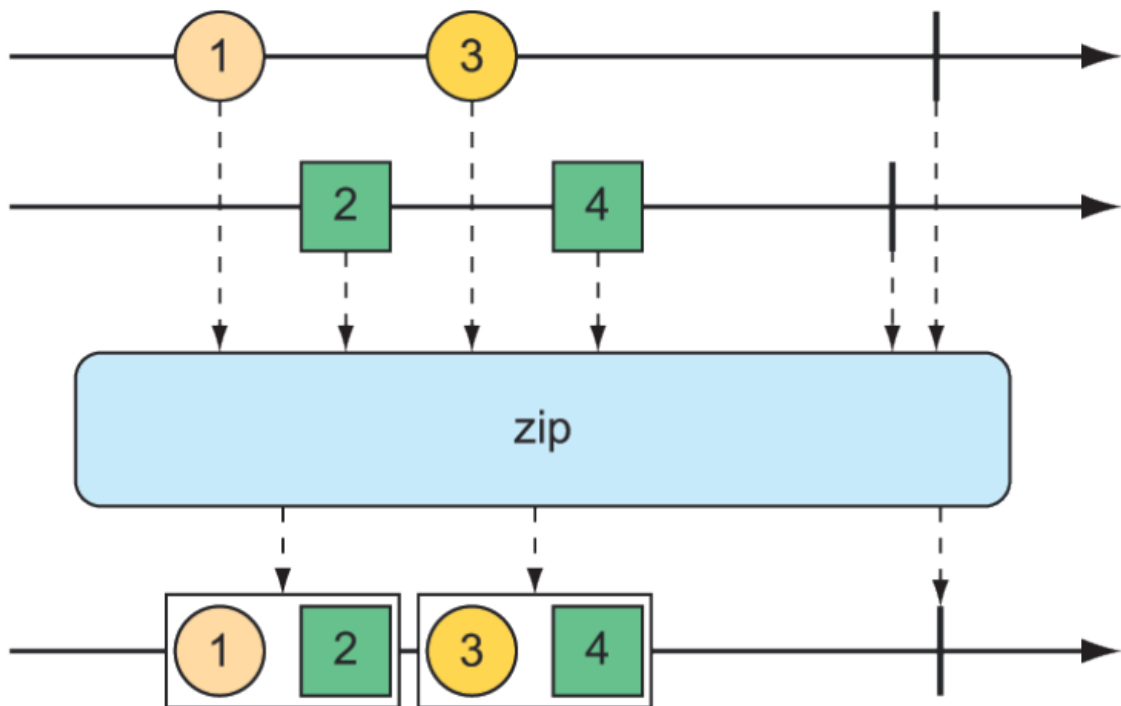


图 11.7 压缩两个 Flux 为一个 Flux。

为了看看 `zip()` 操作的执行情况，参考一下下面的测试方法，它把 `character Flux` 和 `food Flux` 压缩在了一起：

```
@Test
public void zipFluxes() {
    Flux<String> characterFlux = Flux
        .just("Garfield", "Kojak", "Barbossa");
    Flux<String> foodFlux = Flux
        .just("Lasagna", "Lollipops", "Apples");

    Flux<Tuple2<String, String>> zippedFlux =
        Flux.zip(characterFlux, foodFlux);

    StepVerifier.create(zippedFlux)
        .expectNextMatches(p ->
            p.getT1().equals("Garfield") &&
            p.getT2().equals("Lasagna"))
        .expectNextMatches(p ->
            p.getT1().equals("Kojak") &&
            p.getT2().equals("Lollipops"))
        .expectNextMatches(p ->
            p.getT1().equals("Barbossa") &&
            p.getT2().equals("Apples"))
        .verifyComplete();
}
```

注意，与 `mergeWith()` 不同的是，`zip()` 操作是一个静态的创建操作，通过它创建的 Flux 使 `character` 和 `food` 完美对齐。从压缩后的 Flux 发送出来的每个项目都是 `Tuple2`（包含两个对象的容器），其中包含每一个源 Flux 的数据。

如果您不想使用 `Tuple2`，而是想用一些使用其他类型，您可以提供给 `zip()` 您想产生任何对象的 Function 接口。

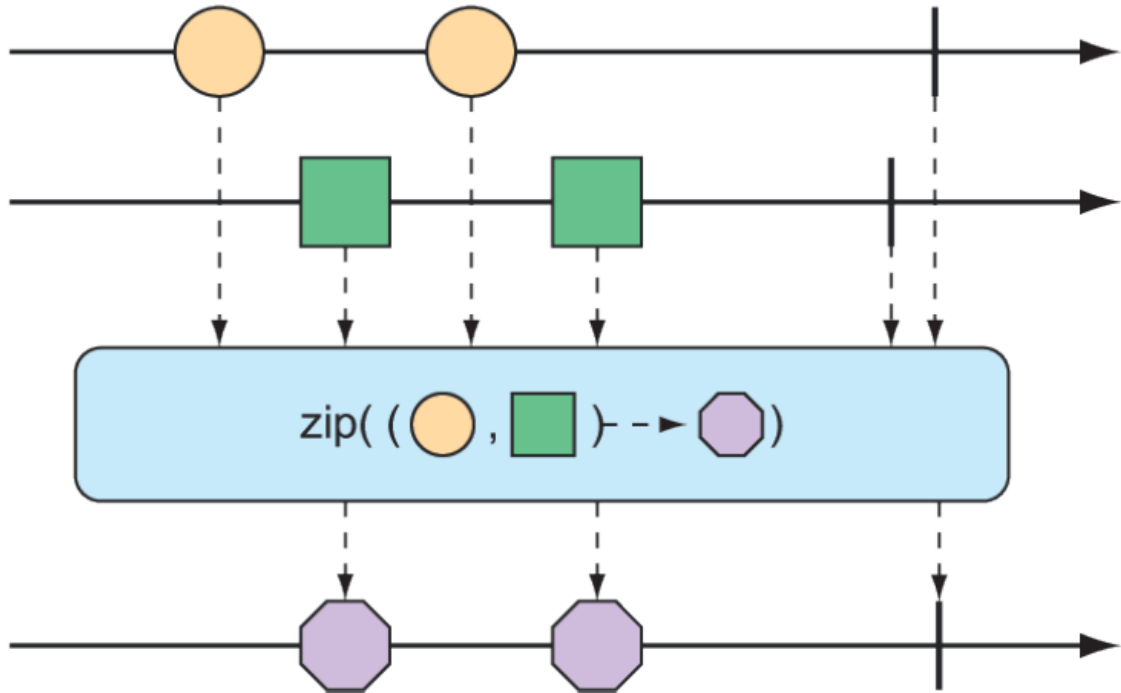


图 11.8 zip 操作的另一种形式导致从每个传入 Flux 的一个元素创建的消息 Flux。

例如，以下的试验方法说明了如何压缩的 character Flux 和 food Flux，使得它产生 String 类型的的 Flux 对象：

```
@Test
public void zipFluxesToObject() {
    Flux<String> characterFlux = Flux
        .just("Garfield", "Kojak", "Barbossa");
    Flux<String> foodFlux = Flux
        .just("Lasagna", "Lollipops", "Apples");

    Flux<String> zippedFlux =
        Flux.zip(characterFlux, foodFlux, (c, f) -> c + " eats " + f);

    StepVerifier.create(zippedFlux)
        .expectNext("Garfield eats Lasagna")
        .expectNext("Kojak eats Lollipops")
        .expectNext("Barbossa eats Apples")
        .verifyComplete();
}
```

给 zip() 的 Function 接口（这里给出一个 lambda 表达式）简单地把两个值连接成一句话，由压缩后的 Flux 进行数据发送。

## 选择第一个响应式类型进行发布

假设您有两个 Flux 对象，您只是想创建一个新的发送从第一个 Flux 产生值的 Flux，而不是将两个 Flux 合并在一起。如图 11.9 所示，first() 操作选择两个 Flux 对象的第一个对象然后输出它的值。

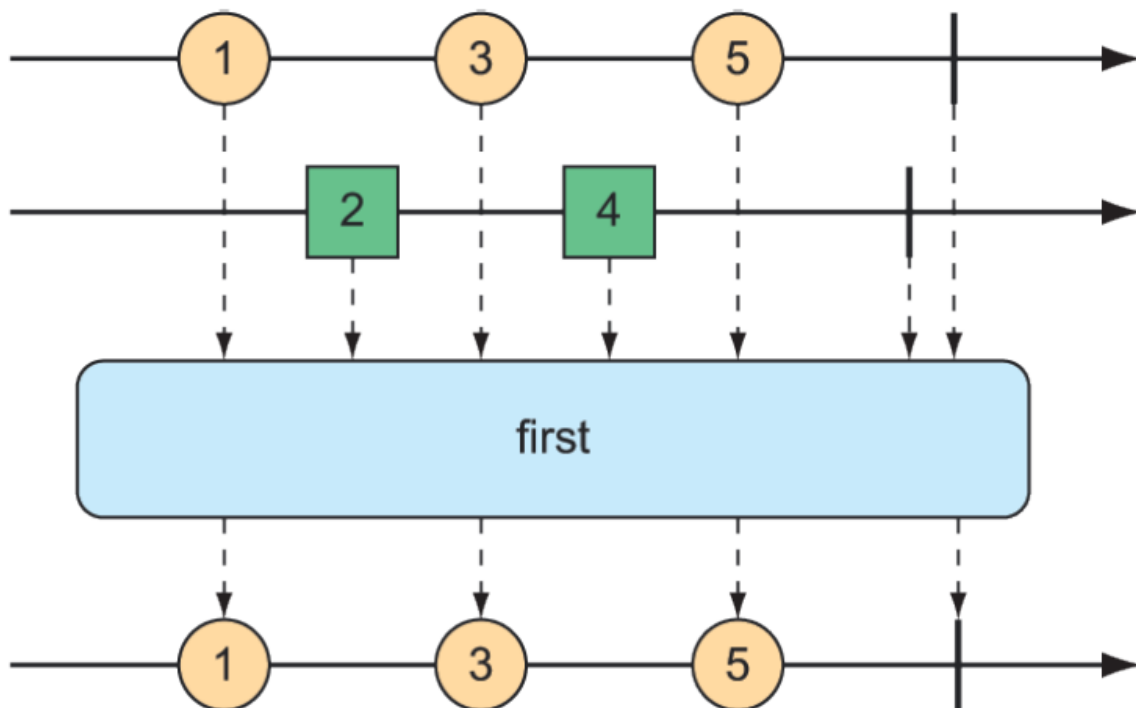


图 11.9 第一个操作选择第一个 Flux 来发送消息，然后仅从该 Flux 生成消息。

下面的测试方法创建一个 fast Flux 和 slow Flux（这里的“slow”的意思是它在订阅之后 100ms 才发布数据）。通过使用 `first()`，它创建了一个新的 Flux，将只会发布从第一个源 Flux 发布的数据：

(译者注：在 reactor 3.4 版本中，`first()` 方法已被废弃，并将在 3.5 版本中被移除，替代方法是 `firstWithSignal()`，选择第一个发出 Signal 的 Flux 对象)

```
@Test
public void firstWithSignalFlux() {

    Flux<String> slowFlux = Flux.just("tortoise", "snail", "sloth")
        .delaySubscription(Duration.ofMillis(100));
    Flux<String> fastFlux = Flux.just("hare", "cheetah", "squirrel");

    Flux<String> firstFlux = Flux.firstWithSignal(slowFlux, fastFlux);

    StepVerifier.create(firstFlux)
        .expectNext("hare")
        .expectNext("cheetah")
        .expectNext("squirrel")
        .verifyComplete();
}
```

在这种情况下，因为在 fast Flux 已经开始发布后 100ms，slow Flux 才开始发布数据，这样导致新创建的 Flux 将完全忽略 slow Flux，而只发布 fast flux 中的数据。

### 11.3.3 转换和过滤响应式流

当数据流过 `stream`，您可能需要过滤或是修改一些值。在本节中，我们将看到的是转换和过滤流过响应式流中的数据。

#### 从响应式类型中过滤数据

当数据从 `Flux` 中流出时，过滤数据的最基本方法之一就是简单地忽略前几个条目。如图 11.10 所示，`skip()` 操作正是这样做的。

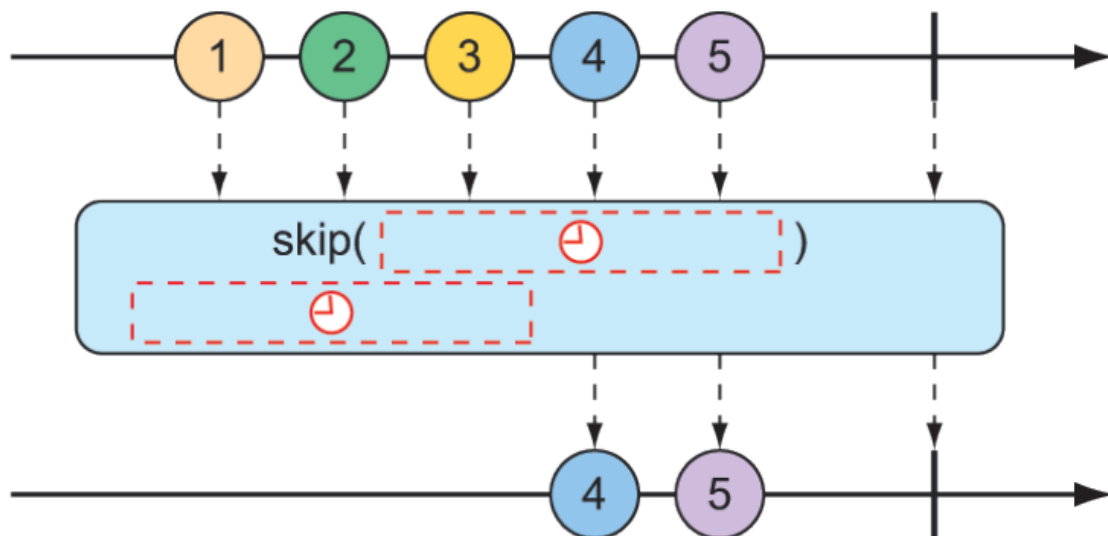


图 11.10 `skip` 操作在将剩余消息传递给结果 `Flux` 之前跳过指定数量的消息。

给定一个包含多个条目的 `Flux`，`skip()` 操作将创建一个新的 `Flux`，该 `Flux` 在从源 `Flux` 发出剩余项之前跳过指定数量的项。下面的测试方法演示如何使用 `skip()`：

```
@Test
public void skipAFew() {
    Flux<String> countFlux = Flux.just(
        "one", "two", "skip a few", "ninety nine", "one hundred")
        .skip(3);

    StepVerifier.create(countFlux)
        .expectNext("ninety nine", "one hundred")
        .verifyComplete();
}
```

在本例中，有五个字符串项的流。对该流调用 `skip(3)` 将生成一个新的流，该流跳过前三个项，并且只发布最后两个项。

您也许不是想跳过特定数量的项目，而是需要过一段时间再跳过前几个项目。`skip()` 操作的另一种形式（如图 10.11 所示）是生成一个流，该流在从源流发出项之前等待一段指定的时间。

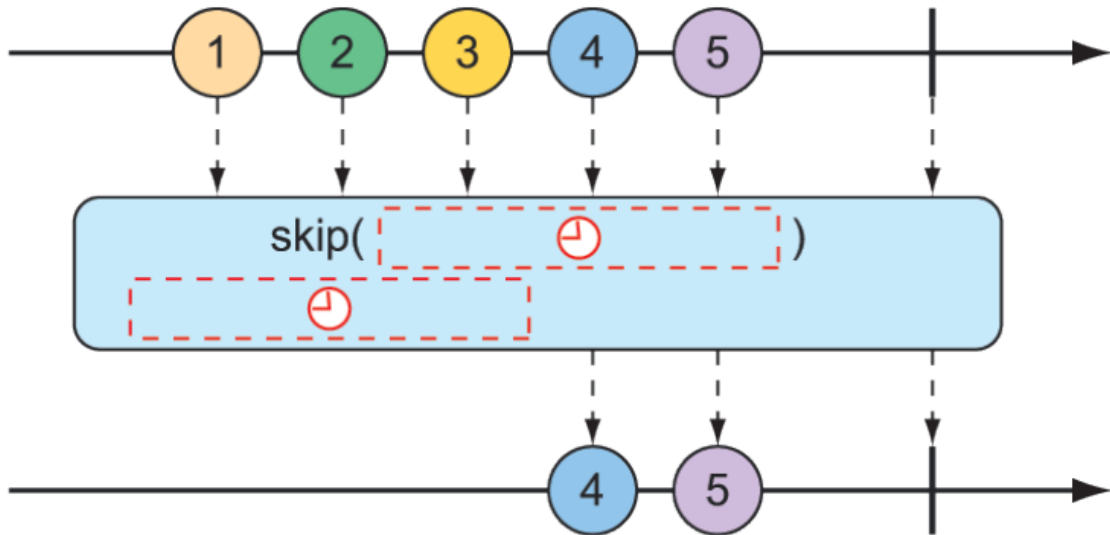


图 11.11 `skip()` 操作的另一种形式是在将消息传递到结果 **Flux** 之前等待一段时间。

下面的测试方法使用 `skip()` 创建一个在发出任何值之前等待 4 秒的 **Flux**。由于该 **Flux** 是从项之间具有 1 秒延迟（使用 `delayElements()`）的 **Flux** 创建的，因此只会发出最后两个项：

```
@Test
public void skipAFewSeconds() {
    Flux<String> countFlux = Flux.just(
        "one", "two", "skip a few", "ninety nine", "one hundred")
        .delayElements(Duration.ofSeconds(1))
        .skip(Duration.ofSeconds(4));

    StepVerifier.create(countFlux)
        .expectNext("ninety nine", "one hundred")
        .verifyComplete();
}
```

您已经看到了 `take()` 操作的一个例子，但是根据 `skip()` 操作，`take()` 可以看作是 `skip()` 的反面。`skip()` 跳过前几个项，`take()` 只发出前几个项（如图 11.12 所示）：

```
@Test
public void take() {
    Flux<String> nationalParkFlux = Flux.just(
        "Yellowstone", "Yosemite", "Grand Canyon", "Zion", "Acadia")
        .take(3);
    StepVerifier.create(nationalParkFlux)
        .expectNext("Yellowstone", "Yosemite", "Grand Canyon")
        .verifyComplete();
}
```

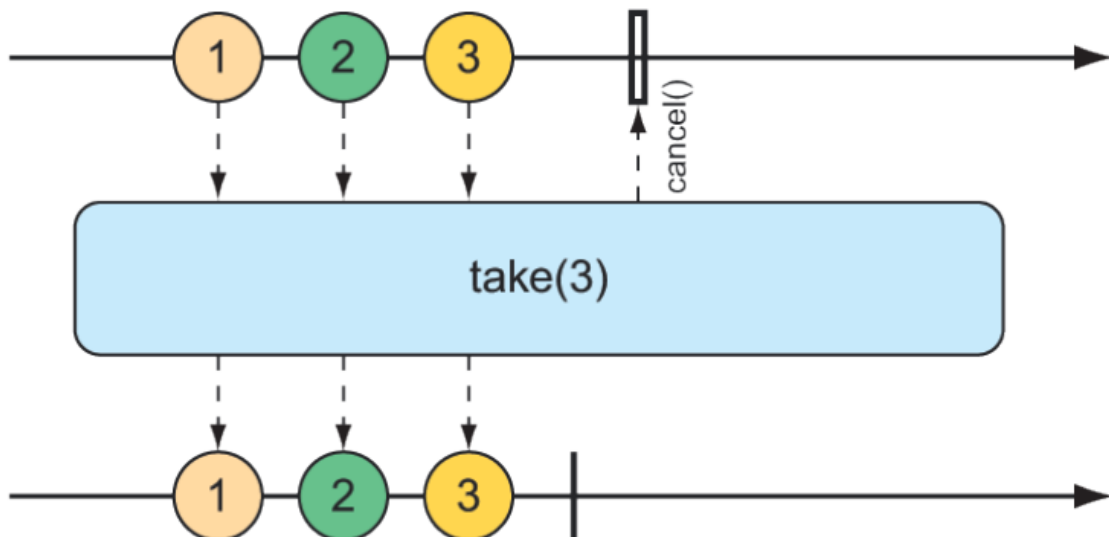


图 11.12 `take` 操作只传递来自传入 `Flux` 的前几个消息，然后取消订阅。

与 `skip()` 一样，`take()` 也有一个基于持续时间而不是项目计数的可选项。它会在一段时间之后，将接收并发出与通过源 `Flux` 一样多的项。如图 11.13 所示：

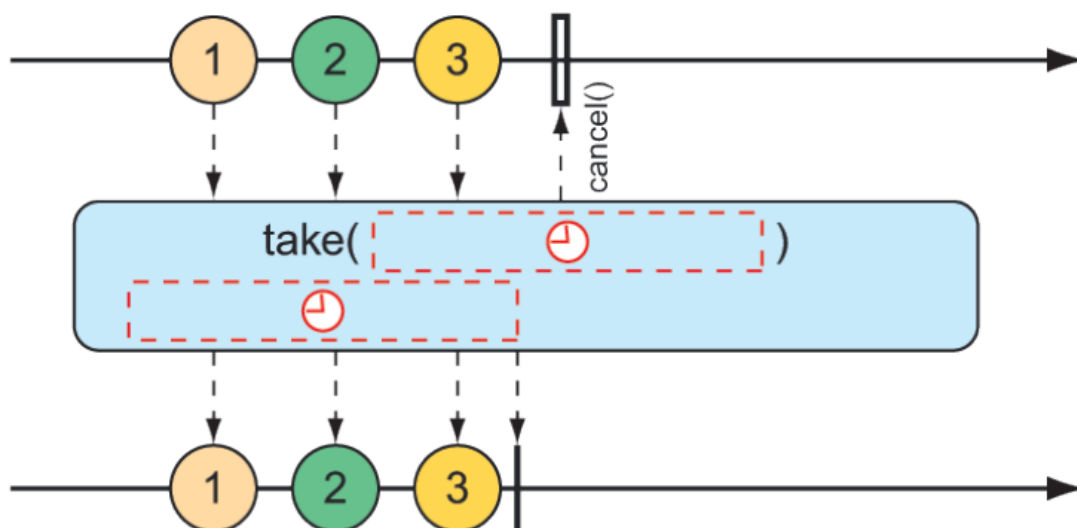


图 11.13 `take` 操作的另一种形式是在某个时间过去后，将消息传递给结果 `Flux`。

以下测试方法使用 `take()` 的替代形式在订阅后的前 3.5 秒内发出尽可能多的项：

```
@Test
public void takeForAwhile() {
    Flux<String> nationalParkFlux = Flux.just(
        "Yellowstone", "Yosemite", "Grand Canyon", "Zion", "Grand Teton")
        .delayElements(Duration.ofSeconds(1))
        .take(Duration.ofMillis(3500));

    StepVerifier.create(nationalParkFlux)
        .expectNext("Yellowstone", "Yosemite", "Grand Canyon")
        .verifyComplete();
}
```

`skip()` 和 `take()` 操作可以看作是基于计数或持续时间的筛选条件的操作。对于更通用的 `Flux` 值过滤，会发现 `filter()` 操作非常有用。

给定一个决定一个项是否通过 Flux 的 Predicate，`filter()` 操作允许您根据需要的任何条件有选择地发布。图 11.14 中的弹珠图显示了 `filter()` 的工作原理。

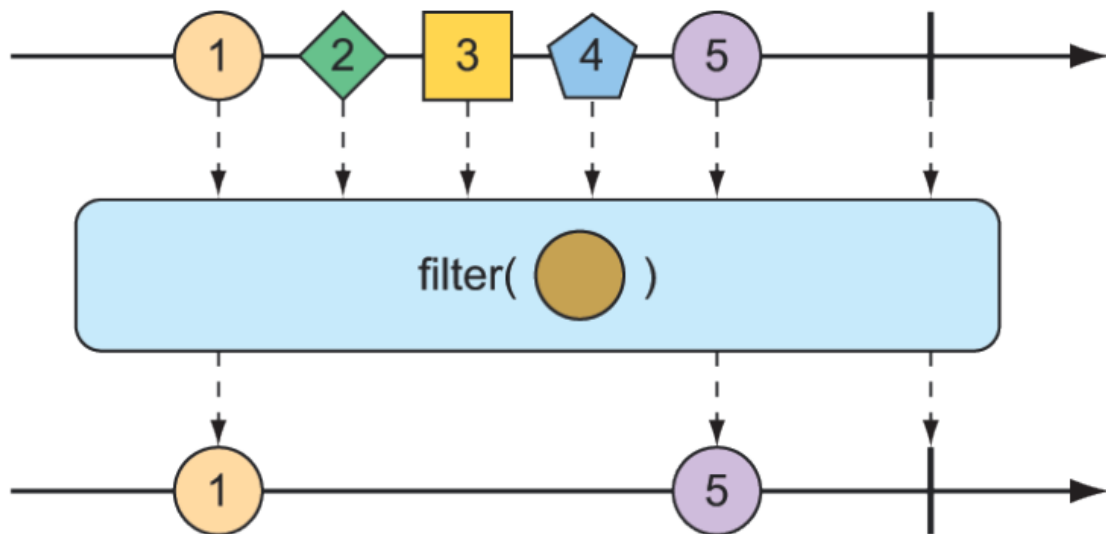


图 11.14 传入的 Flux 可以被过滤，以便生成的 Flux 只接收与给定谓词匹配的消息。

要查看 `filter()` 的运行情况，请考虑以下测试方法：

```
@Test
public void filter() {
    Flux<String> nationalParkFlux = Flux.just(
        "Yellowstone", "Yosemite", "Grand Canyon", "Zion", "Grand Teton")
        .filter(np -> !np.contains(" "));

    StepVerifier.create(nationalParkFlux)
        .expectNext("Yellowstone", "Yosemite", "Zion")
        .verifyComplete();
}
```

这里，`filter()` 被赋予一个 Predicate，它只接受没有空格的 String。因此，“Grand Canyon”和“Grand Teton”被过滤掉。

也许您需要过滤的是您已经收到的任何项目。`distinct()` 操作（如图 11.15 所示）产生一个只发布源 Flux 中尚未发布的项的 Flux。

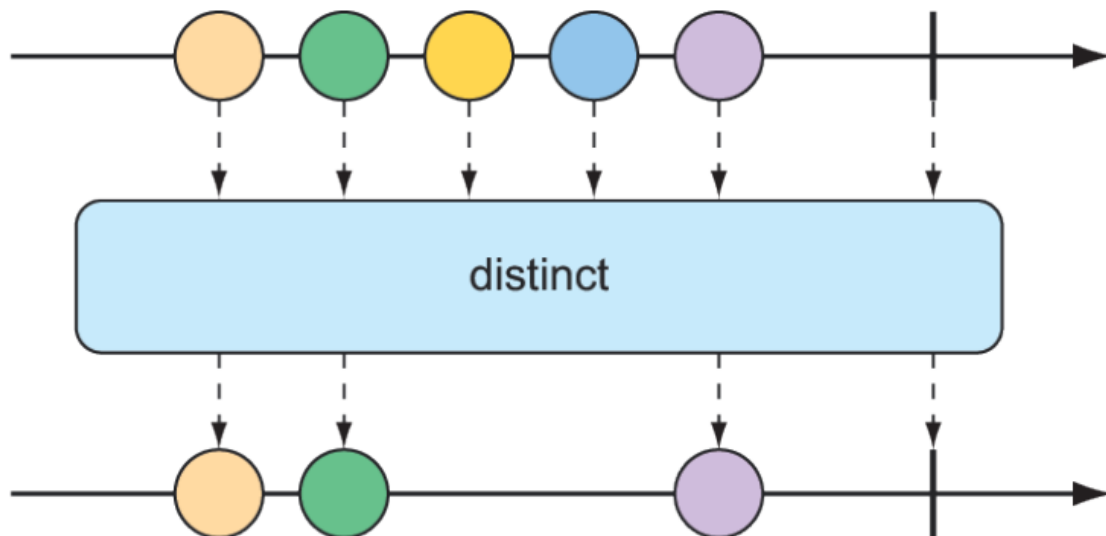


图 11.15 `distinct` 操作过滤掉所有重复的消息。

在下面的测试中，只有唯一的 `String` 值将从不同的 `Flux` 中发出：

```
@Test
public void distinct() {
    Flux<String> animalFlux = Flux.just(
        "dog", "cat", "bird", "dog", "bird", "anteater")
        .distinct();

    StepVerifier.create(animalFlux)
        .expectNext("dog", "cat", "bird", "anteater")
        .verifyComplete();
}
```

尽管“dog”和“bird”分别从源 `Flux` 中发布两次，但在 `distinct Flux` 中只发布一次。

## 映射响应式数据

对于 `Flux` 或 `Mono`，最常用的操作之一是将已发布的项转换为其他形式或类型。`Reactor` 为此提供 `map()` 和 `flatMap()` 操作。

`map()` 操作会创建一个 `Flux`，该 `Flux` 在重新发布之前，按照给定函数对其接收的每个对象执行指定的转换。图 11.16 说明了 `map()` 操作的工作原理。



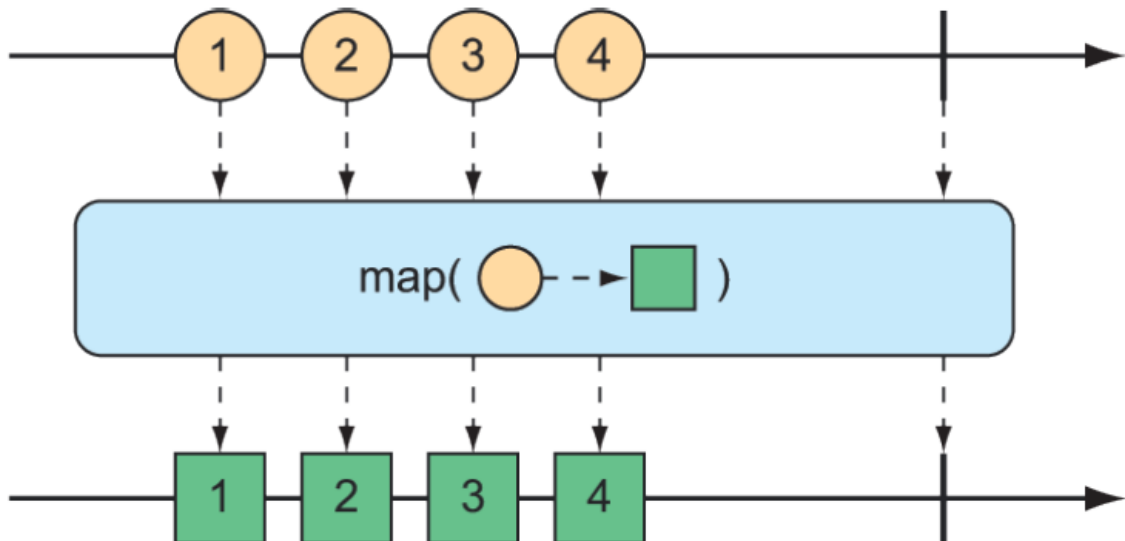


图 11.16 map 操作在结果 Flux 上执行将传入消息转换为新消息。

在以下测试方法中，表示篮球运动员的 String 值的 Flux 映射到 Player 对象的新 Flux：

```
@Test
public void map() {
    Flux<Player> playerFlux = Flux
        .just("Michael Jordan", "Scottie Pippen", "Steve Kerr")
        .map(n -> {
            String[] split = n.split("\\s");
            return new Player(split[0], split[1]);
        });
    StepVerifier.create(playerFlux)
        .expectNext(new Player("Michael", "Jordan"))
        .expectNext(new Player("Scottie", "Pippen"))
        .expectNext(new Player("Steve", "Kerr"))
        .verifyComplete();
}

@Data
private static class Player {
    private final String firstName;
    private final String lastName;
}
```

给 `map()` 的 Function 接口（作为 lambda）将传入 String 以空格进行拆分，并使用生成的字符串数组创建 Player 对象。虽然用 `just()` 创建的流携带的是 String 对象，但是由 `map()` 生成的流携带的是 Player 对象。

关于 `map()` 的重要理解是，映射是同步执行的，因为每个项都是由源 Flux 发布的。如果要异步执行映射，应考虑使用 `flatMap()` 操作。

`flatMap()` 操作需要一些思考和实践才能变得很熟练。如图 11.17 所示，`flatMap()` 不是简单地将一个对象映射到另一个对象，而是将每个对象映射到一个新的 Mono 或 Flux。Mono 或 Flux 的结果被压成一个新的 Flux。当与 `subscribeOn()` 一起使用时，`flatMap()` 可以释放 Reactor 类型的异步能力。

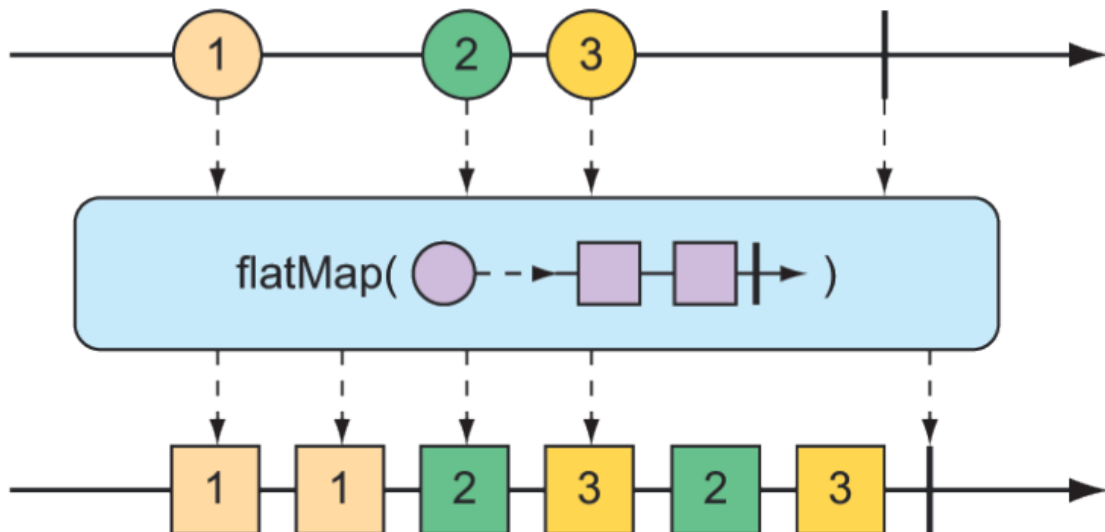


图 11.17 转换映射操作使用中间 **Flux** 来执行转换，从而允许异步转换。

下面的测试方法展示了 `flatMap()` 和 `subscribeOn()` 的用法：

```
@Test
public void flatMap() {
    Flux<Player> playerFlux = Flux
        .just("Michael Jordan", "Scottie Pippen", "Steve Kerr")
        .flatMap(n -> Mono.just(n)
            .map(p -> {
                String[] split = p.split("\\s");
                return new Player(split[0], split[1]);
            })
            .subscribeOn(Schedulers.parallel())
        );

    List<Player> playerList = Arrays.asList(
        new Player("Michael", "Jordan"),
        new Player("Scottie", "Pippen"),
        new Player("Steve", "Kerr"));

    StepVerifier.create(playerFlux)
        .expectNextMatches(p -> playerList.contains(p))
        .expectNextMatches(p -> playerList.contains(p))
        .expectNextMatches(p -> playerList.contains(p))
        .verifyComplete();
}
```

请注意，`flatMap()` 被赋予一个 `lambda` 函数，该函数将传入 `String` 转换为 `String` 类型的 `Mono`。然后对 `Mono` 应用 `map()` 操作，将 `String` 转换为 `Player`。

如果您停在那里，产生的 `Flux` 将携带 `Player` 对象，以与 `map()` 示例相同的顺序同步生成。但是对 `Mono` 做的最后一件事是调用 `subscribeOn()` 来指示每个订阅应该在一个并行线程中进行。因此，可以异步和并行地执行多个传入 `String` 对象的映射操作。

尽管 `subscribeOn()` 的名称与 `subscribe()` 类似，但它们却截然不同。`subscribe()` 是一个动词，它订阅一个响应式流并有效地将其启动，而 `subscribeOn()` 则更具描述性，它指定了应该 *如何* 并发地处理订阅。**Reactor** 不强制任何特定的并发模型；通过 `subscribeOn()` 可以使用 **Schedulers** 程序中的一个静态方法指定要使用的并发模型。在本例中，使用了 `parallel()`，它是使用固定大小线程池的工作线程（大小与 **CPU** 内核的数量一样）。但是调度程序支持多个并发模型，如下表所示：

表 11.1 Schedulers 的并发模型

| Schedulers 方法             | 描述                                                       |
|---------------------------|----------------------------------------------------------|
| <code>.immediate()</code> | 在当前线程中执行订阅                                               |
| <code>.single()</code>    | 在单个可重用线程中执行订阅，对所有调用方重复使用同一线程                             |
| <code>.newSingle()</code> | 在每个调用专用线程中执行订阅                                           |
| <code>.elastic()</code>   | 在从无限弹性池中提取的工作进程中执行订阅，根据需要创建新的工作线程，并释放空闲的工作线程（默认情况下 60 秒） |
| <code>.parallel()</code>  | 在从固定大小的池中提取的工作进程中执行订阅，该池的大小取决于 CPU 核心的数量。                |

使用 `flatMap()` 和 `subscribeOn()` 的好处是，可以通过将工作分成多个并行线程来增加流的吞吐量。但由于这项工作是并行完成的，无法保证先完成哪项工作，因此无法知道产生的 **Flux** 中排放的项目的顺序。因此，**StepVerifier** 只能验证发出的每个项是否存在于 **Player** 对象的预期列表中，并且在 **Flux** 完成之前将有三个这样的项。

## 在响应式流上缓冲数据

在处理流经 **Flux** 的数据的过程中，您可能会发现将数据流分解成比特大小的块是有帮助的。`buffer()` 操作（如图 11.18 所示）可以解决这个问题。

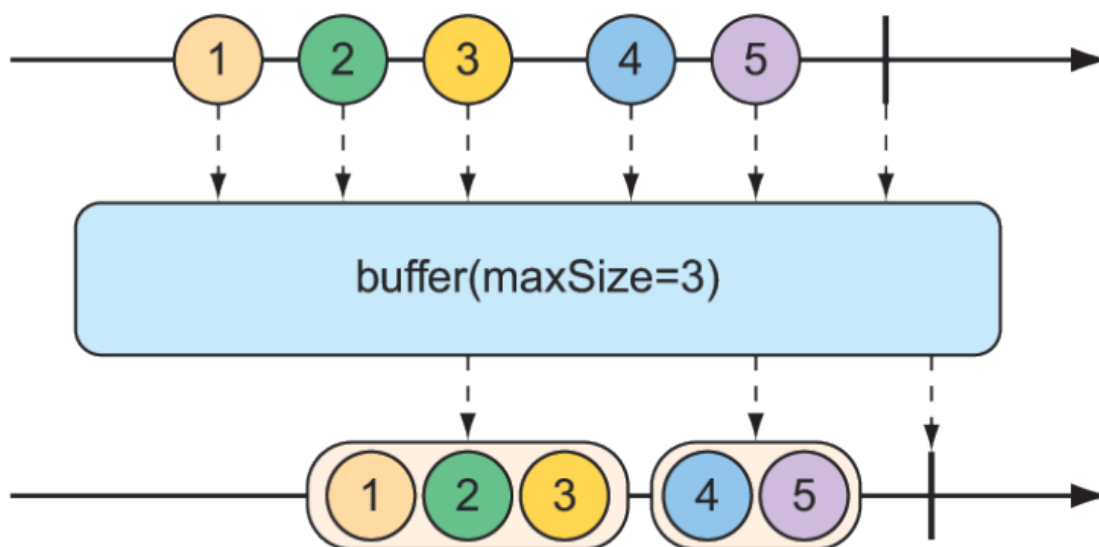


图 11.18 缓冲区操作会产生一个给定最大大小的 **List Flux**，这些 **List** 是从传入的 **Flux** 中收集的。

给定一个 **String** 值的 **Flux**，每个值都包含一个水果的名称，您可以创建一个新的 **List** 集合的 **Flux**，其中每个 **List** 的元素数不超过指定的数目：

```

@Test
public void buffer() {
    Flux<String> fruitFlux = Flux.just(
        "apple", "orange", "banana", "kiwi", "strawberry");

    Flux<List<String>> bufferedFlux = fruitFlux.buffer(3);

    StepVerifier
        .create(bufferedFlux)
        .expectNext(Arrays.asList("apple", "orange", "banana"))
        .expectNext(Arrays.asList("kiwi", "strawberry"))
        .verifyComplete();
}

```

在这种情况下，**String** 元素的 **Flux** 被缓冲到一个 **List** 集合的新 **Flux** 中，每个 **List** 集合包含的项不超过三个。因此，发出 5 个 **String** 的原始 **Flux** 将转换为发出两个 **List** 集合的 **Flux**，一个包含 3 个水果，另一个包含 2 个水果。

那又怎么样？将值从响应式 **Flux** 缓冲到非响应式 **List** 集合似乎适得其反。但是，当将 `buffer()` 与 `flatMap()` 结合使用时，它可以并行处理每个 **List** 集合：

```

@Test
public void bufferAndFlatMap() throws Exception {
    Flux.just(
        "apple", "orange", "banana", "kiwi", "strawberry")
        .buffer(3)
        .flatMap(x ->
            Flux.fromIterable(x)
                .map(y -> y.toUpperCase())
                .subscribeOn(Schedulers.parallel())
                .log()
        ).subscribe();
}

```

在这个新示例中，仍然将 5 个 **String** 值的 **Flux** 缓冲到 **List** 集合的新 **Flux** 中，然后将 `flatMap()` 应用于 **List** 集合的 **Flux**。这将获取每个 **List** 缓冲区并从中元素创建一个新的 **Flux**，然后对其应用 `map()` 操作。因此，每个缓冲 **List** 在单独的线程中进一步并行处理。

为了证明它是有效的，我还包含了一个要应用于每个子 **Flux** 的 `log()` 操作。`log()` 操作只记录所有的 **Reactor Streams** 事件，这样您就可以看到真正发生了什么。因此，以下条目将写入日志（为了简洁起见，删除了时间组件）：

```

[main] INFO reactor.Flux.SubscribeOn.1 - onSubscribe(FluxSubscribeOn.SubscribeOnSubscriber)
[main] INFO reactor.Flux.SubscribeOn.1 - request(32)
[main] INFO reactor.Flux.SubscribeOn.2 - onSubscribe(FluxSubscribeOn.SubscribeOnSubscriber)
[main] INFO reactor.Flux.SubscribeOn.2 - request(32)
[parallel-1] INFO reactor.Flux.SubscribeOn.1 - onNext(APPLE)
[parallel-2] INFO reactor.Flux.SubscribeOn.2 - onNext(KIWI)
[parallel-1] INFO reactor.Flux.SubscribeOn.1 - onNext(ORANGE)
[parallel-2] INFO reactor.Flux.SubscribeOn.2 - onNext(STRAWBERRY)
[parallel-1] INFO reactor.Flux.SubscribeOn.1 - onNext(BANANA)
[parallel-1] INFO reactor.Flux.SubscribeOn.1 - onComplete()
[parallel-2] INFO reactor.Flux.SubscribeOn.2 - onComplete()

```

日志条目清楚地显示，第一个缓冲区（**apple**、**orange** 和 **banana**）中的水果在 **parallel-1** 线程中处理。同时，在第二个缓冲区（**kiwi** 和 **strawberry**）中的水果在 **parallel-2** 线程中进行处理。从每个缓冲区的日志条目交织在一起这一事实可以明显看出，这两个缓冲区是并行处理的。

如果出于某种原因，需要将 Flux 发出的所有内容收集到 List 中，则可以调用不带参数的 `buffer()`：

```
Flux<List<List>> bufferedFlux = fruitFlux.buffer();
```

这将产生一个新的 Flux，该 Flux 会发出一个包含源 Flux 发布的所有项的 List。使用 `collectList()` 操作也可以实现同样的功能，如图 11.19 中的弹珠图所示：

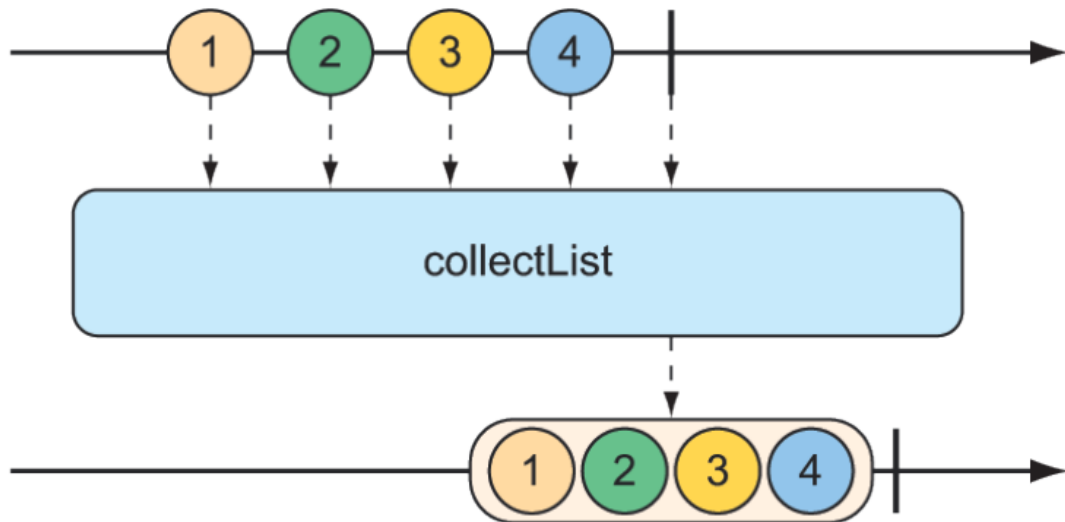


图 11.19 `collect-list` 操作产生一个 **Mono**，其中包含由传入 Flux 发出的所有消息的列表。

`collectList()` 生成一个发布 List 的 Mono，而不是生成一个发布 List 的 Flux。以下测试方法说明了如何使用它：

```
@Test
public void collectList() {
    Flux<String> fruitFlux = Flux.just(
        "apple", "orange", "banana", "kiwi", "strawberry");

    Mono<List<String>> fruitListMono = fruitFlux.collectList();

    StepVerifier
        .create(fruitListMono)
        .expectNext(Arrays.asList(
            "apple", "orange", "banana", "kiwi", "strawberry"))
        .verifyComplete();
}
```

一种更有趣的收集 Flux 发送的项目的方法是把它们存到 Map 中。如图 11.20 所示，`collectMap()` 操作产生一个 Mono，它发布一个 Map，其中填充了由给定 Function 计算其键值的条目。

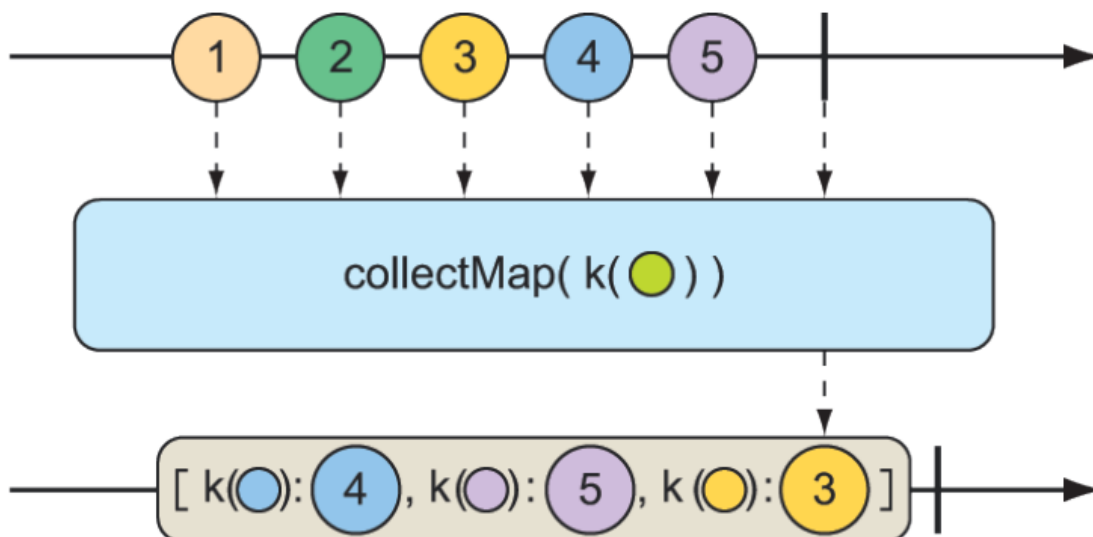


图 11.20 `collect-map` 操作产生一个 **Mono**，其中包含由传入 **Flux** 发出的消息的 **Map**，其中的键来自传入消息的某些特性。

要查看 `collectMap()` 的实际操作，请查看以下测试方法：

```
@Test
public void collectMap() {
    Flux<String> animalFlux = Flux.just(
        "aardvark", "elephant", "koala", "eagle", "kangaroo");

    Mono<Map<Character, String>> animalMapMono =
        animalFlux.collectMap(a -> a.charAt(0));

    StepVerifier
        .create(animalMapMono)
        .expectNextMatches(map -> {
            return
                map.size() == 3 &&
                map.get('a').equals("aardvark") &&
                map.get('e').equals("eagle") &&
                map.get('k').equals("kangaroo");
        })
        .verifyComplete();
}
```

源 **Flux** 发出了一些动物的名字。在该 **Flux** 中，可以使用 `collectMap()` 创建一个新的 **Mono**，该 **Mono** 发送一个 **Map**，其中的键值由动物名称的第一个字母确定，并且该值是动物名称本身。如果两个动物名以同一个字母开头（如 *elephant* 和 *eagle* 或 *koala* 和 *kangaroo*），则流经流的最后一个条目将覆盖所有先前的条目。

### 11.3.4 对响应类型执行逻辑操作

有时您只需要知道 `Mono` 或 `Flux` 发布的条目是否符合某些条件。`all()` 和 `any()` 操作将执行这样的逻辑。图 11.21 和 11.22 说明了 `all()` 和 `any()` 是如何工作的：

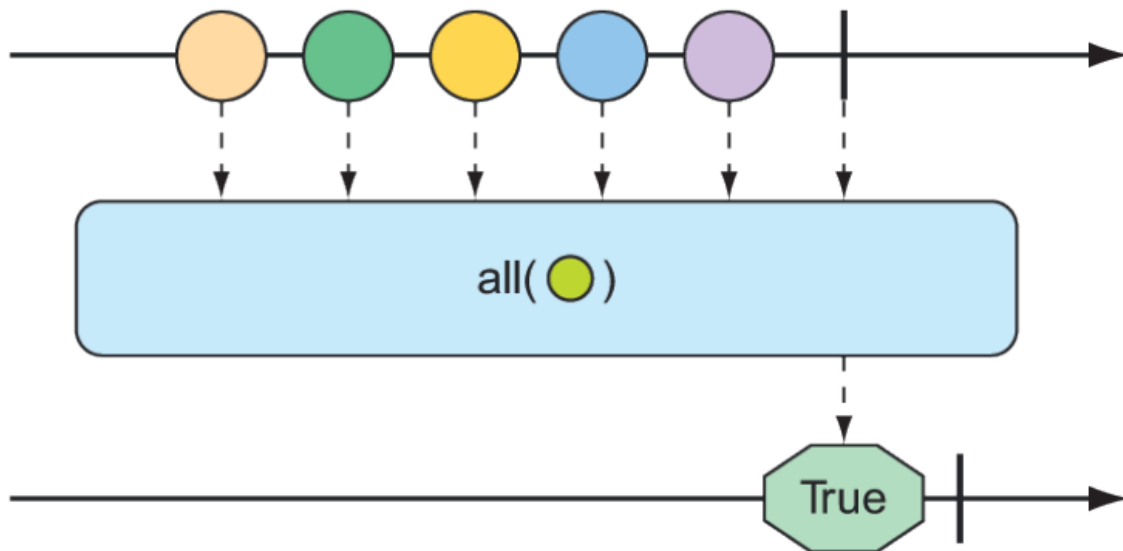


图 11.21 可以对 `Flux` 进行测试以确保所有消息在所有操作中都满足某些条件。

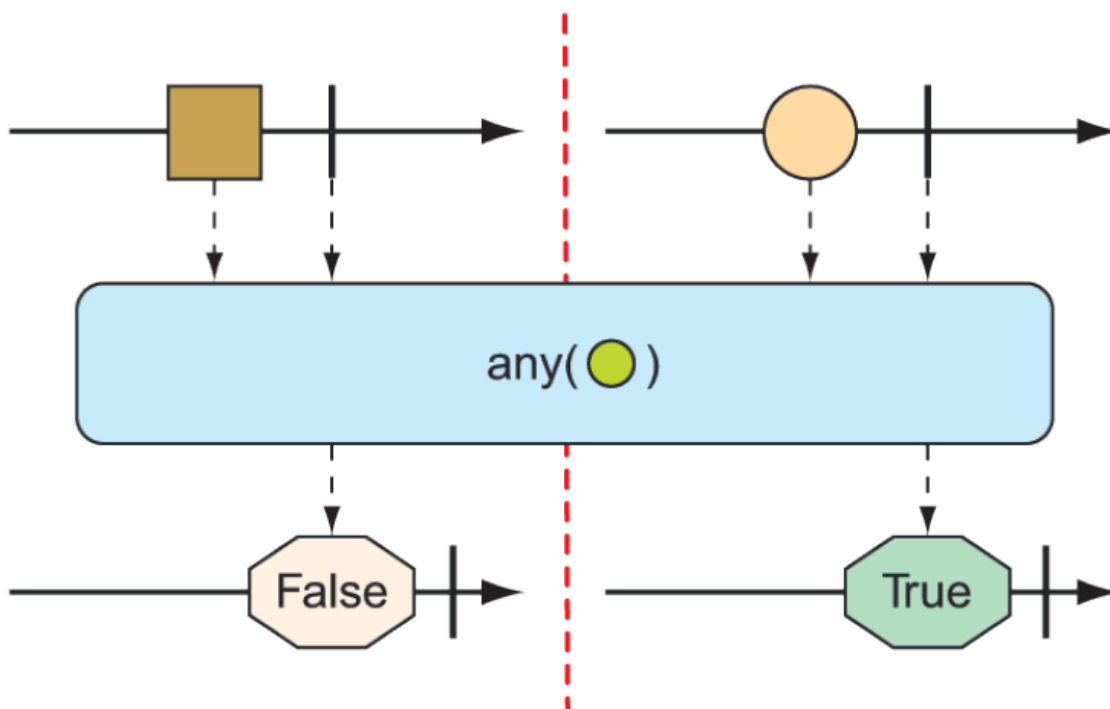


图 11.22 可以对 `Flux` 进行测试以确保在任何操作中至少有一条消息满足某些条件。

假设您想知道由 `Flux` 发布的每个 `String` 都包含字母 `a` 或字母 `k`。下面的测试演示如何使用 `all()` 检查该条件：

```

@Test
public void all() {
    Flux<String> animalFlux = Flux.just(
        "aardvark", "elephant", "koala", "eagle", "kangaroo");

    Mono<Boolean> hasAMono = animalFlux.all(a -> a.contains("a"));
    StepVerifier.create(hasAMono)
        .expectNext(true)
        .verifyComplete();

    Mono<Boolean> hasKMono = animalFlux.all(a -> a.contains("k"));
    StepVerifier.create(hasKMono)
        .expectNext(false)
        .verifyComplete();
}

```

在第一个 `StepVerifier` 中，检查字母 **a**。`all` 操作应用于源 `Flux`，从而生成 `Boolean` 类型的 `Mono`。在本例中，所有的动物名都包含字母 **a**，因此从产生的 `Mono` 发出 `true`。但是在第二个 `StepVerifier` 中，得到的 `Mono` 将发出 `false`，因为并非所有的动物名都包含 **k**。

与其执行全部满足或完全不满足的检查，不如满足至少有一个条目匹配。在这种情况下，`any()` 操作就是您所需要的。这个新的测试用例使用 `any()` 检查字母 **t** 和 **z**：

```

@Test
public void any() {
    Flux<String> animalFlux = Flux.just(
        "aardvark", "elephant", "koala", "eagle", "kangaroo");

    Mono<Boolean> hasAMono = animalFlux.any(a -> a.contains("a"));

    StepVerifier.create(hasAMono)
        .expectNext(true)
        .verifyComplete();

    Mono<Boolean> hasZMono = animalFlux.any(a -> a.contains("z"));
    StepVerifier.create(hasZMono)
        .expectNext(false)
        .verifyComplete();
}

```

在第一个 `StepVerifier` 中，您会看到生成的 `Mono` 发出 `true`，因为至少有一个动物名有字母 **t**（特别是 *elephant*）。在第二个 `StepVerifier` 中，生成的 `Mono` 发出 `false`，因为没有一个动物名包含 **z**。



## 11.4 总结

- 响应式编程包括创建数据流通过的管道。
- **Reactor Stream** 规范定义了四种类型：**Publisher**、**Subscriber**、**Subscription** 和 **Transformer**（**Publisher** 和 **Subscriber** 的组合）。
- **Project Reactor** 实现了 **Reactive Stream**，并将流定义抽象为两种主要类型，**Flux** 和 **Mono**，每种类型都提供数百个操作。
- **Spring** 利用 **Reactor** 创建响应式控制器、**Repository**、**REST** 客户端和其他响应式框架支持。

## 第 12 章 开发响应式 API

本章内容

- 使用 Spring WebFlux
- 编写和测试响应式控制器和客户端
- 消费 REST API
- 保护响应式 web 应用程序安全

既然您已经对响应式编程和 **Project Reactor** 有了很好的了解，就可以开始在 **Spring** 应用程序中应用这些技术了。在本章中，我们将重温您在第 7 章中编写的一些控制器，以利用 **Spring** 的响应式编程模型。

更具体地说，我们将看看 **Spring** 的新的响应式 web 框架 —— **Spring WebFlux**。您将很快发现 **Spring WebFlux** 与 **Spring MVC** 非常相似，同时您已经知道在 **Spring** 中构建 REST API 的知识。有了这些基础，应用起来很容易。

## 12.1 使用 Spring WebFlux

典型的基于 **Servlet** 的 **web** 框架，比如 **Spring MVC**，本质上是阻塞和多线程的，每个连接使用一个线程。在处理请求时，将从线程池中提取一个工作线程来处理该请求。同时，请求线程被阻塞，直到工作线程通知它已完成为止。

因此，在请求量很大的情况下，阻塞 **web** 框架不能有效地扩展。慢工作线程中的延迟使情况更糟，因为工作线程池准备处理另一个请求所需的时间更长。在某些用例中，这种工作方式是完全可以接受的。事实上，这在很大程度上是大多数 **web** 应用程序十多年来的开发方式，但时代在变。

这些 **web** 应用程序伴随着 **HTTP API**，已经从人们偶尔浏览网站成长为人们经常消费内容和使用应用程序。现在，所谓的 **物联网**（其中甚至没有人参与）产生的汽车、喷气发动机以及其他非传统的客户不断地通过 **web API** 交换数据。随着越来越多的客户使用 **web** 应用程序，扩展性比以往任何时候都更加重要。

相比之下，异步 **web** 框架实现用较少的线程达到更高的可扩展性，通常一个 **CPU** 一个线程。通过应用被称为 **event looping** 的技术（如图 12.1 所示），这些框架的每个线程都能够处理许多请求，使得每个连接的成本低

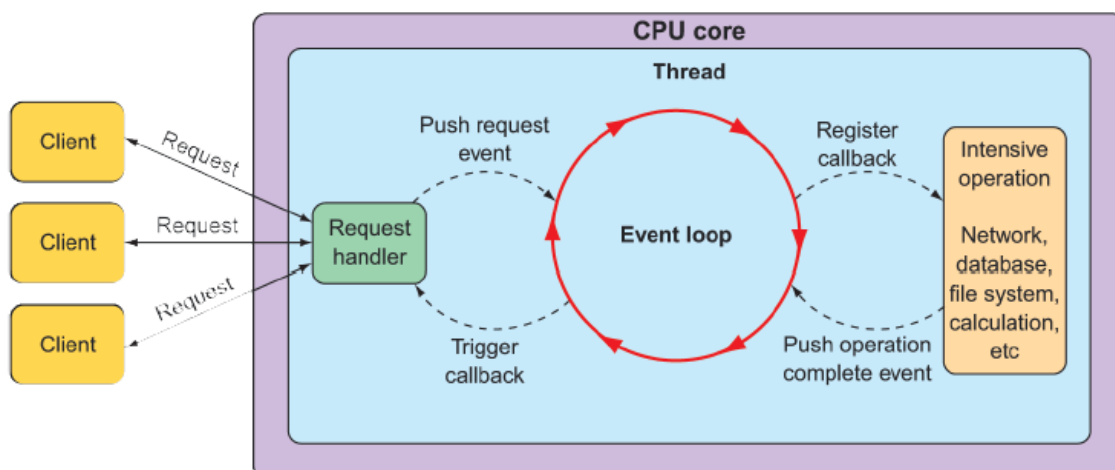


图 12.1 异步 **web** 框架通过应用 **event looping**，使用较少的线程处理更多的请求。

在一个 **event loop** 中，一切皆为事件，其中包括像是数据库和网络操作这种密集操作请求与回调。当需要完成一个重要的操作时，**event loop** 并行地为那个操作注册一个回调，然后它继续去处理其他事件。

当操作完成后，它会被 **event loop** 视为一个 **event**，对于请求也是一样的操作。这样异步 **web** 框架就能够使用更少的线程应对繁重的请求，从而实现更好的扩展性，这样做的结果就是降低了线程管理的开销。

**Spring 5** 已经基于 **Project Reactor** 推出了一个非阻塞异步 **web** 框架，以解决在 **web** 应用程序和 **API** 更大的可扩展性。让我们来看看 **Spring WebFlux**（一个响应式 **web** 框架）。

### 12.1.1 Spring WebFlux 介绍

当 Spring 团队正在考虑如何添加一个响应式编程模型的网络层，很快就发现，如果不在 Spring MVC 做很大的改动，很明显这样做是很困难的。这将涉及到分支代码来决定是否响应式地处理请求。在本质上，其结果将是把两个 web 框架打包成一个，用 if 语句来分离响应式与非响应式。

最终决定创建一个单独的响应式 web 框架，这个框架尽可能的借鉴 Spring MVC，而不是强行把响应式编程模型塞进 Spring MVC 中。Spring WebFlux 就是这个框架了。图 12.2 展示了由 Spring 所定义的完整的 web 开发技术栈。

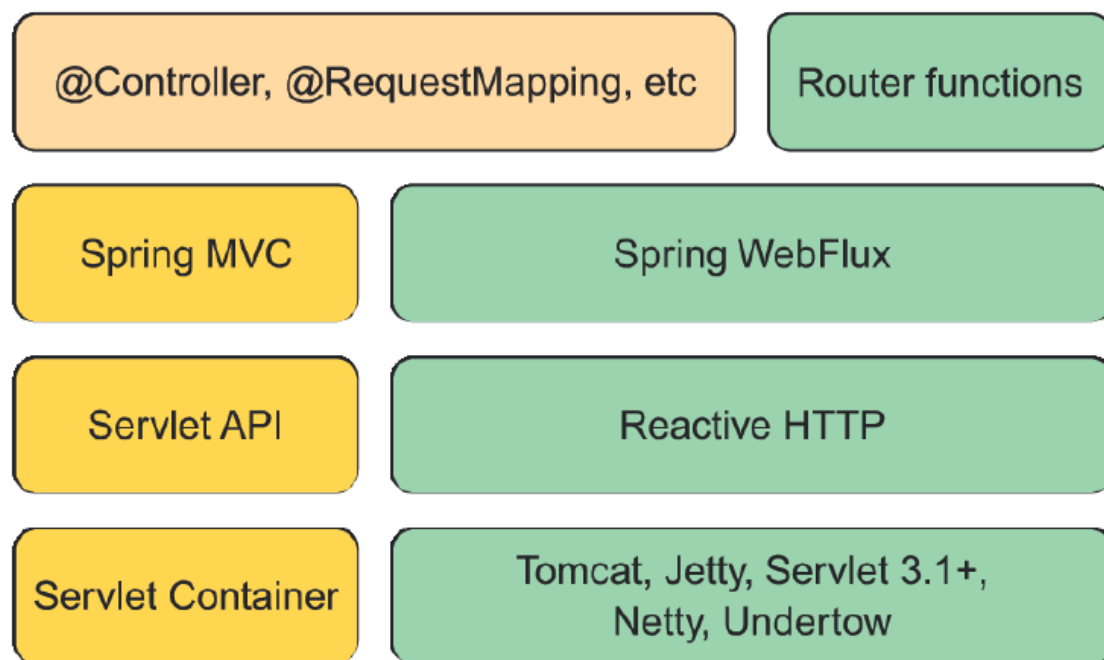


图 12.2 Spring 通过名为 WebFlux 的新 web 框架支持响应式 web 应用程序，WebFlux 是 Spring MVC 的兄弟，它们共享许多核心组件

在图 12.2 的左侧，可以看到 Spring MVC 技术栈，它是在 Spring 框架的 2.5 版中引入的。Spring MVC（在第 2 章和第 6 章中介绍）位于 Java Servlet API 之上，它需要一个 Servlet 容器（比如 Tomcat）来执行。

相比之下，Spring WebFlux（在右侧）与 Servlet API 没有关系，因此它构建在一个响应式 HTTP API 之上，这个方式与使用 Servlet API 提供的相同的响应式功能类似。而且由于 Spring WebFlux 没有耦合到 Servlet API，因此它不需要运行一个 Servlet 容器。相反，它可以在任何非阻塞 web 容器上运行，包括 Netty、Undertow、Tomcat、Jetty 或任何 Servlet 3.1 或更高版本的容器。

图 12.2 最值得注意的是左上角的框，它表示了 Spring MVC 和 Spring WebFlux 之间常见的组件，主要是用于定义 controller 的注解。由于 Spring MVC 和 Spring WebFlux 共享相同的注解，Spring WebFlux 在许多方面与 Spring MVC 没有区别。

右上角的框表示另一种编程模型，该模型使用函数式编程范式而不是使用注解来定义 controller。我们将在第 12.2 节中详细讨论 Spring 的函数式 web 编程模型。

Spring MVC 和 Spring WebFlux 之间最显著的区别就是添加到构建中的依赖项不同。在使用 Spring WebFlux 时，需要添加 Spring Boot WebFlux starter 依赖项，而不是标准的 web starter（例如，spring-boot-starter-web）。在项目的 pom.xml 文件中，如下所示：

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-webflux</artifactId>
</dependency>
```

注意：与大多数 Spring Boot 的 starter 依赖项一样，这个 starter 也可以通过选中 Initializr 中的 Reactive Web 复选框添加到项目中。

使用 WebFlux 而不是 Spring MVC 的一个有趣的副作用是，WebFlux 的默认嵌入式服务器是 Netty 而不是 Tomcat。Netty 是少数几个异步的事件驱动的服务器之一，它自然适合像 Spring WebFlux 这样的响应式 web 框架。

除了使用不同的 starter 依赖项之外，Spring WebFlux controller 方法通常接受并返回响应式类型，比如 Mono 和 Flux，而不是域类型和集合。Spring WebFlux 控制器还可以处理 RxJava 类型，比如 Observable、Single 和 Completable。

## 响应式 Spring MVC?

尽管 Spring WebFlux controller 通常返回 Mono 和 Flux，但这并不意味着 Spring MVC 在处理响应式类型时没有办法。如果您愿意，Spring MVC controller 方法也可以返回 Mono 或 Flux。

不同之处在于如何使用这些类型。Spring WebFlux 是一个真正的响应式 web 框架，允许在 event loop 中处理请求，而 Spring MVC 是基于 Servlet 的，依赖多线程处理多个请求。

让我们通过重写一些 Taco Cloud 的 API controller 来让 Spring WebFlux 工作。

## 12.1.2 编写响应式 Controller

您可能还记得，在第 7 章中，您为 Taco Cloud 的 REST API 创建了一些 controller。这些 controller 具有处理请求的方法，这些方法根据域类型（如 `Order` 和 `Taco`）或域类型的集合，处理输入和输出。提醒一下，请考虑您在第 7 章中写过的 `DesignTacoController` 中的以下片段：

```
@RestController
@RequestMapping(path="/api/tacos",
    produces="application/json")
@CrossOrigin(origins="*")
public class TacoController {

    ...

    @GetMapping(params="recent")
    public Iterable<Taco> recentTacos() {
        PageRequest page = PageRequest.of(
            0, 12, Sort.by("createdAt").descending());
        return tacoRepo.findAll(page).getContent();
    }

    ...

}
```

如前所述，`recentTacos()` controller 处理 `/design/recent` 的 HTTP GET 请求，以返回最近创建的 `tacos` 的列表。更具体地说，它返回一个 `Iterable` 类型的 `Taco`。这主要是因为这是从 `repository` 的 `findAll()` 方法返回的，或者更准确地说，是从 `findAll()` 返回的页面对象的 `getContent()` 方法返回的。

这很好，但是 `Iterable` 不是一个响应式的。您将不能对它应用任何响应式操作，也不能让框架利用它作为响应式类型在多个线程上分割任何工作。您想要的是 `recentTacos()` 返回一个 `Flux<Taco>`。

这里有一个简单但有点有限的选项，就是重写 `recentTacos()` 将 `Iterable` 转换为 `Flux`。而且，当您使用它时，可以去掉分页代码，并用调用 `take()` 来替换它：

```
@GetMapping(params="recent")
public Flux<Taco> recentTacos() {
    return Flux.fromIterable(tacoRepo.findAll()).take(12);
}
```

使用 `Flux.fromIterable()`，可以将 `Iterable<Taco>` 转换为 `Flux<Taco>`。现在您正在使用一个 `Flux`，可以使用 `take()` 操作将返回的 `Flux` 限制为最多 12 个 `Taco` 对象。不仅代码简单，它还处理一个响应式 `Flux`，而不是一个简单的 `Iterable`。

迄今为止，编写响应式代码是一个成功的举措。但是，如果 `repository` 提供了一个可以开始使用的 `Flux`，那就更好了，这样就不需要进行转换。如果是这样的话，那么 `recentTacos()` 可以写成如下：

```
@GetMapping(params="recent")
public Flux<Taco> recentTacos() {
    return tacoRepo.findAll().take(12);
}
```

那就更好了！理想情况下，一个响应式 controller 将是一个端到端的响应式栈的顶端，包括 controller、`repository`、`database` 和任何可能位于两者之间的 `service`。这种端到端的响应式栈如图 12.3 所示：

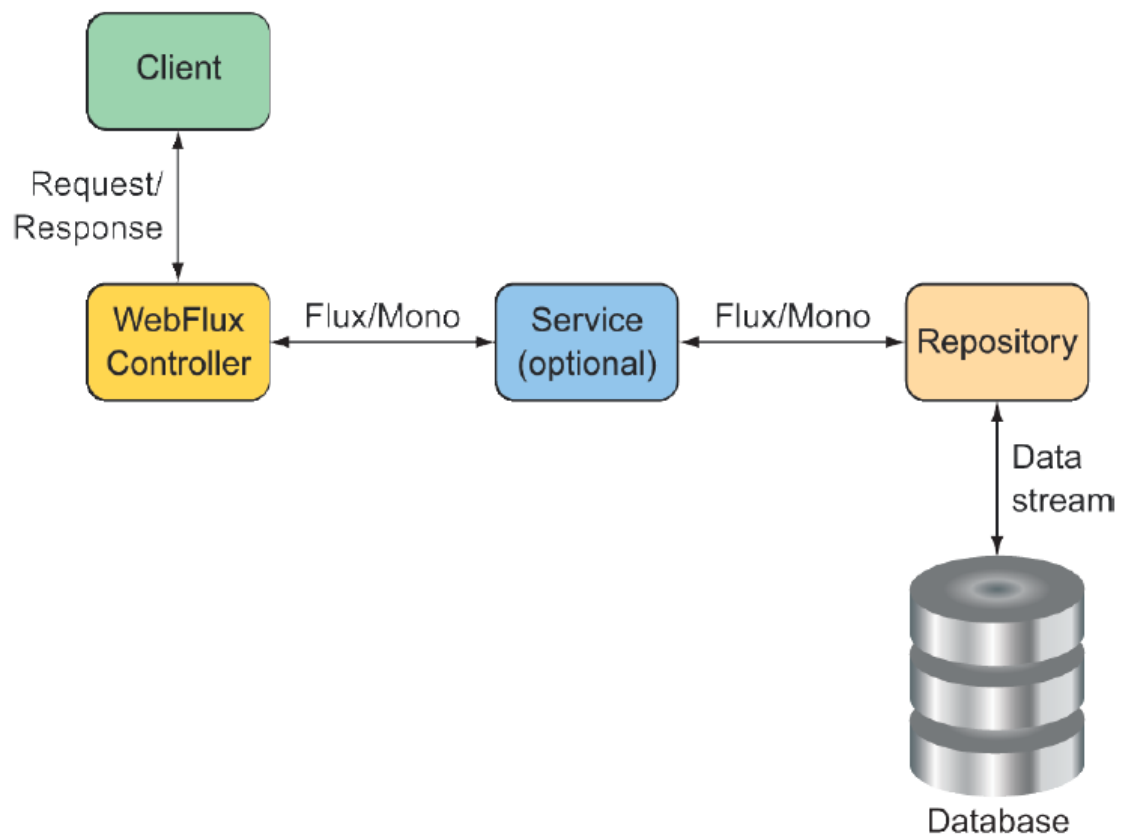


图 12.3 为了最大限度地发挥响应式 **web** 框架的优势，它应该是完整的端到端响应式堆栈的一部分。

这样的端到端的栈要求 **repository** 被写入以返回一个 **Flux**，而不是一个 **Iterable**。在下一章中，我们将探讨如何编写响应式 **repository**，但下面我们将看一看响应式 **TacoRepository** 可能是什么样子：

```
package tacos.data;

import org.springframework.data.repository.reactive.ReactiveCrudRepository;
import tacos.Taco;

public interface TacoRepository
    extends ReactiveCrudRepository<Taco, Long> {
}
```

然而，在这一点上，最重要的是，除了使用 **Flux** 而不是 **Iterable** 以及如何获得 **Flux** 外，定义响应式 **WebFlux controller** 的编程模型与非响应式 **Spring MVC controller** 没有什么不同。两者都用 **@RestController** 和类级别的 **@RequestMapping** 进行了解。它们都有请求处理函数，在方法级别用 **@GetMapping** 进行注解。真正的问题是处理程序方法返回什么类型。

另一个要做的重要观察是，尽管从 **repository** 中获得了一个 `Flux<Taco>`，但您可以在不调用 `subscribe()` 的情况下返回它。实际上，框架将为您调用 `subscribe()`。这意味着当处理对 `/api/tacos?recent` 的请求时，`recentTacos()` 方法将被调用，并在从数据库中获取数据之前返回！

## 返回单个值

作为另一个例子，请考虑 **DesignTacoController** 中的 `tacoById()` 方法，如第 6 章中所述：

```
@GetMapping("/{id}")
public Taco tacoById(@PathVariable("id") Long id) {
    Optional<Taco> optTaco = tacoRepo.findById(id);
    if (optTaco.isPresent()) {
        return optTaco.get();
    }
    return null;
}
```

在这里，这个方法处理 `/design/{id}` 的 GET 请求并返回一个 `Taco` 对象。因为 `repository` 的 `findById()` 返回一个 `Optional`，所以还必须编写一些笨拙的代码来处理这个问题。但是假设 `findById()` 返回 `Mono<Taco>` 而不是 `Optional<Taco>`。在这种情况下，可以重写 `controller` 的 `tacoById()`，如下所示：

```
@GetMapping("/{id}")
public Mono<Taco> tacoById(@PathVariable("id") Long id) {
    return tacoRepo.findById(id);
}
```

哇！这就简单多了。然而，更重要的是，通过返回 `Mono<Taco>` 而不是 `Taco`，可以使 `Spring WebFlux` 以一种被动的方式处理响应。因此，您的API将更好地响应大的负载。

## 使用 RxJava 类型

值得指出的是，虽然在使用 `Spring WebFlux` 时，像 `Flux` 和 `Mono` 这样的 `Reactor` 类型是一个自然的选择，但是您也可以选择使用像 `Observable` 和 `Single` 这样的 `RxJava` 类型。例如，假设 `DesignTacoController` 和后端 `repository` 之间有一个 `service`，它处理 `RxJava` 类型。在这种情况下，`recentTacos()` 方法的编写方式如下：

```
@GetMapping(params = "recent")
public Observable<Taco> recentTacos() {
    return tacoService.getRecentTacos();
}
```

类似地，可以编写 `tacoById()` 方法来处理 `RxJava` 的 `Single` 元素，而不是 `Mono`：

```
@GetMapping("/{id}")
public Single<Taco> tacoById(@PathVariable("id") Long id) {
    return tacoService.lookupTaco(id);
}
```

此外，`Spring WebFlux controller` 方法还可以返回 `RxJava` 的 `Completable`，这相当于 `Reactor` 中的 `Mono<Void>`。`WebFlux` 还可以返回一个 `Flowable`，作为 `Observable` 或 `Reactor` 的 `Flux` 的替代。

## 响应式地处理输入

到目前为止，我们只关心控制器方法返回的响应式类型。但是使用 `Spring WebFlux`，您还可以接受 `Mono` 或 `Flux` 作为处理程序方法的输入。请考虑 `DesignTacoController` 中 `postTaco()` 的原始实现：

```
@PostMapping(consumes="application/json")
@ResponseStatus(HttpStatus.CREATED)
public Taco postTaco(@RequestBody Taco taco) {
    return tacoRepo.save(taco);
}
```



正如最初编写的，`postTaco()` 不仅返回一个简单的 **Taco** 对象，而且还接受一个绑定到请求主体内容的 **Taco** 对象。这意味着在请求有效负载完全解析并用于实例化 **Taco** 对象之前，无法调用 `postTaco()`。这也意味着 `postTaco()` 在对 **repository** 的 `save()` 方法的阻塞调用，在返回之前无法返回。简言之，请求被阻塞了两次：当它进入 `postTaco()` 时，然后在 `postTaco()` 内部被再次阻塞。但是，通过对 `postTaco()` 应用一点响应式编码，可以使其成为一种完全无阻塞的请求处理方法：

```
@PostMapping(consumes = "application/json")
@ResponseStatus(HttpStatus.CREATED)
public Mono<Taco> postTaco(@RequestBody Mono<Taco> tacoMono) {
    return tacoRepo.saveAll(tacoMono).next();
}
```

在这里，`postTaco()` 接受 `Mono<Taco>` 并调用 **repository** 的 `saveAll()` 方法，正如您将在下一章中看到的，该方法接受 **Reactive Streams Publisher** 的任何实现，包括 **Mono** 或 **Flux**。`saveAll()` 方法返回一个 `Flux<Taco>`，但是因为是从 **Mono** 开始的，所以 **Flux** 最多会发布一个 **Taco**。因此，您可以调用 `next()` 来获取将从 `postTaco()` 返回的 `Mono<Taco>`。

通过接受 `Mono<Taco>` 作为输入，可以立即调用该方法，而无需等待 **Taco** 从请求体被解析。由于 **repository** 也是被动的，它将接受一个 **Mono** 并立即返回一个 `Flux<Taco>`，从中调用 `next()` 并返回 `Mono<Taco>`。所有这些都是在处理请求之前完成的！

或者，您也可以像这样实现 `postTaco()`：

```
@PostMapping(consumes = "application/json")
@ResponseStatus(HttpStatus.CREATED)
public Mono<Taco> postTaco(@RequestBody Mono<Taco> tacoMono) {
    return tacoMono.flatMap(tacoRepo::save);
}
```

这种方法将事情翻转过来，使 `tacoMono` 成为行为的驱动者。`tacoMono` 中包含的 **Taco** 通过 `flatMap()` 传递给 **repository** 的 `save()` 方法，导致返回一个新的 `Mono<Taco>`。

任何一种方法都有效，`postTaco()` 可能还有其他几种写法，选择对您最容易理解的方法就好。

**Spring WebFlux** 是 **Spring MVC** 的一个极好的替代品，它提供了使用与 **Spring MVC** 相同的开发模型编写响应式 **web** 应用程序的选项。不过，**Spring** 还有另一个新的窍门。让我们看看如何使用 **Spring** 的新函数式编程风格创建响应式 **API**。

## 12.2 定义函数式请求处理程序

Spring MVC 基于注解的编程模型从 Spring 2.5 开始就出现了，并且广受欢迎。不过，它也有一些缺点。

首先，任何基于注解的编程都涉及到注解应该对做什么以及如何做定义上区分。注解本身定义了什么；如何在框架代码的其他地方定义。当涉及到任何类型的定制或扩展时，这会使编程模型复杂化，因为这样的更改需要在注解外部的代码中工作。此外，调试这样的代码是很棘手的，因为不能在注解上设置断点。

另外，随着 Spring 的不断流行，来自其他语言和框架的新开发人员可能会发现基于注解的 Spring MVC（和 WebFlux）与他们已经知道的非常不同了。作为 WebFlux 的替代，Spring 引入了一个新的函数式编程模型来定义响应式 API。

这个新的编程模型更像是一个库，而不是一个框架，允许您将请求映射到不带注解的处理代码。使用 Spring 的函数式编程模型编写 API 涉及四种主要类型：

- **RequestPredicate** —— 声明将会被处理的请求类型
- **RouteFunction** —— 声明一个匹配的请求应该如何被路由到处理代码中
- **ServerRequest** —— 表示 HTTP 请求，包括对头和正文信息的访问
- **ServerResponse** —— 表示 HTTP 响应，包括头和正文信息

作为将所有这些类型组合在一起的简单示例，请考虑以下Hello World示例：

```
package hello;

import static org.springframework.web
    .reactive.function.server.RequestPredicates.GET;
import static org.springframework.web
    .reactive.function.server.RouterFunctions.route;
import static org.springframework.web
    .reactive.function.server.ServerResponse.ok;
import static reactor.core.publisher.Mono.just;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.web.reactive.function.server.RouterFunction;

@Configuration
public class RouterFunctionConfig {

    @Bean
    public RouterFunction<?> helloRouterFunction() {
        return route(GET("/hello"),
            request -> ok().body(just("Hello World!"), String.class))
        ;
    }

}
```

首先要注意的是，已经选择静态地导入几个 **helper** 类，可以使用这些类来创建前面提到的函数类型。还静态导入了 **Mono**，以使其余代码更易于阅读和理解。

在这个 `@Configuration` 类中，有一个类型为 `RouterFunction<?>`。如前所述，**RouterFunction** 声明一个或多个 **RequestPredicate** 对象与将处理匹配请求的函数之间的映射。

**RouterFunctions** 中的 `route()` 方法接受两个参数：**RequestPredicate** 和处理请求匹配的函数。在本例中，**RequestPredicates** 的 `GET()` 方法声明了一个 **RequestPredicate**，它与 `/hello` 路径的 HTTP GET 请求相匹配。

至于 **handler** 函数，它是作为 **lambda** 编写的，尽管它也可以是方法引用。虽然没有显式声明，但是处理程序 **lambda** 接受一个 **ServerRequest** 作为参数。它使用来自 **ServerResponse** 的 `ok()` 和来自 **BodyBuilder** 的 `body()` 返回一个 **ServerResponse**，后者是从 `ok()` 返回的。这样做是为了创建一个带有 HTTP 200（OK）状态代码和一个表示 **Hello World** 的 **body** 负载的响应！

如前所述，`helloRouterFunction()` 方法声明了一个仅处理单一类型请求的 **RouterFunction**。但是如果需要处理不同类型的请求，不必编写另一个 `@Bean` 方法。只需要调用 `andRoute()` 来声明另一个 **RequestPredicate** 到函数的映射。例如，下面介绍如何为 `/bye` 的 **GET** 请求添加另一个处理程序：

```
@Bean
public RouterFunction<?> helloRouterFunction() {
    return route(GET("/hello"),
        request -> ok().body(just("Hello World!"), String.class))
        .andRoute(GET("/bye"),
            request -> ok().body(just("See ya!"), String.class))
    ;
}
```

**Hello World** 的例子可以让您接触到新的东西。但是让我们把它放大一点，看看如何使用 **Spring** 的函数式 **web** 编程模型来处理类似于真实场景的请求。

为了演示函数式编程模型如何在实际应用程序中使用，让我们将 **DesignTacoController** 的功能重新设计为函数式样式。以下配置类是 **DesignTacoController** 的功能模拟：

```

package tacos.web.api;

import static org.springframework.web.reactive.function.server.RequestPredicates.GET;
import static org.springframework.web.reactive.function.server.RequestPredicates.POST;
import static org.springframework.web.reactive.function.server.RequestPredicates.queryParam;
import static org.springframework.web.reactive.function.server.RouterFunctions.route;

import java.net.URI;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.web.reactive.function.server.RouterFunction;
import org.springframework.web.reactive.function.server.ServerRequest;
import org.springframework.web.reactive.function.server.ServerResponse;

import reactor.core.publisher.Mono;
import tacos.Taco;
import tacos.data.TacoRepository;

@Configuration
public class RouterFunctionConfig {

    @Autowired
    private TacoRepository tacoRepo;

    @Bean
    public RouterFunction<?> routerFunction() {
        return route(GET("/api/tacos").
            and(queryParam("recent", t->t != null )),
            this::recents)
            .andRoute(POST("/api/tacos"), this::postTaco);
    }

    public Mono<ServerResponse> recents(ServerRequest request) {
        return ServerResponse.ok()
            .body(tacoRepo.findAll().take(12), Taco.class);
    }

    public Mono<ServerResponse> postTaco(ServerRequest request) {
        return request.bodyToMono(Taco.class)
            .flatMap(taco -> tacoRepo.save(taco))
            .flatMap(savedTaco -> {
                return ServerResponse
                    .created(URI.create(
                        "http://localhost:8080/api/tacos/" +
                        savedTaco.getId()))
                    .body(savedTaco, Taco.class);
            });
    }
}

```

如您所见，`routerFunction()` 方法声明了一个 `RouterFunction<?>` 的 `bean`，就像 `Hello World` 的例子。但在处理哪些类型的请求以及如何处理这些请求方面有所不同。在本例中，创建 `RouterFunction` 来处理 `/api/tacos` 的 GET 请求和 `/api/tacos` 的 POST 请求。

更突出的是路由是由方法引用处理的。当 `RouterFunction` 后面的行为相对简单和简短时，`lambda` 非常好。但是，在许多情况下，最好将该功能提取到单独的方法中（甚至在单独的类中提取到单独的方法中），以保持代码的可读性。

根据您的需要，`/api/tacos?recent` 的 GET 请求将由 `recents()` 方法处理。它使用注入的 `TacoRepository` 来获取一个 `Mono<Taco>`，从中提取 12 个项目。然后将 `Flux<Taco>` 包装在 `Mono<ServerResponse>` 中，以确保响应显示 HTTP 200 (OK) 状态。这通过在 `ServerResponse` 上调用 `ok()` 来实现。尽管返回了多达 12 个玉米卷，但只有一个服务器响应，理解这一点是很重要的。这就是为什么以 `Mono` 而非 `Flux` 类型来返回。在内部，Spring 仍然会将 `Flux<Taco>` 流作为 `Flux` 返回到客户端。

同时，`postTaco()` 方法处理 `/api/tacos` 的 POST 请求，该方法从传入的 `ServerRequest` 中提取 `Mono<Taco>`。然后 `postTaco()` 方法使用一系列 `flatMap()` 操作将 `taco` 保存到 `TacoRepository`，并创建服务器响应。带有 HTTP 201 (已创建) 状态码以及保存好的 `Taco` 对象在响应体中。

`flatMap()` 操作用于确保在流的每个步骤中，结果被包装在一个 `Mono` 中，从第一个 `flatMap()` 之后的 `Mono<Taco>` 开始，然后最终以 `postTaco()` 返回的 `Mono<ServerResponse>` 结束。

## 12.3 测试响应式 Controller

在测试响应式 Controller 时，Spring 并没有让我们陷入困境。实际上，Spring 引入了 `WebTestClient`，这是一个新的测试程序，它让用 Spring WebFlux 编写的响应式 Controller 变得容易测试。让我们首先使用它测试在第 12.1.2 节中编写的 `TacoController` 中的 `recentTacos()` 方法，来了解如何使用 `WebTestClient` 编写测试用例。

## 12.3.1 测试 GET 请求

对于 `recentTacos()` 方法，我们想声明的一件事是，如果为 `/api/tacos?recent` 路径发出了 HTTP GET 请求，那么响应将包含一个不超过 12 个 `tacos` 的 JSON 数据。下面清单中的测试类是一个很好的开始。

程序清单 12.1 使用 **WebTestClient** 测试 **DesignTacoController**

```
package tacos.web.api;
import static org.mockito.ArgumentMatchers.any;
import static org.mockito.Mockito.when;
import java.util.ArrayList;
import java.util.List;
import org.junit.jupiter.api.Test;
import org.mockito.Mockito;
import org.springframework.http.MediaType;
import org.springframework.test.web.reactive.server.WebTestClient;
import reactor.core.publisher.Flux;
import reactor.core.publisher.Mono;
import tacos.Ingredient;
import tacos.Ingredient.Type;
import tacos.Taco;
import tacos.data.TacoRepository;

public class TacoControllerTest {

    @Test
    public void shouldReturnRecentTacos() {
        Taco[] tacos = {
            testTaco(1L), testTaco(2L),
            testTaco(3L), testTaco(4L),
            testTaco(5L), testTaco(6L),
            testTaco(7L), testTaco(8L),
            testTaco(9L), testTaco(10L),
            testTaco(11L), testTaco(12L),
            testTaco(13L), testTaco(14L),
            testTaco(15L), testTaco(16L)};
        Flux<Taco> tacoFlux = Flux.just(tacos);

        TacoRepository tacoRepo = Mockito.mock(TacoRepository.class);
        when(tacoRepo.findAll()).thenReturn(tacoFlux);

        WebTestClient testClient = WebTestClient.bindToController(
            new TacoController(tacoRepo))
            .build();

        testClient.get().uri("/api/tacos?recent")
            .exchange()
            .expectStatus().isOk()
            .expectBody()
            .jsonPath("$").isArray()
            .jsonPath("$").isNotEmpty()
            .jsonPath("$[0].id").isEqualTo(tacos[0].getId().toString())
            .jsonPath("$[0].name").isEqualTo("Taco 1")
            .jsonPath("$[1].id").isEqualTo(tacos[1].getId().toString())
            .jsonPath("$[1].name").isEqualTo("Taco 2")
            .jsonPath("$[11].id").isEqualTo(tacos[11].getId().toString())
            .jsonPath("$[11].name").isEqualTo("Taco 12")
            .jsonPath("$[12]").doesNotExist();
    }

    ...
}
```

`shouldReturnRecentTacos()` 方法做的第一件事是以 `Flux<Taco>` 的形式设置测试数据。然后，这个 `Flux` 作为模拟 `TacoRepository` 的 `findAll()` 方法的返回值。

对于将由 `Flux` 发布的 `Taco` 对象，它们是用一个名为 `testTaco()` 的方法创建的，当给定一个数字时，该方法将生成一个 `Taco` 对象，其 ID 和名称基于该数字。`testTaco()` 方法实现如下：

```
private Taco testTaco(Long number) {
    Taco taco = new Taco();
    taco.setId(number != null ? number.toString(): "TESTID");
    taco.setName("Taco " + number);
    List<Ingredient> ingredients = new ArrayList<>();
    ingredients.add(
        new Ingredient("INGA", "Ingredient A", Type.WRAP));
    ingredients.add(
        new Ingredient("INGB", "Ingredient B", Type.PROTEIN));
    taco.setIngredients(ingredients);
    return taco;
}
```

为了简单起见，所有的测试 `tacos` 都有相同的两种成分。但它们的 ID 和名字将由给定的号码决定。

同时，在 `shouldReturnRecentTacos()` 方法中，实例化了一个 `TacoController`，将模拟的 `TacoRepository` 注入构造函数。`Controller` 被赋予 `WebTestClient.bindToController()` 以创建 `WebTestClient` 的实例。

完成所有设置后，现在可以使用 `WebTestClient` 向 `/api/tacos?recent` 提交 GET 请求，并验证响应是否满足预期。调用 `get().uri("/api/tacos?recent")` 描述要发出的请求。然后调用 `exchange()` 提交请求，该请求将由绑定到 `TacoController` 的 `Controller` 进行处理。

最后，可以确认响应与预期一致。通过调用 `expectStatus()`，可以断言响应具有 HTTP 200(OK) 状态代码。之后，将看到对 `jsonPath()` 的几个调用，这些调用断言响应体中的 JSON 具有它应该具有的值。最后的断言检查第 12 个元素（在基于零的数组中）是否不存在，因为结果不应超过 12 个元素。

如果 JSON 返回很复杂，包含大量数据或高度嵌套的数据，那么使用 `jsonPath()` 可能会很无聊。实际上，为了节省空间，清单 12.1 中已经省略了对 `jsonPath()` 的许多调用。对于那些使用 `jsonPath()` 可能很笨拙的情况，`WebTestClient` 提供了 `json()`，它接受包含 json 的 `String` 参数来对响应进行响应。

例如，假设在一个名为 `recent-tacos.json` 的文件中创建了完整的响应 JSON，并将其放在路径 `/tacos` 下的类路径中。然后重写 `WebTestClient` 断言，如下所示：

```
ClassPathResource recentsResource =
    new ClassPathResource("/tacos/recent-tacos.json");
String recentsJson = StreamUtils.copyToString(
    recentsResource.getInputStream(), Charset.defaultCharset());

testClient.get().uri("/api/tacos?recent")
    .accept(MediaType.APPLICATION_JSON)
    .exchange()
    .expectStatus().isOk()
    .expectBody()
    .json(recentsJson);
```

因为 `json()` 接受 `String`，所以必须首先将类路径资源加载到 `String` 对象中。谢天谢地，`Spring` 中的 `StreamUtils` 使 `copyToString()` 的使用变得简单。`copyToString()` 返回的 `String` 将包含响应请求时预期的整个 JSON。将它赋给 `json()` 方法可以确保 `Controller` 产生正确的输出。



**WebTestClient** 提供的另一个选项，允许将响应体与值列表进行比较。`expectBodyList()` 方法接受指示列表中元素类型的 `Class` 或 `ParameterizedTypeReference`，并返回要针对其进行断言的 `istBodySpec` 对象。使用 `expectBodyList()`，可以重写测试以使用用于创建模拟 `TacoRepository` 的相同测试数据的子集：

```
testClient.get().uri("/api/tacos?recent")
    .accept(MediaType.APPLICATION_JSON)
    .exchange()
    .expectStatus().isOk()
    .expectBodyList(Taco.class)
        .contains(Arrays.copyOf(tacos, 12));
```

在这里，断言响应体包含的列表，与在测试方法开始时创建的原始 `Taco` 数组的前 12 个元素，具有相同的元素。

## 12.3.2 测试 POST 请求

`WebTestClient` 可以做的不仅仅是针对 `Controller` 的 `GET` 请求进行测试。它还可以用于测试任何类型的 `HTTP` 方法，包括 `GET`、`POST`、`PUT`、`PATCH`、`DELETE` 和 `HEAD` 请求。表 12.1 将 `HTTP` 方法映射到 `WebTestClient` 方法。

表 12.1 `WebTestClient` 针对 `Spring WebFlux` 控制器测试任何类型的请求。

| HTTP 方法 | WebTestClient 方法       |
|---------|------------------------|
| GET     | <code>.get()</code>    |
| POST    | <code>.post()</code>   |
| PUT     | <code>.put()</code>    |
| PATCH   | <code>.patch()</code>  |
| DELETE  | <code>.delete()</code> |
| HEAD    | <code>.head()</code>   |

作为针对 `Spring WebFlux Controller` 中的另一个 `HTTP` 方法请求示例的测试，让我们看看针对 `DesignTacoController` 的另一个测试。这次，将通过向 `/api/tacos` 提交 `POST` 请求来编写针对创建 `taco` 端点 `API` 的测试：

```
@SuppressWarnings("unchecked")
@Test
public void shouldSaveATaco() {
    TacoRepository tacoRepo = Mockito.mock(
        TacoRepository.class);

    WebTestClient testClient = WebTestClient.bindToController(
        new TacoController(tacoRepo)).build();

    Mono<Taco> unsavedTacoMono = Mono.just(testTaco(1L));
    Taco savedTaco = testTaco(1L);
    Flux<Taco> savedTacoMono = Flux.just(savedTaco);

    when(tacoRepo.saveAll(any(Mono.class))).thenReturn(savedTacoMono);

    testClient.post()
        .uri("/api/tacos")
        .contentType(MediaType.APPLICATION_JSON)
        .body(unsavedTacoMono, Taco.class)
        .exchange()
        .expectStatus().isCreated()
        .expectBody(Taco.class)
        .isEqualTo(savedTaco);
}
```

与前面的测试方法一样，`shouldSaveATaco()` 首先设置一些测试数据，模拟 `TacoRepository`，然后构建一个 `WebTestClient`，并绑定到 `Controller`。然后，使用 `WebTestClient` 向 `/api/tacos` 提交 `POST` 请求，请求的 `body` 类型为 `application/json`，有效负载是未保存 `Mono` 中 `Taco` 的 `json` 序列化形式。在执行 `exchange()` 之后，测试断言响应应具有 `HTTP 201 (CREATED)` 状态，并且正文中的有效负载等于保存的 `Taco` 对象。

## 12.3.3 使用线上服务器进行测试

到目前为止，编写的测试依赖于 **Spring WebFlux** 框架的模拟实现，因此不需要真正的服务器。但可能需要在 **Netty** 或 **Tomcat** 等服务器的上下文中测试 **WebFlux Controller**，并且可能需要使用 **repository** 或其他依赖项。也就是说，可能需要编写一个集成测试。

要编写 **WebTestClient** 集成测试，首先使用 `@RunWith` 和 `@SpringBootTest` 对测试类进行注解，就像其他任何 **Spring Boot** 集成测试一样：

```
package tacos;

import java.io.IOException;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.boot.test.context.SpringBootTest.WebEnvironment;
import org.springframework.http.MediaType;
import org.springframework.test.context.junit.jupiter.SpringExtension;
import org.springframework.test.web.reactive.server.WebTestClient;

@ExtendWith(SpringExtension.class)
@SpringBootTest(webEnvironment=WebEnvironment.RANDOM_PORT)
public class TacoControllerWebTest {

    @Autowired
    private WebTestClient testClient;

}
```

通过将 `webEnvironment` 属性设置为 `webEnvironment.RANDOM_PORT`，将要求 **Spring** 启动正在运行的服务器来监听随机选择的端口。

**WebTestClient** 还将自动连接到测试类中。这不仅意味着将不再需要在测试方法中创建一个 **URL**，而且在发出请求时也不需要指定完整的 **URL**。这是因为 **WebTestClient** 将被装配成知道测试服务器在哪个端口上运行。现在可以将 `shouldReturnRecentTacos()` 重写为使用自动连线 **WebTestClient** 的集成测试：

```
@Test
public void shouldReturnRecentTacos() throws IOException {
    testClient.get().uri("/api/tacos?recent")
        .accept(MediaType.APPLICATION_JSON).exchange()
        .expectStatus().isOk()
        .expectBody()
        .jsonPath("$").isArray()
        .jsonPath("$.length()").isEqualTo(3)
        .jsonPath("$[?(@.name == 'Carnivore')]").exists()
        .jsonPath("$[?(@.name == 'Bovine Bounty')]").exists()
        .jsonPath("$[?(@.name == 'Veg-Out')]").exists();
}
```

毫无疑问，您已经注意到这个新版本的 `shouldReturnRecentTacos()` 的代码要少得多。因为将使用自动注入的实例，因此不再需要创建 **WebTestClient**。而且没有必要模拟 **TacoRepository**，因为 **Spring** 将创建 **DesignTacoController** 的一个实例，并为它注入一个真正的 **TacoRepository**。在这个新版本的测试方法中，使用 **JSONPath** 表达式来验证从数据库提供的值。

在测试过程中，当需要使用 **WebFlux Controller** 公开的 **API** 时，**WebTestClient** 非常有用。但是，当应用程序本身使用其他 **API** 时呢？让我们把注意力转向 **Spring** 的响应式 **web** 的客户端，看看 **WebClient** 是如何提供 **REST** 客户端来处理诸如 **Mono** 和 **Flux** 之类的响应式类型的。

## 12.4 响应式消费 REST API

在第 8 章，您已经使用过 `RestTemplate` 通过客户端向 Taco Cloud API 发出请求。`RestTemplate` 是一个老组件了，引入自 Spring 3.0 版本。至今为止，我们使用它向应用程序发出过无数次请求。

但是 `RestTemplate` 提供的所有处理方法都不属于响应式的类型和领域。这就意味着如果您想以响应式的方式处理一个响应的数据，您必须用 `Flux` 或 `Mono` 去包装它。如果您想用 `Post` 或者 `Put` 方法发送已有的 `Flux` 或 `Mono` 请求，您需要将先数据抽取出来再放入一个非响应式的类。

所以，能够有一种办法使用原生的 `RestTemplate` 去处理响应式类型就很棒了。不用担心，Spring 提供了 `WebClient` 作为 `RestTemplate` 的响应式版本。当 `WebClient` 向外部 APIs 发出请求的同时，也可以发出或接收响应式类型。

使用 `WebClient` 和使用 `RestTemplate` 是完全不一样的体验。`RestTemplate` 会使用不同的方法处理不同类型的请求，而 `WebClient` 拥有了一个流畅的构建者风格接口（`builderstyle interface`），可以让您描述并发送请求。`WebClient` 的常用使用方法有以下几种：

- 创建一个 `WebClient` 实例（或者注入一个 `WebClient Bean`）
- 指定发送请求的 HTTP 方法
- 指定请求中所必要的 URI 和 header
- 提交请求
- 消费响应

接下来让我们看几个 `WebClient` 实战的例子，开始了解如何使用 `WebClient` 来发送 HTTP GET 请求。

## 12.4.1 获取资源

作为使用 `WebClient` 的样例，假设您需要通过 Taco Cloud API 根据 ID 获取 `Ingredient` 对象。如果使用 `RestTemplate`，那么您可能会使用 `getForObject()` 方法。但是，借助 `WebClient` 的话，您可以构建请求、获取响应并抽取一个会发布 `Ingredient` 对象的 `Mono`：

```
Mono<Ingredient> ingredient = WebClient.create()
    .get()
    .uri("http://localhost:8080/ingredients/{id}", ingredientId)
    .retrieve()
    .bodyToMono(Ingredient.class);

ingredient.subscribe(i -> { ... });
```

在这里，您使用 `create()` 创建了一个新的 `WebClient` 实例。然后，您可以使用 `get()` 和 `uri()` 定义对 `http://localhost:8080/ingredients/{id}` 的 GET 请求，其中 `{id}` 占位符将会被 `ingredientId` 的值所替换。接着，`retrieve()` 会执行请求。最后，我们调用 `bodyToMono()` 将响应体的载荷抽取到 `Mono<Ingredient>` 中，就可以继续使用 `Mono` 的额外操作了。

为了对 `bodyToMono()` 返回 `Mono` 进行额外的操作，需要注意的很重要的一点是要在请求发送之前对其进行订阅。发送请求获取值的集合是非常容易的。例如，如下的代码片段将获取所有 `Ingredient`：

```
Flux<Ingredient> ingredients = WebClient.create()
    .get()
    .uri("http://localhost:8080/ingredients")
    .retrieve()
    .bodyToFlux(Ingredient.class);
ingredients.subscribe(i -> { ... });
```

大部分而言，获取多个条目与获取单个条目是相同的。最大的差异在于我们不再使用 `bodyToMono()` 将响应体抽取为 `Mono`，而是使用 `bodyToFlux()` 将其抽取为一个 `Flux`。与 `bodyToMono()` 类似，`bodyToFlux()` 返回的 `Flux` 还没有被订阅。在数据流过之前，您可以对 `Flux` 添加一些额外的操作（过滤、映射等）。因此，非常重要的一点就是要订阅结果所形成的 `Flux`，否则请求将始终不会发送。

## 使用基础 URI 发送请求

您可能会发现在很多请求中都会使用一个通用的基础 URI。这样的话，创建 `WebClient bean` 的时候设置一个基础 URI 并将其注入到所需的地方是非常有用的。这样的 `bean` 可以按照如下的方式来声明（任何使用了 `@Configuration` 注解的类）：

```
@Bean
public WebClient webClient() {
    return WebClient.create("http://localhost:8080");
}
```

然后，在想要使用基础 URI 的任意地方，您都可以将 `WebClient bean` 注入进来并按照如下的方式使用：

```

@Autowired
WebClient webClient;
public Mono<Ingredient> getIngredientById(String ingredientId) {
    Mono<Ingredient> ingredient = webClient
        .get()
        .uri("/ingredients/{id}", ingredientId)
        .retrieve()
        .bodyToMono(Ingredient.class);

    ingredient.subscribe(i -> { ... });
}

```

因为 `WebClient` 已经创建好了，所以您可以通过 `get()` 方法直接使用它。对于 `URI` 来说，您只需要调用 `uri()` 指定相对于基础 `URI` 的相对路径即可。

## 对长时间运行的请求进行超时处理

您需要考虑的一件事情就是，网络并不是始终可靠的，或者并不像您预期的那么快，远程服务器在处理请求时有可能会非常缓慢。理想情况下，对远程服务的请求会在一个合理的时间内返回。无法正常返回的话，客户端要是能够避免陷入长时间等待响应的窘境就好了。为了避免客户端请求被缓慢的网络或服务阻塞，您可以使用 `Flux` 或 `Mono` 的 `timeout()` 方法，为等待数据发布的过程设置一个时长限制。作为样例，您可以考虑一下如何为获取配料数据使用 `timeout()` 方法：

```

Flux<Ingredient> ingredients = webclient
    .get()
    .uri("/ingredients")
    .retrieve()
    .bodyToFlux(Ingredient.class);

ingredients
    .timeout(Duration.ofSeconds(1))
    .subscribe(
        i -> { ... },
        e -> {
            //handle timeout error
        });

```

可以看到，在订阅 `Flux` 之前，调用了 `timeout()` 方法，将持续时间设置成了 1 秒。如果请求能够在 1 秒之内返回，就不会有任何问题。但是，如果请求花费的时间超过 1s，则会超时，然后会调用 `subscribe()` 的第二个参数进行错误处理。

## 12.4.2 发送资源

使用 `WebClient` 发送数据与接收数据没有多大区别。例如，假设您拥有一个 `Mono`，并希望使用由 `Mono` 发布的 `Ingredient` 发送一个 `POST` 请求到具有 `/ingredients` 相对路径的 `URI`。您所要做的是使用 `post()` 方法而不是 `get()` 方法，并指定使用 `Mono` 调用 `body()` 来填充请求体：

```
Mono<Ingredient> ingredientMono = Mono.just(
    new Ingredient("INGC", "Ingredient C", Ingredient.Type.VEGGIES));

Mono<Ingredient> result = webClient
    .post()
    .uri("/ingredients")
    .body(ingredientMono, Ingredient.class)
    .retrieve()
    .bodyToMono(Ingredient.class);

result.subscribe(i -> { ... });
```

如果您没有要发送的 `Mono` 或 `Flux`，而是手头有原始领域实体对象，您可以使用 `syncBody()`。例如，假设您要在请求体中发送的是一个 `Ingredient` 对象，而不是 `Mono<Ingredient>` 对象：

```
IIngredient ingredient = ...;

Mono<Ingredient> result = webClient
    .post()
    .uri("/ingredients")
    .bodyValue(ingredient)
    .retrieve()
    .bodyToMono(Ingredient.class);

result.subscribe(i -> { ... });
```

如果不是 `POST` 请求，而是要用 `PUT` 请求更新 `Ingredient`，则调用 `put()` 而不是 `post()` 并相应调整 `URI` 路径：

```
Mono<Void> result = webClient
    .put()
    .uri("/ingredients/{id}", ingredient.getId())
    .bodyValue(ingredient)
    .retrieve()
    .bodyToMono(Void.class);

result.subscribe();
```

`PUT` 请求通常具有空的响应体，因此必须请求 `bodyToMono()` 返回类型为 `Void` 的 `Mono`。订阅该 `Mono` 时，将发送请求。



### 12.4.3 删除资源

**WebClient** 还允许使用 `delete()` 方法删除资源。例如，下面的代码通过给定的 ID 删除 **Ingredient** 对象：

```
Mono<Void> result = webClient
    .delete()
    .uri("/ingredients/{id}", ingredientId)
    .retrieve()
    .bodyToMono(Void.class);

result.subscribe();
```

和 **PUT** 请求一样，**DELETE** 请求通常没有响应体。同样，返回并订阅 `Mono<Void>` 以发送请求。

## 12.4.4 处理错误

到目前为止，所有的 **WebClient** 示例都假设了一个圆满的结局；请求不会返回任何 400 或 500 级别的 HTTP 状态码。当这些错误状态返回时，**WebClient** 应该会记录失败日志；否则，静悄悄地忽略它。

如果需要处理此类错误，则可以使用对 `onStatus()` 的调用来指定如何处理各种 HTTP 状态码。`onStatus()` 接受两个函数：一个 **Predicate** 函数，用于匹配 HTTP 状态。另一个函数给定 **ClientResponse** 对象，返回 `Mono<Throwable>`。

为了演示如何使用 `onStatus()` 创建自定义错误处理程序，考虑以下通过 ID 获取 **Ingredient** 的用法：

```
Mono<Ingredient> ingredientMono = webClient
    .get()
    .uri("http://localhost:8080/ingredients/{id}", ingredientId)
    .retrieve()
    .bodyToMono(Ingredient.class);
```

只要 `ingredientId` 中的值与已知的资源匹配，那么生成的 **Mono** 将在订阅时发布返回 **Ingredient** 对象。但是，如果没有匹配的 **Ingredient** 对象，那会发生什么事情呢？

当订阅 **Mono** 或 **Flux** 可能会发生错误时，注册一个出错处理函数是很重要的。就像在调用 `subscribe()` 方法时注册数据处理函数一样：

```
ingredientMono.subscribe(
    ingredient -> {
        // handle the ingredient data
        ...
    },
    error -> {
        // deal with the error
        ...
    });
```

如果找到了 **Ingredient** 资源对象，则在 `subscribe()` 方法中指定的第一个 **lambda**（数据处理函数）会被调用。如果没有找到资源对象，请求会用状态码 `http 404 (NOT FOUND)` 进行响应，这样会给第二个 **lambda**（错误处理函数）默认传入一个 **WebClientResponseException** 对象。

**WebClientResponseException** 对象的最大问题是它的非特异性，它无法说明具体是什么出了问题导致 **Mono** 出错。它只表明 **WebClient** 发出的请求，在响应时存在错误。您需要进一步深入 **WebClientResponseException** 才能知道具体出了什么问题。在任何情况下，给错误处理函数一个具体的异常值才更好，而不是给一个特定于 **WebClient** 的异常。

通过添加自定义的错误处理函数，您可以自主地将返回的出错状态码转换成一个 **Throwable**。假设您想要让一个失败的 **Ingredient** 请求，返回一个异常 **UnknownIngredientException**，您可以在调用 `retrieve()` 之后添加对 `onStatus()` 的调用：

```
Mono<Ingredient> ingredientMono = webClient
    .get()
    .uri("http://localhost:8080/ingredients/{id}", ingredientId)
    .retrieve()
    .onStatus(HttpStatus::is4xxClientError,
        response -> Mono.just(new UnknownIngredientException()))
    .bodyToMono(Ingredient.class);
```

`onStatus()` 函数调用中的第一个参数是 `Predicate` 类型的 `HttpStatus`，如果是要处理的状态码，则返回 `true`。如果状态码匹配，则响应将会传给第二个参数指定的函数，以进一步进行处理。最终返回一个 `Throwable` 类型的 `Mono`。

在本例中，如果状态码是 400 级别的状态码（例如，客户端错误），则会返回一个带有 `UnknownIngredientException` 的 `Mono`。这会导致 `ingredientMono` 因该异常而失败。

请注意，`HttpStatus::is4xxClientError` 是对 `HttpStatus` 中 `is4xxClientError` 方法的引用。这个方法将在给定 `HttpStatus` 对象时被实际调用。如果需要，可以使用 `HttpStatus` 上的其他方法作为方法引用；或者您提供自己的 `lambda` 或返回布尔值的其他方法引用。

例如，您可以在错误处理函数中更具体的检查处理 `HTTP 404 (NOT FOUND)`，这可以通过对 `onStatus()` 的调用更改来实现，就像这样：

```
Mono<Ingredient> ingredientMono = webClient
    .get()
    .uri("http://localhost:8080/ingredients/{id}", ingredientId)
    .retrieve()
    .onStatus(status -> status == HttpStatus.NOT_FOUND,
        response -> Mono.just(new UnknownIngredientException()))
    .bodyToMono(Ingredient.class);
```

另外值得注意的是，可以根据需要调用任意多个 `onStatus()`，以处理响应中可能返回的各种 HTTP 状态码。

## 12.4.5 请求转换

到目前为止，您已经使用了 `WebClient` 的 `retrieve()` 方法来发送请求。在这些情况中，`retrieve()` 方法返回 `ResponseSpec` 类型的对象。通过它可以使用 `onStatus()`、`bodyToFlux()` 和 `bodyToMono()` 等方法处理响应。使用 `ResponseSpec` 对象适用于简单的一些情况，某些方面会受到限制。举例来说，如果您需要获取响应的 `header` 或 `cookie`，使用 `ResponseSpec` 就无法完成。

当 `ResponseSpec` 不满足需求时，可以使用 `exchange()` 取代 `retrieve()`。`exchange()` 方法返回 `ClientResponse` 类型的 `Mono`，您可以应用反应式操作来检查和使用完整的返回数据，包括有效载荷、`headers` 和 `Cookies`。

在研究 `exchange()` 与 `retrieve()` 的区别之前，让我们先看看他们有多相似。下面的代码片段使用 `WebClient` 和 `exchange()`，按 ID 获取单个 `Ingredient`：

```
Mono<Ingredient> ingredientMono = webClient
    .get()
    .uri("http://localhost:8080/ingredients/{id}", ingredientId)
    .exchangeToMono(cr -> cr.bodyToMono(Ingredient.class));
```

这大致相当于下面使用 `retrieve()` 的示例：

```
Mono<Ingredient> ingredientMono = webClient
    .get()
    .uri("http://localhost:8080/ingredients/{id}", ingredientId)
    .retrieve()
    .bodyToMono(Ingredient.class);
```

在 `exchangeToMono()` 示例中，没有使用 `ResponseSpec` 对象的 `bodyToMono()` 获得 `Mono<Ingredient>`。而是通过一个扁平化映射函数，将 `ClientResponse` 映射到 `Mono<Ingredient>`，从而得到 `Mono<ClientResponse>`，也即一个 `Mono`。

现在让我们看看 `exchangeToMono()` 有什么不同之处。让我们假设，请求中可能包含一个名为 `X_UNAVAILABLE` 的请求头。当其值为 `true` 时，表示（由于某种原因）相关的 `Ingredient` 不可用。为了便于讨论，假设该头存在，您肯定希望得到的结果 `Mono` 为空，不返回任何内容。这可以通过添加另一个对 `flatMap()` 的调用来实现。整个 `WebClient` 调用像如下这样：

```
Mono<Ingredient> ingredientMono = webClient
    .get()
    .uri("http://localhost:8080/ingredients/{id}", ingredientId)
    .exchangeToMono(cr -> {
        if (cr.headers().header("X_UNAVAILABLE").contains("true")) {
            return Mono.empty();
        }
        return Mono.just(cr);
    })
    .flatMap(cr -> cr.bodyToMono(Ingredient.class));
```

新的 `flatMap()` 调用，检查给定 `ClientRequest` 对象的响应头。查找名称为 `X_UNAVAILABLE` 且值为 `true` 的 `header`。如果找到，则返回一个空的 `Mono`。否则，它将返回一个包含 `ClientResponse` 的新 `Mono` 对象。无论是哪种情况，`Mono` 对象都将通过 `flatMap()` 进一步扁平化映射后再返回。

## 12.5 保护响应式 web API

自从 Spring Security（之前广为人知的名称是 Acegi Security）以来，web 安全模型都是围绕 servlet 过滤器构建的。毕竟，这是很明显的逻辑。为了确保请求者拥有适当的权限，拦截基于 servlet 的请求，显而易见的就是选择使用 servlet 过滤器。但是 Spring WebFlux 给这种方法带来了一个难题。

在使用 Spring WebFlux 编写 web 应用程序时，并不能保证 servlet 甚至还参与其中。事实上，一个响应式 web 应用程序，更有可能是构建在 Netty 或其他非 servlet 服务器上的。这是否意味着基于 servlet 过滤器的 Spring Security，不能用于保护 Spring WebFlux 应用程序呢？

在保护 Spring WebFlux 应用程序时，确实不能选择使用 servlet 过滤器。但 Spring Security 仍能胜任这项任务。从 5.0.0 版开始，Spring Security 可以用来保护基于 servlet 的 Spring MVC 和响应式 Spring WebFlux 应用程序。它使用 Spring 的 WebFilter 实现这一点，这是 Spring 特有的，类似于 servlet 过滤器，但并不需要依赖 servlet API。

更值得一提的是，响应式 Spring Security 的配置模型与您在第4章中看到的没有太大区别。事实上，不像 Spring WebFlux 与 Spring MVC 有一个独立的依赖关系，Spring Security 使用和以前一样的 starter，无论您是否打算使用它来保护 Spring MVC 的 web 应用程序或用 Spring WebFlux 的应用程序。再重温一下，starter 的配置是这样的：

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-security</artifactId>
</dependency>
```

也就是说，Spring Security 的响应式和非响应式配置模型没什么大的区别。让我们来快速比较查看一下两种配置的模型。

## 12.5.1 配置响应式 Web 安全

重提一下，配置 Spring Security 以确保 Spring MVC 的 web 应用程序的安全，通常涉及创建一个扩展 `WebSecurityConfigurerAdapter` 的新配置类，并添加 `@EnableWebSecurity` 注解。这样的配置类通过覆写 `configuration()` 方法，以指定 web 安全性规范，例如：哪些请求路径需要什么样的授权。以下简单的配置类，让您重温一下非响应式 Spring MVC 应用程序，是如何配置 Spring Security 的：

```
@Configuration
@EnableWebSecurity
public class SecurityConfig extends WebSecurityConfigurerAdapter {

    @Override
    protected void configure(HttpSecurity http) throws Exception {
        http
            .authorizeRequests()
                .antMatchers("/api/tacos", "/orders").hasAuthority("USER")
                .antMatchers("/**").permitAll();
    }
}
```

现在，让我们看看对于响应式 Spring WebFlux 应用，同样的配置是什么样子。下面的清单显示了一个响应式安全配置类，这大致相当于上面的简单配置类。

程序清单 121.2 为 Spring WebFlux 应用配置 Spring Security。

```
@Configuration
@EnableWebFluxSecurity
public class SecurityConfig {

    @Bean
    public SecurityWebFilterChain securityWebFilterChain(
        ServerHttpSecurity http) {

        return http
            .authorizeExchange()
                .pathMatchers("/api/tacos", "/orders").hasAuthority("USER")
                .anyExchange().permitAll()
            .and()
                .build();
    }
}
```

正如您所见，有很多是熟悉的，但同时又有些不同。比如这个新的配置类没有使用 `@EnableWebSecurity`，而是使用了 `@EnableWebFluxSecurity`。此外，也没有继承像 `WebSecurityConfigurerAdapter` 的任何其他基类。因此，它也不用覆写任何 `configure()` 方法。

代替 `configure()` 方法，您可以使用 `securityWebFilterChain()` 方法声明 `SecurityWebFilterChain` 类型的 bean。而 `securityWebFilterChain()` 的方法体与前面配置的 `configure()` 方法没有太大区别，只有一些微小的变化。

首先，使用给定的 `ServerHttpSecurity` 对象进行配置，而不是 `HttpSecurity` 对象。使用 `ServerHttpSecurity` 声明请求级别安全性，可以调用 `authorizeExchange()`，这大致相当于 `authorizeRequests()`。

注意： `ServerHttpSecurity` 是 Spring Security 5 的中新加的功能，是 `HttpSecurity` 的一种响应式编程的模拟。

在匹配路径时，仍然可以使用 **Ant** 样式的通配符路径，但要使用 `pathMatchers()` 方法而不是 `antMatchers()`。为了方便起见，您不再需要指定 `/**` 来匹配所有路径，因为 `anyExchange()` 会捕获所有路径。

最后，因为您将 `SecurityWebFilterChain` 声明为 **bean**，且没有覆写任何框架方法，所以必须调用 `build()` 方法，来组装所有安全规则以返回给 `SecurityWebFilterChain`。

除了这些小的区别，对于 `Spring WebFlux` 和 `Spring MVC` 的 **web** 安全配置基本一致。但如何指定有关用户的安全配置呢？

## 12.5.2 配置响应式用户信息服务

在扩展 `WebSecurityConfigurerAdapter` 时，可以通过覆写 `configure()` 方法来声明 **web** 安全规则。通常通过定义 `UserDetails` 对象，并使用另一个 `configure()` 方法配置用户身份验证逻辑。重温一下，下面的代码是通过 `UserDetails` 的匿名实现类，使用注入的 `UserRepository` 对象，按用户名查找用户：

```
@Autowired
UserRepository userRepo;

@Override
protected void
configure(AuthenticationManagerBuilder auth)
throws Exception {
auth
.userDetailsService(new UserDetailsService() {
@Override
public UserDetails loadUserByUsername(String username)
throws UsernameNotFoundException {
User user = userRepo.findByUsername(username)
if (user == null) {
throw new UsernameNotFoundException(
username + " not found")
}
return user.toUserDetails();
}
});
}
```

在这个非响应式配置中，您覆写查找用户所需的唯一方法 `UserDetailsService.loadUserByUsername()`。在这个方法中，您使用给定的 `UserRepository`，按给定用户名查找用户。如果找不到，就抛出 `UsernameNotFoundException`。如果找到了，就使用 `toUserDetails()` 返回 `UserDetails` 结果对象。

在响应式安全配置中，不覆写 `configure()` 方法，而是声明一个 `ReactiveUserDetailsService` 类型的 **bean**。`ReactiveUserDetailsService` 是与 `UserDetailsService` 等价的响应式服务。和 `UserDetailsService` 类似，`ReactiveUserDetailsService` 只需要实现一个方法。更明确地说，`findByUsername()` 方法要返回一个 `Mono<UserDetails>`，而不是原始的 `UserDetails` 对象。

在下面的示例中，`ReactiveUserDetailsService` 声明为使用给定的 `UserRepository`，它被认为是一个响应式的 `Spring Data Repository`（我们将在下一章中详细讨论）：

```
@Service
@Bean
public ReactiveUserDetailsService userDetailsService(
    UserRepository userRepo) {
return new ReactiveUserDetailsService() {
@Override
public Mono<UserDetails> findByUsername(String username) {
return userRepo.findByUsername(username)
.map(user -> {
return user.toUserDetails();
});
}
};
}
```

这里，根据需要返回 `Mono<UserDetails>`，但是 `UserRepository.findByUsername()` 方法返回一个 `Mono<User>`。因为它是 **Mono**，您可以进行链式操作。例如通过 `map()` 操作把 `Mono<User>` 映射成 `Mono<UserDetails>`。



在本例中，`map()` 操作使用 `lambda` 表达式的形式，通过 `User` 对象上的 `toUserDetails()` 方法，发布成 `Mono`。把 `User` 对象转换成 `UserDetails` 对象。因此，`.map()` 操作返回一个 `Mono<UserDetails>`，这正是 `ReactiveUserDetailsService.findByUsername()` 所要的。

## 12.6 总结

- Spring WebFlux 提供了一个响应式 web 框架，其编程模型类似 Spring MVC，甚至共享许多相同的注解。
- Spring 还提供了一个函数式编程模型作为 Spring WebFlux 的替代方案。
- 响应式控制器可使用 WebClient 进行测试。
- 在客户端，Spring 提供了 WebClient，它是响应式的，对 RestTemplate 的模拟。
- 为了保护 web 应用程序的安全，尽管 WebFlux 对底层机制有一些重要影响，Spring Security 5 支持响应式编程的安全配置。且与非反应模式的 Spring MVC 应用程序安全配置没有显著的区别。

## 第 13 章 响应式持久化数据

本章内容

- 响应式持久化关系型数据库 R2DBC
- 编写适用于 Cassandra 和 MongoDB 的响应式 Repository
- 测试响应式 Repository

如果说我们从科幻小说中学到了什么，那就是如果想改变过去，所需要做的只是进行一次时间旅行。这在《回到未来》中有过，在《星际迷航》中也有好几集，《复仇者联盟4：终局之战》以及《斯蒂芬·金作品系列：11 22 63》都起过作用。（好吧，也许最后一个并没有变得更好，但您应该明白了。）

在这一章中，我们将回到第 3 章和第 4 章，重温我们为关系数据库、MongoDB 和 Cassandra 创建的 Repository。这一次，我们要利用 Spring Data 的一些响应式 Repository 支持改进它们，我们能够以非阻塞方式使用这些 Repository。

让我们先看看 Spring Data R2DBC，它是 Spring Data JDBC 持久化到关系数据库的一种响应式替代方案。

## 13.1 使用 R2DBC

响应式关系型数据库连接（通常称为 **R2DBC**）是一种使用响应类型处理关系数据新选择。它实际上是一个 **JDBC** 的响应式替代方案，支持针对传统关系型数据库，如 **MySQL**、**PostgreSQL**、**H2** 和 **Oracle** 的非阻塞持久化。因为它是建立在响应式基础上的，与 **JDBC** 有很大不同，是一个独立的规范，与 **Java SE** 无关。

**Spring Data R2DBC** 是 **Spring Data** 的一个子项目。它支持 **R2DBC**，与我们在第 3 章中看到的 **Spring Data JDBC** 非常相似。与 **Spring Data JDBC** 不同的是，**Spring Data R2DBC** 并不要求严格遵守领域驱动设计概念。事实上，您很快就会看到，使用 **Spring Data R2DBC** 通过聚合根持久化数据，需要比使用 **Spring Data JDBC** 要多做一些工作。

要使用 **Spring Data R2DBC**，您需要在项目的构建中添加一个 **starter** 依赖项。对于 **Maven** 构建的项目，依赖项如下所示：

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-r2dbc</artifactId>
</dependency>
```

或者，如果您使用 **Initializr** 配置工程，请在创建项目时勾选“**Spring Data R2DBC**”选择框。

您还需要一个关系数据库以便进行数据持久化，以及相应的 **R2DBC** 驱动。我们将使用内存数据库 **H2**。因此，我们需要增加两个依赖项：**H2** 数据库库和 **H2 R2DBC** 驱动。依赖项如下所示：

```
<dependency>
  <groupId>com.h2database</groupId>
  <artifactId>h2</artifactId>
  <scope>runtime</scope>
</dependency>
<dependency>
  <groupId>io.r2dbc</groupId>
  <artifactId>r2dbc-h2</artifactId>
  <scope>runtime</scope>
</dependency>
```

如果您使用不同的数据库，则需要配置相关依赖项以添加相应的 **R2DBC** 驱动程序。

现在，依赖项已经就位，让我们看看 **Spring Data R2DBC** 是如何工作的。让我们从定义实体类开始。

### 13.1.1 为 R2DBC 定义实体

为了了解 Spring Data R2DBC，我们将重新创建 Taco Cloud 应用程序的持久化层。我们只关保存订单数据，包括为 TacoOrder、Taco 和 Ingredient 创建领域实体，以及每个实体对应的 Repository。

我们要创建的第一个实体类是 Ingredient 类。看起来有点像清单 13.1 所示：

清单 13.1 使用 R2DBC 持久化的 Ingredient 实体类。

```
package tacos;

import org.springframework.data.annotation.Id;
import lombok.Data;
import lombok.EqualsAndHashCode;
import lombok.NoArgsConstructor;
import lombok.NonNull;
import lombok.RequiredArgsConstructor;

@Data
@NoArgsConstructor
@RequiredArgsConstructor
@EqualsAndHashCode(exclude = "id")
public class Ingredient {

    @Id
    private Long id;

    private @NonNull String slug;

    private @NonNull String name;
    private @NonNull Type type;

    public static enum Type {
        WRAP, PROTEIN, VEGGIES, CHEESE, SAUCE
    }
}
```

正如您所看到的，这和我们以前创建过的 Ingredient 类没有太大的不同。有两个值得注意的区别：

- Spring Data R2DBC 要求属性具有 setter 方法。因此，不能定义大多数属性为 Final 属性，它们必须是非 Final 属性。但要帮助 Lombok 创建一个带参构造函数，我们给大多数属性加了 @NonNull 注解。还有 @RequiredArgsConstructor 注解，这将使 Lombok 生成的构造函数中包含这些属性。
- 通过 Spring Data R2DBC Repository 保存对象时，如果对象的 ID 属性为非 null，将其视为更新。对于 Ingredient 对象，id 属性以前是字符串类型，并在创建时指定。但这样会导致 Spring Data R2DBC 报错。因此，这里我们将该字符串 ID 转为用属性 slug 存储，这是一个 Ingredient 的伪 id。我们使用 Long 类型的 id 属性，该属性的值将由数据库自动生成。

对应的数据库表在 schema.sql 中定义：

```
create table Ingredient (
    id identity,
    slug varchar(4) not null,
    name varchar(25) not null,
    type varchar(10) not null
);
```

Taco 实体类也非常类似于它的 Spring Data JDBC 版本，如清单 13.2 所示。

清单 13.2 使用 R2DBC 持久化的 Taco 实体类。

```
package tacos;

import java.util.HashSet;
import java.util.Set;
import org.springframework.data.annotation.Id;
import lombok.Data;
import lombok.NoArgsConstructor;
import lombok.NonNull;
import lombok.RequiredArgsConstructor;

@Data
@NoArgsConstructor
@RequiredArgsConstructor
public class Taco {

    @Id
    private Long id;

    private @NonNull String name;

    private Set<Long> ingredientIds = new HashSet<>();

    public void addIngredient(Ingredient ingredient) {
        ingredientIds.add(ingredient.getId());
    }

}
```

与 `Ingredient` 类一样，我们必须在实体字段上添加 `setter` 方法，因此字段上添加 `@NonNull` 注解，而不是 `final` 类型。

但这里特别有趣的是，我们没有定义集合类型的 `Ingredient` 属性。`Taco` 实体中用一个 `Set<Long>` 属性，引用了所有 `Ingredient` 的ID。使用 `Set` 而不是 `List`，是要保证唯一性。但是为什么我们必须使用 `Set<Long>` 而不是 `Set<Ingredient>` 呢？

与其他 `Spring Data` 项目不同，`Spring Data R2DBC` 目前不支持直接的实体间关联（至少目前不行）。作为一个相对较新的项目，`Spring Data R2DBC` 仍在努力解决，以非阻塞方式处理关系型数据的一些挑战。这可能会在 `Spring Data R2DBC` 的未来版本中做出改进。

在那之前，我们不能让 `Taco` 直接引用一组 `Ingredient` 并进行持久性。在处理实体关联时，我们有几个选择：

- 定义实体中只引用相关对象的ID。在这种情况下，则必须在数据库表中的相应列中使用数组类型。`H2` 和 `PostgreSQL` 是两个支持数组列的数据库，但很多其他数据库没有支持。此外，即使数据库支持数组列，也可能无法将属性定义为引用其他表的外键，来保证关联参照完整性。
- 定义实体及其对应的表，使它们彼此完全匹配。对于集合，这意味着引用的对象将有一个列映射回引用表。例如，`Taco` 对象的表需要有一个指向 `TacoOrder` 的列。
- 将引用的实体序列化为JSON，并将JSON存储在一个大的 `VARCHAR` 列中。如果不需要查询到被引用的对象，这种方法尤其有效。但是，由于 `VARCHAR` 列的长度有限制，它对JSON序列化对象的大小有潜在的限制。此外，没办法保证引用的完整性，因为引用的对象被存储为简单的字符串值了（这可以包含任何内容）。

尽管这些选择都不理想，但在权衡了各种选项后，我们将为 `Taco` 选择第一个选项。`Taco` 类有一个 `Set<Long>`，该集合引用一个或多个 `Ingredient` 的ID。这意味着相应的表必须有一个数组列来存储这些ID。对于 `H2` 数据库，`Taco` 表的定义如下：

```
create table Taco (
    id identity,
    name varchar(50) not null,
    ingredient_ids array
);
```

`ingredient_ids` 列上使用的数组类型特定于 H2。对于 PostgreSQL，这列可能定义为 `integer[]`。若您选择的是其他数据库类型，请参阅您相关的数据库文档，查找如何定义数组列。请注意，并非所有数据库实现都支持数组列，因此您可能需要为关系建模选择其他选项之一。

最后，如清单 13.3 所示，`TacoOrder` 类使用的许多东西我们已经介绍过了。

清单 13.3 使用 R2DBC 持久化的 `TacoOrder` 实体类。

```
package tacos;

import java.util.LinkedHashSet;
import java.util.Set;
import org.springframework.data.annotation.Id;
import lombok.Data;

@Data
public class TacoOrder {

    @Id
    private Long id;

    private String deliveryName;
    private String deliveryStreet;
    private String deliveryCity;
    private String deliveryState;
    private String deliveryZip;
    private String ccNumber;
    private String ccExpiration;
    private String ccCVV;

    private Set<Long> tacoIds = new LinkedHashSet<>();

    private List<Taco> tacos = new ArrayList<>();
    public void addTaco(Taco taco) {
        this.tacos.add(taco);
    }

}
```

正如您所看到的，除了拥有更多的属性之外，类 `TacoOrder` 也遵循和 `Taco` 类同样的规则。它通过一个 `Set<Long>` 引用其 `Taco` 对象。尽管如此，稍后，我们将看到如何将完整的 `Taco` 对象放入 `TacoOrder`，即使 Spring Data R2DBC 不支持这种方式。

`Taco_Order` 表的数据库结构如下所示：

```
create table Taco_Order (  
    id identity,  
    delivery_name varchar(50) not null,  
    delivery_street varchar(50) not null,  
    delivery_city varchar(50) not null,  
    delivery_state varchar(2) not null,  
    delivery_zip varchar(10) not null,  
    cc_number varchar(16) not null,  
    cc_expiration varchar(5) not null,  
    cc_cvv varchar(3) not null,  
    taco_ids array  
);
```

就像 **Taco** 表一样，**TacoOrder** 表使用数组列 **taco\_ids** 引用 **taco** 对象。再说一次，这个模式只用于 **H2** 数据库；请查阅相关数据库文档，以了解有关创建数组列的支持的详细信息。

通常，生产应用程序已经通过其他方式和方法定义了数据库 **schema**，除了测试之外，这样的脚本方式是不可取的。因此，这个 **bean** 是在配置中定义的，它仅在运行自动测试时加载，在应用程序运行时不可用。我们将在定义服务之后，看一个测试 **R2DBC Repository** 的示例。

此外，请注意，这个 **bean** 只使用类路径根目录中的“**schema.sql**”文件（在项目 `src/main/resources` 路径下）。如果您希望其他 **SQL** 脚本也作为数据库初始化的一部分，可在调用 `populator.addPopulators()` 时添加更多 **ResourceDatabasePopulator** 对象。

现在我们已经定义了实体及其对应的数据库 **schema**，让我们创建 **Repository**，通过它我们将保存和获取 **Taco** 数据。



### 13.1.2 定义响应式 Repository

在第 3 章和第 4 章中，我们将 Repository 定义为扩展 Spring Data 的 CrudRepository 接口。但那个基本 Repository 接口处理的是单个对象和可遍历集合。相反，我们希望响应式 Repository 能够处理 Mono 和 Flux 对象。

这就是为什么 Spring 提供了 ReactiveCrudRepository 来定义响应式 Repository。ReactiveCrudRepository 的操作与 CrudRepository 非常相似。要创建响应式 Repository，需扩展 ReactiveCrudRepository 接口，例如：

```
package tacos.data;

import org.springframework.data.repository.reactive.ReactiveCrudRepository;

import tacos.TacoOrder;

public interface OrderRepository
    extends ReactiveCrudRepository<TacoOrder, Long> {
}
```

从表面上看，这个 OrderRepository 与我们在第 3 章和第 4 章中定义的 OrderRepository 之间的唯一区别是，它扩展了 ReactiveCrudRepository，而不是 CrudRepository。但是最显著不同其实是，这个方法返回 Mono 和 Flux 类型，而不是单个 TacoOrder 或 Iterable<TacoOrder>。举两个例子：findById() 方法返回

Mono<TacoOrder>，findAll() 返回 Flux<TacoOrder>。

要了解此响应式 Repository 在实际如何工作，假设您希望获取所有 TacoOrder 对象，并将其名称打印到标准输出。这种情况下，您可以编写一些代码，如清单 13.4 所示。

清单 13.4 调用响应式 Repository 方法。

```
@Autowired
OrderRepository orderRepo;

...

orderRepository.findAll()
    .doOnNext(order -> {
        System.out.println(
            "Deliver to: " + order.getDeliveryName());
    })
    .subscribe();
```

在这里，对 findAll() 的调用返回一个 Flux<TacoOrder>。我们在其上添加了一个 doOnNext() 方法以打印收货人名称。最后，调用 subscribe() 启动了数据传输。

在第 3 章的 Spring Data JDBC 示例中，TacoOrder 是 Taco 的聚合根。因此，Taco 对象作为 TacoOrder 的一部分被持久化了，而没有必要定义一个专门用于 Taco 持久性的 Repository。但是 Spring Data R2DBC 不能以这种方式支持聚合，因此我们需要一个 TacoRepository，通过它持久化 Taco 对象。这个 Repository 请参见清单 13.5。

清单 13.5 使用响应式 Repository 持久化 Taco 对象。

```
package tacos.data;

import org.springframework.data.repository.reactive.ReactiveCrudRepository;
import tacos.Taco;

public interface TacoRepository
    extends ReactiveCrudRepository<Taco, Long> {
}
```

如您所见，`TacoRepository` 与 `OrderRepository` 没有太大区别。它扩展 `ReactiveCrudepository`，在使用 `Taco` 持久化时为我们提供响应式类型。这里没有太多可说的。

另一方面，`IngredientRepository` 稍微有趣一些，如清单 13.6 所示。

清单 13.6 使用响应式 **Repository** 持久化 **Ingredient** 对象。

```
package tacos.data;

import org.springframework.data.repository.reactive.ReactiveCrudRepository;

import reactor.core.publisher.Mono;
import tacos.Ingredient;

public interface IngredientRepository
    extends ReactiveCrudRepository<Ingredient, Long> {

    Mono<Ingredient> findBySlug(String slug);

}
```

与我们的其他两个响应式 `Repository` 一样，`IngredientRepository` 扩展了 `ReactiveCrudRepository`。但是因为我们可能需要一种基于 `slug` 值来查找 `Ingredient` 对象的方法，`IngredientRepository` 包含一个 `findBySlug()` 方法，该方法返回一个 `Mono<Ingredient>`。

现在让我们看看如何编写测试类，来验证我们的 `Repository` 是否工作。

### 13.1.3 测试 R2DBC Repository

Spring Data R2DBC 支持为 R2DBC Repository 编写集成测试类。具体来说，`@DataR2dbcTest` 注解放在测试类上时，会导致 Spring 使用生成的 Spring Data R2DBC Repository 作为 bean 来创建应用程序上下文，并注入到测试类中。就像我们在上一章中使用的 `StepVerifier`，这使我们能够针对我们创建的所有 Repository 编写自动化测试类。

为了简洁起见，我们将只关注一个测试类：`IngredientRepositoryTest`。这将测试 `IngredientRepository`，验证它是否可以保存 `Ingredient` 对象、获取单一 `Ingredient`，并获取所有保存的 `Ingredient` 对象。清单 13.7 显示了这个测试类。

清单 13.7 测试 Spring Data R2DBC Repository。

```
package tacos.data;

import static org.assertj.core.api.Assertions.assertThat;

import java.util.ArrayList;

import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.data.r2dbc.DataR2dbcTest;

import reactor.core.publisher.Flux;
import reactor.test.StepVerifier;
import tacos.Ingredient;
import tacos.Ingredient.Type;

@DataR2dbcTest
public class IngredientRepositoryTest {

    @Autowired
    IngredientRepository ingredientRepo;

    @BeforeEach
    public void setup() {
        Flux<Ingredient> deleteAndInsert = ingredientRepo.deleteAll()
            .thenMany(ingredientRepo.saveAll(
                Flux.just(
                    new Ingredient("FLT0", "Flour Tortilla", Type.WRAP),
                    new Ingredient("GRBF", "Ground Beef", Type.PROTEIN),
                    new Ingredient("CHED", "Cheddar Cheese", Type.CHEESE)
                )))
            .thenMany(ingredientRepo.saveAll(
                Flux.just(
                    new Ingredient("FLTO", "Flour Tortilla", Type.WRAP),
                    new Ingredient("GRBF", "Ground Beef", Type.PROTEIN),
                    new Ingredient("CHED", "Cheddar Cheese", Type.CHEESE)
                )));

        StepVerifier.create(deleteAndInsert)
            .expectNextCount(3)
            .verifyComplete();
    }

    @Test
    public void shouldSaveAndFetchIngredients() {

        StepVerifier.create(ingredientRepo.findAll())
            .recordWith(ArrayList::new)
            .thenConsumeWhile(x -> true)
            .consumeRecordedWith(ingredients -> {
                assertThat(ingredients).hasSize(3);
                assertThat(ingredients).contains(
                    new Ingredient("FLT0", "Flour Tortilla", Type.WRAP));
                assertThat(ingredients).contains(
                    new Ingredient("GRBF", "Ground Beef", Type.PROTEIN));
                assertThat(ingredients).contains(
                    new Ingredient("CHED", "Cheddar Cheese", Type.CHEESE));
            })
            .verifyComplete();

        StepVerifier.create(ingredientRepo.findBySlug("FLT0"))
            .assertNext(ingredient -> {
                ingredient.equals(new Ingredient("FLT0", "Flour Tortilla", Type.WRAP));
            })
            .verifyComplete();
    }
}
```

`shouldSaveAndFetchIngredients()` 方法首先创建 `Flux<Ingredient>` 对象。从这个 `Flux` 中，它使用 `flatMap()` 操作通过注入的 `IngredientRepository` 上的 `save()` 方法保存每一个 `Ingredient`。调用 `subscribe()` 将打开通过 `Flux` 的数据流，导致 `Ingredient` 对象被保存。

接下来，从 `Repository` 的 `findBySlug()` 方法返回的 `Mono<Ingredient>`，创建出了一个 `StepVerifier`，这。 `Mono<Ingredient>` 中应该有一个 `Ingredient` 对象。也是 `assertNext()` 方法验证的内容，这是一个 `slug` 值为“FLTO”的 `Ingredient` 对象。最后，验证了 `Mono` 的完整性。

最后，根据 `Repository` 的 `findAll()` 方法返回的 `Flux<Ingredient>`，创建出了另一个 `StepVerifier`。一次一个，它断言，从该 `Flux` 流出的 `Ingredient` 与最初测试开始时保存的三个 `Ingredient` 对象相匹配。与其他 `StepVerifier` 一样，对 `verifyComplete()` 的调用验证 `Mono` 是完整的，没有更多的 `Ingredient` 对象流出了。

虽然我们只专注于测试 `IngredientRepository`，但同样的技术也可以应用于测试任何 `Spring Data R2DBC` 的 `Repository`。

到目前为止，一切顺利。我们现在已经定义了领域类型及其各自的 `Repository`。我们已经编写了一个测试类来验证它们是否有效。如果您愿意，我们可以就这样使用。但是这些 `Repository` 使 `TacoOrder` 的持久性变得不方便，因为您必须首先创建和持久化属于该 `TacoOrder` 的 `Taco` 对象，然后持久化属于该 `TacoOrder` 对象。当您查看 `TacoOrder` 时，您只会得到 `Taco ID` 而不是完整的 `Taco` 对象集合。

如果我们能够将 `TacoOrder` 作为聚合根，与拥有的所有 `Taco` 对象一起持久化，那就太好了。那样，如果我们找到一个 `TacoOrder`，就能得到完整的 `Taco` 对象（而不仅仅是 `ID`）。让我们定义一个 `Service` 级别类，该类位于 `OrderRepository` 和 `TacoRepository` 之上，以模拟第 3 章的 `OrderRepository` 的行为。

### 13.1.4 定义 OrderRepository 聚合根服务

将 TacoOrder 和 Taco 对象持久化在一起的第一步是使 TacoOrder 成为聚合根，将在 TacoOrder 类中添加一个 Taco 集合属性。如清单 13.8 所示。

清单 13.8 将 Taco 集合添加到 TacoOrder 中。

```
@Data
public class TacoOrder {

    ...

    @Transient
    private transient List<Taco> tacos = new ArrayList<>();

    public void addTaco(Taco taco) {
        this.tacos.add(taco);
        if (taco.getId() != null) {
            this.tacoIds.add(taco.getId());
        }
    }
}
```

除了将名为 `tacos` 的新 `List<Taco>` 属性添加到 `TacoOrder` 类之外，`addTaco()` 方法现在将给定的 `Taco` 添加到该列表中（以及将其 `id` 添加到 `tacoIds` 中）。

但是，请注意 `tacos` 属性加了 `@Transient` 注解（以及 Java 关键字 `transient`），这表明 Spring Data R2DBC 不应该持久化此字段。如果没有 `@Transient` 注释，Spring Data R2DBC 将尝试持久化，会由于不支持而导致错误。

保存 `TacoOrder` 时，只有 `tacoIds` 属性会写入数据库和 `tacos` 属性将被忽略。即使如此，至少现在 `TacoOrder` 有了存放 `Taco` 的地方。这对于保存 `TacoOrder` 时，同时保存 `Taco` 对象，以及获取 `TacoOrder` 时同时读入 `Taco` 对象，都方便很多。

现在，我们可以创建一个服务 `bean` 来保存和读取 `TacoOrder` 对象及其中的 `Taco` 对象。让我们从保存 `TacoOrder` 开始，清单 13.9 中定义的 `TacoOrderAggregateService` 类有一个 `save()` 方法正是这样做的。

清单 13.9 将 `Tacoorder` 和 `Taco` 聚合保存。

```

package tacos.web.api;

import java.util.ArrayList;
import java.util.List;

import org.springframework.stereotype.Service;

import lombok.RequiredArgsConstructor;
import reactor.core.publisher.Mono;
import tacos.Taco;
import tacos.TacoOrder;
import tacos.data.OrderRepository;
import tacos.data.TacoRepository;

@Service
@RequiredArgsConstructor
public class TacoOrderAggregateService {

    private final TacoRepository tacoRepo;
    private final OrderRepository orderRepo;

    public Mono<TacoOrder> save(TacoOrder tacoOrder) {
        return Mono.just(tacoOrder)
            .flatMap(order -> {
                List<Taco> tacos = order.getTacos();
                order.setTacos(new ArrayList<>());
                return tacoRepo.saveAll(tacos)
                    .map(taco -> {
                        order.addTaco(taco);
                        return order;
                    }).last();
            })
            .flatMap(orderRepo::save);
    }
}

```

虽然清单 13.9 中的行不多，但 `save()` 方法中有很多内容需要解释。首先，包装作为参数接收的 `TacoOrder` 在 `Mono` 中使用 `Mono.just()` 方法。这使我们能够将其作为一种响应性类型，在 `save()` 方法所有地方直接使用。

接下来我们要做的是将一个 `flatMap()` 应用到我们刚刚创建的 `Mono<TacoOrder>` 上。`map()` 和 `flatMap()` 是对通过 `Mono` 或 `Flux` 的数据对象进行转换的方法。由于我们在转换过程最终结果是 `Mono<TacoOrder>`，`flatMap()` 操作确保我们继续使用 `Mono<TacoOrder>`，而如果我们改为使用 `map()`，`Mono` 映射后会变成 `Mono<Mono<TacoOrder>>`。

映射的目的是保存这些 `Taco` 对象时，确保 `TacoOrder` 的 `Taco` 对象都最终只有 ID。一个新的 `TacoOrder` 对象，其中的每个 `Taco` 对象的 ID 可能为 `null`，直到 `Taco` 对象被保存之后，我们才知道 ID 的值。

从 `TacoOrder` 获取 `List<Taco>` 后，我们将在保存 `Taco` 对象时使用该列表，我们将 `tacos` 属性重置为空列表。我们将用新的 `Taco` 对象重建列表，使用保存后已分配的 ID 值。

对注入的 `TacoRepository` 调用 `saveAll()` 方法将保存所有 `Taco` 对象。`saveAll()` 方法返回一个 `Flux<Taco>`，然后通过 `map()` 循环处理。在这种情况下，转换操作是次要的，因为每个 `Taco` 对象会被添加回 `TacoOrder`。但是为了确保结果 `Flux` 是一个 `TacoOrder` 而不是一个 `Taco`，最后，映射操作返回的是 `TacoOrder`，而不是 `Taco`。对 `last()` 的调用确保映射操作的结果不会有重复的 `TacoOrder` 对象（每个 `Taco` 对应一个）。

此时，应保存所有 `Taco` 对象，然后将其推回父对象 `TacoOrder` 对象，以及它们新分配的 ID。剩下的就是保存 `TacoOrder`，这就是最后的 `flatMap()` 调用所做的。再次使用 `flatMap()` 确保 `OrderRepository.save()` 返回的是 `Mono<TacoOrder>`，没有被包装在另一个 `Mono` 中，我们希望 `save()` 方法返回 `Mono<TacoOrder>` 而不是 `Mono<Mono<TacoOrder>>`。

现在让我们来看一个方法，该方法将通过 `TacoOrder` 的 ID 读取 `TacoOrder`，并重新构造所有 `Taco` 对象。清单 13.10 显示了这个新的 `findById()` 方法。

清单 13.10 将 `TacoOrders` 和 `Tacos` 聚合读取。

```
public Mono<TacoOrder> findById(Long id) {
    return orderRepo
        .findById(id)
        .flatMap(order -> {
            return tacoRepo.findAllById(order.getTacoIds())
                .map(taco -> {
                    order.addTaco(taco);
                    return order;
                }).last();
        });
}
```

新 `findById()` 方法比 `save()` 短一些。但些方法中仍然有很多需要说的点。

方法中做的第一件事是通过调用 `OrderRepository` 的 `findById()` 获取 `TacoOrder`。方法返回一个 `Mono<TacoOrder>`，然后通过扁平化映射对其进行变换，从只有 `Taco ID` 的 `TacoOrder`，变成包含完整 `Taco` 对象的 `TacoOrder`。

提供给 `flatMap()` 方法的 `lambda` 调用 `TacoRepository.findAllById()` 方法，获取 `TacoIds` 中引用的所有 `Taco` 对象。这会产生一个 `Flux<Taco>`，它通过 `map()` 循环处理，将每个 `Taco` 添加到父对象 `TacoOrder` 中，就像我们在 `save()` 方法中所做的那样，使用 `saveAll()` 保存所有 `Taco` 对象。

同样，`map()` 操作更多地被用作对 `Taco` 对象进行迭代的一种方法，而不是作为一种转变。但是每次给 `map()` 的 `lambda` 都会返回父对象 `TacoOrder`。我们最终得到的是 `Flux<TacoOrder>`，而不是 `Flux<Taco>`。对 `last()` 的调用使用最后一项，并返回一个 `Mono<TacoOrder>`，这是 `findById()` 方法的返回值。

如果您还没有习惯响应式编程方式，`save()` 和 `findById()` 方法中的代码可能让您感到迷惑。响应式编程需要一种不同的思维方式，而且有时令人困惑，但当您的响应式编程技巧提高以后，就会发现它非常优雅。

就像 `TacoOrderAggregateService` 中的代码，这可能让您感觉迷惑。所以编写测试以确保其正常工作是一个好主意。该测试还将作为 `TacoOrderAggregateService` 如何运行的示例。清单 13.11 显示了对 `TacoOrderAggregateService` 的测试类。

清单 13.11 测试 `TacoOrderAggregateService`。



```

package tacos.web.api;

import static org.assertj.core.api.Assertions.assertThat;

import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.data.r2dbc.DataR2dbcTest;
import org.springframework.test.annotation.DirtiesContext;

import reactor.test.StepVerifier;
import tacos.Taco;
import tacos.TacoOrder;
import tacos.data.OrderRepository;
import tacos.data.TacoRepository;

@DataR2dbcTest
@DirtiesContext
public class TacoOrderAggregateServiceTests {

    @Autowired
    TacoRepository tacoRepo;

    @Autowired
    OrderRepository orderRepo;

    TacoOrderAggregateService service;

    @BeforeEach
    public void setup() {
        this.service = new TacoOrderAggregateService(tacoRepo, orderRepo);
    }

    @Test
    public void shouldSaveAndFetchOrders() {
        TacoOrder newOrder = new TacoOrder();
        newOrder.setDeliveryName("Test Customer");
        newOrder.setDeliveryStreet("1234 North Street");
        newOrder.setDeliveryCity("Notrees");
        newOrder.setDeliveryState("TX");
        newOrder.setDeliveryZip("79759");
        newOrder.setCcNumber("4111111111111111");
        newOrder.setCcExpiration("12/24");
        newOrder.setCcCVV("123");

        newOrder.addTaco(new Taco("Test Taco One"));
        newOrder.addTaco(new Taco("Test Taco Two"));

        StepVerifier.create(service.save(newOrder))
            .assertNext(this::assertOrder)
            .verifyComplete();
        StepVerifier.create(service.findById(1L))
            .assertNext(this::assertOrder)
            .verifyComplete();
    }

    private void assertOrder(TacoOrder savedOrder) {
        assertThat(savedOrder.getId()).isEqualTo(1L);
        assertThat(savedOrder.getDeliveryName()).isEqualTo("Test Customer");
        assertThat(savedOrder.getDeliveryName()).isEqualTo("Test Customer");
        assertThat(savedOrder.getDeliveryStreet()).isEqualTo("1234 North Street");
        assertThat(savedOrder.getDeliveryCity()).isEqualTo("Notrees");
        assertThat(savedOrder.getDeliveryState()).isEqualTo("TX");
        assertThat(savedOrder.getDeliveryZip()).isEqualTo("79759");
        assertThat(savedOrder.getCcNumber()).isEqualTo("4111111111111111");
        assertThat(savedOrder.getCcExpiration()).isEqualTo("12/24");
    }
}

```

```

        assertThat(savedOrder.getCcCVV()).isEqualTo("123");
        assertThat(savedOrder.getTacoIds()).hasSize(2);
        assertThat(savedOrder.getTacos().get(0).getId()).isEqualTo(1L);
        assertThat(savedOrder.getTacos().get(0).getName())
            .isEqualTo("Test Taco One");
        assertThat(savedOrder.getTacos().get(1).getId()).isEqualTo(2L);
        assertThat(savedOrder.getTacos().get(1).getName())
            .isEqualTo("Test Taco Two");
    }
}

```

清单 13.11 中有很多行，但其中大部分是 `assertOrder()` 方法中的断言。此测试类，我们将重点关注其他部分。

测试类用了 `@DataR2dbcTest` 注解，以使 Spring 创建的应用程序上下文中包含相关 `Repository` 的 `bean`。`@DataR2dbcTest` 查找带有注解 `@SpringBootTest` 的配置类，来定义 Spring 应用程序上下文。在单模块项目中，引导类用 `@SpringBootApplication` 注解可以达到这个目的（它本身会有 `@SpringBootTest` 注解）。但在我们的多模块项目中，测试类与引导类不在同一个项目中，因此我们需要一个简单的配置：

```

package tacos;

import org.springframework.boot.SpringBootConfiguration;
import org.springframework.boot.autoconfigure.EnableAutoConfiguration;

@SpringBootConfiguration
@EnableAutoConfiguration
public class TestConfig {

}

```

这不仅满足了 `@SpringBootTest` 注解类的需要，而且还支持自动配置，确保 `Repository` 被创建（还有许多其他功能）。

就其本身而言，`TacoOrderAggregateServiceTests` 应该可以通过测试。但是在 IDE 环境中，可能在测试运行之间共享 JVM 和 Spring 应用程序上下文，与其他测试一起运行此测试，可能会导致将冲突数据写入内存中的 H2 数据库。这个 `@DirtiesContext` 注解来确保重置 Spring 应用程序上下文。在测试运行时，每次运行都会产生一个新的空 H2 数据库。

`setup()` 方法使用注入测试类的 `TacoRepository` 和 `OrderRepository` 对象，创建 `TacoOrderAggregateService` 实例。这个 `TacoOrderAggregateService` 被分配给一个实例变量，以便测试方法可以使用它。

现在我们终于准备好测试我们的聚合服务了。前几行 `shouldSaveAndFetchOrders()` 构建一个 `TacoOrder` 对象，并加入了几个 `Taco` 对象。然后 `TacoOrder` 对象通过 `TacoOrderAggregateService` 的 `save()` 方法保存，返回 `Mono<TacoOrder>`，表示保存的顺序。使用 `StepVerifier`，我们断言返回的 `Mono` 中的 `TacoOrder` 符合我们的期望，包括它包含的子 `Taco` 对象。

接下来，我们调用服务的 `findById()` 方法，该方法返回一个 `Mono<TacoOrder>`。像调用 `save()` 方法那样，一个 `StepVerifier` 用于遍历返回 `Mono` 中的每个 `TacoOrder`（应该只有一个）并声称它符合我们的期望。

在这两种 `StepVerifier` 情况下，对 `verifyComplete()` 的调用确保 `Mono` 中没有更多对象，并确认 `Mono` 已完成处理。

值得注意的是，尽管我们可以应用类似的聚合操作，来确保 `Taco` 对象总是包含完整的 `Ingredient` 对象。但我选择不这样做，`Ingredient` 是它自己的聚合根，可能被多个 `Taco` 对象引用。因此，每个 `Taco` 将只携带 `Set<Long>` 指向 `Ingredient` ID，然后通过 `IngredientRepository` 单独查找。

尽管聚合实体可能需要更多的工作，但 **Spring Data R2DBC** 提供了以响应式方式处理关系数据的方式。但这并不是 **Spring** 提供的唯一的响应式持久化数据的选择。让我们看看如何使用 **Spring Data Repostotry** 响应式的使用 **MongoDB**。

## 13.2 使用 MongoDB 响应式保存文档

在第 4 章中，我们使用 Spring Data MongoDB 定义了基于 MongoDB 文档型数据库的文档数据持久化。在本节中，我们将使用 Spring Data 对 MongoDB 的响应式支持。

首先，您需要使用 Spring Data Reactive MongoDB starter 创建一个项目。实际上，这是使用 Initializr 创建项目时要选择的复选框的名称。或者，您可以使用以下依赖项手动将其添加到 Maven 构建中：

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-mongodb-reactive</artifactId>
</dependency>
```

在第 4 章中，我们还借助于 Flapdoodle 嵌入式 MongoDB 数据库进行测试。不幸的是，以响应式持久化时，Flapdoodle 的表现不是很好。在运行测试时，您需要运行一个实际的 Mongo 数据库，并侦听端口 27017。

现在，我们已经准备好开始为响应式 MongoDB 持久化编写代码了。我们先从创建文档类型开始。

### 13.2.1 定义文档类型

与前面一样，我们需要定义应用程序实体的相关类。我们将使用 Spring Data MongoDB 的 `@Document` 注解，就像我们在第 4 章中所做的那样，表明它们是要存储在 MongoDB 中的文档。让我们先从 `Ingredient` 类开始，如清单 13.12 所示。

清单 13.12 为 Mongo 持久化的 `Ingredient` 类。

```
package tacos;

import org.springframework.data.annotation.Id;
import org.springframework.data.mongodb.core.mapping.Document;

import lombok.AccessLevel;
import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;

@Data
@AllArgsConstructor
@NoArgsConstructor(access=AccessLevel.PRIVATE, force=true)
@Document
public class Ingredient {

    @Id
    private String id;
    private String name;
    private Type type;

    public static enum Type {
        WRAP, PROTEIN, VEGGIES, CHEESE, SAUCE
    }

}
```

您可能敏锐的发现这个 `Ingredient` 类与我们在第 4 章中创建的类是相同的。事实上，MongoDB `@Document` 类确实完全一致，无论是通过响应式还是非响应式方式进行持久化。这意味着 `Taco` 和 `TacoOrder` 类也与我们在第 4 章中创建的是相同的。但是为了完整性，以便您不需要回到第 4 章，我们在这里列出。

清单 13.13 中显示了 `Taco` 类。

清单 13.13 为 Mongo 持久性添加注解的 `Taco` 类。

```

package tacos;

import java.util.ArrayList;
import java.util.Date;
import java.util.List;

import javax.validation.constraints.NotNull;
import javax.validation.constraints.Size;

import org.springframework.data.annotation.Id;
import org.springframework.data.mongodb.core.mapping.Document;
import org.springframework.data.rest.core.annotation.RestResource;

import lombok.Data;

@Data
@RestResource(rel = "tacos", path = "tacos")
@Document
public class Taco {

    @Id
    private String id;

    @NotNull
    @Size(min = 5, message = "Name must be at least 5 characters long")
    private String name;

    private Date createdAt = new Date();

    @Size(min=1, message="You must choose at least 1 ingredient")
    private List<Ingredient> ingredients = new ArrayList<>();

    public void addIngredient(Ingredient ingredient) {
        this.ingredients.add(ingredient);
    }
}

```

注意，与 `Ingredient` 不同，`Taco` 类没有用 `@Document` 注解。那是因为它不会保存为自己的文档，而是保存为 `TacoOrder` 聚合根的一部分。另一方面，由于 `TacoOrder` 是聚合根，因此它用 `@Document` 注解，如清单 13.14 所示。

清单 13.14 为 **Mongo** 持久化注解的 **TacoOrder** 类。

```

package tacos;

import java.io.Serializable;
import java.util.ArrayList;
import java.util.Date;
import java.util.List;

import org.springframework.data.annotation.Id;
import org.springframework.data.mongodb.core.mapping.Document;

import lombok.Data;

@Data
@Document
public class TacoOrder implements Serializable {
    private static final long serialVersionUID = 1L;

    @Id
    private String id;
    private Date placedAt = new Date();

    private User user;

    private String deliveryName;

    private String deliveryStreet;

    private String deliveryCity;

    private String deliveryState;

    private String deliveryZip;

    private String ccNumber;

    private String ccExpiration;

    private String ccCVV;

    private List<Taco> tacos = new ArrayList<>();

    public void addTaco(Taco taco) {
        this.tacos.add(taco);
    }
}

```

同样，对于响应式 **MongoDB Repository** 与非响应式 **MongoDB Repository**，文档类没有什么不同。正如您接下来将看到的，响应式 **MongoDB Repository** 本身与非响应性的 **MongoDB Repository** 也只有很小的区别。

### 13.2.2 定义响应式 MongoDB Repository

现在我们需要定义两个 **Repository**，一个用于 **TacoOrder** 聚合根，另一个用于 **Ingredient**。我们不需要 **Taco** 的 **Repository**，因为它是 **TacoOrder** 的一个属性。

您现在应该已经熟悉 **IngredientRepository** 接口：

```
package tacos.data;

import org.springframework.data.repository.reactive.ReactiveCrudRepository;
import org.springframework.web.bind.annotation.CrossOrigin;

import tacos.Ingredient;

@CrossOrigin(origins="*")
public interface IngredientRepository
    extends ReactiveCrudRepository<Ingredient, String> {
}
```

此 **IngredientRepository** 接口与我们在第 4 章中定义的略有不同，它扩展了 **ReactiveCrudRepository**，而不是扩展 **CrudRepository**。且与我们为 **Spring Data R2DBC** 持久化创建的 **Repository** 也不一样，不同之处在于它没有 **findBySlug()** 方法。

同样，**OrderRepository** 与我们在第 4 章中创建的几乎相同：

```
package tacos.data;

import org.springframework.data.domain.Pageable;
import org.springframework.data.repository.reactive.ReactiveCrudRepository;

import reactor.core.publisher.Flux;
import tacos.TacoOrder;
import tacos.User;

public interface OrderRepository
    extends ReactiveCrudRepository<TacoOrder, String> {

    Flux<TacoOrder> findByUserOrderByPlacedAtDesc(
        User user, Pageable pageable);

}
```

最后，响应式和非响应式 **MongoDB Repository** 之间的唯一区别是：它们是否扩展为 **ReactiveCrudRepository**。在选择扩展 **ReactiveCrudRepository** 时，这些 **Repository** 的客户端必须准备好处理响应类型，如 **Flux** 和 **Mono**。这一点在我们编写响应式测试时变得显而易见，这是我们下一步要做的。



### 13.2.3 测试响应式 MongoDB Repository

为 MongoDB Repository 编写测试的关键是使用 `@DataMongoTest`。此注解执行与 `@DataR2dbcTest` 类似的功能，这是我们在本章前面使用过的。它确保 Spring 应用程序上下文生成 Repository 相关 bean，并且可注入测试类。从那里，测试类可以使用这些注入的 Repository 来设置测试数据，并对其执行其他操作。

例如，考虑清单 13.15 中的 `IngredientRepositoryTest` 类测试 `IngredientRepository`，断言 `Ingredient` 对象可以写入数据库。

清单 13.15 测试响应式 Mongo Repository。

```

package tacos.data;

import static org.assertj.core.api.Assertions.assertThat;
import java.util.ArrayList;

import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.data.mongo.DataMongoTest;
import reactor.core.publisher.Flux;
import reactor.test.StepVerifier;
import tacos.Ingredient;
import tacos.Ingredient.Type;

@DataMongoTest
public class IngredientRepositoryTest {

    @Autowired
    IngredientRepository ingredientRepo;

    @BeforeEach
    public void setup() {
        Flux<Ingredient> deleteAndInsert = ingredientRepo.deleteAll()
            .thenMany(ingredientRepo.saveAll(
                Flux.just(
                    new Ingredient("FLT0", "Flour Tortilla", Type.WRAP),
                    new Ingredient("GRBF", "Ground Beef", Type.PROTEIN),
                    new Ingredient("CHED", "Cheddar Cheese", Type.CHEESE)
                )))
        StepVerifier.create(deleteAndInsert)
            .expectNextCount(3)
            .verifyComplete();
    }

    @Test
    public void shouldSaveAndFetchIngredients() {

        StepVerifier.create(ingredientRepo.findAll())
            .recordWith(ArrayList::new)
            .thenConsumeWhile(x -> true)
            .consumeRecordedWith(ingredients -> {
                assertThat(ingredients).hasSize(3);
                assertThat(ingredients).contains(
                    new Ingredient("FLT0", "Flour Tortilla", Type.WRAP));
                assertThat(ingredients).contains(
                    new Ingredient("GRBF", "Ground Beef", Type.PROTEIN));
                assertThat(ingredients).contains(
                    new Ingredient("CHED", "Cheddar Cheese", Type.CHEESE));
            })
            .verifyComplete();

        StepVerifier.create(ingredientRepo.findById("FLT0"))
            .assertNext(ingredient -> {
                ingredient.equals(new Ingredient("FLT0", "Flour Tortilla", Type.WRAP));
            });
    }
}

```

此测试与我们前面编写的基于 R2DBC 的 `Repository` 测试类略有不同。它首先将三个 `Ingredient` 对象写入数据库。然后使用两个 `StepVerifier` 验证 `Ingredient` 是否可以通过 `Repository` 读到，先获取所有 `Ingredient` 对象的集合，然后通过 ID 获取单个 `Ingredient`。

此外，与前面基于 R2DBC 的测试一样，`@DataMongoTest` 注解将查找添加了 `@SpringBootTestConfiguration` 注解的类，以创建应用程序上下文。前面创建的那个，这里也可以使用。

这里的独特之处在于，第一步 `StepVerifier` 将所有 `Ingredient` 对象收集到一个 `ArrayList`，然后断言 `ArrayList` 包含每个 `Ingredient`。`findAll()` 方法不能保证结果文档的顺序，这使得 `assertNext()` 或 `expectNext()` 容易失败。通过收集所有产生的 `Ingredient`，将对象放入列表中，我们可以断言该列表包含所有三个对象，而不管它们的顺序如何。

`OrderRepository` 的测试看起来很类似，如清单 13.16 所示。

#### 清单 13.16 测试 `Mongo OrderRepository`。

```
package tacos.data;

import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.data.mongo.DataMongoTest;

import reactor.test.StepVerifier;
import tacos.Ingredient;
import tacos.Taco;
import tacos.TacoOrder;
import tacos.Ingredient.Type;

@DataMongoTest
public class OrderRepositoryTest {

    @Autowired
    OrderRepository orderRepo;

    @BeforeEach
    public void setup() {
        orderRepo.deleteAll().subscribe();
    }

    @Test
    public void shouldSaveAndFetchOrders() {
        TacoOrder order = createOrder();

        StepVerifier
            .create(orderRepo.save(order))
            .expectNext(order)
            .verifyComplete();

        StepVerifier
            .create(orderRepo.findById(order.getId()))
            .expectNext(order)
            .verifyComplete();

        StepVerifier
            .create(orderRepo.findAll())
            .expectNext(order)
            .verifyComplete();
    }

    private TacoOrder createOrder() {
        TacoOrder order = new TacoOrder();
        ...
        return order;
    }
}
```

`shouldSaveAndFetchOrders()` 方法所做的第一件事是构造一个订单，有完整的客户和付款信息和一些玉米卷。（为了简洁起见，`createOrder()` 方法的详细信息被省略。）然后它使用一个 **StepVerifier** 保存 **TacoOrder** 对象，并断言 `save()` 方法返回保存的 **TacoOrder**。然后尝试通过其 ID 获取订单，并断言它收到了完整的 **TacoOrder**。最后，它获取所有 **TacoOrder** 对象（应该只有一个），并断言它是预期的 **TacoOrder** 对象。

如前所述，您将需要一个 MongoDB 服务器，并侦听端口 27017，以运行此测试。而 **Flapdoodle** 这个嵌入式 MongoDB 不能很好地与响应式 **Repository** 配合使用。如果您的计算机上安装了 **Docker**，则可以轻松启动 MongoDB 服务器，并侦听端口 27017，如下所示：

```
$ docker run -p27017:27017 mongo
```

还有其他方法可以获得 MongoDB 服务器。更多细节请参阅以下网址的文档：<https://www.mongodb.com/>。

现在，我们已经了解了如何为 **R2BDC** 和 **MongoDB** 创建响应式 **Repository**，下面让我们看一看另一个 **Spring Data** 选择：**Cassandra**。

## 13.3 使用 **Cassandra** 响应式保存数据

针对 **Cassandra** 数据库的响应式持久化，您需要添加以下 **starter** 依赖项到项目中：

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-cassandra-reactive</artifactId>
</dependency>
```

此依赖项替代了我们前面使用的任何 **Mongo** 或 **R2DBC** 依赖项。

然后，您需要声明有关 **Cassandra** 键空间的一些细节，以及 **schema** 的管理方式。在 **application.yml** 文件中，添加以下行：

```
spring:
  data:
    rest:
      base-path: /data-api
    cassandra:
      keyspace-name: tacocloud
      schema-action: recreate
      local-datacenter: datacenter1
```

这与我们在第 4 章中使用非响应式 **Cassandra Repository** 时使用的 **YAML** 配置完全相同。需要注意的是 **keyspace-name**，需要在 **Cassandra** 集群中创建具有该名称的键空间。

您还需要在本地计算机上运行 **Cassandra** 群集，侦听端口 **9042**，最简单的方法是使用 **Docker**：

```
$ docker network create cassandra-net
$ docker run --name my-cassandra --network cassandra-net \
  -p 9042:9042 -d cassandra:latest
```

如果您的 **Cassandra** 群集位于另一台计算机或其他端口上，则需要在 **application.yml** 中指定连接信息，如第 4 章所示。

要创建键空间，请运行 **CQL shell** 并使用如下 **create keyspace** 命令：

```
$ docker run -it --network cassandra-net --rm cassandra cqlsh my-cassandra
cqlsh> create keyspace tacocloud
WITH replication = {'class': 'SimpleStrategy', 'replication_factor' : 1};
```

现在您有了一个 **Cassandra** 集群、一个新的“tacocloud”键空间和 **Spring Data Cassandra Reactive starter** 在您的项目中，您已经准备好开始定义领域实体类了。

### 13.2.1 为 **Cassandra** 定义实体类

与持久化 **Mongo** 时一样，选择响应式还是非响应式 **Cassandra** 持久化，在定义领域实体类上完全没区别。我们将使用的 **Ingredient**、**Taco** 和 **TacoOrder** 类，与我们在第 4 章创建的完全相同。

**Cassandra** 注解的 **Ingredient** 类如清单 13.17 所示。

清单 13.17 注解 **Cassandra** 持久化 **Ingredient** 类。

```
package tacos;

import org.springframework.data.cassandra.core.mapping.PrimaryKey;
import org.springframework.data.cassandra.core.mapping.Table;

import lombok.AccessLevel;
import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;

@Data
@AllArgsConstructor
@NoArgsConstructor(access=AccessLevel.PRIVATE, force=true)
@Table("ingredients")
public class Ingredient {

    @PrimaryKey
    private String id;
    private String name;
    private Type type;

    public static enum Type {
        WRAP, PROTEIN, VEGGIES, CHEESE, SAUCE
    }

}
```

至于 **Taco** 类，它是用类似的 **Cassandra** 持久化注解定义的，如清单 13.18 所示：

清单 13.18 为 **Cassandra** 持久化注解的 **Taco**。

```

package tacos;

import java.util.ArrayList;
import java.util.Date;
import java.util.List;
import java.util.UUID;

import javax.validation.constraints.NotNull;
import javax.validation.constraints.Size;

import org.springframework.data.cassandra.core.cql.Ordering;
import org.springframework.data.cassandra.core.cql.PrimaryKeyType;
import org.springframework.data.cassandra.core.mapping.Column;
import org.springframework.data.cassandra.core.mapping.PrimaryKeyColumn;
import org.springframework.data.cassandra.core.mapping.Table;
import org.springframework.data.rest.core.annotation.RestResource;

import com.datastax.oss.driver.api.core.uuid.Uuids;

import lombok.Data;

@Data
@RestResource(rel = "tacos", path = "tacos")
@Table("tacos")
public class Taco {

    @PrimaryKeyColumn(type=PrimaryKeyType.PARTITIONED)
    private UUID id = Uuids.timeBased();

    @NotNull
    @Size(min = 5, message = "Name must be at least 5 characters long")
    private String name;

    @PrimaryKeyColumn(type=PrimaryKeyType.CLUSTERED,
        ordering=Ordering.DESCENDING)
    private Date createdAt = new Date();

    @Size(min=1, message="You must choose at least 1 ingredient")
    @Column("ingredients")
    private List<IngredientUDT> ingredients = new ArrayList<>();

    public void addIngredient(Ingredient ingredient) {
        this.ingredients.add(new IngredientUDT(ingredient.getName(), ingredient.getType()));
    }
}

```

由于通过用户定义的类型引用对象，因此您还需要 `IngredientUDT` 类，如清单 13.19 所示。

清单 13.19 是 **Cassandra** 持久化的用户定义类型。

```

package tacos;

import org.springframework.data.cassandra.core.mapping.UserDefinedType;

import lombok.AccessLevel;
import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;

@Data
@AllArgsConstructor
@NoArgsConstructor(access = AccessLevel.PRIVATE, force = true)
@UserDefinedType("ingredient")
public class IngredientUDT {
    private String name;
    private Ingredient.Type type;
}

```

三个领域实体类中的最后一个，**TacoOrder** 持久化如清单 13.20 所示。

清单 13.20 为 **Cassandra** 持久化注解 **TacoOrder**。



```

package tacos;

import java.io.Serializable;
import java.util.ArrayList;
import java.util.Date;
import java.util.List;
import java.util.UUID;

import org.springframework.data.cassandra.core.mapping.Column;
import org.springframework.data.cassandra.core.mapping.PrimaryKey;
import org.springframework.data.cassandra.core.mapping.Table;

import com.datastax.oss.driver.api.core.uuid.Uuids;

import lombok.Data;

@Data
@Table("tacoorders")
public class TacoOrder implements Serializable {
    private static final long serialVersionUID = 1L;

    @PrimaryKey
    private UUID id = Uuids.timeBased();
    private Date placedAt = new Date();

    @Column("user")
    private UserUDT user;

    private String deliveryName;

    private String deliveryStreet;

    private String deliveryCity;

    private String deliveryState;

    private String deliveryZip;

    private String ccNumber;

    private String ccExpiration;

    private String ccCVV;

    @Column("tacos")
    private List<TacoUDT> tacos = new ArrayList<>();

    public void addTaco(Taco taco) {
        this.addTaco(new TacoUDT(taco.getName(), taco.getIngredients()));
    }
    public void addTaco(TacoUDT tacoUDT) {
        this.tacos.add(tacoUDT);
    }
}

```

就像 **Taco** 通过用户定义的类型引用配料一样，**TacoOrder** 引用通过 **TacoUDT** 类实现 **Taco**，如清单 13.21 所示。

清单 13.21 **Cassandra** 持久化的 **Taco** 用户定义类型。

```
package tacos;

import java.util.List;

import org.springframework.data.cassandra.core.mapping.UserDefinedType;

import lombok.Data;

@Data
@UserDefinedType("taco")
public class TacoUDT {

    private final String name;
    private final List<IngredientUDT> ingredients;

}
```

再重复一遍，它们与非响应式的对应类是相同的。我只重复在这里把它们放了一遍，这样您就不必到其他单来记住它们的样子。

现在让我们定义保存这些对象的 **Repository**。

## 13.2.2 创建响应式 **Cassandra Repository**

到目前为止，您可能已经期望响应式 **Cassandra Repository** 与等效的非响应性 **Repository** 看起来是一样的。如果是这样，那真是太好了！您跟上了 **Spring Data** 的步伐。无论 **Repository** 是否是响应式的，都尽可能的使用类似的编程模型。

您可能已经猜到了，使 **Repository** 是响应式的，主要就是扩展 **ReactiveCrudRepository** 接口。如下图中的 **IngredientRepository** 所示：

```
package tacos.data;

import org.springframework.data.repository.reactive.ReactiveCrudRepository;

import tacos.Ingredient;

public interface IngredientRepository
    extends ReactiveCrudRepository<Ingredient, String> {
}
```

当然，**OrderRepository** 也是如此：

```
package tacos.data;

import java.util.UUID;

import org.springframework.data.domain.Pageable;
import org.springframework.data.repository.reactive.ReactiveCrudRepository;

import reactor.core.publisher.Flux;
import tacos.TacoOrder;
import tacos.User;

public interface OrderRepository
    extends ReactiveCrudRepository<TacoOrder, UUID> {

    Flux<TacoOrder> findByUserOrderByPlacedAtDesc(
        User user, Pageable pageable);
}
```

事实上，这些 **Repository** 不仅让人联想到它们的非响应式对应库，它们与我们在本章前面编写的 **MongoDB Repository** 也没有太大区别。除了 **Cassandra** 使用 **UUID** 作为 ID 类型，而不是 **TacoOrder** 的字符串，它们实际上是完全相同的。这再次证明了 **Spring Data** 中采用的一致性设计思想（尽可能的）。

让我们通过编写几个测试，来结束编写响应式 **Cassandra Repository** 的工作，并验证它们是否有效。

### 13.2.3 测试响应式 **Cassandra Repository**

到此，您可能不会惊讶，测试响应式 **Cassandra Repository** 与测试响应式 **MongoDB** 存储库的方式完全类似。例如，看看清单 13.22 中的 **IngredientRepositoryTest**，看看您是否能够发现它与清单 13.15 的区别。

清单 13.22 测试 **Cassandra Ingredient Repository**。

```
package tacos.data;

import static org.assertj.core.api.Assertions.assertThat;

import java.util.ArrayList;

import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.data.cassandra.DataCassandraTest;

import reactor.core.publisher.Flux;
import reactor.test.StepVerifier;
import tacos.Ingredient;
import tacos.Ingredient.Type;

@DataCassandraTest
public class IngredientRepositoryTest {

    @Autowired
    IngredientRepository ingredientRepo;

    @BeforeEach
    public void setup() {
        Flux<Ingredient> deleteAndInsert = ingredientRepo.deleteAll()
            .thenMany(ingredientRepo.saveAll(
                Flux.just(
                    new Ingredient("FLT0", "Flour Tortilla", Type.WRAP),
                    new Ingredient("GRBF", "Ground Beef", Type.PROTEIN),
                    new Ingredient("CHED", "Cheddar Cheese", Type.CHEESE)
                )))
        StepVerifier.create(deleteAndInsert)
            .expectNextCount(3)
            .verifyComplete();
    }

    @Test
    public void shouldSaveAndFetchIngredients() {

        StepVerifier.create(ingredientRepo.findAll())
            .recordWith(ArrayList::new)
            .thenConsumeWhile(x -> true)
            .consumeRecordedWith(ingredients -> {
                assertThat(ingredients).hasSize(3);
                assertThat(ingredients).contains(
                    new Ingredient("FLT0", "Flour Tortilla", Type.WRAP));
                assertThat(ingredients).contains(
                    new Ingredient("GRBF", "Ground Beef", Type.PROTEIN));
                assertThat(ingredients).contains(
                    new Ingredient("CHED", "Cheddar Cheese", Type.CHEESE));
            })
            .verifyComplete();

        StepVerifier.create(ingredientRepo.findById("FLT0"))
            .assertNext(ingredient -> {
                ingredient.equals(new Ingredient("FLT0", "Flour Tortilla", Type.WRAP));
            });
    }
}
```

您看到了吗？在 MongoDB 版本用 `@DataMongoTest` 注解的地方，这个新的 Cassandra 版本用 `@DataMongoTest` 注解。就这样！其他地方完全一样。`OrderRepositoryTest` 也是如此。将 `@DataMongoTest` 替换为 `@DataCassandraTest`，其他一切都是相同的：

```
@DataCassandraTest
public class OrderRepositoryTest {
    ...
}
```

再一次，各种 **Spring Data** 项目之间的一致性甚至扩展到测试的方式上。这使得在持久化到不同类型的项目时，切换数据库变得很容易，而不必考虑它们是如何开发。

## 13.4 总结

- Spring Data 支持多种数据库类型的响应式持久化，包括关系型数据库（使用 R2DBC）、MongoDB 和 Cassandra。
- Spring Data R2DBC 为关系型数据库持久化提供了一个响应式选择，但还没有直接支持领域实体类中的关联关系。
- 由于缺乏直接关系支持，Spring Data R2DBC Repository 需要使用不同的方法设计领域对象和数据库表。
- Spring Data MongoDB 和 Spring Data Cassandra 提供了几乎相同的编程模型，为 MongoDB 和 Cassandra 数据库编写响应式 Repository。
- 使用 Spring Data 测试注解和 StepVerifier，您可以自动测试通过 Spring 应用程序上下文创建的响应式 Repository。

## 第 14 章 使用 RSocket

本章内容：

- 使用 RSocket 进行响应式网络通信
- 使用 RSocket 的四种通信模型
- 在 WebSocket 上进行 RSocket 传输

曾经有一段时期，在电话和现代电子技术出现之前，与远在他乡的朋友和家人最好的沟通方式是写一封信，然后把信投到邮箱里。这不是一种快速的沟通方式，需要几天甚至几周才能收到回信。但这是一种有效的方式，而且是唯一的选择。

多亏了亚历山大·格雷厄姆·贝尔（Alexander Graham Bell）发明了电话。电话提供了一种与远方朋友和家人交流的新方式，这是一种近实时的异步通信方式。电话发展得相当快，从贝尔先生的第一项发明开始有一段时间了，但它仍然是一种保持联系的流行方式，几乎没有人再写信了。

当涉及到应用程序之间的通信时，由 HTTP 和 REST 服务提供的 请求/响应 模式非常常见，但也有其局限性。就像写信一样，请求/响应 要先发送消息，然后等待响应。这种模式不容易提供异步通信。在异步通信模式中，服务器可能会以数据流进行响应，或允许开放双向通道，客户端和服务端可以在该通道上相互反复发送数据。

在本章中，我们将介绍 RSocket。这是一种相对较新的应用程序间协议，不仅仅提供简单的 请求/响应 通信模式。因为它天然就是响应式的，所以它可以比阻塞式的 HTTP 请求更高效。

在此过程中，我们将了解如何使用 Spring 开发 RSocket 通信。首先，让我们深入了解 RSocket 与基于 HTTP 的通信有哪些区别。



## 14.1 介绍 RSocket

RSocket 是一种二进制应用程序协议，它是异步的，基于响应式流。换句话说，RSocket 提供了应用程序之间的异步通信，支持我们在第 12 章学习的，完全一致的响应模型类型（如 Flux 和 Mono）。

作为基于 HTTP 通信的替代方案，它更加灵活，提供了四种不同的通信模型：请求/响应、请求/流、即发即忘和通道。

请求/响应（Request-Response）是 RSocket 最熟悉的通信模型，它模仿了典型的 HTTP 通信方式。在请求/响应模型中，客户端发出一个请求到服务器，服务器以单个响应进行反馈。如图 14.1 所示，使用响应式的 Mono 类型定义请求/响应。

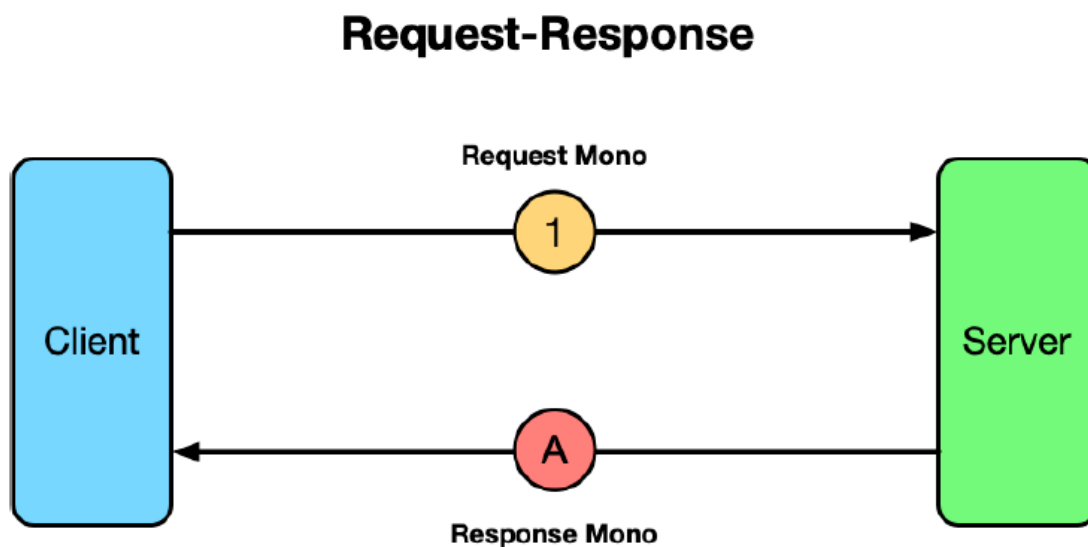


图 14.1 RSocket 的请求/响应通信模型。

虽然请求/响应模型看起来与 HTTP 提供的通信模型等效，但要理解 RSocket 本质上是非阻塞的，并且基于响应式类型。尽管客户端仍将等待服务器的答复，但底层是非阻塞和响应式的，那么就可以更高效地被线程使用。

请求/流通信模型类似于请求/响应，只是在客户端向服务器发送了一个请求后，服务器以零个或多个流进行响应。图 14.2 说明了请求/流模型，使用 Mono 进行请求，使用 Flux 进行响应。

## Request-Stream

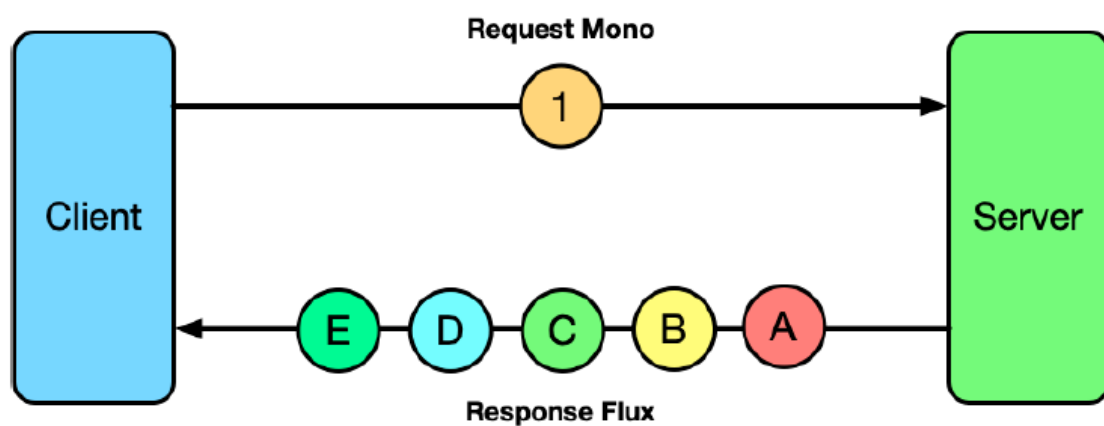


图 14.2 RSocket 的请求/流通信模型。

在某些情况下，客户端可能需要向服务器发送数据，但不需要响应。RSocket 为这些情况提供了 即发即忘 模型，如图 14.3 所示。

## Fire-and-Forget

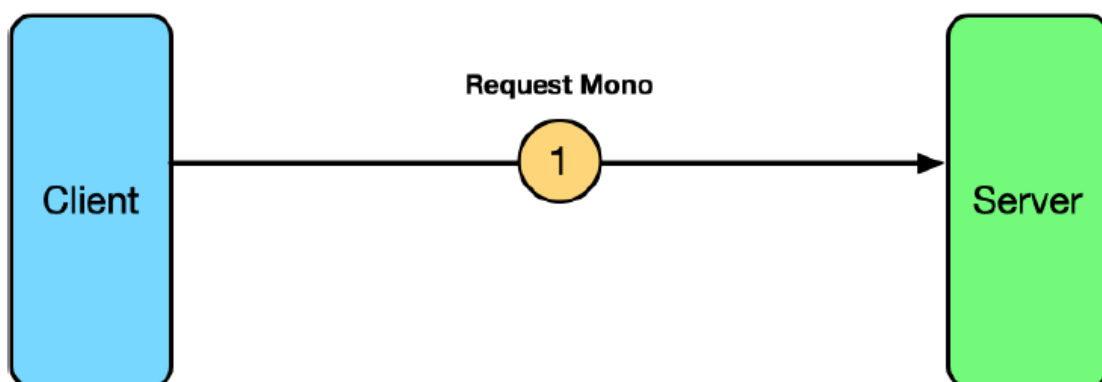


图 14.3 RSocket 的即发即忘通信模型。

在 即发即忘 模型中，客户端向服务器发送请求，但服务器进行回复。

最后，RSocket 最灵活的通信模型是 通道 模型。在 通道 模型，客户端与服务器打开一个双向通道，每个通道都可以向对方随时发送数据。图 14.4 说明了 通道 通信方式。

## Channel

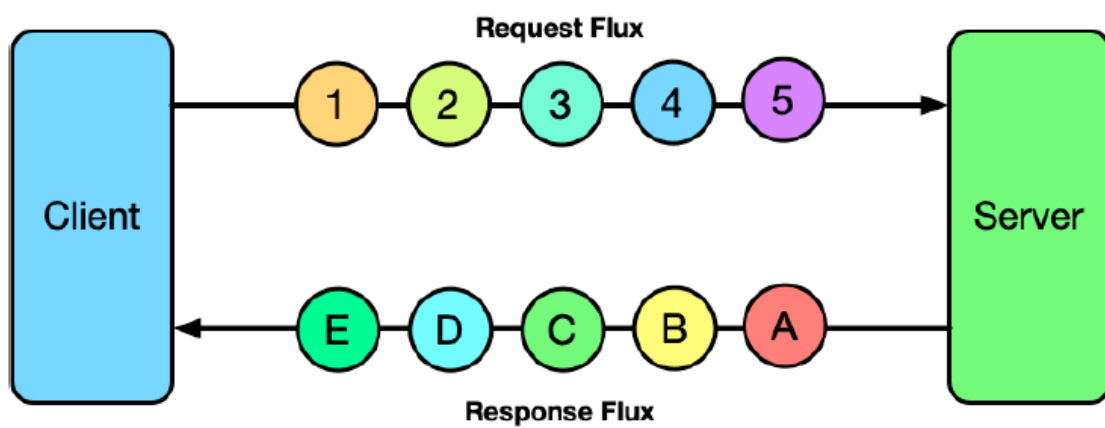


图 14.4 RSocket 的 通道 通信模型。

RSocket 支持多种语言 and 平台，包括 Java、JavaScript、Kotlin、.NET、GO 和 C++。Spring 的最新版本提供了对 RSocket 的完美支持，可以用熟悉的 Spring 用法轻松创建服务器和客户端。

让我们深入了解如何创建四种通信模型的 RSocket 服务器和客户端。

## 14.2 创建一个简单的 RSocket 服务端和客户端

Spring 为 RSocket 的消息传递提供了非常好的支持，包括所有四种通信方式模型。要开始使用 RSocket，您需要将 Spring Boot RSocket starter 添加到项目构建中。在 Maven 的 POM 文件中，RSocket starter 依赖项如下所示：

清单 14.1 Spring Boot 的 RSocket starter 依赖项

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-rsocket</artifactId>
</dependency>
```

RSocket 的服务器和客户端应用程序都需要添加这个依赖。

使用 Spring Initializr 选择依赖项时，您可能会看到类似命名的 WebSocket 依赖项。虽然 RSocket 和 WebSocket 名称相似，且您可以使用 WebSocket 作为 RSocket 的底层（我们将在本章后面介绍），但您并不需要选择 WebSocket 依赖。

接下来，您需要决定哪种通信模型最适合您的应用程序。没有一种方式会适合所有情况，因此您需要依据期望的应用程序通信行为，来权衡选择。但是，正如您将在接下来的几节中看到的，每个通信模型的开发模型没有太大的不同，所以如果您选错了，很容易切换。

让我们看看如何在 Spring 中使用每种通信创建 RSocket 服务器和客户端。因为每个 RSocket 的通信模型都不同，最适合在特定的用例场景中，我们将暂时搁置 Taco Cloud 应用程序，看看如何在不同的问题域上应用 RSocket。我们将从了解如何使用 请求/响应 通信模型开始。

### 14.2.1 使用 请求/响应 模型

在 Spring 中创建 RSocket 服务器与创建 控制器类一样简单，与在 web 应用程序或 REST 服务中那样。下面的控制器是一个示例 RSocket 服务，处理来自客户端的问候语，并以另一个问候语回应：

清单 14.2 一个简单的 RSocket 请求/响应 服务器。

```
package rsocket;
import org.springframework.messaging.handler.annotation.MessageMapping;
import org.springframework.stereotype.Controller;

import lombok.extern.slf4j.Slf4j;
import reactor.core.publisher.Mono;

@Controller
@Slf4j
public class GreetingController{

    @MessageMapping("greeting")
    public Mono<String> handleGreeting(Mono<String> greetingMono) {
        return greetingMono
            .doOnNext(greeting ->
                log.info("Received a greeting: " + greeting))
            .map(greeting -> "Hello back to you!");
    }

}
```

如您所见，web 控制器和 RSocket 控制器之间的关键区别在于：RSocket 控制器不处理给定路径的 HTTP 请求（使用 @GetMapping 或 @PostMapping 注解），而是使用 @MessageMapping 注解，处理给定路由上的传入消息。在本例中，当请求被调用时，从客户端发送到名为“greeting”的路由，调用 handleGreeting() 方法。

handleGreeting() 方法以 Mono<String> 为参数。在这种情况下，问候语非常简单，一个字符串就足够了。但是如果需要的话，传入的有效载荷可以是更复杂的类型。在收到 Mono<String> 之后，它只是记录收到的问候语，然后使用 map() 函数在 Mono 上创建一个新的 Mono<String>，以返回给客户端。

尽管 RSocket 控制器不处理 HTTP 请求，但路由名称可以具有类似路径的形式，包括路径占位符都可以传递到处理方法中。例如，考虑以下形式的 handleGreeting() 方法：

```
@MessageMapping("greeting/{name}")
public Mono<String> handleGreeting(
    @DestinationVariable("name") String name,
    Mono<String> greetingMono) {

    return greetingMono
        .doOnNext(greeting ->
            log.info("Received a greeting from " + name + " : " + greeting))
        .map(greeting -> "Hello to you, too, " + name);
}
```

在本例中，@MessageMapping 中指定的路由包含一个名为“name”的变量。它用大括号括起来，与 Spring MVC 中的路径变量表示方法相同。类似地，该方法接受带 @DestinationVariable 注解的字符串参数，来引用占位符变量。就像 Spring MVC 的 @PathVariable 注解一样，@DestinationVariable 用于提取指定的占位符值，并将其传递给处理方法。

一旦运行到新版本的 handleGreeting()，将使用路由中指定的名称，向客户端返回更个性化的问候语。

在创建 **RSocket** 服务器时，还必须记住一件事：指定要监听的端口。默认情况下，**RSocket** 服务是基于 **TC** 的，并且侦听特定端口。`spring.rsocket.server.port` 配置属性可以设置 **RSocket** 服务器使用的端口：

```
spring:
  rsocket:
    server:
      port: 7000
```

`spring.rsocket.server.port` 属性有两个用途：启用服务器和指定服务器应侦听的端口。如果没有设置，那么 **Spring** 将假定您的应用程序将仅作为客户端使用，不会做为服务端进行端口侦听。在这种情况下，我们启动一台服务器，设置属性 `spring.rsocket.server.port`，如下所示，在端口 **7000** 上启动侦听。

现在，让我们把注意力转向 **RSocket** 客户端。在 **Spring** 中，使用 **RSocketRequester** 实现了 **RSocket** 客户端。**RSocket** 的 **Spring Boot** 自动配置将自动在 **Spring** 应用程序上下文中创建 **RSocketRequester.Builder** 类型的 **bean**。您可以将这个 **bean** 注入任何其他 **bean** 中。

例如，下面的 **ApplicationRunner** **bean** 注入了 **RSocketRequester.Builder**：

```
package rsocket;
import org.springframework.boot.ApplicationRunner;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.messaging.rsocket.RSocketRequester;

@Configuration
@Slf4j
public class RSocketClientConfiguration {

    @Bean
    public ApplicationRunner sender(RSocketRequester.Builder requesterBuilder) {
        return args -> {
            RSocketRequester tcp = requesterBuilder.tcp("localhost", 7000);
            // ... send messages with RSocketRequester ...
        };
    }
}
```

在本例中，创建的 **RSocketRequester** 侦听 **localhost** 端口 **7000**。返回的 **RSocketRequester** 可用于向服务器发送消息。

在 **请求/响应** 模型中，请求需要（至少）指定路由和有效数据载荷。您一定还记得，服务端控制器会处理名为 **“greeting”** 的请求，且需要字符串类型的输入。它还返回一个字符串作为输出。以下是完整的客户端代码，显示了如何向服务器发送问候语并处理响应：

**清单 14.3** 发送请求的客户端。

```
RSocketRequester tcp = requesterBuilder.tcp("localhost", 7000);

// ... send messages with RSocketRequester ...

tcp
    .route("greeting")
    .data("Hello RSocket!")
    .retrieveMono(String.class)
    .subscribe(response -> log.info("Got a response: " + response));
```

这将通过“greeting”路由向服务器发送问候语“Hello RSocket! ”。请注意，它需要一个返回，是在调用 `retrieveMono()` 时指定的。这个 `subscribe()` 方法订阅返回的 `Mono`，并通过日志记录返回值。

现在，让我们假设您希望向另一个路径发送问候语，该路径在其内部接受一个变量。客户端代码基本相同，只是在 `route()` 中包含了占位符变量的值：

```
String who = "Craig";
tcp
    .route("greeting/{name}", who)
    .data("Hello RSocket!")
    .retrieveMono(String.class)
    .subscribe(response -> log.info("Got a response: " + response));
```

在这里，消息将被发送到名为“greeting/Craig”的路由。该路由将由控制器中的方法处理，这个方法使用了 `@MessageMapping` 注解，指定了路由为“greeting/{name}”。同时，您也可以在路由中硬编码名称，或使用字符串连接来创建路由名称。在客户机中使用占位符，可以很容易地避免字符串连接造成的混乱。

请求/响应 模型可能是 `RSocket` 最简单的通信模型。这只是个开始，让我们看看如何使用可能返回多个响应的请求/流 模型。

## 14.2.2 处理 请求/流 消息

并非所有的交互都是一个请求和一个响应的交互。例如，在股票报价场景中，请求给定股票的报价流可能很有用。在 请求/响应 模型中，客户端需要重复轮询当前股价。但是在 请求/流 模型中，客户机只需要询问一次股票价格，然后订阅定期更新流。

为了说明 请求/流 模型，让我们实现股票报价的服务器和客户端。首先，我们需要定义一个可以携带股票报价信息的对象。清单 14.4 中的 `StockQuote` 类用于此目的。

清单 14.4 表示股票报价的模型类。

```
package rsocket;
import java.math.BigDecimal;
import java.time.Instant;

import lombok.AllArgsConstructor;
import lombok.Data;

@Data
@AllArgsConstructor
public class StockQuote {
    private String symbol;
    private BigDecimal price;
    private Instant timestamp;
}
```

如您所见，股票报价带有股票号、价格和表示价格的时间戳。为了简洁起见，我们使用 `Lombok` 来帮助构造和访问方法。

现在，让我们编写一个控制器来处理股票报价请求。您会发现清单 14.5 中的 `StockQuoteController` 与上一节的 `GreetingController` 有些相似。

清单 14.5 一个用于流式传输股票报价的 `RSocket` 控制器。

```
package rsocket;
import java.math.BigDecimal;
import java.time.Duration;
import java.time.Instant;

import org.springframework.messaging.handler.annotation.DestinationVariable;
import org.springframework.messaging.handler.annotation.MessageMapping;
import org.springframework.stereotype.Controller;

import reactor.core.publisher.Flux;

@Controller
public class StockQuoteController {

    @MessageMapping("stock/{symbol}")
    public Flux<StockQuote> getStockPrice(
        @DestinationVariable("symbol") String symbol) {
        return Flux
            .interval(Duration.ofSeconds(1))
            .map(i -> {
                BigDecimal price = BigDecimal.valueOf(Math.random() * 10);
                return new StockQuote(symbol, price, Instant.now());
            });
    }
}
```



这里，`getStockPrice()` 法处理“`stock/{symbol}`”路由上的传入请求，接受带有 `@DestinationVariable` 注解的路由中的股票号码。为了简单起见，价格不是查找实际股票价格，而是随机值计算的（这可能不会准确地模拟某些实际股票的波动性）。

然而，最值得注意的是 `getStockPrice()` 返回一个 `Flux<StockQuote>`，而不是 `Mono<StockQuote>`。这给了 Spring 一个提示，该处理方法支持 请求/流 模型。在内部，`Flux` 按每秒一次的间隔来持续创。该 `Flux` 被映射到另一个生成随机股票报价的 `Flux` 上。简单地说，`getStockPrice()` 方法处理的单个请求将返回多个值，每秒钟一次。

请求/流 模型的客户端与 请求/响应 模型的客户端差别不大。唯一的关键区别在于，不是对请求程序调用 `retrieveMono()`，而是 应该调用 `retrieveFlux()`。股票报价服务的客户端可能如下所示：

```
String stockSymbol = "XYZ";

RSocketRequester tcp = requesterBuilder.tcp("localhost", 7000);
tcp
    .route("stock/{symbol}", stockSymbol)
    .retrieveFlux(StockQuote.class)
    .doOnNext(stockQuote -> {
        log.info(
            "Price of " + stockQuote.getSymbol() +
            " : " + stockQuote.getPrice() +
            " (at " + stockQuote.getTimestamp() + ")");
    })
    .subscribe();
```

现在，我们已经了解了如何创建 `RSocket` 服务器和客户端来处理单个和多个响应。但是，如果服务器没有要发送的响应，或者客户端没有响应，该怎么办呢？让我们看看如何使用 即发即忘 沟通模式。

### 14.2.3 发送 即发即忘 消息

想象一下，您在一艘刚刚受到敌舰攻击的星际飞船上。您发出全舰范围的“红色警报”，以便所有人员都处于战斗模式。您不需要等待飞船计算机的响应来确认警报发送状态。在这种情况下，您也没有时间等待阅读任何回应。您设置了警报，然后继续执行更多关键操作。

这是一个 即发即忘 的例子。考虑到目前的情况，虽然您可能不会忘记您处于红色警戒状态。对您来说，应对这场战争危机比处理警报响应显然更重要。

为了模拟这个场景，我们将创建一个 **RSocket** 服务器，它处理警报状态，但不进行任何响应。首先，我们需要定义一个请求数据类。像清单 14.6 中的 **Alert** 类。

清单 14.6 表示警报的模型类。

```
package rsocket;

import java.time.Instant;

import lombok.AllArgsConstructor;
import lombok.Data;

@Data
@AllArgsConstructor
public class Alert {

    private Level level;
    private String orderedBy;
    private Instant orderedAt;

    public static enum Level {
        YELLOW, ORANGE, RED, BLACK
    }
}
```

警报对象捕获警报级别、警报的请求者以及警报发出时的时间戳（定义为 **Instant** 类型）。同样，为了保持代码简短，我们使用 **Lombok** 自动生成构造和访问方法。

在服务器端，清单 14.7 中的 **AlertController** 类将处理警报消息。

清单 14.7 处理警报更新的 **RSocket** 控制器。

```

package rsocket;
import org.springframework.messaging.handler.annotation.MessageMapping;
import org.springframework.stereotype.Controller;
import lombok.extern.slf4j.Slf4j;
import reactor.core.publisher.Mono;

@Controller
@Slf4j
public class AlertController {

    @MessageMapping("alert")
    public Mono<Void> setAlert(Mono<Alert> alertMono) {
        return alertMono
            .doOnNext(alert -> {
                log.info(alert.getLevel() + " alert"
                    + " ordered by " + alert.getOrderedBy()
                    + " at " + alert.getOrderedAt());
            })
            .thenEmpty(Mono.empty());
    }
}

```

`setAlert()` 方法处理“**alert**”路由上的消息。为保持简单（尽管在实际作战情况下毫无用处），它只记录警报。但重要的是返回一个 `Mono<Void>`，表示没有响应。因此此处理方法支持 **即发即忘** 模型。

在客户端中，代码与 **请求/响应** 或 **请求/流** 的代码差别不大：

```

RSocketRequester tcp = requesterBuilder.tcp("localhost", 7000);
tcp
    .route("alert")
    .data(new Alert(
        Alert.Level.RED, "Craig", Instant.now()))
    .send()
    .subscribe();
log.info("Alert sent");

```

但是，请注意，客户端没有调用 `retrieveMono()` 或 `retrieveFlux()`，而只是调用 `send()`，不需要响应。

现在，让我们看看如何处理 **通道** 通信模型，在该模型中，服务器和客户端相互发送多条消息。

## 14.2.4 双向发送消息

到目前为止，在我们看到的所有通信模型中，客户端发送一个请求，而服务器响应为零、一或多个。在 `请求/流` 模型中，服务器能够将多个响应返回到客户端，但客户端仍仅限于发送单个请求。但是为什么只有服务器能发送多次数据呢？为什么客户不能发送多个请求呢？

这就是 `通道` 模型的用武之地。在 `通道` 模式下，客户端可以将多个请求流式传输到服务器，服务器也可以将多个请求流式传输回客户端。双方双向对话，它是 `RSocket` 中最灵活的通信模型，尽管也是最复杂的。

为了演示如何在 `Spring` 中使用 `RSocket` 通道通信，让我们创建一个计算账单上的小费、接收一个 `Flux` 请求并以 `Flux` 响应的服务。首先，我们需要定义表示请求和响应数据的模型对象。清单 14.8 中所示的类表示客户端发出，服务器接收的数据模型类。

清单 14.8 表示入站小费请求的模型。

```
package rsocket;

import java.math.BigDecimal;

import lombok.AllArgsConstructor;
import lombok.Data;

@Data
@AllArgsConstructor
public class GratitudeIn {

    private BigDecimal billTotal;
    private int percent;

}
```

`GratitudeIn` 包含计算小费所需的两项基本信息：账单总数和百分比。

清单 14.9 中的 `GratitudeOut` 类表示响应，如下所示，包含账单总数以及包含计算出的小费金额。

清单 14.9 表示出站小费响应的模型。

```
package rsocket;

import java.math.BigDecimal;

import lombok.AllArgsConstructor;
import lombok.Data;

@Data
@AllArgsConstructor
public class GratitudeOut {

    private BigDecimal billTotal;
    private int percent;
    private BigDecimal gratuity;

}
```

清单 14.10 中的 `GratitudeController` 处理小费请求，看起来非常像我们在本章之前写的控制器。

清单 14.10 一个 `RSocket` 控制器，它在一个通道上接收并返回多条消息。

```

package rsocket;
import java.math.BigDecimal;
import org.springframework.messaging.handler.annotation.MessageMapping;
import org.springframework.stereotype.Controller;

import lombok.extern.slf4j.Slf4j;
import reactor.core.publisher.Flux;

@Controller
@Slf4j
public class GratuityController {

    @MessageMapping("gratuity")
    public Flux<GratuityOut> calculate(Flux<GratuityIn> gratuityInFlux) {
        return gratuityInFlux
            .doOnNext(in -> log.info("Calculating gratuity: " + in))
            .map(in -> {
                double percentAsDecimal = in.getPercent() / 100.0;
                BigDecimal gratuity = in.getBillTotal()
                    .multiply(BigDecimal.valueOf(percentAsDecimal));
                return new GratuityOut(in.getBillTotal(), in.getPercent(), gratuity);
            });
    }
}

```

然而，有一个显著的区别：它不仅返回 **Flux**，而且还接受 **Flux** 作为输入。与 请求/流 模型一样，返回的 **Flux** 使控制器能够将多个值流式传输到客户端。但是 **Flux** 参数是 通道 模式与 请求/流 模型主要区别。输入的 **Flux** 参数，允许控制器处理来自客户端的请求流。

通道 模式的客户端与 请求/流 模式的客户端仅在以下方面不同：它向服务器发送 **Flux<GratuityIn>** 而不是 **Mono<GratuityIn>**，如清单 14.11 所示。

清单 14.11 通过通道发送和接收多条消息的客户端。

```

RSocketRequester tcp = requesterBuilder.tcp("localhost", 7000);

Flux<GratuityIn> gratuityInFlux =
    Flux.fromArray(new GratuityIn[] {
        new GratuityIn(new BigDecimal(35.50), 18),
        new GratuityIn(new BigDecimal(10.00), 15),
        new GratuityIn(new BigDecimal(23.25), 20),
        new GratuityIn(new BigDecimal(52.75), 18),
        new GratuityIn(new BigDecimal(80.00), 15)
    })
    .delayElements(Duration.ofSeconds(1));

tcp
    .route("gratuity")
    .data(gratuityInFlux)
    .retrieveFlux(GratuityOut.class)
    .subscribe(out ->
        log.info(out.getPercent() + "% gratuity on "
            + out.getBillTotal() + " is "
            + out.getGratuity()));

```

在本例中，**Flux<GratuityIn>** 是使用 **fromArray()** 方法静态创建的，但可以从任何数据源创建的 **Flux**。很可能是从响应式数据 **Repository**（我们将在下一章了解更多信息）。

您可能已经观察到，服务器接受和返回的响应类型，用于确定支持的 **RSocket** 通信模式。表 14.1 总结了服务器的输入/输出类型与 **RSocket** 通信模型。

表14.1 支持的 **RSocket** 模型由 **handler** 方法的参数和返回类型决定。

| <b>RSocket</b> 通信模式 | <b>Handler</b> 参数类型 | <b>Handler</b> 返回类型 |
|---------------------|---------------------|---------------------|
| 请求/响应               | Mono                | Mono                |
| 请求/流                | Mono                | Flux                |
| 即发即忘                | Mono                | Mono                |
| 通道                  | Flux                | Flux                |

您可能想知道服务器是否可以接受并返回一个 **Mono**。简言之，这不行。尽管您可以想象传入一个 **Flux**，处理多个请求后以 `Mono<Void>` 进行回应，这`即发即忘`模型也可以混搭，但没有该场景的 **RSocket** 模型。因此，它不受支持。

## 14.3 在 WebSocket 上传输 RSocket

默认情况下，RSocket 通信通过 TCP 套接字进行。但也有一些情况使 TCP 不是一个好的选择。考虑这两种情况：

- 客户端是用 JavaScript 编写的，并在用户的 web 浏览器中运行。
- 客户端必须跨越网关或防火墙边界才能访问服务器，防火墙不允许通过随意指定的端口进行通信。

此外，WebSocket 本身缺乏对路由的支持，要求路由细节在应用程序级别定义。通过将 RSocket 置于 WebSocket 之上，WebSocket 将受益来自 RSocket 的内置路由支持。

在这些情况下，RSocket 可以通过 WebSocket 进行运输。WebSocket 通信通过 HTTP 进行，HTTP 是所有 web 浏览器中的主要通信方式，并且通常是允许通过防火墙的。

要从 TCP 传输切换到 WebSocket 传输，只需在服务器和客户端中做几处简单的修改。首先，由于 WebSocket 是通过 HTTP 传输的，您需要确保服务器端应用程序支持处理 HTTP 请求。简言之，您需要将 WebFlux starter 依赖项添加到项目构建中（如果项目中还没有）：

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-webflux</artifactId>
</dependency>
```

您还需要配置属性 `spring.rsocket.server.transport`，指出要在服务器端使用 WebSocket 进行传输。还需要设置 `spring.rsocket.server.mapping-path` 属性，指定 RSocket 将在 HTTP 上通信的路径。服务器的 `application.yml` 配置如下所示：

```
spring:
  rsocket:
    server:
      transport: websocket
      mapping-path: /rsocket
```

与通过特定端口进行通信的 TCP 传输不同，WebSocket 传输在特定的 HTTP 路径上。因此，不需要设置 `spring.rsocket.server.port`。

这就是在服务器端为 RSocket 启用 WebSocket 传输所需要做的全部工作。其他一切都将与使用 TCP 时完全相同。

在客户端，只需要一个小改动。您不再创建基于 TCP 的请求，而是需要通过调用 `RSocketRequester.Builder` 上的方法 `websocket()`，创建基于 WebSocket 的请求：

```
RSocketRequester requester = requesterBuilder.websocket(
    URI.create("ws://localhost:8080/rsocket"));

requester
    .route("greeting")
    .data("Hello RSocket!")
    .retrieveMono(String.class)
    .subscribe(response -> log.info("Got a response: " + response));
```

这就是通过 WebSocket 传输 RSocket 的全部内容！

## 14.4 总结

- **RSocket**是一种异步二进制协议，提供四种通信模型：`请求/响应`、`请求/流`、`即发即忘` 以及 `通道`。
- **Spring** 通过带 `@MessageHandler` 注解的控制器处理方法，在服务端支持 **RSocket**。
- 通过 **RSocketRequester** 支持客户端的 **RSocket** 通信。
- 在两种情况下，**Spring** 的 **RSocket** 支持通过响应式的 **Flux** 和 **Mono** 支持完全响应式通信。
- 默认情况下，**RSocket** 通信通过 **TCP** 进行，但也可以通过 **WebSocket**，以避免防火墙限制和支持浏览器客户端。



## 第 15 章 使用 Spring Boot Actuator

本章内容

- 在 Spring Boot 项目中启用 Actuator
- 探索 Actuator 的 endpoint
- 定制 Actuator
- 为 Actuator 增加安全机制

您有没有猜过，一个礼物包装盒里面到底是什么礼物呢？摇一摇、掂一掂、量一下尺寸，可能对里面的东西猜个八九不离十。但是直到打开它之前，是没有办法百分百的确定的。

一个正在运行的应用程序，就像一份包装好的礼物。您可以访问它，然后根据反馈做出相对合理的猜测。但您能保证一定猜测正确吗？如果能有什么办法，能让您看到应用程序的内部：查看是如何运行的、查看它的健康状况，甚至触发一些操作以改变其运行方式，那就可以确定应用程序的实际情况了！

在本章中，我们将探讨 Spring Boot 的 Actuator。Actuator 可以对 Spring Boot 应用程序进行监视，并提供一些度量指标。这些都是可以应用于生产环境的特性。Actuator 的特征通过几个 endpoint 提供，这些 endpoint 可通过 HTTP 和 JMX mbean 获得。本章主要关注 HTTP endpoint，而在第 17 章讨论 JMX endpoint。

## 15.1 介绍 Actuator

在机器设备中，Actuator 指驱动器，是负责控制和移动机器的机械装置。在 Spring Boot 应用程序中，Spring Boot Actuator 扮演同样的角色，使我们能够看到正在运行的应用程序的内部。在某种程度上，控制应用程序的行为。

使用 Actuator 公开的 endpoint，我们可以获取正在运行的 Spring Boot 应用程序的一些情况：

- 应用程序环境中有哪些配置属性可用？
- 应用程序中各种包的日志记录级别是什么？
- 应用程序现在消耗了多少内存？
- 给定的 HTTP endpoint 被请求了多少次？
- 应用程序及其协作的其他服务的健康状况如何？

要在 Spring Boot 应用程序中启用 Actuator，只需添加 Actuator 的依赖。在 Spring Boot 应用程序 pom.xml 文件中，添加如下部分：

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
```

一旦 Actuator 依赖添加到了项目构建中，应用程序就有了几个现成的 endpoint，包括表 16.1 中所描述的这些。

**表 15.1：窥视 Spring Boot 应用程序内部，并操纵其运行状态的 Actuator endpoint**

| HTTP 方法           | 路径              | 描述                                                       | 是否默认启用 |
|-------------------|-----------------|----------------------------------------------------------|--------|
| GET               | /auditevents    | 生成已触发的任何审核事件的报告。                                         | 否      |
| GET               | /beans          | 描述 <b>Spring</b> 应用程序上下文中的所有 <b>bean</b> 。               | 否      |
| GET               | /conditions     | 对于在应用程序上下文中创建的 <b>bean</b> ，自动配置条件校验通过或失败的报告。            | 否      |
| GET               | /configprops    | 描述所有配置属性以及当前值。                                           | 否      |
| GET, POST, DELETE | /env            | 生成 <b>Spring</b> 应用程序可用的所有属性源及其属性的报告。                    | 否      |
| GET               | /env/{toMatch}  | 描述单个环境属性的值。                                              | 否      |
| GET               | /health         | 返回应用程序的聚合运行状况和（可能）外部相关应用程序的运行状况。                         | 是      |
| GET               | /heapdump       | 下载堆转储信息。                                                 | 否      |
| GET               | /httptrace      | 生成最近 100 个请求的跟踪记录。                                       | 否      |
| GET               | /info           | 返回开发人员自定义的应用程序信息。                                        | 是      |
| GET               | /loggers        | 生成应用程序中包的列表，及其配置的有效日志级别。                                 | 否      |
| GET, POST         | /loggers/{name} | 返回某个 <b>logger</b> 的有效日志记录级别，并可以通过 <b>POST</b> 请求设置日志级别。 | 否      |
| GET               | /mappings       | 生成关于所有 HTTP 映射及其相应处理方法的报告。                               | 否      |
| GET               | /metrics        | 返回所有度量指标类别的列表。                                           | 否      |
| GET               | /metrics/{name} | 返回某度量的多维值的集合。                                            | 否      |
| GET               | /scheduledtasks | 列出所有计划任务                                                 | 否。     |
| GET               | /threaddump     | 返回所有应用程序线程的报告。                                           | 否      |

除了基于 HTTP 的 endpoint，表 15.1 中的所有 **Actuator** 的 endpoint，除 **/heapdump** 以外，同时暴露为 **JMX MBean**。我们将在第 17 章讨论 **JMX** 的 **Actuator**。

### 15.1.1 配置 Actuator 基本路径

默认情况下，在表 15.1 中显示的所有路径都以 `/exactor` 为前缀。这意味着，如果您希望从 Actuator 获取应用的健康状况信息，您可以对路径 `/actuator/health` 发送 GET 请求来获得。

您可以通过设置 `management.endpoint.web.base-path` 属性来更改前缀路径。例如，如果您希望将前缀路径更改为 `/management`，您可以进行如下设置：

```
management:
  endpoints:
    web:
      base-path: /management
```

上述的属性设置之后，您就可以对路径 `/management/health` 发出 GET 请求，来获得应用程序的健康状况信息。

无论您是否更改 Actuator 基本路径，为了简洁起见，本章中用到的 Actuator 将会省略基本路径。例如，当提到 `/health` 端点时，实际是 `/{{base path}}/health` 端点。更准确地说，如果基本路径没有更改，则应该为 `/exactor/health` 端点。

## 15.1.2 启用和禁用 Actuator 端点

您可能已经注意到，默认情况下只启用了 `/health` 端点。由于大多数 Actuator 携带有敏感信息，所以应予以保护。Actuator 自身并没有提供安全措施，但您可以使用 Spring Security 来保护 Actuator。对于禁用的端点，您可以手动将其打开。

有两个配置属性：`management.endpoints.web.exposure.include` 和 `management.endpoints.web.exposure.exclude`，可用于控制端点的启用和关闭。使用 `management.endpoints.web.exposure.include`，您可以指定要启用的端点。例如，如果您只想公开 `/health`、`/info`、`/beans` 和 `/conditions`，可以使用以下配置：

```
management:
  endpoints:
    web:
      exposure:
        include: health,info,beans,conditions
```

`management.endpoints.web.exposure.include` 属性也可以使用星号（`*`）通配符，将所有 Actuator 端点都启用：

```
management:
  endpoints:
    web:
      exposure:
        include: '*'
```

如果要启用除少数端点以外的所有端点，通常更方便的做法是：使用通配符将它们全部包含在内，然后显式排除一些。例如，启用所有但除了 `/threaddump` 和 `/heapdump` 之外的 Actuator 端点。您可以像下面这样设置

`management.endpoints.web.exposure.include` 和 `management.endpoints.web.exposure.exclude` 属性：

```
management:
  endpoints:
    web:
      exposure:
        include: '*'
        exclude: threaddump,heapdump
```

如果您决定不仅仅启用 `/health` 和 `/info`，那么配置 Spring Security 以限制对其他端点的任意访问是个好主意。我们将在第 15.4 节讨论如何保护 Actuator 端点。现在，让我们看看如何访问这些已经启用的 HTTP 端点。

## 15.2 使用 Actuator 端点

Actuator 可以通过表 15.1 中列出的 HTTP 端点，提供一个有关正在运行的应用程序的，有趣和有用的名副其实的宝库。作为 HTTP 端点，它们可以像任何 REST API 一样使用，使用您熟悉的任何 HTTP 客户端，包括 Spring 的 RestTemplate 和 WebClient，来自基于浏览器的 JavaScript 应用程序，或者简单地使用 curl 命令行客户端。

为了探索 Actuator 端点，您将在本章中使用 curl 命令行客户端。在第 16 章中，我将向您介绍 Spring Boot Admin，它在应用程序的 Actuator 端点之上构建出了一个用户友好的 Web 应用程序。

为了解 Actuator 都提供了哪些端点，可以对 Actuator 基本路径发送 GET 请求，这将返回所有端点的 HATEOAS 链接。使用 curl 向 `/actuator` 发出请求，您可能会得到类似这样的响应（为了节省篇幅而进行了删节）：

```
$ curl localhost:8081/actuator
{
  "_links": {
    "self": {
      "href": "http://localhost:8081/actuator",
      "templated": false
    },
    "auditevents": {
      "href": "http://localhost:8081/actuator/auditevents",
      "templated": false
    },
    "beans": {
      "href": "http://localhost:8081/actuator/beans",
      "templated": false
    },
    "health": {
      "href": "http://localhost:8081/actuator/health",
      "templated": false
    }
  },
  ...
}
```

因为不同的库可能会贡献自己额外的 Actuator 端点，并且因为某些端点可能不会被暴露，所以实际结果可能因应用程序而异。

不论差异如何，从 Actuator 的基本路径返回的这些链接，都明确展示了 Actuator 所提供的端点。让我们开始探索 Actuator，从提供应用程序的基本信息的两个端点 `/health` 和 `/info` 开始。

## 15.2.1 获取重要的应用程序信息

在一次典型的看医生开始时，我们通常会被问到两个非常基本的问题：您是谁，您感觉如何？尽管医生或护士选择的词可能不同，但他们最终目标是了解一些基本情况：关于他们正在治疗的人，以及您为什么要见他们。

这些相同的基本问题是 **Actuator** 的 `/info` 和 `/health` 端点为 **Spring Boot** 应用程序回答的。`/info` 端点告诉您有关应用程序的一些信息，`/health` 端点告诉您应用程序的健康状况。

### 询问有关应用程序的信息

要了解有关正在运行的 **Spring Boot** 应用程序的一些信息，您可以询问 `/info` 端点。但是，默认情况下，`/info` 端点的信息量并不大。

当您使用 **curl** 发出请求时，您可能会看到以下内容：

```
$ curl localhost:8081/actuator/info
{}

```

虽然 `/info` 端点似乎不是很有用，但最好将其视为一个干净的画布，您可以在其上绘制您想要呈现的任何信息。

有多种方法可以提供 `/info` 端点返回所需的信息，最直接的方法是创建一个或多个配置属性，其中属性名称以 `info.` 为前缀。例如，假设您希望来自 `/info` 端点以包含支持联系信息，包括电子邮件地址和电话号码。为此，您可以在 `application.yml` 文件中配置以下属性：

```
info:
  contact:
    email: support@tacocloud.com
    phone: 822-625-6831

```

`info.contact.email` 和 `info.contact.phone` 属性可能对于 **Spring Boot**，或应用程序上下文中的任何 **bean** 都没有任何特殊意义。然而，由于它以 `info.` 为前缀，`/info` 端点现在将在其响应中回显属性的值：

```
{
  "contact": {
    "email": "support@tacocloud.com",
    "phone": "822-625-6831"
  }
}

```

在第 15.3.1 节中，我们将了解一些其他方法来使 `/info` 端点返回有关应用程序的更多有用信息。

### 检查应用程序健康状态

向 `/health` 端点发送 **HTTP GET** 请求会产生一个简单的 **JSON** 响应，其中包含应用程序的健康状态。例如，以下是您在使用 **curl** 获取 `/health` 端点时可能会看到的内容：

```
$ curl localhost:8080/actuator/health
{"status":"UP"}

```

您可能想知道，一个只能报告应用程序已启动的端点有多大用处？如果应用程序关闭，它会报告什么？

实际情况是，此处显示的状态是一个或多个健康指标的聚合状态。健康指标报告应用程序与之交互的外部系统的健康状况，例如数据库、消息代理，甚至 **Eureka** 和 **Config Server** 等 **Spring Cloud** 组件。每个指标的健康状态可以是以下之一：

- **UP** - 外部系统已启动并可访问。
- **DOWN** - 外部系统已关闭或无法访问。
- **UNKNOWN** - 外部系统的状态尚不清楚。
- **OUT\_OF\_SERVICE** - 外部系统可访问，但当前不可用。

然后将所有健康指标的健康状态，聚合到应用程序的整体健康状态中，应用以下规则：

- 如果所有健康指标都为 **UP**，则应用程序健康状态为 **UP**。
- 如果一个或多个健康指标为 **DOWN**，则应用程序健康状态为 **DOWN**。
- 如果一个或多个健康指标为 **OUT\_OF\_SERVICE**，则应用健康状态为 **OUT\_OF\_SERVICE**。
- **UNKNOWN** 健康状况将被忽略，并且不会被纳入应用程序的总体健康状况。

默认情况下，仅返回聚合状态以响应对 `/health` 的请求。但是，您可以配置 `management.endpoint.health.show-details` 属性以显示所有健康指标的完整详细信息：

```
management:
  endpoint:
    health:
      show-details: always
```

`management.endpoint.health.show-details` 属性默认为 **never**，但也可以设置为 **always** 始终显示所有健康指标的完整详细信息，或 **when-authorized** 仅在请求客户端时显示完整详细信息完全授权。

现在，当您向 `/health` 端点发出 **GET** 请求时，您将获得完整的健康指标详细信息。以下是与 **Mongo** 文档数据库集成的服务的示例：

```
{
  "status": "UP",
  "details": {
    "mongo": {
      "status": "UP",
      "details": {
        "version": "3.2.2"
      }
    },
    "diskSpace": {
      "status": "UP",
      "details": {
        "total": 499963170816,
        "free": 177284784128,
        "threshold": 10485760
      }
    }
  }
}
```

无论任何其他外部依赖项如何，所有应用程序都将具有名为 **diskSpace** 的文件系统的运行状况指示器。**diskSpace** 运行状况指示器指示文件系统的运行状况（希望是 **UP**），这取决于剩余的可用空间量。如果可用磁盘空间低于阈值，它将报告 **DOWN** 状态。

在前面的示例中，还有一个 **mongo** 健康指示器，用于报告 **Mongo** 数据库的状态。显示的详细信息包括 **Mongo** 数据库版本。



自动配置确保只有与应用程序相关的健康指标，才会出现在来自 `/health` 端点的响应中。除了 `mongo` 和 `diskSpace` 健康指标，`Spring Boot` 还为其他几个外部数据库和系统提供健康指标，包括：

- `Cassandra`
- `Config Server`
- `Couchbase`
- `Eureka`
- `Hystrix`
- `JDBC data sources`
- `Elasticsearch`
- `InfluxDB`
- `JMS message brokers`
- `LDAP`
- `Email servers`
- `Neo4j`
- `Rabbit message brokers`
- `Redis`
- `Solr`

此外，第三方库可能会贡献自己的健康指标。我们将在 15.3.2 节中了解如何编写自定义健康指标。

如您所见，`/health` 和 `/info` 端点提供有关正在运行的应用程序的一般信息。同时，还有其他 `Actuator` 端点可以深入了解应用程序配置。让我们看看 `Actuator` 如何显示应用程序的配置信息。

## 15.2.2 查看配置详细信息

除了获取有关应用程序的一般信息之外，了解应用程序的配置信息也很有用。应用程序上下文中有哪些 **bean**？哪些自动配置条件通过或失败？应用程序可以使用哪些环境属性？HTTP 请求如何映射到控制器？一个或多个包或类设置为怎样的日志级别？

这些问题由 **Actuator** 的 `/beans`、`/conditions`、`/env`、`/configprops`、`/mappings` 和 `/loggers` 端点回答。在某些情况下，例如 `/env` 和 `/loggers`，您还可以动态调整正在运行的应用程序的配置。从 `/beans` 端点开始，我们将查看每一个端点，以深入了解正在运行的应用程序的配置。

### 获取 **Bean Wiring** 报告

探索 **Spring** 应用程序上下文最重要的端点是 `/beans` 端点。此端点返回一个 **JSON** 文档，描述应用程序上下文中的每个 **bean**、它的 **Java** 类型以及它注入的任何其他 **bean**。

对 `/beans` 的 **GET** 请求所得到的完整响应信息非常多，可以轻松地填满本章。作为完整响应的替代，让我们考虑以下片段，它专注于单个 **bean** 条目：

```
{
  "contexts": {
    "application-1": {
      "beans": {
        ...
        "ingredientsController": {
          "aliases": [],
          "scope": "singleton",
          "type": "tacos.ingredients.IngredientsController",
          "resource": "file [/Users/habuma/Documents/Workspaces/
TacoCloud/ingredient-service/target/classes/tacos/ingredients/
IngredientsController.class]",
          "dependencies": [
            "ingredientRepository"
          ]
        },
        ...
      },
      "parentId": null
    }
  }
}
```

响应的根是 **contexts** 元素，它包含应用程序中每个 **Spring** 应用程序上下文的一个子元素。在每个应用程序上下文中都有一个 **beans** 元素，其中包含应用程序上下文中所有 **bean** 的详细信息。

在前面的示例中，显示的 **bean** 是名称为 `ingredientsController` 的 **bean**。您可以看到它没有别名，范围为单例，并且类型为 `tacos.ingredients.IngredientsController`。此外，资源属性给出了定义 **bean** 的类文件的路径。并且 **dependencies** 属性列出了注入给定 **bean** 的所有其他 **bean**。在这种情况下，`ingredientsController` **bean** 被注入了一个名称为 `ingredientRepository` 的 **bean**。

### 解释自动配置

如您所见，自动配置是 **Spring Boot** 提供的最强大的功能之一。但是，有时您可能想知道为什么会自动配置某些内容。或者您可能期望某些东西被自动配置，想知道它为什么没有被自动配置。在这种情况下，您可以向 `/conditions` 发出 **GET** 请求，以了解自动配置的情况。

`/conditions` 返回的自动配置报告分为三部分：**positiveMatches**（条件配置通过）、**negativeMatches**（条件配置失败）和 **unconditionalClasses**（无条件类）。以下是 `/conditions` 请求的响应的片段，示例显示了每个部分：

```
{
  "contexts": {
    "application-1": {
      "positiveMatches": {
        ...
        "MongoDataAutoConfiguration#mongoTemplate": [
          {
            "condition": "OnBeanCondition",
            "message": "@ConditionalOnMissingBean (types
:org.springframework.data.mongodb.core.MongoTemplate;
SearchStrategy: all) did not find any beans"
          }
        ],
        ...
      },
      "negativeMatches": {
        ...
        "DispatcherServletAutoConfiguration": {
          "notMatched": [
            {
              "condition": "OnClassCondition",
              "message": "@ConditionalOnClass did not find required
class 'org.springframework.web.servlet.
DispatcherServlet'"
            }
          ],
          "matched": []
        },
        ...
      },
      "unconditionalClasses": [
        ...
        "org.springframework.boot.autoconfigure.context.
ConfigurationPropertiesAutoConfiguration",
        ...
      ]
    }
  }
}
```

在 **positiveMatches** 部分，您会看到 **MongoTemplate bean** 是由自动配置配置的，且因为它不存在。导致这种情况的自动配置是因为有 `@ConditionalOnMissingBean` 注释，如果尚未显式配置要配置的 **bean**，则它会传递要配置的 **bean**。在这种情况下，由于没有找到 **MongoTemplate** 类型的 **bean**，因此自动配置介入并配置了一个。

在 **negativeMatches** 下，Spring Boot 自动配置考虑配置一个 **DispatcherServlet**。但是 `@ConditionalOnClass` 条件注解失败了，因为找不到 **DispatcherServlet**。

最后，如 **unconditionalClasses** 部分所示，配置了 **ConfigurationPropertiesAutoConfiguration bean**。配置属性是 Spring Boot 运行的基础，因此任何与配置属性相关的配置，都应该在没有任何条件的情况下自动配置。

## 检查环境和配置属性

除了了解您的应用程序 **bean** 如何连接在一起之外，您可能还对了解哪些环境属性可用，以及哪些配置属性注入了 **bean** 中感兴趣。

当您向 `/env` 端点发出 **GET** 请求时，您将收到一个相当长的，来自 **Spring** 应用程序正在运行的，所有属性来源的属性列表。这包括来自环境变量的属性、**JVM** 系统属性、`application.properties` 和 `application.yml` 文件，甚至 **Spring Cloud Config Server**（如果应用程序是 **Config Server** 的客户端）。

下面的清单显示了，您可能从 `/env` 端点获得的响应数据的一个大大简化的示例，让您对它提供的信息有所了解。

清单 15.1 来自 `/env` 端点的数据

```
$ curl localhost:8081/actuator/env
{
  "activeProfiles": [
    "development"
  ],
  "propertySources": [
    ...
    {
      "name": "systemEnvironment",
      "properties": {
        "PATH": {
          "value": "/usr/bin:/bin:/usr/sbin:/sbin",
          "origin": "System Environment Property \"PATH\""
        },
        ...
        "HOME": {
          "value": "/Users/habuma",
          "origin": "System Environment Property \"HOME\""
        }
      }
    },
    {
      "name": "applicationConfig: [classpath:/application.yml]",
      "properties": {
        "spring.application.name": {
          "value": "ingredient-service",
          "origin": "class path resource [application.yml]:3:11"
        },
        "server.port": {
          "value": 8081,
          "origin": "class path resource [application.yml]:9:9"
        },
        ...
      }
    },
    ...
  ]
}
```

尽管 `/env` 的完整响应提供了更多信息，但清单 15.1 中包含了那些需要关注的元素。首先，请注意响应顶部是一个名为 `activeProfiles` 的字段。现在的配置表明 `development` 配置文件处于活动状态。如果任何其他配置文件处于活动状态，也将列出这些配置文件。

接下来，`propertySources` 字段是一个数组，其中包含 **Spring** 应用程序环境中每个属性源的条目。在清单 15.1 中，只显示了 `systemEnvironment` 和引用 `application.yml` 文件的 `applicationConfig` 属性源。

在每个属性源中，列出了该源提供的所有属性，并与其值配对。在 `application.yml` 属性源的情况下，每个属性的 `origin` 字段准确说明属性设置的位置，包括在 `application.yml` 中的行和列。

当该属性的名称作为路径的第二个元素给出时，`/env` 端点还可用于获取特定属性。例如，要检查 `server.port` 属性，请提交对 `/env/server.port` 的 **GET** 请求，如下所示：

```
$ curl localhost:8081/actuator/env/server.port
{
  "property": {
    "source": "systemEnvironment", "value": "8081"
  },
  "activeProfiles": [ "development" ],
  "propertySources": [
    { "name": "server.ports" },
    { "name": "mongo.ports" },
    { "name": "systemProperties" },
    {
      "name": "systemEnvironment",
      "property": {
        "value": "8081",
        "origin": "System Environment Property \"SERVER_PORT\""
      }
    },
    { "name": "random" },
    {
      "name": "applicationConfig: [classpath:/application.yml]",
      "property": {
        "value": 0,
        "origin": "class path resource [application.yml]:9:9"
      }
    },
    { "name": "springCloudClientHostInfo" },
    { "name": "refresh" },
    { "name": "defaultProperties" },
    { "name": "Management Server" }
  ]
}
```

如您所见，所有属性源仍被表示，但只有那些设置了指定属性的源才会包含附加信息。在这种情况下，系统环境属性源和 `application.yml` 属性源都具有 `server.port` 属性的值。因为 `systemEnvironment` 属性源优先于它下面列出的任何属性源，所以它的值 `8081` 胜出。获胜值反映在属性字段下方的顶部附近。

`/env` 端点不仅可以用于读取属性值。通过向 `/env` 端点提交 `POST` 请求以及带有名称和值字段的 `JSON` 数据，您还可以在正在运行的应用程序中设置属性。例如，要将名为 `tacocloud.discount.code` 的属性设置为 `TACOS1234`，您可以使用 `curl` 在命令行提交 `POST` 请求，如下所示：

```
$ curl localhost:8081/actuator/env \
  -d '{"name":"tacocloud.discount.code","value":"TACOS1234"}' \
  -H "Content-type: application/json"
{"tacocloud.discount.code":"TACOS1234"}
```

提交属性后，新设置的属性及其值将在响应中返回。稍后，如果您决定不再需要该属性，您可以向 `/env` 端点提交 `DELETE` 请求，以删除通过该端点创建的所有属性：

```
$ curl localhost:8081/actuator/env -X DELETE
{"tacocloud.discount.code":"TACOS1234"}
```

与通过 `Actuator` 的 `API` 设置属性一样重要，要注意，使用 `/env` 端点的 `POST` 请求设置的任何属性，仅适用于接收请求的应用程序实例，是临时的，并且会在应用程序重新启动时丢失。

## 浏览 HTTP 请求映射

尽管 Spring MVC（和 Spring WebFlux 的）编程模型，通过简单地使用请求映射注释，对方法进行注释来轻松处理 HTTP 请求，但有时很难全面了解，应用程序可以处理的所有类型的 HTTP 请求，以及处理这些请求的组件类型。

Actuator 的 `/mappings` 端点提供了应用程序中每个 HTTP 请求处理程序的一站式视图，无论是来自 Spring MVC 控制器还是 Actuator 自己的端点之一。要获取 Spring Boot 应用程序中所有端点的完整列表，请向 `/mappings` 端点发出 GET 请求，您可能会收到类似于下面显示的简化响应的内容。

清单 15.2 如 `/mappings` 端点所示的 HTTP 请求映射

```
$ curl localhost:8081/actuator/mappings | jq
{
  "contexts": {
    "application-1": {
      "mappings": {
        "dispatcherHandlers": {
          "webHandler": [
            ...
            {
              "predicate": "{[/ingredients],methods=[GET]}",
              "handler": "public
reactor.core.publisher.Flux<tacos.ingredients.Ingredient>
tacos.ingredients.IngredientsController.allIngredients()",
              "details": {
                "handlerMethod": {
                  "className": "tacos.ingredients.IngredientsController",
                  "name": "allIngredients",
                  "descriptor": "()Lreactor/core/publisher/Flux;"
                },
                "handlerFunction": null,
                "requestMappingConditions": {
                  "consumes": [],
                  "headers": [],
                  "methods": [
                    "GET"
                  ],
                  "params": [],
                  "patterns": [
                    "/ingredients"
                  ],
                  "produces": []
                }
              }
            }
          ],
          ...
        ]
      }
    },
    "parentId": "application-1"
  },
  "bootstrap": {
    "mappings": {
      "dispatcherHandlers": {}
    },
    "parentId": null
  }
}
```

在这里，来自命令 `curl` 的响应通过管道传输到一个名为 `jq` 的解析器中，以更美观易读的方式打印 JSON 数据。为简洁起见，此响应已被删节以仅显示单个请求处理程序。具体来说，它表明对 `/ingredients` 的 GET 请求将由 `IngredientsController` 的 `allIngredients()` 方法处理。

## 管理日志级别

日志记录对任何应用程序来说都是非常重要的功能。日志记录可以提供一种审计手段，以及一种粗略的调试手段。

设置日志记录级别是需要进行反复权衡的行为。如果日志级别设置得低，日志中可能会出现过多的噪音，可能很难找到有用的信息。另一方面，如果您将日志记录级别设置得太高，则日志对于了解应用程序正在执行的操作可能没有多大价值。

日志级别通常应用在包级别上。如果您想知道正在运行的 **Spring Boot** 应用程序中设置了哪些日志记录级别，您可以向 `/loggers` 端点发出 **GET** 请求。以下 **JSON** 显示了对 `/loggers` 的响应的摘录：

```
{
  "levels": [ "OFF", "ERROR", "WARN", "INFO", "DEBUG", "TRACE" ],
  "loggers": {
    "ROOT": {
      "configuredLevel": "INFO", "effectiveLevel": "INFO"
    },
    ...
    "org.springframework.web": {
      "configuredLevel": null, "effectiveLevel": "INFO"
    },
    ...
    "tacos": {
      "configuredLevel": null, "effectiveLevel": "INFO"
    },
    "tacos.ingredients": {
      "configuredLevel": null, "effectiveLevel": "INFO"
    },
    "tacos.ingredients.IngredientServiceApplication": {
      "configuredLevel": null, "effectiveLevel": "INFO"
    }
  }
}
```

响应数据以所有有效日志记录级别的列表开始。之后，**loggers** 元素会列出应用程序中每个包的日志记录级别的详细信息。**configureLevel** 属性显示已显式配置的日志记录级别（如果尚未显式配置，则为 **null**）。

**EffectiveLevel** 属性给出了有效的日志级别，它可能是从父包或根记录器继承的。

此示例仅显示了根记录器和四个包的日志记录级别，完整的响应会包括应用程序中每个包的日志记录级别条目，包含所有使用到的库中的包。如果您希望查看特定包，您可以将包名称指定为请求路径中的一部分。

例如，如果您只想知道为 **tacocloud.ingredients** 设置了哪些日志记录级别。您可以向 `/loggers/tacos.ingredients` 发出请求：

```
{
  "configuredLevel": null,
  "effectiveLevel": "INFO"
}
```

除了返回应用程序包的日志级别外，`/loggers` 端点还允许您通过发出 **POST** 请求来更改配置的日志级别。例如，假设您要 **tacocloud.ingredients** 包的日志记录级别设置为 **DEBUG**。用以下 **curl** 命令将实现这一点：

```
$ curl localhost:8081/actuator/loggers/tacos/ingredients \
-d '{"configuredLevel": "DEBUG"}' \
-H "Content-type: application/json"
```

现在日志级别已更改，您可以向 `/loggers/tacos/ingredients` 发出 GET 请求以查看更改：

```
{
  "configuredLevel": "DEBUG",
  "effectiveLevel": "DEBUG"
}
```

请注意，`configuredLevel` 以前为 `null`，现在是 `DEBUG`。这种变化也传递到了 `effectiveLevel`。但最重要的是，如果该包中的任何代码在调试级别记录任何内容，日志文件将包含该调试级别信息。



### 15.2.3 查看应用程序活动

关注应用程序中的活动是非常有用的。这些活动包括应用程序正在处理的各种 HTTP 请求，以及应用程序中的所有线程等等。为此，Actuator 提供了 `/httptrace`、`/threaddump` 和 `/heapdump` 端点。

`/heapdump` 端点可能是 Actuator 最难详细描述的一端点。简单来说，它下载一个 gzip 压缩的 HPROF 堆转储文件，以便对内存或线程问题进行分析跟踪。因篇幅原因，且堆转储的使用是一个相当高级的特性，所以仅在这里提一下 `/heapdump` 端点。

### 跟踪 HTTP 活动

`/httptrace` 端点报告应用处理的最近 100 个请求。详细信息包括请求的方法和路径、时间戳、请求和响应的头信息，以及处理请求的耗时。

下面的 JSON 片段显示了 `/httptrace` 端点的返回：

```
{
  "traces": [
    {
      "timestamp": "2020-06-03T23:41:24.494Z",
      "principal": null,
      "session": null,
      "request": {
        "method": "GET",
        "uri": "http://localhost:8081/ingredients",
        "headers": {
          "Host": ["localhost:8081"],
          "User-Agent": ["curl/7.54.0"],
          "Accept": ["*/*"]
        },
        "remoteAddress": null
      },
      "response": {
        "status": 200,
        "headers": {
          "Content-Type": ["application/json;charset=UTF-8"]
        }
      },
      "timeTaken": 4
    },
    ...
  ]
}
```

这些信息可能对调试很有用，但更有趣的是，随着时间的推移跟踪这些数据，可以深入了解应用程序运行状况。可以了解在给定的时间段内，应用的繁忙程度，或根据响应状态的值，统计有多少成功的请求和失败的请求。在第 16 章中，您将看到 Spring Boot Admin 如何捕获这些信息，并且将这些 HTTP 跟踪信息可视化到一个运行图中。

### 监视线程

除了对 HTTP 请求进行跟踪之外，要确定应用程序的运行情况，监视线程活动状态也非常有用的。`/threaddump` 端点生成一个当前线程活动的快照。下面是 `/threaddump` 响应数据片段，可以了解此端点都提供了哪些信息：

```

{
  "threadName": "reactor-http-nio-8",
  "threadId": 338,
  "blockedTime": -1,
  "blockedCount": 0,
  "waitedTime": -1,
  "waitedCount": 0,
  "lockName": null,
  "lockOwnerId": -1,
  "lockOwnerName": null,
  "inNative": true,
  "suspended": false,
  "threadState": "RUNNABLE",
  "stackTrace": [
    {
      "methodName": "kevent0",
      "fileName": "KQueueArrayWrapper.java",
      "lineNumber": -2,
      "className": "sun.nio.ch.KQueueArrayWrapper",
      "nativeMethod": true
    },
    {
      "methodName": "poll",
      "fileName": "KQueueArrayWrapper.java",
      "lineNumber": 198,
      "className": "sun.nio.ch.KQueueArrayWrapper",
      "nativeMethod": false
    },
    ...
  ],
  "lockedMonitors": [
    {
      "className": "io.netty.channel.nio.SelectedSelectionKeySet",
      "identityHashCode": 1039768944,
      "lockedStackDepth": 3,
      "lockedStackFrame": {
        "methodName": "lockAndDoSelect",
        "fileName": "SelectorImpl.java",
        "lineNumber": 86,
        "className": "sun.nio.ch.SelectorImpl",
        "nativeMethod": false
      }
    },
    ...
  ],
  "lockedSynchronizers": [],
  "lockInfo": null
}

```

完整的线程转储报告，包括正在运行的应用程序中的每个线程。为了节省篇幅，这里的线程转储信息只显示了单个线程的一个简化条目。您可以看到，它包括线程有关的阻塞和锁定状态的详细信息，以及其他一些细节。还有一个堆栈跟踪，可以提供一些线程在代码的哪个区域所花费时间的信息。

因为 `/threaddump` 端点只会把请求时的线程快照信息取出来，很难全面了解线程的行为是如何随着时间的推移变化的。在第 16 章中，您将看到 **Spring Boot Admin** 如何实时监视 `/threaddump` 端点。

## 15.2.4 利用运行时指标

`/metrics` 端点能够报告正在运行应用程序的多项指标，包括有关内存、处理器、垃圾收集以及 HTTP 请求的相关指标。`Actuator` 提供了二十多种开箱即用的度量指标类别。向 `/metrics` 发出 GET 请求，返回的度量指标列表示例如下：

```
$ curl localhost:8081/actuator/metrics | jq
{
  "names": [
    "jvm.memory.max",
    "process.files.max",
    "jvm.gc.memory.promoted",
    "http.server.requests",
    "system.load.average.1m",
    "jvm.memory.used",
    "jvm.gc.max.data.size",
    "jvm.memory.committed",
    "system.cpu.count",
    "logback.events",
    "jvm.buffer.memory.used",
    "jvm.threads.daemon",
    "system.cpu.usage",
    "jvm.gc.memory.allocated",
    "jvm.threads.live",
    "jvm.threads.peak",
    "process.uptime",
    "process.cpu.usage",
    "jvm.classes.loaded",
    "jvm.gc.pause",
    "jvm.classes.unloaded",
    "jvm.gc.live.data.size",
    "process.files.open",
    "jvm.buffer.count",
    "jvm.buffer.total.capacity",
    "process.start.time"
  ]
}
```

由于涉及的指标太多，不可能在本文中全部讨论。作为如何使用 `/metrics` 端点的示例，我们仅讨论 `http.server.requests` 指标。

如果您不是简单地请求 `/metrics`，而是请求 `/metrics/{METRICS CATEGORY}`，您将收到关于该指标的更多详细信息。以 `http.server.requests` 为例，GET 请求 `/metrics/http.server.requests` 的返回的数据如下所示：

```
$ curl localhost:8081/actuator/metrics/http.server.requests
{
  "name": "http.server.requests",
  "measurements": [
    { "statistic": "COUNT", "value": 2103 },
    { "statistic": "TOTAL_TIME", "value": 18.086334315 },
    { "statistic": "MAX", "value": 0.028926313 }
  ],
  "availableTags": [
    { "tag": "exception",
      "values": [ "ResponseStatusException",
                  "IllegalArgumentException", "none" ] },
    { "tag": "method", "values": [ "GET" ] },
    { "tag": "uri",
      "values": [
        "/actuator/metrics/{requiredMetricName}",
        "/actuator/health", "/actuator/info", "/ingredients",
        "/actuator/metrics", "/*" ] },
    { "tag": "status", "values": [ "404", "500", "200" ] }
  ]
}
```

该响应最重要的部分是 **measurements** 部分，该部分包括请求类别的所有度量。在本例中，它报告已收到 **2103** 个 HTTP 请求。处理这些请求的总时间为 **18.086334315** 秒，处理其中某个请求所花费的最大时间为 **0.028926313** 秒。

查看这些通用指标很有趣，您还可以使用 **availableTags** 下列出的标记，进一步缩小结果范围。例如，您知道已经有 **2103** 个请求，但还不知道分别有多少个请求导致了 HTTP 200、HTTP 404、HTTP 500。使用状态标记，您可以获取导致 HTTP 404 状态的所有请求的指标，如下所示：

```
$ curl localhost:8081/actuator/metrics/http.server.requests?tag=status:404
{
  "name": "http.server.requests",
  "measurements": [
    { "statistic": "COUNT", "value": 31 },
    { "statistic": "TOTAL_TIME", "value": 0.522061212 },
    { "statistic": "MAX", "value": 0 }
  ],
  "availableTags": [
    { "tag": "exception",
      "values": [ "ResponseStatusException", "none" ] },
    { "tag": "method", "values": [ "GET" ] },
    { "tag": "uri",
      "values": [
        "/actuator/metrics/{requiredMetricName}", "/*" ] }
  ]
}
```

通过使用 **tag** 属性指定标记名称和值，您现在可以看到针对导致 HTTP 404 响应的请求度量。返回数据表明有 **31** 个请求导致 **404**，服务时间为 **0.522061212** 秒。此外，很明显，一些失败的请求是对 `/actuator/metrics/{requiredMetricsName}` 的 GET 请求（尽管不清楚 `{requiredMetricsName}` 路径变量解析成了什么值）。还有一些是另一个路径，由 `/*` 通配符路径匹配处理了。

如果您想知道这些 HTTP 404 响应中有多少响应是对 `/*` 路径的调用？可进一步过滤，只需在请求中增加 **tag**，像这样：

```
% curl "localhost:8081/actuator/metrics/http.server.requests?tag=status:404&tag=uri:/**"
{
  "name": "http.server.requests",
  "measurements": [
    { "statistic": "COUNT", "value": 30 },
    { "statistic": "TOTAL_TIME", "value": 0.519791548 },
    { "statistic": "MAX", "value": 0 }
  ],
  "availableTags": [
    { "tag": "exception", "values": [ "ResponseStatusException" ] },
    { "tag": "method", "values": [ "GET" ] }
  ]
}
```

现在您可以看到，有 30 个请求与 `/**` 匹配，且导致了 HTTP 404 响应。处理这些响应总共花费了 0.519791548 秒。

您还将注意到，当您细化请求时，可用的标记会越少。提供的可用标记仅与显示的指标捕获的请求相匹配。在这种情况下，`exception` 和 `method` 标记都只有一个值：很明显，所有 30 个请求都是 GET 请求，都由于 `ResponseStatusException` 异常导致了状态 404。

浏览 `/metrics` 端点可能是一项棘手的工作，但只要稍加练习，要获得您需要的数据并非不可能。在第 16 章中，您将看到如何通过 **Spring Boot Admin** 来使获取 `/metrics` 端点的数据变得更加容易。

尽管 **Actuator** 端点提供的信息，有助于了解 **Spring Boot** 应用程序的内部运行情况，但它们并不适合人工使用。作为 **REST** 端点，它们旨在供其他应用程序使用，可能是 **UI**。考虑到这一点，让我们看看如何将 **Actuator** 信息，展示在一个用户友好的 **web** 应用程序中。

## 15.3 自定义 Actuator

Actuator 的最大特点之一是可定制，以满足特定应用程序的需求。一些端点本身允许定制，同时，Actuator 本身允许您创建自定义端点。

让我们看一些定制 Actuator 的方法，首先是添加信息到 `/info` 端点。

### 15.3.1 向 /info 端点提供信息

正如您在第 15.2.1 节中所看到的，`/info` 端点开始时是空的，没有信息。但是，您可以通过创建以 `info.` 为前缀的属性，轻松地往里边添加数据。

为属性添加 `info.` 前缀，可以方便的将自定义数据添加到 `/info` 端点，但这并不是唯一的方法。Spring Boot 提供了一个名为 `InfoContributor` 的接口，允许您以代码方式，将任何信息添加到 `/info` 端点的响应数据中。Spring Boot 甚至已经提供了一些实现类，您会发现它们非常有用。

让我们看看如何编写自己的 `InfoContributor`，并向 `/info` 端点添加一些自定义信息。

### 创建自定义 `Info Contributor` 实现类

假设您想在 `/info` 端点中，添加一些有益于 `Taco Cloud` 的简单统计信息。例如，有多少玉米卷已经制作出来了。为此，您可以编写一个实现 `InfoContributor` 接口的实现类，并注入 `TacoRepository`。然后就可以把 `TacoRepository` 能够提供的统计数据，提供给 `/info` 端点使用。以下的程序清单展示了如何实现这样的类。

程序清单 15.3 `InfoContributor` 的自定义实现类

```
package tacos.tacos;
import org.springframework.boot.actuate.info.InfoContributor;
import org.springframework.stereotype.Component;
import java.util.HashMap;
import java.util.Map;
import org.springframework.boot.actuate.info.Info.Builder;
@Component
public class TacoCountInfoContributor implements InfoContributor {
    private TacoRepository tacoRepo;
    public TacoCountInfoContributor(TacoRepository tacoRepo) {
        this.tacoRepo = tacoRepo;
    }
    @Override
    public void contribute(Builder builder) {
        long tacoCount = tacoRepo.count();
        Map<String, Object> tacoMap = new HashMap<String, Object>();
        tacoMap.put("count", tacoCount);
        builder.withDetail("taco-stats", tacoMap);
    }
}
```

实现 `InfoContributor` 接口时，`TacoCountInfoContributor` 需要实现 `contribute()` 方法。此方法传入了一个 `builder` 对象，在该对象上调用 `withDetail()` 以添加详细信息。在上述实现中，通过调用 `TacoRepository` 的 `count()` 方法，查找已经制作了多少玉米卷。然后把这个数字放到一个 `Map` 对象中，最后将标签 `taco-stats` 提供给 `Builder` 对象。`/info` 端点的结果将包括该计数，如下所示：

```
{
  "taco-stats": {
    "count": 44
  }
}
```

如您所见，`InfoContributor` 的实现类可以提供动态统计信息。这与简单地属性加 `info.` 前缀形成对比，虽然简单，但仅限于静态值。

### 将构建信息注入 `/INFO` 端点

Spring Boot 附带了几个 `InfoContributor` 的内置实现，可以自动将信息添加到 `/info` 端点的结果数据中。其中包括 `BuildInfoContributor`，它将项目构建信息添加到 `/info` 端点。这包括项目版本、时间戳等基本信息，以及执行构建的主机和用户。

要使生成信息包含在 `/info` 端点的结果数据中，需要添加 `build-info` 到 Spring Boot Maven 插件执行目标中，如下所示：

```
<build>
  <plugins>
    <plugin>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-maven-plugin</artifactId>
      <executions>
        <execution>
          <goals>
            <goal>build-info</goal>
          </goals>
        </execution>
      </executions>
    </plugin>
  </plugins>
</build>
```

如果使用 Gradle 构建项目，只需将以下行添加到您的 `build.gradle` 文件：

```
springBoot {
    buildInfo()
}
```

在这两种情况下，构建都将在 JAR 或 WAR 文件中生成名为 `build-info.properties` 的文件，`BuildInfoContributor` 会把这些信息附加到 `/info` 端点返回数据中。以下代码片段显示了 `/info` 端点中的构建信息：

```
{
  "build": {
    "artifact": "tacocloud",
    "name": "taco-cloud",
    "time": "2021-08-08T23:55:16.379Z",
    "version": "0.0.15-SNAPSHOT",
    "group": "sia"
  },
}
```

此信息有助于准确了解，正在运行的应用程序的版本、构建时间。通过对 `/info` 端点执行 GET 请求，您将知道您是否正在运行项目的最新版本。

## 公开 GIT 提交信息

假设您的项目使用 Git 进行源代码版本控制，您可能希望在 `/info` 端点中包括 Git 提交信息。要做到这一点，您需要在 Maven 项目 `pom.xml` 中添加以下插件：



```

<build>
  <plugins>
    ...
    <plugin>
      <groupId>pl.project13.maven</groupId>
      <artifactId>git-commit-id-plugin</artifactId>
    </plugin>
  </plugins>
</build>

```

如果您是 **Gradle** 用户，别担心，可以在 **build.gradle** 文件中添加一个同样功能的插件：

```

plugins {
  id "com.gorylenko.gradle-git-properties" version "2.2.4"
}

```

基本上这两个插件做的是相同的事情：它们生成一个构建产物 **git.properties**，包含项目的所有 **git** 元数据。专门的 **InfoContributor** 实现类在运行时扫描该文件，并将其内容作为 **/info** 端点返回数据的一部分。

最简单的形式，**/info** 端点中显示的 **Git** 信息包括，构建应用程序所依据的 **Git** 分支、提交哈希和时间戳：

```

{
  "git": {
    "branch": "main",
    "commit": {
      "id": "df45505",
      "time": "2021-08-08T21:51:12Z"
    }
  },
  ...
}

```

此信息非常明确的描述项目启动时的代码状态。还可以将 **management.info.git.mode** 属性设置为 **full**，您就可以获得项目构建时有关 **Git** 提交的详细信息。

```

management:
  info:
    git:
      mode: full

```

下面的清单显示了完整 **Git** 的示例信息。

清单 15.4 通过 **/info** 端点公开的完整 **Git** 提交信息

```

"git":{
  "local":{
    "branch":{
      "ahead":"8",
      "behind":"0"
    }
  },
  "commit":{
    "id":{
      "describe-short":"df45505-dirty",
      "abbrev":"df45505",
      "full":"df455055daaf3b1347b0ad1d9dca4ebbc6067810",
      "describe":"df45505-dirty"
    },
    "message":{
      "short":"Apply chapter 18 edits",
      "full":"Apply chapter 18 edits"
    },
    "user":{
      "name":"Craig Walls",
      "email":"craig@habuma.com"
    },
    "author":{
      "time":"2021-08-08T15:51:12-0600"
    },
    "committer":{
      "time":"2021-08-08T15:51:12-0600"
    },
    "time":"2021-08-08T21:51:12Z"
  },
  "branch":"master",
  "build":{
    "time":"2021-08-09T00:13:37Z",
    "version":"0.0.15-SNAPSHOT",
    "host":"Craigs-MacBook-Pro.local",
    "user":{
      "name":"Craig Walls",
      "email":"craig@habuma.com"
    }
  },
  "tags":"",
  "total":{
    "commit":{
      "count":"196"
    }
  },
  "closest":{
    "tag":{
      "commit":{
        "count":""
      },
      "name":""
    }
  },
  "remote":{
    "origin":{
      "url":"git@github.com:habuma/spring-in-action-6-samples.git"
    }
  },
  "dirty":"true"
},

```

除了时间戳和 **Git** 提交哈希之外，完整版本还包括，提交代码的用户姓名和电子邮件，以及提交消息和其他信息。允许您精确地识别构建项目时使用的代码。事实上，请注意清单 15.4 中的 **dirty** 字段为 **true**，指出构建目录中有一些未提交的更改，这样的代码状态，真是没有什么情况比这更糟糕了！

## 15.3.2 自定义健康指标

对于一些常见的外部系统，与 **Spring** 应用程序集成时，**Spring Boot** 附带了几个现成的健康状况指示器，可提供它们的健康状况。但有时，您可能发现有一些外部系统，**Spring Boot** 并没有为其提供健康状况指示器。

例如，您的应用程序可能与遗留大型机应用程序集成，应用程序的健康状况，可能会受到遗留系统健康状况的影响。要创建自定义健康状况指示器，需创建一个实现 **HealthIndicator** 接口的类。

实际上，**Taco Cloud** 服务并不需要自定义健康指标，因为 **Spring Boot** 所提供的已经足够了。但为了展示如何开发自定义健康指示器，考虑以下程序清单，它展示了 **HealthIndicator** 的简单实现。其中选择一天中某个时间，随机地展示其健康状况。

程序清单 **15.5 HealthIndicator** 的非正常实现。

```
package tacos.tacos;
import java.util.Calendar;
import org.springframework.boot.actuate.health.Health;
import org.springframework.boot.actuate.health.HealthIndicator;
import org.springframework.stereotype.Component;
@Component
public class WackoHealthIndicator implements HealthIndicator {
    @Override
    public Health health() {
        int hour = Calendar.getInstance().get(Calendar.HOUR_OF_DAY);
        if (hour > 12) {
            return Health
                .outOfService()
                .withDetail("reason",
                    "I'm out of service after lunchtime")
                .withDetail("hour", hour)
                .build();
        }
        if (Math.random() < 0.1) {
            return Health
                .down()
                .withDetail("reason", "I break 10% of the time")
                .build();
        }
        return Health
            .up()
            .withDetail("reason", "All is good!")
            .build();
    }
}
```

这个疯狂的健康指示器首先检查当前时间，如果是中午之后，返回 **OUT\_OF\_SERVICE** 的健康状况，并提供一些详细信息来解释故障原因。

即使是在午餐前，健康指标也有 **10%** 的可能性会报告 **DOWN** 的状态，因为它使用一个随机数来决定它是否正常运行。如果随机数字小于 **0.1**，状态将报告为 **DOWN**。否则，状态为 **UP**。

显然，清单 **15.5** 中的健康状况指示器，在真实环境中任何情况下都不会有用。但是想象一下，如果代码中不是用随机数，而是远程调用某个外部系统并确定其状态。在这种情况下，这将是一个非常有用的健康状况指示器。

### 15.3.3 注册自定义指标

在第 15.2.4 节，我们介绍了如何发送 HTTP 请求，获取 `/metrics` 端点的数据，以使用 Actuator 发布的各种指标。Actuator 提供的指标非常有用，但是并不仅限于这些内置的度量指标。

其实，Actuator 指标由 Micrometer 实现 <https://micrometer.io/>。这是一种与供应商无关的度量，使应用程序能够发布任何想要的指标，并将其显示在第三方监控系统中。包括对 Prometheus、Datadog 和 New Relic 等的支持。

使用 Micrometer 发布度量的最基本方法是，使用 Micrometer 的 MeterRegistry。在 Spring Boot 应用程序中，当需要发布计数器、计时器或其他指标时，您只需发布指标到 MeterRegistry 中。

作为发布自定义度量的示例，假设您希望知道，有多少玉米卷用不同的配料制作出来了。就是，您想知道有多少玉米卷是用莴苣、牛肉或土豆或其他配料作成的。下面的清单显示了，TacoMetrics 是如何使用 MeterRegistry 收集这些信息的。

程序清单 15.6 TacoMetrics 注册有关玉米卷配料的指标。

```
package tacos.tacos;
import java.util.List;
import
org.springframework.data.rest.core.event.AbstractRepositoryEventListener;
import org.springframework.stereotype.Component;
import io.micrometer.core.instrument.MeterRegistry;

@Component
public class TacoMetrics extends AbstractRepositoryEventListener<Taco> {
    private MeterRegistry meterRegistry;
    public TacoMetrics(MeterRegistry meterRegistry) {
        this.meterRegistry = meterRegistry;
    }

    @Override
    protected void onAfterCreate(Taco taco) {
        List<Ingredient> ingredients = taco.getIngredients();
        for (Ingredient ingredient : ingredients) {
            meterRegistry.counter("tacocloud",
                "ingredient", ingredient.getId()).increment();
        }
    }
}
```

如您所见，TacoMetrics 通过其构造函数注入 MeterRegistry。它还扩展了 AbstractRepositoryEventListener，支持拦截 Repository 事件并重写了 onAfterCreate() 方法。这样就可以在保存新的 Taco 对象时随时获取事件通知。

在 onAfterCreate() 方法中，为每种配料声明一个计数器，名称为 ingredient，标签值等于 ingredient 的 ID。如果计数器标记已存在，将重新使用，使计数值递增，这表示另一个玉米卷已经为这种配料添加过标识。

创建了几个 taco 之后，就可以开始查询 /metrics 端点了。对 /metrics/tacocloud 发关 GET 请求，会产生一些未过滤的度量计数：

```
$ curl localhost:8087/actuator/metrics/tacocloud
{
  "name": "tacocloud",
  "measurements": [ { "statistic": "COUNT", "value": 84 }
],
  "availableTags": [
    {
      "tag": "ingredient",
      "values": [ "FLTO", "CHED", "LETC", "GRBF",
                  "COTO", "JACK", "TMT0", "SLSA" ]
    }
  ]
}
```

**measurements** 下的计数值在这里意义不大，因为它是计算所有配料的所有数据的总和。但假设您想知道有多少玉米卷由玉米面制成（**FLTO**），您需要做的就是指定成分值为 **FLTO** 的标记：

```
$ curl localhost:8087/actuator/metrics/tacocloud?tag=ingredient:FLTO
{
  "name": "tacocloud",
  "measurements": [
    { "statistic": "COUNT", "value": 39 }
  ],
  "availableTags": []
}
```

现在很明显，**39** 个玉米卷的配料是玉米面。

### 15.3.4 创建自定义端点

乍一看，您可能认为 **Actuator** 的端点实现，和 **Spring MVC** 的 **Controller** 一样。正如您将在第 18 章中看到的，端点可以作为 **JMX MBean**，还可以通过 **HTTP** 访问。因此，对于这些端点来说，并不仅仅是 **Controller** 类。

事实上，**Actuator** 端点的定义与 **Controller** 完全不同。不同于用 **@Controller** 或 **@RestController** 注解的类，**Actuator** 端点使用带有 **@Endpoint** 注解的类定义。

更重要的是，不再使用 **HTTP** 命名注解，诸如 **@GetMapping**、**@PostMapping** 或 **@DeleteMapping**。**Actuator** 端点用 **@ReadOperation**、**@WriteOperation** 和 **@DeleteOperation** 注解。这些注解并不意味着任何特定的通信机制，事实上，允许 **Actuator** 通过各种通信机制进行通信，**HTTP** 和 **JMX** 开箱即用。

为了演示如何编写自定义 **Actuator** 端点，请参考如下的 **NotesEndpoint**。

程序清单 15.7 记录便笺的自定义端点。

```
package tacos.ingredients;
import java.util.ArrayList;
import java.util.Date;
import java.util.List;
import org.springframework.boot.actuate.endpoint.annotation.DeleteOperation;
import org.springframework.boot.actuate.endpoint.annotation.Endpoint;
import org.springframework.boot.actuate.endpoint.annotation.ReadOperation;
import org.springframework.boot.actuate.endpoint.annotation.WriteOperation;
import org.springframework.stereotype.Component;
import lombok.Getter;
import lombok.RequiredArgsConstructor;

@Component
@Endpoint(id="notes", enableByDefault=true)
public class NotesEndpoint {

    private List<Note> notes = new ArrayList<>();

    @ReadOperation
    public List<Note> notes() {
        return notes;
    }

    @WriteOperation
    public List<Note> addNote(String text) {
        notes.add(new Note(text));
        return notes;
    }

    @DeleteOperation
    public List<Note> deleteNote(int index) {
        if (index < notes.size()) {
            notes.remove(index);
        }
        return notes;
    }

    @RequiredArgsConstructor
    private class Note {
        @Getter
        private Date time = new Date();
        @Getter
        private final String text;
    }
}
```

该端点是一个简单的便笺端点。在该端点中，用户可以使用写入操作增加一个便笺，使用读取操作读取便笺列表，然后使用删除操作删除便笺。诚然，这个端点对于 **Actuator** 来说不是很有用，但是当您考虑开箱即用的 **Actuator** 端点会涉及太多领域时，很难想出一个实用的自定义示例。

可以看到，**NotesEndpoint** 类合使用 **@Component** 注解，以便通过 **Spring** 的组件扫描获取，并在 **Spring** 中实例化为 **bean**。它还添加了注解 **@Endpoint**，使其成为 ID 为 **notes** 的 **Actuator** 端点。并且它是默认启用的，这样您就不需要通过将其包含在 `management.web.endpoints.web.exposure.include` 中来显式启用它。

如您所见，**NotesEndpoint** 提供了如下操作：

- **notes()** 方法用 **@ReadOperation** 注解。调用时，它将返回可用的便笺列表。使用 **HTTP** 时，这意味着它将处理对 `/actuator/notes` 的 **HTTP GET** 请求，并响应 **JSON** 形式的便笺列表。
- **addNote()** 方法用 **@WriteOperation** 注解。当被调用时，它将根据给定文本创建新便笺，并将其添加到列表中。使用 **HTTP** 时，它处理 **POST** 请求，其中请求体是包含文本属性的 **JSON** 对象。处理完成后返回列表的当前状态。
- **deleteNote()** 方法用 **@DeleteOperation** 注解。调用时，它将删除给定索引的便笺。使用 **HTTP** 时，该端点处理 **DELETE** 请求，在请求参数中给出便笺索引。

要查看实际运行效果，可以使用 **curl** 测试新端点。首先，添加几个便笺，使用两个单独的 **POST** 请求：

```
$ curl localhost:8080/actuator/notes \
  -d '{"text": "Bring home milk"}' \
  -H 'Content-type: application/json'
[{"time": "2020-06-08T13:50:45.085+0000", "text": "Bring home milk"}]

$ curl localhost:8080/actuator/notes \
  -d '{"text": "Take dry cleaning"}' \
  -H 'Content-type: application/json'
[{"time": "2021-07-03T12:39:13.058+0000", "text": "Bring home milk"},
{"time": "2021-07-03T12:39:16.012+0000", "text": "Take dry cleaning"}]
```

如您所见，每次发布新便笺时，端点都会返回最新便笺列表。如果以后要查看便笺列表，可以执行以下操作：

```
$ curl localhost:8080/actuator/notes
[{"time": "2021-07-03T12:39:13.058+0000", "text": "Bring home milk"},
{"time": "2021-07-03T12:39:16.012+0000", "text": "Take dry cleaning"}]
```

如果您决定删除其中一个便笺，则发送带有索引参数的 **DELETE** 请求：

```
$ curl localhost:8080/actuator/notes?index=1 -X DELETE
[{"time": "2021-07-03T12:39:13.058+0000", "text": "Bring home milk"}]
```

需要注意的是，尽管这里只演示了使用 **HTTP** 如何与端点交互，但它还作为 **MBean** 公开了，可以使用任何您熟悉的 **JMX** 客户端进行访问。如果您想将其限制为仅公开 **HTTP** 端点，您可以使用 **@WebEndpoint** 而不是 **@endpoint** 注解端点类：

```
@Component
@WebEndpoint(id="notes", enableByDefault=true)
public class NotesEndpoint {
    ...
}
```

同样，如果您喜欢只使用 **MBean** 的端点，请使用 **@JmxEndpoint** 注解。



## 15.4 保护 Actuator

对于 Actuator 提供的信息，可能您并不想被间谍窥探到。此外，由于 Actuator 提供了可以更改环境属性，以及调整日志记录级别的操作，那么使只允许具有适当访问权限的客户，才能使用 Actuator 是必要的。

尽管确保 Actuator 的安全非常重要，但 Actuator 并没有提供安全措施的责任。相反，您需要使用 Spring Security 来保护 Actuator。因为 Actuator 端点只是应用程序中的路径，这与应用程序中的任何其他路径没什么区别，所以确保 Actuator 的安全措施，与保证应用程序其他路径的安全措施相比，没有什么独特之处。我们在第 4 章中讨论的所有内容都适用 Actuator 端点。

因为所有的 Actuator 端点都集中在路径 `/actuator` 下（或者其他路径，如果设置了 `management.endpoints.web.base-path` 属性），很容易将授权规则应用于 Actuator 端点。例如，要求用户具有 `ROLE_ADMIN` 权限，以调用 Actuator 端点，您可以重写 `WebSecurityConfigurerAdapter` 的 `configure()` 方法，如下所示：

```
@Override
protected void configure(HttpSecurity http) throws Exception {
    http
        .authorizeRequests()
        .antMatchers("/actuator/**").hasRole("ADMIN")
        .and()
        .httpBasic();
}
```

这要求，所有请求都要来自具有 `ROLE_ADMIN` 权限的用户。另外还配置了 HTTP 基本身份验证，以便客户端应用程序，可以在请求头 `Authorization` 属性中，传输编码过的身份验证信息。

以这种方式保护 Actuator 的真正问题是，端点的路径硬编码为 `/actuator/**` 了。如果修改了 `management.endpoints.web.base-path` 属性，它将不再工作。为解决这个问题，Spring Boot 还提供了 `EndpointRequest`，这是一个请求匹配器类。它更容易使用，而且不依赖于给定的字符串路径。使用 `EndpointRequest`，您可以对 Actuator 端点应用相同的安全配置，而无需硬编码 `/actuator/**` 路径：

```
@Override
protected void configure(HttpSecurity http) throws Exception {
    http
        .requestMatcher(EndpointRequest.toAnyEndpoint())
        .authorizeRequests()
        .anyRequest().hasRole("ADMIN")
        .and()
        .httpBasic();
}
```

`EndpointRequest.toAnyEndpoint()` 方法返回一个请求匹配器，该匹配器匹配任何 Actuator 端点。如果要从中排除某些端点，可以调用 `excluding()`，并按名称指定它们：

```
@Override
protected void configure(HttpSecurity http) throws Exception {
    http
        .requestMatcher(EndpointRequest.toAnyEndpoint().excluding("health", "info"))
        .authorizeRequests()
        .anyRequest().hasRole("ADMIN")
        .and()
        .httpBasic();
}
```

另一方面，您如果希望对少数 **Actuator** 进行保护，您可以通过调用 `to()` 方法，而不是 `toAnyEndpoint()` 方法：

```
@Override
protected void configure(HttpSecurity http) throws Exception {
    http
        .requestMatcher(EndpointRequest.to("beans", "threaddump", "loggers"))
        .authorizeRequests()
        .anyRequest().hasRole("ADMIN")
        .and()
        .httpBasic();
}
```

这将 **Actuator** 的安全性仅限于 `/beans`、`/threaddump` 和 `/loggers` 端点。所有其他 **Actuator** 端点完全开放。

## 15.5 总结

- Spring Boot Actuator 提供了几个端点，可做为 HTTP 和 JMX MBean，让您可以窥探 Spring Boot 应用程序的内部工作情况。
- 默认情况下，大多数 Actuator 端点处于禁用状态，但可以有选择地通过设置 `management.endpoints.web.exposure.include` 和 `management.endpoints.web.exposure.exclude` 调整启用状态。
- 某些端点（如 `/logger` 和 `/env` 端点）允许写操作，动态更改正在运行的应用程序的配置。
- 有关应用程序的构建和 Git 提交的详细信息，可以通过 `/info` 端点获取。
- 应用程序的运行状况，可能会受到自定义运行状况指示器的影响，运行状况来自于外部集成应用程序的运行状况。
- 自定义应用程序度量指标，可以通过 Micrometer 注册。Micrometer 支撑了 Spring Boot 应用程序与几个流行指标引擎集成，包括 Datadog、New Relic 和 Prometheus 等。
- Actuator web 端点可以使用 Spring Security 进行保护，就像保护任何 Spring Web 应用程序中的其他端点那样。

## 第 16 章 管理 Spring

本章内容

- 设置 Spring Boot Admin
- 注册客户端应用程序
- 使用 Actuator 端点
- 保护 Admin 服务器的安全

俗话说，一图胜千言。对于众多的应用使用者来说，一个对用户友好的 **Web** 界面抵得上一千次 **API** 调用。别误解我，我是个命令行迷，非常喜欢使用 **curl** 和 **HTTPie** 来进行 **REST API** 调用。但是，有时手动键入命令行以调用 **REST**，然后目视检查返回结果，效率可能低于只需单击链接并在 **web** 浏览器中读取结果。

在上一章中，我们讨论了 **Spring Boot Actuator**。这些 **HTTP** 端点返回 **JSON** 响应数据，可以任意使用它们。在本章中，我们将了解如何创建底层基于 **Actuator** 的前端用户界面（**UI**）。使 **Actuator** 更易于使用，且可以实时捕获难以直接从 **Actuator** 获取的数据。

## 16.1 使用 SpringBoot Admin

使用 Actuator 端点的数据，构建一个易于查看的应用是不是有意义？开发这样的 web 应用是不是很难？我被问过多次这样的问题。我回答说，Actuator 端点只是一个 REST API，因此，这是可以做的。但为什么要自己做呢？codecentric AG (<https://www.codecentric.de/>)，一家总部位于德国的软件和咨询公司，已经为您做了这项工作。

Spring Boot Admin 是一个管理型前端 web 应用程序，它使 Actuator 端点更易于人类使用。它分为两个主要组件：Spring Boot Admin 服务端及其客户端。服务端收集和管理数据，显示从一个或多个 Spring Boot 应用程序获得的 Actuator 数据。提供数据的应用程序，被标识为 Spring Boot Admin 客户端，如图 16.1所示：

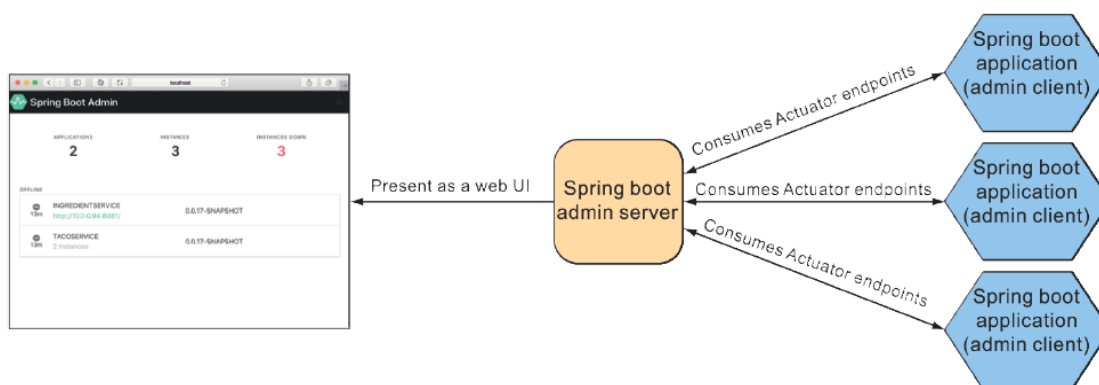


图 16.1 Spring Boot Admin 服务端，使用来自一个或多个 Spring Boot 应用程序的 Actuator 端点数据，并显示在基于 web 的 UI 中。

您要将 Taco Cloud 中的每个应用程序（微服务），都注册为 Spring Boot Admin 的客户端。首先，您需要设置 Spring Boot Admin 服务端接收每个客户端的 Actuator 信息。

### 16.1.1 创建 Admin 服务端

要启用 Admin 服务端，首先需要创建一个新的 Spring Boot 应用程序，并将 Admin 服务端依赖项添加到项目的构建中。Admin 服务端通常是作为独立应用程序使用的，与其他应用程序分开。所以，最简单的方法是，使用 Spring Boot Initializr 创建一个新的 Spring Boot 项目，并选中标记为 Spring Boot Admin（服务端）的复选框。这样就会在 `<dependencies>` 中包含以下依赖：

```
<dependency>
  <groupId>de.codecentric</groupId>
  <artifactId>spring-boot-admin-starter-server</artifactId>
</dependency>
```

接下来，您需要通过在主配置类上添加 `@EnableAdminServer` 注解，以启用 Admin 服务端，如下所示：

```
package tacos.bootadmin;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import de.codecentric.boot.admin.server.config.EnableAdminServer;

@SpringBootApplication
@EnableAdminServer
public class BootAdminServerApplication {
    public static void main(String[] args) {
        SpringApplication.run(BootAdminServerApplication.class, args);
    }
}
```

最后，因为 Admin 服务端可能不是唯一一个在本地运行的应用程序，您应该设置共用使用一个唯一的端口（不能是端口0）。在这里，我选择端口 9090 作为 Spring Boot Admin 服务端的端口：

```
server:
  port: 9090
```

注意：与微服务架构中的任何其他服务应用程序一样，在 Spring Boot 应用程序中，`server.port` 属性可以在不同的配置文件中设置不同的值，最终使用的端口值会由底层平台机制决定。

现在，您的 Admin 服务端已准备就绪。如果您在此时启动它，并在您的 web 浏览器访问 <http://localhost:9090>，您会看到类似于图 16.2 的页面。

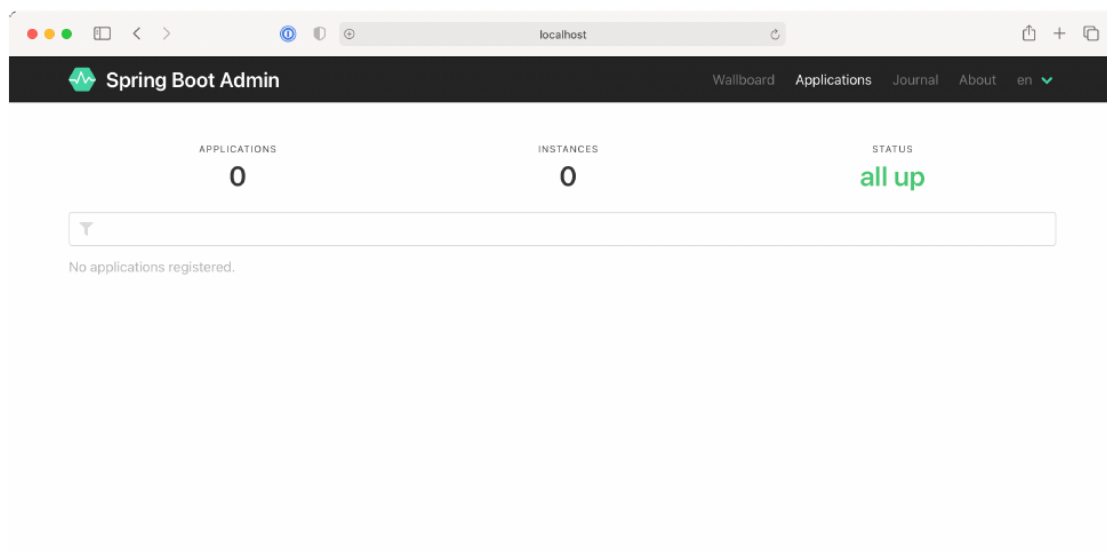


图 16.2 显示在 **Spring Boot Admin UI** 中的新创建的服务器。尚未登记任何应用。

如您所见，Spring Boot Admin 显示，启动了 0 个应用程序的 0 个实例。但是当您看到，计数器下方显示的信息 **No Applications Registered**，就明白现在这些计数值根本毫无意义。要使用 Admin 服务端有用，您需要向它注册一些应用。

## 16.1.2 注册 Admin 客户端

Admin 服务端，是一个独立于其他 Spring Boot 应用程序的应用程序。要显示来自于其他应用程序的 Actuator 数据，您必须以某种方式使 Admin 服务端感知到其他应用程序。所在要注册 Spring Boot Admin 客户端，有如下两种方式：

- 每个应用程序都向 Admin 服务端显式注册自己。
- 管理服务器通过 Eureka 服务注册中心自动发现客户端。

我们将重点讨论，如何把各个 Spring Boot 应用程序配置为 Spring Boot Admin 客户端，以便可以在 Admin 服务端上注册自己。有关使用 Eureka 的详细信息，请参阅 Spring Cloud 文档 <https://docs.spring.io/spring-cloud-netflix/docs/current/reference/html/> 或由约翰·卡内尔和伊利·瓦伊卢波·桑切斯合著的《Spring 微服务实战》第二版。

为了把 Spring Boot 应用程序注册为 Admin 客户端，您必须在其构建时包含 Spring Boot Admin 客户端 starter。您可以使用 Initializr 轻松添加依赖关系，只需选中 Spring Boot Admin (Client) 的复选框。或者您可以为 Maven 构建的 Spring Boot 应用程序添加如下的 `<dependency>`：

```
<dependency>
  <groupId>de.codecentric</groupId>
  <artifactId>spring-boot-admin-starter-client</artifactId>
</dependency>
```

客户端依赖库就绪后，还需要配置 Admin 服务端位置，以便客户端可以自行注册。为此，您需要设置 `spring.boot.admin.client.url` 属性，将其设置为 Admin 服务端的根 url：

```
spring:
  boot:
    admin:
      client:
        url: http://localhost:9090
```

请注意，还设置了 `spring.application.name` 属性。此属性在多个 Spring 项目中使用，用于识别应用程序。在本例中，作为一个名称标签在 Admin 服务端注册。



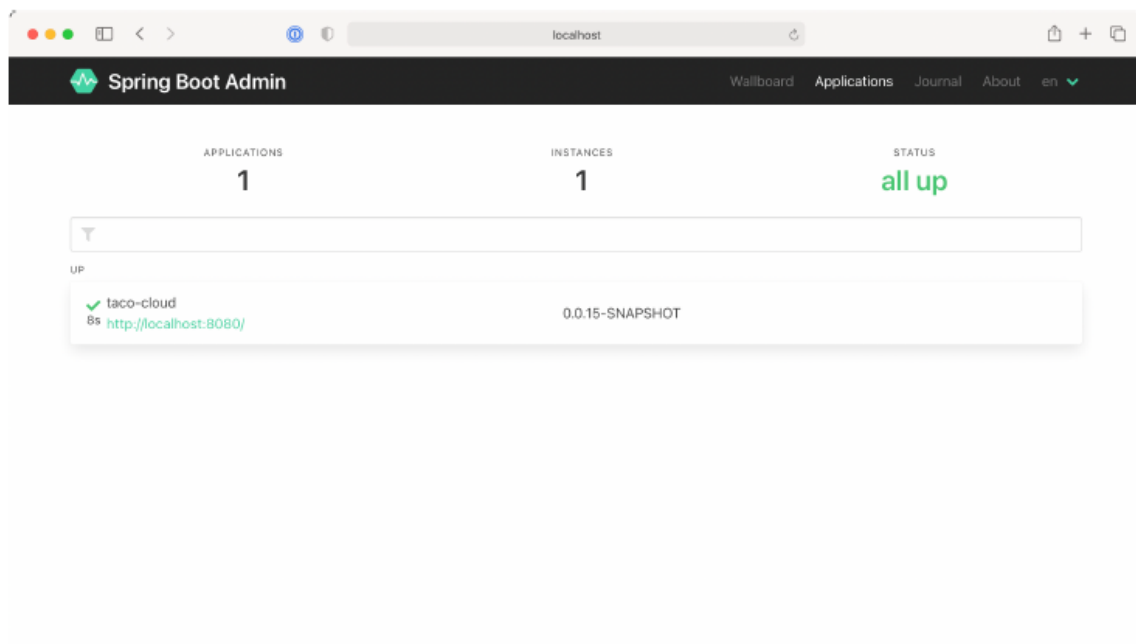


图 16.3 Spring Boot Admin UI 显示了一个注册的应用程序。

虽然在图 16.3 中，没有太多关于 Taco Cloud 服务的信息，但仍然显示了应用程序的正常运行时间，以及 Spring Boot Maven 插件配置的构建产物（正如我们在第 16.3.1 节中讨论的），还有构建产物的版本号。请放心，在您单击 Admin 服务端中的应用之后，还有很多其他运行时细节可以查看。我们将在第 16.2 节中，更深入地了解 Admin 服务端的功能。

现在，您已经向 Admin 服务端注册了一些 Taco Cloud 服务，让我们看看 Admin 服务端都提供了什么。

## 16.2 深入 Admin 服务端

一旦您将所有 **Spring Boot** 应用程序都注册为 **Admin** 客户端，就可以在 **Admin** 服务端就查看到大量信息，这就可以确定每个应用程序内都发生了什么。这些信息包括：

- 一般健康信息
- 通过 **Micrometer** 和 `/metrics` 端点发布的度量指标
- 环境属性信息
- 包和类的日志记录级别

事实上，几乎所有 **Actuator** 公开的内容，都可以在 **Admin** 服务端界面中查看，而且显示格式更人性化。这包括图形化界面和过滤器，以帮助定位所需信息。**Admin** 服务端显示的信息量，比我们在本章中将要讨论的内容要丰富得多。让我用这一章的其余部分来介绍 **Admin** 服务端的一些亮点。

## 16.2.1 查看普通应用程序运行状况和信息

如第 15.2.1 节所述，Actuator 通过 `/health` 和 `/info` 端点，显示应用程序运行状况和一般信息。Admin 服务端在 **Details** 选项卡下显示该信息，如图 16.4 所示。

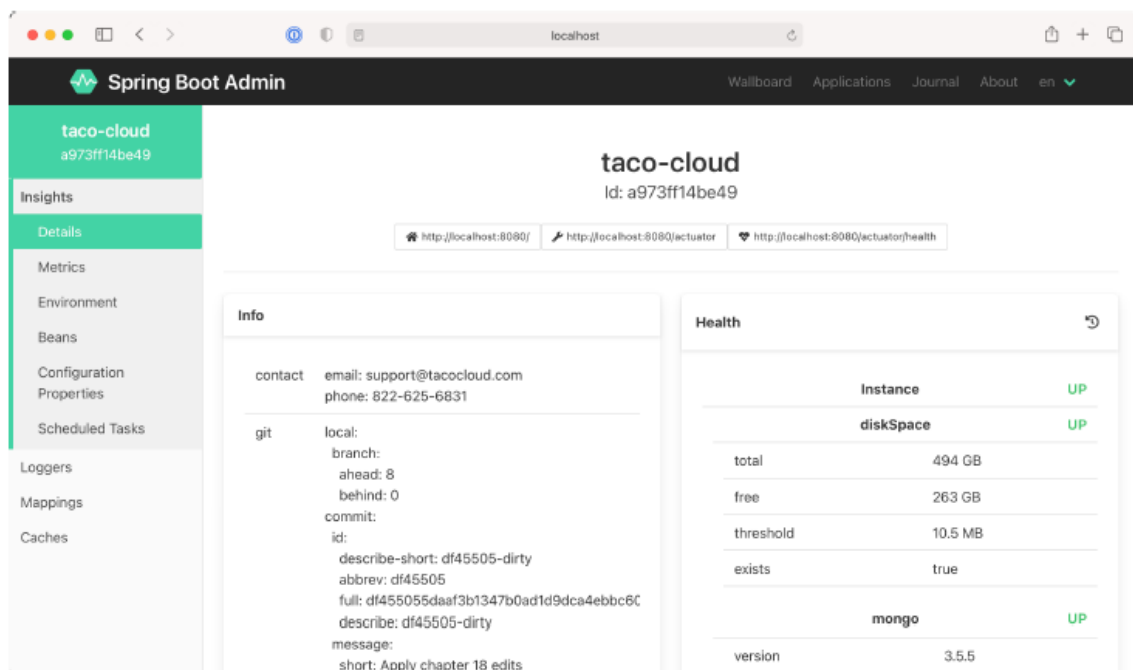


图 16.4 Spring Boot Admin UI 的 **Details** 选项卡显示了应用的一般运行状况和信息。

如果您滚动查看 Detail 选项卡中的 **Health** 和 **Info** 部分，您将发现来自应用程序 JVM 的统计信息，包括图形化显示的内存、线程和处理器使用情况（图 16.5）。

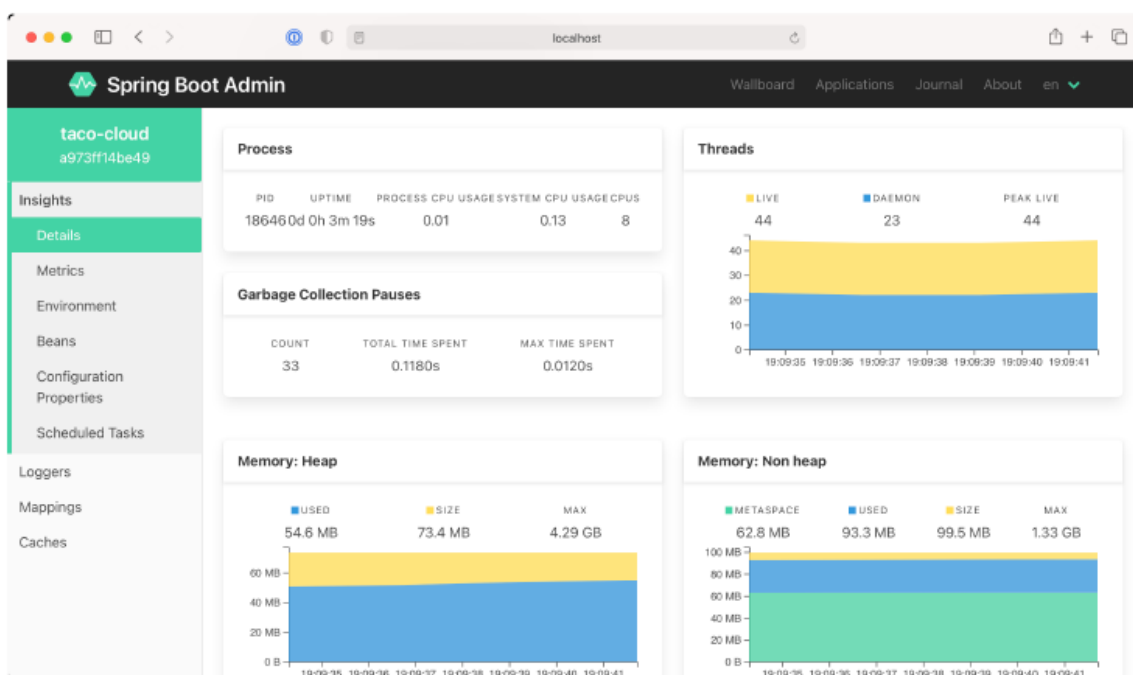


图 16.5 当您向下滚动 **Details** 选项卡时，您可以查看额外的 **JVM** 内部信息，包括处理器、线程和内存的统计信息。

图表中显示的信息，以及 **Processes** 和 **Garbage Collection Pauses** 两部分的信息，可以提供有关应用程序如何利用 **JVM** 资源的相关情况。

## 16.2.2 观察关键指标

在 Actuator 的所有端点中，`/metrics` 端点提供的信息，可能是人类可读性最差的信息。但是 Admin 服务端，让我们很容易使用应用程序中生成的度量指标，其 UI 位于 Metrics 选项卡之下。

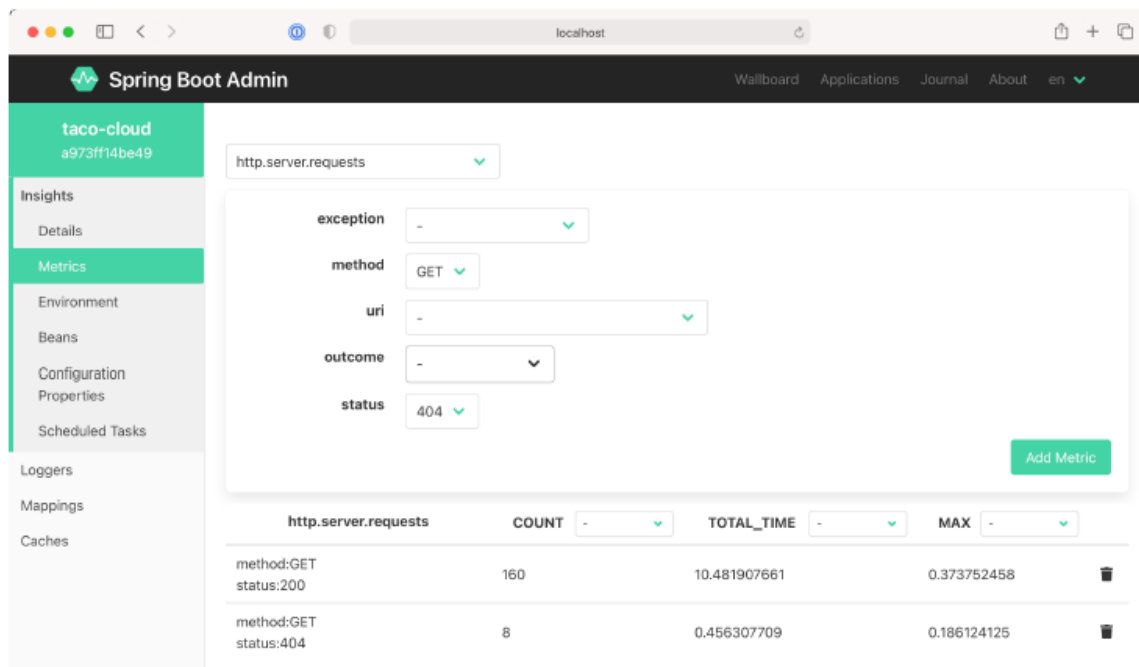


图 16.6 在 Metrics 选项卡上，您可以对通过应用程序发布的任何度量设置监视 `/metrics` 端点。

最初，Metrics 选项卡不显示任何度量指标。但是在顶部，允许您关注任何指标，可以设置一个或多个进行观察。

在图 16.6 中，我在 `http.server.requests` 下设置了两个度量类别。第一个在 `/ingredients` 端点收到 HTTP GET 请求，并返回状态 200（OK）时报告。第二个在导致 HTTP 400（NOT FOUND）响应的请求时进行报告。

这些指标的好处（事实上，几乎所有显示在 Admin 管理端的指标都是如此）是它们实时的，它们会自动更新，而无需刷新页面。

### 16.2.3 检查环境属性

Actuator 的 `/env` 端点，返回来自于所有属性源的 Spring 可用的属性。尽管 JSON 格式的响应人为读取并不是太困难，但 Admin 服务端以更美观的形式，在 Environment 选项卡下进行展示（图 16.7）。

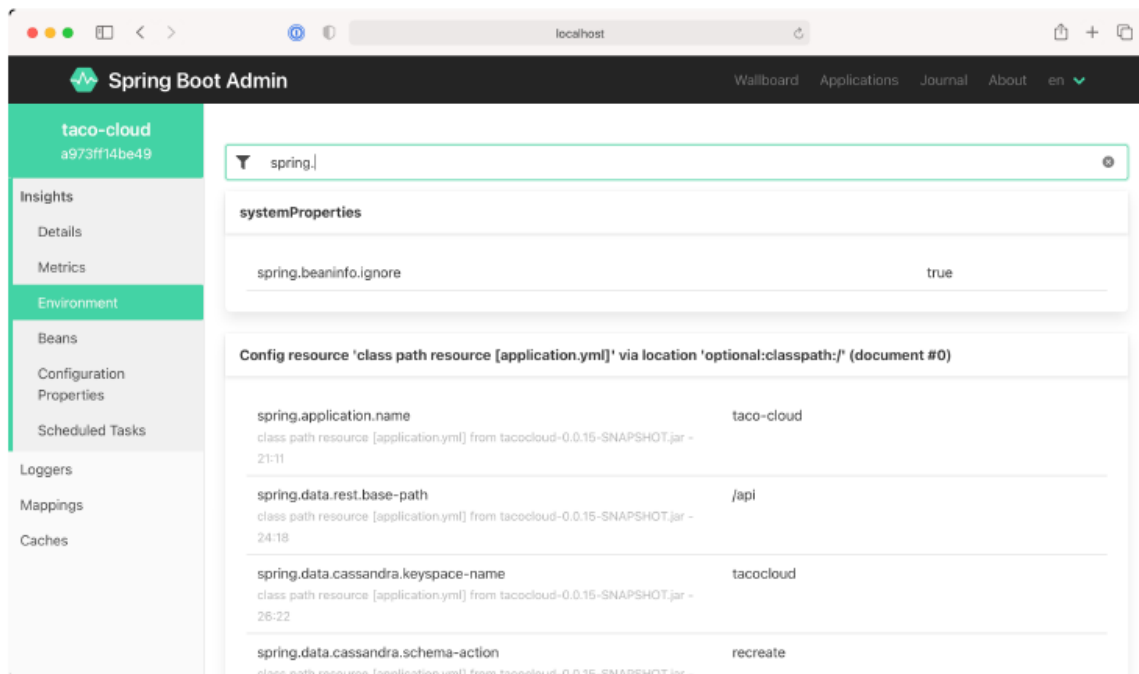


图 16.7 Environment 选项卡显示环境属性，还包括覆盖和筛选这些属性值的选择。

因为可能有数百个属性，所以可以根据属性名称或值进行筛选。图 16.7 显示了筛选出的属性列表，其名称或值包含文本“spring.”。Admin 服务端也允许您设置或覆盖属性，使用 Environment Manager 部分提供的表单进行。

## 16.2.4 查看并设置 log 级别

Actuator 的 `/loggers` 端点，有助于了解或修改应用程序的日志记录级别。Admin 服务端的 Loggers 选项卡，为 `/loggers` 端点添加了一个易于使用的 UI。在图 16.8 中，显示了按名称 `org.springframework.boot` 筛选的日志记录列表。

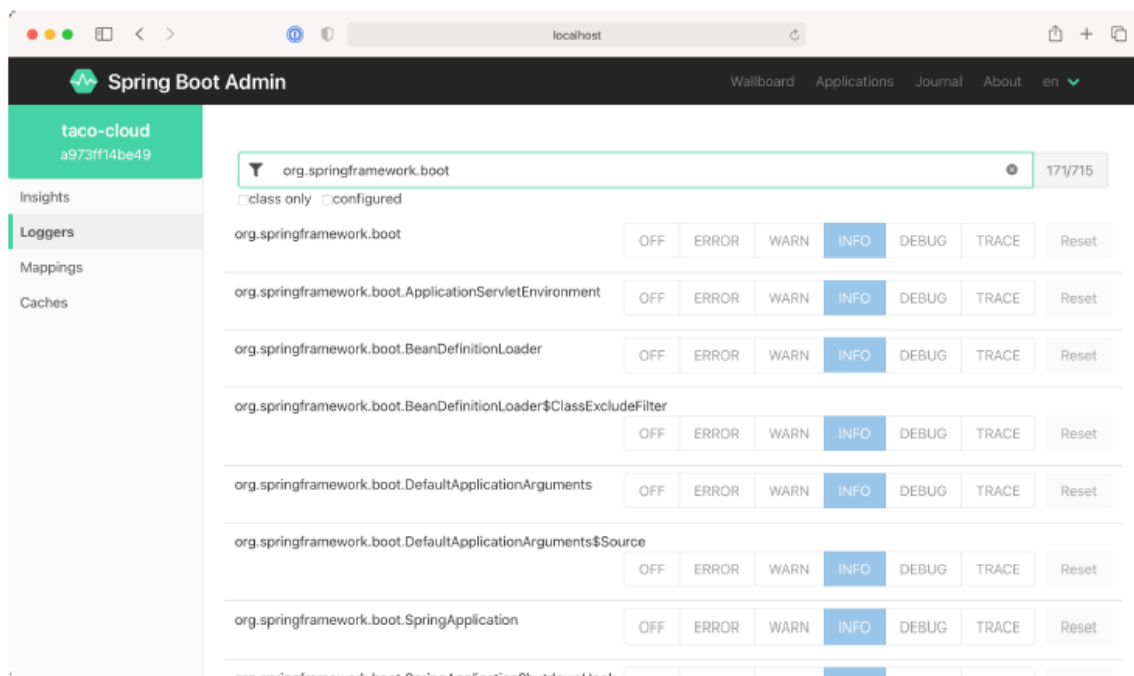


图 16.8 Loggers 选项卡显示应用程序中，包和类的日志记录级别，并允许您覆盖这些级别。

默认情况下，Admin 服务端显示所有包和类的日志记录级别。可以按名称（仅适用于类）或日志记录级别（显示配置，而不是从根记录器继承）筛选这些内容。

## 16.3 保护 Admin 服务端

正如我们在前一章中所讨论的那样，**Actuator** 的端点是不应该随便使用的。端点暴露的应用程序详细信息，只有管理员才应该看见。此外，一些端点允许修改配置，这当然不应该随便暴露给任何人。

正如安全性对 **Actuator** 非常重要一样，它对 **Admin** 服务端也同样重要。此外，如果 **Actuator** 端点需要身份验证，则 **Admin** 服务端需要知道，能够访问这些端点的凭据。让我们看看如何为 **Admin** 服务端添加一点安全性。我们将从要求身份验证开始。



### 16.3.1 在 Admin 服务端中启用登录

Admin 服务端默认没有安全措施，所以给 Admin 服务端添加安全保护是一个好主意。Admin 服务端本质就是一个 Spring Boot 应用程序，所以您可以像保护任何其他 Spring Boot 应用程序一样进行设置，而且与任何受 Spring Security 保护的应用程序一样，您可以自由决定哪种安全方案最适合您的需要。

最简单的方式，您只需将 **Spring Boot Security starter** 添加到 Admin 服务端构建中。若使用 **Initializr**，可通过选中 **Security** 复选框，或者添加以下内容到项目 **pom.xml** 文件中：

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-security</artifactId>
</dependency>
```

这样您就不必一直查看 Admin 服务端的日志来查找随机生成的密码了。您可以在 **application.yml** 中配置一个简单的管理用户名和密码：

```
spring:
  security:
    user:
      name: admin
      password: 53cr3t
```

现在，当在浏览器访问 Admin 服务端时，会弹出 **Spring Security** 默认登录表单，提示您输入用户名和密码。输入上边代码片段中的 **admin** 和 **53cr3t**，就可以登录进入。

默认情况下，**Spring Security** 将在 **Spring Boot Admin** 服务端上启用 **CSRF**，这将防止客户端应用程序在 Admin 服务端上注册。因此，我们需要花一点时间禁用 **CSRF** 的安全配置，如下所示：

```
package tacos.admin;
import org.springframework.context.annotation.Bean;
import org.springframework.security.config.annotation.web.reactive.EnableWebFluxSecurity;
import org.springframework.security.config.web.server.ServerHttpSecurity;
import org.springframework.security.web.server.SecurityWebFilterChain;

@EnableWebFluxSecurity
public class SecurityConfig {

    @Bean
    public SecurityWebFilterChain filterChain(ServerHttpSecurity http) throws Exception {
        return http
            .csrf()
            .disable()
            .build();
    }
}
```

当然，这是一种极其基础的安全配置，我建议您参考第 5 章，了解 **Spring Security** 为管理服务器提供的更丰富的安全方案。

## 16.3.2 使用 Actuator 进行认证

在第 15.4 节中，我们讨论了如何使用 HTTP 基本验证，保护 Actuator 端点。这样做，就可以把所有不知道密码的人挡在门外。不幸的是，这也意味着 Admin 服务端也将无法使用 Actuator 端点，除非它知道用户名和密码。但是 Admin 服务端将如何获得那些凭据呢？

Admin 客户端应用程序，可以向 Admin 服务端提供其凭据。可以直接向 Admin 服务端注册，也可以通过 Eureka 被发现。如果应用程序直接向 Admin 服务端注册，那么它可以在注册时，将其凭据发送到 admin 服务端。您需要配置一些属性来启用该功能。

属性 `spring.boot.admin.client.instance.metadata.user.name` 和

`spring.boot.admin.client.instance.metadata.user.password` 指定的凭据信息，Admin 服务端可以用来访问应用程序的 Actuator 端点。下面是 `application.yml` 中的代码片段，展示了如何设置这些属性：

```
spring:
  boot:
    admin:
      client:
        url: http://localhost:9090
        username: admin
        password: 53cr3t
```

必须在每个注册的应用程序中设置用户名和密码属性，它本身与 Admin 服务端连接。且给定的值必须与 Actuator 端点 HTTP 基本身份验证中，所需的用户名和密码匹配。在本例中，它们被设置为 `admin` 和 `password`，这是为访问 Actuator 端点所需的凭据。

## 16.4 总结

- Spring Boot Admin 服务端消费一个或多个 Spring Boot 应用程序 Actuator 端点的数据，并以用户友好的方式显示在 web 界面中。
- Spring Boot 应用程序可以主动将自己注册为 Admin 客户端，也可以由 Admin 服务端通过 Eureka 发现它们。
- 与捕获应用程序快照的 Actuator 端点不同，Admin 服务端能够显示应用程序的实时状态。
- Admin 服务端可以方便地过滤 Actuator 端点数据，并在图形中直观地显示。
- 因为 Admin 服务端本质就是一个 Spring Boot 应用程序，所以可以应用任何 Spring Security 提供的安全措施。

## 第 17 章 使用 JMX 监控 Spring

本章内容

- 使用 Actuator 端点 MBean
- 将 Spring Bean 公开为 MBean
- 发布通知

十多年来，Java 管理扩展（JMX）一直是监视和管理 Java 应用程序的标准方法。通过公开托管组件即 MBean（托管bean），外部 JMX 客户端可以执行调用、检查属性和监视来自 MBeans 的事件。

我们开始探索 Spring 和 JMX，先看看 Actuator 端点是如何暴露为 MBean 的。

## 17.1 使用 Actuator MBean

默认情况下，所有 Actuator 端点都作为 MBean 公开了。但从 Spring Boot 2.2 开始，JMX 默认是禁用的。要在您的 Spring Boot 应用程序中开始 JMX，您可以设置 `spring.jmx.enabled` 属性为 `true`。在 `application.yml` 文件中，像这样：

```
spring:
  jmx:
    enabled: true
```

设置了此属性后，JMX 就启用了，所有 Actuator 端点都作为 MBean 公开了。您可以使用 JMX 客户端连接这些 Actuator 端点 MBean。如图 17.1 所示，使用 Java JDK 附带的 JConsole 进行连接，您可以在 `org.springframework.boot` 下找到 MBean 列表。

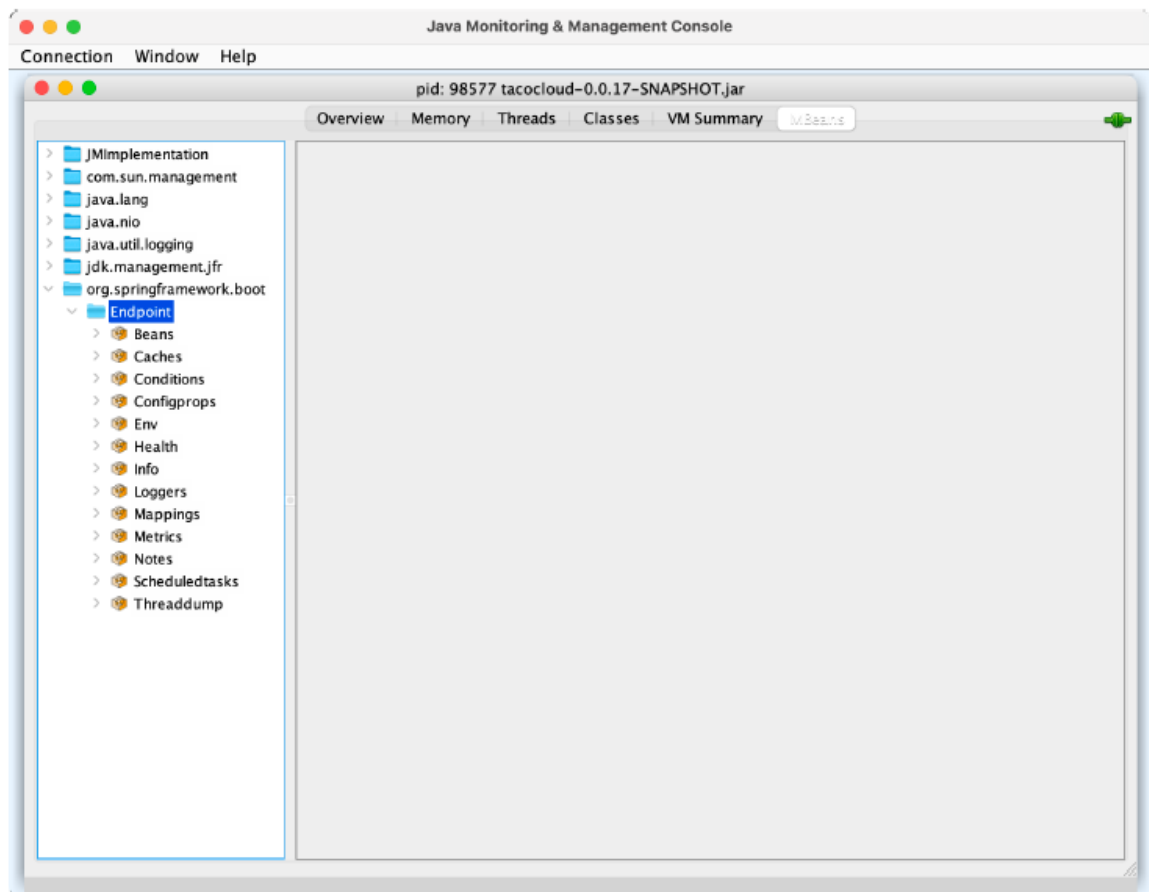


图 17.1 Actuator 端点自动显示为 JMX MBean。

Actuator 端点 MBean 的一个优点是，默认情况下它们都是公开的。没有必要像使用 HTTP 那样显式指定启用其中任何一个。但是，您可以通过设置 `management.endpoints.jmx.exposure.include` 和 `management.endpoints.jmx.exposure.exclude` 来调整范围。例如，将 Actuator 端点 MBean 限制为仅公开 `/health`、`/info`、`/bean` 和 `/conditions` 端点，可以设置 `management.endpoints.jmx.exposure.include` 只包括以下内容：

```
management:
  endpoints:
    jmx:
      exposure:
        include: health,info,bean,conditions
```

或者，如果只有少数几个要排除，则可以设置 `management.endpoints.jmx.exposure.exclude`，像下面这样：

```
management:
  endpoints:
    jmx:
      exposure:
        exclude: env,metrics
```

在这里，您可以使用 `management.endpoints.jmx.exposure.exclude` 来排除 `/env` 和 `metrics` 端点。所有其他 `Actuator` 端点仍将作为 `MBean` 公开。

要在 `JConsole` 中的一个 `MBean` 上执行调用操作，展开左侧树中的端点 `MBean`，然后选择所需的操作。

例如，如果您想检查 `tacos.ingredients` 包的日志记录级别，展开 `Loggers MBean` 并单击名 `loggerLevels`，如图 17.2 所示。在右上角的表单中，输入包名称 `tacos.ingredients`，然后单击 `LoggerLevel` 按钮。

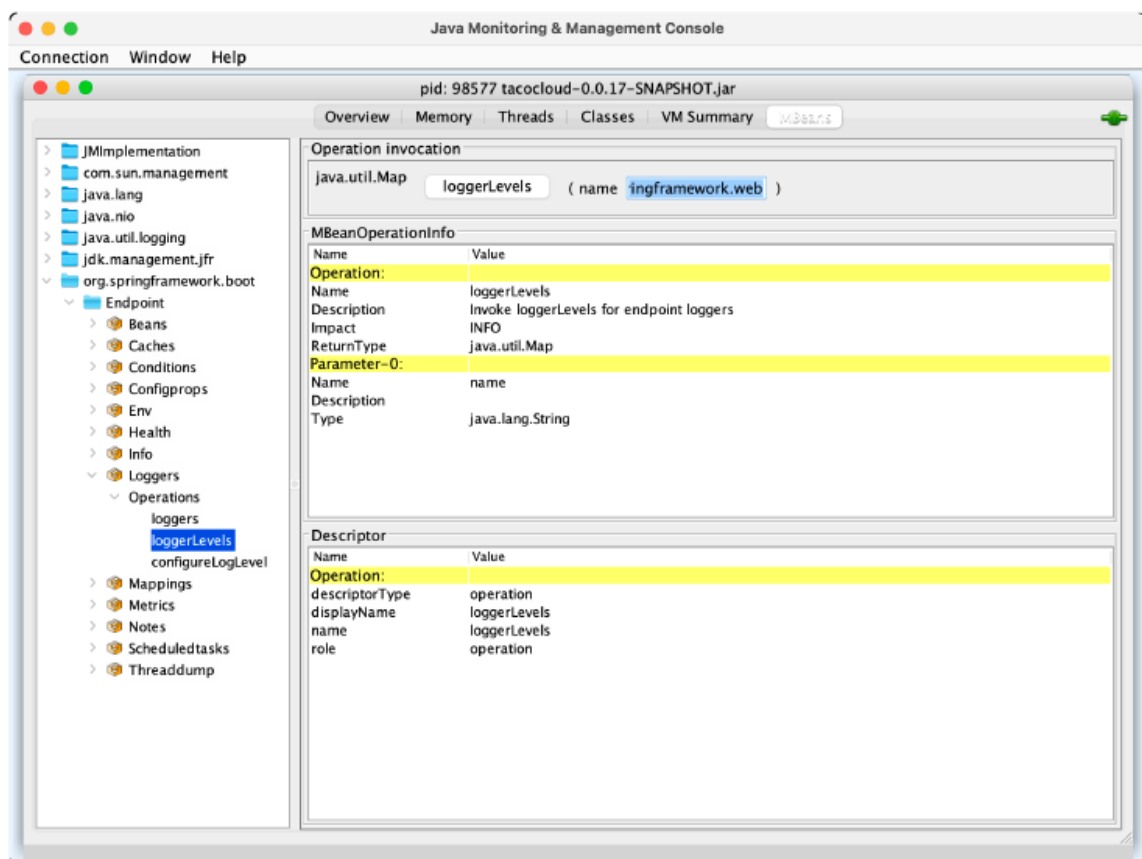


图 17.2 使用 `JConsole` 显示 `Spring Boot` 应用程序的日志记录级别。

单击 `LoggerLevel` 按钮后，将弹出一个对话框，显示来自 `/loggers` 端点 `MBean` 的响应。如图 17.3。

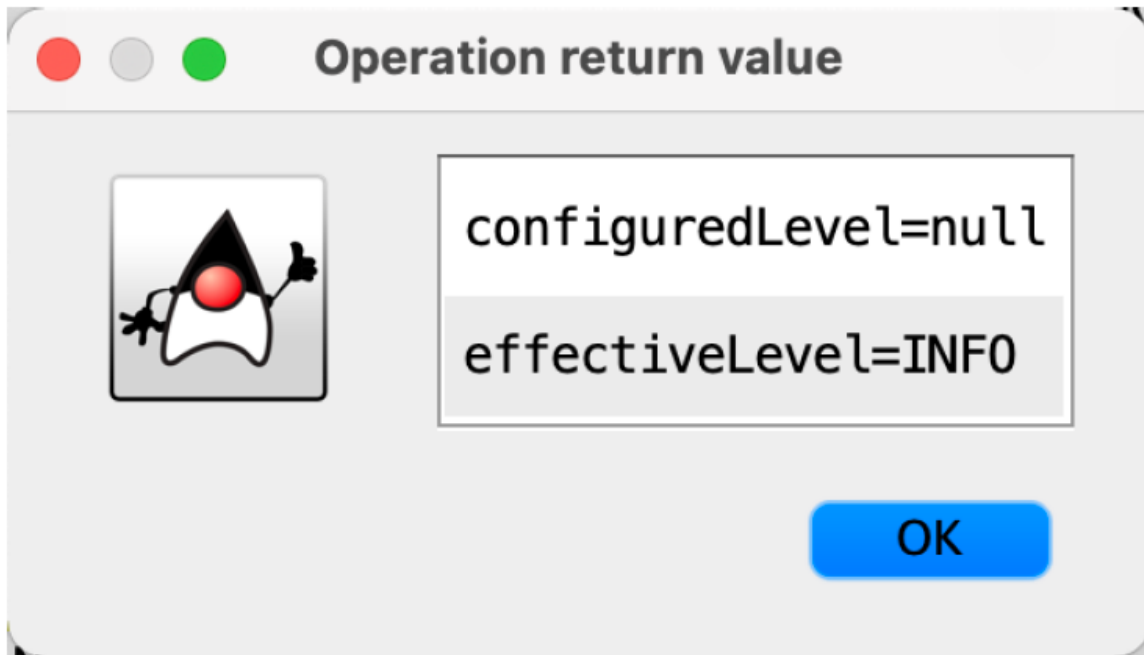


图 17.3 在 JConsole 中显示的 `/loggers` 端点 MBean 的日志级别

尽管 JConsole UI 使用起来有点笨拙，但您应该了解了使用 Mbean 的诀窍，并以基本相同的方式来探索任何 Actuator 端点。如果您不喜欢 JConsole，那没关系，还有很多其他 JMX 客户端可供选择使用。

## 17.2 创建自己的 MBean

Spring 可以轻松地将任何您想要的 bean 公开为 JMX MBean。您要做的就是添加 `@ManagedResource` 注解，然后在方法或属性上添加 `@ManagedOperation` 或 `@ManagedAttribute` 注解。Spring 会进行其他必须的工作。

例如，假设您想提供一个 MBean，来跟踪通过 Taco Cloud 订购了多少玉米卷。您可以定义一个服务 bean 来记录已订购的玉米卷数量。下面的列表显示了实现这个功能的服务。

程序清单 17.1 一个 MBean，它统计创建了多少个玉米卷

```
package tacos.jmx;
import java.util.concurrent.atomic.AtomicLong;
import org.springframework.data.rest.core.event.AbstractRepositoryEventListener;
import org.springframework.jmx.export.annotation.ManagedAttribute;
import org.springframework.jmx.export.annotation.ManagedOperation;
import org.springframework.jmx.export.annotation.ManagedResource;
import org.springframework.stereotype.Service;
import tacos.Taco;
import tacos.data.TacoRepository;

@Service
@ManagedResource
public class TacoCounter
    extends AbstractRepositoryEventListener<Taco> {

    private AtomicLong counter;
    public TacoCounter(TacoRepository tacoRepo) {
        tacoRepo
            .count()
            .subscribe(initialCount -> {
                this.counter = new AtomicLong(initialCount);
            });
    }

    @Override
    protected void onAfterCreate(Taco entity) {
        counter.incrementAndGet();
    }

    @ManagedAttribute
    public long getTacoCount() {
        return counter.get();
    }

    @ManagedOperation
    public long increment(long delta) {
        return counter.addAndGet(delta);
    }
}
```

TacoCounter 类加了 `@Service` 注解，将通过组件扫描自动在 Spring 上下文中生成 bean 实例。还可以用 `@ManagedResource` 注解，以表明这个 bean 也应该是 MBean。作为 MBean，它将公开一个属性和一个操作。getTacoCount() 方法用了 `@ManagedAttribute` 注解，因此它作为 MBean 属性公开，而 increment() 方法用了 `@ManagedOperation` 注解，将其作为 MBean 操作公开。图 17.4 显示了 TacoCounter MBean 在 JConsole 中的显示方式。



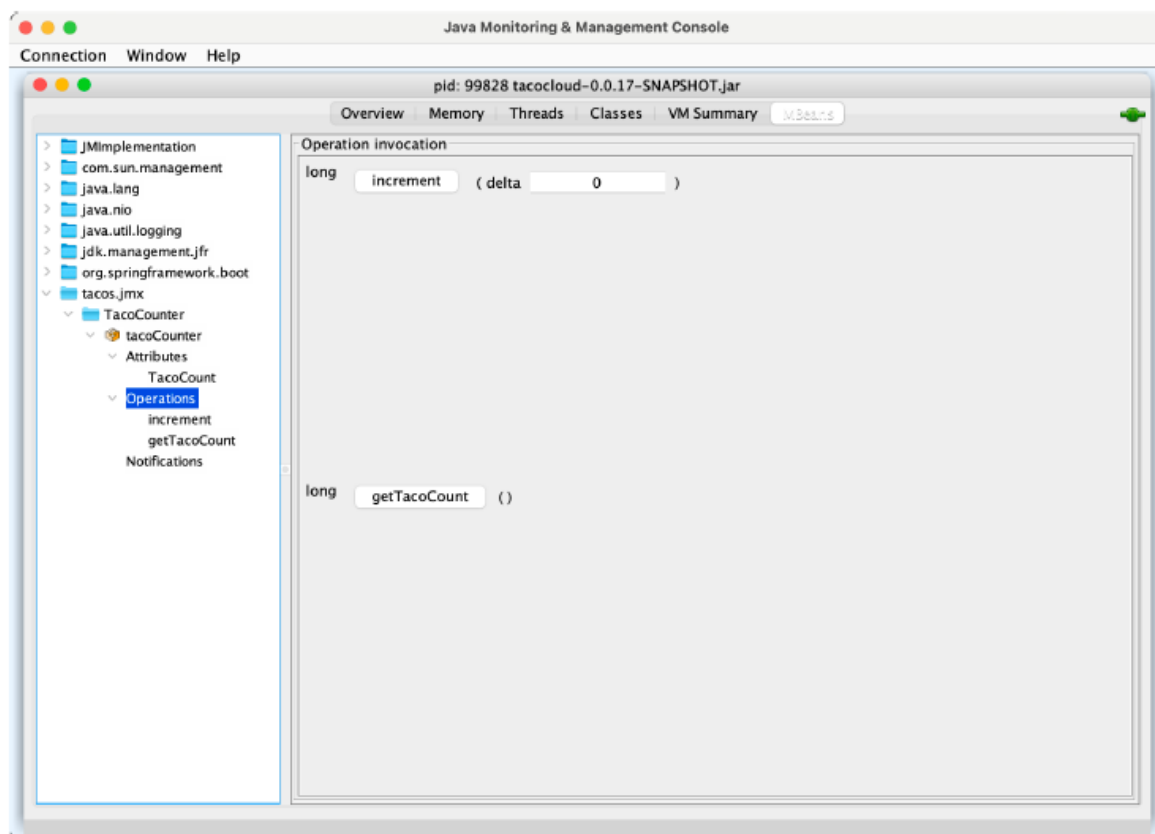


图 17.4 TacoCounter 公开的操作和属性如 JConsole 所示

TacoCounter 还有另一个秘密，尽管它与 JMX 无关。因为它扩展了 `AbstractRepositoryEventListener`，所以当通过 `TacoRepository` 保存 Taco 时，将发送持久化事件。在这种情况下，只要创建新的 Taco 对象并保存，就会调用 `onAfterCreate()` 方法，它将计数器加 1。但是 `AbstractRepositoryEventListener` 还提供了几种处理前后事件的方法，如在创建、保存或删除对象操作时。

使用 MBean 操作和属性在很大程度上是一种拉式操作。也就是，即使 MBean 属性的值发生更改，在通过 JMX 客户端再次访问查看前是不知道的。让我们反过来看看，如何从 MBean 推送通知到 JMX 客户端。

## 17.3 发送通知

MBean 可以使用 Spring 的 `NotificationPublisher`，将通知推送到感兴趣的 JMX 客户端。`NotificationPublisher` 有一个 `sendNotification()` 方法，当给定 `Notification` 对象时，会将通知发布到任何已经订阅此 MBean 的 JMX 客户端。

为了使 MBean 能够发布通知，它必须实现 `NotificationPublisherAware` 接口，该接口要求实现 `setNotificationPublisher()` 方法。例如，假设每生产 100 个玉米卷，您要发布一次通知，可以修改 `TacoCounter` 类，实现 `NotificationPublisherAware` 并使用注入的 `NotificationPublisher` 发送通知。下面的清单显示了，实现此功能后的 `TacoCounter`。

程序清单 17.2 每生产 100 个玉米卷就发送通知

```
package tacos.jmx;

import java.util.concurrent.atomic.AtomicLong;
import org.springframework.data.rest.core.event.AbstractRepositoryEventListener;
import org.springframework.jmx.export.annotation.ManagedAttribute;
import org.springframework.jmx.export.annotation.ManagedOperation;
import org.springframework.jmx.export.annotation.ManagedResource;
import org.springframework.stereotype.Service;

import org.springframework.jmx.export.notification.NotificationPublisher;
import org.springframework.jmx.export.notification.NotificationPublisherAware;
import javax.management.Notification;

import tacos.Taco;
import tacos.data.TacoRepository;

@Service
@ManagedResource
public class TacoCounter
    extends AbstractRepositoryEventListener<Taco>
    implements NotificationPublisherAware {

    private AtomicLong counter;
    private NotificationPublisher np;

    @Override
    public void setNotificationPublisher(NotificationPublisher np) {
        this.np = np;
    }

    ...

    @ManagedOperation
    public long increment(long delta) {
        long before = counter.get();
        long after = counter.addAndGet(delta);
        if ((after / 100) > (before / 100)) {
            Notification notification = new Notification(
                "taco.count", this,
                before, after + "th taco created!");
            np.sendNotification(notification);
        }
        return after;
    }
}
```

在 JMX 客户端中，您需要订阅 **TacoCounter MBean** 以接收通知。然后，随着玉米卷的生产，每生产 100 个玉米卷，客户端将收到一条通知。图 17.5 显示了在 JConsole 中显示的通知。

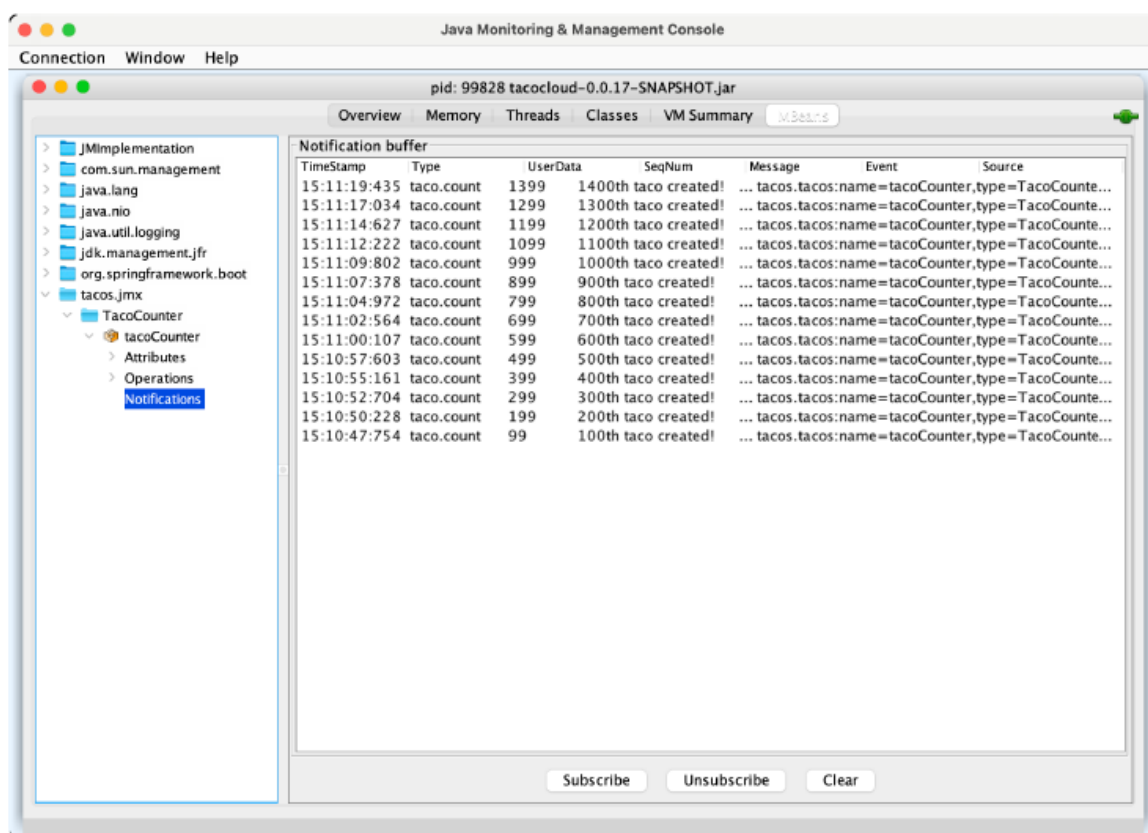


图 17.5 订阅 **TacoCounter MBean** 的 JConsole，每 100 个玉米卷被制造出来就收到一次通知。

推送通知是应用程序主动向客户端发送数据和警报的好方法，而不要求客户端轮询或主动发起调用操作。

## 17.4 总结

- 大多数 Actuator 端点都是 MBean，可以使用任何 JMX 客户端进行连接。
- Spring 会自动启用 JMX 来监视 Spring 应用程序上下文中的 bean。
- 通过在实体 bean 上添加 @ManagedResource 注解，可以将其公开为 MBean。它们的方法和属性，可以通过使用 @ManagedOperation 和 @ManagedAttribute 进行公开。
- 可以使用 NotificationPublisher 将通知发布到已订阅的 JMX 客户端。

## 第 18 章 部署 Spring

本章内容

- 将 Spring 应用程序构建为 WAR 或 JAR
- 将 Spring 应用程序推送至 Cloud Foundry
- 使用 Docker 容器化部署 Spring 应用程序

想像一部您最喜欢的动作电影，并且您现在正在电影院看它。伴随着高速的追逐、爆炸、战斗，真是一段惊心动魄的视听旅程。但是如果还没有看到电影中正义战胜邪恶，播放就停止了。剧院的灯光亮起来，每个人都开始往外走，您一定会明白，虽然情节的铺垫过程不可忽视，但最令人兴奋的是电影的高潮，结局是非常重要的。没有结局，那就成了只问耕耘，不问收获。

这就像为解决业务问题，投入了大量精力开发的应用程序却不部署，不让其他人使用和享受成果。虽然我们编写的大多数应用程序都不涉及汽车追逐或爆炸（至少我希望不是这样），但在开发的过程中，一定经历过困难。即使我们都清楚，并不是写的每一行代码都注定要投入生产，但如果开发的应用从来不实际部署，那是多么遗憾的事啊。

到目前为止，我们一直专注于使用 **Spring Boot** 的特性，来帮助我们开发一个应用程序。在这一过程中有一些令人兴奋的步骤。现在一切开发过程都结束了。如果您没有越过终点线来部署应用程序，那么您将一无所获。

在本章中，我们将跨过开发 **Spring** 应用程序这一阶段，了解如何部署应用程序。虽然对于任何部署过基于 **Java** 应用程序的人来说，似乎太简单了。但 **Spring Boot** 以及 **Spring** 相关的项目有一些独特特性，您可以利用这些特性部署 **Spring Boot** 应用程序。

事实上，与大多数 **Java web** 应用程序不同，因为 **Java web** 应用程序通常将应用程序作为 **WAR** 文件部署到服务器。**Spring Boot** 提供了几个部署选项。在我们了解如何部署 **Spring Boot** 应用程序前，让我们先看看有哪些部署选项可供您选择。

## 18.1 权衡部署选项

您可以通过多种方式构建和运行 **Spring Boot** 应用程序。附录中介绍了这些方式，包括：

- 使用 **Spring Tool Suite** 或 **IntelliJ IDEA** 在 **IDE** 中运行应用程序
- 使用 **Maven springboot:run** 或 **Gradle bootRun**，从命令行运行应用程序
- 使用 **Maven** 或 **Gradle** 生成可执行 **JAR**，然后在命令行或部署在云中运行
- 使用 **Maven** 或 **Gradle** 生成 **WAR** 文件，然后部署到传统 **Java** 应用服务器中运行
- 使用 **Maven** 或 **Gradle** 生成容器镜像，可以部署到任何支持容器的位置，包括 **Kubernetes** 环境。

这些选项中的任何一个都适合在开发过程中运行应用程序。但是，当您准备将应用程序部署到生产环境，或其他非开发环境中，应该怎么选择呢？

虽然从 **IDE** 或通过 **Maven** 或 **Gradle** 运行应用程序的方式，不适用于生产环境，但可执行 **JAR** 文件和传统 **Java WAR** 文件是适用于生产环境的。那如何选择是部署 **WAR** 文件，还是 **JAR** 文件呢？一般来说，这取决于您是想将应用程序部署到传统 **Java** 应用服务器还是云平台：

- 部署到云端 —— 如果您计划将应用程序部署到 **PaaS** 平台，如 **Cloud Foundry**，可执行的 **JAR** 文件是最好的选择。即使云平台支持 **WAR** 文件部署，但 **JAR** 文件格式比 **WAR** 格式简单得多，**WAR** 格式是为应用服务器部署设计的。
- 部署到 **Java** 应用服务器 —— 如果必须将应用程序部署到 **Tomcat**、**WebSphere**、**WebLogic** 或任何其他传统 **Java** 应用程序服务器，真的别无选择，只能将应用程序构建为 **WAR** 文件。
- 部署到 **Kubernetes** —— 现代云平台越来越多地基于 **Kubernetes**。部署到 **Kubernetes** 时，它本身就是一个容器编排系统，最明显的选择是将应用程序构建为容器映像。

在本章中，我们将重点介绍三种部署场景：

- 将 **Spring Boot** 应用程序构建为可执行 **JAR** 文件，可推送到 **Paas** 平台。
- 将 **Spring Boot** 应用程序构建为 **WAR** 文件，部署到 **Java** 应用程序服务器，比如 **Tomcat**。
- 将 **Spring Boot** 应用程序打包到 **Docker** 容器中，以便部署到支持 **Docker** 的云平台。

首先，让我们来看看构建 **Spring Boot** 应用程序最常见的方法：可执行 **JAR** 文件。

## 18.2 构建可执行 JAR 文件

将 Spring Boot 应用程序构建成可执行 JAR 文件相当简单。假设在初始化项目时您选择了 JAR 打包方式，那么您应该能够用以下 Maven 命令构建可执行 JAR 文件：

```
$ mvnw package
```

成功构建后，生成的 JAR 文件将被放置到 `target` 目录中，并带有基于项目 `pom.xml` 中的 `<artifactId>` 和 `<version>` 条目的名称和版本文件（例如，“`tacocloud-0.0.19-SNAPSHOT.jar`”）。

或者，您用的是 Gradle，可以这样做：

```
$ gradlew build
```

对于 Gradle 构建，生成的 JAR 将在 `build/libs` 目录中。JAR 文件名称将基于 `settings.gradle` 文件中的 `rootProject.name` 属性以及 `build.gradle` 中的 `version` 属性。

一旦有了可执行的 JAR 文件，就可以用 `java -JAR` 运行它，如下所示：

```
$ java -jar tacocloud-0.0.19-SNAPSHOT.jar
```

应用程序将运行。假设它是一个 web 应用程序，这启动一个嵌入式服务器（Netty 或 Tomcat，取决于项目是否为反应式 web 项目）并开始侦听配置的 `server.port`（默认为 8080）上的请求。

这非常适合在本地运行应用程序。但是如何部署可执行 JAR 文件呢？

这实际上取决于您将在哪里部署应用程序。如果您要部署到 Cloud Foundry，您可以使用 CF 命令行工具：

```
$ cf push tacocloud -p target/tacocloud-0.0.19-SNAPSHOT.jar
```

`cf push` 的第一个参数是 Cloud Foundry 中应用程序的名称。这个名字用于 Cloud Foundry 和 cf CLI 中的应用程序，并用作访问此应用程序的子域名。例如，如果 Cloud Foundry 的应用程序子域名是“`cf.myorg.com`”，那么 Taco Cloud 应用程序访问地址将是 <https://tacocloud.cf.myorg.com>。

部署可执行 JAR 文件的另一种方法是，将它们打包到 Docker 容器中，并在 Docker 或 Kubernetes 中运行它们。让我们看看这是如何做到的。

## 18.3 构建容器镜像

在云端部署各种应用程序，Docker 已经成为了事实上的标准 <https://www.docker.com>。许多不同的云端环境，包括 AWS、Microsoft Azure、Google Cloud Platform 和 Pivotal Web Service（举几个例子），都接受用于部署应用程序的 Docker 容器。

容器化应用程序（如使用 Docker 创建的应用程序）的思想吸引了很多人，这是自真实世界集装箱的类比。集装箱都有一个标准尺寸和格式，无论在其中放什么货物。正因为如此，集装箱很容易堆放在船上，或用火车、卡车来运输。

以类似的方式，容器化应用程序共享一个公共的容器格式，这种格式可以在任何位置部署和运行，而不用考虑其内部的应用程序如何。

从 Spring Boot 应用程序创建镜像的最基本方法是使用 `docker build` 命令和 Dockerfile 文件。Dockerfile 文件将可执行 JAR 文件从项目构建中复制到容器镜像中。一个极简单的 Dockerfile 如下：

```
FROM openjdk:11.0.12-jre
ARG JAR_FILE=target/*.jar
COPY ${JAR_FILE} app.jar
ENTRYPOINT ["java","-jar","/app.jar"]
```

Dockerfile 描述了如何创建容器镜像。上面的示例非常简单，让我们逐行进行说明：

- 第 1 行：声明我们创建的镜像，将基于预定义的容器镜像，它提供了 Open JDK11 Java 运行时。
- 第 2 行：创建一个变量，该变量引用项目 `target` 目录中的所有 JAR 文件。对于大多数 Maven 构建项目，其中应该只有一个 JAR 文件。通过使用通配符，可将 Dockerfile 定义与 JAR 文件的名称和版本分离。JAR 文件的路径也假定了 Dockerfile 位于 Maven 项目的根目录中。
- 第 3 行：将 JAR 文件从项目的 `target` 目录复制到容器镜像中，名称为“app.jar”。
- 第 4 行：定义一个入口点，也就是说，定义了由镜像生成容器，在启动时要执行的命令。这里是使用 `java -jar /app.jar` 执行 JAR 文件。

有了这个 Dockerfile，您可以使用 Docker 命令行工具创建镜像了，如下所示：

```
$ docker build . -t habuma/tacocloud:0.0.19-SNAPSHOT
```

此命令中的“.”指明了 Dockerfile 位置的相对路径。如果您是从其他路径运行 `docker build`，请替换 Dockerfile 的路径（不带文件名）。例如，如果您正在从父级项目中运行 `docker build`，命令应该像下面这样：

```
$ docker build tacocloud -t habuma/tacocloud:0.0.19-SNAPSHOT
```

在 `-t` 参数之后给出的值是镜像标记，它由名称和版本组成。在本例中，镜像名称为“habuma/tacocloud”，版本为 0.0.19-SNAPSHOT。

如果您想运行一下，可以使用 `docker run` 运行此新创建的镜像：

```
$ docker run -p8080:8080 habuma/tacocloud:0.0.19-SNAPSHOT
```

参数 `-p8080:8080` 将请求从主机上的端口 8080（例如，您正在运行 Docker 的主机）转发到容器的 8080 端口（Tomcat 或 Netty 正在监听的位置）。



在构建 Docker 镜像时，如果您已经有了一个可执行的 JAR，那么这种方法就足够简单。但这不是从 Spring Boot 应用程序创建镜像的最简单的方式。

从 Spring Boot 2.3.0 开始，您可以构建容器镜像，而无需添加任何特殊的依赖项、配置文件或以任何方式修改项目。这是因为 Spring Boot 集成了直接构建镜像的插件，支持 Maven 和 Gradle 两种方式。

要将 Maven 构建的 Spring 项目构建成容器镜像，可以指定 Spring Boot Maven 插件的构建目标为 build-image，如下所示：

```
$ mvnw spring-boot:build-image
```

同样，Gradle 构建的项目也可以构建成容器镜像，如下所示：

```
$ gradlew bootBuildImage
```

这将基于 pom.xml 文件中 `<artifactId>` 和 `<version>` 属性构建带有默认标记的镜像。对于 Taco Cloud 应用程序，这类似于“library/tacocloud:0.0.19-SNAPSHOT”。稍后我们将了解如何指定自定义镜像标签。

Spring Boot 的构建插件依赖 Docker 来创建镜像。因此，您需要在生成镜像的计算机上安装 Docker 运行时。无论如何，一旦镜像被创建后，就可以按如下方式运行：

```
$ docker run -p8080:8080 library/tacocloud:0.0.19-SNAPSHOT
```

这将运行镜像并映射端口 8080（由嵌入式 Tomcat 或 Netty 进行侦听）到主机的端口 8080。

镜像标记的默认格式为“docker.io/library/\${project.artifactId}:\${project.version}”，这就解释了为什么标记以“library”开头。如果您只在本地运行容器，这没什么问题。但您很可能希望将镜像推送到镜像注册中心，如 DockerHub，这将需要使用标签名称以引用镜像 Repository 的构建。

例如，假设您的组织在 DockerHub 中的 Repository 名称为“tacocloud”。在这种情况下，您将希望映像名称为“tacocloud/tacocloud:0.0.19-SNAPSHOT”，要将“library”默认前缀替换为“tacocloud”。要做到这一点，只要生成镜像时指定生成属性。对于 Maven，您需指定镜像名称使用 spring-boot.build-image.imageName 这个 JVM 系统属性，如下所示：

```
$ mvnw spring-boot:build-image \
  -Dspring-boot.build-image.imageName=tacocloud/tacocloud:0.0.19-SNAPSHOT
```

对于 Gradle 构建的项目，它稍微简单一些。您可以使用 --imageName 参数，如下所示：

```
$ gradlew bootBuildImage --imageName=tacocloud/tacocloud:0.0.19-SNAPSHOT
```

这两种指定镜像名称的方法，任何一种都要求您执行构建时要记住指定镜像名称，而且不能犯错。为了让构建变得更简单，您可以将镜像名称指定为构建本身的一部分。

在 Maven 中，您可以在 Spring Boot Maven 插件中将映像名指定为配置条目。例如，以下项目的 pom.xml 文件中的 `<configuration>` 块，显示了如何指定镜像名称：

```

<plugin>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-maven-plugin</artifactId>
  <configuration>
    <image>
      <name>tacocloud/${project.artifactId}:${project.version}</name>
    </image>
  </configuration>
</plugin>

```

注意，我们可以利用变量，而不是硬编码，因为这些值已在其他地方指定了。这就消除了需要手动在镜像名称中添加版本号麻烦。

对于 **Gradle** 构建的项目，**build.gradle** 中的以下条目可实现相同的效果：

```

bootBuildImage {
  imageName = "habuma/${rootProject.name}:${version}"
}

```

有了此配置，您无需再在命令行中指定镜像名称，就像我们前面所做的那样。

此时，您可以像以前一样使用 **docker run** 运行镜像（使用新的镜像名称），或您可以使用 **docker push** 将镜像推送到镜像注册中心，如 **DockerHub**：

```
$ docker push habuma/tacocloud:0.0.19-SNAPSHOT
```

一旦镜像推到了镜像注册中心，就可以从任何能够访问该注册中心的环境拉取运行镜像。**Kubernetes** 是一个越来越常见的运行镜像的环境。让我们了解一下如何在 **Kubernetes** 中运行镜像。

### 18.3.1 部署到 Kubernetes

Kubernetes 是一个惊人的容器编排平台，提供了非常强大的功能。可以运行镜像、按需缩放容器实例、自动重启异常容器增加健壮性等等。

Kubernetes 是部署应用程序的强大平台。事实上我们不可能在本章中详细介绍它。我们将只关注将 Spring Boot 应用程序构建成镜像，并部署到 Kubernetes 集群中。要更详细地了解 Kubernetes，请参阅 Marko Lukša 的《Kubernetes 实战》第二版。

Kubernetes 被认为难以使用（也许不公平），但开发 Spring 应用程序并部署在 Kubernetes 中是非常简单的。

您需要一个 Kubernetes 环境来将应用程序部署到其中。目前有几种选择，包括亚马逊的 AWS EKS 和谷歌 Kubernetes 引擎（GKE）。对于本地实验，还可以使用各种 Kubernetes 实现以运行 Kubernetes 群集，如 MiniKube (<https://minikube.sigs.k8s.io/docs/>)、MicroK8s (<https://microk8s.io/>)，还有我个人最喜欢的 Kind (<https://kind.sigs.k8s.io/>)。

您需要做的第一件事是创建部署清单。部署清单是一个描述映像应如何部署的 YAML 文件。作为一个简单的例子，考虑以下部署清单部署前面创建的 Taco Cloud 镜像到 Kubernetes 集群：

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: taco-cloud-deploy
  labels:
    app: taco-cloud
spec:
  replicas: 3
  selector:
    matchLabels:
      app: taco-cloud
  template:
    metadata:
      labels:
        app: taco-cloud
    spec:
      containers:
        - name: taco-cloud-container
          image: tacocloud/tacocloud:latest
```

您可以随意命名此清单。但是为了便于讨论，让我们假设您将其命名为 `deploy.yaml`，并将其放置在项目根目录下名为 `k8s` 的目录中。

在不深入了解 Kubernetes 部署规范细节的情况下，这里只需要注意，我们的部署名为“taco-cloud-deploy”，并且（靠近底部）设置了基于名称为“tacocloud/tacocloud:latest”的镜像部署和启动容器。通过将“latest”作为版本而不是“0.0.19-SNAPSHOT”，我们可以知道，将使用推送到容器注册中心的最新镜像。

另一件需要注意的事情是副本设置为了 3。这告诉 Kubernetes 运行时，应该有 3 个容器实例在运行。如果出于任何原因三个实例失败后，Kubernetes 将自动启动新的实例。

要应用部署，可以使用 `kubectl` 命令行工具，如下所示：

```
$ kubectl apply -f deploy.yaml
```

稍等一会儿，您应该能够使用 `kubectl get all` 查看部署情况，包括三个 pod，每个 pod 运行一个容器实例。您会看到类似的输出：

```
$ kubectl get all
```

| NAME                                   | READY | STATUS  | RESTARTS | AGE |
|----------------------------------------|-------|---------|----------|-----|
| pod/taco-cloud-deploy-555bd8fdb4-dln45 | 1/1   | Running | 0        | 20s |
| pod/taco-cloud-deploy-555bd8fdb4-n455b | 1/1   | Running | 0        | 20s |
| pod/taco-cloud-deploy-555bd8fdb4-xp756 | 1/1   | Running | 0        | 20s |

| NAME                              | READY | UP-TO-DATE | AVAILABLE | AGE |
|-----------------------------------|-------|------------|-----------|-----|
| deployment.apps/taco-cloud-deploy | 3/3   | 3          | 3         | 20s |

| NAME                                         | DESIRED | CURRENT | READY | AGE |
|----------------------------------------------|---------|---------|-------|-----|
| replicaset.apps/taco-cloud-deploy-555bd8fdb4 | 3       | 3       | 3     | 20s |

第一部分显示了 3 个 pod，每个 pod 对应于我们在 `replicas` 属性中请求的实例。中间部分是部署资源本身。最后一部分是 `ReplicaSet` 资源，Kubernetes 用来记住要给应用程序保留几个副本。

如果您想试用该应用程序，您需要为 pod 公开一个端口，`kubectl port forward` 命令很方便做到这一点：

```
$ kubectl port-forward pod/taco-cloud-deploy-555bd8fdb4-dln45 8080:8080
```

在本例中，我选择了 `kubectl get all` 列出的 3 个 pod 中的第一个，并设置了转发来自主机（Kubernetes 群集所在的计算机）的请求，从端口 8080 至 pod 的端口 8080。有了它，您就能够访问 <http://localhost:8080>，查看在指定 pod 上运行的 Taco Cloud 应用程序。

## 18.3.2 启用优雅关闭

有几种方法可以使 **Spring** 应用程序变得对 **Kubernetes** 友好，但是您需要做两件最基本的事情，实现优雅的关闭和活跃性和可用性探测。

在任何时候，**Kubernetes** 都可能决定关闭应用程序正在运行的一个或多个 **pod**。这可能是因为它感觉到了一些问题，也可能是因为有人感觉到了问题，明确要求关闭或重新启动 **pod**。不管是什么原因，如果 **pod** 正在处理一个请求，将 **pod** 立即关闭，使用请求未处理，是一种非常糟糕的情况。这样做将导致客户端收到错误响应，并要求客户端重试请求。

您可以在 **Spring** 中启用优雅的关闭，而不是让客户端承担错误。应用程序只需将 **server.shutdown** 属性设置为“**graceful**”。这可以在第 6 章中讨论的任何属性来源文件中指定，包括 **application.yml**，如下所示：

```
server:
  shutdown: graceful
```

通过启用优雅关闭，**Spring** 将允许应用程序延迟关闭一段时间，最多 30 秒，以处理任何正在进行的请求。在所有请求处理完之后，或关闭超时时间过期，应用程序将被允许关闭。

默认情况下，关机超时为 30 秒。但您可以通过 **spring.lifecycle.timeout-per-shutdown-phase** 属性进行调整。例如，更改超时为 20 秒，您可以将属性设置为：

```
spring:
  lifecycle.timeout-per-shutdown-phase: 20s
```

关闭挂起时，服务器将停止接受新请求。这允许在关机前清空所有处理中的请求。

关闭期间并不是应用程序无法处理请求的唯一时机。启动时，应用程序可能需要一段时间来准备处理流量。

**Spring** 应用程序向 **Kubernetes** 表明它还没有准备好处理流量的方式之一，是准备就绪的探测器。接下来，我们将看一看如何启用存活性和就绪性探测器。

### 18.3.3 应用存活性和就绪性

正如我们在第 15 章中所看到的，**Actuator** 的 **health** 端点提供了一个系统的健康状态。但这种健康状况只与应用程序依赖的下层组件有关，例如数据库或消息代理。即使应用程序是完全健康的，但数据库连接不正常，这并不意味着它已经准备好处理请求了，或者确保可以在当前状态下继续运行。

**Kubernetes** 支持存活性和就绪性探测的概念：应用程序的健康探测器帮助 **Kubernetes** 确定是否应将流量发送到应用程序上，或应用程序是否应该重新启动以解决某些问题。**Spring Boot** 支持存活性和就绪性探测，通过 **Actuator health** 端点的子集（称为 **health groups**）。

存活性是应用程序是否健康，以便可以继续运行，而无需重新启动的指标。如果应用程序指示其存活性为 **DOWN**，则 **Kubernetes** 运行时可以通过终止应用程序正在运行的 **pod**，并启动新的取而代之。

另一方面，就绪性告诉 **Kubernetes** 应用程序是否准备好处理请求。例如，启动时应用程序开始处理请求之前，可能需要执行一些初始化。在此期间，应用程序的就绪状态可能会显示 **Down**，而应用程序一直处于存活状态，因此 **Kubernetes** 不会重新启动它。一旦应用程序已完成初始化，它可以设置就绪探测器为 **UP**，以指示它已启动，**Kubernetes** 将把请求转到到应用上。

### 启用存活性和就绪性探测器

为了在 **Spring Boot** 应用程序中启用存活性和就绪性探测器，需要设置 **management.health.probes.enabled** 为 **true**，在 **application.yml** 文件中像这样设置：

```
management:
  health:
    probes:
      enabled: true
```

启用探测器后，对 **Actuator health** 端点的请求将返回类似如下的数据（假设应用程序完全健康）：

```
{
  "status": "UP",
  "groups": [
    "liveness",
    "readiness"
  ]
}
```

就其本身而言，基本 **health** 端点并没有告诉我们，一个项目的存活性和就绪性的太多信息。但对 **/actuator/health/liveness** 或 **/actuator/health/readiness** 的请求，将提供应用程序存活性和就绪性状态。在任何一种情况下，“**UP**”状态会像这样：

```
{
  "status": "UP"
}
```

另一方面，如果存活性和就绪性为 **DOWN**，则返回结果如下所示：

```
{
  "status": "DOWN"
}
```

在就绪性为“DOWN”状态下，Kubernetes 不会将请求转发到应用程序。如果存活性探测器指示“DOWN”状态，Kubernetes 将通过删除 pod 并在其他位置启动新实例来解决此问题。

## 在部署中配置存活性和就绪性探测器

随着 **Actuator** 在这两个端点上产生存活性和就绪性状态，我们需要做的就是部署清单中告诉 Kubernetes 有关它们的信息。以下部署清单内容在结尾处显示了让 Kubernetes 知道如何检查存活性和就绪性：

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: taco-cloud-deploy
  labels:
    app: taco-cloud
spec:
  replicas: 3
  selector:
    matchLabels:
      app: taco-cloud
  template:
    metadata:
      labels:
        app: taco-cloud
    spec:
      containers:
        - name: taco-cloud-container
          image: tacocloud/tacocloud:latest
          livenessProbe:
            initialDelaySeconds: 2
            periodSeconds: 5
            httpGet:
              path: /actuator/health/liveness
              port: 8080
          readinessProbe:
            initialDelaySeconds: 2
            periodSeconds: 5
            httpGet:
              path: /actuator/health/readiness
              port: 8080
```

这告诉 Kubernetes，对于每个探测器，都要向端口 8080 上的给定路径发出 GET 请求获得存活性和就绪性状态。按照这里的配置，第一个请求应该发生在应用程序 pod 运行后的 2 秒后，此后每 5 秒一次。

## 管理存活性和就绪性

那么，是如何设置存活性和就绪性状态的呢？在内部，Spring 本身或应用程序所依赖的某个库组件，可以通过发布可用性更改事件来设置状态。但这种能力并不局限于 Spring 及其相关库，您还可以在应用程序中编写代码发布这些事件。

例如，假设您希望将应用程序的就绪性延迟到某些初始化之后。在应用程序生命周期的早期，在 **ApplicationRunner** 或 **CommandLineRunner** bean 中，您可以发布就绪性状态以拒绝处理请求：

```
@Bean
public ApplicationRunner disableLiveness(ApplicationContext context) {
    return args -> {
        AvailabilityChangeEvent.publish(context, ReadinessState.REFUSING_TRAFFIC);
    };
}
```

这里，`ApplicationRunner` 被提供一个 `Spring` 应用程序上下文的实例，作为 `@Bean` 方法的参数。这是必需的，因为静态 `publish()` 方法需要它来发布事件。

初始化完成后，可以更新应用程序的就绪性状态以接受流量，类似这样：

```
AvailabilityChangeEvent.publish(context, ReadinessState.ACCEPTING_TRAFFIC);
```

存活性状态可以以非常相同的方式更新。关键的区别在于不是发布 `ReadinessState.ACCEPTING_TRAFFIC` 或 `ReadinessState.REFUSING_TRAFFIC`，而是发布 `LivenessState.CORRECT` 或 `LivenessState.BROKEN`。

例如，如果在应用程序代码中检测到不可恢复的致命错误，则应用程序可以通过发布 `LivenessState.BROKEN` 请求终止并重新启动它，如下所示：

```
AvailabilityChangeEvent.publish(context, LivenessState.BROKEN);
```

此事件发布后不久，`liveness` 端点将指示应用程序状态为 `DOWN`。`Kubernetes` 将采取重新启动应用程序的行动。这留给您发布 `LivenessState.CORRECT` 事件的时间很短。但是，如果您确定，应用程序是健康的，然后您可以通过发布新事件撤消以前的事件，像这样：

```
AvailabilityChangeEvent.publish(context, LivenessState.CORRECT);
```

只要 `Kubernetes` 在您将状态设置为“`DOWN`”后还没有请求 `liveness` 端点，您的应用程序可以将此记录为一个结束呼叫，并继续为请求提供服务。



## 18.4 构建并部署 WAR 文件

在阅读本书的整个过程中，随着您开发 Taco Cloud 应用程序，您已经在 IDE 中或通过命令运行过可执行 JAR 文件。无论是内嵌的 Tomcat 服务器还是 Netty、SpringWebFlux 应用程序，一直都在提供服务。

这在很大程度上要感谢 Spring Boot 的自动配置。您得以不必创建 web.xml 文件或 servlet 初始类，以声明 Spring MVC 的 DispatcherServlet。但是如果您要部署应用程序到 Java 应用服务器，您需要构建一个 WAR 文件。应用服务器需要知道如何运行应用程序，您还需要在 WAR 文件中包含一个 servlet 初始化类，以充当 web.xml 文件的一部分，并声明 DispatcherServlet。

事实证明，将 Spring Boot 应用程序构建成 WAR 文件并不困难。实际上，如果在使用 Initializr 时选择了 WAR 选项，那么您就没什么需要做的了。

Initializr 确保生成的项目将包含 servlet 初始类，且构建产物是 WAR 文件。但是，如果您使用 Initializr 时选择的是 JAR 文件（或者如果您想知道相关的差异是什么），您需要阅读下面的内容。

首先，您需要使用某种方式配置 Spring 的 DispatcherServlet。可以通过 web.xml 文件完成，但 Spring Boot 通过 SpringBootServletInitializer 很容易实现这一点。SpringBootServletInitializer 是对 WebApplicationInitializer 接口的一种特殊的 SpringBootApplication 实现。除了配置 Spring 的 DispatcherServlet，SpringBootServletInitializer 也寻找 Spring 应用程序上下文中的任何类型为 Filter、Servlet 或 ServletContextInitializer 的类，并将它们实例化到 servlet 容器。

要使用 SpringBootServletInitializer，请创建一个子类并重写 `configure()` 方法指定 Spring 配置类。清单 19.1 显示了 IngredientServiceServletInitializer。这是您用于配料服务应用程序的 SpringBootServletInitializer 的子类。

程序清单 18.1 通过 Java 启用 Spring Web 应用程序

```
package tacos;

import org.springframework.boot.builder.SpringApplicationBuilder;
import org.springframework.boot.context.web.SpringBootServletInitializer;

public class IngredientServiceServletInitializer
    extends SpringBootServletInitializer {
    @Override
    protected SpringApplicationBuilder configure(
        SpringApplicationBuilder builder) {
        return builder.sources(IngredientServiceApplication.class);
    }
}
```

如您所见，`configure()` 方法参数中传入了 `SpringApplicationBuilder` 对象，并将其作为结果返回。方法中调用 `sources()` 注册 Spring 配置类。在这种情况下，它注册 `IngredientServiceApplication` 类，起到两个作用。一是指定启动类（用于可执行 JAR 文件），另一个指定 Spring 配置类。

即使配料服务应用程序还有其他 Spring 配置类，但并不需要使用 `sources()` 方法全部注册它们。这个 `IngredientServiceApplication` 类，加了 `@SpringBootApplication` 注解，隐式启用了组件扫描。组件扫描发现并会找到任何其他配置类。

大多数情况下，`SpringBootServletInitializer` 的子类就是一个模板式文件。除引用应用程序主配置类之外，完全都一样。对于每个要构建 WAR 文件的应用程序，您几乎永远不会需要对其进行更改。

既然您已经编写了一个 **servlet** 初始化类，那么您必须对项目构建做一些小的修改。如果您使用 **Maven** 进行构建，要确保 **pom.xml** 中的 `<packaging>` 元素设置为 **war**，所需的更改如下：

```
<packaging>war</packaging>
```

如果使用 **Gradle** 构建，所做的修改也很简单。在 **build.gradle** 文件中引入 **war** 插件：

```
apply plugin: 'war'
```

现在，您已经准备好构建应用程序了。对于 **Maven**，您将使用 **Maven** 包装器执行打包脚本：

```
$ mvnw package
```

如果构建成功，那么可以在目标目录中找到 **WAR** 文件。另一方面，如果您使用 **Gradle** 来构建项目，那么您将使用 **Gradle** 包装器：

```
$ gradlew build
```

构建完成后，**WAR** 文件将位于 **build/libs** 目录中。然后就是部署应用程序了。部署过程因应用服务器不同而异，因此请参阅应用服务器的相关部署文档。

值得注意的是，尽管您已经构建的是一个适合部署到任何 **Servlet 3.0**（或更高版本）容器的 **WAR** 文件，但此 **WAR** 文件仍然可以在命令行上执行，就像它是一个可执行的 **JAR** 文件一样：

```
$ java -jar target/taco-cloud-0.0.19-SNAPSHOT.war
```

实际上，您可以从一个构建产品中获得两种部署选择！

## 18.5 终章

在前面的几百页中，我们从 [start.spring.io](http://start.spring.io) 创建工程开始，到在云端部署应用程序。我希望您也像我写这些章节时那样，获得了很多乐趣。

虽然这本书必须结束了，但您使用 **Spring** 的冒险旅程才刚刚开始。希望您利用在本书中学到的知识，用 **Spring** 构建出一些令人惊奇的东西。我迫不及待地想看看您的项目！

## 18.6 总结

- Spring 应用程序可以部署在许多不同的环境中，包括传统的应用服务器，以及平台即服务（PaaS）环境，比如 Cloud Foundry 或者 Docker 容器。
- 将 Spring Boot 应用程序构建为可执行 JAR 文件，就可以部署到多个云平台，避免 WAR 文件的较大的开销。
- 在构建 WAR 文件时，应该包含一个 SpringBootServletInitializer 子类，以确保 Spring 的 DispatcherServlet 正确配置。
- 容器化 Spring 应用程序可以简单的使用 Spring Boot 构建插件支持构建镜像。这些镜像可以部署在任何 Docker 容器环境，包括在 Kubernetes 集群。